preface

More than a decade ago, my journey into AI began with statistical pattern classification, where I learned how far relatively simple models can go when they capture useful structure in data. Over time, that curiosity shifted from models that recognize patterns to models that can explain, plan, and solve multistep problems.

With the release of ChatGPT in 2022, large language models (LLMs) moved into the mainstream. A later shift, especially visible in late 2024 and throughout 2025, was the extension of LLMs to create so-called reasoning models, which can solve more complex problems, especially in technical domains such as math and code.

At the same time, reasoning models can feel opaque, and this book is my attempt to make reasoning models more approachable. Rather than training a giant model from scratch, we'll begin with a small pretrained LLM and apply reasoning techniques step by step. Along the way, you will implement text generation, evaluation with verifiers, inference-time scaling methods, and training methods, such as reinforcement learning with verifiable rewards and distillation from scratch. By the end, you will understand how the main reasoning methods work in practice and how they fit into a modern LLM development workflow.

Like my earlier book, *Build a Large Language Model (From Scratch)*, this one takes a code-first approach, but the focus here is different. Instead of explaining the Transformer architecture in depth or covering large-scale pretraining, here we'll concentrate on the methods that turn a conventional LLM into a reasoning model. If you have read the earlier book, this one should feel like a natural next step. If you haven't, you can still follow along here without needing all the underlying LLM details up front.

We'll use math examples throughout the book because they are practical for reasoning research and easy to verify automatically. That makes them a useful test bed for understanding whether a model is actually solving a problem correctly. More broadly, my goal is not to teach one narrow benchmark but to give you the tools and intuition needed to build, evaluate, and improve reasoning systems in your own work.

When I first started working on reasoning models, I had to piece together ideas from papers, repositories, and scattered implementation notes. I hope this book saves you that effort and gives you a clearer path from first principles to working code. I strongly believe that the best way to understand reasoning models is to build one yourself.

Happy reading and coding!

acknowledgments

Writing a book is a significant undertaking, and I would like to express my sincere gratitude to my wife, Liza, for her patience and support throughout this process. Her unconditional love and constant encouragement have been absolutely essential.

I am incredibly grateful to Daniel Kleine and Dmitry Labazkin, whose invaluable feedback on the draft chapters and code went far above and beyond. Daniel's keen eye for detail and style, along with Dmitry's sharp insights into code performance, have undoubtedly made this book a smoother and more enjoyable read for all.

I would also like to thank the wonderful team at Manning Publications. Michael Stephens, thank you for the many productive conversations that helped shape the direction of this book. Dustin Archibald, I greatly appreciate your constructive guidance on adhering to Manning's guidelines, as well as your flexibility in accommodating the unique demands of this unconventional from-scratch approach. Special thanks also go to Aleksandar Dragosavljevic and Ivan Martinović for their work on layout and formatting. Finally, technical editor David Caswell, thank you for your thoughtful and generous feedback throughout the process. David is an industry-leading consultant focused on applying LLMs to journalism. He previously led AI innovation at the BBC, Tribune Publishing, and Yahoo!, he publishes peer-reviewed research on AI automation of information workflows, and he is a frequent speaker and writer on LLMs in the emerging AI-mediated information ecosystem.

Finally, I extend my thanks to the reviewers Alejandro Cuevas Rivero, Muhammad Ali Shafique, Arun Prakash A, Christopher Brousseau, David Curran, Fabio Montagna, Hamza Farooq, Hongming Zheng, Julien Thomazo, Lindo William Khoza, Michael Anthony Garcia, Naman Dwivedi, Pere Martra, Pradeep Dasigi, Salvatore Raieli, Sifal Klioui, Syed Baqir Ali, Thomas Viehmann, Tianrui Liu, Toni Ramchandani, and Viton Vitanis for their thorough feedback on the drafts. Your keen eyes and insightful comments have been essential in improving the quality of this book.

To everyone who has contributed to this journey, I am sincerely grateful. Your support, expertise, and dedication have been instrumental in bringing this book to fruition. Thank you!

Previous chapter
‹ preface

Next chapter
about this book ›

about this book

Build a Reasoning Model (From Scratch) is a hands-on guide to understanding how modern reasoning LLMs work by implementing their core techniques yourself. Rather than training a base model from the ground up, this book starts with a pretrained open source base LLM and shows you how to extend it step by step with reasoning capabilities. Along the way, you will learn how to generate text with a base model, evaluate answers with verifiers, improve performance through inference-time scaling, train with reinforcement learning, and distill reasoning behavior into smaller models. By the end of the book, you will understand how the main reasoning methods fit together in a practical LLM development workflow and how to build small but functional reasoning systems of your own.

The book takes a code-first approach. The "from scratch" part refers to implementing the reasoning methods yourself so that the underlying mechanics become transparent. If you are interested in how base LLMs are built and pretrained in detail, my earlier book, Build a Large Language Model (From Scratch) (**http://mng.bz/M96o**), provides that foundation, but it is not required for reading this book.

Who should read this book

Build a Reasoning Model (From Scratch) is for machine learning enthusiasts, engineers, researchers, students, and software developers who want a practical understanding of how reasoning models work and how to implement the main techniques behind them. It is written for readers who want to go beyond high-level descriptions and see how these systems are built, evaluated, and improved in code.

The most important prerequisite is solid Python experience. Prior exposure to machine learning, deep learning, or large language models is helpful, but you do not need to be an expert. The book is intended to be approachable for motivated beginners while still offering enough technical depth for experienced practitioners.

The book uses math examples frequently because they are practical for reasoning research and easy to verify automatically. However, advanced mathematics is not required. Familiarity with basic algebra and a general comfort level with vectors and matrices is useful, though, to follow some of the code implementations.

Some prior PyTorch experience may help you move more quickly through the code, but it is not required. The focus of the book is on understanding reasoning methods and implementing them clearly, not on mastering every detail of a deep learning framework. Readers who have already read Build a Large Language Model (From Scratch) may recognize some broader LLM concepts, but this book is fully self-contained.

How this book is organized: A road map

This book is organized around a practical development pipeline for reasoning models. We'll begin with a conventional pretrained LLM, add evaluation so that we can measure progress, and then explore two main ways of improving reasoning: inference-time methods and additional training. Because later chapters build directly on the code and ideas introduced earlier, the book is best read in sequence.

Chapter 1 introduces the practical meaning of reasoning in the context of LLMs. It explains how reasoning models differ from conventional LLMs, it reviews the standard LLM training pipeline at a high level, and it introduces the main approaches used to improve reasoning.

Chapter 2 establishes the starting point by loading and using a pretrained base LLM. It covers the coding environment, tokenization, step-by-step text generation, and practical speedups such as key-value caching and model code compilation.

Chapter 3 adds evaluation. It shows how to extract final answers reliably, verify correctness against reference solutions, and build an evaluation pipeline for math-oriented reasoning tasks.

With that foundation in place, chapters 4 and 5 focus on inference-time scaling, improving reasoning without changing the model weights. Chapter 4 covers prompt-based reasoning, diverse decoding, and self-consistency. Chapter 5 extends this direction with self-refinement, simple scoring strategies, confidence estimation, and iterative answer improvement.

Chapters 6 through 8 implement training-time methods. Chapter 6 introduces reinforcement learning for reasoning models, with a focus on reinforcement learning with verifiable rewards (RLVR) and Group Relative Policy Optimization (GRPO). Chapter 7 builds on that implementation by covering practical monitoring, common training-failure modes, reward hacking, and useful extensions such as KL regularization and response-format rewards. Chapter 8 concludes the main text with distillation, showing how to create teacher-generated reasoning datasets, train a smaller student model, and evaluate the distilled result.

The appendices complement the main chapters. Appendix A provides references and further reading, appendix B summarizes the exercise solutions, appendix C presents the Qwen3 source code used throughout the book, appendix D discusses larger LLM variants, appendix E covers batching and throughput-oriented execution, appendix F surveys additional LLM evaluation approaches, and appendix G shows how to build a chat interface around the model.

About the code

To make it easy to follow along, the code examples in this book are available from the Manning book page at **https://www.manning.com/books/build-a-reasoning-model-from-scratch** as well as in the companion GitHub repository at **https://github.com/rasbt/reasoning-from-scratch**. Solutions to the exercises are summarized in appendix B, and the corresponding code files are linked from the chapter folders in the repository. You can get executable snippets of code from the liveBook (online) version of this book at **https://livebook.manning.com/book/build-a-reasoning-model-from-scratch**.

This book contains many examples of source code both in numbered listings and in line with normal text. In both cases, source code is formatted in a `fixed-width font like this` to separate it from ordinary text.

In many cases, the original source code has been reformatted; we've added line breaks and reworked indentation to accommodate the available page space in the book. Additionally, comments in the source code have often been removed from the listings when the code is described in the text. Code annotations accompany many of the listings, highlighting important concepts.

One of the key goals of this book is accessibility, so the code examples have been carefully designed to run efficiently on a regular laptop, without the need for any special hardware. But if you do have access to a GPU, certain sections provide helpful tips on scaling up the datasets and models to take advantage of that extra power.

The code in the main chapters is designed to be approachable and to run on consumer hardware. The inference and evaluation chapters work well on CPUs and GPUs, while the training-oriented chapters benefit from access to a GPU if you want to reproduce the larger experiments. Throughout the book, we use PyTorch together with a pretrained open source Qwen3 model, and appendix C provides the underlying Qwen3 implementation for readers who want to inspect that code in more detail.

liveBook discussion forum

Purchase of Build a Reasoning Model (From Scratch) includes free access to liveBook, Manning's online reading platform. Using liveBook's exclusive discussion features, you can attach comments to the book globally or to specific sections or paragraphs. It's a snap to make notes for yourself, ask and answer technical questions, and receive help from the author and other users. To access the forum, go to **https://livebook.manning.com/book/build-a-reasoning-model-from-scratch/discussion**.

Manning's commitment to our readers is to provide a venue where a meaningful dialogue between individual readers and between readers and the author can take place. It is not a commitment to any specific

amount of participation on the part of the author, whose contribution to the forum remains voluntary (and unpaid). We suggest you try asking them some challenging questions lest their interest stray! The forum and the archives of previous discussions will be accessible from the publisher's website as long as the book is in print.

Other online resources

Interested in the latest AI and large language model (LLM) research trends?

- Check out my blog at **https://magazine.sebastianraschka.com**, where I regularly discuss the latest AI research with a focus on LLMs.

Need help getting up to speed with deep learning and PyTorch?

- I offer several free courses on my website at **https://sebastianraschka.com/teaching**. These resources can help you quickly get up to speed with the latest techniques.

Looking for bonus materials related to the book?

- Visit the book's GitHub repository at **https://github.com/rasbt/reasoning-from-scratch** to find additional resources and examples to supplement your learning.

about the author

Sebastian Raschka is an LLM Research Engineer with over a decade of experience in artificial intelligence. His work spans industry and academia, including implementing LLM solutions as a senior engineer at Lightning AI, as well as researching and teaching as a statistics professor at the University of Wisconsin–Madison.

Sebastian collaborates with industry partners on AI solutions and serves on the Open Source Board at University of Wisconsin–Madison. He specializes in LLMs and the development of high-performance AI systems, with a deep focus on practical, code-driven implementations. He is the author of the bestselling books Build a Large Language Model (From Scratch), as well as Machine Learning with PyTorch and Scikit-Learn, and Machine Learning Q and AI.

You can learn more about Sebastian at **https://sebastianraschka.com**.

Previous chapter
‹ about this book

Next chapter
about the cover illustration ›

about the cover illustration

The figure on the cover of *Build a Reasoning Model (From Scratch)*, titled “La Lionne,” or “The Lioness,” is taken from a book by Louis Curmer published in 1841. Each illustration is finely drawn and colored by hand.

In those days, it was easy to identify where people lived and what their trade or station in life was just by their dress. Manning celebrates the inventiveness and initiative of the computer business with book covers based on the rich diversity of regional culture centuries ago, brought back to life by pictures from collections such as this one.

Previous chapter
< about the author

Next chapter
1 Understanding reasoning models >

1 Understanding reasoning models

This chapter covers

- What "reasoning" means for a large language model
- Reviewing the conventional pretraining and post-training stages of LLMs
- Introducing key approaches to improving reasoning abilities in LLMs
- How reasoning differs from pattern matching
- Why we should build reasoning models from scratch

Welcome to the next stage of large language models (LLMs): *reasoning*. LLMs have transformed how we process and generate text, but their success has been largely driven by statistical pattern recognition. New advances in reasoning methods now enable LLMs to tackle more complex tasks, such as solving logical puzzles and multistep math problems.

Moreover, reasoning is an essential technique for making AI agents practical, such as when an agent has to break a task into steps, use tools, and recover from mistakes. Reasoning LLMs are already used in agent applications such as OpenClaw.

In this book, you'll learn the inner workings of LLM reasoning methods through a hands-on, code-first approach. We'll start with a pretrained LLM and extend it step by step with reasoning capabilities. We'll implement these reasoning components ourselves, from scratch, to see how these methods work in practice.

If you are curious about how LLMs themselves are built and trained, my earlier book, *Build a Large Language Model (From Scratch)*, also published by Manning (**http://mng.bz/orYv**), covers these foundations in detail, but it's not required to follow along here.

Even if you just have a cursory knowledge of how LLMs are built, by the end of this book, you will understand how reasoning models work and be equipped to design, prototype, and evaluate the main methods for improving reasoning in LLMs.

When working with reasoning models, it's crucial to know how to automatically check whether answers are correct on tasks such as math and how to turn a general-purpose model into a smaller reasoning model. We will use math examples throughout the book because they are easy to check automatically, but you do not need advanced math knowledge to follow along. Understanding reasoning methods is the central focus of this book.

With its focus on practical applications and explanations, this book is intended for LLM engineers, machine learning researchers, applied scientists, and software developers. Understanding how reasoning models are set up will help us judge when a standard LLM is sufficient and when extra reasoning machinery is worth the added complexity, cost, and latency. It will also help us design better evaluations, debug failures on multistep tasks, and adapt reasoning techniques to real products and research workflows.

1.1 Defining reasoning in the context of LLMs

In this book, I use the term *reasoning* in a practical engineering sense rather than a philosophical one. In the context of LLMs, reasoning refers to a model generating intermediate steps before producing its final answer.

These intermediate steps may appear as visible step-by-step explanations, or they may be enclosed in special tags such as `<think>...</think>` and be hidden from the user. In either case, the core idea remains the same. The model is encouraged or trained to use tokens for intermediate problem-solving steps rather than jumping directly to the conclusion.

Correspondingly, I use *reasoning model* to mean an LLM that has been improved, either through training or prompting techniques, to produce such intermediate steps, which often increases accuracy on complex tasks such as coding, logical puzzles, and math problems.

Before we get to the coding portions of this book, I will briefly discuss the main techniques that improve these reasoning behaviors and how they relate to pattern matching and logical reasoning. This will lay the groundwork for later discussions of how LLMs are currently built, how they handle reasoning tasks, and what they are and are not so good at.

CHAIN-OF-THOUGHT (COT)

This style of generating intermediate steps is often referred to as *chain of thought* (CoT). Researchers and engineers often say that the model "thinks" through the problem step by step, meaning that it makes its intermediate reasoning process explicit and easier to follow. In this book, I use the terms *reasoning* and *thinking* in that common engineering sense. This does not imply that LLMs actually reason or think in the same way humans do.

Figure 1.1 illustrates how a conventional LLM generates the answer to a user's question. A conventional LLM might not show how it came up with its answer, and while the answer might be correct, it doesn't help the user understand the process behind it.

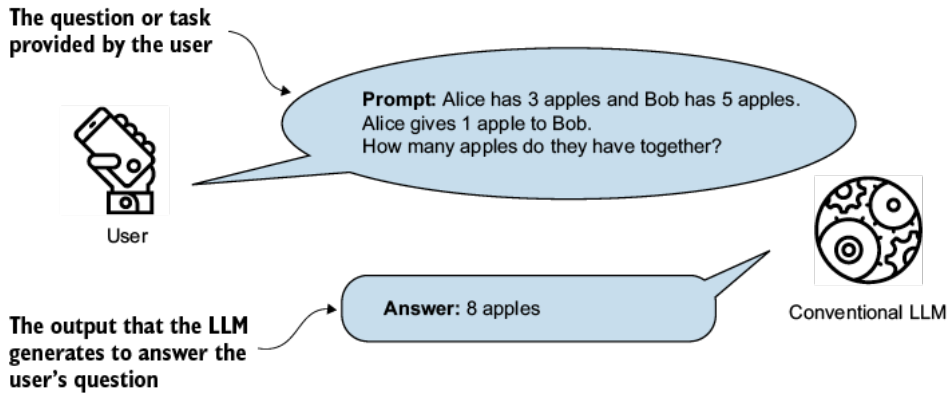


Figure 1.1 A simplified illustration of how a conventional, nonreasoning LLM might respond to a question with a short answer

Figure 1.2 illustrates a simple example of multistep (CoT) reasoning in an LLM. The LLM-produced intermediate reasoning steps look very much like a person articulating internal thoughts aloud. How closely these methods (and the resulting reasoning processes) mirror human reasoning remains an open research question, one this book does not attempt to answer.

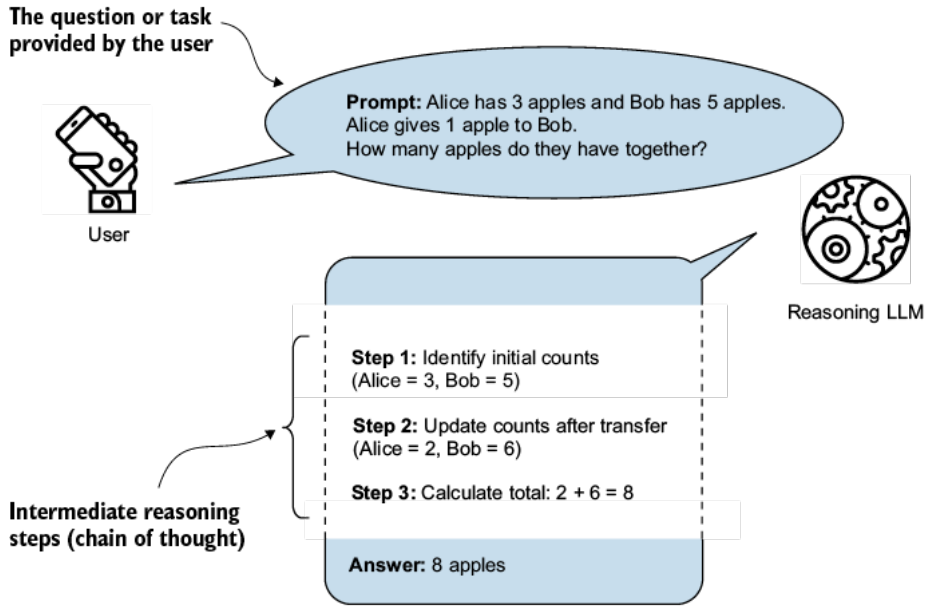


Figure 1.2 A simplified illustration of how a reasoning LLM might tackle a multistep reasoning task using a chain of thought. Rather than just recalling a fact, the model combines several intermediate reasoning steps to arrive at the correct conclusion. The intermediate reasoning steps may or may not be shown to the user, depending on the implementation.

While figure 1.2 is a typical example of chain-of-thought reasoning, it's important to emphasize that LLM reasoning differs from traditional, deterministic reasoning. For instance, a symbolic logic engine or theorem prover follows strict, rule-based steps that guarantee consistency and correctness. It works like following a recipe in which every step is fixed and must produce the same result each time. If you follow the steps exactly, you will always end up with the same dish.

In contrast to a symbolic logic engine, an LLM generates reasoning *autoregressively*, which means it predicts one token at a time based on statistical patterns in its training data. As a result, the LLM's "reasoning steps" are not guaranteed to be logically sound, even if they look convincing.

This book focuses on explaining and implementing the fundamental techniques that improve LLM-based reasoning and make LLMs better at handling complex tasks. My hope is that by gaining hands-on experience with these methods, you will be better prepared to understand and improve the reasoning methods being developed and maybe even explore how they compare to human reasoning.

LLM VS. HUMAN REASONING

Reasoning processes in LLMs may superficially resemble human thought, particularly in how intermediate steps are articulated. But it is important to recognize a key difference: humans can engage in deterministic reasoning by deliberately applying rules of logic or by reasoning over an internal model of the world. Deterministic, here, means that if we start with the same facts and follow the same steps, we will always reach the same conclusion. In contrast, current LLM reasoning is probabilistic, meaning that it generates one token at a time based on statistical patterns in training data, without guarantees of logical consistency.

Humans often reason by consciously manipulating concepts, intuitively understanding abstract relationships, or generalizing from a few examples. LLMs, by contrast, work by picking up patterns from huge amounts of text rather than relying on built-in reasoning rules or any kind of conscious thought.

In short, although the outputs of reasoning-enhanced LLMs can appear human-like, the underlying mechanisms differ substantially and remain an active area of exploration.

1.2 Understanding the standard LLM training pipeline

Let's briefly look at how conventional (nonreasoning) LLMs are typically trained so that we can understand where their limitations lie. This background will also help frame our upcoming discussions on the differences between pattern matching and logical reasoning.

Before applying any specific reasoning methodology, conventional LLM training is usually structured into two stages: *pretraining* and *post-training*, which are illustrated in figure 1.3. Some recent papers also distinguish a *mid-training* stage between the other two, such as to continue training a model on code, math, or long-context data, but I'll group that stage under pretraining to keep the terminology simple in this book.

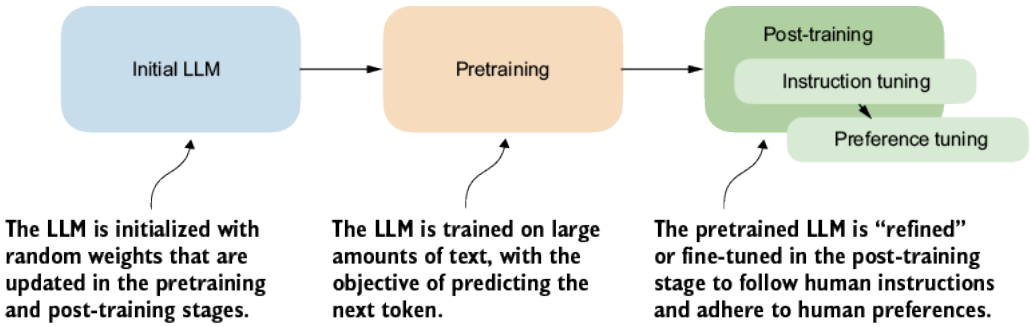


Figure 1.3 Overview of a typical LLM training pipeline. The process begins with a model initialized with random weights, followed by pretraining on large-scale text data to learn language patterns by predicting the next token. Post-training then refines the model through instruction fine-tuning and preference fine-tuning, enabling the LLM to better follow human instructions and align with human preferences.

In the pretraining stage of a typical LLM training pipeline, LLMs are trained on massive amounts of unlabeled text, often many terabytes of data or trillions of tokens, including books, websites, research articles, and many other sources. The pretraining objective for the LLM is to learn to predict the next word (that

is, the next *token*) in these texts.

WORDS AND TOKENS

A token is a small unit of text that a language model processes. A token can be a full word, part of a word, or even punctuation, depending on how the text is split by a tokenizer.

For example, the sentence "An LLM can be useful." might be broken into tokens like "An", " L", "LM", " can", " be", " useful", and "." by a common tokenizer. These tokens are then converted into numerical IDs that the model can ingest.

A tokenizer is not part of the LLM itself, but it is nonetheless a critical component of the LLM text-processing and generation pipeline. We will see how tokenization works in the next chapter.

LLMs become highly capable when they're pretrained on massive datasets, which typically involve several terabytes of text (equivalent to trillions of tokens). This training requires thousands of GPUs running for many months and can cost millions of dollars. Here, "capable" means that LLMs begin to generate text that closely resembles human writing. To some extent, pretrained LLMs also begin to exhibit so-called *emergent properties*, which means they can perform tasks that they were not explicitly trained to do, including translation and code generation.

These pretrained models serve as base models for the post-training stage, which uses two key techniques: *supervised fine-tuning* (often abbreviated as *SFT* in the literature and also known as *instruction tuning*) and *preference tuning* to teach LLMs to respond to user queries.

As shown in figure 1.4, instruction tuning improves an LLM's capabilities for personal-assistance tasks such as question answering, summarizing, translating text, and more. The preference-tuning stage then refines these capabilities. As the term implies, preference tuning helps tailor responses to user preferences. Some readers may be familiar with the term reinforcement learning with human feedback (RLHF), which is a specific technique for implementing preference tuning.

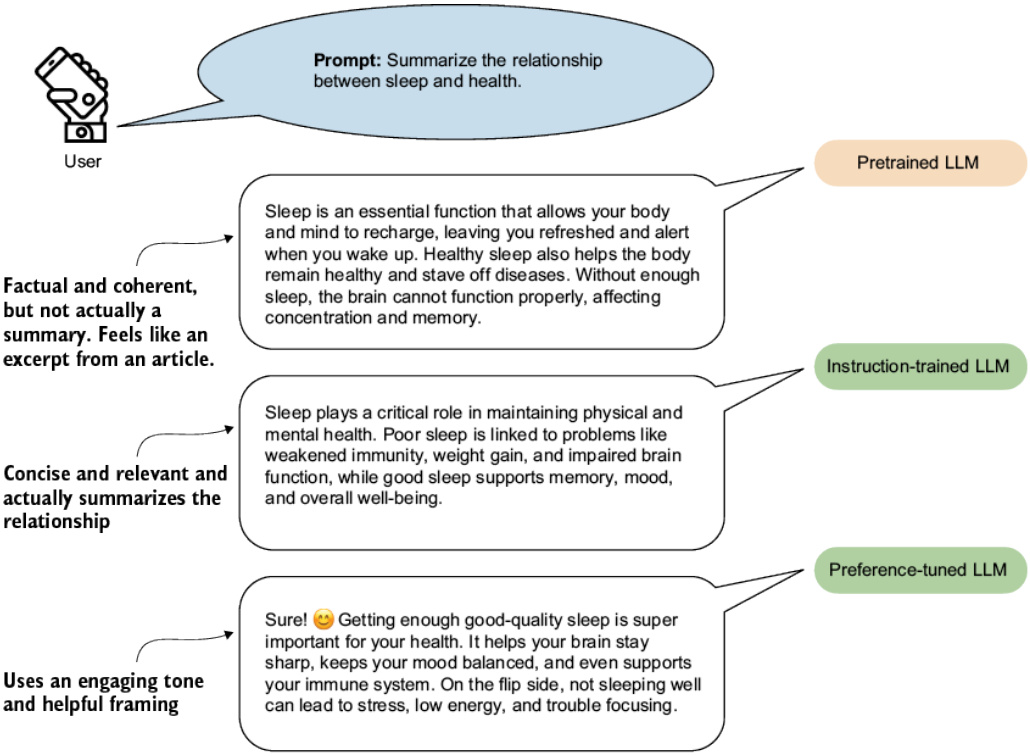


Figure 1.4 Example responses from a language model at different training stages. The prompt asks for a summary of the relationship between sleep and health. The pretrained LLM produces a relevant but unfocused answer without directly following the instructions. The instruction-tuned LLM generates a concise, accurate summary aligned with the prompt. The preference-tuned LLM further improves the response by using a friendly tone and engaging language, making the answer more relatable and user centered.

In short, we can think of pretraining as "raw language prediction" (via next-token prediction) that gives the LLM some basic properties and capabilities to produce coherent texts. The post-training stage then improves the task understanding of LLMs via instruction tuning and refines the LLM to create answers

with preferred stylistic choices via preference tuning.

It is worth noting that even an instruction-tuned model is not yet a "chatbot." A chat interface adds another layer that guides the model's responses in an interactive, multi-turn setting. This typically involves a system prompt, conversation history management, and other orchestration (an example of this is implemented in appendix G).

NOTE These pretraining and post-training stages are covered in my book *Build a Large Language Model (From Scratch)*, published by Manning (**http://mng.bz/orYv**). The book you are reading now does not require detailed knowledge of those stages. Here, we will load a model that has already undergone the expensive pretraining and post-training stages so that we can focus on the methodology specific to reasoning models.

1.3 Improving LLM reasoning with training and inference techniques

Reasoning, in the context of LLMs, entered the public eye with the announcement of OpenAI's o1 in ChatGPT on September 12, 2024. In the announcement article, OpenAI stated, "We've developed a new series of AI models designed to spend more time thinking before they respond" (**https://openai.com/index/introducing-openai-o1-preview/**). Furthermore, OpenAI wrote, "These enhanced reasoning capabilities may be particularly useful if you're tackling complex problems in science, coding, math, and similar fields."

A few months later, in January 2025, DeepSeek released the DeepSeek-R1 model and technical report, which details training methods for developing reasoning models (**https://arxiv.org/abs/2501.12948**). This made big waves because the company not only made freely and openly available a model that competes with and exceeds the performance of the proprietary o1 model but they also shared a blueprint for training such a model. In this book, we'll look at how the methodologies used to develop reasoning models work by implementing similar methods from scratch.

The different approaches to developing and improving an LLM's reasoning capabilities can be grouped into three broad categories, as illustrated in figure 1.5. These methods are typically applied to LLMs that have undergone the conventional pretraining and post-training phases, including instruction and preference tuning.

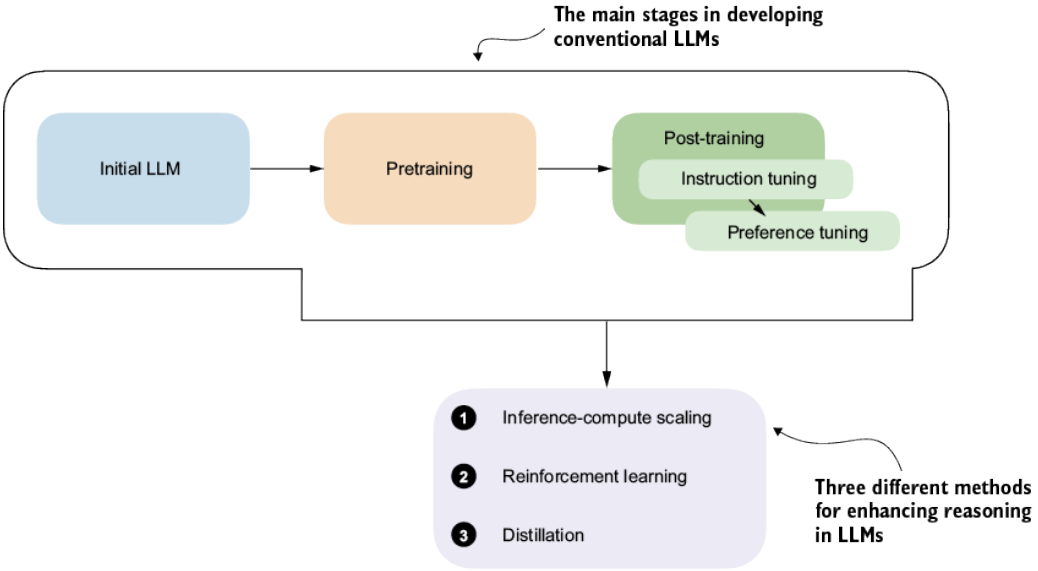


Figure 1.5 Three approaches are commonly used to improve reasoning capabilities in LLMs: inference-compute scaling, reinforcement learning, and distillation. These methods are typically applied after the conventional training stages (initial model training, pretraining, and post-training with instruction and preference tuning), but reasoning techniques can also be applied to the pretrained base model.

We will examine these common approaches in more detail throughout the book:

- *Inference-time compute scaling* (also often called *inference-compute scaling*, *test-time scaling*, and other variations) includes methods that improve the model's reasoning capabilities at inference time (when a user prompts the model) without training or modifying the underlying model weights. The core idea

is to trade increased computational resources for improved performance, making even fixed models more capable through techniques such as chain-of-thought reasoning and various sampling procedures. This will be the focus of chapters 4 and 5.

- *Reinforcement learning* (RL) refers to training methods that improve a model's reasoning capabilities by encouraging it to take actions that lead to high-reward signals. These rewards can be broad, such as task success or heuristic scores, or they can be narrowly defined and verifiable, such as correct answers in math problems or coding tasks.
  Unlike inference-time compute scaling, which can improve reasoning performance without modifying the model, RL updates the model's weights during training. This enables the model to learn and refine reasoning strategies through trial and error based on the feedback it receives from the environment. We will explore RL in more detail in chapters 6 and 7.
- *Distillation* involves transferring complex reasoning patterns learned by larger, more capable models into smaller or more efficient models. In the context of LLMs, this typically means performing supervised fine-tuning (SFT) using high-quality labeled instruction datasets generated by a larger, more capable model. This technique is commonly referred to as *knowledge distillation* or simply *distillation* in LLM literature. It's important to note that this differs slightly from traditional knowledge distillation in deep learning, where a smaller ("student") model typically learns from both the outputs and the logits produced by a larger ("teacher") model. This topic is discussed further in chapter 8.

REINFORCEMENT LEARNING FOR REASONING AND PREFERENCE TUNING

In the context of developing reasoning models, it is important to distinguish the reinforcement learning (RL) approach from reinforcement learning with human feedback (RLHF), which is used during preference tuning when developing a conventional LLM. Both settings use the same underlying process (RL), but they differ primarily in how the reward is obtained and validated (human judgments for RLHF versus automated verifiers or environments for reasoning RL).

RLHF incorporates explicit human evaluations or the rankings of model outputs as reward signals, directly guiding the model toward human-preferred behaviors. In contrast, RL in the context of reasoning models typically relies on automated or environment-based reward signals, which can be more objective but less aligned with human preferences. For instance, RL in a reasoning-model development pipeline might train a model to excel at mathematical proofs by providing explicit rewards for correctness. By contrast, RLHF would involve human evaluators ranking various responses to encourage outputs that align closely with human standards and subjective preferences.

1.4 Pattern matching vs. logical reasoning

During pretraining, LLMs are exposed to vast quantities of text and learn to predict the next token by identifying and reproducing statistical associations in that data. This process enables them to generate fluent and coherent text, but it is fundamentally based on learned statistical regularities rather than explicit rules or guaranteed logical inference.

LLMs respond to prompts by generating text continuations that are statistically consistent with the patterns seen during training. This includes many premise-to-conclusion patterns that can look like logical inference. The key difference is that the model does not explicitly represent premises and apply formal rules but instead predicts likely continuations from similar examples in its training data.

Consider the following example:

The capital of Germany is…

Berlin.

An LLM producing the answer "Berlin" is not logically deducing the answer. Instead, it is recalling a strong statistical association learned from training data. This behavior reflects what we refer to as *pattern matching*, which means that the model completes text based on learned correlations and not by applying structured reasoning steps.

But what about tasks that go beyond pattern recognition, tasks where a correct answer depends on drawing conclusions from given facts? This brings us to a different kind of capability: logical reasoning.

Logical reasoning involves systematically coming up with conclusions using rules. Unlike pattern matching, it depends on intermediate reasoning steps and the ability to recognize contradictions or draw implications based on formal relationships.

Consider the following prompt as an example:

SR All birds can fly. A penguin is a bird. Can a penguin fly?

There are two ways to evaluate this.

First, in a *closed-world* (prompt-only) setting, given the two premises (claims or assumptions) in the prompt, the valid answer is "Yes, a penguin can fly."

Second, in an *open-world* (with background knowledge) setting, if we allow knowledge not included in the prompt (such as that penguins cannot fly), this external fact conflicts with the conclusion derived from the premises, as shown in figure 1.6. A reasoning system should notice the inconsistency and either ask for clarification or weaken the first statement (for example, "Most birds can fly, with exceptions such as penguins.").

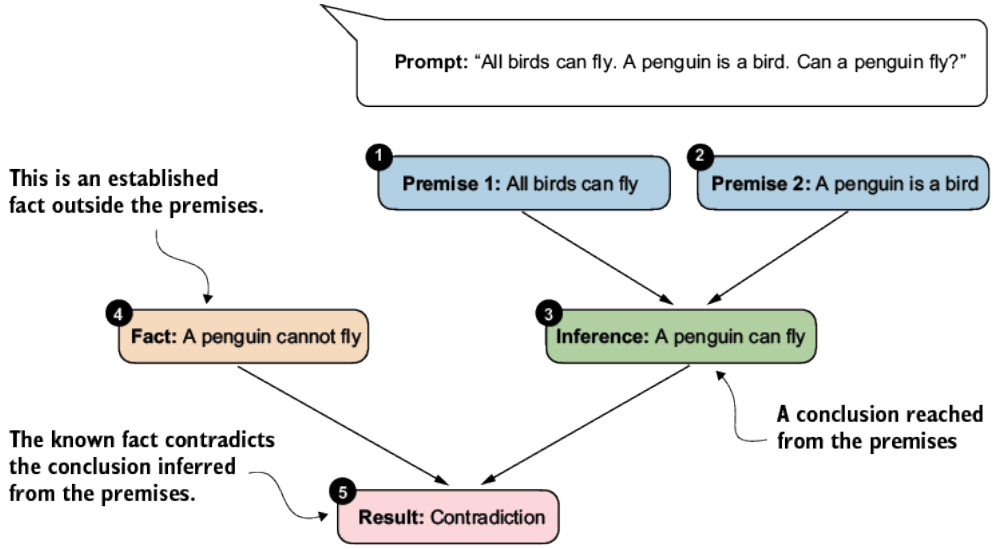


Figure 1.6 Contradictory premises lead to a logical inconsistency. From "All birds can fly" and "A penguin is a bird," we infer "A penguin can fly." This conclusion conflicts with the established fact "A penguin cannot fly," which results in a contradiction.

In contrast, a statistical (pattern-matching) LLM does *not* explicitly track contradictions but instead predicts based on learned text distributions. For instance, if information such as "All birds can fly" is strongly reinforced in the training data, the model may confidently answer, "Yes, penguins can fly."

In the next section, we'll look at an example of how an LLM handles this "All birds can fly..." prompt.

LOGICAL REASONING AND RULE-BASED SYSTEMS

Why are explicit rule-based systems not more popular? Rule-based systems were widely used in the '80s and '90s for medical diagnosis, legal decisions, and engineering. They are still used in critical domains (medicine, law, aerospace), which often require explicit inference and transparent decision processes. They are hard to implement because they largely rely on human-crafted heuristics. (Heuristics are simple decision rules that give a good-enough answer quickly without guaranteeing an optimal one.) In contrast, deep neural networks, including LLMs, do not implement handwritten rules; they learn decision patterns from data and can be highly flexible when trained at scale.

1.5 Simulating reasoning without explicit rules

We saw how contradictory premises can lead to logical inconsistencies. A conventional LLM does not explicitly track contradictions but generates responses based on learned text distributions.

Figure 1.7 shows how a non-reasoning-enhanced LLM like GPT-4o in OpenAI's ChatGPT responds to the

"All birds can fly..." prompt discussed in the previous section. As you can see, GPT-4o appears to answer correctly even though it is not considered a reasoning model (unlike OpenAI's other offerings, such as o1, o3, o4-mini, and the more recent GPT-5, which have been explicitly developed with reasoning methodology).

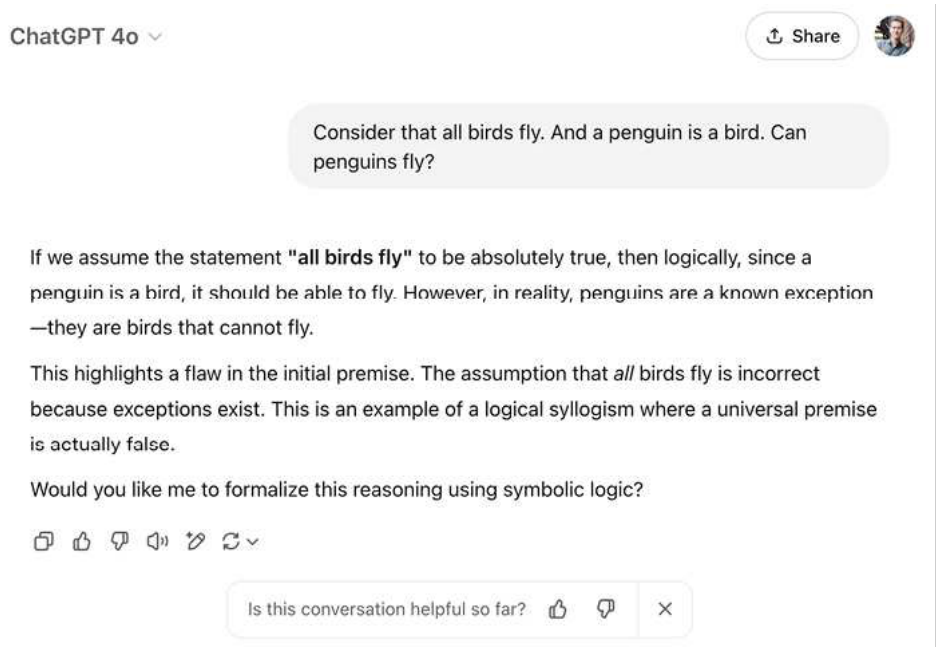


Figure 1.7 An illustrative example of how a language model (GPT-4o in ChatGPT) appears to "reason" about a contradictory premise

So how did the 4o model generate its answer? Does this mean GPT-4o explicitly reasons logically? No, not necessarily. At a minimum, GPT-4o is highly effective at simulating logical reasoning in familiar contexts.

GPT-4o does not implement explicit contradiction checking; it instead generates answers based on probability-weighted patterns. This approach works well enough if the training data includes many instances that correct the contradiction (for example, text like "penguins cannot fly") so that the model learns a statistical association between "penguins" and "not flying." In other words, the model recognizes the contradiction implicitly because it has frequently encountered this exact reasoning scenario during training. This effect relies heavily on statistical associations built from extensive exposure to reasoning-like patterns in training data. So, even when a conventional LLM seems to perform logical deduction, as shown in figure 1.7, it's not executing explicit, rule-based logic but instead leveraging patterns from its vast training data.

Still, GPT-4o's success here is a good illustration of how powerful implicit pattern matching can become when trained at a massive scale. These pattern-based reasoning models usually struggle in a couple of scenarios:

- When the logical scenario is novel (not previously encountered in training data)
- When reasoning complexity is high, involving intricate, multistep logical relationships

LOGICAL REASONING VS. REASONING MODELS CURRENTLY AVAILABLE TODAY

Although GPT-4o is not officially labeled as a reasoning model, OpenAI offers several dedicated reasoning models, including o1, o3, o4-mini, and GPT-5. Moreover, other companies have been developing LLMs with explicit reasoning capabilities. As of this writing, popular examples include Anthropic's Claude 4, xAI's Grok 4, Google's Gemini 2.5, DeepSeek's R1, Alibaba's Qwen3, and many more. The techniques employed by these models are the focus of this book. As we'll see, this happens without implementing a rule-based reasoning pipeline. Instead, the LLM will learn or improve its reasoning capabilities through modified inference and training methodologies.

We might say that LLMs simulate logical reasoning through learned patterns. We can improve this further with specific reasoning methods, including inference-compute scaling and post-training strategies such as reinforcement learning, but the LLMs are not explicitly executing any rule-based logic internally.

Moreover, LLMs exist on a spectrum. Even before the advent of dedicated reasoning models such as Ope-

nAI's o1 and DeepSeek-R1, LLMs were capable of simulating reasoning behavior. These models exhibited behaviors aligning with the chapter's earlier definition of reasoning in section 1.1, namely, generating intermediate steps before producing a final answer. What we now explicitly label a "reasoning model" is essentially a more refined version of this capability. These improved reasoning capabilities are achieved by using specific inference-compute scaling techniques (discussed in chapters 4 and 5) and targeted post-training methods, such as reinforcement learning (chapters 6 and 7), designed to improve and reinforce reasoning-like behavior.

1.6 Why build reasoning models from scratch?

Following the release of DeepSeek-R1 in January 2025, improving the reasoning abilities of LLMs has become one of the hottest topics in AI, and for good reason. Stronger reasoning skills allow LLMs to tackle more complex problems, making them more capable across various tasks that users care about.

This shift is also reflected in a February 12, 2025, statement from OpenAI's CEO:

> *We will next ship GPT-4.5, the model we called Orion internally, as our last non-chain-of-thought model. After that, a top goal for us is to unify o-series models and GPT-series models by creating systems that can use all our tools, know when to think for a long time or not, and generally be useful for a very wide range of tasks.*

The preceding quote underlines the major shift among leading LLM providers toward reasoning models. Here, "chain-of-thought" refers to a prompting technique that guides language models to reason step by step to improve their reasoning capabilities; we'll cover it in more detail in chapters 4 and 5.

Also noteworthy is the mention of knowing "when to think for a long time or not." This hints at an important design consideration: reasoning is not always necessary or desirable. For instance, reasoning models are designed for complex tasks such as solving puzzles, advanced math problems, and challenging coding tasks. They are also often useful in agent systems, especially for task decomposition, planning, and choosing which tools to use. They are not necessary for simpler tasks like summarization, translation, or knowledge-based question answering. In fact, using reasoning models for everything can be inefficient and expensive. For instance, they are typically more expensive to use, more verbose, and sometimes more prone to errors due to "overthinking." Here, the simple rule applies: use the right tool (or type of LLM) for the task.

Reasoning models are often more expensive than non-reasoning models for two reasons. First, they tend to produce longer outputs because they include intermediate steps that explain how an answer is derived. As figure 1.8 illustrates, LLMs generate text one token at a time, and each token requires a *full forward pass*. If a reasoning model's answer is twice as long, generation involves roughly twice as many forward passes, which increases compute costs.

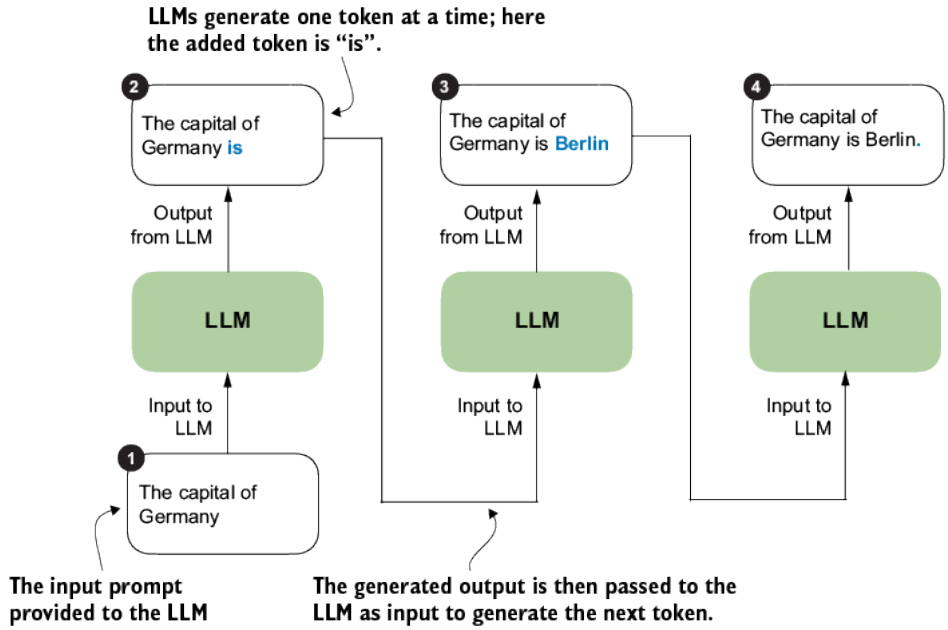


Figure 1.8 Token-by-token generation in an LLM. At each step, the LLM takes the full sequence generated so far and predicts the next token, which may represent a word, subword, or punctuation mark, depending on the tokenizer. The newly generated token is appended to the sequence and used as input for the next step. This iterative decoding process is used in both standard language models and reasoning-focused models.

Second, many reasoning workflows require running the model several times for a single task, such as to sample multiple candidate solutions, call tools, or run a verifier. These additional calls multiply the total number of tokens processed and further increase cost beyond the single-call behavior shown in figure 1.8.

That's exactly why it helps to implement these models and methods from scratch. It's one of the best ways to understand how they work. And if we understand how LLMs and reasoning models work, we can better understand these tradeoffs.

1.7 A road map to building reasoning models from scratch

Now that we've discussed reasoning in LLMs from a bird's-eye view, the subsequent chapters will guide you through coding and applying reasoning methods from scratch. At a high level, the road map is simple. We'll start with a conventional LLM, learn how to evaluate its reasoning behavior, and then explore two broad ways to improve it: inference techniques and training techniques. Figure 1.9 illustrates the main steps.

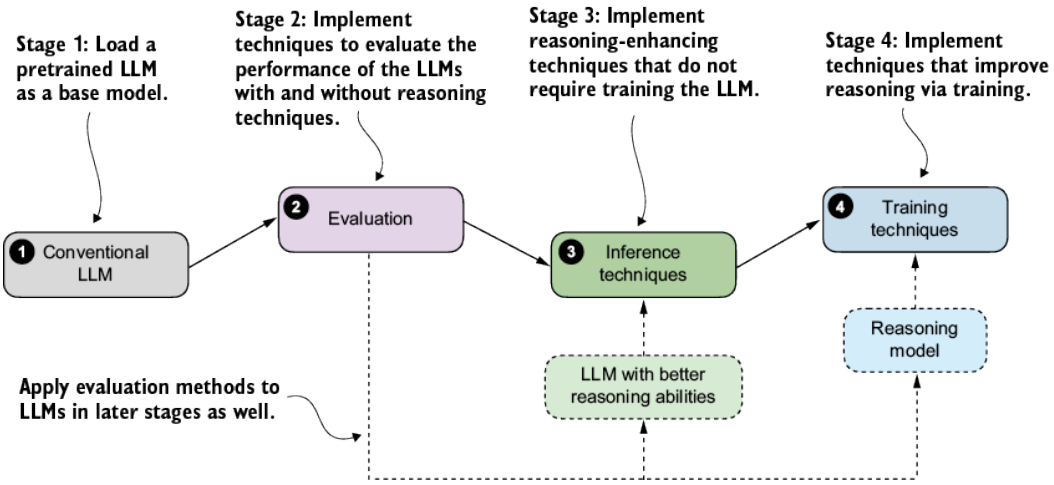


Figure 1.9 A high-level road map of what we'll build in this book. We'll start with a conventional LLM, add evaluation methods so we can measure its progress, and then explore two broad families of reasoning improvements: inference techniques and training techniques.

Stage 1 is an explicit step in figure 1.9 because, in practice, reasoning methods are usually applied to an existing base LLM rather than one trained from scratch. We'll begin with a conventional LLM that has already been pretrained, and we'll treat it as the baseline we want to improve.

Stage 2 implements the evaluation tools needed to tell whether a method actually helps. That's why stages 3 and 4 loop back to evaluation: after improving reasoning behavior through inference-time methods or additional training, we return to stage 2 to measure whether the change helped.

Stage 3 improves reasoning behavior at inference time without changing the model weights, while stage 4 improves the model itself through additional training, turning the pretrained LLM into a dedicated reasoning model.

Figure 1.10 provides more detail and summarizes the different substeps covered in each chapter.

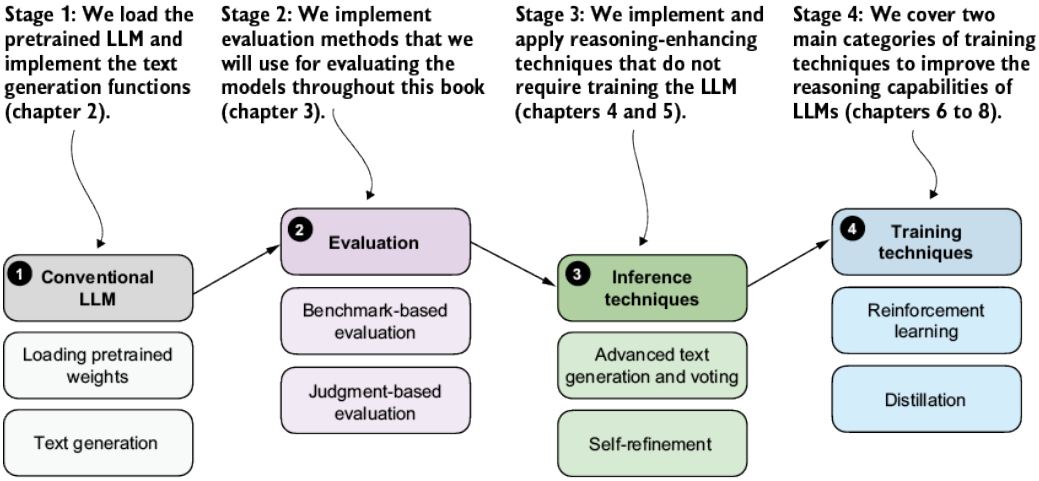


Figure 1.10 A detailed road map of the chapter-level substeps. After loading the base model, we'll cover benchmark-based and judgment-based evaluation, then inference-time methods such as advanced text generation, voting, and self-refinement, and finally training-time methods based on reinforcement learning and distillation.

I'm looking forward to the journey ahead and hope you are as well.

Summary

- Conventional LLM training occurs in several stages:
  - Pretraining, in which the model learns language patterns from vast amounts of text
  - Instruction fine-tuning, which improves the model's responses to user prompts
  - Preference tuning, which aligns model outputs with human preferences
- Reasoning methods are applied on top of a conventional LLM.
- Reasoning in LLMs refers to improving a model so that it explicitly generates intermediate steps (chain of thought) before producing a final answer, which often increases accuracy on multistep tasks.
- Reasoning in LLMs is different from rule-based reasoning, and it likely also works differently from human reasoning. Currently, the consensus is that reasoning in LLMs relies on statistical pattern matching.
- Pattern matching in LLMs relies purely on statistical associations learned from data, enabling fluent text generation but lacking explicit logical inference.
- Improving reasoning in LLMs can be achieved in a few ways:
  - Inference-time compute scaling improves reasoning without retraining (e.g., chain-of-thought prompting).
  - Reinforcement learning involves training models explicitly with reward signals.
  - Supervised fine-tuning and distillation, using examples from stronger reasoning models.
- Building reasoning models from scratch provides practical insights into LLM capabilities, limitations, and computational tradeoffs.

2 Generating text with a pretrained LLM

This chapter covers

- Setting up the code environment for working with LLMs
- Using a tokenizer to prepare input text for an LLM
- The step-by-step process of generating text with a pretrained LLM
- Caching and compilation techniques for speeding up LLM text generation

In the previous chapter, we discussed the difference between *conventional large language models* (LLMs) and *reasoning models*. We also introduced several techniques that can improve the reasoning capabilities of LLMs. These techniques are usually applied on top of a conventional (base) LLM.

In this chapter, we'll lay the groundwork for the upcoming chapters by loading a pretrained base model, as shown in figure 2.1. Previously, we discussed how reasoning methods are often added after the usual post-training stages. But starting from a base model makes it easier to see which capabilities come from the reasoning methods themselves. The conventional LLM in this chapter is a nonreasoning LLM, and more specifically, a pretrained base model rather than an instruction- or preference-tuned assistant.

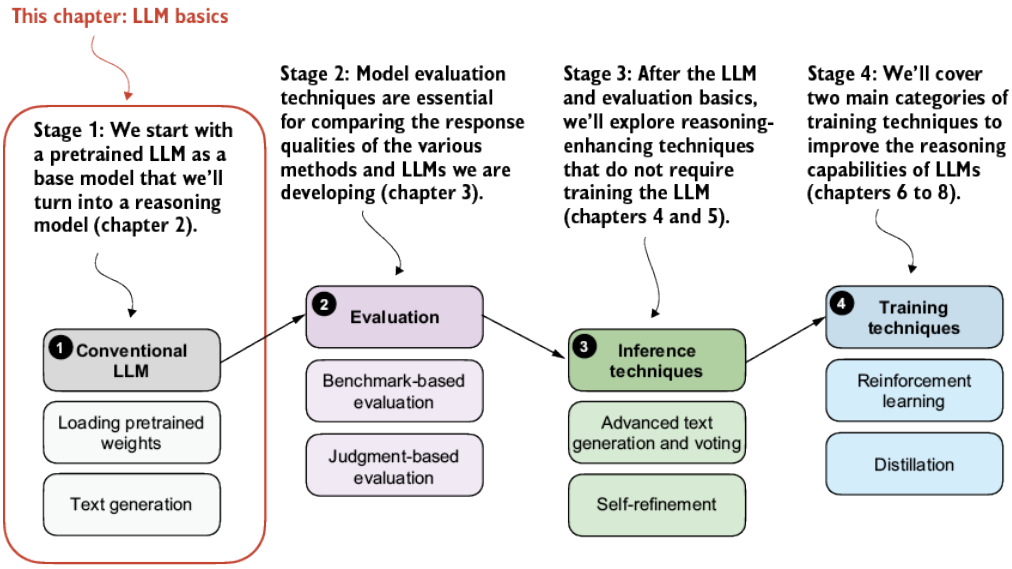


Figure 2.1 The four main stages of developing a reasoning model. This chapter focuses on stage 1: loading a conventional LLM and implementing the text generation functionality.

In addition to setting up the coding environment and loading a pretrained LLM, you will learn how to use a *tokenizer* to prepare text input for the model. As shown in figure 2.1, you will also implement a text generation function, enabling practical use of the LLM to generate text. This functionality will be used and improved on in later chapters.

2.1 Introducing LLMs for text generation

The LLM we'll use in this chapter is a base model that has undergone only pretraining, but it will already be capable of generating coherent text and, in some cases, of following basic instructions. Figure 2.2 summarizes the components of an LLM text generation pipeline, and we'll discuss and implement these steps in more detail later in this chapter.

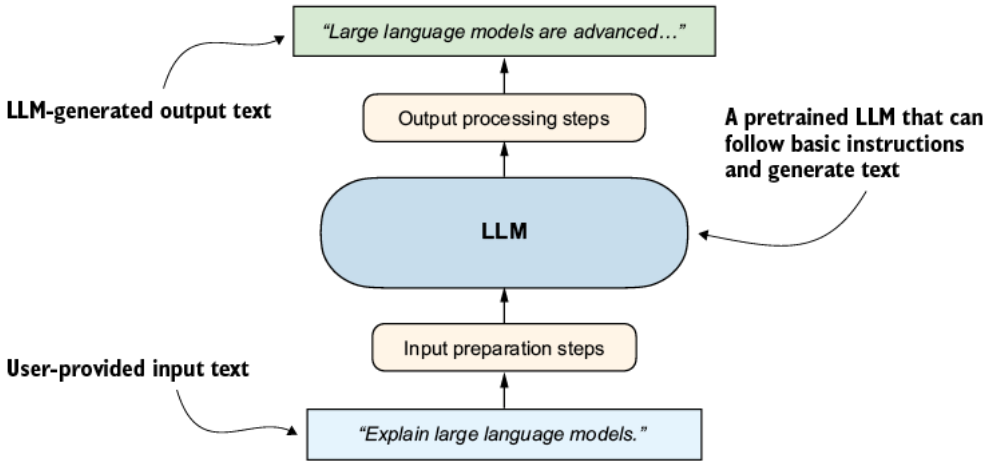


Figure 2.2 An overview of an LLM generating a response (output text) given a user query (input text)

NOTE By convention, diagrams involving neural networks such as LLMs are drawn and read vertically from bottom (inputs) to top (outputs). Arrows indicate the flow of information upward through the model.

If you have not coded an LLM or used LLMs programmatically before, this chapter will teach you how the text generation process works. We won't go deep into the internals of an LLM, such as the attention mechanism and other architectural components; this is the topic of my other book, *Build a Large Language Model (From Scratch)*. Understanding those internals is not required for this book; if you are curious, you can learn about them afterwards. This chapter will also be helpful if you have already read my other book, since it adds new material on speeding up inference and other practical optimizations.

Before we begin implementing the components shown in figure 2.2, including input preparation, loading the LLM, and generating text, we first need to set up our coding environment.

2.2 Setting up the coding environment

We'll look at two main ways to set up your Python coding environment so you can follow along with the examples in this book: a straightforward `pip`-based setup and a `uv`-based workflow. I recommend reading through this section before deciding which option is best for you. If you already have a Python environment set up (with Python 3.10 or newer), the simplest way to install dependencies is to use Python's package installer (`pip`) in your terminal. If you have downloaded the code from the publisher's website, use the `requirements.txt` file to install the Python libraries required for this book:

```
pip install -r requirements.txt
```

Alternatively, to install the required packages directly without downloading the `requirements.txt` file, use this command:

```
pip install -r https://raw.githubusercontent.com/\
rasbt/reasoning-from-scratch/refs/heads/main/requirements.txt
```

PYTHON PACKAGES USED IN THIS CHAPTER

If you prefer to install only the packages used in this chapter, you can start with the following command:

```
pip install torch>=2.10.0 tokenizers>=0.22.2 reasoning-from-scratch
```

PyTorch installation can be platform- and hardware-specific, especially if you want GPU acceleration. If the preceding command does not install the right build for your system, I recommend using the installation selector on the official PyTorch website (**https://pytorch.org/get-started/locally/**) and then installing the remaining packages separately:

- `torch` refers to PyTorch, a widely used deep learning library that provides tools for building and training neural networks.
- `tokenizers` is a library that provides efficient tokenization algorithms used to prepare input data for LLMs.
- `reasoning-from-scratch` is a custom library that I developed for this book. It includes all the code examples used throughout the chapters, along with additional utility functions we will be using.

While `pip` is the canonical way to install Python packages, my preferred way to use Python is with the widely recommended `uv` Python package and project manager instead. Like many others, I now recommend `uv` because it is significantly faster and handles dependency resolution more reliably than `pip`. It also creates isolated environments automatically and comes with its own Python executable (but will use the system Python if a compatible version is already installed), so it's a great option if you do not have Python installed on your system yet. Figure 2.3 outlines the four-step process, from installing `uv` to getting ready to execute the code in this chapter.

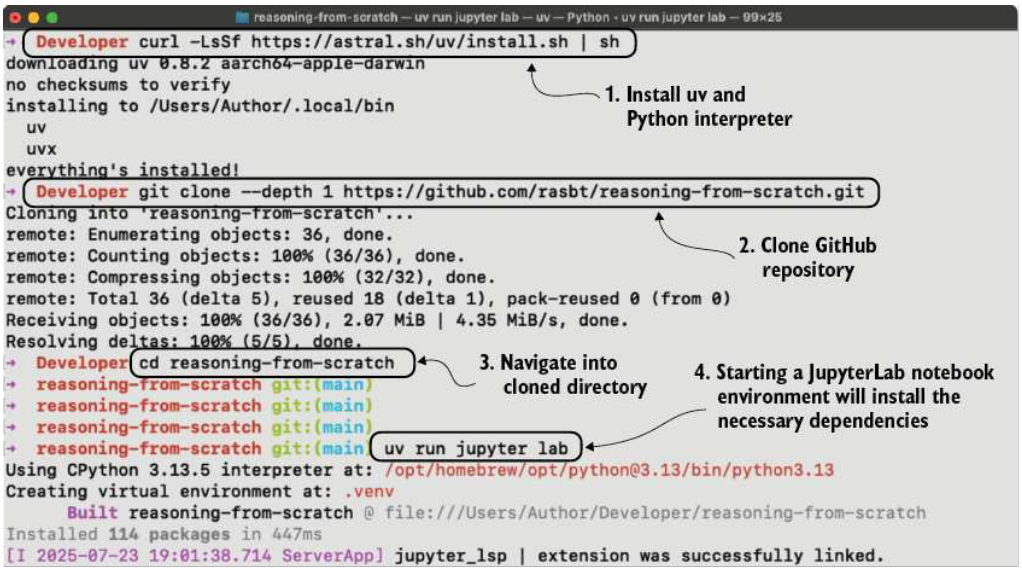


Figure 2.3 Installing and using the uv Python package and project manager via the macOS terminal

Figure 2.3 shows the installation and use of `uv` in a macOS terminal, but it's also supported on Linux and Windows:

1. To install `uv`, run the installer for your OS from the official website: **https://docs.astral.sh/uv/getting-started/installation/**.
2. Next, clone the GitHub repo:

```
git clone --depth 1 https://github.com/rasbt/reasoning-from-scratch.git
```

Here, the `--depth 1` option tells `git` to perform a shallow clone, which means it only downloads the latest version of the code without the full version history. This makes the download faster and uses less space.

If you don't have `git` installed, you can manually download the source code repository from the publisher's website or by opening this link in your browser: **https://github.com/rasbt/reasoning-from-scratch/archive/refs/heads/main.zip** (unzip it after downloading).

3. In the terminal, navigate to the `reasoning-from-scratch` folder.
4. Inside the `reasoning-from-scratch` folder, execute

```
uv run jupyter lab
```

This command launches JupyterLab, where you can open a blank Jupyter notebook to type and run code, or you can open the chapter 2 notebook that contains all the code covered in this chapter. (You don't need to create or activate a virtual environment first.)

Python script files can be executed via `uv run script-name.py`. This command also sets up a local virtual environment (usually inside an invisible `.venv/` folder) and automatically installs all dependencies from the `pyproject.toml` file in the `reasoning-from-scratch` folder, so manually installing code dependencies via the requirements file is not needed. However, if you plan to install additional packages, you can use the following command:

```
uv add packagename
```

The supplementary code repository contains additional installation instructions and details inside the `ch02` subfolder if you need them.

2.3 Understanding hardware needs and recommendations

You may have heard that training LLMs is very expensive. For leading LLM companies, it is not uncommon to spend anywhere from $1–$10 million on the low end to more than $50 million on the high end in compute costs to train a new base-model LLM before adding any reasoning techniques.

For example, the DeepSeek V3 model, which serves as the base checkpoint for the DeepSeek R1 reasoning system, was trained on 2,048 NVIDIA H800 GPUs for about 11 weeks at an estimated cost of US$5.5 million. (DeepSeek V3 is one of the few recent models with a fully transparent compute disclosure.) According to the technical report, the final training run used 14.8 million GPU-hours of compute. The energy usage was roughly 620 MWh, about the amount of electricity an average American household uses in 55 years.

These resource requirements and the high price tag would make developing an LLM unfeasible for me and most readers. Instead, we will use a relatively small (but capable) pretrained LLM, on top of which we'll implement reasoning techniques. This smaller LLM is a scaled-down version that otherwise follows the same architecture as contemporary state-of-the-art models. The reasoning methods that we'll apply are the same as those used by larger LLMs. The difference is that the smaller LLM will let us explore these methods in a budget-friendly way.

As an analogy, imagine you are curious to learn how cars work. If you are new to cars, you probably wouldn't start out building an expensive Ferrari right away. Instead, you would start with a more basic car, like a Volkswagen Beetle, which will still teach you a lot about how engines and transmissions work. In fact, I would even say that working on a simpler car would help you *better* understand how the engine and transmission work because it doesn't have the complicated refinements of a more expensive car.

While we'll use a relatively small model in this book, using, developing, and applying the reasoning techniques are still computationally intensive. The techniques in later chapters, such as chapters 5–8, will benefit from a GPU. You can check whether your computer has a PyTorch-supported GPU by running the following PyTorch code in Python:

```
import torch

print(f"PyTorch version {torch.__version__}")

if torch.cuda.is_available():
    print(f"CUDA/ROCm GPU: {torch.cuda.get_device_name(0)}")

elif torch.xpu.is_available():
    print(f"Intel GPU: {torch.xpu.get_device_name(0)}")

elif torch.backends.mps.is_available():
    print("Apple Silicon GPU")

else:
    print("Only CPU")
```

Depending on your machine, the code may return the following results:

```
PyTorch version 2.10.0
Only CPU
```

Don't worry if your machine does not have a GPU to run the code. Chapters 2–5 can be run in a reasonable time on a CPU. For some chapters, the code will automatically use an NVIDIA GPU if one is available; otherwise, it will run on the CPU (or an Apple Silicon GPU if recommended for a particular section or chapter). I will provide more information in the relevant sections and chapters.

NOTE If you want GPU acceleration and are unsure which PyTorch build to install, use the official PyTorch install selector (**https://pytorch.org/get-started**) to choose the correct CPU, CUDA, or ROCm build for your system. For this book, any recent supported PyTorch CUDA build is fine.

Like many other AI researchers who work on and with LLMs daily, I don't have a machine with the necessary GPU hardware to train LLMs at home, so I use cloud resources instead. If you are looking for cloud provider options, my personal preference is Lightning AI Studio (**https://lightning.ai/**) because of its ease of use and feature support, as shown in figure 2.4. Google Colab (**https://colab.research.google.com/**) is another good choice.

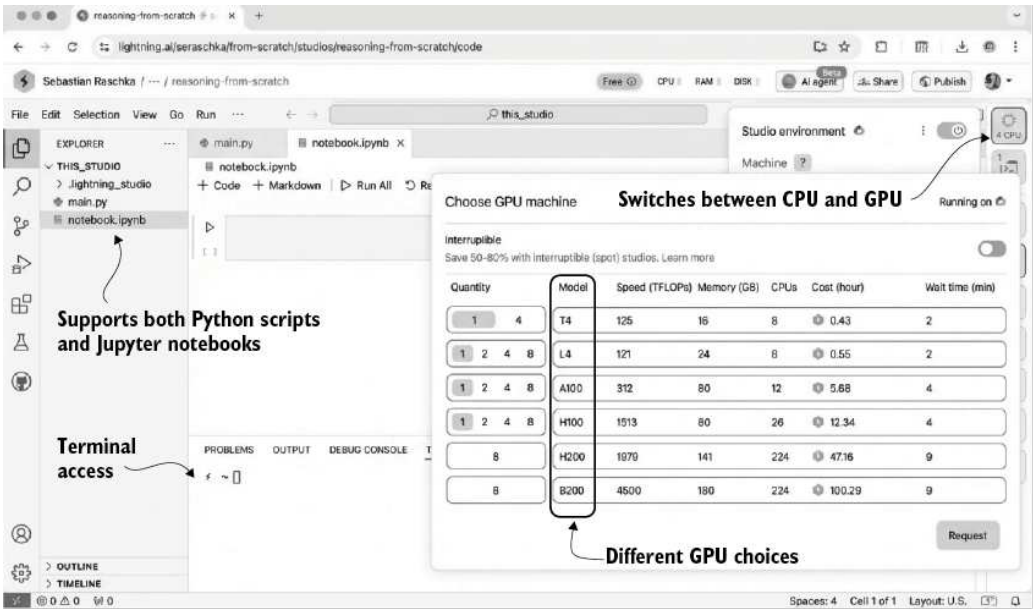


Figure 2.4 An overview of the Lightning AI GPU cloud platform in a web browser. The interface supports Python scripts, Jupyter notebooks, and terminal access, and it lets users switch between CPU and various GPU types based on their compute needs. (Colors were inverted and the image was converted to grayscale for print reproduction.)

As of this writing, Lightning AI also offers users free compute credits after the sign-up and verification process, and these can be used for the different GPU choices shown in figure 2.4. (As I've mentioned, a GPU is not needed for this chapter, but if you want to use one, the L4 GPU is more than sufficient.)

NOTE For disclosure: I helped build and launch the Lightning AI platform in 2023, but I no longer have any financial interest in the company. I am not sponsored to recommend it, and I pay for it myself. I use it because I find it the most convenient option. It supports multiple types of GPUs, allows easy switching between them and back to CPU to save costs, and lets me pause or resume environments without redoing the setup.

The supplementary code repository contains additional GPU platform recommendations inside the `ch02` subfolder.

USING PYTORCH

In this section, we imported and used the PyTorch library, which is currently the most widely used general-purpose library. We will use it throughout this book to run and train LLMs, including the reasoning methods we will develop. If you are new to PyTorch, I recommend reading my "PyTorch in One Hour: From Tensors to Training Neural Networks on Multiple GPUs" tutorial to get the most out of this book. It is freely available on my website at **https://sebastianraschka.com/teaching/pytorch-1h**.

2.4 Preparing input texts for LLMs

Let's now explore how we can use a tokenizer to process input and output text for an LLM. Tokenizers are essential because every prompt, reasoning trace, and generated answer ultimately has to be represented as token IDs before the model can process them. Figure 2.5 provides a more detailed view of the input and output preparation steps shown earlier in figure 2.2 .

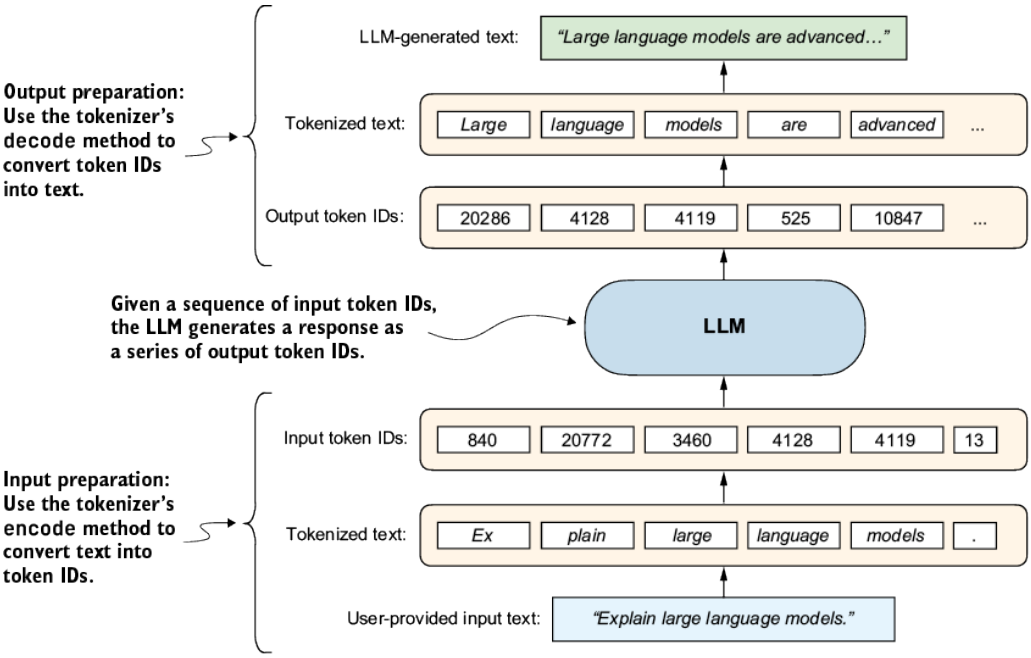


Figure 2.5 A simplified illustration of how an LLM receives input data and generates output. The user-provided text is tokenized into IDs using the tokenizer's encode method, which are then processed by the LLM to generate output token IDs. These are decoded back into human-readable text using the tokenizer's decode method.

To see how this works in practice, we'll begin by loading a tokenizer from this book's `reasoning-from-scratch` Python package (installed in section 2.2). To download the tokenizer files (corresponding to the Qwen3 0.6B base LLM), run this command:

```
from reasoning_from_scratch.qwen3 import download_qwen3_small
download_qwen3_small(kind="base", tokenizer_only=True, out_dir="qwen3")
```

This will display a progress bar:

```
tokenizer-base.json: 100% (6 MiB / 6 MiB)
```

The command downloads the `tokenizer-base.json` file (approximately 6 MB in size) and saves it in a `qwen3` subdirectory.

Now we can load the tokenizer settings from the tokenizer file into `Qwen3Tokenizer`:

```
from pathlib import Path
from reasoning_from_scratch.qwen3 import Qwen3Tokenizer

tokenizer_path = Path("qwen3") / "tokenizer-base.json"
tokenizer = Qwen3Tokenizer(tokenizer_file_path=tokenizer_path)
```

Since we have not loaded the LLM yet (the central component in figure 2.5), we will start with a simpler dry run using just the tokenizer. Specifically, we will do a tokenization round trip: encode a text into token IDs and then decode those IDs back to text, as illustrated in figure 2.6.

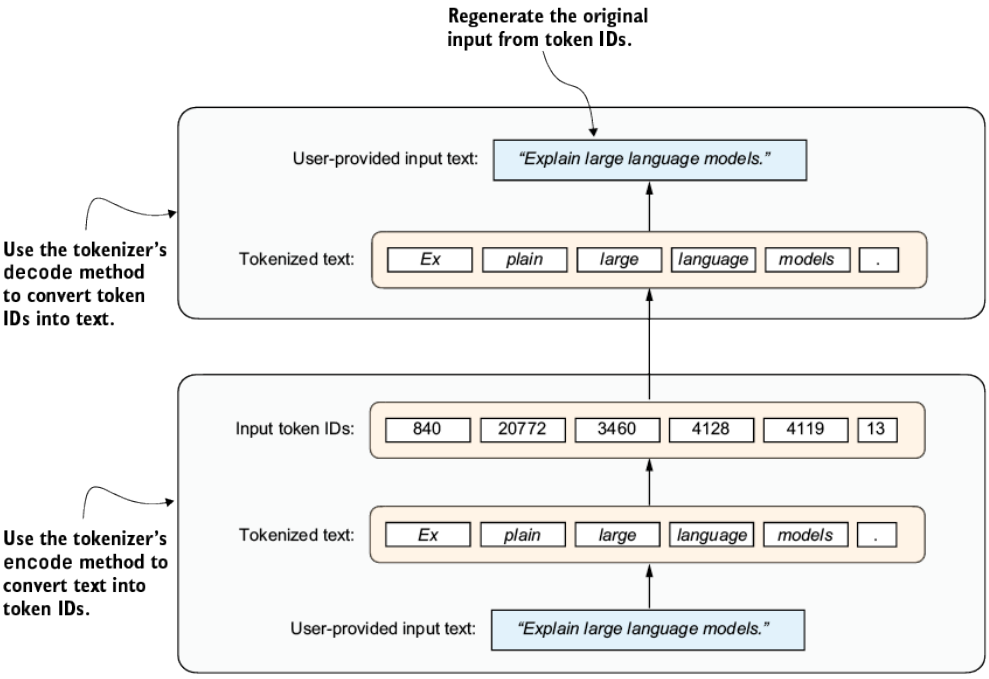


Figure 2.6 The round-trip tokenization process using a tokenizer. The user-provided input text is first converted into token IDs using the encode method and then reconstructed into the original text using the decode method.

The following code snippet implements the encoding process shown at the bottom of figure 2.6:

```
prompt = "Explain large language models."
input_token_ids_list = tokenizer.encode(prompt)
```

The following code implements the decoding process, converting the token IDs back into text, shown at the top of figure 2.6:

```
text = tokenizer.decode(input_token_ids_list)
print(text)
```

Based on the following printed result, we can see that the tokenizer reconstructed the original input prompt from the token IDs:

```
'Explain large language models.'
```

Before we move on to the LLM, let's take a look at the token IDs generated by the `encode` method. The following code prints each token ID and its corresponding decoded string to illustrate how the tokenizer works:

```
for i in input_token_ids_list:
    print(f"{i} --> {tokenizer.decode([i])}")
```

The output is as follows:

```
840 --> Ex
20772 --> plain
3460 -->  large
4128 -->  language
4119 -->  models
13 --> .
```

As you can see in the output, the original text is split into six token IDs. Each token represents a word or subword, depending on how the tokenizer segments the input.

For example, “Explain” was split into two separate tokens, “Ex” and “plain.” This is because the tokenizer algorithm uses a subword-based method called *byte pair encoding* (BPE). BPE can represent both common and rare words using a mix of full words and subword units. Spaces are also often included in tokens (for example, “ large”), which helps the LLM detect word boundaries.

The `Qwen3Tokenizer` has a vocabulary of about 151,000 tokens, which is relatively large as of this writing (for comparison, early GPT-2 has a vocabulary of approximately 50,000 tokens, and Llama 3 has a vocabulary of approximately 128,000 tokens).

A larger vocabulary in a language model increases the model’s size because the embedding and output layers must store more token representations. It can also increase the per-token compute cost of producing next-token probabilities. A larger vocabulary also allows more words to be represented as single tokens rather than being split into subword components, which can reduce sequence length. For example, splitting a word like “Explain” into “Ex” and “plain” results in more input tokens. More tokens lead to longer input sequences, which increases processing time and resource usage. The tradeoff is between a larger vocabulary with a somewhat higher per-token cost and a smaller vocabulary that often produces longer token sequences. For instance, doubling the number of tokens in a sequence can roughly double the computational cost of running the model, since the model has to process and generate more tokens overall.

Unfortunately, the details of tokenizers and their implementations are outside the scope of this book. If you’re interested, you can find additional resources, including my from-scratch implementation, in appendix A.

EXERCISE 2.1: ENCODING UNKNOWN WORDS

Experiment with the tokenizer to see if and how it handles unknown words. To do this, get creative and make up words that don’t exist. Also, if you speak multiple languages, try to encode words in a language other than English.

2.5 Loading pretrained models

In the previous section, we loaded and familiarized ourselves with the tokenizer, which prepares the input data for an LLM and converts LLM outputs back into human-readable text. In this section, we’ll load the LLM itself with its pretrained weights, as shown in figure 2.7. Once this base model is in place, the following chapters will build directly on it by evaluating it, prompting it in different ways, and eventually training it into a stronger reasoning model.

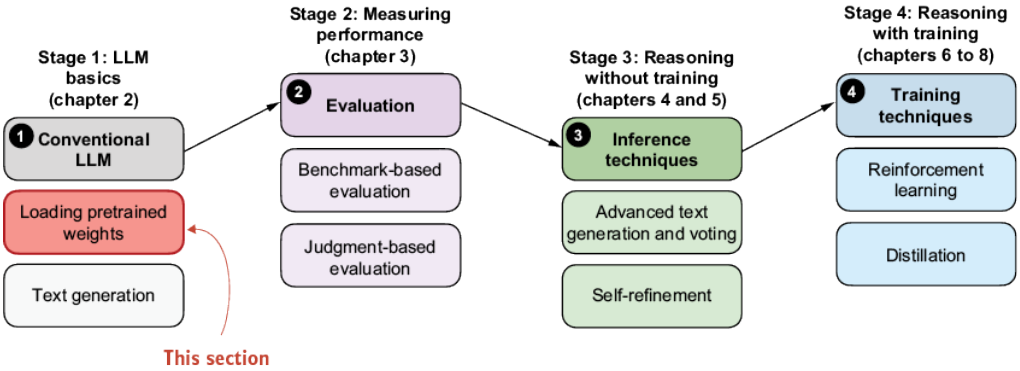


Figure 2.7 An overview of the four key stages in developing a reasoning model in this book. This section focuses on loading a pretrained LLM in stage 1.

This book uses Qwen3 0.6B as its pretrained base model. The "0.6B" in the name indicates that the model contains approximately 0.6 billion weight parameters.

Why use Qwen3? After carefully evaluating several open-weight base models, I chose Qwen3 0.6B for the following reasons:

- For this book, we want a small yet capable open-weight model (open-weight models are models whose trained weights are publicly downloadable) that can run on consumer hardware.
- Qwen3 offers both a base model (the focus of our reasoning model development) and an official reasoning variant that we can use as a reference for evaluation purposes.

NOTE The canonical spelling of "Qwen3" does not include whitespace, whereas, for example, "Llama 3" does.

In the spirit of building things "from scratch," this book uses a custom reimplementation of Qwen3 that I wrote in pure PyTorch and that is entirely independent of external LLM libraries. The emphasis of this reimplementation is on code readability and tweakability in case you want to modify it later for your own experiments. Despite being built from scratch, this implementation remains fully compatible with the original pretrained Qwen3 model weights.

This book does not cover the Qwen3 code implementation in depth. That topic alone would fill a separate book, similar to my *Build a Large Language Model (From Scratch)* book. Instead, this book focuses on implementing reasoning methods on top of a base model, in this case, Qwen3. If you're interested in more details about Qwen3 and the model code, see appendix C.

NOTE This reimplemented Qwen3 LLM runs entirely locally, just like any other neural network implemented in PyTorch. There are no server-side components or external API calls involved. All model usage happens on your own machine, and your data stays on your device. If you are concerned about privacy, the setup we are using ensures full control over both the LLM inputs and outputs.

Before we load the model, we can specify the device we are going to use, namely, a CPU or GPU. The code in the following listing automatically selects the best available device.

Listing 2.1 Get the device automatically

```
def get_device(enable_tensor_cores=True):
    if torch.cuda.is_available():
        device = torch.device("cuda")
        print("Using NVIDIA CUDA GPU")

        if enable_tensor_cores:
            major, minor = map(int, torch.__version__.split(".")[:2])
            if (major, minor) >= (2, 9):
                torch.backends.cuda.matmul.fp32_precision = "tf32"
                torch.backends.cudnn.conv.fp32_precision = "tf32"
            else:
                torch.backends.cuda.matmul.allow_tf32 = True
                torch.backends.cudnn.allow_tf32 = True

    elif torch.backends.mps.is_available():
        device = torch.device("mps")
        print("Using Apple Silicon GPU (MPS)")

    elif torch.xpu.is_available():
        device = torch.device("xpu")
        print("Using Intel GPU")

    else:
        device = torch.device("cpu")
        print("Using CPU")

    return device
```

Note that if you have a modern NVIDIA GPU (based on the Ampere architecture or newer), the `get_device()` function automatically enables Tensor Cores for faster matrix multiplications when `enable_tensor_cores=True`. This can slightly change floating-point rounding, but it does not noticeably affect results in this book, and the speed advantage on newer cards is worth it. On non-NVIDIA devices, these settings are ignored.

Using the code from listing 2.1, we can obtain the device as follows:

```
device = get_device()
```

While GPUs generally provide substantial speed and performance gains, it can be helpful to run the remaining code in this chapter on the CPU for compatibility and debugging. You can temporarily override the automatic selection by explicitly setting the device:

```
device = torch.device("cpu")
```

After you've finished the chapter and verified that the code works properly on the CPU, you can remove or comment out the manual override and rerun the code. If your system has a GPU, you should then see better performance.

NOTE The code in the remainder of this chapter was executed on a Mac Mini with an Apple M4 CPU. Performance comparisons with the Apple Silicon M4 GPU and the NVIDIA H100 GPU are included at the end of the chapter.

Before we load the model and put it onto the selected `device`, we first need to download the weights for Qwen3 0.6B. These files are required to initialize the pretrained model correctly. In the previous section, we called the `download_qwen3_small()` function with `tokenizer_only=True` to download only the tokenizer files. Here, we'll use the same function but set `tokenizer_only=False` so that the model weights are downloaded as well:

```
download_qwen3_small(kind="base", tokenizer_only=False, out_dir="qwen3")
```

The output is as follows:

```
qwen3-0.6B-base.pth: 100% (1433 MiB / 1433 MiB)
✓ qwen3/tokenizer-base.json already up-to-date
```

(There is a checkmark next to the tokenizer because we already downloaded it in the previous section.)

Now that we've downloaded the model weights, we can instantiate a `Qwen3Model` class and load the pretrained weights via PyTorch's `load_state_dict` method:

```
from reasoning_from_scratch.qwen3 import Qwen3Model, QWEN_CONFIG_06_B

model_path = Path("qwen3") / "qwen3-0.6B-base.pth"
model = Qwen3Model(QWEN_CONFIG_06_B)  #1
model.load_state_dict(torch.load(model_path))  #2
model.to(device)  #3
```

#1 Instantiate a Qwen3 model with random weights as placeholders.
#2 Load the pretrained weights into the model.
#3 Transfer the model to the designated device (e.g., "cuda").

Note that if the device setting is `" cpu "`, the `model.to(device)` operation is skipped because the model already resides in CPU memory by default.

After executing the preceding code, you should see the following output. (If you are not running the code in an interactive environment like a Jupyter notebook, you will have to run `print(model)` to see the output.)

```
Qwen3Model(
  (tok_emb): Embedding(151936, 1024)
  (trf_blocks): ModuleList(
    (0-27): 28 x TransformerBlock(
      (att): GroupedQueryAttention(
        (W_query): Linear(in_features=1024, out_features=2048, bias=False)
        (W_key): Linear(in_features=1024, out_features=1024, bias=False)
        (W_value): Linear(in_features=1024, out_features=1024, bias=False)
        (out_proj): Linear(in_features=2048, out_features=1024, bias=False)
        (q_norm): RMSNorm()
        (k_norm): RMSNorm()
      )
      (ff): FeedForward(
        (fc1): Linear(in_features=1024, out_features=3072, bias=False)
        (fc2): Linear(in_features=1024, out_features=3072, bias=False)
        (fc3): Linear(in_features=3072, out_features=1024, bias=False)
      )
      (norm1): RMSNorm()
      (norm2): RMSNorm()
    )
  )
  (final_norm): RMSNorm()
  (out_head): Linear(in_features=1024, out_features=151936, bias=False)
)
```

This output is a summary of the Qwen3 0.6B base model architecture, as printed by PyTorch. It highlights the model's core components: an embedding layer, a stack of 28 transformer blocks, and a final linear projection head. Each transformer block includes a grouped-query attention mechanism and a multilayer feed-forward network, along with normalization layers throughout.

These components are also illustrated in figure 2.8 for readers familiar with LLM architectures, but a detailed understanding of this architecture is not required for this book. Since we are not modifying the

base model itself, but rather building reasoning methods on top of it, you can safely treat the architecture as a black box for now. If you're interested, you can find more information on these components, such as RMSNorm, in appendix C.

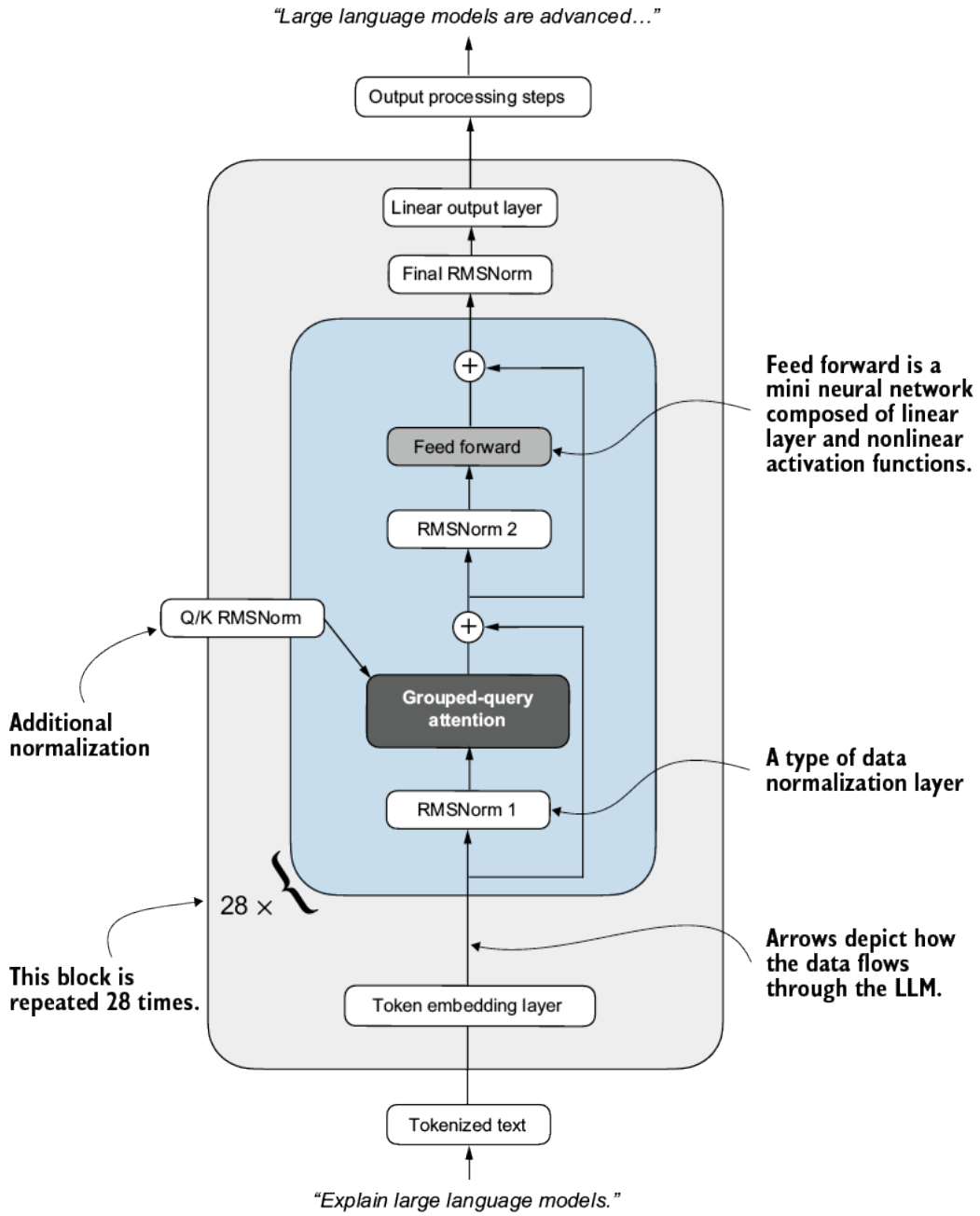


Figure 2.8 Overview of the Qwen3 0.6B model architecture, as instantiated earlier. Input text is tokenized and passed through an embedding layer, followed by 28 repeated transformer blocks. Each block contains grouped-query attention, feed-forward layers, and RMS normalization. The model ends with a final normalization and linear output layer. Arrows show the data flow through the model.

We have now loaded a pretrained model that should be capable of generating coherent text. In the next section, we'll code a text generation function that feeds tokenized data into the model and returns the output in a human-readable format.

2.6 Understanding the sequential LLM text generation process

Now that we've loaded the pretrained LLM, our goal is to write a function that uses the LLM to generate text. This function will form the foundation for reasoning-improving methods that we'll implement later in the book, as shown in figure 2.9.

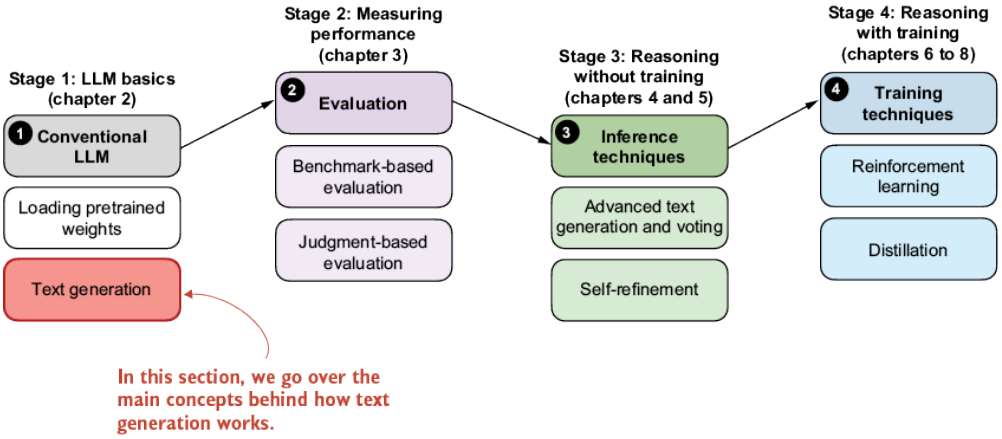


Figure 2.9 An overview of the four key stages in developing a reasoning model in this book. This section explains the main concept behind text generation in LLMs, which will enable us to implement a text generation function that uses the pretrained LLM.

You may already know that text generation in LLMs is a sequential process in which LLMs generate one word (or token) at a time. This is often also called *autoregressive* text generation. Figure 2.10 presents a broad overview, with one generated output token (in the top row) shown at each step as it is fed an input prompt.

CHATBOT INTERFACES

A chat interface simply wraps the underlying next-token prediction process (shown in figure 2.10) in a conversational loop. The chatbot model still predicts one token at a time, but the system feeds the entire dialogue history back into the model so that each new reply feels context aware and coherent from turn to turn. This book focuses on single-turn conversations, where the current answer is independent of previous answers. If you're interested, you can find an implementation of a chat interface with answer history in appendix G.

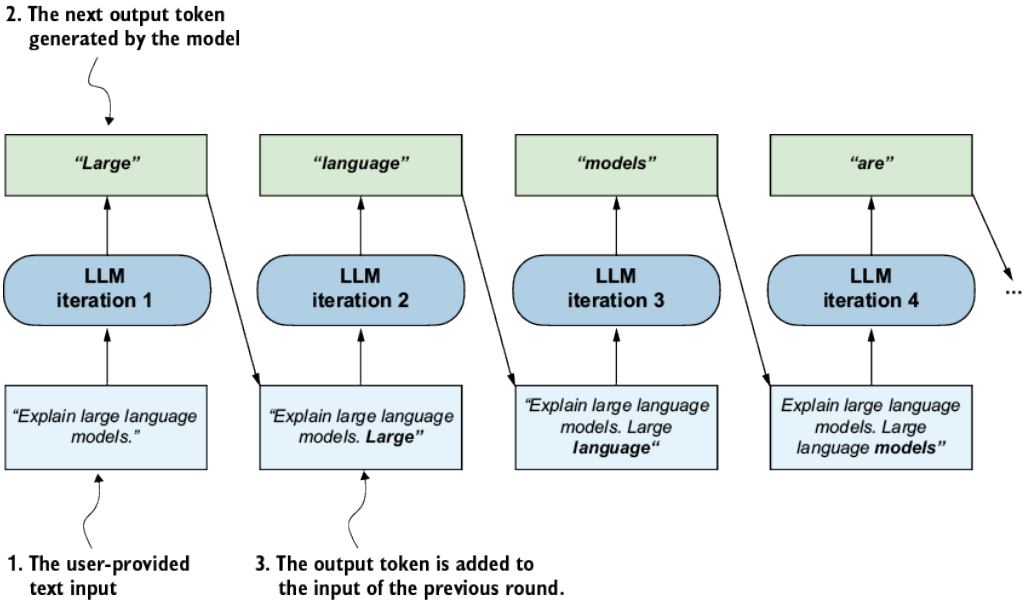


Figure 2.10 Sequential (autoregressive) text generation in LLMs. At each iteration, the model generates the next token based on the input and previously generated tokens, which are cumulatively fed back into the model to produce coherent output.

If we zoom in on one of figure 2.10's iterations, we can see that the LLM processes the full input sequence in one forward pass and produces one prediction for each input position. This means that if we have six input tokens, the model returns six corresponding predictions, as illustrated in figure 2.11. The figure makes the position-by-position structure easy to see, but model outputs are not, in general, just the input prompt shifted by one position. Rather, each position produces a distribution over possible next tokens. For text generation, we only use the prediction at the last position, because that's the one that tells us which new token to append next.

Before we implement a text generation function that uses the concept shown in figure 2.11 in each iteration of the autoregressive text generation process, let's look at a code example that further illustrates figure 2.11. We'll reuse the "Explain large language models." prompt from section 2.4:

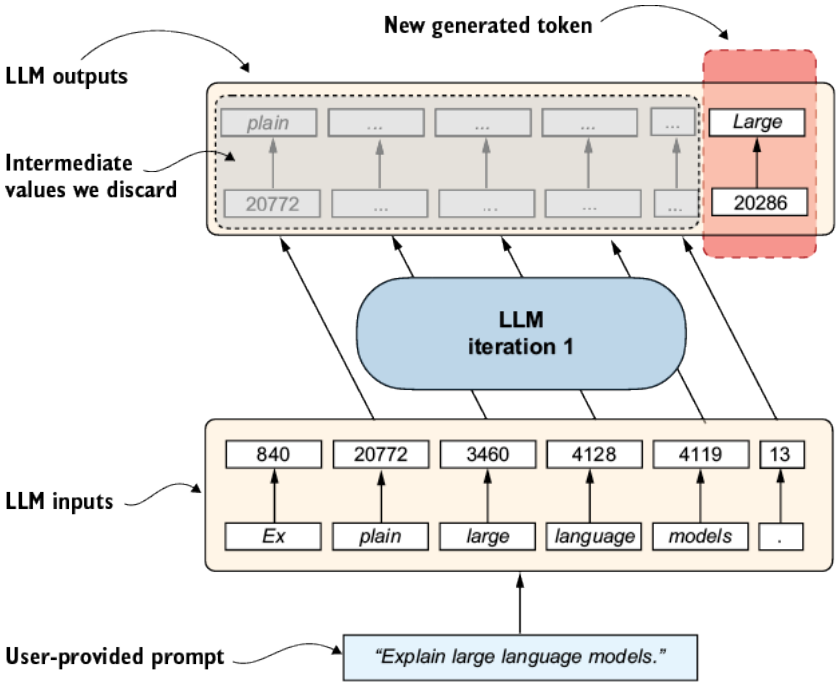


Figure 2.11 The LLM processes the full input sequence and produces one next-token prediction per input position. These predictions are conceptually shifted one step ahead because each position predicts the following token. The illustration is position aligned for intuition, but the outputs are not literally the input prompt shifted by one position. During generation, we keep only the final prediction and use it to continue the input prompt one token at a time.

```
prompt = "Explain large language models."
input_token_ids_list = tokenizer.encode(prompt)
print(f"Number of input tokens: {len(input_token_ids_list)}")

input_tensor = torch.tensor(input_token_ids_list)  #1
input_tensor_fmt = input_tensor.unsqueeze(0)  #2
input_tensor_fmt = input_tensor_fmt.to(device)

with torch.inference_mode():
    output_tensor = model(input_tensor_fmt)  #3
output_tensor_fmt = output_tensor.squeeze(0)  #4
print(f"Formatted Output tensor shape: {output_tensor_fmt.shape}")
```

#1 Convert Python list into PyTorch tensor.

#2 Add an additional dimension.

#3 Generate the output.

#4 Remove the extra dimension.

The preceding code produces the following output:

```
Number of input tokens: 6
Formatted Output tensor shape: torch.Size([6, 151936])
```

As we can see, we feed six input tokens into the model, which returns a 6×151,936-dimensional matrix. The 6 in this matrix corresponds to the six input tokens. The second dimension, 151,936, corresponds to the vocabulary size that the model supports. For instance, each of the six tokens is represented by a vector with 151,936 values. We can think of the values in these vectors as scores for each possible word in the vocabulary, where the highest score corresponds to the most likely word or subword (in the 151,936-entry vocabulary) to be chosen as the generated token.

SQUEEZING AND UNSQUEEZING TENSORS

Tensors are generalized matrices of *n* dimensions. Many PyTorch functions and model components expect tensors with specific dimensions, so being able to add or remove dimensions is important for making data compatible with these operations.

The `.squeeze()` and `.unsqueeze()` operations in PyTorch are used to change the shape of a tensor by removing or adding dimensions of size 1. This is often useful for reshaping a tensor to match what a model expects. For example, a model might expect input tensors with two dimensions (e.g., rows and columns) so it can process batches of inputs (see appendix E). But if the input is just a row vector, we can use `.unsqueeze(0)` to add an extra dimension and make it compatible:

```
example = torch.tensor([1, 2, 3])
print(example)
print(example.unsqueeze(0))
```

This returns the following:

```
tensor([1, 2, 3])
tensor([[1, 2, 3]])
```

Here, `.unsqueeze(0)` adds a new dimension at position 0, turning a 1D tensor into a 2D tensor with shape `(1, 3)`. Conversely, `.squeeze(0)` removes a dimension of size 1 from position 0:

```
example = torch.tensor([[1, 2, 3]])
print(example)
print(example.squeeze(0))
```

This returns the following:

```
tensor([[1, 2, 3]])
tensor([1, 2, 3])
```

This is useful when you want to remove extra dimensions that are not needed.

To get the next generated word, we extract the last row of this 6×151,936-dimensional matrix, find the token ID corresponding to the largest score value in this row, and convert this token ID back into text via the tokenizer, as illustrated in figure 2.12.

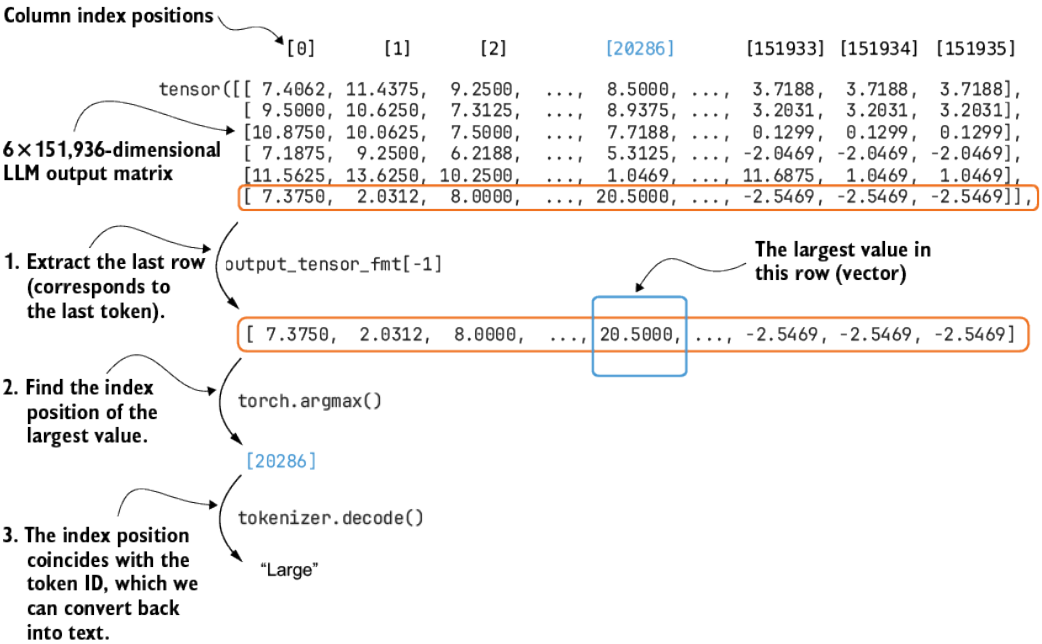


Figure 2.12 A closer look at how the raw scores output by an LLM, in a single text generation iteration, are converted into a token ID and its corresponding text representation.

Let's see how we can convert the LLM output matrix into the generated text token (shown in figure 2.12) in code.

Note that for text generation we run the model in inference mode rather than training mode, so PyTorch does not need to track gradients. As shown in figure 2.11, the earlier positions correspond to predictions for tokens that are already inside the prompt. Only the last position predicts the next unseen token that we want to append, which we can obtain via the `[-1]` index:

```
last_token = output_tensor_fmt[-1]
print(last_token)
```

Using `torch.inference_mode()` in the previous step avoids storing gradient information that we do not need during generation. This reduces memory usage and usually improves speed.

This prints the 151,936 values corresponding to the last token:

```
tensor([ 7.3750,  2.0312,  8.0000,  ..., -2.5469, -2.5469, -2.5469],
       dtype=torch.bfloat16)
```

The `dtype=torch.bfloat16` suffix indicates that these scores are stored in bfloat16, a reduced-precision format commonly used to lower memory usage and improve efficiency in LLM inference. Depending on your hardware and settings, you may instead see a different `dtype`.

Then, we can use the `argmax` function to obtain the position with the largest value score (value) in this tensor:

```
print(torch.argmax(last_token, dim=-1, keepdim=True))
```

The result is

```
tensor([20286])
```

This returned integer value is the position of the largest value in this vector, and it also corresponds to the token ID of the generated token (`last_token`), which we can translate back into text via the tokenizer:

```
print(tokenizer.decode([20286]))
```

This prints the generated token:

```
 Large
```

MAX VS. ARGMAX

It is helpful to briefly recall how `max` and `argmax` work and how they differ, since we will use `torch.argmax()` later when we select the next token when implementing the text generation function. In this chapter, using `argmax` corresponds to greedy decoding, where we always pick the single highest-scoring next token. We will discuss alternatives to this later in the book.

The `torch.max()` function returns the largest value in a tensor, whereas `torch.argmax()` returns the index of that value. For example:

```
example = torch.tensor([-2, 1, 3, 1])
print(torch.max(example))
print(torch.argmax(example))
```

This returns:

```
tensor(3)
tensor(2)
```

The maximum value is 3, and it first appears at index 2.

We can also use `keepdim=True` with `torch.argmax()` to keep the output shape consistent by retaining the reduced dimension:

```
print(torch.argmax(example, keepdim=True))
```

This returns:

```
tensor([2])
```

Here, `keepdim=True` keeps the result as a 1D tensor with the same number of dimensions as the input, which can be helpful for keeping the shape required by the tokenizer and for concatenation later on in our text generation function.

To recap, figure 2.10 illustrated the iterative (autoregressive) text generation process in an LLM. Then, figure 2.11 zooms in on one of the iterations in this process. Figure 2.12 then further zooms in on this one iteration and shows how the score matrix (output by an LLM), gets converted into a token ID (and its corresponding text representation).

While we have seen how to use the LLM to generate a single token, in the next section, we will put these concepts into action and implement a function that applies this concept sequentially to generate coherent output text.

2.7 Coding a minimal text generation function

The previous section explained a single iteration in the basic, sequential text generation process in LLMs. In this section, building on that concept, we will implement a text generation function that uses the pre-trained LLM to generate coherent text following a user prompt, as illustrated in figure 2.13 in the chapter overview.

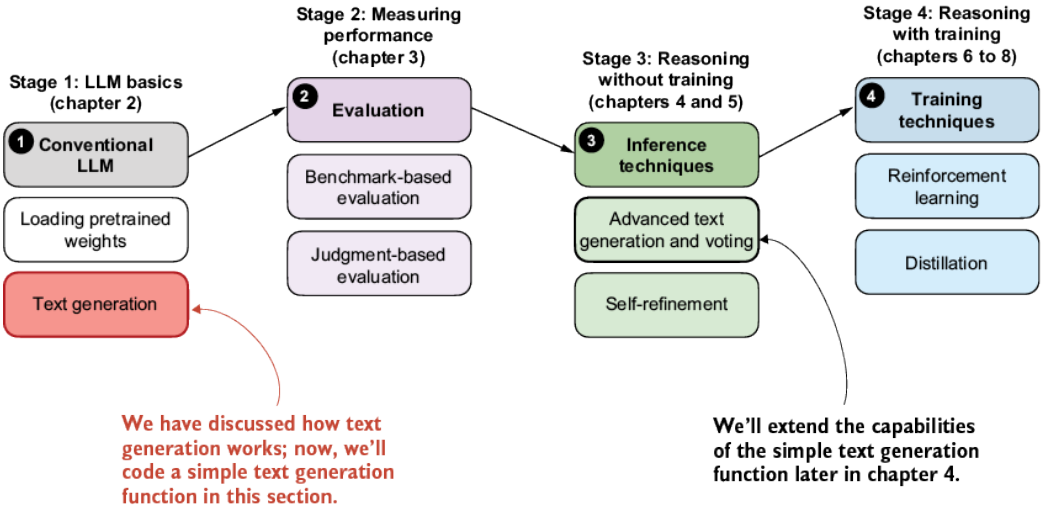


Figure 2.13 An overview of the four key stages in developing a reasoning model in this book. In this section, we implement a text generation function for the pretrained LLM.

This text generation function, mentioned in figure 2.13, works by first converting the input prompt into token IDs that the model can process. The model then predicts the next most likely token, appends it to the sequence, and reprocesses the extended sequence to generate the next token. This iterative process continues until a stopping condition is met, and the generated token IDs are then decoded back into text.

Figure 2.14 shows this process step by step, with both the generated token IDs and their corresponding text at each stage. (This figure is similar to figure 2.10 shown at the beginning of the previous section, except figure 2.14 also shows the generated token IDs below their text representation.)

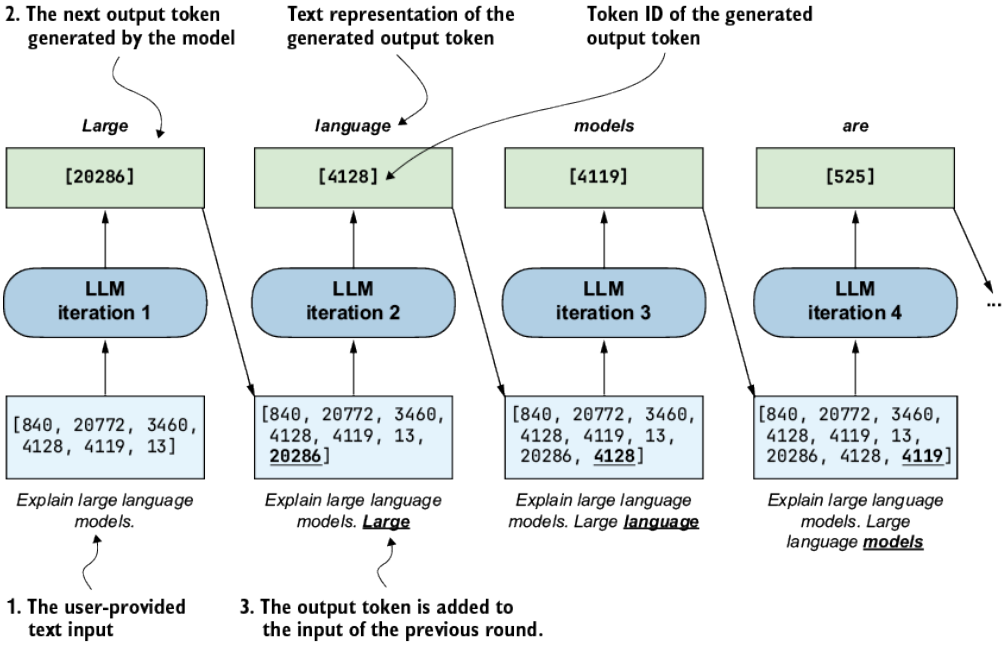


Figure 2.14 An illustration of sequential (autoregressive) text generation in large language models (LLMs), with token IDs shown explicitly. At each iteration, the model generates the next token based on the original input and all previously generated tokens. The predicted token is added to the sequence in both its textual and token ID form.

The `generate_text_basic_stream` function in listing 2.2 implements the sequential text generation process (figure 2.14) using the `argmax` function introduced in the previous section.

Listing 2.2 A basic text generation function

```
@torch.inference_mode()                                          #1
def generate_text_basic_stream(
    model,
    token_ids,
    max_new_tokens,
    eos_token_id=None
):

    model.eval()                                                 #2

    for _ in range(max_new_tokens):
        out = model(token_ids)[:, -1]                            #3
        next_token = torch.argmax(out, dim=-1, keepdim=True)

        if (eos_token_id is not None                             #4
                and torch.all(next_token == eos_token_id)):
            break

        yield next_token                                         #5

        token_ids = torch.cat([token_ids, next_token], dim=1)  #6
```

#1 Disable gradient tracking for speed and memory efficiency.
#2 Switch model to evaluation mode to enable deterministic behavior (best practice).
#3 Get the scores of the last token.
#4 Stop if all sequences in the batch have generated EOS.
#5 Yield each token as soon as it's generated.
#6 Append the newly predicted token to the sequence.

In essence, the `generate_text_basic_stream` function in listing 2.2 applies the `argmax`-based token ID extraction via a `for` loop for a user-specified number of iterations (`max_new_tokens`). It returns the generated token IDs, similar to what's shown in figure 2.14, which we can then convert back into text.

Let's use the function to generate a 100-token response to a simple `" Explain large language models in a single sentence. "` prompt to make sure that the `Qwen3Model` and `generate_text_basic_stream` function work (we get to the reasoning task examples in later chapters).

Please note that the following code will be slow and can take 1–3 minutes to complete, depending on your computer (we will speed it up in later sections):

```
prompt = "Explain large language models in a single sentence."
input_token_ids_tensor = torch.tensor(
    tokenizer.encode(prompt),
    device=device                       #1
    ).unsqueeze(0)

max_new_tokens = 100                    #2



for token in generate_text_basic_stream(
    model=model,
    token_ids=input_token_ids_tensor,
    max_new_tokens=max_new_tokens,
):
    token_id = token.squeeze(0).tolist()     #3
    print(
        tokenizer.decode(token_id),
        end="",
        flush=True  #4
    )
```

#1 Transfer the input token IDs onto the same device (CPU, GPU) where the model is located.
#2 Let the model generate up to 100 new tokens.
#3 Convert output token IDs from PyTorch tensor to Python list.
#4 Deactivates buffering so tokens are printed live

The generated output text is as follows:

```
 Large language models are artificial intelligence systems that can
understand, generate, and process human language, enabling them to
perform a wide range of tasks, from answering questions to writing
articles, and even creating creative content.<|endoftext|>Human language
is a complex and dynamic system that has evolved over millions of
years to enable effective communication and social interaction. It is
composed of a vast array of symbols, including letters, numbers, and
words, which are used to convey meaning and express thoughts and
ideas. The evolution of language has
```

Note that the output was generated on a CPU. Depending on the device (e.g., CPU versus GPU), the exact wording may vary slightly due to differences in floating-point behavior on different hardware.

As we can see based on the output, the model follows the instruction quite well by producing a single, clear sentence in response to the prompt. It continues generating additional, off-topic text after the special token `<|endoftext|>`. This token is used during training to mark the end of a document and separate different samples.

TIP The leading whitespace in `" Large "` (the first output word) is expected because many tokenizers encode words with a preceding space when they follow earlier text. In listing 2.2, tokens are streamed one by one, following the input text, so this leading space appears naturally in the first emitted token. If we want a cleaner output, we can call `.lstrip()` on the first token or the final assembled string.

When using the model for inference (generating text after training), we typically want it to stop as soon as it produces the special token `<|endoftext|>`. This token is represented by the ID 151643, which we can confirm using

```
print(tokenizer.encode("<|endoftext|>"))
```

For convenience, this token ID is also saved via the `tokenizer.eos_token_id` attribute. We can pass this ID to the `generate_text_basic_stream` function to signal when generation should stop:

```
for token in generate_text_basic_stream(
    model=model,
    token_ids=input_token_ids_tensor,
    max_new_tokens=max_new_tokens,
    eos_token_id=tokenizer.eos_token_id  #1
):
    token_id = token.squeeze(0).tolist()
    print(
        tokenizer.decode(token_id),
        end="",
        flush=True
    )
```

#1 Pass end-of-sequence (eos) token ID.

The output looks like this:

```
 Large language models are artificial intelligence systems that can
understand, generate, and process human language, enabling them to
perform a wide range of tasks, from answering questions to writing
articles, and even creating creative content.
```

If we compare this response to the previous response, we can see that the text generation stopped once the end-of-sequence token was encountered.

You may have noticed that generating the response is relatively slow and might take several seconds up to multiple minutes, depending on the hardware. Before we wrap up and learn how to speed up this function substantially, let's implement a simple utility function in the next listing that measures the runtime of the text generation process.

Listing 2.3 Token generation speed and memory usage

```
import warnings

def generate_stats(output_token_ids, tokenizer, start_time,
                   end_time):
    total_time = end_time - start_time
    print(f"\n\nTime: {total_time:.2f} sec")
    print(f"{int(output_token_ids.numel() / total_time)} tokens/sec")

    for name, backend in (("CUDA", getattr(torch, "cuda", None)),
                          ("XPU", getattr(torch, "xpu", None))):
        if backend is not None and backend.is_available():

            device_type = output_token_ids.device.type#1
            if device_type != name.lower():
                warnings.warn(
                    f"{name} is available but tensors are on "
                    f"{device_type}. Memory stats may be 0."
                )

            if hasattr(backend, "synchronize"):#2
                backend.synchronize()

            max_mem_bytes = backend.max_memory_allocated()
            max_mem_gb = max_mem_bytes / (1024 ** 3)
            print(f"Max {name} memory allocated: {max_mem_gb:.2f} GB")

            backend.reset_peak_memory_stats()
```

#1 Check whether we are actually using this backend.

#2 Synchronize if supported (important for async backends).

The `generate_stats` function in listing 2.3 will calculate the total runtime, given a start and end timestamp, the generation speed in terms of tokens per second (tokens/sec), and the GPU memory used. Note

that the GPU memory usage is currently only computed for CUDA-supported GPUs (and some newer Intel GPUs via XPU), as PyTorch lacks similar utility functions for CPUs and Apple Silicon GPUs.

To apply the `generate_stats` function, we obtain a `start_time` and `end_time` stamp immediately before and after running the `generate_text_basic_stream` function via Python's `time` module:

```
import time

start_time = time.time()
generated_ids = []

for token in generate_text_basic_stream(
    model=model,
    token_ids=input_token_ids_tensor,
    max_new_tokens=max_new_tokens,
    eos_token_id=tokenizer.eos_token_id
):
    token_id = token.squeeze(0).tolist()
    print(
        tokenizer.decode(token_id),
        end="",
        flush=True
    )
    next_token_id = token.squeeze(0)
    generated_ids.append(next_token_id)  #1

end_time = time.time()
output_token_ids_tensor = torch.cat(generated_ids, dim=0)
generate_stats(output_token_ids_tensor, tokenizer, start_time, end_time)
```

#1 Collect generated tokens.

The output, on a Mac Mini M4 CPU, is as follows:

```
Time: 7.94 sec
5 tokens/sec
 Large language models are artificial intelligence systems that can
understand, generate, and process human language, enabling them to
perform a wide range of tasks, from answering questions to writing
articles, and even creating creative content.
```

At five tokens per second, the generation speed is relatively slow. In the next section, we will implement a caching technique that speeds up the generation process 5–6 fold.

TEXT GENERATION AND INFERENCE TERMINOLOGY

When reading LLM literature or software documentation, you will often see the term *inference* used where you might expect *text generation*. In a neural network context, inference means something very specific, namely, taking a model whose parameters are already learned and fixed, running a forward pass, and producing a prediction (for example, generating the next token). Nothing is being estimated or learned during this stage. In the forward pass, we are simply applying a function.

This is different from inference in statistics, where the goal is to learn unknown information from data. *Statistical inference* involves estimating parameters, quantifying uncertainty, or testing hypotheses about a population or data-generating process.

Note that, while in large language model contexts the model is computing the next-token distribution, and people sometimes say the model is "estimating" or "inferring" the next token, this is not estimation in the statistical sense. At this stage, the model is a fixed function and its parameters are already learned.

So when we call the `generate_text_basic_stream` function, we are not performing statistical inference. We are performing neural network inference, which is just the forward application of a trained model to produce the next-token distribution and select the next generated token from this distribution.

2.8 Faster inference via KV caching

Now that we have a basic text generation function in place, we can turn our attention to what happens when we actually run it in practice. As you may have noticed, the text generation in the previous section can be a bit slow. That slowdown points us to a key concern: performance during inference.

When running inference with LLMs, which in this context means generating text from a prompt, runtime performance (efficiency) quickly becomes important, especially for long sequences. While the code in this book emphasizes clarity over speed, real-world systems often use engineering tricks to make inference more efficient.

This is useful for the later chapters as well, because evaluation and reasoning workflows often require generating many tokens or many candidate answers, so small speedups here compound quickly.

In the remaining two sections, we will cover two fundamental techniques, KV caching and model compilation, as shown in the overview in figure 2.15, to speed up the text generation.

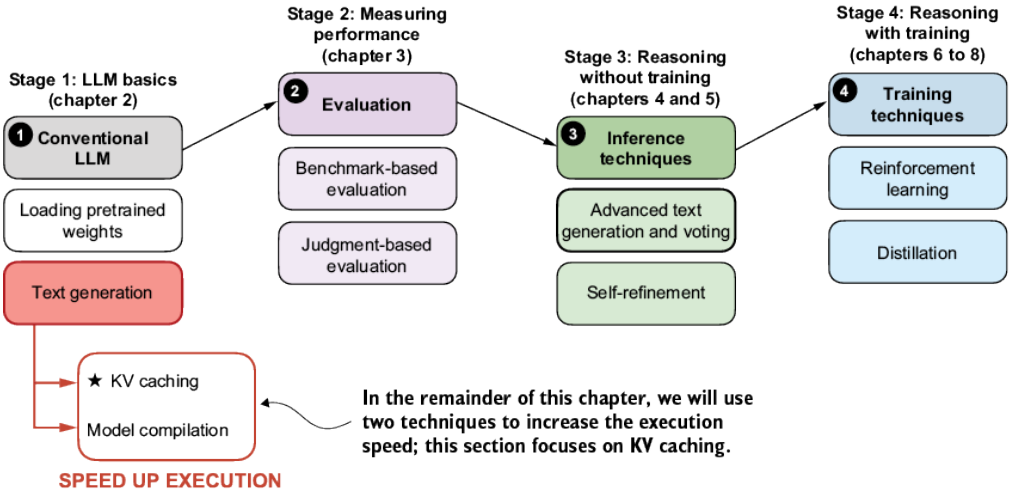


Figure 2.15 An overview of the four key stages in developing a reasoning model in this book. This section builds on pretrained LLM and the basic text generation function we coded earlier and applies KV caching to speed up execution.

As mentioned in figure 2.15, one engineering trick that increases the text generation speed is *KV caching*, where *KV* refers to the keys and values used in the model's attention mechanism. If you are not familiar with these terms, that's okay. The key idea is that we can cache certain intermediate values and reuse them at each step of text generation, as shown in figure 2.16, which helps speed up inference.

The key idea of KV caching, as shown in figure 2.16, is to store intermediate values computed in each iteration in a cache. Previously, each new token generated by the network was concatenated to the entire input sequence and fed back into the model repeatedly (indicated by crossed-out boxes in the diagram). This approach was inefficient because all tokens, except the newly generated one, remained identical in subsequent iterations. By using a KV cache, we avoid redundant computation and instead directly retrieve stored intermediate representations.

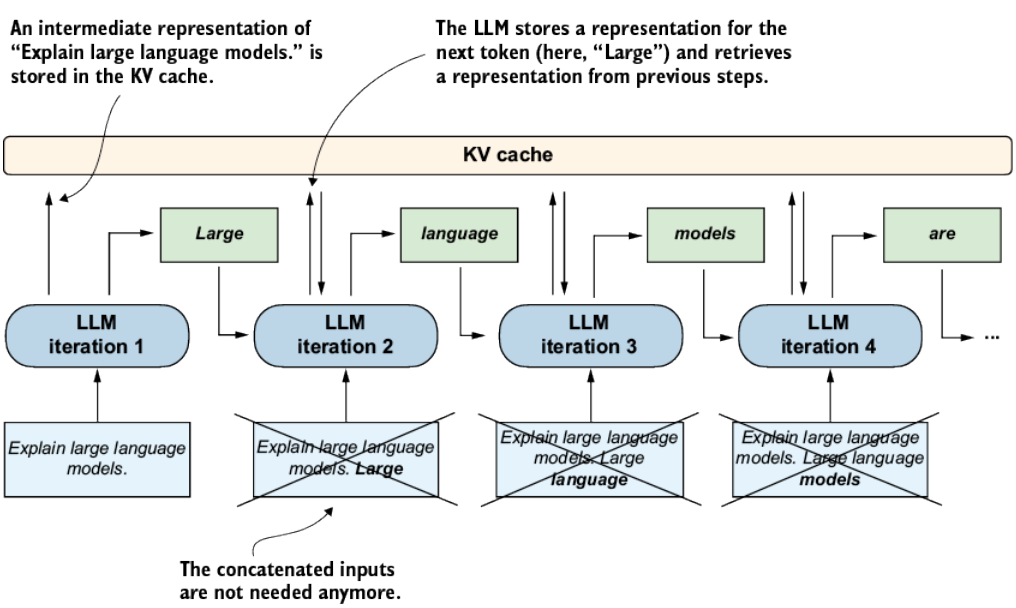


Figure 2.16 Illustration of how a KV cache improves efficiency during autoregressive text generation. Instead of reprocessing the entire input sequence at each step, the KV cache stores intermediate representations so that the LLM can reuse them to generate the next token. This eliminates the need to concatenate the generated token with prior inputs in each subsequent iteration.

In rough terms, generating *n* new tokens without KV caching requires repeatedly recomputing work over an increasingly long sequence, which adds up to about O(*n*2) total decoding work. Here, O(*n*2) means the total work grows roughly with the square of the output length. With KV caching, after the initial pass, each new step processes only the newly added token, reducing this to roughly O(*n*), which means the total work grows approximately in direct proportion to the number of generated tokens.

As mentioned earlier, the nonreasoning focused LLM details like KV caching, which we used to improve the token generation speed, are outside the scope of this book, and they are not required for the topics covered later in this book. However, interested readers can find more information on the mechanics of KV caching in my freely available article: *Understanding and Coding the KV Cache in LLMs from Scratch* (**https://magazine.sebastianraschka.com/p/coding-the-kv-cache-in-llms**).

Following is a modified version of the `generate_text_basic_stream` function that incorporates a KV cache, which is almost identical to the basic text generation function in listing 2.2, except for the KV cache-related change highlighted via the comments:

Listing 2.4 A basic text generation function with KV cache

```
from reasoning_from_scratch.qwen3 import KVCache

@torch.inference_mode()
def generate_text_basic_stream_cache(
    model,
    token_ids,
    max_new_tokens,
    eos_token_id=None
):

    model.eval()
    cache = KVCache(n_layers=model.cfg["n_layers"])  #1
    model.reset_kv_cache()                            #1

    out = model(token_ids, cache=cache)[:, -1]       #2
    for _ in range(max_new_tokens):
        next_token = torch.argmax(out, dim=-1, keepdim=True)

        if (eos_token_id is not None
                and torch.all(next_token == eos_token_id)):
            break

        yield next_token
        out = model(next_token, cache=cache)[:, -1]  #3
```

#1 Initialize the KV cache.
#2 In the first round, the whole input is provided to the model as before.
#3 Consequent iterations only feed the next_token to the input.

The `generate_text_basic_stream_cache` function in listing 2.4 differs only slightly from the `generate_text_basic_stream` function in listing 2.2. The main difference is the introduction of a `KVCache` object.

During the first iteration, the model is given the full input token sequence as before, using `model(token_ids, cache=cache)`. Behind the scenes, the KV cache stores the attention keys and values computed for all these input tokens, so they do not need to be recomputed in later iterations.

In the following iterations, we no longer need to pass the entire sequence. Instead, we only provide the `next_token` to the model using `model(next_token, cache=cache)`. The model then retrieves the necessary context from the previously stored KV cache.

Let's time this function to see whether it provides any performance benefits:

```
start_time = time.time()
generated_ids = []

for token in generate_text_basic_stream_cache(
    model=model,
    token_ids=input_token_ids_tensor,
    max_new_tokens=max_new_tokens,
    eos_token_id=tokenizer.eos_token_id
):
    token_id = token.squeeze(0).tolist()
    print(
        tokenizer.decode(token_id),
        end="",
        flush=True
    )
    next_token_id = token.squeeze(0)
    generated_ids.append(next_token_id)

end_time = time.time()

output_token_ids_tensor = torch.cat(generated_ids, dim=0)
generate_stats(output_token_ids_tensor, tokenizer, start_time, end_time)
```

The output is

```
Time: 1.40 sec
29 tokens/sec

 Large language models are artificial intelligence systems that can
understand, generate, and process human language, enabling them to
perform a wide range of tasks, from answering questions to writing
articles, and even creating creative content.
```

As we can see, this approach is significantly faster, generating 29 tokens per second compared to just 5 tokens per second previously (measured on a Mac Mini M4 CPU).We also see that the generated text is the same as before, which is an important sanity check to ensure that the KV cache is implemented and used correctly.

In the next section, we will learn about another technique we can use to further improve the generation speed, which will come in handy when we evaluate the model in the upcoming chapters. Faster generation allows us to run more evaluations in less time and makes it easier to compare different models or settings efficiently.

2.9 Faster inference via PyTorch model compilation

In the previous section, we covered KV caching as a technique to improve runtime efficiency as shown in the overview in figure 2.17.

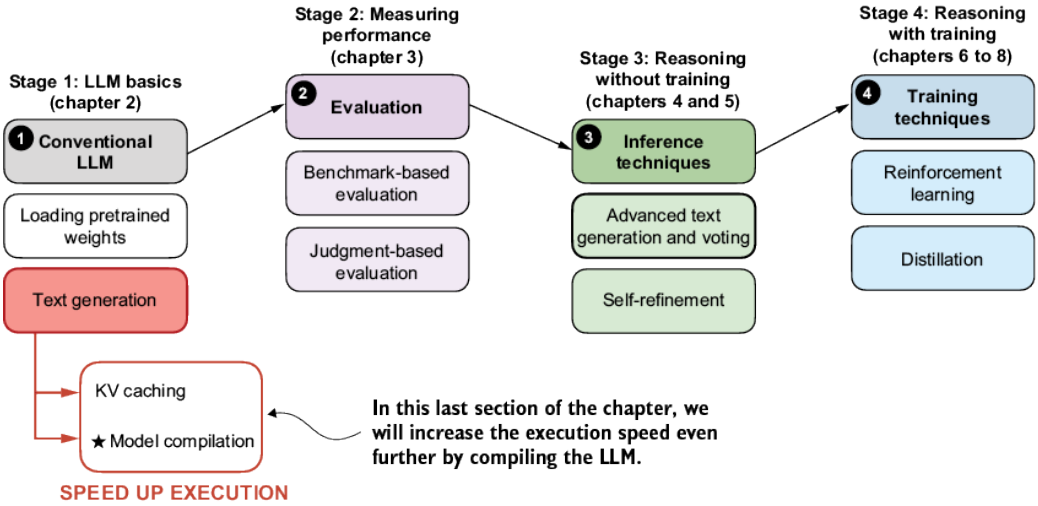


Figure 2.17 An overview of the four key stages in developing a reasoning model in this book. This section builds on pretrained LLM and the basic text generation function we coded earlier, including KV caching, and adds model compilation to speed up the execution speed even further.

As shown in figure 2.17, in this remaining section of this chapter, we will apply another technique that can substantially speed up model inference: model compilation using `torch.compile`. In simple terms, `torch.compile` analyzes the model's internal computation graph and tries to turn groups of operations into more optimized kernels, which reduces Python overhead and other execution inefficiencies. This can improve runtime performance during text generation, especially when we call the same model code repeatedly in a loop.

That will matter later when we run larger evaluations and more elaborate reasoning workflows, where even modest per-step speedups can save substantial total runtime. We can compile the model as follows:

```
major, minor = map(int, torch.__version__.split(".")[:2])
if (major, minor) >= (2, 8):
    # This avoids retriggering model recompilations
    # in PyTorch 2.8 and newer
    # if the model contains code like self.pos = self.pos + 1
    torch._dynamo.config.allow_unspec_int_on_nn_module = True

model_compiled = torch.compile(model)
```

If you are using a Mac with Apple Silicon and encounter an `InductorError`, please make sure to use PyTorch 2.9 or newer.

It is worth noting that the first execution using the compiled model may be slower than usual due to the initial compilation and optimization steps. To better measure the performance improvement, we will repeat the text generation process multiple times.

To begin, we will test this using the noncached version of the generation function. The code in listing 2.5 is similar to what we used before except that we run it three times in a row. The code execution may take a few minutes to finish, depending on the system.

Listing 2.5 Generating text with the compiled model

```
for i in range(3):            #1

    start_time = time.time()

    generated_ids = []

    for token in generate_text_basic_stream(
        model=model_compiled,
        token_ids=input_token_ids_tensor,
        max_new_tokens=max_new_tokens,
        eos_token_id=tokenizer.eos_token_id

    ):
        token_id = token.squeeze(0).tolist()
        print(
            tokenizer.decode(token_id),
            end="",
            flush=True
        )

        next_token_id = token.squeeze(0)
        generated_ids.append(next_token_id)

    end_time = time.time()


    if i == 0:                #2
        print("\n\nWarm-up run")   #2
    else:
        print(f"\n\nTimed run {i}:")

    output_token_ids_tensor = torch.cat(generated_ids, dim=0)
    generate_stats(output_token_ids_tensor, tokenizer, start_time, end_time)

    print(f"\n{30*'-'}\n")
```

#1 We run the token generation three times.

#2 The first run is labeled as "Warm-up run."

The output is as follows:

```
 Large language models are artificial intelligence systems that can
understand, generate, and process human language, enabling them to
perform a wide range of tasks, from answering questions to writing
articles, and even creating creative content.

Warm-up run

Time: 11.68 sec
3 tokens/sec

------------------------------

 Large language models are artificial intelligence systems that can
understand, generate, and process human language, enabling them to
perform a wide range of tasks, from answering questions to writing
articles, and even creating creative content.

Timed run 1:

Time: 6.78 sec
6 tokens/sec

------------------------------

 Large language models are artificial intelligence systems that can
understand, generate, and process human language, enabling them to
perform a wide range of tasks, from answering questions to writing
articles, and even creating creative content.

Timed run 2:

Time: 6.80 sec
6 tokens/sec

------------------------------
```

As we can see from the results, the compiled model achieves a slight improvement in speed, with around 6 tokens per second compared to the previous 5 tokens per second.

Next, let's see how the KV cache version performs in comparison, using the same code as before except for swapping `generate_text_basic_stream` with `generate_text_basic_stream_cache`.

Listing 2.6 Generating text with the compiled model using a KV cache

```
for i in range(3):
    start_time = time.time()
    generated_ids = []

    for token in generate_text_basic_stream_cache(
        model=model_compiled,
        token_ids=input_token_ids_tensor,
        max_new_tokens=max_new_tokens,
        eos_token_id=tokenizer.eos_token_id

    ):
        token_id = token.squeeze(0).tolist()
        print(
            tokenizer.decode(token_id),
            end="",
            flush=True
        )

        next_token_id = token.squeeze(0)
        generated_ids.append(next_token_id)

    end_time = time.time()

    if i == 0:
        print("\n\nWarm-up run")
    else:
        print(f"\n\nTimed run {i}:")

    output_token_ids_tensor = torch.cat(generated_ids, dim=0)
    generate_stats(
        output_token_ids_tensor, tokenizer, start_time, end_time
    )

    print(f"\n{30*'-'}\n")
```

The output is as follows:

```
 Large language models are artificial intelligence systems that can
understand, generate, and process human language, enabling them to
perform a wide range of tasks, from answering questions to writing
articles, and even creating creative content.

Warm-up run

Time: 8.07 sec
5 tokens/sec

------------------------------

 Large language models are artificial intelligence systems that can
understand, generate, and process human language, enabling them to
perform a wide range of tasks, from answering questions to writing
articles, and even creating creative content.

Timed run 1:

Time: 0.60 sec
68 tokens/sec

------------------------------

 Large language models are artificial intelligence systems that can
understand, generate, and process human language, enabling them to
perform a wide range of tasks, from answering questions to writing
articles, and even creating creative content.

Timed run 2:

Time: 0.60 sec
68 tokens/sec

------------------------------
```

As we can see based on the outputs, the model generation speed improved from 29 tokens per second for the uncompiled model with KV cache to 68 tokens per second when the same model is compiled (on a Mac Mini M4 CPU), which is more than a two-fold speed-up.

If you don't see any improvement, try running `torch.compile` with the `" max-autotune "` mode instead of the default settings. For instance, replace

```
model = torch.compile(model)
```

with

```
model = torch.compile(model, mode="max-autotune")
```

EXERCISE 2.2: RERUNNING CODE ON NON-CPU DEVICES

If you have access to a GPU, rerun the code in this chapter on a GPU device and compare the runtimes to the CPU runtimes.

In case you are curious about how the different model configurations compare on an Apple Silicon GPU and a high-end NVIDIA GPU, see table 2.1.

Table 2.1 Token generation speeds and GPU memory usage for different model configurations on different hardware

Mode	Hardware	Tokens/sec	GPU memory
Regular	Mac Mini M4 CPU	5	-
Regular compiled	Mac Mini M4 CPU	6	-
KV cache	Mac Mini M4 CPU	28	-
KV cache compiled	Mac Mini M4 CPU	68	-
Regular	Mac Mini M4 GPU	27	-
Regular compiled	Mac Mini M4 GPU	43	-
KV cache	Mac Mini M4 GPU	41	-
KV cache compiled	Mac Mini M4 GPU	71	-
Regular	NVIDIA H100 GPU	51	1.55 GB
Regular compiled	NVIDIA H100 GPU	164	1.81 GB
KV cache	NVIDIA H100 GPU	48	1.52 GB
KV cache compiled	NVIDIA H100 GPU	141	1.81 GB

As shown in table 2.1, the NVIDIA GPU delivers the best performance, which is expected. The CPU still performs surprisingly well once a KV cache and a compiled model are enabled. However, there are a few important details that explain why the GPU gains are not larger for this particular setup.

First, the model in this chapter is not optimized for GPUs. The implementation aims for a good balance between memory usage and compute speed because memory is the main bottleneck for most readers. A GPU-optimized variant would preallocate the full K and V tensors up to the maximum context length (in this case, 40 thousand tokens). This preallocation avoids repeated concatenation via `torch.cat`, but it also increases memory consumption.

With a large context size, preallocating, for example, 40 thousand entries for both K and V adds a noticeable footprint. Another approach would be to let users specify a fixed context size at construction time, but that introduces extra configuration overhead. The KV cache here grows on demand via `torch.cat`, which is simpler and more memory friendly, although concatenating non-preallocated tensors is a bit slower on GPUs.

I included a GPU-optimized version in the bonus materials, where the KV cache variant is slightly faster than the regular version, but it uses more memory (see **https://github.com/rasbt/reasoning-from-scratch/tree/main/ch02/03_optimized-LLM**).

Second, the model used for benchmarking is small. Larger models benefit much more from KV caching and from GPU-optimized memory layouts. The same holds for batched inference, where GPUs can better saturate their compute units. With a larger model or a GPU-focused implementation, the performance gap in favor of the NVIDIA GPU would be more pronounced.

All examples were run using a single prompt (i.e., a batch size of 1). For readers interested in how performance scales with multiple inputs, batched inference is discussed in appendix E.

Summary

- Using LLMs to generate text involves multiple key steps:
  - Setting up the coding environment to run LLM code and install necessary dependencies
  - Loading a pretrained base LLM (such as Qwen3 0.6B), which will be extended with reasoning capabilities in later chapters
  - Initializing and using a tokenizer, which converts text input into token IDs and decodes output back to human-readable form
- Text generation in LLMs follows a sequential (autoregressive) process, where the model generates one token at a time by predicting the next most likely token.
- The speed and efficiency of text generation can be improved through
  - KV caching, which stores intermediate states to avoid recomputing previously encountered input tokens at each step
  - Model compilation using `torch.compile`, which optimizes runtime performance
- This chapter lays the technical foundation for reasoning capabilities in upcoming chapters by implementing a functional, efficient text generation pipeline using a pretrained base LLM.

Previous chapter
‹ 1 Understanding reasoning models

Next chapter
3 Evaluating reasoning models ›

3 Evaluating reasoning models

This chapter covers

- Extracting final answers reliably from an LLM response
- Verifying answer correctness by comparing an LLM's output to the reference solution using a symbolic math solver
- Running a full evaluation pipeline that loads a pretrained model, generates outputs, and grades them against a dataset

Evaluation is what lets us distinguish between LLMs that merely sound convincing and those that can solve problems correctly. LLM evaluation techniques span a broad range of approaches, from measuring task accuracy to making sure that LLMs adhere to specific safety standards. In this chapter, "evaluate" means quantitatively testing a model on many examples and scoring its outputs against reference answers.

More specifically, we'll focus on implementing a *verification*-based method that checks whether an LLM can solve math problems accurately by comparing its answers against reference solutions using a calculator-like implementation.

This verifier is particularly useful because it not only evaluates performance on math tasks but also introduces the principle of *verifiable rewards*, the foundation of the reinforcement learning approach to reasoning models that we'll implement in chapter 6.

3.1 Building a math verifier

There are four common ways to evaluate trained LLMs in practice: *multiple choice*, *verifiers*, *leaderboards*, and *LLM judges*. These methods are widely used across research papers, technical reports, marketing materials, and model cards (documents that summarize how a model was trained, evaluated, and intended to be used), and results often draw from more than one category.

As figure 3.1 illustrates, these evaluation approaches can be grouped into two broader types: *benchmark-based evaluation* and *judgment-based evaluation*. The former is usually more quantitative, whereas the latter relies more on qualitative judgments. All four evaluation methods are useful in different contexts, but verifiers are especially relevant for reasoning models.

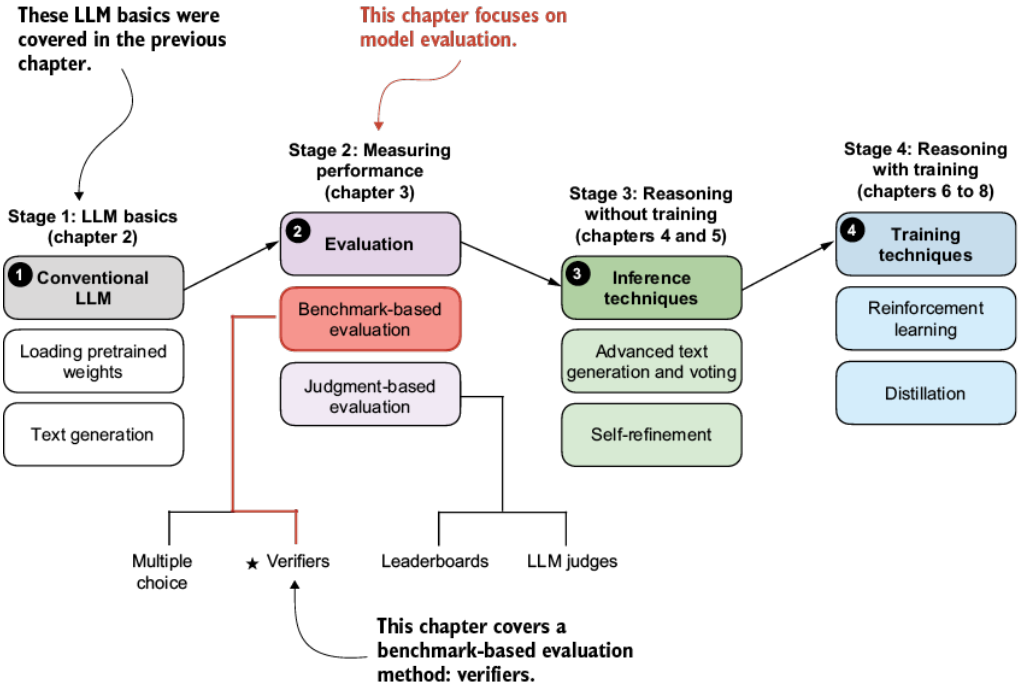


Figure 3.1 A model of the topics covered in this book. This chapter covers evaluation methods (stage 2), with a special focus on implementing verifiers, which we'll reuse as the basis for verifiable rewards in chapter 6.

Math problems provide a natural example: depending on the problem's complexity, they benefit from step-by-step reasoning, yet evaluation is straightforward because the final answer can be checked against the correct answer. In this setting, the verifier provides a simple and reliable way to measure whether a model's reasoning steps lead to the correct outcome.

In this chapter, we'll focus on verifiers as a benchmark-based approach for measuring answer correctness in math problems. Verifiers compare the extracted answer with the reference solution, as shown in figure 3.2, often by relying on external tools such as code interpreters or calculator programs.

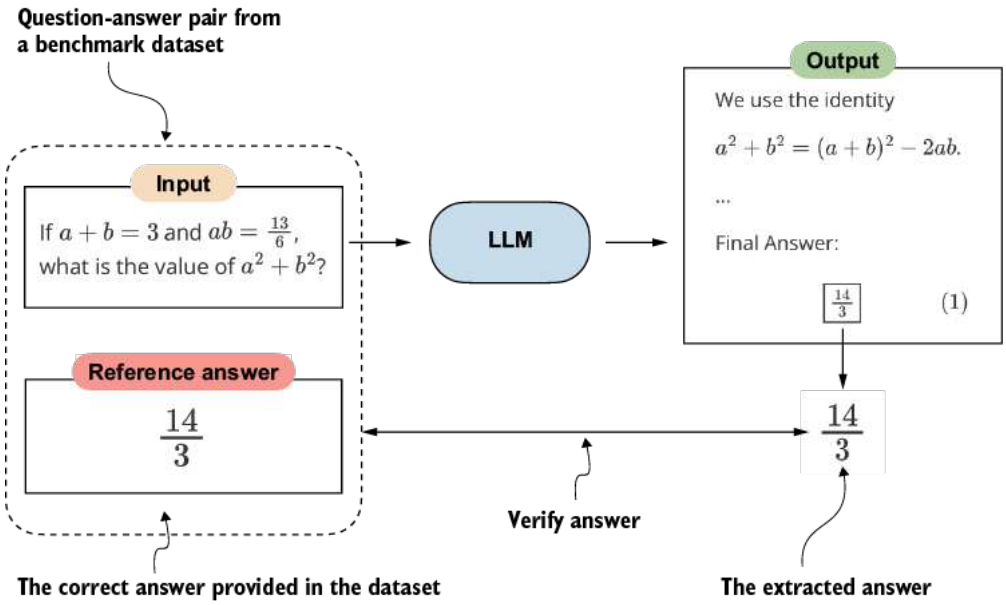


Figure 3.2 Evaluating an LLM with a verification-based method in free-form question answering. The model generates a free-form answer (which may include multiple steps) and a final boxed answer, which is extracted and compared with the correct answer from the dataset.

NOTE Appendix F covers other evaluation methods, such as multiple-choice benchmarks, preference-based leaderboards, and LLM-as-a-judge approaches.

While our immediate focus is evaluation, verifiers will reappear later in this book. They serve not only as a way to measure performance but also as the feedback signal used in reinforcement learning methods to train reasoning models, which we'll explore in chapter 6.

The downside is that verifier methods can be applied only to domains that can be easily (and, ideally, deterministically) verified, such as math and code. Also, this approach can introduce additional complexity

and dependencies, and it may shift part of the evaluation burden from the model itself to the external tool.

Because math problem solving can be generated in unlimited variations programmatically and because it benefits from step-by-step reasoning, this task has become a cornerstone of evaluating and developing reasoning models. In this chapter, we'll build a math verifier step by step, following the eight steps shown in figure 3.3.

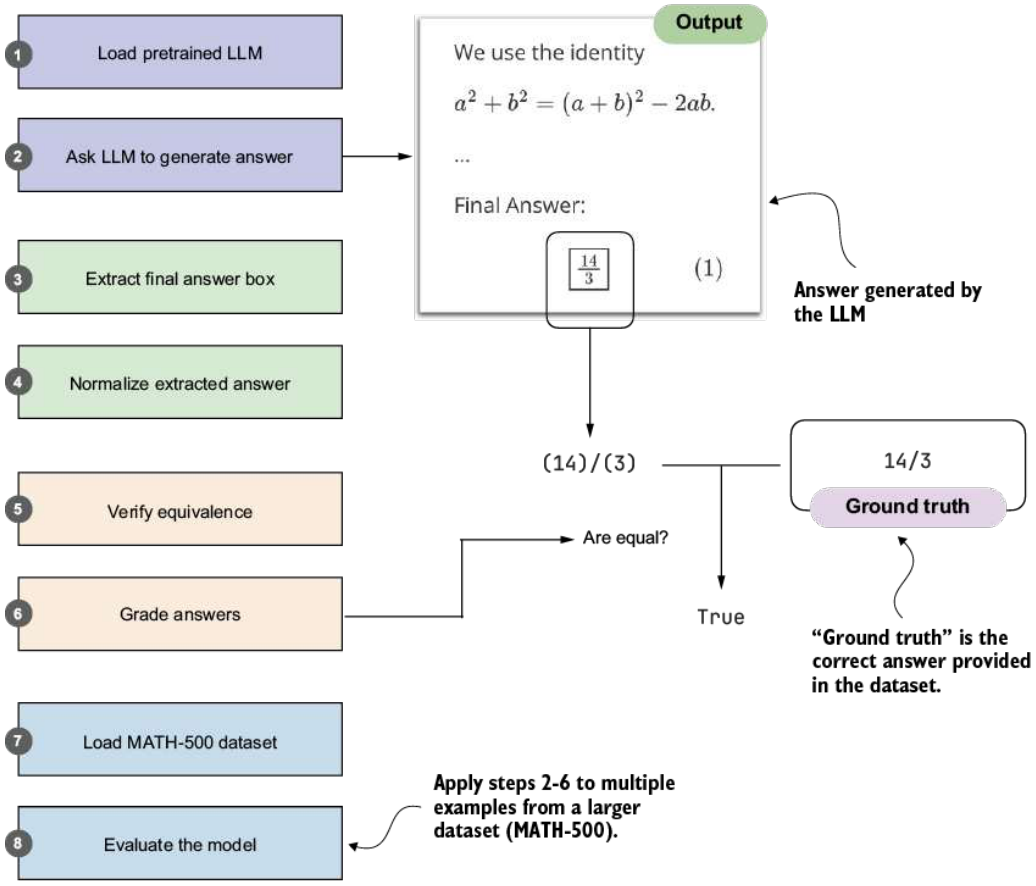


Figure 3.3 A step-by-step workflow for building and applying a math verifier. Starting with a pretrained LLM, we'll generate answers, extract and normalize them, and then compare them with the ground-truth solutions. The verified answers are then graded, and the process is repeated across a dataset (MATH-500) to evaluate overall model performance.

The next section will go through steps 1 and 2 in figure 3.3: loading a pretrained LLM and setting it up to generate answers.

3.2 Loading a pretrained model to generate text

In this section, we begin implementing the verifier by loading the pretrained LLM introduced in the previous chapter and configuring it to generate answers. This will lay the foundation for the later steps in figure 3.3, where we'll extract, normalize, and verify those answers. It will also set up a reusable model-loading path for later chapters, where we'll compare different reasoning methods and need a consistent way to measure whether they actually improve the model.

We will use the same pretrained base model that we used in chapter 2. For convenience and reuse in future chapters, we'll wrap the model-loading logic in a `load_model_and_tokenizer` function call, as shown in the following listing.

Listing 3.1 Loading a pretrained model

```
from pathlib import Path
import torch

from reasoning_from_scratch.ch02 import (
    get_device
)
from reasoning_from_scratch.qwen3 import (
```

```
    download_qwen3_small,
    Qwen3Tokenizer,
    Qwen3Model,
    QWEN_CONFIG_06_B
)


def load_model_and_tokenizer(
    which_model, device, use_compile, local_dir="qwen3"
):
    if which_model == "base":

        download_qwen3_small(
            kind="base", tokenizer_only=False, out_dir=local_dir
        )

        tokenizer_path = Path(local_dir) / "tokenizer-base.json"
        model_path = Path(local_dir) / "qwen3-0.6B-base.pth"
        tokenizer = Qwen3Tokenizer(tokenizer_file_path=tokenizer_path)

    elif which_model == "reasoning":

        download_qwen3_small(
            kind="reasoning", tokenizer_only=False, out_dir=local_dir
        )

        tokenizer_path = Path(local_dir) / "tokenizer-reasoning.json"
        model_path = Path(local_dir) / "qwen3-0.6B-reasoning.pth"
        tokenizer = Qwen3Tokenizer(
            tokenizer_file_path=tokenizer_path,
            apply_chat_template=True,
            add_generation_prompt=True,
            add_thinking=True,
        )

    else:
        raise ValueError(f"Invalid choice: which_model={which_model}")

    model = Qwen3Model(QWEN_CONFIG_06_B)
    model.load_state_dict(torch.load(model_path))

    model.to(device)

    if use_compile:  #1
        torch._dynamo.config.allow_unspec_int_on_nn_module = True
        model = torch.compile(model)

    return model, tokenizer


WHICH_MODEL = "base"   #2
device = get_device()
# device = torch.device("cpu")  #3

model, tokenizer = load_model_and_tokenizer(
    which_model=WHICH_MODEL,
    device=device,
    use_compile=False
)
```

#1 Optionally set to true to enable model compilation.

#2 Uses the base model, similar to chapter 2, by default

#3 Uncomment this line if you have compatibility issues with your device.

By default, listing 3.1 loads the base model, just as in chapter 2.

An optional variant is the official Qwen3 reasoning model, which the Qwen3 team trained on top of the base model using reasoning-specific methods. This is not the custom reasoning model that we'll build lat-

er in the book. Instead, we'll use it here as a reference point. You can load it by setting `WHICH_MODEL = " reasoning "` in listing 3.1, so that you can later compare its evaluation results with those of the base model.

Now that we have loaded the model, we can use the text generation function from chapter 2 to generate text, as shown in the following listing.

Listing 3.2 Generating model outputs

```
from reasoning_from_scratch.ch02 import (
    generate_text_basic_stream_cache
)

prompt = (  #1
    r"If $a+b=3$ and $ab=\tfrac{13}{6}$, "
    r"what is the value of $a^2+b^2$?"
)

input_token_ids_tensor = torch.tensor(  #2
    tokenizer.encode(prompt),
    device=device
).unsqueeze(0)  #3

all_token_ids = []

for token in generate_text_basic_stream_cache(  #4
    model=model,
    token_ids=input_token_ids_tensor,
    max_new_tokens=2048,
    eos_token_id=tokenizer.eos_token_id
):
    token_id = token.squeeze(0)  #5
    decoded_id = tokenizer.decode(token_id.tolist())
    print(  #6
        decoded_id,
        end="",
        flush=True
    )
    all_token_ids.append(token_id)

all_tokens = tokenizer.decode(all_token_ids)  #7
```

#1 Define the math problem as a string prompt.
#2 Convert the prompt into token IDs that the model can process.
#3 Adds batch dimension
#4 Generate output tokens from the model, one at a time.
#5 Removes batch dimension
#6 Prints token as it is generated
#7 Decode the full generated sequence into text.

In this listing, we start by encoding a simple math problem into token IDs that the model can process. The model then generates tokens one by one in a streaming fashion. We print them immediately as they appear so we can read the output while it's being generated. At the same time, we collect the generated tokens into a list so that we can later decode them into the final answer string. This pattern of both streaming and collecting tokens is handy because it lets us monitor the generation live while still storing the full answer text (in `all_tokens`) so we can process it later.

The response generated by the code in listing 3.2 is as follows:

```
To find the value of \( a^2 + b^2 \) given that \( a + b = 3 \)
and \( ab = \frac{13}{6} \), we can use the following algebraic identity:

\[
a^2 + b^2 = (a + b)^2 - 2ab
\]

**Step 1:** Substitute the given values into the equation.

\[
a^2 + b^2 = (3)^2 - 2 \left( \frac{13}{6} \right)
\]

[...]  #1

**Final Answer:**

\[
\boxed{\dfrac{14}{3}}
\]
```

#1 Shortened for brevity

As you can see from this response, even though we're using a base model, it provides a reasoning-model-like explanation. This is likely because the Qwen3 team included chain-of-thought data during pretraining, as stated in their technical report. But even though the model shows some reasoning-model-like behavior, adding additional reasoning methods can further improve these capabilities. (Note that the generated response may differ depending on whether you executed the code on a CPU, CUDA, or MPS device.)

If you're unfamiliar with the LaTeX syntax commonly used for mathematics, the preceding response may be hard to decipher. In that case, you can use IPython's `Latex` class to render it:

```
from IPython.display import Latex, display
display(Latex(all_tokens))
```

The result of running the preceding code in a code notebook is shown in figure 3.4.

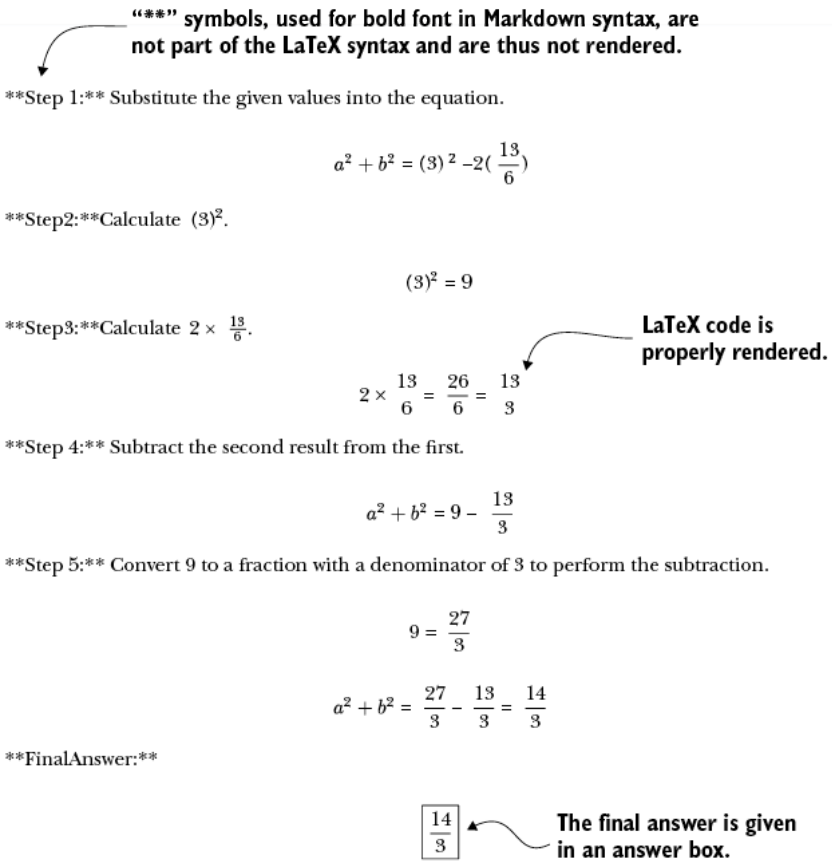


Figure 3.4 Rendered response with step-by-step calculations and the final boxed answer

The final answer shown in figure 3.4 is indeed the correct answer to this problem.

3.3 Implementing a wrapper for easier text generation

We have loaded the pretrained LLM and set up the text generation functionality (as illustrated in figure 3.5). Now, for convenience in later sections, we'll create a wrapper (see listing 3.3) for the text generation function so that we only have to pass in the model, tokenizer, and prompt, along with some additional settings, instead of repeating the tokenization and input preparation steps each time.

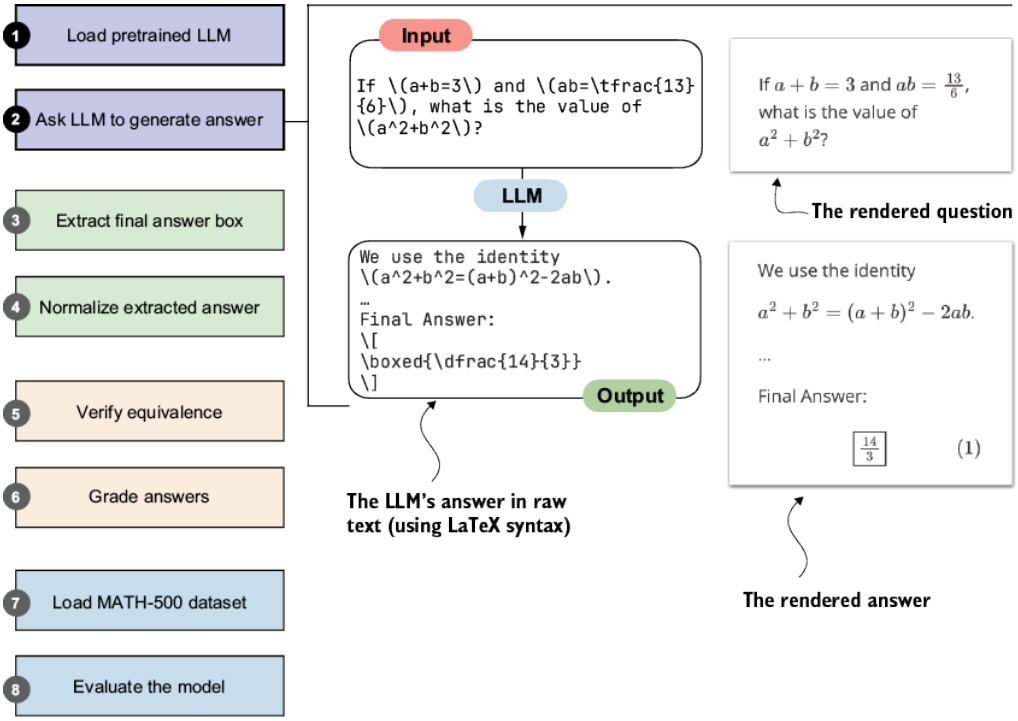


Figure 3.5 In this section, we'll wrap steps 1 and 2 of the verifier workflow into a reusable text-generation helper. Instead of repeating tokenization and input preparation each time, the wrapper handles prompting and streamed generation and returns the model's raw LaTeX output.

Listing 3.3 A wrapper for streamed text generation

```
def generate_text_stream_concat(
    model, tokenizer, prompt, device, max_new_tokens,
    verbose=False,
):
    input_ids = torch.tensor(                    #1
        tokenizer.encode(prompt), device=device
        ).unsqueeze(0)

    generated_ids = []
    for token in generate_text_basic_stream_cache(  #2
        model=model,
        token_ids=input_ids,
        max_new_tokens=max_new_tokens,
        eos_token_id=tokenizer.eos_token_id,
    ):
        next_token_id = token.squeeze(0)
        generated_ids.append(next_token_id.item())


        if verbose: #3
            print(
                tokenizer.decode(next_token_id.tolist()),
                end="",
                flush=True
            )

    return tokenizer.decode(generated_ids)  #4
```

#1 Encode prompt text into token IDs and place on device.
#2 Stream tokens one by one using cached generation.
#3 Optionally print tokens as they are generated.
#4 Decode all generated IDs into final text string.

This wrapper function handles the full cycle of text generation: it tokenizes the input prompt, streams new tokens from the model, and then decodes the results into a final string.

As mentioned before, the optional verbose flag allows us to see tokens as they are generated in real time. The function can be used as follows:

```
generated_text = generate_text_stream_concat(
    model, tokenizer, prompt, device,
    max_new_tokens=2048,
    verbose=True  #1
)
```

#1 Using False will suppress the live token-by-token printing.

This prints the exact same response you saw in section 3.2:

```
[...]  #1

**Final Answer:**

\[
\boxed{\dfrac{14}{3}}
\]
```

#1 Shortened for brevity

3.4 Extracting the final answer box

Now that we have the model loaded and ready, we can get to the interesting part: evaluating the model.

It is worth recalling that this book takes a from-scratch approach, which naturally includes some de-

tailed, sometimes tedious, steps. This is intentional. We want to build the evaluation pipeline ourselves so we can better understand how it works, rather than just calling predefined wrapper functions and building blocks.

In the previous section, you saw that the model returned the final answer in an answer box (written as `r " \boxed{\dfrac{14}{3}} "` in raw text). We did not enforce that format; the model likely produced it because boxed answers are a common convention in math benchmarks and training data. This behavior is not necessarily proof of overfitting to the evaluation tasks we'll run later, but it reflects that pretrained models often encounter many problem formats online and learn to reproduce those stylistic conventions. As a general rule, it's fair to assume that any information available on the internet when a model was trained was part of the training data.

Although it was not necessary here, when we later evaluate the model on the MATH-500 dataset, we will add a specific prompt instructing the model to return answers in this boxed form. (MATH-500 is a curated collection of 500 problems widely used as a benchmark dataset for reasoning models.) This step is straightforward, and it will reappear whenever we need to turn a model response into something that can be scored automatically at scale. This common convention makes evaluation more consistent across models and simplifies data extraction.

We will now write code that performs this extraction of the boxed answer content. This section implements step 3 in figure 3.6. (The next section will implement the normalization method for step 4.)

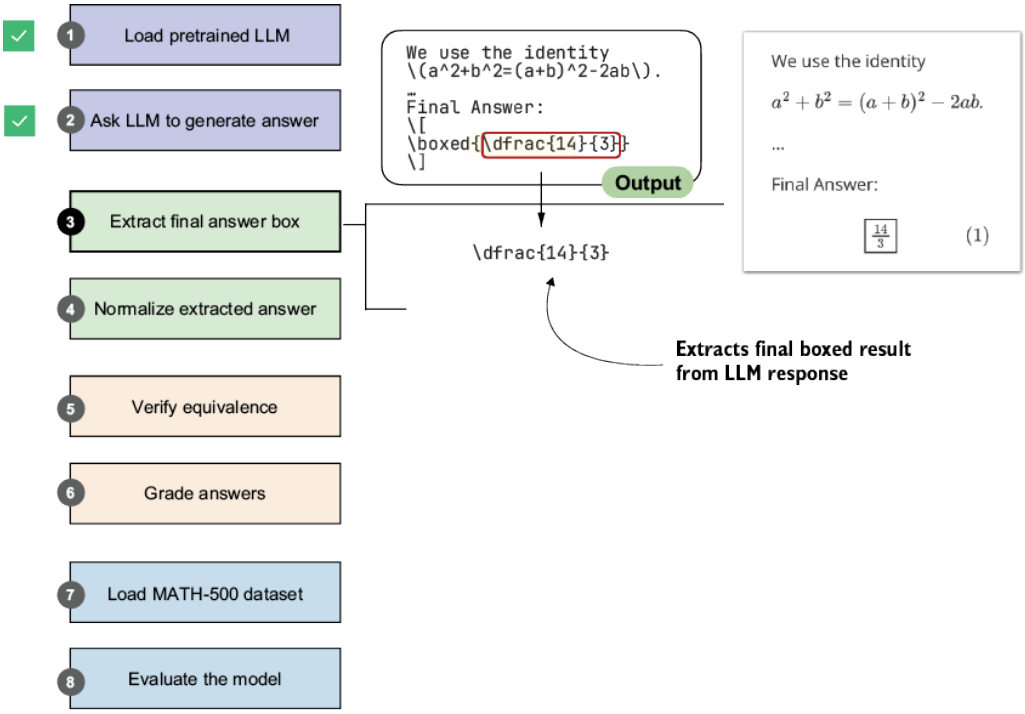


Figure 3.6 An illustration of how the boxed result is extracted from the LLM output

Because your model may produce slightly different responses than mine, depending on your hardware, the following code uses a hardcoded placeholder answer rather than generating a response with the model (but we'll pretend that this answer was generated by the model). In later sections, we'll revisit the model and have it generate answers for the tasks in the MATH-500 dataset:

```
model_answer = (
r"""... some explanation...
**Final Answer:**

\[                      #1
\boxed{\dfrac{14}{3}}   #1
\]                      #1
""")
```

#1 The answer box we want to extract

NOTE We are using a raw string (`r """ ... """` instead of a regular string `""" ... """`). Raw strings make it easier to handle backslash (`\`) characters, which would otherwise be treated as escape se-

quences and require each backslash to be doubled.

Next, let's define a function to extract the answer box from `model_answer`.

Listing 3.4 Extracting answer boxes

```
def get_last_boxed(text):
    boxed_start_idx = text.rfind(r"\boxed")  #1
    if boxed_start_idx == -1:
        return None

    current_idx = boxed_start_idx + len(r"\boxed")  #2

    while current_idx < len(text) and text[current_idx].isspace():#3
        current_idx += 1


    if current_idx >= len(text) or text[current_idx] != "{":#4
        return None

    current_idx += 1
    brace_depth = 1
    content_start_idx = current_idx

    while current_idx < len(text) and brace_depth > 0:#5
        char = text[current_idx]
        if char == "{":
            brace_depth += 1
        elif char == "}":
            brace_depth -= 1
        current_idx += 1

    if brace_depth != 0:  #6
        return None

    return text[content_start_idx:current_idx-1]  #7
```

#1 Find the last occurrence of "\boxed".
#2 Get the position after "\boxed".
#3 Skip any whitespace after "\boxed".
#4 Expect an opening brace "{".
#5 Parse the braces with nesting.
#6 Account for unbalanced braces.
#7 Extract content inside the outermost braces.

The `get_last_boxed` helper utility function in listing 3.4 extracts the content of the last `\boxed{...}` expression from a model's output. More specifically, it scans for the final `\boxed`, skips over whitespace, checks for braces, and handles any nesting so that it captures the intended answer string.

While it may look a bit tedious, having this parser in place will pay off when we run evaluations on datasets like MATH-500, where extracting the correct final answer is the first step toward measuring a model's reasoning ability.

Now, let's test this function on the model answer:

```
extracted_answer = get_last_boxed(model_answer)
print(extracted_answer)
```

The output of this function call is `" \dfrac{14}{3} "`, which is the boxed answer we wanted to extract.

RENDERING MATH FORMULAS

We can render math formulas via the `Latex` class introduced earlier. Alternatively, for standalone math formulas without accompanying answer text, we can use the simpler `Math` class:

```
from IPython.display import Math
display(Math(r"\dfrac{14}{3}"))
```

This renders the fraction as

$$\frac{14}{3}$$

The preceding `get_last_boxed` function correctly extracted the text, but we can make answer extraction a bit more robust with the `extract_final_candidate` function (shown in the following listing), which accounts for cases where a final answer box is missing or incomplete.

Listing 3.5 Extracting the final answer candidate

```
import re

RE_NUMBER = re.compile(  #1
    r"-?(?:\d+/\d+|\d+(?:\.\d+)?(?:[eE][+-]?\d+)?)"
)

def extract_final_candidate(text, fallback="number_then_full"):

    result = ""  #2

    if text:  #3
        boxed = get_last_boxed(text.strip())
        if boxed:
            result = boxed.strip().strip("$ ")

        elif fallback in ("number_then_full", "number_only"):#4
            m = RE_NUMBER.findall(text)
            if m:
                result = m[-1]  #5
            elif fallback == "number_then_full":

                result = text  #6
    return result
```

#1 Regular expression for extracting numeric values from the text
#2 Default return value if nothing matches
#3 Prefer the last boxed expression if present.
#4 If there is no boxed expression, try the fallback.
#5 Use the last number.
#6 Else return the full text if no number is found.

This `extract_final_candidate` function provides `fallback` settings in case no boxed answer is found, as follows:

- `" number_then_full "` (the default): Pick the last simple number; otherwise, pick the whole text.
- `" number_only "`: Pick the last simple number; otherwise, return an empty string `""`.
- `" none "`: Extract only the boxed content; otherwise, return an empty string `""`.

For the fallback setting, the code in listing 3.5 uses *regular expressions* (regex for short) via Python's `re` library. Regexes are a way to search for patterns in text. In our case, the regex pattern is designed to recognize numbers, including fractions, decimals, and scientific notation. While the regex syntax looks intimidating, you don't need to worry about how it works here. What matters is that this gives us a reliable tool for extracting the last numeric candidate from the model's output when no boxed answer is available.

Let's try it on our model answer:

```
print(extract_final_candidate(model_answer))
```

This correctly returns `" \dfrac{14}{3} "`.

Now let's try some additional examples. First, here's another boxed candidate:

```
print(extract_final_candidate(r"\boxed{ 14/3. }"))
```

This correctly returns `" 14/3. "`, stripping the extra whitespace but not the punctuation. The punctuation character will be handled correctly by the equality check we implement later.

Next, let's try a candidate without a box, which should trigger the fallback setting:

```
print(extract_final_candidate("abc < > 14/3 abc"))
```

Thanks to the default fallback setting, it will find the last number in the answer and correctly return `" 14/3 "`.

In this section, we defined utility functions to extract the LLM's answer from its response text. This brings us one step closer to our overall goal of verifying whether that answer is correct. In the next section, we'll normalize the response into a more general, canonical form before implementing the checking functionality.

WHY NOT USE AN LLM TO EXTRACT THE ANSWER?

We could use an LLM to extract the boxed answer itself, but that would introduce unnecessary complexity and potential errors. Extraction is a simple, mechanical task, assuming that the LLM outputs the answer in a specific format: we just need to locate the last boxed expression or, if that is missing, fall back to a number or the raw text.

Regular expressions may look complicated at first, but we now have a small, reusable utility function that is cheap to execute and handles the extraction deterministically and reproducibly, without depending on the variability of another model's output.

3.5 Normalizing the extracted answer

Previously, we extracted the boxed answer `" \dfrac{14}{3} "` from the model's response. Models may print the same value in many ways, such as `" \frac{14}{3} "`, `" 14/3 "`, `" $14/3$ "`, or `" (14)/(3) "`. To implement and use a robust checking system to determine whether the answer is correct, we first need a consistent method of comparing results. In this section, we'll implement a normalization (or canonicalization) pass (step 4 in figure 3.7) that strips formatting and standardizes the answer.

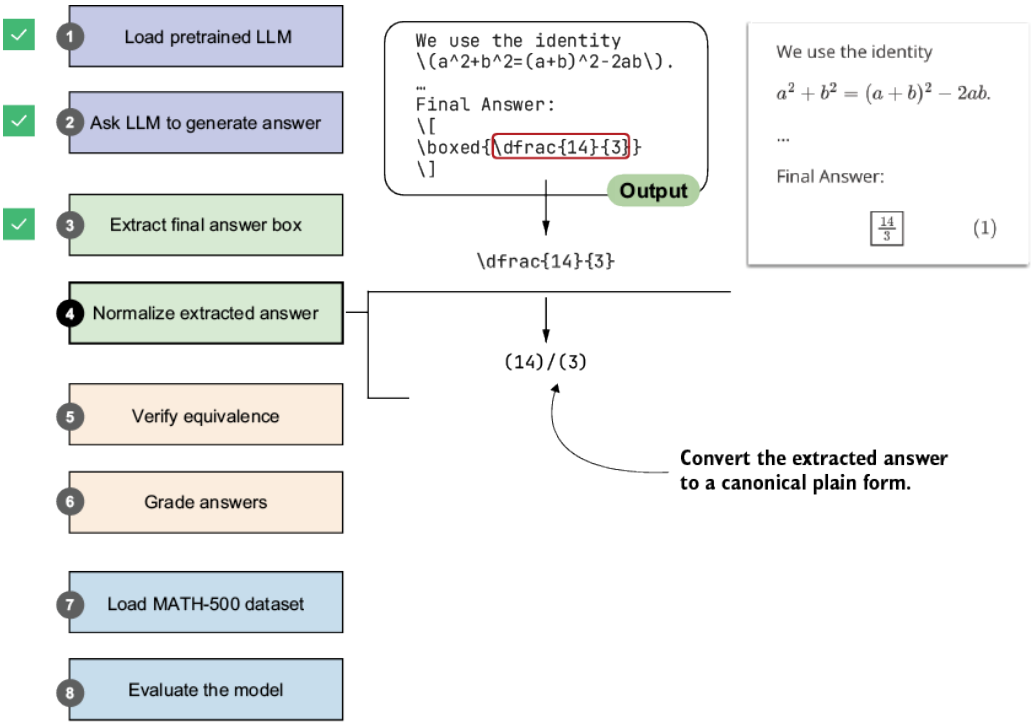


Figure 3.7 An illustration of how the boxed result from the LLM output is extracted and converted into a canonical plain form. This normalized answer will later be verified against the correct answer.

The normalization step shown in figure 3.7 is implemented in the following `normalize_text` function.

Listing 3.6 Normalizing extracted answers

```
LATEX_FIXES = [  #1
    (r"\\left\s*", ""),
    (r"\\right\s*", ""),
    (r"\\,|\\!|\\;|\\:", ""),
    (r"\\cdot", "*"),
    (r"\u00B7|\u00D7", "*"),
    (r"\\\^\\circ", ""),
    (r"\\dfrac", r"\\frac"),
    (r"\\tfrac", r"\\frac"),
    (r"°", ""),
]

RE_SPECIAL = re.compile(r"<\|[^>]+?\|>")  #2
SUPERSCRIPT_MAP = {
    "⁰": "0", "¹": "1", "²": "2", "³": "3", "⁴": "4",  #3
    "⁵": "5", "⁶": "6", "⁷": "7", "⁸": "8", "⁹": "9",     #3
    "⁺": "+", "⁻": "−", "⁽": "(", "⁾": ")",               #3
}

def normalize_text(text):
    if not text:
        return ""
    text = RE_SPECIAL.sub("", text).strip()


    match = re.match(r"^[A-Za-z]\s*[.:]\s*(.+)$", text)#4
    if match: #4
        text = match.group(1) #4
 #4
 #4
    text = re.sub(r"\^\s*\{\s*\\circ\s*\}", "", text)   #4
    text = re.sub(r"\^\s*\\circ", "", text)            #5
    text = text.replace("°", "")                        #5

    match = re.match(r"^\\text\{(?P<x>.+?)\}$", text)  #6
    if match:
        text = match.group("x")
```

```
    text = re.sub(r"\\\(|\\\)|\\\[|\\\]", "", text)  #7


    for pat, rep in LATEX_FIXES:  #8
        text = re.sub(pat, rep, text)


    def convert_superscripts(s, base=None):
        converted = "".join(
            SUPERSCRIPT_MAP[ch] if ch in SUPERSCRIPT_MAP else ch
            for ch in s
        )
        if base is None:
            return converted
        return f"{base}**{converted}"

    text = re.sub(
        r"([0-9A-Za-z\)\]\}])([⁰¹²³⁴⁵⁶⁷⁸⁹⁺⁻]+)",
        lambda m: convert_superscripts(m.group(2), base=m.group(1)),
        text,
    )
    text = convert_superscripts(text)


    text = text.replace("\\%", "%").replace("$", "").replace("%", "")#9
    text = re.sub(
        r"\\sqrt\s*\{([^}]*)\}",
        lambda match: f"sqrt({match.group(1)})",
        text,
    )
    text = re.sub(
        r"\\sqrt\s+([^\\\s{}]+)",
        lambda match: f"sqrt({match.group(1)})",
        text,
    )


    text = re.sub(#10
        r"\\frac\s*\{([^{}]+)\}\s*\{([^{}]+)\}",
        lambda match: f"({match.group(1)})/({match.group(2)})",
        text,
    )
    text = re.sub(
        r"\\frac\s+([^\s{}]+)\s+([^\s{}]+)",
        lambda match: f"({match.group(1)})/({match.group(2)})",
        text,
    )

    text = text.replace("^", "**")#11
    text = re.sub(
        r"(?<=\d)\s+(\d+/\d+)",
        lambda match: "+" + match.group(1),
        text,
    )

    text = re.sub(#12
        r"(?<=\d),(?=\d\d\d(\D|$))",
        "",
        text,
    )

    return text.replace("{", "").replace("}", "").strip().lower()
```

#1 LaTeX formatting to be replaced (left: original value, right: new value)

#2 Strip chat special tokens like <|assistant|>.

#3 Dictionary mapping to convert unicode superscripts to plaintext superscripts

#4 Strip leading multiple choice labels (e.g., like "c. 3" → 3).

#5 Remove angle-degree markers.

#6 Unwrap "\text{...}" if the whole string is wrapped.
#7 Strip inline/display mathwrappers: \(\) \[\].
#8 LaTeX canonicalization
#9 Normalize number and root expressions.
#10 Convert LaTeX fractions into division form.
#11 Handle exponents and mixed numbers.
#12 Remove thousands separators in numbers.

This `normalize_text` function takes an extracted answer string and rewrites it into a standardized format that we can reliably compare against reference solutions. It first strips away special tokens and unnecessary LaTeX clutter, such as `\left`, `\right`, or degree symbols. It then unwraps cases like `\text{...}`, removes inline math markers, and rewrites common structures into a calculator-style form. For example, it turns `\sqrt{a}` into `sqrt(a)` and `\frac{a}{b}` into `(a)/(b)`. Finally, it normalizes exponents, mixed numbers, and thousands separators and cleans up braces and casing. In short, the function transforms differently formatted LaTeX outputs into a clean, standardized string representation.

Let's try the `normalize_text` function on our model answer:

```
print(normalize_text(extract_final_candidate(model_answer)))
```

Instead of printing the answer with LaTeX formatting (`r " \dfrac{14}{3} "`), it returns the answer in a standardized, LaTeX-free form:

```
"(14)/(3)"
```

Next, let's try a differently formatted answer:

```
print(normalize_text(r"\text{\[\frac{14}{3}\]}"))
```

This also returns `" (14)/(3) "`, as intended.

We now have a robust method that extracts answer texts from an LLM's response. In the next section, we'll implement a function that compares the LLM answer with a correct reference answer.

3.6 Verifying mathematical equivalence

So far, we have implemented steps that ask an LLM to generate an answer, extract the relevant portion, and normalize it. The next step, as illustrated in figure 3.8, is to verify the extracted answer by comparing it to a correct reference answer, which in technical contexts is referred to as the *ground truth*. We will use such a verifier again in chapter 6 to create a verifiable reward signal, and in later chapters we'll use it to check whether our training changes improved our model.

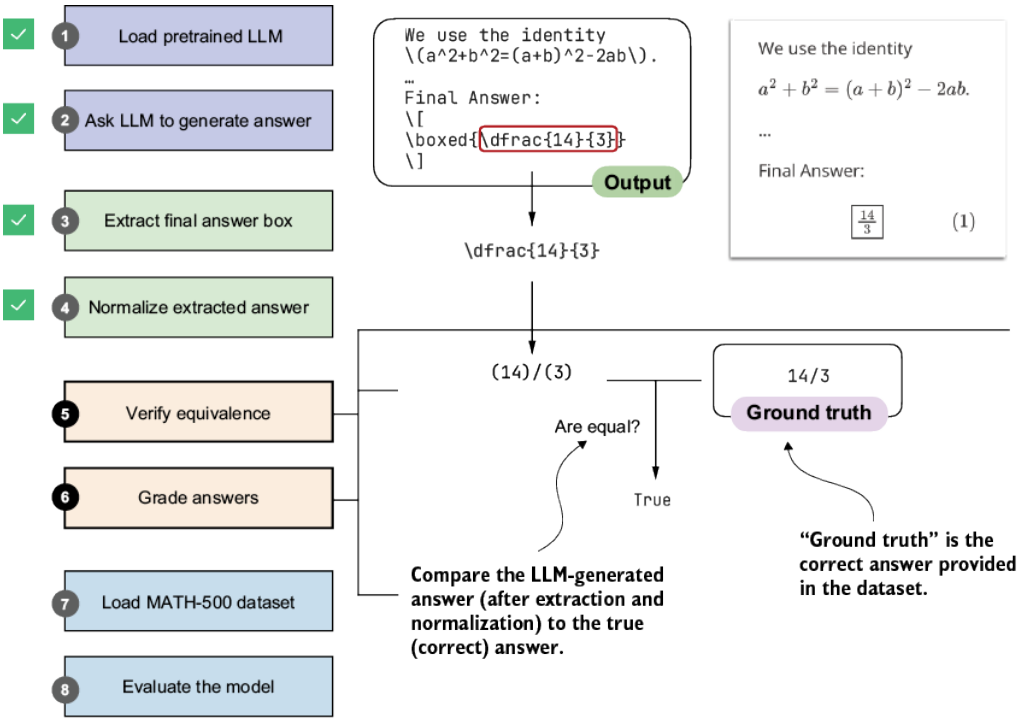


Figure 3.8 An illustration of how an LLM-generated answer is checked against the correct reference answer (ground truth). The LLM's final boxed answer is extracted, normalized, and compared to the correct answer provided in the dataset. If they match, the response is graded as correct.

Note that to implement the equality check shown in figure 3.8, a direct comparison using Python's `==` operator is not sufficient, since expressions like `" 14/3 "` and `" (14)/(3) "` would not match, and equivalent but unnormalized fractions such as `" (28)/(6) "` and `" (14)/(3) "` would also be treated as unequal.

As part of our equality check, we'll add an intermediate step: parsing the extracted, normalized answer with a symbolic math engine. For this, we'll use the open source SymPy math library (**https://sympy.org**), which has been developed and tested over two decades and has become a staple of scientific computing in Python. The parsing function is implemented in listing 3.7.

NOTE If you haven't installed the dependencies from chapter 2, you can manually install SymPy with `uv pip install sympy` (or `uv add sympy`).

Listing 3.7 Using the SymPy parser for a mathematical equality check

```
from sympy.parsing import sympy_parser as spp
from sympy.core.sympify import SympifyError
from sympy.polys.polyerrors import PolynomialError
from tokenize import TokenError

def sympy_parser(expr):
    if expr is None or len(expr) > 2000: #1
        return None

    try:
        return spp.parse_expr(
            expr,
            transformations=(
                *spp.standard_transformations,  #2

                spp.implicit_multiplication_application,#3
            ),

            evaluate=True,  #4
        )
    except (SympifyError, SyntaxError, TypeError, AttributeError,
            IndexError, TokenError, ValueError, PolynomialError):
        return None
```

#1 To avoid crashing on long garbage responses
#2 Standard transformations like handling parentheses
#3 Allow omitted multiplication symbols (e.g., 2y → 2*y).
#4 Evaluate during parsing so simple constants simplify (e.g., 2+3 → 5).

This `sympy_parser` function takes an input expression, such as the normalized answers we extract from the LLM response, and converts it into a SymPy object that can be reliably compared for mathematical equivalence. To do so, it applies SymPy's standard parsing rules, supports implicit multiplication (for example, 2*y* instead of 2 * *y*), and simplifies basic arithmetic (so 2 + 3 becomes 5).

NOTE The `sympy_parser` accounts for what may seem like an excessive number of error cases, but these are all errors I've encountered when evaluating the model on the 500 MATH-500 problems. The model does not always generate perfectly formatted outputs.

Let's see `sympy_parser` in action and apply it to a normalized answer candidate:

```
print(sympy_parser(normalize_text(
    extract_final_candidate(model_answer)
)))
```

This returns the fraction `14/3`. Next, let's try an unnormalized fraction:

```
print(sympy_parser("28/6"))
```

Similarly, this returns `14/3`.

Using `sympy_parser`, we can now implement the equality check function.

Listing 3.8 Equality check function using SymPy

```
from sympy import simplify

def equality_check(expr_gtruth, expr_pred):
    if expr_gtruth == expr_pred:  #1
        return True


    gtruth, pred = sympy_parser(expr_gtruth), sympy_parser(expr_pred)#2

    if gtruth is not None and pred is not None:  #3
        try:
            return simplify(gtruth - pred) == 0  #4
        except (SympifyError, TypeError):
            pass

    return False
```

#1 First, check if the two expressions are exactly the same string.
#2 Parse both expressions into SymPy objects (returns None if parsing fails).
#3 If both expressions were parsed successfully, try symbolic comparison.
#4 If the difference is 0, they are equivalent.

The `equality_check` function in the preceding listing determines whether a model's answer matches the ground-truth solution. It first looks for an exact string match, which is the simplest case. If the strings differ, it parses both expressions into SymPy objects (via the `sympy_parser` function we implemented in listing 3.7) and checks whether their difference simplifies to zero. This lets us recognize answers that may look different on the surface (for example, 14/3 and 28/6) but are mathematically the same.

Let's try the equality checker from listing 3.8 on an example:

```
print(equality_check(
    normalize_text("13/4."),
    normalize_text(r"(13)/(4)")
))
```

As intended, this ignores the formatting and returns `True`. Next, let's try a more challenging example and see whether the symbolic math parser recognizes that 0.5 is the same as 1/2:

```
print(equality_check(
    normalize_text("0.5"),
    normalize_text(r"(1)/(2)")
))
```

This also returns `True`. Now, let's try a negative example:

```
print(equality_check(
    normalize_text("14/3"),
    normalize_text("15/3")
))
```

This returns `False`, since the expressions are different.

So far, so good. Based on these encouraging results, we might conclude that we now have a robust equality checker that we can use to evaluate the LLM on a math benchmark dataset. To make sure it's ready for prime time, let's try one more example:

```
print(equality_check(
    normalize_text("(14/3, 2/3)"),
    normalize_text("(14/3, 4/6)")
))
```

In this case, we are comparing two tuples. Since 2/3 and 4/6 are mathematically equivalent, we would expect the result to be `True`. Instead, the function returns `False` because it currently handles only simple expressions, not tuples. We will address this limitation in the next section.

3.7 Grading answers

Now we'll build on the previous section's mathematical equality-checking function to implement a robust grading function that can also handle tuple-like expressions, such as correctly comparing `" (14/3, 2/3) "` and `" (14/3, 4/6) "`.

First, we'll implement a Python helper function that splits such tuple-like expressions into individual subparts.

Listing 3.9 A helper function to split tuple-like expressions

```
def split_into_parts(text):
    result = [text]

    if text:  #1
        if (
            len(text) >= 2
            and text[0] in "([" and text[-1] in ")]"
            and "," in text[1:-1]
        ):
            items = [p.strip() for p in text[1:-1].split(",")]  #2
            if all(items):
                result = items
    else:  #3
        result = []

    return result
```

#1 Check if text looks like a tuple or list, e.g., "(a, b)" or "[a, b]".
#2 Split on commas inside brackets and strip whitespace.
#3 If text is empty, return an empty list.

The preceding `split_into_parts` function helps us handle answers with multiple components. If the input looks like a tuple or list, such as `(a, b)` or `[a, b]`, it splits the content on the commas and returns the individual pieces. (If the string is empty, it simply returns an empty list.) In essence, this function breaks down multipart answers into smaller parts that can be checked one by one.

Before we implement the grading function, let's take `split_into_parts` for a test drive on the tuple-like expression from earlier:

```
split_into_parts(normalize_text(r"(14/3, 2/3)"))
```

This returns `[ ' 14/3 ' , ' 2/3 ' ]`, as desired.

Now, we can implement the `grade_answer` function (listing 3.10), which splits tuple-like expressions (if present) into subparts and then uses the `equality_check` function from the previous section to compare a generated answer to a reference (ground truth) answer.

Listing 3.10 Function to grade predicted answers against the ground truth

```
def grade_answer(pred_text, gt_text):
    result = False   #1
    if pred_text is not None and gt_text is not None:  #2
        gt_parts = split_into_parts(
            normalize_text(gt_text)
        )
        pred_parts = split_into_parts(
            normalize_text(pred_text)
        )

        if (gt_parts and pred_parts                #3
           and len(gt_parts) == len(pred_parts)):   #3
            result = all(
                equality_check(gt, pred)
                for gt, pred in zip(gt_parts, pred_parts)
            )  #4

    return result  #5
```

#1 Default outcome if checks fail
#2 Only continue if both inputs are non-empty strings.
#3 Ensure both sides have the same number of valid parts.
#4 Check each part for mathematical equivalence.
#5 True only if all checks passed

This implementation of the `grade_answer` function first assumes the prediction is incorrect (`False`) and continues only if both the prediction and the ground truth are non-empty. It then normalizes each side and splits them into subparts (for example, breaking `"(14/3, 2/3)"` into `["14/3", "2/3"]`). If the number of subparts matches, it compares them one by one using `equality_check`. The result is returned as correct (`True`) only if all pairs match mathematically.

We can think of the `grade_answer` function as an advanced version of the `equality_check` function from the previous section. It can split tuple-like expressions and normalize the answers before applying the `equality_check` function.

On simple expressions, it works similarly to `equality_check`, returning `True` if two expressions are mathematically equivalent:

```
grade_answer("14/3", r"\frac{14}{3}")
```

In addition, it now also returns `True` for two mathematically equivalent tuple-like expressions:

```
grade_answer(r"(14/3, 2/3)", "(14/3, 4/6)")
```

To check the `grade_answer` function more comprehensively, the following listing contains more diverse test cases.

Listing 3.11 Test cases and demo function to test the grader

```
tests = [  #1
        ("check_1", "3/4", r"\frac{3}{4}", True),
        ("check_2", "(3)/(4)", r"3/4", True),
        ("check_3", r"\frac{\sqrt{8}}{2}", "sqrt(2)", True),
        ("check_4", r"\( \frac{1}{2} + \frac{1}{6} \)", "2/3", True),
        ("check_5", "(1, 2)", r"(1,2)", True),
        ("check_6", "(2, 1)", "(1, 2)", False),
        ("check_7", "(1, 2, 3)", "(1, 2)", False),
        ("check_8", "0.5", "1/2", True),
        ("check_9", "0.3333333333", "1/3", False),
        ("check_10", "1,234/2", "617", True),
        ("check_11", r"\text{2/3}", "2/3", True),
        ("check_12", "50%", "1/2", False),
        ("check_13", r"2\cdot 3/4", "3/2", True),
        ("check_14", r"90^\circ", "90", True),
        ("check_15", r"\left(\frac{3}{4}\right)", "3/4", True),
        ("check_16", r"2²", "2**2", True),
    ]


def run_demos_table(tests):
    header = ("Test", "Expect", "Got", "Status")
    rows = []
    for name, pred, gtruth, expect in tests:
        got = grade_answer(pred, gtruth)  #2
        status = "PASS" if got == expect else "FAIL"
        rows.append((name, str(expect), str(got), status))

    data = [header] + rows

    col_widths = [  #3
        max(len(row[i]) for row in data)
        for i in range(len(header))
    ]

    for row in data:  #4
        line = " | ".join(
            row[i].ljust(col_widths[i])
            for i in range(len(header))
        )
        print(line)

    passed = sum(r[3] == "PASS" for r in rows)  #5
    print(f"\nPassed {passed}/{len(rows)}")      #5
```

#1 Defines the test cases: (name, prediction, ground truth, expected result)

#2 Runs an equality check

#3 Computes the max width for each column to align table nicely

#4 Prints the table row by row

#5 Prints the summary of passed tests

The code in listing 3.11 is a simple test suite that uses a selection of `tests` to check whether the `grade_answer` function works as intended. The `tests` list contains tuples covering fractions, LaTeX notation, tuple inputs, decimals, percentages, and other tricky formats. The `run_demos_table` function then runs each test by calling `grade_answer`, collects the outcomes, and organizes the results into a formatted table.

Calling the `run_demos_table(tests)` function in listing 3.11 prints the following table:

```
Test      | Expect | Got   | Status
check_1   | True   | True  | PASS
check_2   | True   | True  | PASS
check_3   | True   | True  | PASS
check_4   | True   | True  | PASS
check_5   | True   | True  | PASS
check_6   | False  | False | PASS
check_7   | False  | False | PASS
check_8   | True   | True  | PASS
check_9   | False  | False | PASS
check_10  | True   | True  | PASS
check_11  | True   | True  | PASS
check_12  | False  | False | PASS
check_13  | True   | True  | PASS
check_14  | True   | True  | PASS
check_15  | True   | True  | PASS
check_16  | True   | True  | PASS

Passed 16/16
```

As you can see from the `PASS` results, the `grade_answer` function is relatively robust and can handle a variety of differently formatted expressions.

EXERCISE 3.1: ADDING MORE TEST CASES

Try to think of additional test cases, ideally, challenging ones, and add them to the `run_demos_table()` function. Can you find cases where the check fails incorrectly?

With `grade_answer` implemented, we now have the building blocks to evaluate the LLM. In the next section, we'll load a math dataset on which to evaluate it.

3.8 Loading the evaluation dataset

As you have seen so far in this chapter, implementing a robust verification pipeline can be tedious. Fortunately, we now have all the pieces in place, from answer extraction to grading, and we're ready to evaluate the LLM on a benchmark dataset. As shown in figure 3.9, we'll use the MATH-500 dataset (**https://huggingface.co/datasets/HuggingFaceH4/MATH-500**), a widely used benchmark for reasoning models. It is a curated collection of 500 problems sampled from the original MATH dataset.

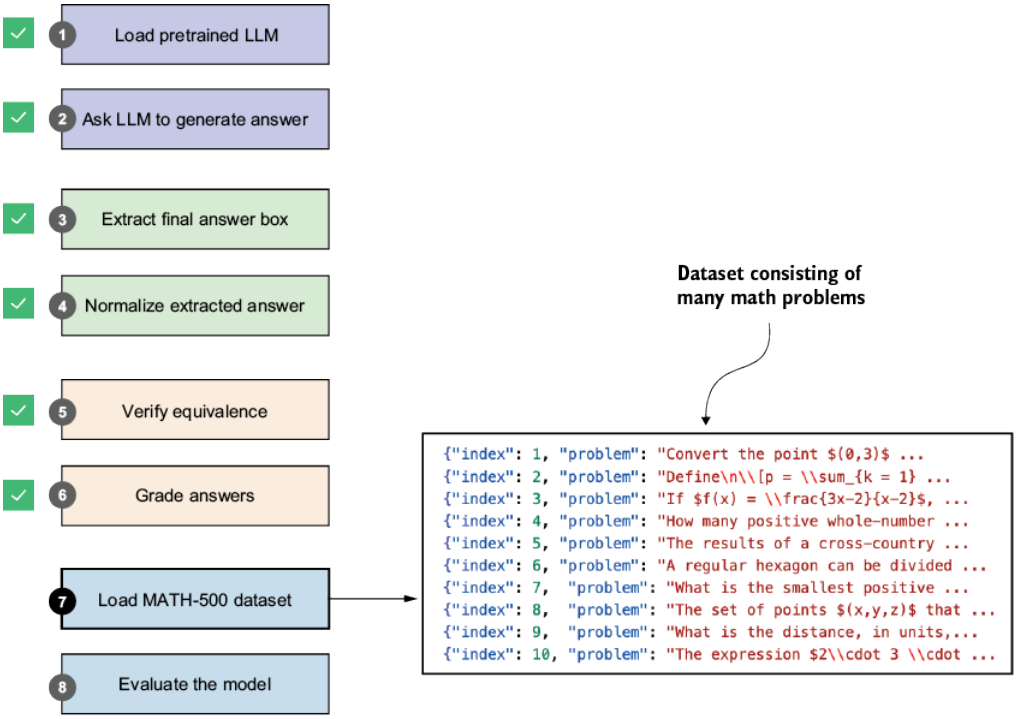


Figure 3.9 Loading the evaluation dataset. We completed steps 2-6 (generate, extract, normalize, verify, and grade answers) in the previous sections; the two remaining steps are to load the full dataset (step 7) and apply the same procedure across all the problems to evaluate the model (step 8).

Using MATH-500 is a practical choice for three reasons. First, MATH-500 is large enough to be meaningful but still small enough to keep repeated evaluations throughout the book manageable, whereas running the full MATH dataset each time would be much slower. Second, in later chapters we'll use a non-overlapping training subset derived from the original MATH dataset, so keeping MATH-500 as a held-out evaluation set gives us a clean reference point. Third, MATH-500 is a common benchmark dataset in the reasoning-model literature, which makes the results in this book easier to compare with prior work. I also prefer MATH-500 over simpler datasets such as GSM8K because it is more challenging for modern reasoning models and it better matches the kind of multistep symbolic verification pipeline we are building in this chapter.

We'll load the MATH-500 dataset using the following code.

Listing 3.12 Loading the MATH-500 dataset

```
import json
import requests

def load_math500_test(local_path="math500_test.json", save_copy=True):
    local_path = Path(local_path)
    url = (
        "https://raw.githubusercontent.com/rasbt/reasoning-from-scratch/"
        "main/ch03/01_main-chapter-code/math500_test.json"
    )

    if local_path.exists():
        with local_path.open("r", encoding="utf-8") as f:
            data = json.load(f)
    else:
        r = requests.get(url, timeout=30)
        r.raise_for_status()
        data = r.json()

        if save_copy:  # Saves a local copy
            with local_path.open("w", encoding="utf-8") as f:
                json.dump(data, f, indent=2)

    return data

math_data = load_math500_test()
print("Number of entries:", len(math_data))
```

This prints the following result:

```
Number of entries: 500
```

LOADING THE DATASET FROM HUGGING FACE MODEL HUB

The MATH-500 dataset split was originally proposed in the PRM800K repository (**https://github.com/openai/prm800k/tree/main?tab=readme-ov-file#math-splits**), and it is also available on the Hugging Face Hub (**https://huggingface.co/datasets/HuggingFaceH4/MATH-500**). The code we use in this chapter loads a copy from the code repository so you can ensure reproducibility if the external sources change.

If you prefer to download the dataset directly from Hugging Face, you can use the following optional code. Note that this requires the `datasets` library, which can be installed via `pip install datasets` or `uv add datasets`:

```
from datasets import load_dataset
dset = load_dataset("HuggingFaceH4/MATH-500", split="test")
```

Before we implement the model evaluation pipeline in the next section, let's take a closer look at the structure of the dataset by printing its first entry (we'll use the built-in `pprint` library for nicer format-

ting):

```
from pprint import pprint
pprint(math_data[0])
```

This produces the following output:

```
{'answer': '\\left( 3, \\frac{\\pi}{2} \\right)',
 'level': 2,
 'problem': 'Convert the point $(0,3)$ in rectangular coordinates to polar '
            'coordinates.  Enter your answer in the form $(r,\\theta),$ where '
            '$r > 0$ and $0 \\le \\theta < 2 \\pi.$',
 'solution': 'We have that $r = \\sqrt{0^2 + 3^2} = 3.$  Also, if we draw the '
             'line connecting the origin and $(0,3),$ this line makes an angle '
             'of $\\frac{\\pi}{2}$ with the positive $x$-axis.\n'
             '\n'
             '[asy]\n'
             'unitsize(0.8 cm);\n'
             '\n'
             'draw((-0.5,0)--(3.5,0));\n'
             'draw((0,-0.5)--(0,3.5));\n'
             'draw(arc((0,0),3,0,90),red,Arrow(6));\n'
             '\n'
             'dot((0,3), red);\n'
             'label("$(0,3)$", (0,3), W);\n'
             'dot((3,0), red);\n'
             '[/asy]\n'
             '\n'
             'Therefore, the polar coordinates are $\\boxed{\\left( 3, '
             '\\frac{\\pi}{2} \\right)}.$',
 'subject': 'Precalculus',
 'unique_id': 'test/precalculus/807.json'}
```

As you can see, the dataset entry is formatted as a Python dictionary with keys and values, sorted in alphabetical order. These are the relevant keys:

- `problem` : The math question or problem for the LLM to solve
- `answer` : The correct (ground truth) answer we want to compare the LLM answer against
- `solution` : A worked-out, step-by-step explanation of the problem (not used in this chapter, but useful for training or analysis)

Now that we have a pretrained LLM, evaluation functions, and a benchmark dataset to work with, we can evaluate the model.

3.9 Evaluating the model

In this section, we'll put the LLM text generation and evaluation tools from steps 2–6 in figure 3.10 into practice and apply them to the MATH-500 dataset (step 8), which we loaded in the previous section. This full pipeline will be important in future chapters because it will be the main way we'll measure progress when we compare prompting methods and training-based improvements.

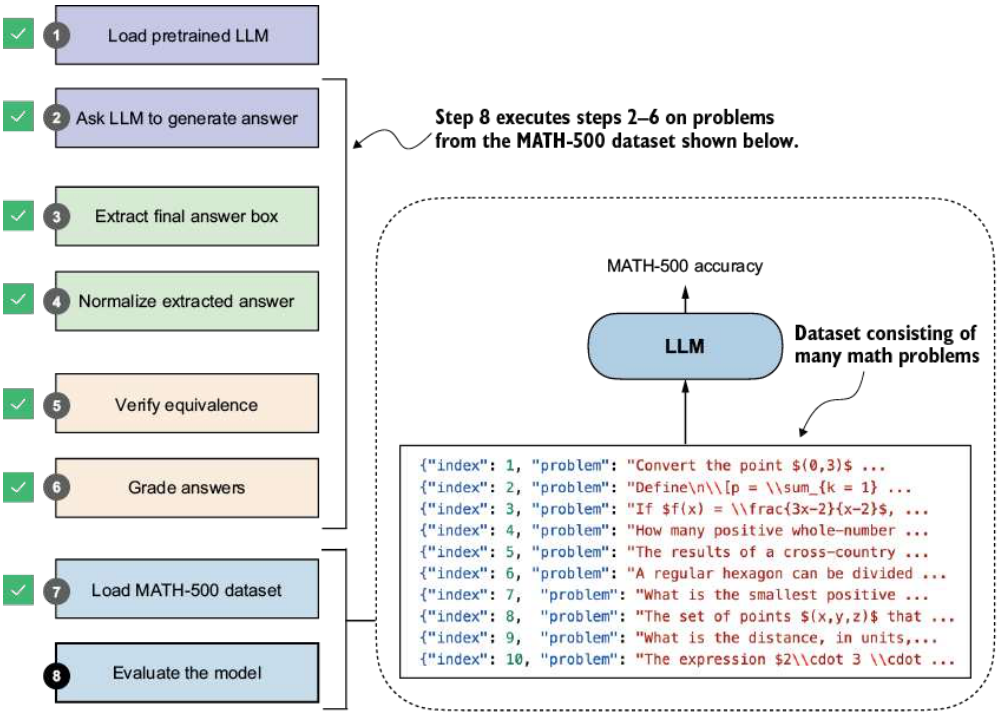


Figure 3.10 The complete evaluation pipeline for the MATH-500 dataset. After loading the dataset (step 7), steps 2–6 are applied systematically across all problems to obtain the final model evaluation (step 8).

As you may recall from section 3.4, our answer-checking pipeline expects the model to return the answer in boxed form, which is a common convention when evaluating reasoning models on math problems. To increase the likelihood that the model adheres to this format, we can format the prompt as follows.

Listing 3.13 Function to render a prompt template for math evaluation

```
def render_prompt(prompt):
    template = (
        "You are a helpful math assistant.\n"
        "Answer the question and write the final result on a new line as:\n"
        "\\boxed{ANSWER}\n\n"
        f"Question:\n{prompt}\n\nAnswer:"
    )
    return template
```

Let's now apply this prompt template to the example prompt introduced in section 3.2 and shown again here:

```
prompt = (
    r"If $a+b=3$ and $ab=\tfrac{13}{6}$, "
    r"what is the value of $a^2+b^2$?"
)
prompt_fmt = render_prompt(prompt)
print(prompt_fmt)
```

The formatted prompt is now as follows:

```
You are a helpful math assistant.
Answer the question and write the final result on a new line as:
\boxed{ANSWER}

Question:
If $a+b=3$ and $ab=\tfrac{13}{6}$, what is the value of $a^2+b^2$?

Answer:
```

Next, we'll pass the prompt to the text generation wrapper function we defined in listing 3.3 to recap the

text generation process before we construct the model evaluation function:

```
generated_text = generate_text_stream_concat(
    model, tokenizer, prompt_fmt, device,
    max_new_tokens=2048,
    verbose=True
)
```

Using this prompt example, the model responds with a relatively brief answer: `" \boxed{10} "`. (Note that the generated response may differ depending on whether you run the code on a CPU, CUDA, or MPS device.)

While brevity can speed up generation by reducing the number of tokens, the response is incorrect. In section 3.3, by contrast, without a prompt template, the model produced a longer response, which led to the correct answer, 14/3.

Whether the prompt template is well suited to a given model and task needs to be determined on a larger set of examples, such as the MATH-500 dataset, before we can draw any conclusions.

PROMPT TEMPLATE CHOICES

The prompt template in listing 3.13 is used here to show how you can implement a model evaluation pipeline with answers that are automatically checked for correctness. The chosen template encourages short outputs, which will let you work through this chapter efficiently on a first read. Afterward, I recommend revisiting the chapter's examples with alternative settings to optimize accuracy on the official Qwen3 reasoning model variant used here as a reference model.

As it turns out, using no prompt template boosts the base model's performance by 50%, but it reduces the reasoning model's accuracy by 40%.

Additionally, we can experiment with alternative prompt templates. For instance, the standard prompt commonly used for the MATH-500 benchmark is a variant of listing 3.13 that swaps "Question:" for "Problem:". This seemingly minor change improves the base model's accuracy by approximately 20%, likely because it better matches the memorized training data (assuming the MATH-500 test set was included in the training corpus). While the base model benefits from this change, the reasoning model variant's accuracy drops by 30%.

Earlier, I mentioned that answer extraction is a simple, mechanical task that can be solved with deterministic code rather than by employing another LLM for answer extraction. Based on the change in accuracy across different prompting templates, it looks like our extraction method may not be reliable. This is not necessarily the case.

Also, it's not necessarily the case that the LLM generates misformatted answers. It may simply be incorrect, and switching to another LLM for extraction would not solve that. Smaller base models are often quite sensitive to prompt phrasing. In the next chapter, you'll see that this becomes even more apparent once we introduce additional prompt variations.

Before we implement the final model evaluation function, let's test our model evaluation pipeline from end to end on a smaller example using the following demo function.

Listing 3.14 Demo function to run the evaluation pipeline

```
def mini_eval_demo(model, tokenizer, device):
    ex = {  #1
        "problem": "Compute 1/2 + 1/6.",
        "answer": "2/3"
    }
    prompt = render_prompt(ex["problem"])     #2
    gen_text = generate_text_stream_concat(   #3
        model, tokenizer, prompt, device,      #3
        max_new_tokens=64,                     #3
    )                                          #3
    pred_answer = extract_final_candidate(gen_text)  #4
    is_correct = grade_answer(                       #5
        pred_answer, ex["answer"]                     #5
    )                                                 #5

    print(f"Device: {device}")
    print(f"Prediction: {pred_answer}")
    print(f"Ground truth: {ex['answer']}")
    print(f"Correct: {is_correct}")
```

#1 Test the example with "problem" and "answer" fields.
#2 1. Apply prompt template
#3 2. Generate response
#4 3. Extract and normalize answer
#5 4. Grade answer

This `mini_eval_demo` function combines all the aspects we have covered so far in this chapter:

1. Applying a prompt template
2. Feeding the formatted prompt to the LLM to generate an answer
3. Extracting and normalizing the answer
4. Grading the answer

The `mini_eval_demo` function essentially connects the evaluation components into a small function that we can use to test the code before building our final evaluation pipeline for the MATH-500 dataset. The code starts with a toy example (`ex`), renders the problem into the prompt template (`prompt`), and streams a response from the model (`generate_text_stream_concat`). It then parses the model output into a final candidate answer (`pred_answer`) and grades it against the ground truth with `grade_answer` . Lastly, it prints the results for us to evaluate.

Calling the `mini_eval_demo(model, tokenizer, device)` function results in the following output:

```
Device: mps
Prediction: 1/3
Ground truth: 2/3
Correct: False
```

You can see that the generated answer (`" 1/3 "`) was correctly extracted, but it doesn't match the correct answer (`" 2/3 "`), so the check returns `False` . (Note that your results may differ depending on whether you run the code on a CPU, CUDA, or MPS device.)

Now that we have tested our workflow on a simple example, let's implement it on the MATH-500 dataset.

Listing 3.15 End-to-end model evaluation pipeline for MATH-500 dataset

```
import time


def eta_progress_message(  #1
    processed,
    total,
    start_time,
```

```
    show_eta=False,
    label="Progress",
):
    progress = f"{label}: {processed}/{total}"
    pad_width = len(f"{label}: {total}/{total} | ETA: 00h 00m 00s")
    if not show_eta or processed <= 0:
        return progress.ljust(pad_width)

    elapsed = time.time() - start_time
    if elapsed <= 0:
        return progress.ljust(pad_width)

    remaining = max(total - processed, 0)

    if processed:
        avg_time = elapsed / processed
        eta_seconds = avg_time * remaining
    else:
        eta_seconds = 0

    eta_seconds = max(int(round(eta_seconds)), 0)
    minutes, rem_seconds = divmod(eta_seconds, 60)
    hours, minutes = divmod(minutes, 60)
    if hours:
        eta = f"{hours}h {minutes:02d}m {rem_seconds:02d}s"
    elif minutes:
        eta = f"{minutes:02d}m {rem_seconds:02d}s"
    else:
        eta = f"{rem_seconds:02d}s"

    message = f"{progress} | ETA: {eta}"
    return message.ljust(pad_width)


def evaluate_math500_stream(
    model,
    tokenizer,
    device,
    math_data,
    out_path=None,
    max_new_tokens=512,
    verbose=False,
):

    if out_path is None:
        dev_name = str(device).replace(":", "-")      #2
        out_path = Path(f"math500-{dev_name}.jsonl")

    num_examples = len(math_data)
    num_correct = 0
    start_time = time.time()

    with open(out_path, "w", encoding="utf-8") as f:   #3
        for i, row in enumerate(math_data, start=1):
            prompt = render_prompt(row["problem"])     #4
            gen_text = generate_text_stream_concat(    #5
                model, tokenizer, prompt, device,
                max_new_tokens=max_new_tokens,
                verbose=verbose,
            )

            extracted = extract_final_candidate(       #6
                gen_text
            )
            is_correct = grade_answer(                 #7
                extracted, row["answer"]
            )
            num_correct += int(is_correct)
```

```
            record = {                                  #8
                "index": i,
                "problem": row["problem"],
                "gtruth_answer": row["answer"],
                "generated_text": gen_text,
                "extracted": extracted,
                "correct": bool(is_correct),
            }
            f.write(json.dumps(record, ensure_ascii=False) + "\n")

            progress_msg = eta_progress_message(
                processed=i,
                total=num_examples,
                start_time=start_time,
                show_eta=True,
                label="MATH-500",
            )
            print(progress_msg, end="\r", flush=True)

            if verbose:                                 #9
                print(
                    f"\n\n{'='*50}\n{progress_msg}\n"
                    f"{'='*50}\nExtracted: {extracted}\n"
                    f"Expected:  {row['answer']}\n"
                    f"Correct so far: {num_correct}\n{'-'*50}"
                )

    seconds_elapsed = time.time() - start_time
    acc = num_correct / num_examples if num_examples else 0.0
    print(f"\nAccuracy: {acc*100:.1f}% ({num_correct}/{num_examples})")
    print(f"Total time: {seconds_elapsed/60:.1f} min")
    print(f"Logs written to: {out_path}")
    return num_correct, num_examples, acc
```

#1 Helper function to print progress with optional ETA (estimated time to arrival)
#2 Make the filename compatible with Windows.
#3 Save the results for inspection.
#4 Apply the prompt template.
#5 Generate a response.
#6 Extract and normalize the answer.
#7 Grade the answer.
#8 Record to be saved for inspection
#9 Print the responses during generation.

The `evaluate_math500_stream` function in listing 3.15 uses the same main steps as the smaller demo function from listing 3.14: for each problem, it renders the prompt, streams a model response, extracts the answer candidate, and grades it against the reference answer.

In addition to iterating over a dataset with multiple entries, it adds a few extra bells and whistles. For instance, it saves the generated responses to a JSON file in a Python dictionary-like format for record keeping and closer inspection.

Let's now run this function on a subset: the first 10 examples of MATH-500. This takes about 0.7 minutes on a Mac Mini with an M4 chip. (Evaluating the reasoning model variant takes about 7 minutes because it generates longer responses.)

```
print("Model:", WHICH_MODEL)
print("Device:", device)
num_correct, num_examples, acc = evaluate_math500_stream(
    model, tokenizer, device,
    math_data=math_data[:10],  #1
    max_new_tokens=2048,
    verbose=False              #2
)
```

#1 Only evaluate on the first 10 examples.

#2 Set to true to read the responses as they are being generated.

In the preceding code example, we set `max_new_tokens` to a generous 2,048 because the reasoning model variant, by design, tends to generate much longer responses, and we don't want to cut it off prematurely. This leads to much longer evaluation times, and it may appear that generation is stuck. Optionally, you could set `verbose=True` to see the response being generated live, token by token.

The result of running `evaluate_math500_stream` is as follows:

```
Model: base
Device: mps
MATH-500: 10/10 | ETA: 00s
Accuracy: 30.0% (3/10)
Total time: 0.4 min
```

(As usual, the results may differ depending on whether you execute the code on a CPU, CUDA, or MPS device.)

As you can see, the model achieves a relatively low accuracy of 30%. We can open the `math500_base-mps.jsonl` file in a text editor to analyze the results together with the generated response. For instance, the answers have been successfully extracted in all cases, but they are simply wrong, which indicates that the model does not have very strong math-problem-solving capabilities (yet). This is expected because it is merely a base model.

LOADING THE .JSONL FILE PROGRAMMATICALLY

The `.jsonl` file suffix is a convention used for JSON files with one data entry per row. You can view it in your favorite text editor. Optionally, you can load the `.jsonl` file created during the evaluation in Python using the following code:

```
dev_name = str(device).replace(":", "-")
local_path = f"math500_{WHICH_MODEL}-{dev_name}.jsonl"
results = []
with open(local_path, "r") as f:
    for line in f:
        if line.strip():
            results.append(json.loads(line))
```

The reasoning model variant, which you can enable by setting `WHICH_MODEL = " reasoning "` in listing 3.1, performs much better, achieving 90% accuracy on the same 10-sample subset and 50.8% on the complete 500-sample dataset, as shown in table 3.1.

Table 3.1 MATH-500 task accuracy on different devices

Mode	Device	Accuracy	MATH-500 size
Base	CPU	30%	10
Base	CUDA	30%	10
Base	MPS	30%	10
Reasoning	CPU	90%	10
Reasoning	CUDA	90%	10
Reasoning	MPS	80%	10
Base	CUDA	15.3%	500
Reasoning	CUDA	50.8%	500

As shown in table 3.1, the reasoning variant, with its longer responses, has drastically improved accuracy, but it also increases compute intensity and answer generation time substantially (from 0.4 minutes for the base model to 7 minutes for the reasoning model on a Mac Mini with an M4 chip on the 10-sample subset, and from 13.3 minutes to 185.4 minutes on an NVIDIA H100 GPU on the 500-sample dataset), which highlights one of the tradeoffs of using reasoning models. Note that these numbers were obtained in PyTorch 2.8 and can differ across PyTorch versions.

TIP The book's code repository contains a bonus script that runs the code in this chapter in batched mode (**https://github.com/rasbt/reasoning-from-scratch/blob/main/ch03/02_math500-verifier-scripts/evaluate_math500_batched.py**). This means it processes multiple examples per forward pass to speed up evaluation, but it requires more RAM. Using a batch size of 128 to evaluate all 500 samples reduces the running time of the base model from 13.3 minutes to 3.3 minutes on an H100 GPU. Similarly, it reduces the running time of the reasoning model from 185.4 minutes to 14.6 minutes. Note that the H100 is used as an example here; the script is compatible with other GPUs as well.

EXERCISE 3.2: CALCULATING THE AVERAGE RESPONSE LENGTH

Try modifying the code in this chapter to also report the average response length in the `evaluate_math500_stream` function (listing 3.15). Instead of modifying the function directly, you could also compute the response length from the generated JSON report files.

EXERCISE 3.3: EXTENDING OR CHANGING THE EVALUATION DATASET

We chose a subset of only 10 examples for computational efficiency. You can also run the code on larger or different portions of the dataset to see whether the 10-sample subset is representative. Ideally, you could also experiment with your own data. (For reference, evaluating the base model on the complete MATH-500 dataset takes about 13.3 minutes for the base model and 185.4 minutes for the reasoning model on an H100.)

EXERCISE 3.4: EXPERIMENTING WITH DIFFERENT PROMPT TEMPLATES

Models can be sensitive to different prompt templates. Experiment with different prompt templates in listing 3.13 to see how they affect the results. Also, while the Qwen3 team recommends using the base model without an additional chat template, you can enable the `apply_chat_template=True` setting in the tokenizer (listing 3.1) and observe whether it improves base model performance.

This concludes our implementation of a verification-based approach for math tasks (figure 3.11). I chose math because it is both natural to implement and widely used in reasoning-specific training, particularly reinforcement learning with verifiable rewards, which we'll cover in chapter 6. The same concept can be

extended to other domains, such as code; we didn't explore that here because executing code would involve setting up a secure virtual environment.

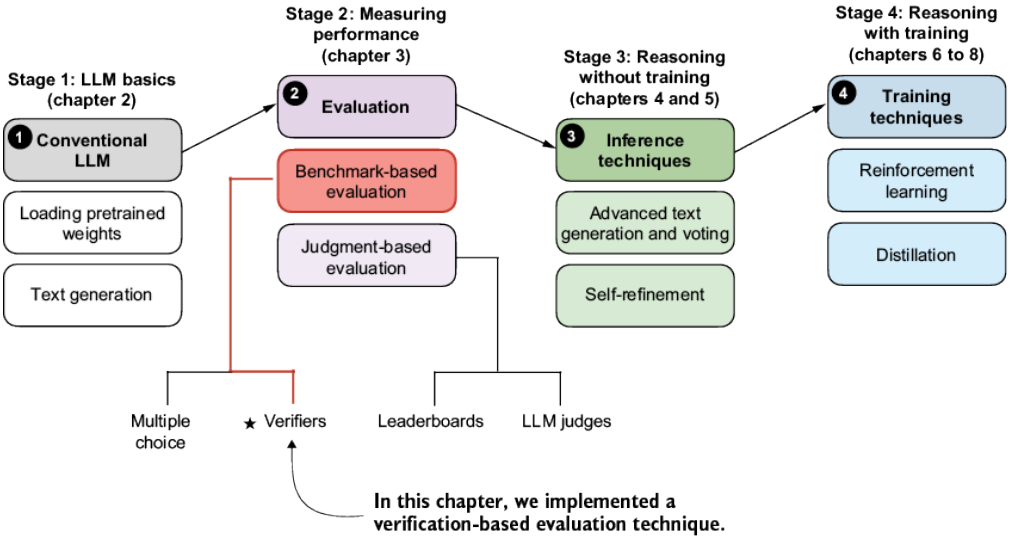


Figure 3.11 A model of the topics covered in this book. This chapter implements a verification-based evaluation pipeline. In the next chapter, we improve the reasoning capabilities of the LLM through more advanced inference techniques.

Before we move on, it's worth noting that evaluation also comes in many other flavors. In this chapter, we focused on verification-based accuracy for math problems because it is a popular method and because we will reuse this verifier as part of the reinforcement learning pipeline in chapters 6 and 7. For a broader overview, appendix F walks through other common evaluation strategies, such as multiple--choice benchmarks, verifiers, leaderboards, and LLM-as-judge setups. It also includes hands-on examples if you want a quick tour of how these methods work in practice.

Now that we have an evaluation framework in place, the next chapter will focus on improving reasoning capabilities through more advanced inference (text generation) techniques.

Summary

- There are four main evaluation methods for LLMs: multiple choice, verifiers, leaderboards, and LLM judges.
- Verification-based evaluation methods allow free-form answers and use external tools to check correctness.
- This chapter focuses on verification-based evaluation by building a math verifier that extracts, normalizes, and checks answers with SymPy.
  - The verification pipeline involves several core steps, from loading the LLM to running the evaluation on a dataset.
  - As part of the verification pipeline, answer extraction uses string parsing to locate boxed content (with fallback mechanisms for missing boxes).
  - Another step implements normalization, which standardizes diverse answer formats by stripping LaTeX and converting mathematical notation.
- Finally, the pipeline uses mathematical equivalence checking (via SymPy) to compare expressions symbolically.
- The MATH-500 dataset provides 500 curated math problems for evaluation.
- Prompt templates significantly impact model performance.
- The reasoning model achieves higher accuracy than the base model, but it requires a much longer running time.

4 Improving reasoning with inference-time scaling

This chapter covers

- Prompting an LLM to explain its reasoning to improve answer accuracy
- Modifying the text generation function to produce diverse responses
- Improving reasoning reliability by sampling multiple responses

Reasoning performance and answer accuracy can be improved without retraining or modifying the model itself. As shown in the overview in figure 4.1, this chapter covers two inference-time-scaling methods. These methods operate during inference, when the model generates text. As you'll see later in this chapter, both methods more than double the accuracy of the base model used in previous chapters.

We'll first look at what inference-time scaling is, before we discuss the inference methods shown in figure 4.1.

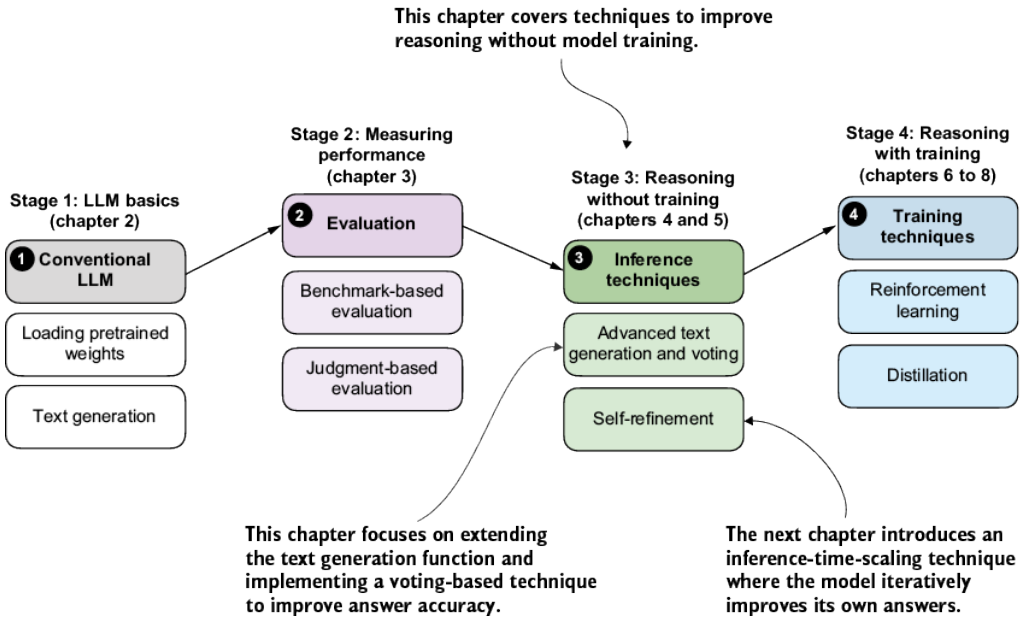


Figure 4.1 A model of the topics covered in this book. This chapter focuses on inference-time-scaling techniques, which improve reasoning without additional training (stage 3). In particular, it extends the text generation function and implements a voting-based method to improve answer accuracy. The next chapter will introduce another inference-time-scaling approach where the model iteratively refines its own answers.

4.1 Introduction to inference-time scaling

There are two main strategies to improve reasoning:

- Increasing training compute
- Increasing inference compute (also known as inference-time scaling or test-time scaling)

NOTE In machine learning and AI, "compute" refers to the computational resources required to train or run a model.

These two approaches are illustrated in figure 4.2, and the plots make it look like we can improve reasoning either by increasing training-time compute *or* inference-time compute. In reality, LLMs are usually designed to improve reasoning by combining heavy training-time compute (a topic of future chapters) *and* increased inference-time compute (the topic of this chapter).

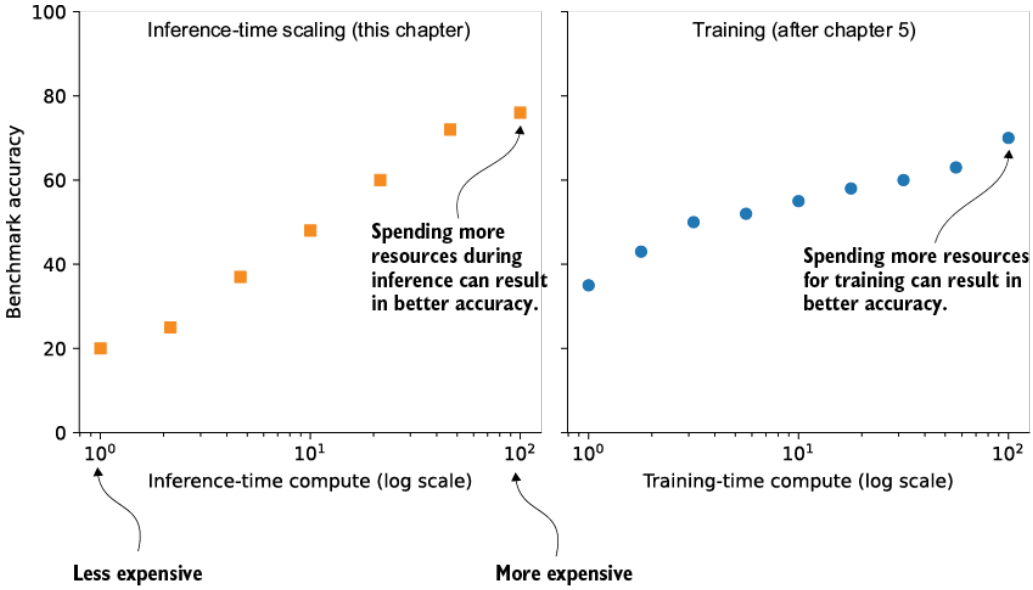


Figure 4.2 Comparing inference-time scaling (discussed in this chapter) and training-time scaling (discussed in chapters 6-8). Both approaches improve accuracy by using more compute, but inference-time scaling does this on the fly, without changing the model's weight parameters. The plots are inspired by OpenAI's article introducing its first reasoning model (https://openai.com/index/learning-to-reason-with-llms/).

Inference-time compute scaling (also called *inference-time scaling* or *test-time compute scaling*) means increasing computation at answer-generation time, after the model has already been trained, to improve the quality of its response. Instead of changing the model's weights, we let the trained model do more work per question, such as by generating more tokens and "thinking" longer, sampling multiple answers, or successively refining its answer.

In this book, we'll focus on three practical and foundational inference-time techniques (illustrated in figure 4.3):

- Method 1: Extending the *chain-of-thought* response to prompt the model to explain its reasoning. This simple technique can substantially improve accuracy.
- Method 2: Parallel sampling via *self-consistency*, where the model generates multiple responses and selects the most frequent one.
- Method 3: Iterative *self-refinement*, where the model reviews and improves its own reasoning and answers across multiple steps.

These methods fall under the category of inference-time scaling because they cause the model to generate more tokens, which increases compute requirements during inference. They improve accuracy while making inference more expensive, which is a common theme of inference-time-scaling techniques. In this chapter, we'll discuss the first two methods; the third will be covered in chapter 5.

The first method improves answer accuracy by prompting the model to explain its reasoning, a simple yet highly effective approach.

The second method samples multiple responses for the same input and selects the most frequent one. We'll do this by extending the text generation function introduced in the previous chapter and implementing *self-consistency*, a voting-based inference-time- scaling technique.

In total, I evaluated more than ten different inference-time-scaling techniques across thousands of experiments. I chose the three methods illustrated in figure 4.3 because they combine three desirable properties: they improve accuracy on the model used here, they represent the main practical paradigms of inference-time scaling, and they are simple enough to implement from scratch without too much extra machinery. They also cover the two main patterns: generating longer responses and generating multiple responses.

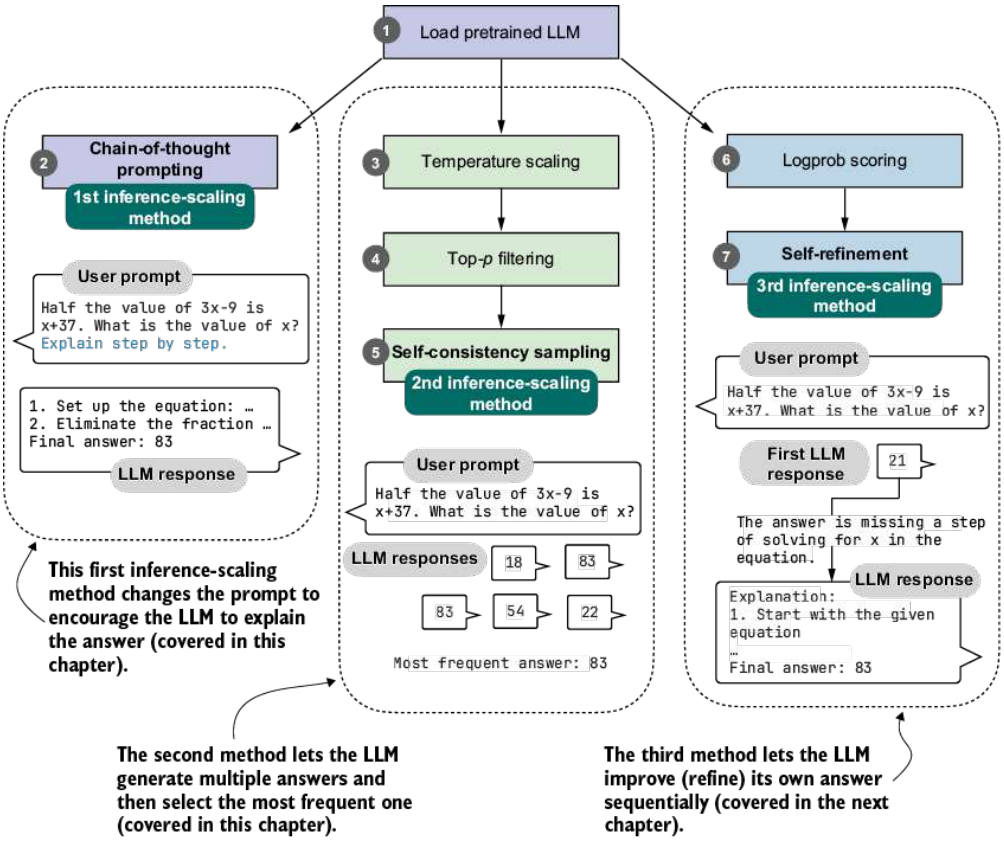


Figure 4.3 Three inference-time methods to improve reasoning. The first modifies the prompt to encourage step-by-step reasoning, and the second samples multiple answers and selects the most frequent one. Those two approaches are discussed in this chapter. The third method, in which the model iteratively refines its own response, is introduced in the next chapter.

Longer responses that include reasoning and explanations (methods 1 and 3) are what we typically expect from reasoning models. Parallel sampling (method 2) is also widely used in production systems, such as Claude 4 (as described in the May 2025 introduction of Claude 4: **https://www.anthropic.com/news/claude-4**).

4.2 Loading a pretrained model

Before we begin implementing inference-time-scaling methods, we'll load the pretrained base model and tokenizer. The following code is similar to the code we used in the previous chapter.

Listing 4.1 Load tokenizer and base model

```
import torch
from reasoning_from_scratch.ch02 import get_device
from reasoning_from_scratch.ch03 import (
     load_model_and_tokenizer
)

device = get_device()
device = torch.device("cpu")  #1

model, tokenizer = load_model_and_tokenizer(
    which_model="base",
    device=device,
    use_compile=False
)
```

#1 Delete this line to run the code on a GPU (if supported by your machine).

Since the code in this chapter is cheap to run, I recommend running it on the `" cpu "` device the first time to get the same results that are shown in this chapter (running it on a GPU can subtly alter the results).

Let's try the model on a prompt from the MATH-500 dataset, which we worked with in the previous chapter:

```
from reasoning_from_scratch.ch03 import render_prompt

raw_prompt = (
    "Half the value of $3x-9$ is $x+37$. "
    "What is the value of $x$?"
)
prompt = render_prompt(raw_prompt)
print(prompt)
```

The formatted prompt is as follows:

```
You are a helpful math assistant.
Answer the question and write the final result on a new line as:
\boxed{ANSWER}

Question:
Half the value of $3x-9$ is $x+37$. What is the value of $x$?

Answer:
```

We can use the preceding prompt as input to the `generate_text_stream_concat` function we defined in the previous chapter.

We want to compare several inference-time-scaling strategies, so we'll make a small modification to the text generation wrapper that will let us swap in different generation functions and settings without changing the surrounding prompt-handling and output code. The changes are highlighted in the code comments in the following listing.

Listing 4.2 Modified `generate_text_stream_concat` function

```
from reasoning_from_scratch.ch02 import generate_text_basic_stream_cache


def generate_text_stream_concat_flex(
    model, tokenizer, prompt, device, max_new_tokens,
    verbose=False,
    generate_func=None,  #1
    **generate_kwargs     #1
):

    if generate_func is None:  #2
        generate_func = generate_text_basic_stream_cache

    input_ids = torch.tensor(
        tokenizer.encode(prompt), device=device
        ).unsqueeze(0)

    generated_ids = []
    for token in generate_func(
        model=model,
        token_ids=input_ids,
        max_new_tokens=max_new_tokens,
        eos_token_id=tokenizer.eos_token_id,
        **generate_kwargs,  #3
    ):
        next_token_id = token.squeeze(0)
        generated_ids.append(next_token_id.item())

        if verbose:
            print(
                tokenizer.decode(next_token_id.tolist()),
                end="",
                flush=True
            )
    return tokenizer.decode(generated_ids)
```

#1 We add parameters to accept a text generation function and additional arguments.
#2 If the text generation function is undefined, we use generate_text_basic_stream_cache similar to chapter 3.
#3 We can pass additional arguments to the text generation function if needed.

In short, the preceding `generate_text_stream_concat_flex` function is similar to the `generate_text_stream_concat` function from the previous chapter, except that we can now pass the text generation function (such as `generate_text_basic_stream_cache`) as an argument instead of hardcoding it. The practical benefit is that the outer wrapper can keep handling the prompt encoding, streaming, and decoding in one place, while the inner generation step can be replaced with different decoding strategies. This makes it easier to compare methods fairly, because we are changing only the generation logic rather than the entire pipeline. In future sections, we'll swap the `generate_text_basic_stream_cache` function for more advanced functions.

The use of the `generate_text_basic_stream_cache` function is also similar to before, except that we can now pass the function explicitly:

```
response = generate_text_stream_concat_flex(
    model, tokenizer, prompt, device,
    max_new_tokens=2048, verbose=True,
    generate_func=generate_text_basic_stream_cache
)
```

This is the generated output:

```
\boxed{20}
```

Note that this answer is wrong; the correct solution is 83. In the remainder of this chapter and the next, we'll implement inference-time-scaling methods to get the model to generate the correct answer.

4.3 Generating better responses with chain-of-thought prompting

We've loaded the pretrained base model and set up the text generation function; now we'll focus on improving model output through so-called *chain-of-thought* prompting. This prompting technique is classic, simple, and effective. It modifies the input prompt to encourage the LLM to generate an explanation, or a chain of thought (also called a reasoning chain), as illustrated in figure 4.4.

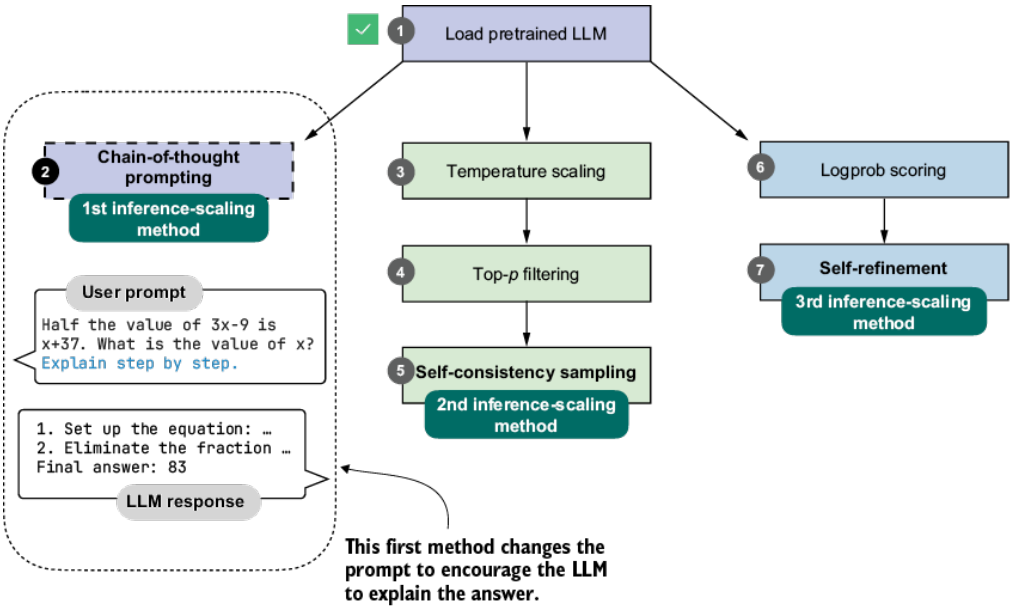


Figure 4.4 Our first inference-time method, chain-of-thought prompting, modifies the prompt to encourage the model to explain its reasoning, step by step, before producing a final answer.

The simplest way to try chain-of-thought prompting is to append an extra instruction that asks the model to reason step by step. There are multiple ways to phrase this. For example, the original paper on the topic used `" Let ' s think step by step. "` (https://arxiv.org/abs/2205.11916), but different phrasings may work better depending on the model and task. Here, we'll use `" Explain step by step. "` because it worked well in my experiments for this chapter, and it keeps the example simple:

```
prompt_cot = prompt + " \n\nExplain step by step."

response_cot = generate_text_stream_concat_flex(
    model, tokenizer, prompt_cot, device,
    max_new_tokens=2048, verbose=True,
)
```

The response is now as follows:

```
 To solve the problem, we need to find the value of \( x \) such that
half the value of \( 3x - 9 \) is equal to \( x + 37 \).

# ...  #1

### Step 3: Solve for \( x \)
Subtract \( 2x \) from both sides to isolate \( x \):
\[
3x - 2x - 9 = 74
\]
Simplify:
\[
x - 9 = 74
\]
Add 9 to both sides to solve for \( x \):
\[
x = 74 + 9
\]
\[
x = 83
\]

### Final Answer:
\[
\boxed{83}
\]
```

#1 The response was truncated to save space.

As you can see, the model now writes a lengthy step-by-step explanation and, in this case, arrives at the correct answer.

This simple chain-of-thought prompt is a good demonstration of the inference-time- scaling tradeoff. While the model now answers correctly, it expends many more tokens than before. As discussed in chapter 2, LLMs generate text one token at a time, and each additional token requires another forward pass through the model. So these intermediate reasoning steps do not just make the answer longer; they also directly increase latency, compute cost, and often API cost.

Note that while the model generated the correct answer in this case, not all problems benefit from chain-of-thought prompting. On simple problems, it can sometimes even degrade the model's performance, as the model might generate erroneous explanations and mislead itself. This phenomenon is known as "overthinking."

Lastly, not every model benefits from the "Explain step by step" (or similar) instruction. In this case, we used a simple base model that doesn't always generate explanations, so chain-of-thought prompting can help. Trained reasoning models, such as the "reasoning" variant of the Qwen3 model we used in the previous chapter, already write explanations alongside their responses and don't need or benefit from this type of chain-of-thought prompting.

WHY CHAIN-OF-THOUGHT CAN IMPROVE ACCURACY

Chain-of-thought prompting asks the model to write out the intermediate steps that lead to a final answer. This helps in two practical ways.

First, walking through the steps gives the model more opportunities to correct itself.

Second, step-by-step reasoning matches the way many training examples are written. For instance, large math and logic datasets often contain detailed solutions, so asking for a chain of thought aligns the model with patterns it has already learned.

At the same time, chain-of-thought reasoning is no guarantee of correctness. It can still produce faulty reasoning, and for very simple problems it may even introduce unnecessary steps that lead to more mistakes. In other words, chain-of-thought can improve accuracy on many reasoning tasks, but it is not universally beneficial.

Overall, chain-of-thought prompting does not provide the model with new knowledge, but it changes how the model uses its existing knowledge. Often, this shift can lead to more reliable answers. This is especially true for math, code, logic, and other multistep problems.

EXERCISE 4.1: USING CHAIN-OF-THOUGHT PROMPTING ON MATH-500

Modify the `evaluate_math500_stream` function from listing 3.15 to see whether chain-of-thought prompting improves the MATH-500 accuracy of the base model.

4.4 Controlling output diversity with temperature scaling

The previous section gave a brief taste of inference-time scaling by extending the model's answer via chain-of-thought prompting. Chain-of-thought prompting can be seen as a sequential technique because it extends the number of next-token prediction steps.

In the remainder of this chapter, we'll implement a self-consistency technique (also called *self-consistency sampling*) that generates multiple answers, as illustrated in figure 4.5. This technique was formally described in the Google Research paper "Self-Consistency Improves Chain of Thought Reasoning in Language Models" (**https://arxiv.org/abs/2203.11171**).

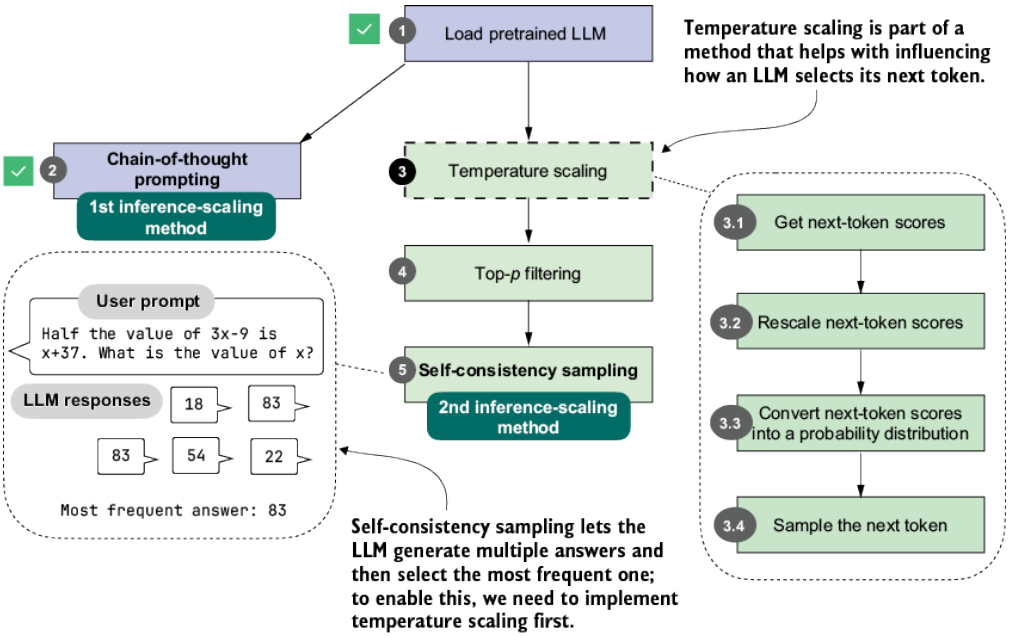


Figure 4.5 Self-consistency sampling, our second inference-time method, generates multiple answers and selects the most frequent one. This method relies on temperature scaling, which influences how the model samples its next token.

Since the multiple answers generated with this technique are independent of each other, this can be implemented through parallel sampling (if you have the necessary resources, such as multiple GPUs), which won't increase the time for the user to get an answer. (Chain-of-thought prompting can be combined with this technique, but more on that later.)

Before we implement self-consistency sampling (step 5 in figure 4.5), we first need to extend the text generation function so that it can produce different answers for the same prompt. To achieve this, we'll implement two techniques that allow the model to sample different responses: *temperature scaling* (step 3) and *top-p filtering* (step 4).

Temperature scaling is the main focus of this section; it will be one of the key controls for output diversity in the rest of this chapter, and it forms the basis for the top-*p* filtering method that follows. First, though, we'll look at how the next token is sampled in an LLM. These low-level sampling controls may seem overly technical, but they are what will let us generate diverse candidate answers for self-consistency sampling.

4.4.1 Understanding the process of selecting the next token

Let's take a closer look at the text generation process we've implemented so far and see how the next-token selection process works under the hood. This will help you understand the motivation behind temperature scaling.

Suppose we have the following simple prompt:

```
ex_prompt = "The capital of Germany is"

response = generate_text_stream_concat_flex(
    model, tokenizer, ex_prompt, device,
    max_new_tokens=1, verbose=True
)
```

The model's response is `" Berlin "`. This looks relatively simple, but multiple steps are happening under the hood, as illustrated in figure 4.6.

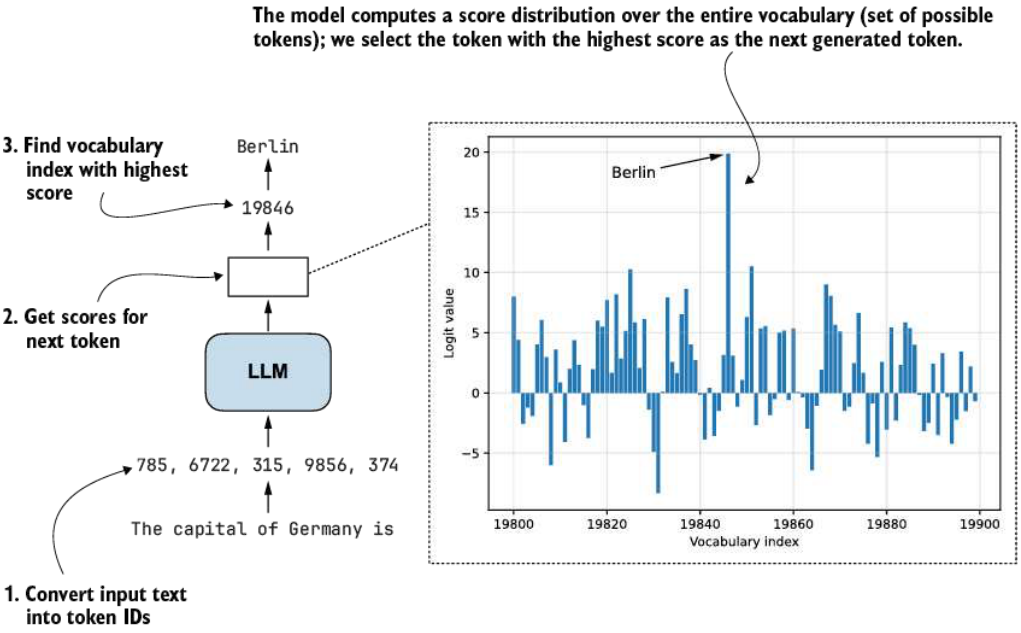


Figure 4.6 How an LLM generates the next token. As in all process diagrams in this book, the flow runs from bottom to top. The model converts the input into token IDs, computes scores for all possible next tokens, and selects the one with the highest score as the next output.

When generating text, the inputs are first converted into token IDs (as discussed in chapter 2):

```
input_token_ids = torch.tensor(
    tokenizer.encode(ex_prompt), device=device
).unsqueeze(0)
print(input_token_ids)
```

In this case, these are the token IDs:

```
tensor([[ 785, 6722,  315, 9856,  374]])
```

In the second step in figure 4.6, we get the scores for the output token we want to generate. These output scores are also called *logits*. An LLM generates one output token for each input token, but we are interested only in the last token, which we select via the `[:, -1]` tensor indexing. This last token corresponds to the token we want to generate:

```
with torch.inference_mode():
    next_token_logits = model(input_token_ids)[:, -1]
print(next_token_logits.shape)
```

The printed output shape is `[1, 151936]`, where `151936` is the vocabulary size for this tokenizer and model. The vocabulary includes all the unique tokens the tokenizer can handle and the LLM can generate.

To obtain the next generated token (`" Berlin "` in this example), we have to find the vocabulary entry associated with the largest score (step 3 in figure 4.6):

```
max_token_id = torch.argmax(next_token_logits)
print(f"Token ID: {max_token_id}")
print(f"Decoded token: '{tokenizer.decode([max_token_id])}'")
```

The output follows:

```
Token ID: 19846
Decoded token: ' Berlin'
```

We've now covered the three main steps in generating the next token. Before we move on to the next section, let's take a closer look at the score distribution of the `next_token_logits` tensor we passed to the `torch.argmax` function to obtain the token IDs. We'll plot it in Matplotlib.

Listing 4.3 Plotting the next-token logit scores

```
import matplotlib.pyplot as plt

def plot_scores_bar(
    next_token_logits, start=19_800, end=19_900,
    arrow=True, ylabel="Logit value"
):
    x = torch.arange(start, end)  #1
    logits_section = next_token_logits[0, start:end].float().cpu() #2

    plt.bar(x, logits_section)  #3
    plt.xlabel("Vocabulary index")
    plt.ylabel(ylabel)

    if arrow:  #4
        max_idx = torch.argmax(logits_section)
        plt.annotate(
            "Berlin",
            xy=(x[max_idx], logits_section[max_idx]),
            xytext=(x[max_idx] - 25, logits_section[max_idx] - 2),
            arrowprops={
                "facecolor": "black", "arrowstyle": "->", "lw": 1.5
            },
            fontsize=10,
        )

    plt.grid(alpha=0.3)
    plt.tight_layout()
    plt.show()

plot_scores_bar(next_token_logits)
```

#1 Select the vocabulary subsection.

#2 .cpu() is a shortcut for to(torch.device("cpu")).

#3 Plot the logits (scores) within the selected range.

#4 Draw an arrow to highlight the largest score.

We are restricting the plot to the 100 vocabulary index tokens between 19,800 and 19,900, rather than plotting the scores for all 151,936 entries, which would make the plot too crowded. I selected this range because it contains the entry with the highest score. The resulting plot is shown in figure 4.7.

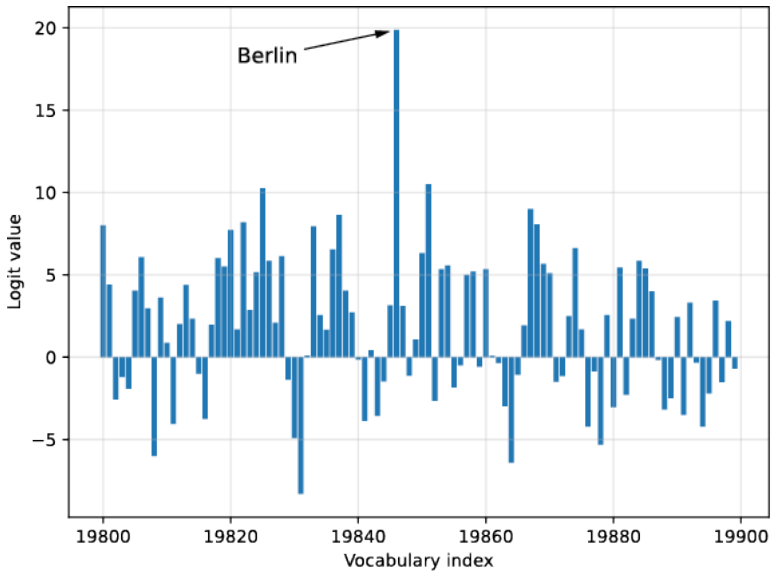


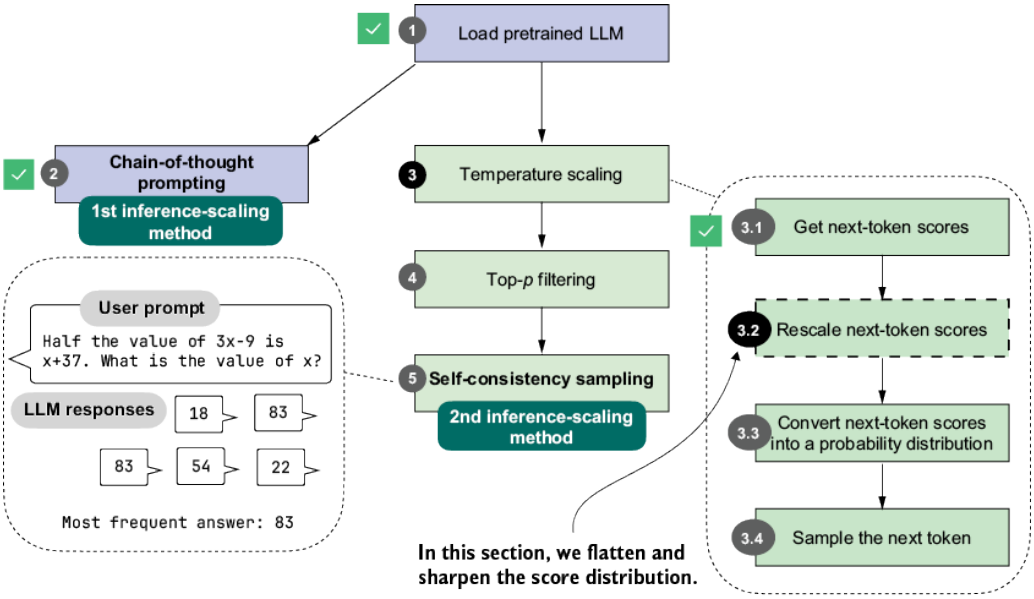
Figure 4.7 Next-token logits for a 100-token slice of a language model's much larger vocabulary. Each bar represents one possible token's score within this slice, with "Berlin" having the highest logit value and being selected as the next token.

The plot in figure 4.7 shows the 100 logit values (scores) for vocabulary indices 19,800–19,899. The values range approximately from –8 to 20, and the value 20 corresponds to the vocabulary index for the token `" Berlin"`.

4.4.2 Rescaling token scores (logits) via a temperature parameter

Now that you know how the model selects its next token, we can introduce the concept of temperature. Temperature, or, more precisely, the temperature parameter, rescales the logits, which makes the corresponding probability distribution sharper or flatter and thereby affects how the next token is selected.

As shown in figure 4.8, this section focuses on rescaling the next-token logits with a temperature parameter before using them for sampling. Rescaling here means adjusting the magnitude of the scores so the sampling step becomes more or less sensitive to differences in the scores.



In this section, we'll implement the core part of temperature scaling (step 3.2), which adjusts the next-token scores. This lets us control how confidently the model selects its next token in later steps.

The code for implementing the temperature rescaling step (step 3.2 in figure 4.8), is relatively short and simple. In essence, the following code rescales the logit values before converting them to probabilities (discussed in the next section).

Listing 4.4 Rescaling next-token scores via temperature scaling

```
def scale_logits_by_temperature(logits, temperature):
    if temperature <= 0:
        raise ValueError("Temperature must be positive")
    return logits / temperature
```

In practice, temperature values are expected to be positive numbers, so we raise an error if the temperature is zero or less. We will add more temperature-scaling safeguards later, when we add temperature scaling to our text generation function.

For all other values, the logits are divided by the temperature. A temperature of 1.0 means no change, because dividing a number by 1 leaves it unchanged. A temperature lower than 1.0 makes the distribution sharper, which makes the model more confident when it selects the next token. Temperatures higher than 1.0 flatten the logit distribution, which can make sampling (step 3.4 in figure 4.8) more diverse. In other words, higher temperatures reduce the probability gap between the top token and lower-ranked tokens, increasing the chance that sampling will pick a non-maximum token.

Let's see this `scale_logits_by_temperature` function in action. We'll try it out with relatively extreme temperature values of 0.5 and 5.0 for a stronger effect.

Listing 4.5 Plotting the temperature-rescaled logits

```
def plot_logits_with_temperature(
    next_token_logits, start=19_800, end=19_900,
    temps=(0.5, 5.0),
):
    x = torch.arange(start, end)
    logits_orig = next_token_logits[0, start:end].float().cpu()

    logits_scaled = [   #1
        scale_logits_by_temperature(logits_orig, T) for T in temps  #2
    ]   #2

    plt.plot(x, logits_orig, label="Original logits", lw=2)  #2
    plt.plot(
        x, logits_scaled[0],
        label=f"T={temps[0]} (sharper)", ls="--", lw=1       #3
    )
    plt.plot(
        x, logits_scaled[1],
        label=f"T={temps[1]} (flatter)", ls=":", lw=3
    )

    # Highlight max logit
    max_idx = torch.argmax(logits_orig)                     #4
    plt.annotate(                                            #4
        "Berlin",
        xy=(x[max_idx], logits_orig[max_idx]),
        xytext=(x[max_idx] - 25, logits_orig[max_idx] + 2),
        arrowprops={"facecolor": "black", "arrowstyle": "->", "lw": 1.5},
        fontsize=12,
    )

    plt.xlabel("Vocabulary index")
    plt.ylabel("Logit value")
    plt.legend()
    plt.grid(alpha=0.3)
    plt.tight_layout()
    plt.show()

plot_logits_with_temperature(
    next_token_logits,
    temps=(0.5, 5.0)                                        #5
)
```

#1 Apply temperature scaling.

#2 Plot the logits (scores) within the selected range.

#3 Plot the logits (scores) within the selected range.

#4 Draw an arrow to highlight the largest score.

#5 Run the rescaling and plotting with temperatures 0.5 and 5.

As shown in figure 4.9, the larger temperature (5.0) yields a much flatter score distribution, while the smaller temperature (0.5) yields a much sharper one.

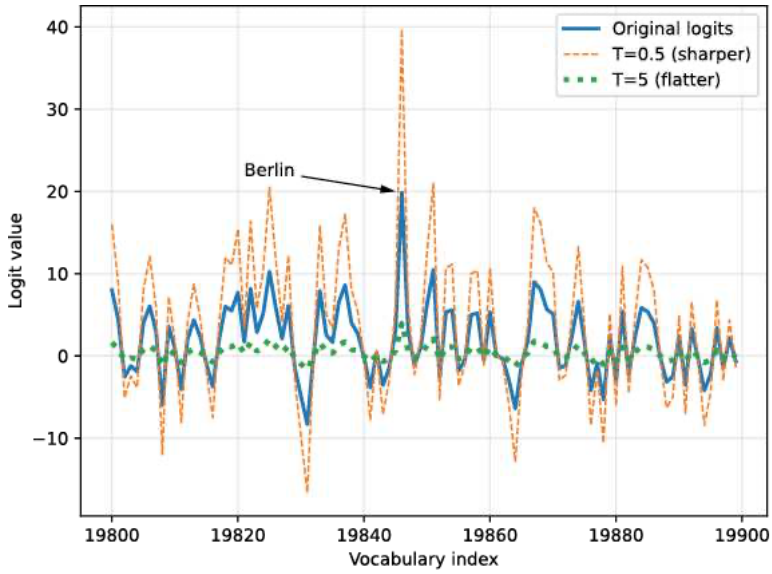


Figure 4.9 The effect of temperature scaling on logits. Lower temperatures make the distribution sharper, while higher temperatures flatten it. (Note that this visualization is shown as a line plot for readability; a bar plot would more accurately represent the individual vocabulary scores.)

The plotting code in listing 4.5 is very similar to the plotting code in listing 4.3. In addition to applying temperature scaling, we are now using a line plot (`plt.plot`) instead of a bar plot (`plt.bar`). Although a bar plot is technically a better choice for the discrete vocabulary indices on the *x*-axis, a line plot makes it easier to visualize and compare the rescaled logits in this case.

The important point is not the absolute height of a logit by itself, but how the differences between logits change. If a token such as `" Berlin "` stands out more strongly than the alternatives, it is more likely to be selected later. If the differences shrink, lower-ranked tokens become more likely to be selected. The next section will illustrate this by converting the logits into probabilities.

WHY TEMPERATURE?

The term *temperature* comes from physics, where temperature controls how much randomness or movement there is in a system. In LLMs, we use the same idea to control how confidently or creatively the model chooses its next token, as you'll see in the following sections.

4.4.3 Sampling the next token from a probability distribution

In the previous section, we rescaled the logit values with different temperature values. This lets us influence how the model selects the next token. Before we get to the next-token sampling portion in the next section, we'll add one more intermediate step: converting the rescaled logits into probability scores, as shown in figure 4.10.

To demonstrate how rescaled logits are converted into probability scores (step 3.3 of figure 4.10), we'll use a temperature of 5.0, which will make it easier to visualize the resulting probabilities in a plot. The `" Berlin "` token, for example, has such a high logit value that it would otherwise dominate the scale, making it difficult to see the probabilities of the surrounding tokens.

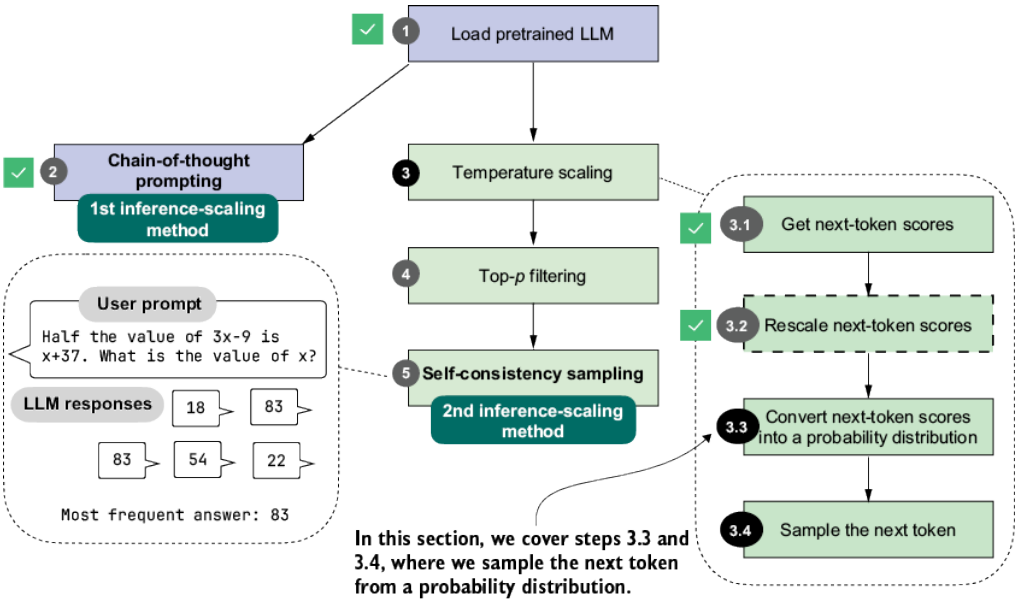


Figure 4.10 Overview of the sampling process for generating tokens. Here, we focus on steps 3.3 and 3.4, where the next-token scores are converted into a probability distribution. The next token will be sampled based on that distribution.

The conversion from rescaled logits to probability scores can be done with a single function call, `torch.softmax`, as shown in the following listing.

Listing 4.6 Sampling the next token from a probability distribution

```
rescaled_logits = scale_logits_by_temperature(next_token_logits, 5.0)  #1

next_token_probas = torch.softmax(  #2
    rescaled_logits, dim=-1
)
```

#1 Step 3.2: Rescale next-token scores

#2 Step 3.3: Convert rescaled logits into probability scores

This `torch.softmax` function normalizes the logit values to the range from 0 to 1, such that they sum to 1. We can confirm that with the following code:

```
print("Probability sum:", torch.sum(next_token_probas))
```

Additionally, let's visualize the converted scores by reusing the `plot_scores_bar` function from listing 4.3:

```
plot_scores_bar(
    next_token_probas, arrow=False, ylabel="Probability value"
)
```

The resulting plot, now with the probability values on the *y*-axis, is shown in figure 4.11, where we can see that token ID 19,846 (`" Berlin "`) has the highest value in this selected vocabulary range. The probability is 0.0003, which we can confirm with the following code:

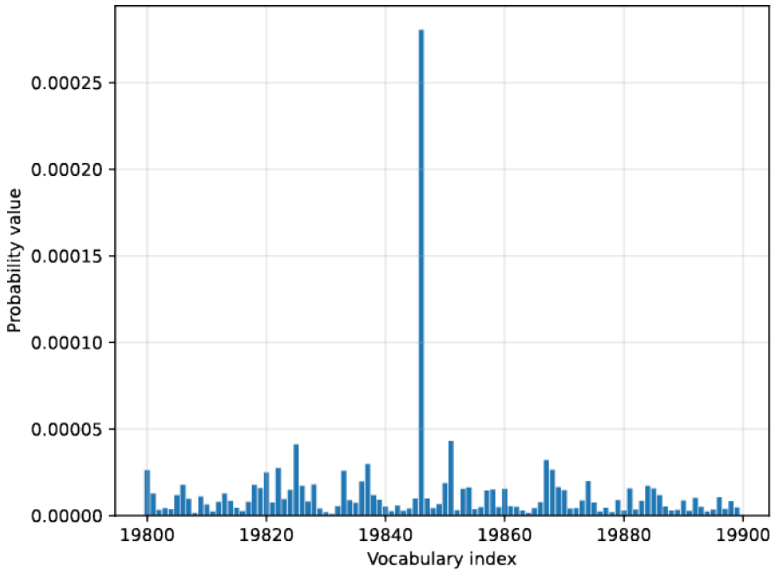


Figure 4.11 Token probabilities obtained by applying the softmax function to the rescaled logits. The token with the highest probability (corresponding to " Berlin", but with the label omitted for code simplicity) is selected as the next output.

```
print("Token ID 19,846 probability:", next_token_probas[:, 19846])
```

Note that while this 0.0003 value is relatively small, there is no larger value outside the selected vocabulary range in this plot. For instance, using the following code, we can confirm that 0.0003 is indeed the largest value:

```
print("Highest probability:", max(next_token_probas.squeeze(0)))
```

We can interpret this score as the model's confidence. This means that the model is more confident in choosing `" Berlin "` (19,846) as the next token than any other token.

The reason the value is so small is that we are using a high temperature, which makes it easier to plot this value next to the other token values. If we change the temperature from 5 to 0.5, the probability score increases from 0.0003 to 0.3398, and the probability scores of the other tokens become even closer to 0.

THE SOFTMAX FUNCTION UNDER THE HOOD

The softmax function converts a vector of raw scores (logits) into a probability distribution where each value lies between 0 and 1 and all values sum to 1. This conversion makes the values easier to interpret and to sample from later.

If you are familiar with mathematical notation, the formula behind the softmax function is

$\text{softmax}(z_i) = \exp(z_i) / \Sigma_j \exp(z_j)$

Here, z is a vector of real-valued inputs

$z = [z_1, z_2, \ldots, z_n]$,

where

- n is the number of elements in the vector.
- i is the index of the current element ($1 \le i \le n$).
- j is the index used to sum over all elements ($1 \le j \le n$).

This produces a normalized probability for each element z_i, such that

$\Sigma_i \text{softmax}(z_i) = 1$

In code, the softmax function can be implemented in three lines:

```
def softmax_with_temperature(logits, temperature):
    scaled_logits = logits / temperature
    return torch.softmax(scaled_logits, dim=0)
```

In practice, I prefer to use the `torch.softmax` function because it has some additional numerical stability improvements to handle very small and very large values more reliably.

The purpose of converting the logits into these probability scores is that they are somewhat more interpretable, and we can now sample from them using a `torch.multinomial` function. For instance, if we draw a sample from this probability distribution, we have a 0.03% chance of getting `" Berlin "` with our temperature setting of 5.0 (and a 33.98% chance of getting `" Berlin "` with a temperature of 0.5).

The sampling process, corresponding to step 3.4 in figure 4.10, can be implemented as follows:

```
torch.manual_seed(123)
print(
    "Sampled token:",
    torch.multinomial(next_token_probas.cpu(), num_samples=1)
)
```

This code returns token ID 65,094, which corresponds to the word `" mistress "`. Note that the word doesn't make sense in our context, `" The capital of Germany is "`. In this case, it was selected randomly because of the high-temperature setting, which encourages sampling tokens other than `" Berlin "`.

The `torch.multinomial` function samples vocabulary indices in proportion to their probabilities. In other words, vocabulary indices with higher probabilities are more likely to be sampled. If we repeated the sampling a very large number of times, we would sample the vocabulary index corresponding to the token `" Berlin "` with a probability of 0.03%, given a temperature of 5.

Before we look at some additional examples, note that we specified a random seed to make the code in this chapter reproducible. The `torch.multinomial()` function may still yield different results on CUDA and MPS devices and may even crash when drawing larger numbers of samples (I observed this issue in PyTorch 2.9 on both CUDA and MPS devices), which is why we are running the sampling on the CPU via `.cpu()`.

Let's now sample some more next-token candidates to get a more representative sample.

Listing 4.7 Sampling multiple next-token candidates

```
def count_samples(probas, num_samples=1000, threshold=1, tokenizer=None):
    samples = torch.multinomial(  #1
        probas.cpu(), num_samples=num_samples, replacement=True
    )
    counts = torch.bincount(samples.squeeze(0), minlength=1)  #2

    for i, c in enumerate(counts):
        if c > threshold:   #3
            if tokenizer is None:
                print(f"Vocab index {i}: {c.item()}x")
            else:
                print(f"'{tokenizer.decode([i])}': {c.item()}x")
```

#1 Draw samples according to probabilities.
#2 Count how often each index was selected.
#3 Print frequently sampled vocabulary indices (entries).

This `count_samples` function in listing 4.7 samples token indices from a probability distribution and counts how often each token is drawn. It uses `torch.multinomial` to randomly select `num_samples` indices in proportion to their probabilities. The `replacement=True` setting allows us to draw the same

token multiple times.

Then, `torch.bincount` counts how often each index appears. It prints only tokens that occur more than a specified threshold so it doesn't clutter the output with less frequently drawn tokens.

MULTINOMIAL SAMPLING

Multinomial sampling is the procedure used to pick the next token, given a probability distribution over the vocabulary, such as the softmax probability scores. In multinomial sampling, instead of always selecting the most likely token (known as *greedy decoding*), we draw one token at random, with tokens of higher probability being more likely to be selected. This randomness is important for generating diverse responses, which we'll later use in self-consistency sampling.

To illustrate this further, we can think of the probability scores as a set of weighted choices. For example, a token with probability 0.40 is four times as likely to appear as one with probability 0.10, but both remain possible outcomes.

Suppose the model assigns the following probabilities:

- "Berlin": 0.70
- "Munich": 0.20
- "Hamburg": 0.10

Greedy decoding would always return "Berlin." Multinomial sampling instead draws one token according to these weights. Across a very large number of draws, "Berlin" will appear most often (70% of the time), "Munich" sometimes (20% of the time), and "Hamburg" only occasionally (10% of the time).

This variability is what allows us to generate multiple candidate answers for self-consistency in later sections.

NOTE This `count_samples` function is meant for illustration only. In real text generation, we only draw one token at a time. This function takes a large number of samples from the distribution to show how often each token is selected, making the underlying probabilities easier to visualize and understand.

First, let's run the `count_samples` function on the probability scores that we obtained by applying a temperature of 5:

```
torch.manual_seed(123)
count_samples(next_token_probas, tokenizer=tokenizer)
```

The output is as follows:

```
'}': 2x
' </': 2x
' represent': 2x
' Inf': 2x
'()*': 2x
' beside': 2x
' Kob': 2x
'⍰': 2x
```

As you can see, even though the default sample size is 1,000, none of the sampled tokens appear more than twice. Also, all of these are nonsense tokens in the context of the `" The capital of Germany is "` query. The reason for these nonsensical results is that we used a temperature value that is much too high.

Let's try a lower temperature value of 0.35, which sharpens the score distribution and makes it more likely to select a meaningful next token:

```
torch.manual_seed(123)
probas_lowT = torch.softmax(
    scale_logits_by_temperature(next_token_logits, 0.35), dim=-1
)
```

In this case, we see the following output:

```
' __': 158x
' Berlin': 435x
' ____': 169x
' _______': 209x
' Munich': 3x
' Hamburg': 3x
' _____': 18x
```

This output makes much more sense as a set of next-token candidates for our `" The capital of Germany is "` query. Out of the 1,000 samples, the `"  Berlin "` token was drawn 435 times. You can check via `print(probas_lowT[0, 19_846])` that the probability of drawing this token is approximately 42% at the temperature value 0.35. To make it even more likely that `"  Berlin "` will be sampled, we could further reduce the temperature.

Notice that some of the other, rarer candidates also make sense. For example, both `"  Munich "` and `" Hamburg "` are big cities in Germany, so they are not completely unrelated to the query. The underscore responses (`"  _____ "`) are likely due to the model having seen text in the form of a quiz with a placeholder; e.g., `" The capital of Germany is _____ "`.

You may wonder, if `"  Berlin "` is the correct answer, what's the point of making the model occasionally give the wrong answer by adding this temperature rescaling and multinomial sampling? For different kinds of queries, introducing randomness during sampling helps the model explore alternative responses instead of always choosing the single most likely token. This variability can be useful for creative or open-ended tasks, where there may be multiple valid completions.

Specifically, in reasoning tasks, we can use this sampling diversity through techniques such as self-consistency (discussed in section 4.6), which generate multiple candidate answers and compare them to improve answer accuracy.

4.4.4 Adding temperature scaling to the text generation function

Before we introduce another improvement to the text generation process by adding a token-probability selection filter, let's add the temperature-scaling modification to the text generation function so we can more readily generate new tokens with the model.

Listing 4.8 Text generation with temperature scaling

```
from reasoning_from_scratch.qwen3 import KVCache

@torch.inference_mode()
def generate_text_temp_stream_cache(
    model,
    token_ids,
    max_new_tokens,
    eos_token_id=None,
    temperature=0.
):
    model.eval()
    cache = KVCache(n_layers=model.cfg["n_layers"])
    model.reset_kv_cache()

    out = model(token_ids, cache=cache)[:, -1]  #1
    for _ in range(max_new_tokens):

        ######################################
        # NEW:
        orig_device = token_ids.device

        if temperature is None or temperature == 0.0:
            next_token = torch.argmax(out, dim=-1, keepdim=True)

        else:
            logits = scale_logits_by_temperature(out, temperature)  #2
            probas = torch.softmax(logits, dim=-1)  #3
            next_token = torch.multinomial(probas.cpu(), num_samples=1)  #4
            next_token = next_token.to(orig_device)

        ######################################
        if (eos_token_id is not None
                and torch.all(next_token == eos_token_id)):
            break

        yield next_token
        out = model(next_token, cache=cache)[:, -1]
```

#1 Step 3.1: Get logits
#2 Step 3.2: Apply temperature scaling on logits
#3 Step 3.3: Convert to probabilities
#4 Step 3.4: Sample token according to probabilities

The `generate_text_temp_stream_cache` in listing 4.8 is similar to the `generate_text_stream_cache` from chapter 2. What's new is that it now includes temperature rescaling and sampling (below the `# New` comment indicator in the code).

The following code shows how temperature scaling and sampling are integrated into the token-generation loop itself. This function can be passed into the more flexible `generate_text_stream_concat_flex` wrapper introduced in listing 4.2, which keeps the outer prompt-handling code unchanged while letting us swap in different decoding strategies:

```
torch.manual_seed(123)
response = generate_text_stream_concat_flex(
    model, tokenizer, prompt, device,
    max_new_tokens=2048, verbose=True,
    generate_func=generate_text_temp_stream_cache,  #1
    temperature=1.1
)
```

#1 Use the new temperature scaling-based text generation function.

The output is `\boxed{$x = \frac{90}{7}$}`. The correct answer is 83, so the model is still wrong, but this was meant as a demonstration to show that we can tweak the answer by using temperature scaling

and sampling.

CHOOSING TEMPERATURE SETTINGS

In practice, temperature selection depends on the goal. A temperature of 0.0 corresponds to greedy decoding, where we always pick the highest-probability token. Small nonzero values such as 0.3–0.8 are often useful when we want a bit more diversity without making the output too erratic. Much higher values make the model explore more broadly, which can be useful for creative generation or broad search, but often hurts reliability on tasks where we want the single most likely answer.

In the next section, we'll improve the sampling process.

4.5 Balancing diversity and coherence with top-p sampling

You've seen how temperature scaling and sampling (via `torch.multinomial`) can increase the diversity of the LLM responses, for better or for worse. Specifically, you saw that this approach may end up sampling tokens that are unrelated to the user query.

We'll now improve the sampling process by adding a top-*p* filter (figure 4.12) so that very low-confidence tokens are not sampled by accident. The top-*p* sampling process described in this section is also known as *nucleus sampling*.

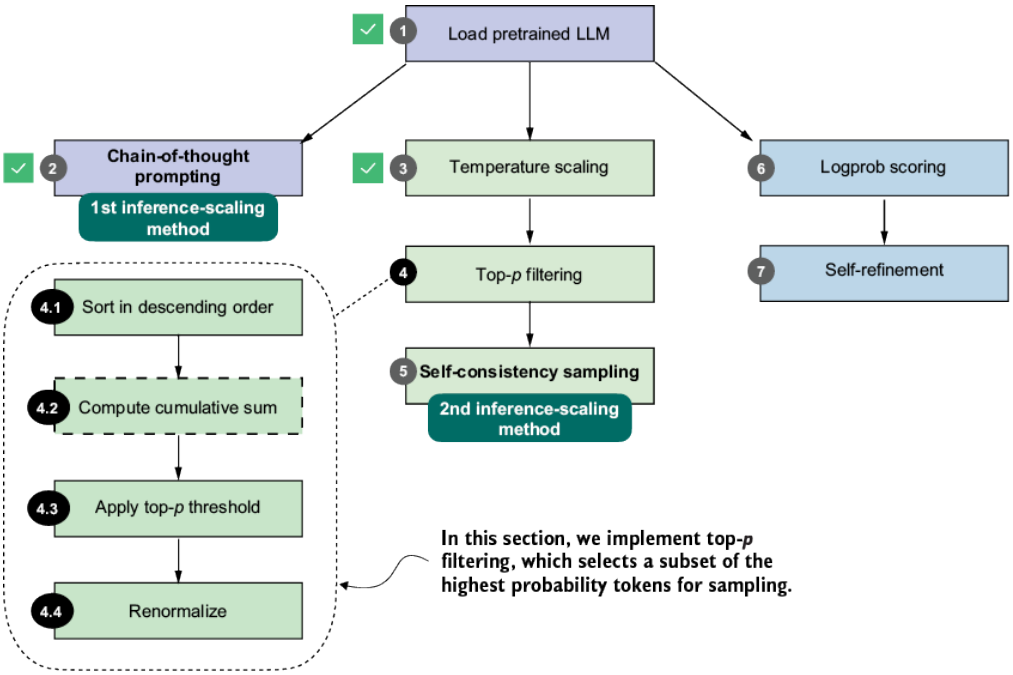


Figure 4.12 Overview of the top-*p* filtering process. The filter keeps only the highest-probability tokens by sorting them, applying a cumulative cutoff, selecting the top-*p* subset, and renormalizing the result.

Figure 4.12 summarizes the four steps that make up the top-*p* filter: sorting the token probabilities, computing their cumulative sum, selecting the subset that satisfies the top-*p* cutoff, and renormalizing the remaining probabilities so that they again form a valid distribution. The renormalization step is necessary, because once we remove all low-probability tokens, the remaining probabilities no longer sum to one. To sample correctly, we need to rescale the remaining values so that they represent a proper probability distribution.

We'll now walk through steps 4.1 through 4.4 from figure 4.12 in detail.

4.5.1 Selecting a subset of top-p tokens

Before we implement the top-*p* filter, let's introduce a simple toy dataset that will make the temperature-scaling and -sampling process easier to illustrate. To start, let's assume the model's and tokenizer's vocabularies have only 10 entries, rather than 151,936.

Listing 4.9 Temperature scaling and sampling with toy data

```
toy_logits = torch.tensor(  #1
    [-0.7, -3.0, 0.1, -1.2, 2.0, -1.0, -0.5, -2.0, 0.3, 1.5]
)

toy_logits_scaled = scale_logits_by_temperature(toy_logits, 1.0)  #2
toy_probas = torch.softmax(toy_logits_scaled, dim=-1)   #3

plt.bar(  #4
    torch.arange(len(toy_probas)), toy_probas,
    alpha=0.5
)

plt.ylim([0, 1])
plt.xlabel("Vocabulary index")
plt.ylabel("Probability")
plt.show()
```

#1 Step 3.1: Get logits (here: use toy logits for 10 tokens)

#2 Step 3.2: Apply temperature scaling (we'll use 1.0 as a placeholder)

#3 Step 3.3: Convert to probabilities

#4 Plot probabilities in a bar plot

Note that the `toy_logits` variable holds example values for the next-token logit scores from the `model(token_ids, cache=cache)[:, -1]` call in listing 4.8, assuming that the vocabulary size is 10.

The resulting bar plot in figure 4.13 shows the next-token logit scores after they are rescaled to probabilities.

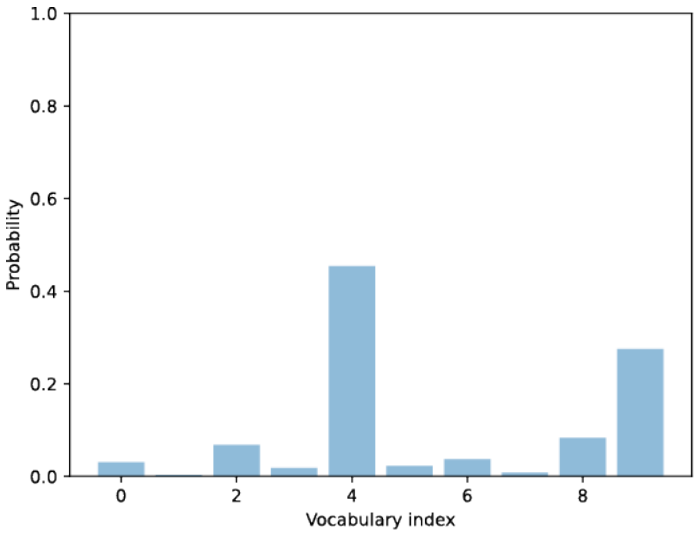


Figure 4.13 Example of token probabilities before top-p filtering. The distribution includes many low-probability tokens, which will later be truncated by applying a cumulative probability threshold.

So far, this is basically a recap of the previous section. Now we'll add the first two top-*p* filtering steps (steps 4.1 and 4.2) illustrated in figure 4.12. This involves sorting the probability scores in descending order and computing the cumulative sum.

Listing 4.10 Computing the cumulative probability sum

```
sorted_probas, sorted_idx = torch.sort(toy_probas, descending=True)  #1
cumsum = torch.cumsum(sorted_probas, dim=-1)  #2

plt.bar(
    torch.arange(len(sorted_probas)), sorted_probas,
        alpha=0.5
)
plt.step(
    torch.arange(len(cumsum)), cumsum,
    where="mid", color="C1", label="Cumulative sum"
)

plt.ylim([0, 1])
plt.xlabel("Token rank (sorted by probability)")
plt.ylabel("Probability")
plt.show()
```

#1 Step 4.1: Sort by descending probability

#2 Step 4.2: Compute cumulative sum

The `torch.cumsum` function used in listing 4.10 computes the cumulative sum of elements along a given dimension. In this example, it takes the sorted token probabilities and adds them up step by step, so that each position in the result represents the total probability accumulated up to that token.

For instance, the first element equals the highest probability, the second equals the sum of the top two, and so on, until the final value reaches 1. This is best explained by looking at the cumulative step plot produced by the code in listing 4.10 and shown in figure 4.14.

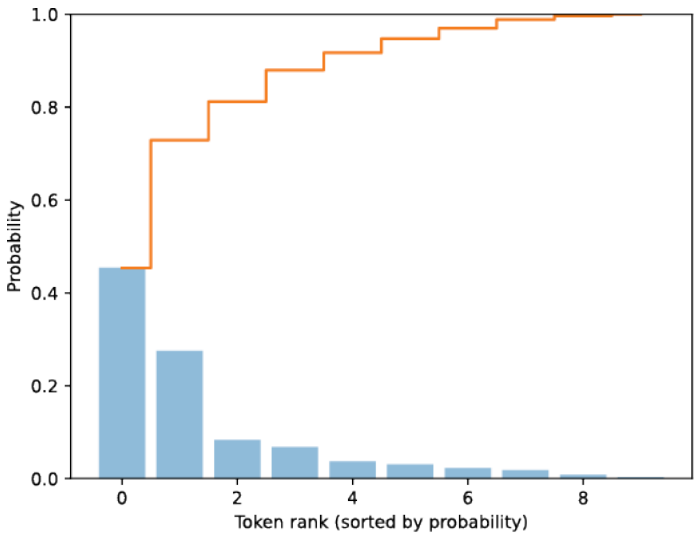


Figure 4.14 Visualization of sorted token probabilities and their cumulative sum. This step prepares for top-p filtering by showing how probabilities accumulate when ordered from highest to lowest, which helps determine where to set the cutoff threshold.

Now that we have the cumulative probability sum, we can implement the core top-*p* filtering step. The *p* in top-*p* stands for *probability*, and top-*p* can be translated as "keep the smallest set of tokens whose cumulative probability stays below or equal to *p*." The following listing shows a simple implementation of top-*p* filtering.

Listing 4.11 A simple-top-*p* filtering rule

```
top_p = 0.8
keep_mask = cumsum <= top_p
n_kept = torch.sum(keep_mask).item()
print("Cumulative sum:", cumsum)
print("Tokens kept:", n_kept)
```

In code, this is implemented as `keep_mask = cumsum <= top_p`, which marks all tokens whose cumulative probability mass does not yet exceed the threshold *p* (here defined as `top_p=0.8`). The num-

ber of retained tokens is then computed and assigned to the variable `n_kept`. The output follows:

```
Cumulative sum: tensor([0.4538, 0.7290, 0.8119, 0.8798,
0.9170, 0.9475, 0.9701, 0.9886, 0.9969, 1.0000])
Tokens kept: 2
```

Looking at the returned cumulative probabilities, the second entry (`0.7290`) is just below the top-*p* threshold of `0.8`, and the third entry (`0.8119`) exceeds it, which is why only the first two tokens are kept.

A more common variant of top-*p* filtering includes the token that exceeds the threshold, as shown in the next listing.

Listing 4.12 A common top-*p* filtering variant

```
keep_mask = (cumsum - sorted_probas) < top_p
n_kept = keep_mask.sum().item()
print("Tokens kept:", n_kept)
```

This code returns 3 as the number of tokens kept. This follows the definition of top-*p* filtering: the smallest set of tokens is kept such that the cumulative probability mass is at least *p*.

Let's illustrate this top filtering with a plot.

Listing 4.13 Visualizing top-*p* filtering

```
plt.bar(
    torch.arange(len(sorted_probas)), sorted_probas,
    alpha=0.5, label="Sorted probabilities"
)
plt.step(
    torch.arange(len(cumsum)), cumsum, where="mid",
    color="darkorange", label="Cumulative sum"
)

plt.axhline(#1
    top_p, color="red", linestyle="--",
    label=f"top_p = {top_p}"
)
plt.axvline(
    n_kept - 0.5, color="gray", linestyle=":",
    label=f"Top-p cutoff at {n_kept} tokens"
)

plt.xlabel("Token rank (sorted by probability)")
plt.ylabel("Probability")
plt.legend()
plt.grid(alpha=0.3)
plt.ylim(0, 1.05)
plt.show()
```

#1 Highlight cutoff

The plot produced by executing the code in listing 4.13 is shown in figure 4.15. This plot shows the `top_p = 0.8` threshold value (the dashed horizontal line) that defines which tokens are kept. In this case, the cumulative sum of the first two tokens is below the threshold, so we exclude all other tokens to the right of the cutoff (the vertical dotted line).

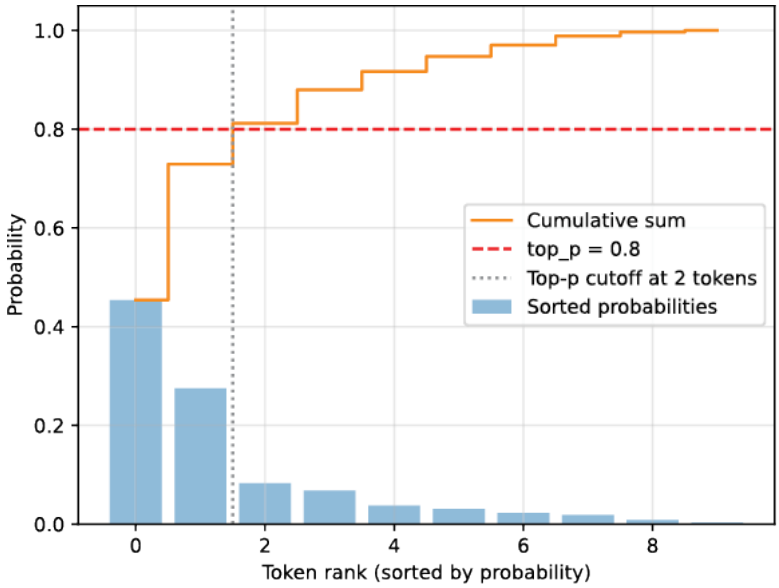


Figure 4.15 Top-p (nucleus) filtering. Tokens are sorted by probability, and the smallest subset whose cumulative probability exceeds the threshold (p = 0.8) is kept for sampling.

To implement this threshold cutoff, we can use the following code, which first zeroes out all values to the right of the cutoff (the vertical dashed line in figure 4.15) and then restores the original sorting order.

Listing 4.14 Applying top-*p* filtering

```
kept_sorted = torch.where(
    keep_mask, sorted_probas,
    torch.zeros_like(sorted_probas)
)
filtered = torch.zeros_like(toy_probas).scatter(0, sorted_idx, kept_sorted)
print(filtered)
```

The resulting tensor follows:

```
tensor([0.0000, 0.0000, 0.0000, 0.0000, 0.4538, 0.0000, 0.0000, 0.0000, 0.0829, 0.2752])
```

You can see that all values except the ones at index positions 4, 8, and 9 are zeroed out. This means that if we now use the multinomial function, only those three tokens will be considered.

Finally, we renormalize these values so that they sum to one again:

```
denom = torch.sum(filtered).clamp_min(1e-12)
renormalized = filtered / denom
print(renormalized)
```

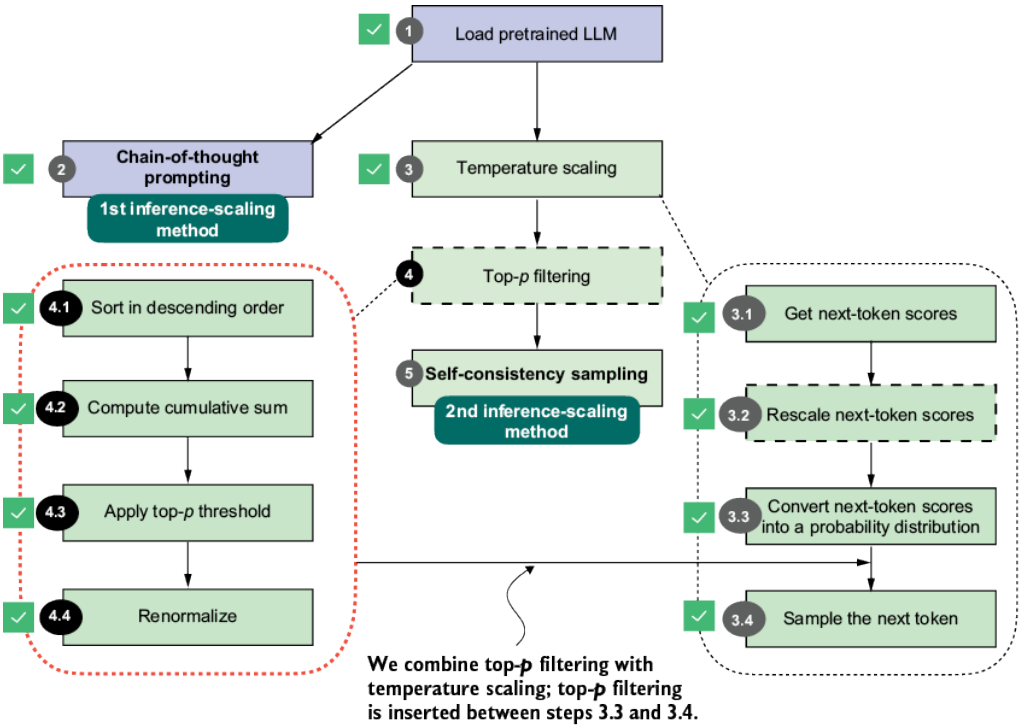
The resulting normalized tensor is as follows:

```
tensor([0.0000, 0.0000, 0.0000, 0.0000, 0.5589, 0.0000, 0.0000, 0.0000,
0.1021, 0.3390])
```

The goal of this top-*p* filtering is to remove tokens with low probabilities so they aren't sampled later. This helps reduce nonsensical token responses in a given context.

4.5.2 Adding a top-p filter to the text generation function

We've walked through the top-*p* filtering steps using a toy example. Now let's add the four top-*p* filtering steps (steps 4.1–4.4 in figure 4.16) to our existing text generation function between the probability conversion and the sampling.



Integrating top-p filtering with temperature scaling. After rescaling the next-token scores, top-p filtering is applied between steps 3.3 and 3.4 to limit sampling to the most probable tokens.

We'll first assemble the top-*p* filtering steps from the previous section into a single, convenient function we can call.

Listing 4.15 Top-*p* filtering function

```
def top_p_filter(probas, top_p):
    if top_p is None or top_p >= 1.0:
        return probas

    sorted_probas, sorted_idx = torch.sort(probas, dim=1, descending=True) #1
    cumprobas = torch.cumsum(sorted_probas, dim=1)  #2

    prefix = cumprobas - sorted_probas  #3
    keep = prefix < top_p                #3
    keep[:, 0] = True          #4

    kept_sorted = torch.where(          #5
        keep, sorted_probas,             #5
        torch.zeros_like(sorted_probas)  #5
    )

    filtered = torch.zeros_like(probas).scatter(1, sorted_idx, kept_sorted)#6

    denom = torch.sum(filtered, dim=1, keepdim=True).clamp_min(1e-12)#7
    return filtered / denom
```

#1 Step 4.1: Sort by descending probability
#2 Step 4.2: Cumulative sum
#3 Step 4.3.1: Keep tokens where prefix cumulative mass (before each token) is < top_p
#4 For top_p <= 0, only the highest-probability token is guaranteed to be kept as a fallback.
#5 Step 4.3.2: Zero out beyond cutoff
#6 Step 4.3.3: Map back to original order
#7 Step 4.4: Renormalize to sum to 1

The `top_p_filter` function in listing 4.15 first sorts token probabilities and computes their cumulative sum. It then keeps tokens whose prefix cumulative mass (the cumulative probability before each token) is below the `top_p` threshold, which includes the token that first crosses the threshold. It zeroes out the rest, maps the kept values back to their original order, and renormalizes the remaining probabilities so they sum to 1 again.

Before we add this `top_p_filter` function to our text generation functions, let's give it a try and see

how it works with the previous temperature-scaling approach. First, we get the logits:

```
with torch.inference_mode():
    next_token_logits = model(input_token_ids)[:, -1]
print(next_token_logits.shape)
```

The preceding code prints the dimensions `[1, 151936]` for the `next_token_logits` tensor. We are now working with the real data and full vocabulary, which has 151,936 entries.

Next, we rescale the logits into probability scores and apply temperature scaling with a temperature of 0.35, as before:

```
torch.manual_seed(123)
probas_lowT = torch.softmax(
    scale_logits_by_temperature(next_token_logits, 0.35), dim=-1
)
count_samples(probas_lowT, threshold=1, tokenizer=tokenizer)
```

This code is similar to what we used in the previous temperature-scaling examples, and it prints the following sampled outputs:

```
' __': 158x
' Berlin': 435x
' ____': 169x
' _______': 209x
' Munich': 3x
' Hamburg': 3x
' _____': 18x
```

Now, let's add the top-*p* filter and see how it changes the results:

```
torch.manual_seed(123)
probas_lowT = torch.softmax(
    scale_logits_by_temperature(next_token_logits, 0.35), dim=-1
)
probas_lowT_filtered = top_p_filter(probas_lowT, top_p=0.8)
count_samples(probas_lowT_filtered, threshold=1, tokenizer=tokenizer)
```

With a `top_p` threshold of 0.8, which is a typical value, we exclude the lowest-probability tokens whose cumulative probability makes up the remaining 20%. The sampled outputs are now as follows:

```
' Berlin': 534x
' ____': 217x
' _______': 249x
```

As you can see, we now either select the correct city (removing `" Munich "` and `" Hamburg "` as sampled options) or we print the underscore token, which the model might use to format the text as a quiz question, with `' _______ '` as a placeholder.

Now let's return to our math query and add the top-*p* filter to the `generate_text_temp_stream_cache` function we coded in listing 4.8. The updated function, now called `generate_text_top_p_stream_cache`, is shown next.

Listing 4.16 Text generation with top-*p* filtering

```
@torch.inference_mode()
def generate_text_top_p_stream_cache(
    model,
    token_ids,
    max_new_tokens,
    eos_token_id=None,
    temperature=0.,
    top_p=None
):
    model.eval()
    cache = KVCache(n_layers=model.cfg["n_layers"])
    model.reset_kv_cache()

    out = model(token_ids, cache=cache)[:, -1]  #1
    for _ in range(max_new_tokens):

        orig_device = token_ids.device

        if temperature is None or temperature == 0.0:
            next_token = torch.argmax(out, dim=-1, keepdim=True)

        else:
            logits = scale_logits_by_temperature(out, temperature)  #2
            probas = torch.softmax(logits, dim=-1)  #3

            probas = top_p_filter(probas, top_p)  #4

            next_token = torch.multinomial(probas.cpu(), num_samples=1) #5
            next_token = next_token.to(orig_device)

        if (eos_token_id is not None
                and torch.all(next_token == eos_token_id)):
            break

        yield next_token
        out = model(next_token, cache=cache)[:, -1]
```

#1 Step 3.1: Get logits
#2 Step 3.2: Apply temperature scaling on logits
#3 Step 3.3: Convert to probabilities
#4 (New) Step 4: Apply top-p filter to probabilities
#5 Step 3.4: Sample token according to probabilities

Let's plug this new text generation function into `generate_text_stream_concat_flex`, as we did before with the temperature-scaling version of the text generation function:

```
torch.manual_seed(123)
response = generate_text_stream_concat_flex(
    model, tokenizer, prompt, device,
    max_new_tokens=2048, verbose=True,
    generate_func=generate_text_top_p_stream_cache,
    temperature=0.5,
    top_p=0.8,
)
```

The output is `" \boxed{18} "`, which is still incorrect. Everything we have done so far is to let us sample different outputs. In the next section, we'll use `generate_text_stream_concat_flex` with our augmented text generation function (`generate_text_top_p_stream_cache`) to sample different outputs when implementing the self-consistency inference-time-scaling technique.

TOP-*K* FILTERING

Top-*k* filtering is another way to limit the set of candidate next tokens during sampling. Instead of keeping all tokens whose cumulative probability stays below a threshold (as in top-*p*), top-*k* keeps only the *k* most likely tokens based on the model's logits.

After sorting the vocabulary by probability, everything past the first *k* entries is removed. The remaining *k* tokens are then renormalized and sampled from.

In short, top-*k* keeps a fixed number of the most likely tokens, whereas top-*p* keeps a variable number of tokens depending on their cumulative mass.

Top-*k* is simpler to implement, and I covered it in my *Build a Large Language Model (From Scratch)* book. Top-*p* sampling has become more popular recently.

EXERCISE 4.2: USING TEMPERATURE SCALING AND TOP-*P* FILTERING ON MATH-500

Modify the `evaluate_math500_stream` function from listing 3.15 to see whether adding temperature scaling and top-*p* sampling will change the MATH-500 accuracy of the base model. You can use a setting of 0.9 for both temperature scaling and top-*p*.

4.6 Improving response accuracy with self-consistency

Now that we've coded the temperature scaling and top-*p* filtering, we are ready to implement our second inference-time-scaling technique in this chapter: self-consistency sampling.

The idea behind self-consistency sampling was introduced in the Google Research paper "Self-Consistency Improves Chain-of-Thought Reasoning in Language Models" (**https://arxiv.org/abs/2203.1117**). Despite the fancy name, self-consistency sampling is essentially a form of majority voting: we use temperature scaling and top-*p* filtering to generate multiple answers, and then we select the most frequent one, as shown in figure 4.17.

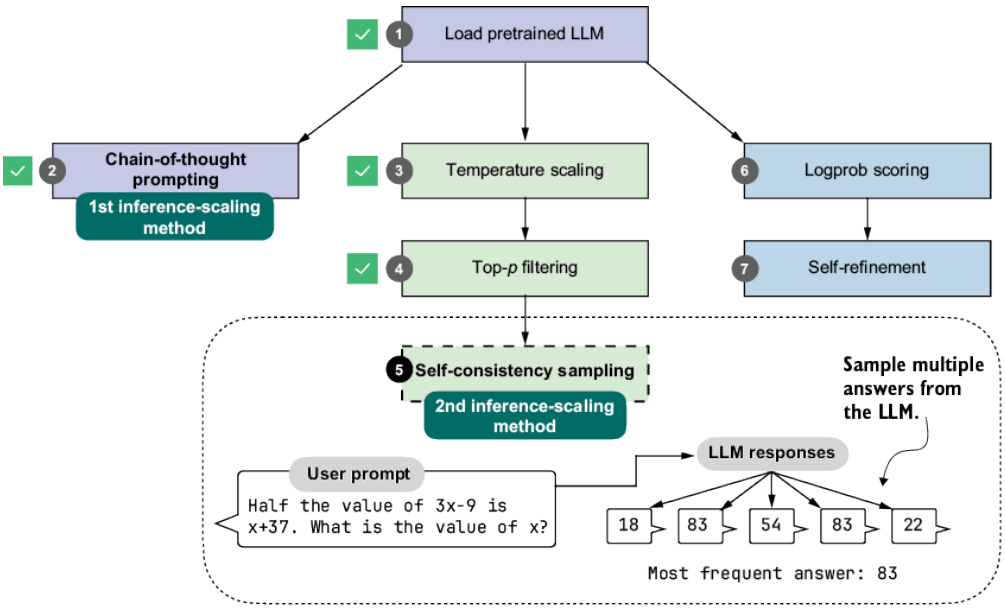


Figure 4.17 The self-consistency sampling method generates multiple responses from the LLM and selects the most frequent answer, improving answer accuracy through majority voting.

The self-consistency sampling technique shown in figure 4.17 counts as an inference-time-scaling technique because we don't update the model itself, and we expend more compute resources to improve response accuracy.

To build our voting-based inference-time pipeline, we'll reuse the generation code from chapter 2, the answer extraction logic from chapter 3, and the sampling controls from this chapter. Thanks to the `generate_text_stream_concat_flex` function, the self-consistency code implementation is relatively straightforward. The main procedure can be summarized in three steps, as illustrated in figure 4.18.

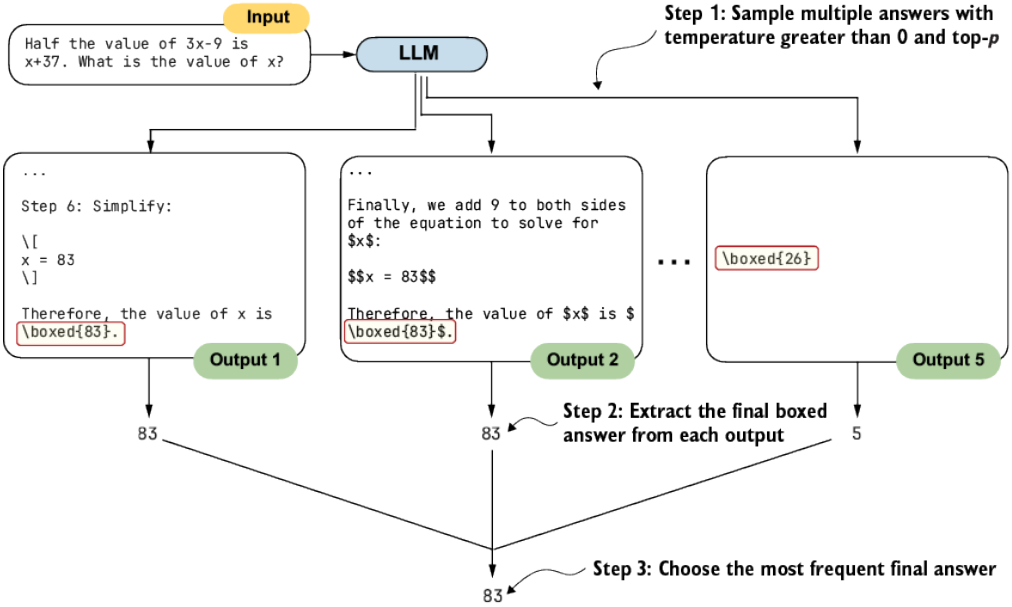


Figure 4.18 The three main steps for implementing self-consistency sampling. First, we generate multiple answers for the same prompt using a temperature greater than zero and top-*p* filtering. Second, we extract the final boxed answer from each generated solution. Third, we select the most frequently extracted answer as the final prediction.

To implement the three steps illustrated in figure 4.18, we simply call `generate_text_stream_concat_flex` repeatedly to generate multiple answers and reuse the `extract_final_candidate` function from chapter 3. The only new code is to implement the majority voting based on answer frequency.

Listing 4.17 Text generation with top-*p* filtering

```
from reasoning_from_scratch.ch03 import extract_final_candidate
from collections import Counter

def self_consistency_vote(
    model, tokenizer, prompt, device,
    num_samples=10, temperature=0.8, top_p=0.9, max_new_tokens=2048,
    show_progress=True, show_long_answer=False, seed=None,
):
    full_answers, short_answers = [], []

    for i in range(num_samples):  #1
        if seed is not None:
            torch.manual_seed(seed + i + 1)

        answer = generate_text_stream_concat_flex(
            model=model, tokenizer=tokenizer, prompt=prompt, device=device,
            max_new_tokens=max_new_tokens, verbose=show_long_answer,
            generate_func=generate_text_top_p_stream_cache,
            temperature=temperature, top_p=top_p,
        )

        short = extract_final_candidate(           #2
            answer, fallback="number_then_full"     #2
        )                                           #2
        full_answers.append(answer)
        short_answers.append(short)
        if show_progress:
            print(f"[Sample {i+1}/{num_samples}] → {short!r}")

    counts = Counter(short_answers)
    groups = {s: [] for s in counts}
    for idx, s in enumerate(short_answers):
        groups[s].append(idx)

    mc = counts.most_common()  #3
    if not mc:
        majority_winners, final_answer = [], None
    else:
        top_freq = mc[0][1]
        majority_winners = [s for s, f in mc if f == top_freq]
        final_answer = mc[0][0] if len(majority_winners) == 1 else None

    return {
        "full_answers": full_answers,
        "short_answers": short_answers,
        "counts": dict(counts),
        "groups": groups,
        "majority_winners": majority_winners,
        "final_answer": final_answer,
    }
```

#1 1) Sample multiple answers

#2 2) Extract the final (short) answer from each answer

#3 3) Choose the most frequent final answer (self-consistency vote)

In short, the self-consistency method in listing 4.17 does the following:

1. Samples multiple answers with temperature greater than 0 and top-*p*
2. Extracts the final boxed answer from each response
3. Chooses the most frequent final answer

Note that in our implementation, we use a `for` loop to generate these answers sequentially. In practice, it's also common to generate the answers on different devices so the sampling can be parallelized.

Furthermore, in the preceding code, we set the random seed for each round individually:

```
            if seed is not None:
                torch.manual_seed(seed + i + 1)
```

Technically, this is not necessary; setting the random seed once should be sufficient to generate diverse samples. This explicit seeding is useful if we want to rerun some of the rounds individually.

Let's try this function and see what we get:

```
results = self_consistency_vote(
    model,
    tokenizer,
    prompt,
    device=device,
    num_samples=5,
    temperature=0.8,
    top_p=0.9,
    max_new_tokens=2048,
    seed=123,
    show_progress=True,
)
```

The printed results are as follows:

```
[Sample 1/5] → '83'
[Sample 2/5] → '22'
[Sample 3/5] → '54'
[Sample 4/5] → '83'
[Sample 5/5] → '61'
```

Since 83 is the most frequent answer, the final answer is 83 (we can access this number programmatically via `results[ " final_answer " ]`). If you want to read the explanation and full answer, you can access one of the correct solutions returning 83; for example, `print(results[ " full_answers " ][0])` prints the following:

```
To find the value of x, let's solve the equation step by step.

1. **Given Equation:**
   \[
   \frac{1}{2} \times (3x - 9) = x + 37
   \]

# ...

**Final Answer:**
\[
\boxed{83}
\]
```

With this self-consistency approach, we were finally able to generate the correct answer. (Note that the results may vary when running the code on an `" mps "` or `" cuda "` device.)

The `self_consistency_vote` function does not currently handle ties, so if multiple samples have the same frequency, it returns `None` as the final answer. In the next chapter, we'll implement a scoring method to calculate the confidence of a given answer, which can be used as a tiebreaker.

EXERCISE 4.3: USING SELF-CONSISTENCY SAMPLING ON MATH-500

Modify the `evaluate_math500_stream` function from listing 3.15 to evaluate whether self-consistency sampling improves the MATH-500 accuracy of the base model. Use a sample size of 3 and set both temperature and top-*p* to 0.9.

As part of this exercise, implement tiebreaking so that ties are resolved by the first answer appearing in the sample list. For example, if the sample answers are 13, 15, 13, 15, 16, the selected answer should be 13.

Note that you don't have to modify `self_consistency_vote` itself to implement this tiebreaking rule; you can work with the `results` dictionary returned by the function instead.

EXERCISE 4.4: EARLY STOPPING IN SELF-CONSISTENCY SAMPLING

To improve computational efficiency, implement an early-stopping version of self-consistency that ends the sampling once more than half of the answers agree.

CHOOSING TEMPERATURE AND TOP-*P* SETTINGS

We've already discussed temperature, but the practical question is how much randomness we should introduce for self-consistency sampling. Our goal is not to make the model as random as possible, but to generate answers that are diverse enough for majority voting to help, while still keeping most samples sensible. In practice, temperature values around 0.5 to 0.9 and top-*p* values around 0.7 to 0.9 are reasonable starting points.

As a rule of thumb, if all long answers look nearly identical, it's a good idea to gently increase the temperature or the top-*p* value to encourage more diversity. If the long answers start to look off or nonsensical, the settings are likely too aggressive and should be reduced, usually by lowering the temperature first. Here, a "long" answer refers to the full answer before extracting the final boxed result. You can inspect the long answers in the results dictionary returned by the `self_consistency_vote` function or run the function with the `show_long_answer=True` setting.

You may recall that the paper describing this technique was titled "Self-Consistency Improves Chain-of-Thought Reasoning in Language Models." How does the "chain-of-thought reasoning" aspect factor into this? Chain-of-thought reasoning here simply refers to the chain-of-thought prompting we introduced earlier in this chapter, when we modified the prompt via `" \n\n Explain step by step. "` so that the LLM generates longer responses.

Let's combine chain-of-thought prompting with self-consistency sampling:

```
results = self_consistency_vote(
    model,
    tokenizer,
    prompt + "\n\nExplain step by step.",
    device=device,
    num_samples=5,
    temperature=0.8,
    top_p=0.9,
    max_new_tokens=2048,
    seed=123,
    show_progress=True,
)
```

In this case, all five answers are 83 (correct), likely because the question is relatively simple for the LLM when using chains of thought. It would be more interesting to see how well the model performs on the entire MATH-500 dataset. The results from various experiments are shown in table 4.1.

Table 4.1 MATH-500 task accuracy for different methods

	Method	Model	Accuracy	Time
1	Baseline (chapter 3), greedy decoding	Base	15.2%	10.1 min
2	Baseline (chapter 3), greedy decoding	Reasoning	48.2%	182.1 min
3	Chain-of-thought prompting (CoT)	Base	40.6%	84.5 min
4	Temperature and top-*p* (Top-*p*)	Base	17.8%	30.7 min
5	Top-*p* + self-consistency (*n*=3)	Base	29.6%	97.6 min
6	Top-*p* + self-consistency (*n*=5)	Base	27.8%	116.8 min
7	Top-*p* + self-consistency (*n*=10)	Base	31.6%	300.4 min
8	Top-*p* + CoT	Base	33.4%	129.2 min
9	Self-consistency (*n*=3) + Top-*p* + CoT	Base	42.2%	211.6 min
10	Self-consistency (*n*=5) + Top-*p* + CoT	Base	48.0%	452.9 min
11	Self-consistency (*n*=10) + Top-*p* + CoT	Base	52.0%	862.6 min
12	Self-consistency (*n*=3) + Top-*p* + CoT	Reasoning	55.2%	544.4 min

For brevity, methods labeled "Top-*p*" in table 4.1 use both temperature scaling and top-*p* sampling. The accuracy values shown in the table were computed on all 500 samples in the MATH-500 test set using a `" cuda "` GPU (DGX Spark).

Let's go through the results one by one. The *n* = 3 abbreviation means that we used a sample size of 3 for self-consistency sampling. Rows 1 and 2 show the results of the base and reasoning variants using the code from chapter 3; that is, we used the text generation function without temperature scaling or top-*p* filtering. This text generation function simply selects the token with the highest score at each step (greedy decoding). We can see that the reasoning variant has approximately three times the accuracy but also a substantially longer running time, since it generates more tokens.

In row 3, we see the results for the chain-of-thought prompting approach using the `" \n\nExplain step by step. "` prompt modification. As you can see, this boosts the base model's accuracy from approximately 15% to 40%.

Row 4 shows what happens when we add temperature scaling and top-*p* sampling to the base model. (All experiments involving temperature scaling and top-*p* sampling in table 4.1 used a setting of 0.9 for both.) The accuracy over the base model in row 1 improves only moderately, from 15.2% to 17.8%. This is expected because temperature scaling and top-*p* sampling merely help us control sampling diversity but are not inference-time-scaling techniques themselves. We can also see that the running time increased from 10.1 min to 30.7 min. This is not due to sampling code overhead, but rather because the model now generates longer responses in some cases.

Rows 5–7 show the self-consistency scaling results. Increasing the number of samples from 3 to 10 further improves accuracy to 31.6%, but it also significantly increases the running time. In this case, there is almost no accuracy advantage to using 10 instead of 3 samples.

Row 8 shows temperature scaling and top-*p* sampling combined with chain-of-thought prompting. In this case, sampling makes the chain-of-thought results worse (33.4% compared to 40.6% in row 3). Combining

chain-of-thought prompting with self-consistency improves accuracy to 52% with a sample size of 10, but it also substantially increases the running time to a staggering 862.6 minutes.

In practice, if you have access to multiple GPUs, the different samples in self-consistency sampling can be computed in parallel rather than sequentially. This would still use the same amount of compute, but it could be spread across GPUs to generate the results faster.

Lastly, in row 12, we can see that the reasoning variant also benefits from self-consistency sampling, increasing its accuracy from 48.2% in row 2 to 55.2%. Again, this comes with increased running time.

The takeaway is that the results in table 4.1 highlight the tradeoff in inference-time scaling: better accuracy for more compute.

One major downside of the self-consistency sampling approach in this chapter is that it relies on a final boxed answer that we can extract for majority voting. This method is trickier to apply to problems that don't have numeric or short final answers.

In the next chapter, as illustrated in figure 4.19, we will implement a different and more versatile inference-time-scaling method called *self-refinement*, in which the model iteratively improves its own answers.

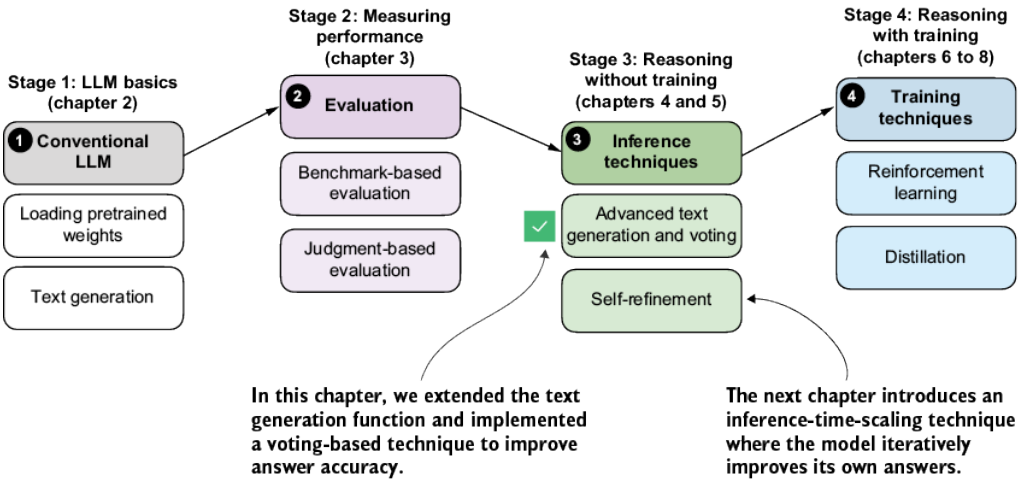


Figure 4.19 Summary of this chapter's focus on inference-time techniques. Here, the text generation function was extended with a voting-based method to improve answer accuracy. The next chapter introduces self-refinement, in which the model iteratively improves its responses.

Summary

- Reasoning abilities and answer accuracy can be improved without retraining the model by increasing compute at inference time (inference-time scaling).
- A flexible text generation wrapper (`generate_text_stream_concat_flex`) allows different sampling strategies to be plugged in without changing the surrounding code.
- Next tokens are produced from logits via softmax.
- Temperature scaling changes logits to control the diversity of the generated text.
- Top-*p* (nucleus) sampling filters out low-probability tokens to reduce the chance of generating nonsensical answers.
- Chain-of-thought prompting (such as "Explain step by step.") often yields more accurate answers by encouraging the model to write out its intermediate reasoning, though it increases the number of generated tokens and, in turn, the runtime cost.
- Self-consistency sampling generates multiple answers, extracts the final boxed result from each, and selects the most frequent answer via majority vote to improve answer accuracy.
- Experiments on the MATH-500 dataset show that combining chain-of-thought prompting with self-consistency can substantially boost accuracy compared to the baseline without sampling, at the cost of much longer running times.
- The central tradeoff of inference-time scaling is higher accuracy in exchange for more compute.

5 Inference-time scaling via self-refinement

This chapter covers

- Scoring LLM answers with a simple rule-based scorer
- Computing an LLM's confidence in its answers
- Coding a self-refinement loop where the LLM iteratively improves its answers

The previous chapter introduced the concept of *inference-time scaling* (*inference scaling* for short), which improves the accuracy of model responses without further training. In particular, it focused on *self-consistency*, where the model generates multiple answers, and the final answer is the most frequent one.

As outlined in figure 5.1, this chapter will move beyond self-consistency for inference scaling and cover another popular and useful inference-scaling technique, *self-refinement*. Instead of generating and choosing from multiple answers, self-refinement focuses on iteratively refining a single answer to correct potential mistakes.

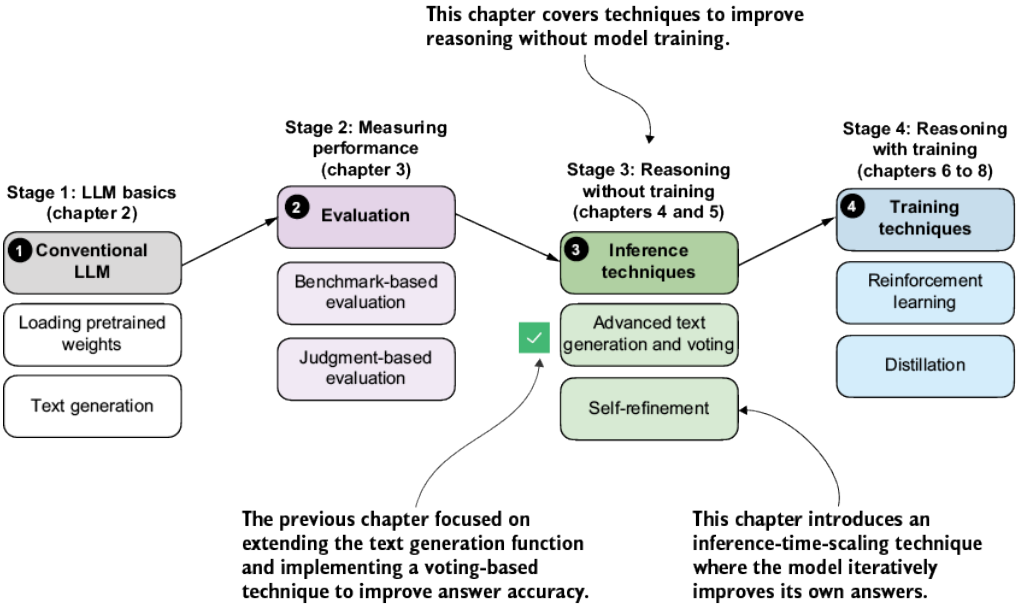


Figure 5.1 A model of the topics covered in this book. This chapter continues with stage 3 and focuses on inference-time techniques for improving reasoning without additional training. It introduces self-refinement, where the model iteratively critiques and improves its own answers.

5.1 Scoring and iteratively improving model responses

As you've seen, inference scaling trades additional compute for better accuracy. We've covered two inference-scaling techniques so far: *chain-of-thought prompting* and *self-consistency*.

Chain-of-thought prompting, the first method illustrated in figure 5.2, modifies the prompt, such as by adding the phrase `" Explain step by step. "`, which can trigger a base model to write longer explanations that can in turn improve answer accuracy. This method is particularly useful for base models that don't naturally provide reasoning-like explanations. Models trained as reasoning models usually don't benefit from this type of inference scaling because they already explain their answers.

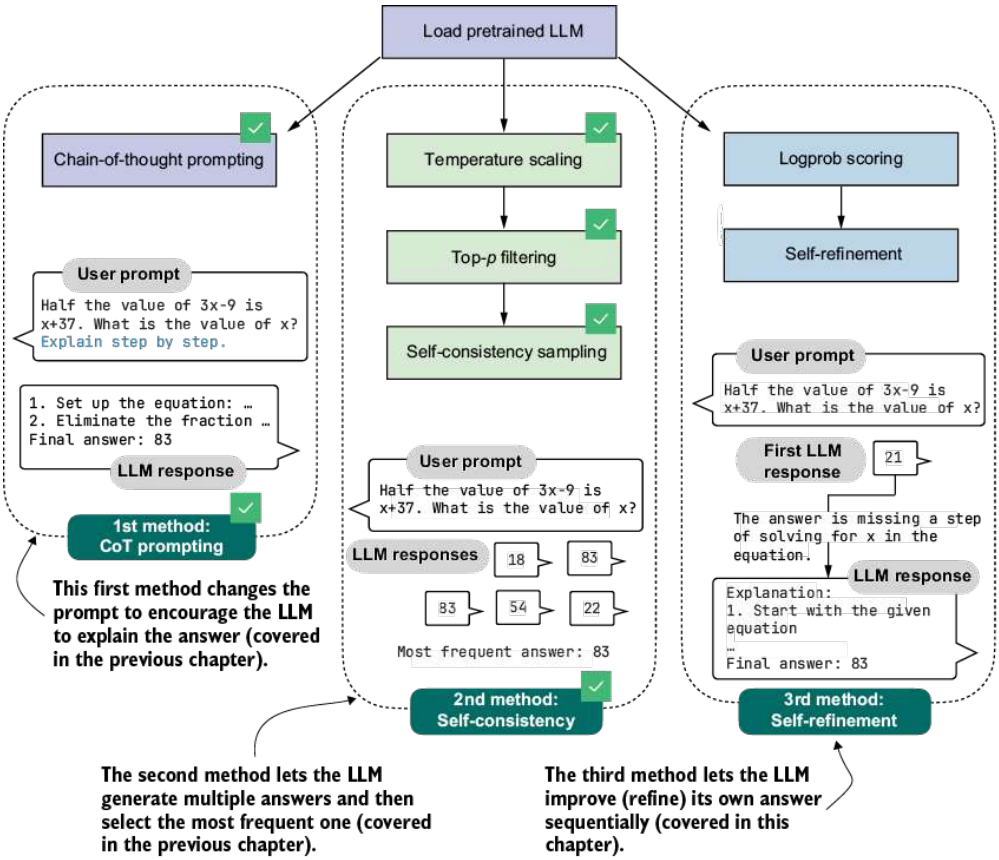


Figure 5.2 The three inference-time methods to improve reasoning covered in this book. The first two methods were covered in the previous chapter. This chapter covers the third method, self-refinement, where the model iteratively improves its own answers.

Self-consistency, the second method in figure 5.2, lets the model produce several answers in parallel. We then choose the final answer by selecting the most frequent of these candidates.

Even though self-consistency is quite simple, it often yields large gains in answer accuracy, which is why it has become a common choice in LLM applications where accuracy is a higher priority than latency. Recent examples include DeepSeekMath-V2 and Google's Gemini 3 Deep Think mode (see appendix A for references). One downside of self-consistency is that selecting the most common answer requires short answers that can be compared.

In this chapter, we'll implement a more versatile technique, *self-refinement*, in which the LLM learns to improve its own answers iteratively (method 3 in figure 5.2). Before we do so, though, we'll first implement scoring functions to compare and rank different answers, as outlined in figure 5.3.

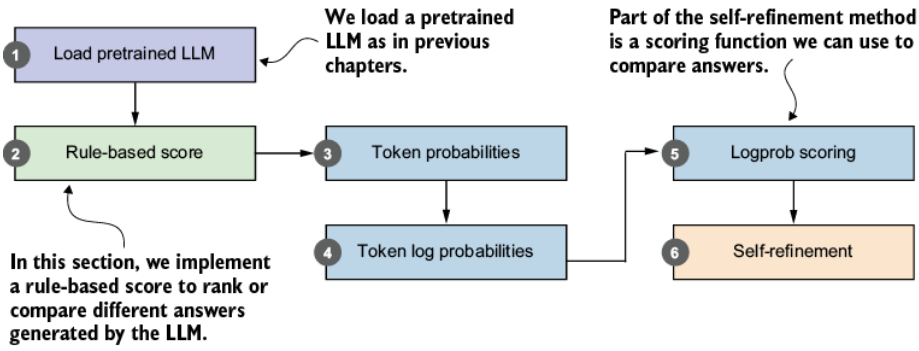


Figure 5.3 We'll build a simple rule-based score, compute token probabilities and log probabilities, and use these scores as part of a self-refinement method in which the model iteratively improves its own answers.

After loading the pretrained LLM, as in previous chapters (step 1 in figure 5.3), we'll implement a simple *rule-based scoring* function to illustrate the concept of scoring (step 2). Then we'll go over the concepts of *token probabilities* (step 3) and *token log probabilities* (step 4), which we'll need to implement the log-probability scoring (logprob scoring) method (step 5) for our self-refinement loop (step 6).

We will primarily use the logprob scoring function (step 5 in figure 5.3) to track progress within the self-refinement loop. These scoring functions can also be used to break ties in self-consistency or to select the

best response instead of relying on a majority vote.

The overview in figure 5.3, leading up to self-refinement, looks relatively short and straightforward, but the topics of token probability and log probability are somewhat complex. These topics will also be relevant in the next chapter, where we'll implement reinforcement learning methods to train the LLM. To that end, we'll spend some time here discussing the concept of log-probability scoring.

5.2 Loading a pretrained model

As in the previous chapter, we'll begin by loading the model and tokenizer we'll use throughout this chapter.

Listing 5.1 Loading the tokenizer and base model

```
import torch
from reasoning_from_scratch.ch02 import get_device
from reasoning_from_scratch.ch03 import (
     load_model_and_tokenizer
)

device = get_device()
device = torch.device("cpu")  #1

model, tokenizer = load_model_and_tokenizer(
    which_model="base",
    device=device,
    use_compile=False
)
```

#1 Delete this line to run the code on a GPU (if supported by your machine).

The preceding code runs on the CPU by default, to ensure results that are generally more consistent with those shown in this chapter. Small numerical differences can still occur on the CPU, depending on the operating system and machine, but these differences are typically smaller and more predictable than those observed across different accelerator backends.

Sections 5.4 and 5.5 will use computations involving very small numbers with many digits after the decimal point, and these are more sensitive to device choice and low-level implementation details. This is not a problem in practice, but such mismatches can be confusing on a first read-through. For that reason, I recommend starting with the CPU device and considering MPS or CUDA devices later.

To ensure that the model is loaded correctly, let's use it with the temperature and top-*p* sampling code from the previous chapter on a MATH-500 prompt.

Listing 5.2 Generating text with temperature scaling and top-*p* sampling

```
from reasoning_from_scratch.ch03 import render_prompt
from reasoning_from_scratch.ch04 import (
    generate_text_stream_concat_flex,
    generate_text_top_p_stream_cache
)

raw_prompt = (
    "Half the value of $3x-9$ is $x+37$. "
    "What is the value of $x$?"
)
prompt = render_prompt(raw_prompt)
prompt_cot = prompt + "\n\nExplain step by step."

torch.manual_seed(0)
response_1 = generate_text_stream_concat_flex(
    model, tokenizer, prompt_cot, device,
    max_new_tokens=2048, verbose=True,
    generate_func=generate_text_top_p_stream_cache,
    temperature=0.9,
    top_p=0.9
)
```

The model produces the following response:

```
 The problem states that half the value of 3x-9 is equal to x+37. We need to find the value of \(x\).

### Step 2: Translate the problem into an equation
... #1

### Final Answer

\[
\boxed{83}
\]
```

#1 Response truncated to preserve space

Because we are using temperature scaling and top-*p* sampling, changing the random seed produces a different response:

```
torch.manual_seed(3)
response_2 = generate_text_stream_concat_flex(
    model, tokenizer, prompt_cot, device,
    max_new_tokens=2048, verbose=True,
    generate_func=generate_text_top_p_stream_cache,
    temperature=0.9,
    top_p=0.9,
)
```

This time, the model responds as follows:

```
 We start with the given equation:
\[
\frac{1}{2} \times (3x - 9) = x + 37
\]
... #1
Final Answer:
\[
\boxed{83}
\]
```

#1 Response truncated to preserve space

In both cases, the model produces the correct final answer (`83`). The second response is much shorter, which we can confirm by printing the number of characters or tokens in each response:

```
print("Response 1 characters:", len(response_1))
print("Response 1 tokens:", len(tokenizer.encode(response_1)))
print("\nResponse 2 characters:", len(response_2))
print("Response 2 tokens:", len(tokenizer.encode(response_2)))
```

This is the result:

```
Response 1 characters: 1419
Response 1 tokens: 534

Response 2 characters: 533
Response 2 tokens: 231
```

A shorter response is not necessarily better. If two responses reach the same correct answer, judging which is better is not straightforward. It often depends on human preferences regarding clarity, usefulness, and the partial correctness of the intermediate steps.

Scoring the intermediate steps of a response remains an active research area (see appendix A), and methods such as process reward models, which evaluate the reasoning itself, do not always yield better outputs in practice.

If the qualitative value of two responses is comparable, one thing is certain: shorter responses are cheaper because they require fewer tokens to be generated, and these responses are therefore preferred.

5.3 Scoring LLM responses with a rule-based score

In the previous section, the LLM generated two correct responses. In this section, we'll develop a simple rule-based scoring function to compare them, as illustrated in figure 5.4.

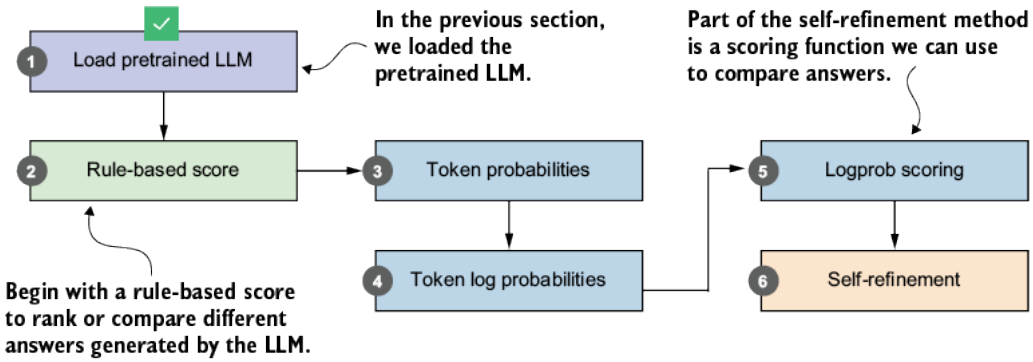


Figure 5.4 This section implements a rule-based scorer to rank different answers generated by the pretrained LLM.

The rule-based scoring function will assign a score to each of two LLM responses (see figure 5.5), which will allow us to rank them and select the better one. Here, "better" refers to format and brevity, not correctness.

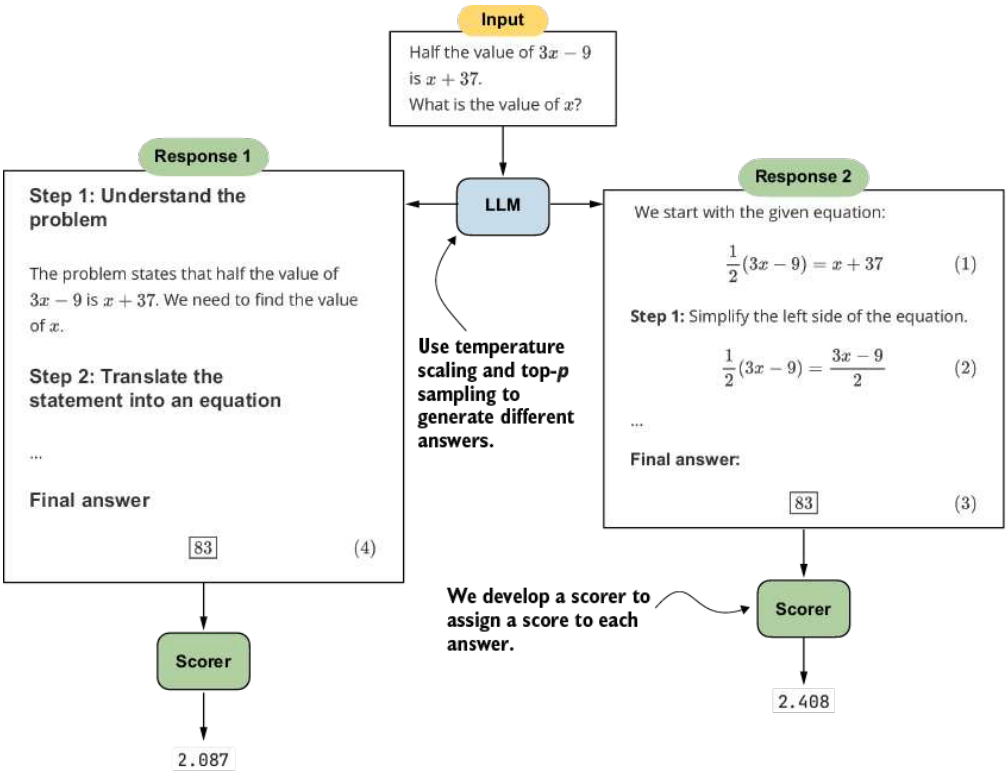


Figure 5.5 Two generated responses reach the same correct answer but differ in their explanations. A scorer evaluates the responses and assigns a score to each response.

There are several ways to implement a scoring function (scorer) for evaluating responses, as shown in figure 5.5. The scorer can be *heuristic*, meaning a simple rule-based method. It can also be another LLM that rates the answers, often referred to as *LLM-as-a-judge* (see appendix F). It can also rely on internal *probability scores* or *likelihoods*, which we will explore later in this chapter.

In this section, we'll begin with a heuristic, a rule-based scorer, implemented in listing 5.3. We begin with this version not because it is the most sophisticated option, but because it gives us a simple baseline that will make the later probability-based scorers easier to compare and reason about.

Listing 5.3 A simple rule-based scorer

```
from reasoning_from_scratch.ch03 import extract_final_candidate
import math

def heuristic_score(
    answer,
    prompt=None,  #1
    brevity_bonus=500.0,
    boxed_bonus=2.0,
    extract_bonus=1.0,
    fulltext_bonus=0.0,
):
    score = 0.0

    cand = extract_final_candidate(answer, fallback="none")    #2
    if cand:
        score += boxed_bonus

    else:     #3
        cand = extract_final_candidate(answer, fallback="number_only")
        if cand:
            score += extract_bonus
        else:
            cand = extract_final_candidate(
                answer, fallback="number_then_full"
            )
            if cand:
                score += fulltext_bonus

    score += 1.5 * math.exp(-len(answer) / brevity_bonus)    #4
    return score
```

#1 A placeholder that we ignore in this section
#2 Reward answers that have a final boxed value.
#3 Give weaker rewards if the answer doesn't have a boxed value.
#4 Add a brevity reward that decays with text length.

This `heuristic_score` function assigns a numerical score to an LLM answer based on how cleanly the answer can be extracted and on the answer's length. For instance, we award a bonus if the answer has a `\boxed{}` response (`boxed_bonus`). Otherwise, we give a smaller bonus if it contains at least a number (`extract_bonus`). The `brevity_bonus` assigns points based on how short the answer is.

The absolute values are not very important. What matters more is their relative scale. Here, `boxed_bonus=2.0` is intentionally larger than `extract_bonus=1.0` so that a cleanly boxed final answer is preferred over one where we can only extract a number. The brevity term contributes at most 1.5 additional points, so it mainly helps to break ties between answers of similar quality rather than dominating the extraction bonuses. We also keep `fulltext_bonus=0.0` so that arbitrary long-form answers are not rewarded unless we can extract a clearer final candidate from them.

NOTE We are developing a scorer here, and not using the verifier from chapter 3, because we are assuming we don't know the true answer to the question. We only use the verifier for evaluation when we're evaluating the model on an existing test set.

The `prompt` argument is a placeholder that doesn't serve a purpose in the function, but it will make life easier (and the code simpler) when we develop a self-refinement function with swappable scorer plugins later in the chapter.

Although `brevity_bonus=500.0` may appear high, it is used as an exponentially decaying term via `1.5 * math.exp(-len(answer) / brevity_bonus)`. The following plot, whose results are shown in figure 5.6, illustrates the effect of the `brevity_bonus` on the score.

Listing 5.4 Plotting the brevity penalty curve

```
import matplotlib.pyplot as plt

def plot_brevity_curve(brevity_bonus, max_len=2048):
    lengths = torch.arange(1, max_len)
    scores = 1.5 * torch.exp(-lengths / brevity_bonus)

    plt.figure(figsize=(4, 3))
    plt.plot(lengths, scores)
    plt.xlabel("Text length (number of characters)")
    plt.ylabel("Score contribution")
    plt.tight_layout()
    plt.show()

plot_brevity_curve(500)
```

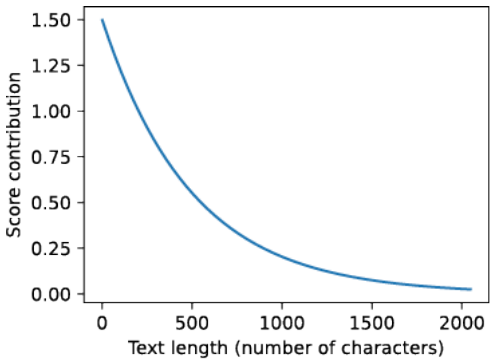


Figure 5.6 A simple rule-based length penalty used by our scorer. Longer explanations receive a smaller score contribution.

The score bonus approaches 1.5 for shorter answers, while answers longer than 1,000 characters receive a bonus of 0.2 or less. A response of a few hundred characters corresponds to a short paragraph or a concise worked solution, whereas 1,000 characters or more usually means a fairly long, multisentence explanation. This term mildly favors compact answers, but it does not force the model to collapse everything into a one-line response.

For simplicity, the brevity bonus is computed using the number of characters rather than the number of tokens, which avoids passing the `tokenizer` to the scoring function. Using token counts would also be reasonable and may even be preferable.

We can now apply the heuristic scorer to the first (longer) response from section 5.2:

```
print(round(heuristic_score(response_1), 3))
```

The resulting score is `2.088`. Next, let's try the second (shorter) response:

```
print(round(heuristic_score(response_2), 3))
```

The computed score is `2.517`, which means the second (shorter) response would be the preferred answer.

You've now seen the basic idea of scoring methods. Later in the chapter, we'll develop another scorer and return to the heuristic scorer as part of the self-refinement method.

EXERCISE 5.1: USING THE HEURISTIC SCORER AS A TIEBREAKER IN SELF-CONSISTENCY

Extend the self-consistency implementation (`self_consistency_vote`) in the previous chapter (listing 4.17) so that it can handle ties among candidate answers. For instance, when two or more answers receive the same number of votes, apply the heuristic scorer (`heuristic_score`) from this section to the tied candidates, and select the one with the highest score. Tip: You don't need to modify the `self_consistency_vote` function itself to apply the tiebreaking; instead, apply the tiebreaking to the `results` dictionary returned by `self_consistency_vote` .

EXERCISE 5.2: USING THE HEURISTIC SCORER IN A BEST-OF-*N* SETUP

Modify the self-consistency implementation so that the final answer is chosen using the heuristic scorer rather than majority voting. Generate *N* candidate answers for each problem (where *N* = *2* or higher), score each candidate with the heuristic scorer, and select the one with the highest score as the final prediction. (If we use a scorer instead of majority vote, the method is called best-of-*N* in the literature, as opposed to self-consistency.)

Apply this method to a small subset of MATH-500 and compare the results to both plain best-of-*N* and the self-consistency tiebreaker from exercise 5.1.

Note that the scorer includes several parameters chosen by intuition, which is why we refer to it as a heuristic score. To optimize the extraction and brevity reward settings, we could plug this scorer into the self-consistency or best-of-*N* methods from exercises 5.1 and 5.2 and evaluate which settings yield higher accuracy on a benchmark dataset such as MATH-500.

As a practical rule of thumb, increase the extraction-related bonuses if the scorer too often prefers answers whose final result is hard to parse. Increase the brevity pressure if the scorer keeps favoring long, rambling answers that do not improve the final result. On the other hand, if the scorer starts preferring overly short but incomplete answers, reduce the brevity penalty or increase the reward for clearly extractable final answers. Thinking about these tradeoffs is often more useful than focusing on the exact numeric values in isolation.

5.4 Understanding token probability scores

Let's now take the first step toward building a scorer based on the model's own confidence (step 5 in figure 5.7), where confidence is a probability that the model assigns. At each position, the model will distribute the probability mass over the possible next tokens. If the tokens in a proposed answer consistently receive high probability, this suggests that the answer is more compatible with the model's own internal preferences. Conversely, if the model assigns very low probability to many of the answer tokens, that's a sign that the model itself does not strongly support that answer.

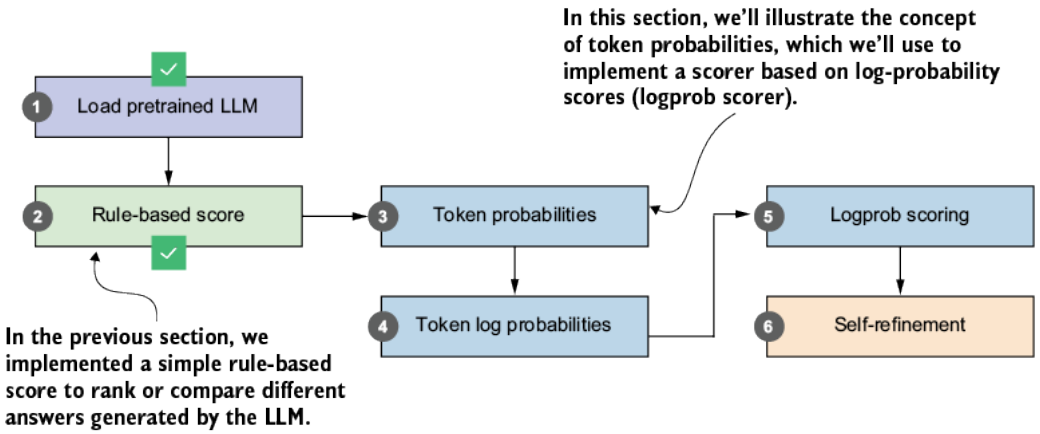


Figure 5.7 In this section, we'll move from a simple rule-based scorer to token-level probabilities. These probabilities will form the basis for the logprob scoring method that we'll use later in the self-refinement approach.

In this section, we'll start by computing the token probability scores for a proposed answer and use them to estimate how likely the model considers that answer to be (step 3 in figure 5.7). This may seem like a detour, but these probability and log-probability concepts will become important again in the next chapter, where they'll reappear inside the training objective rather than only being an inference-time scoring

signal.

The token probability scores we'll compute in this section are also known in the literature as *next-token probabilities*, *per-token probabilities*, *sequence likelihoods*, or, more loosely, *token-level likelihoods*. They represent the probability that the model assigns to each possible next token, expressed as a normalized (softmax) distribution over the vocabulary.

NOTE The term *logprob* is common shorthand in the AI literature for *log probability*.

Although this discussion of probabilities may sound complicated, it relies on the same mechanism used for text generation that we covered in the previous chapter. For instance, in chapter 4, you saw that the model assigns a logit score to each token in the vocabulary at each stage, and it selects the next token based on these scores. This process, which we discussed in section 4.4, is illustrated again in figure 5.8.

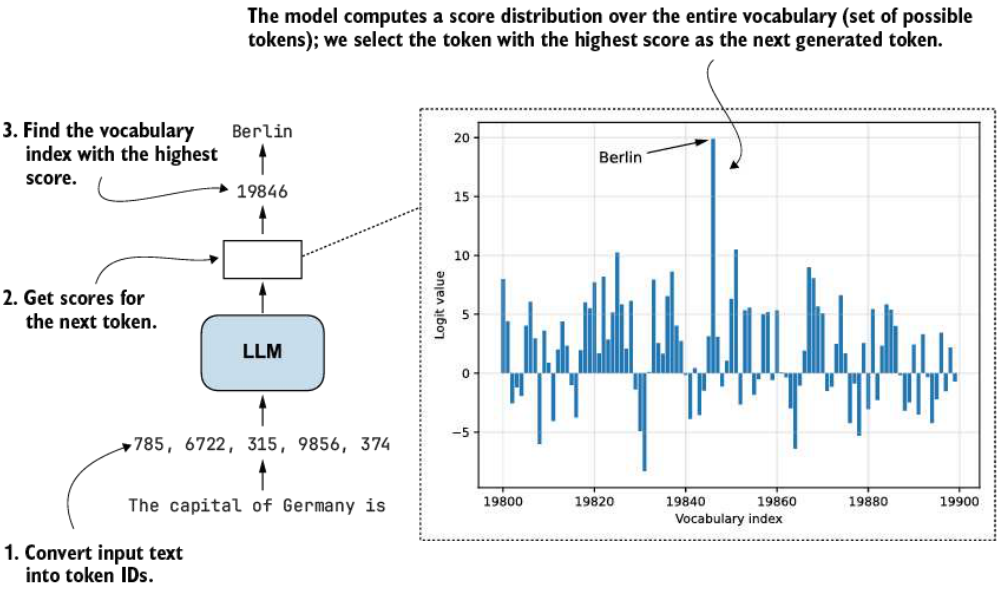


Figure 5.8 How an LLM selects the next token. The model converts the input text into token IDs, computes a score for every vocabulary token, and chooses the token with the highest score. The plot on the right shows the logit values for a subset of the vocabulary, with the token for Berlin having the highest score.

In the previous chapter, we looked up the vocabulary entry corresponding to the highest score (vocabulary index `19846`, corresponding to the token `" Berlin "` in figure 5.8) to get the next generated token. Here, we'll revisit the logits with a different goal in mind. Instead of generating the next token, we want to use the scores to quantify the model's confidence in a particular answer. In other words, we are not going to generate anything based on the scores here; we are just inspecting them.

For example, consider two candidate answers we want to score:

- `" The capital of Germany is Berlin "`
- `" The capital of Germany is Bridge "`

Our goal is to quantify how confident the model is in each answer. It is important to keep in mind that model confidence does not automatically imply correctness. A model

can assign high probability to an answer simply because it fits the patterns it has learned well, even if the answer is factually wrong. When this happens, we still need additional methods, such as retrieval-augmented generation, external tools such as web search, calculators, or code execution, or comparison-based approaches like self-consistency to detect and correct confidently wrong answers.

In figure 5.8, the input text `" The capital of Germany is "` is fed to the model, and the next token with the highest score (`" Berlin "`) is selected. Instead of selecting a token, here we'll compare the scores assigned to two candidate next tokens, `" Berlin "` and `" Bridge "`, as shown in figure 5.9. (`" Bridge "` is used as an alternative because it appears within the vocabulary range shown in figure 5.8.)

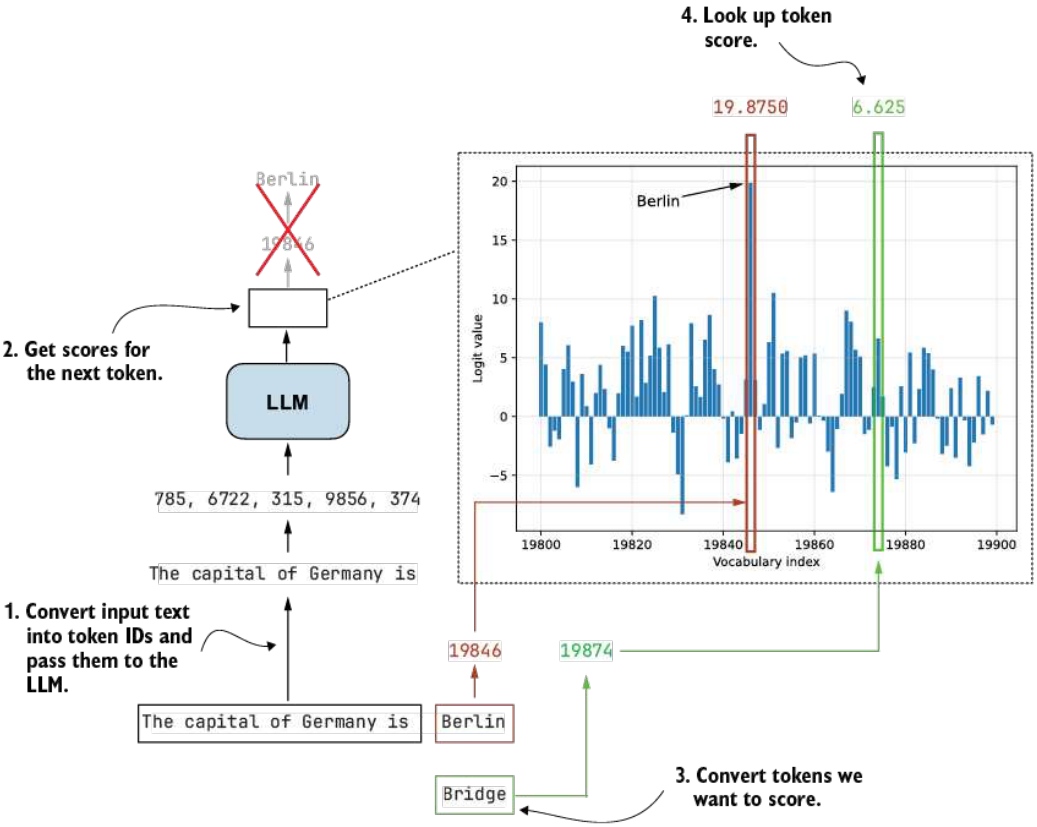


Figure 5.9 How we look up the logit scores for specific tokens. After passing the input text through the model, we obtain a logit value for every vocabulary token. We then convert the candidate tokens we want to score into token IDs and read off their corresponding logit values from the distribution.

Rather than working with the raw logits shown in figure 5.9, we convert them to probabilities. Probabilities are easier to interpret, they're comparable across inputs, and they form the basis for the logprob scoring method we'll use later in the chapter.

As discussed in chapter 4, applying `torch.softmax` converts logits into probability values. Figure 5.10 shows the resulting probability distribution. The token probabilities shown in figure 5.10 are computed using `torch.softmax` as we did in section 4.4.3, as part of the multinomial sampling procedure. The difference here is that we do notsample from the distribution. Instead, we simply look up the probability assigned to specific tokens.

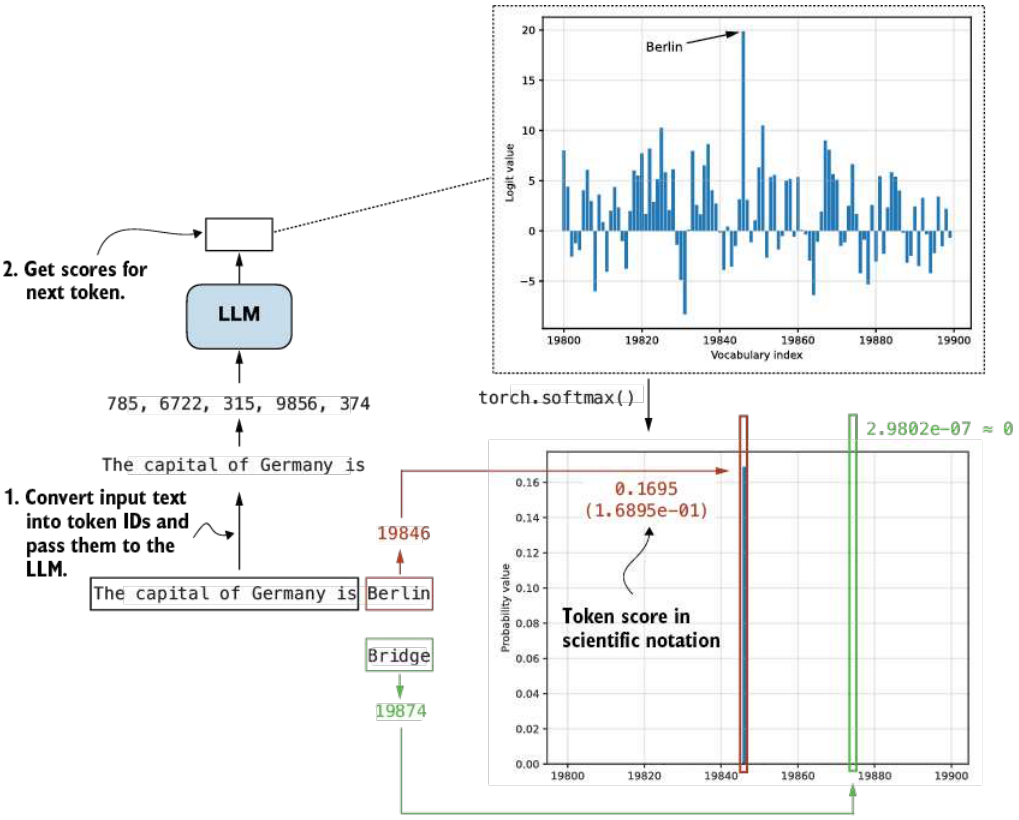


Figure 5.10 Next-token scoring. The input text is converted into token IDs and fed to the LLM, which outputs logits for the next token. After applying a softmax, these logits become probabilities, where tokens like `" Berlin "` receive a high probability and unlikely alternatives such as `" Bridge "` receive values near zero.

In figure 5.10, only the bar for `" Berlin "` is visible, with a probability of `0.1695`. The probabilities for other tokens in the plotted range are near zero. This indicates that the model assigns higher confidence to `" Berlin "` than to `" Bridge "` as the next token, given the input text `" The capital of Germany is "`.

NOTE The bars shown in figure 5.10 do not sum to 1 because the full vocabulary contains 151,000 tokens, and the figure displays only a small slice of the distribution (token indices 19,800–19,900).

In practice, we usually want to compare complete answers rather than individual tokens. This extension from single-token to sequence-level scoring is illustrated in figure 5.11, where we compute the probability of each token given its preceding tokens. This is the same procedure as in figure 5.10, except that we repeat it for every token in the sequence.

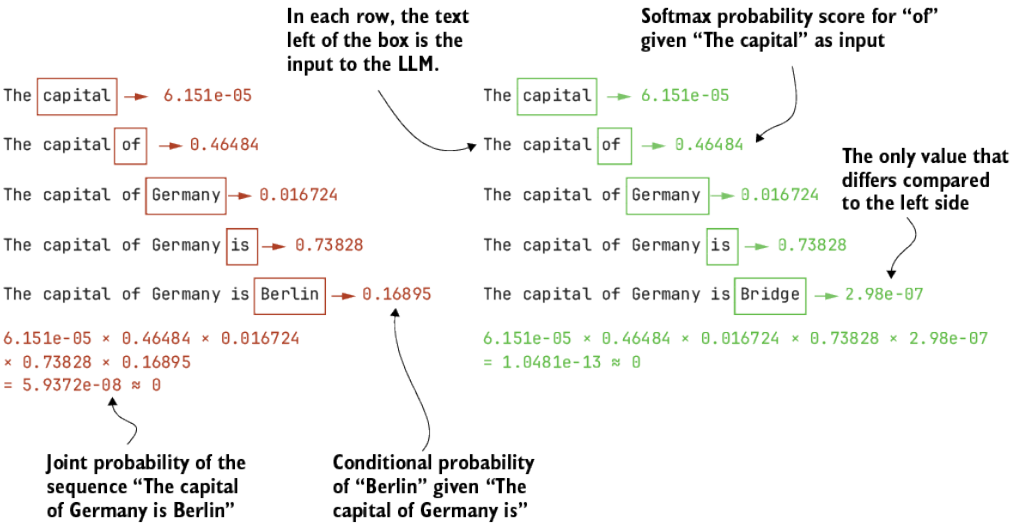


Figure 5.11 Computing token probability scores for a given sequence. For each position, we feed the preceding text into the model and read off the softmax probability of the next token. Multiplying these conditional probabilities yields the joint probability of the full sequence.

During ordinary text generation, the model does this one step at a time as it goes along, selecting the next token. Here, we use the same mechanism in a different way: instead of sampling new tokens, we take a completed candidate answer and score each of its tokens one by one under the model. We then multiply

these probabilities to obtain the probability of the full sequence, also called the *joint probability*.

JOINT PROBABILITY OF A TOKEN SEQUENCE

For those comfortable with mathematical notation, the joint probability of a token sequence can be written compactly as a product of *conditional probabilities*. For a sequence $x1$, $x2$, ..., xT and model weights W, this is

$$P(x_1, x_2, \ldots, x_T|W) = \prod_{t=1}^{T} P(x_t|x_{1:t-1}, W)$$

Expanded, this becomes

$$P(x_1|W) \cdot P(x_2|x_1, W) \cdot P(x_3|x_{1:2}, W) \cdot \ldots \cdot P(x_T|x_{1:t-1}, W)$$

This matches exactly what we compute in figure 5.11. At each position, we feed the context into the model, obtain the probability of the next token, and multiply these scores across the sequence.

Ideally, the probability assigned to the sequence `" The capital of Germany is Berlin "` should be much higher than that of the nonsensical answer `" The capital of Germany is Bridge "`. Because the joint probability is obtained by multiplying many small values, the resulting probabilities for both sequences in figure 5.11 are close to zero. We address this issue in the next section.

The two answers in this example are nearly identical, differing only in the final token (`" Berlin "` versus `" Bridge "`). The same method can also be applied to sequences that differ more substantially, such as the answers generated in section 5.2 for the MATH-500 prompt (“`Half the value of $3x-9$ is $x+37$. What is the value of $x$? "`).

HOW THIS PROBABILITY SCORING DIFFERS FROM GREEDY SAMPLING AND TEMPERATURE-PLUS-TOP-*P* SAMPLING

When we compute token probability scores, we are not generating text. The full sequence already exists, and we are simply querying the model for the probability of each next token, given the fixed context as input. Nothing about these probabilities influences later inputs. It is a retrospective scoring procedure, not a generation step.

Generation methods like the `generate_text_stream_cache` function (greedy sampling) and the `generate_text_top_p_stream_cache` function that we coded in chapter 4 behave very differently. Greedy sampling always selects the most likely next token, while temperature sampling and top-*p* sampling draw from a reshaped probability distribution. These procedures modify the distribution at each step and then commit to a single token, which becomes the input for the next position.

None of these sampling strategies, including greedy sampling, is guaranteed to produce the sequence with the highest overall probability under the model. A token that looks suboptimal at one step can lead to a future step where the following tokens are much more likely. On the other hand, a locally optimal choice may steer the model toward low-probability sequences later.

Even if we did find the globally highest-probability sequence under the model, that still would not guarantee that it is the correct answer or the most useful answer for the user. It would only mean that it is the sequence the model prefers most under its learned distribution.

Temperature and top-*p* sampling add even more variability by flattening or truncating the distribution before drawing samples, which may further disconnect the generated text from the globally most likely sequence.

By scoring an existing answer, we avoid all these complications. We do not run a sampling algorithm, and the model does not choose any tokens. We only evaluate how probable the given sequence would have been, if it had already been written.

We’ve looked at the conceptual explanations, so now let’s see the token probability calculation in action.

Listing 5.5 Calculating next-token probabilities

```
@torch.inference_mode()
def calc_next_token_probas(model, tokenizer, prompt, device, show=True):

    token_ids = torch.tensor(tokenizer.encode(prompt), device=device)

    logits = model(token_ids.unsqueeze(0)).squeeze(0)     #1
    all_probas = torch.softmax(logits, dim=-1)             #1

    t_idx = torch.arange(0, token_ids.shape[0] - 1, device=device) #2

    next_ids = token_ids[1:] #3

    next_token_probas = all_probas[t_idx, next_ids] #4

    prod_next_token_probas = torch.prod(next_token_probas)  #5

    if show:
        print("Next-token probabilities:", next_token_probas)
        print("Joint probability:", prod_next_token_probas)

    else:
        return next_token_probas, prod_next_token_probas
```

#1 Get logits and probabilities as in text generation functions.

#2 Select positions we score (here: all).

#3 Since we have the text, we know the true next tokens.

#4 Get probabilities for each next token.

#5 The likelihood of the sequence is the product of the probability scores.

As you can see, the `calc_next_token_probas` function, which computes the next-token probabilities (illustrated in figure 5.11), is relatively short and appears simple at first glance. But a lot is happening in this function to carry out the computation. Figure 5.12 illustrates this with a simpler input text example and a tiny five-word vocabulary.

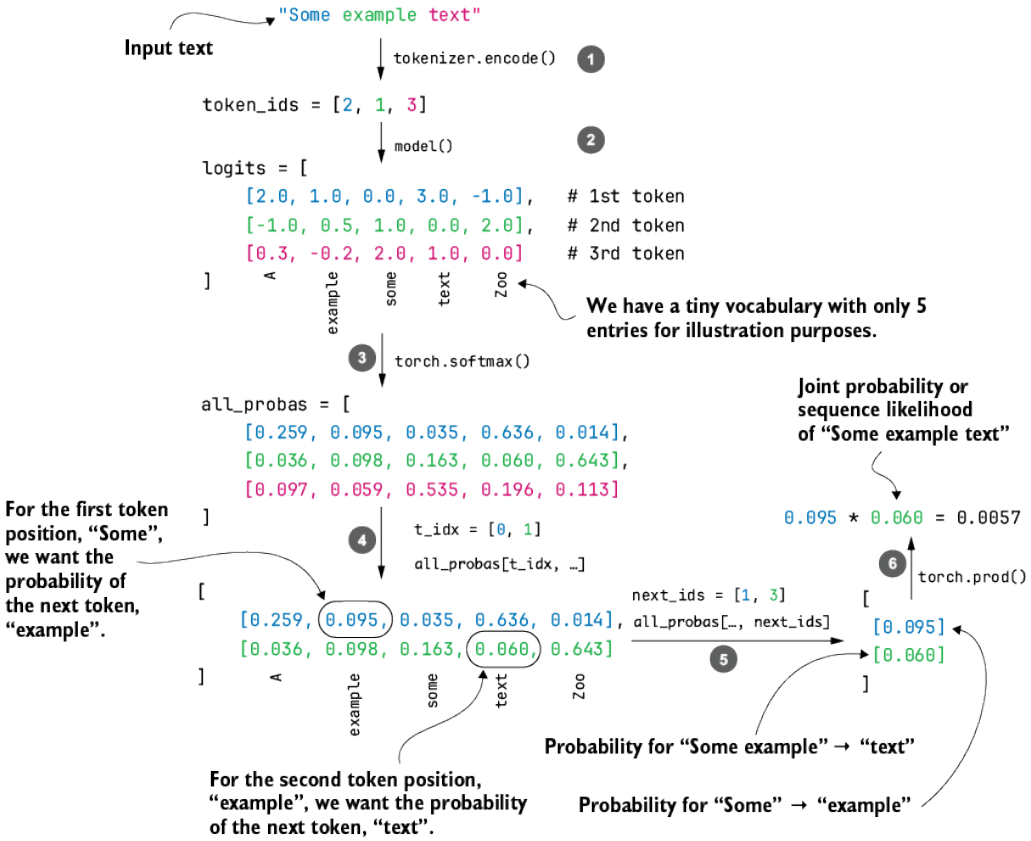


Figure 5.12 Extracting next-token probabilities. After converting the input text into token IDs, the model computes logits that are transformed into probabilities with a softmax function. Using index tensors for positions and true next tokens, we then get the model's computed probability for each next token.

The `calc_next_token_probas` function, illustrated in figure 5.12, computes next-token probabilities in a few steps. First, it computes the logits in the same way as the text-generation functions introduced earlier (for example, the `out = model(token_ids)` line in `generate_text_basic` from section 2.7).

The resulting logits (step 2 in figure 5.12) have shape `[sequence_length, vocab_size]` and are then converted into normalized probability distributions using `torch.softmax` (step 3), so that the probabilities at each position sum to 1.

To extract the probability assigned to the actual next token at each position, we construct an index tensor `t_idx` for the positions we want to score and another tensor, `next_ids`, containing the corresponding target tokens. For example, if the input text is `" The capital of Germany is Berlin "`, then `t_idx` refers to the positions corresponding to `" The capital of Germany is "`, and `next_ids` contains the tokens shifted by one position: `" capital of Germany is Berlin "`.

These target tokens are simply the input tokens shifted by one position because an LLM is trained to predict the next token in a sequence. For example, given the input `" The capital of Germany is Berlin "`, the model is asked at position 2 (the token for `" capital "`) to predict position 3 (the token for `" of "`). Using `all_probas[t_idx, next_ids]` (steps 4 and 5) retrieves exactly these probability values for each position in the sequence.

Finally, the function uses `torch.prod` to compute the sequence likelihood as the product of the per-token probabilities (step 6).

Let's see this function in action:

```
torch.set_printoptions(precision=4, sci_mode=True)
calc_next_token_probas(
        model, tokenizer, device=device,
        prompt="The capital of Germany is Berlin"
    )
```

This is the resulting output:

```
Next-token probabilities: tensor([6.1512e-05, 4.6484e-01,
1.6724e-02, 7.3828e-01, 1.6895e-01], dtype=torch.bfloat16)
Joint probability: tensor(5.9372e-08, dtype=torch.bfloat16)
```

Let's try another text:

```
calc_next_token_probas(
        model, tokenizer, device=device,
        prompt="The capital of Germany is Bridge"
    )
```

This results in the following output:

```
Next-token probabilities: tensor([6.1512e-05, 4.6484e-01, 1.6724e-02,
7.3828e-01, 2.9802e-07], dtype=torch.bfloat16)
Joint probability: tensor(1.0481e-13, dtype=torch.bfloat16)
```

Analyzing these results, the first response (ending in `" Berlin "`) had a joint probability (sequence likelihood) of `5.9372e-08`, which is larger than the joint probability of the second response, `1.0481e-13`. (In decimal form, `5.9372e-08` is `0.000000059372` and `1.0481e-13` is `0.00000000000010481`.)

These results make sense because the `" Berlin "` answer receives a higher sequence likelihood than the `" Bridge "` answer. Both values are extremely small because multiplying many probabilities less than one quickly produces numbers that approach zero. This makes raw likelihoods difficult to work with in practice, especially for longer sequences where underflow becomes a problem.

NOTE These scores reflect the model's internal likelihood, not a calibrated probability of correctness. In other words, they tell us how strongly the model prefers one answer over another under its own distribution, not whether the answer is actually true, correct, or useful. A high score means the answer fits the model's learned patterns well, not that it is guaranteed to be right.

In the next section, we'll modify this calculation with log probabilities, which will let us avoid these numerical issues and give us a more stable way to score entire sequences.

PROBABILITIES VS. LIKELIHOODS

You may have noticed that we've used both the terms "probability" and "likelihood." Is there a difference? In statistics, probabilities describe how likely an event is before we observe any data, and they must sum to one across all possible outcomes. Likelihoods, in contrast, measure how well a specific model explains observed data, and they are viewed as a function of the model's weights or parameters, not the data. In short, probabilities predict data given a model, while likelihoods evaluate models given data. (For a more detailed example, see my article on probabilities versus likelihoods: "What is the difference between likelihood and probability?" at **https://sebastianraschka.com/faq/docs/probability-vs-likelihood.html**.)

In LLMs, the next-token probability is technically a probability, not a likelihood, because it comes from a normalized distribution over all possible next tokens that sums to one. In other words, the values in `next_token_probas` are probabilities, not likelihoods. They come directly from the model's softmax over the vocabulary for each position, which is a normalized probability distribution.

The product, computed as the joint probability (`torch.prod(next_token_probas)`), is the model-assigned probability of the sequence. This quantity is often referred to as the sequence likelihood (especially when viewed as a function of the model parameters).

5.5 From token probability scores to log probabilities

The token probabilities computed in the previous section can be used as a scoring function to rank different responses. As we saw, the probability scores can be very small, especially when multiplied to obtain the joint probability or sequence likelihood.

In this section, to avoid the numerical stability issues that often arise when working with such small values, we'll apply *logarithmic scaling* (also called *log scaling*) to these probabilities, as shown in the overview in figure 5.13.

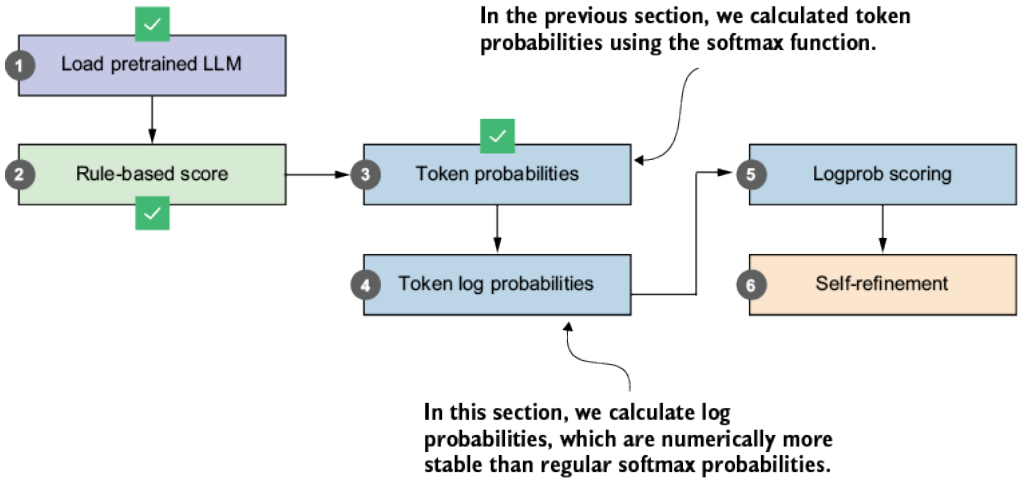


Figure 5.13 Overview of moving from token probabilities to token log probabilities, which provide a numerically more stable basis for the log-probability scoring used later in self-refinement.

NOTE This section focuses on applying a scaling transformation to the probabilities from the previous section. Our overall goal is still to compute the token probabilities, or, in this case, log probabilities (which you'll see soon).

Consider this simple example of computing probabilities for a selection of logit values:

```
torch.set_printoptions(precision=4, sci_mode=False)
logits = torch.linspace(-2, 2, steps=7)
probas = torch.softmax(logits, dim=-1)
print(probas)
```

The probability values are as follows:

```
tensor([0.0090, 0.0175, 0.0341, 0.0665, 0.1295, 0.2522, 0.4912])
```

We can turn them into log-probability scores via the `torch.log` function:

```
print(torch.log(probas))
```

These are the log-probability values:

```
tensor([-4.7109, -4.0442, -3.3776, -2.7109, -2.0442, -1.3776, -0.7109])
```

Here, `torch.log` applies the natural logarithm, where $log(0.0090) = -4.7109$, and vice versa, $e^{-4.7109} = 0.0090$.

Instead of chaining `torch.log(torch.softmax(...))`, PyTorch also has an optimized `torch.log_softmax` function that combines these two operations:

```
log_probas = torch.log_softmax(logits, dim=-1)
print(log_probas)
```

Similar to before, this returns the following:

```
tensor([-4.7109, -4.0442, -3.3776, -2.7109, -2.0442, -1.3776, -0.7109])
```

Note that log scaling changes only the magnitude of the values, not their ordering, so a sequence with a higher likelihood will always have a higher log likelihood as well. To see this visually, let's plot the logits, probabilities, and log probabilities side by side.

Listing 5.6 Plotting logits, softmax probabilities, and log-softmax values

```
plt.figure(figsize=(9, 4))

plt.subplot(1, 3, 1)#1
plt.bar(range(len(logits)), logits, color="C0", alpha=0.7) #1
plt.title("Logits") #1
plt.xlabel("Token index") #1
plt.ylabel("Value") #1
plt.grid(alpha=0.3) #1
 #1
plt.subplot(1, 3, 2)#2
plt.bar(range(len(probas)), probas, color="C1", alpha=0.7)
plt.title("torch.softmax(logits)")
plt.xlabel("Token index")
plt.ylabel("Probability")
plt.ylim(0, 1)#3
plt.grid(alpha=0.3)

plt.subplot(1, 3, 3)#4
plt.bar(range(len(log_probas)), log_probas, color="C2", alpha=0.7) #4
plt.title("torch.log_softmax(logits)") #4
plt.xlabel("Token index") #4
plt.ylabel("Log-probability") #4
plt.grid(alpha=0.3) #4
 #4
plt.tight_layout()
plt.savefig("logits_softmax_log_softmax.pdf")
plt.show()
```

#1 Plotting logits

#2 Plotting softmax probabilities

#3 Plotting softmax probabilities

#4 Plotting log-softmax values

The resulting plot is shown in figure 5.14. As you can see, the log probabilities have the same order as the logits and probability values. A larger logit corresponds to a larger probability and a less negative log probability. Smaller logits, in turn, produce smaller probabilities and more negative log probabilities.

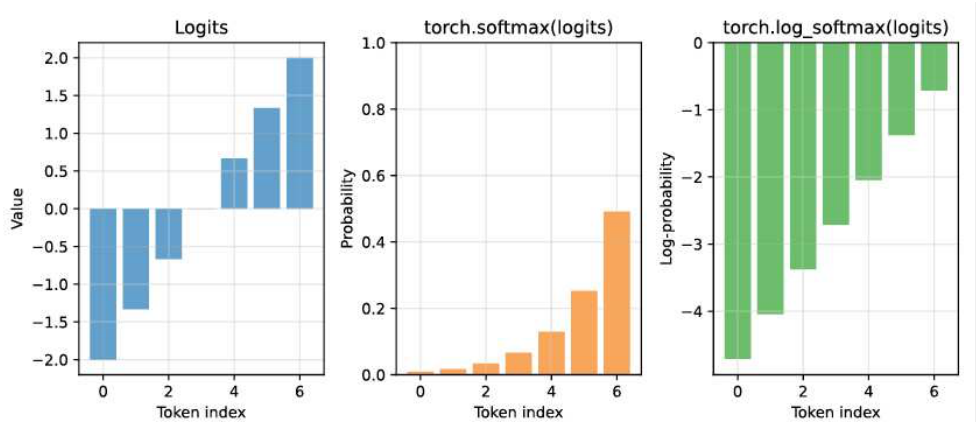


Figure 5.14 Comparing logits, softmax probabilities, and log probabilities for a simple example. The log probabilities preserve the ordering of the probabilities.

In practical terms, higher probabilities are better because they indicate the model assigns more confidence to a token. For log probabilities, values closer to zero are better, since zero corresponds to a probability of one, while very negative values correspond to extremely small probabilities. This makes it easy to compare tokens: the least negative log probability is the most likely one, and the most negative log probability is the least likely.

While figure 5.14 illustrates the relationship between logits, probabilities, and log probabilities for a simple example, we can apply it to the probability values from the earlier `" The capital of Germany is "` prompt, as shown in figure 5.15.

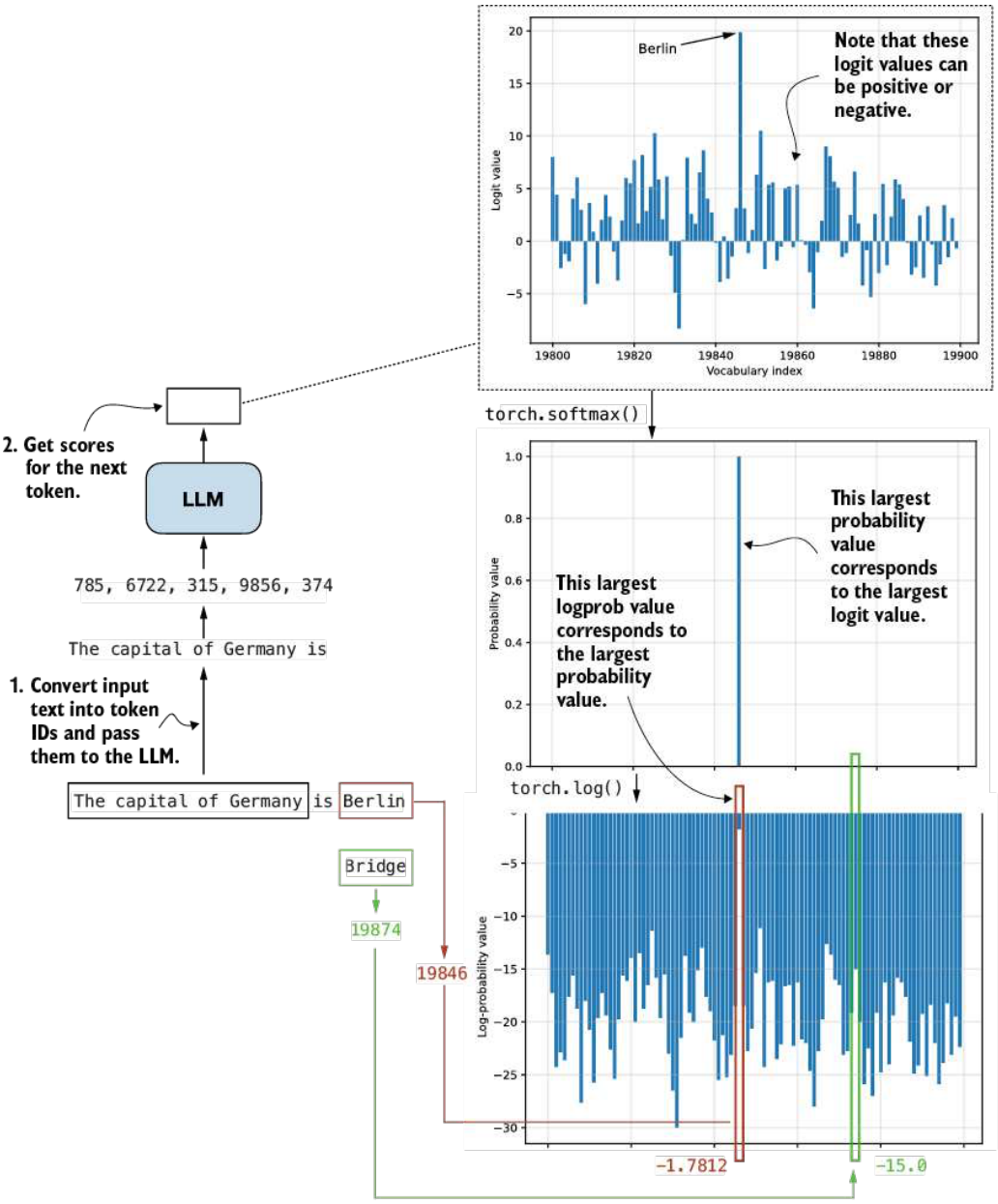


Figure 5.15 How logits are converted to probabilities and log probabilities for next-token scoring. The correct next token (`" Berlin "`) receives a high logit, which becomes a high probability and a less negative log probability, while unlikely candidates like `" Bridge "` map to very small probabilities and large negative log probabilities.

We use probabilities instead of logits because probabilities are normalized and easier to interpret as confidence values. Earlier, we also noted that log probabilities offer an additional benefit, since they allow more numerically stable calculations than working with raw probabilities.

The numerical stability comes from the fact that even very small probability scores map to log probabilities in a reasonable numeric range. Another reason is that multiplying many probabilities quickly drives the result toward zero, while taking the log turns this multiplication into addition, which avoids underflow and is much more stable for long sequences.

Figure 5.16 shows how the joint log probability (or sequence log likelihood) is computed for the two example texts we used earlier. When we computed the regular joint probability score, we saw that both sequences had values close to zero. Now, with the joint log probability, we can see that even changing just the final token (for example, `" Berlin "` versus `" Bridge "`) results in a noticeably different total log probability (`-16.6250` versus `-29.8750`).

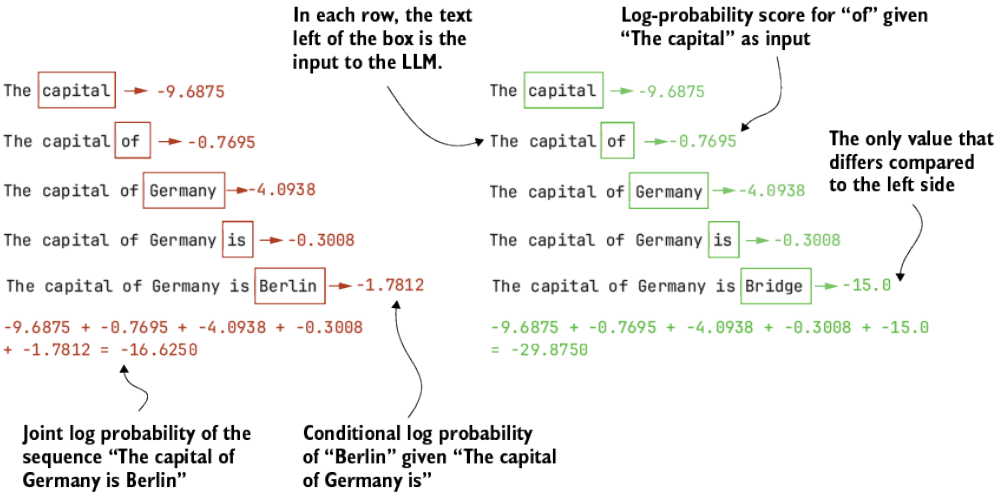


Figure 5.16 How token-level log probabilities accumulate to form sequence log probabilities. Each row shows the log probability of the next token, given the preceding text. Summing these values results in the joint log probability of the full sequence.

WORKING WITH LOG PROBABILITIES

When we compute the joint probability of a sequence, we multiply many numbers between 0 and 1:

$$P(x_1, x_2, \ldots, x_T|W) = \prod_{t=1}^{T} P(x_t|x_{1:t-1}, W)$$

We can write the expanded form as follows:

$$P(x_1|W) \cdot P(x_2|x_1, W) \cdot P(x_3|x_{1:2}, W) \cdot \ldots \cdot P(x_T|x_{1:t-1}, W)$$

These products quickly become extremely small, which is inconvenient and can cause numerical underflow. When numbers become too tiny, the computer may round them down to zero, and we lose useful information. To avoid this, we often work in log space. Taking the logarithm of the joint probability gives us this:

$$\log(P(x_1, x_2, \ldots, x_T|W)) = \log\left(\prod_{t=1}^{T} P(x_1|x_{1:t-1}|W)\right)$$

Using the fact that the logarithm of a product is the sum of the logarithms, this expands to

$$\log P(x_1|W) + \log P(x_2|x_1, W) + \log P(x_3|x_{1:2}, W) + \ldots + \log P(x_T|x_{1:t-1}, W)$$

In other words, the log probability of a sequence is just the sum of the log probabilities of its individual tokens. This is both numerically more stable and easier to work with. It also matches the values returned by `torch.log_softmax`, which provides the log probabilities directly.

Summing log probabilities is therefore the standard approach in machine learning and AI.

Implementing the token log probability function is straightforward because it requires only two small changes to the code in listing 5.6: changing `torch.softmax` to `torch.log_softmax` and changing `torch.prod` to `torch.sum`.

WARNING Using mismatched combinations, such as `torch.softmax` with `torch.sum` or `torch.log_softmax` with `torch.prod`, is technically possible but mathematically incorrect.

Listing 5.7 Calculating next-token log probabilities

```
@torch.inference_mode()
def calc_next_token_logprobas(model, tokenizer, prompt, device, show=True):

    token_ids = torch.tensor(tokenizer.encode(prompt), device=device)

    logits = model(token_ids.unsqueeze(0)).squeeze(0)
    all_logprobas = torch.log_softmax(logits, dim=-1)    #1

    t_idx = torch.arange(0, token_ids.shape[0] - 1, device=device)
    next_ids = token_ids[1:]
    next_token_logprobas = all_logprobas[t_idx, next_ids]

    sum_next_token_logprobas = torch.sum(next_token_logprobas)    #2

    if show:
        print("Next-token log-probabilities:", next_token_logprobas)
        print("Joint log-probability:", sum_next_token_logprobas)
    else:
        return next_token_logprobas, sum_next_token_logprobas
```

#1 We now use log_softmax.

#2 We replace the product with a sum.

Let's now try the updated function on the example prompts from earlier:

```
calc_next_token_logprobas(
        model, tokenizer, device=device,
        prompt="The capital of Germany is Berlin"
    )
```

The result is as follows:

```
Next-token log-probabilities: tensor([-9.6875, -0.7695, -4.0938,
-0.3008, -1.7812], dtype=torch.bfloat16)
Joint log-probability: tensor(-16.6250, dtype=torch.bfloat16)
```

Now let's try the second sequence:

```
calc_next_token_logprobas(
        model, tokenizer, device=device,
        prompt="The capital of Germany is Bridge"
    )
```

This returns the following:

```
Next-token log-probabilities: tensor([ -9.6875,  -0.7695,  -4.0938,
-0.3008, -15.0000], dtype=torch.bfloat16)
Joint log-probability: tensor(-29.8750, dtype=torch.bfloat16)
```

As you can see, the difference between the `" Berlin "` and `" Bridge "` is now much more pronounced (`-16.6250` and `-29.8750`), with `" Berlin "` scoring much higher (better).

5.6 Scoring model confidence with log probabilities

The previous two sections explained the concepts of token probabilities and log probabilities in great detail. In this section, we'll make slight modifications to the log-probability computation to develop a log-probability-based scorer function analogous to the heuristic scorer we developed at the beginning of this chapter (see figure 5.17).

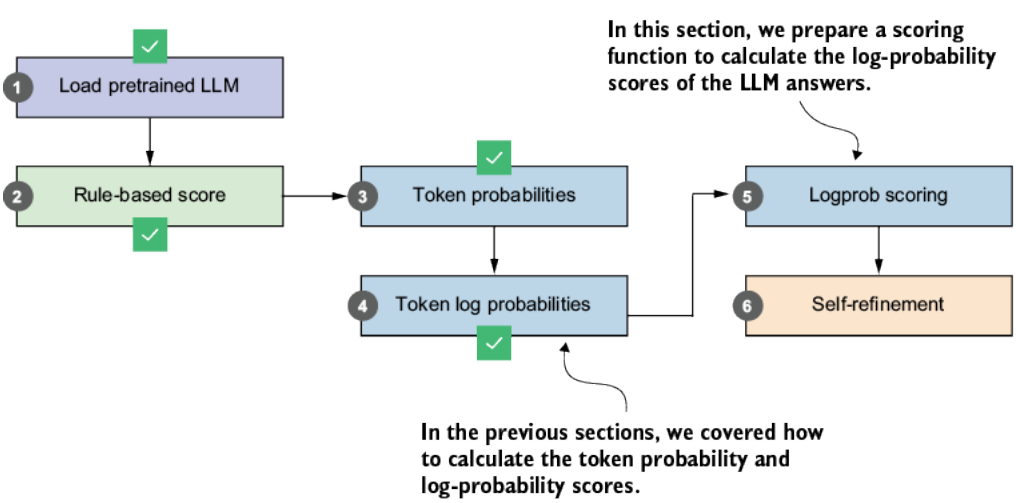


Figure 5.17 This section implements a logprob scorer based on token log probabilities, which we'll use in the self-refinement method later in this chapter.

The logprob scoring method we'll develop is very similar to the procedure we covered in the previous section. We'll make two main modifications. First, we'll exclude the prompt from the score calculation and calculate the score only for the answer tokens. Second, we'll average (instead of sum) the token log probabilities so that it is fairer to compare two sequences of different lengths. The updated calculation with these two modifications is shown in figure 5.18.

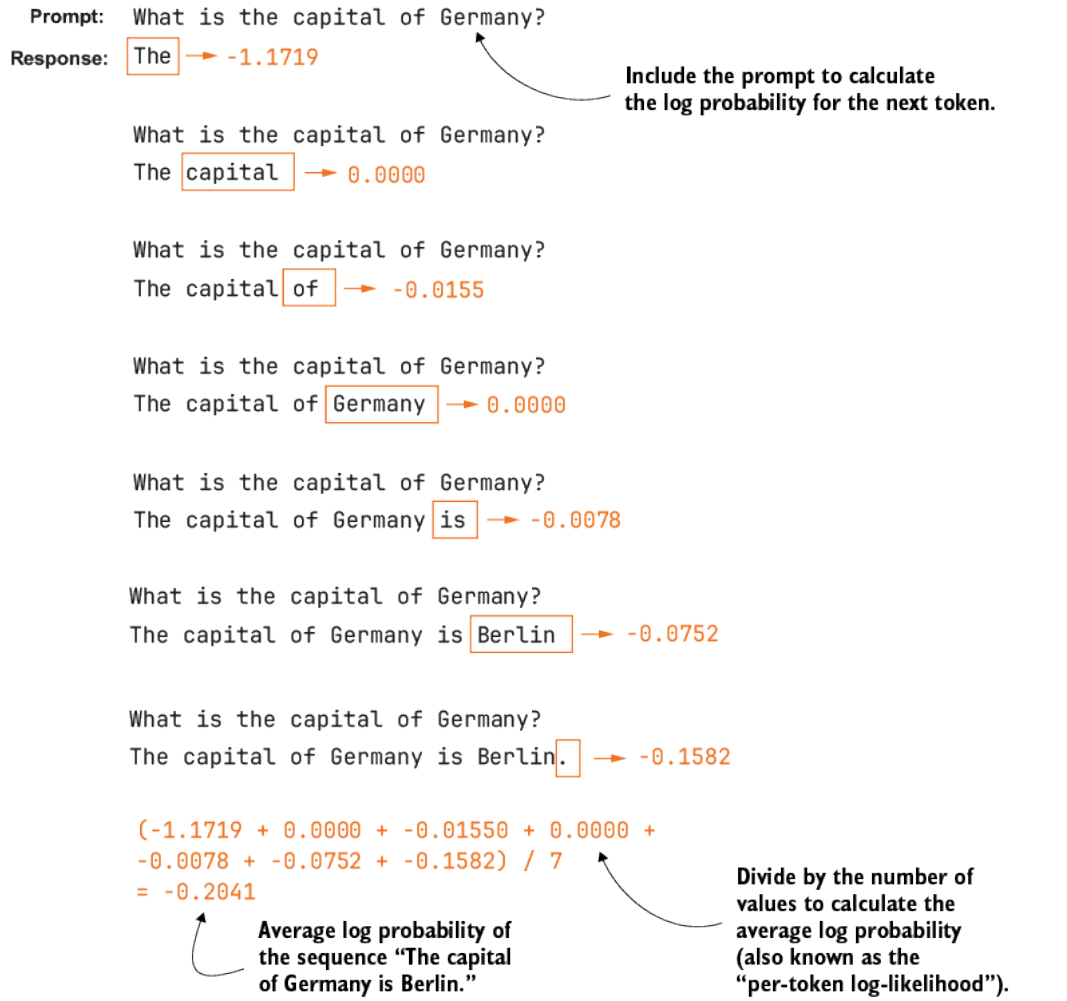


Figure 5.18 The modified logprob scoring procedure. The prompt tokens are excluded from the calculation, and only the log probabilities of the answer tokens are collected. These values are then averaged to obtain a length-normalized score, allowing us to compare answers of different lengths fairly.

For illustration, let's implement the two modifications shown in figure 5.18 using the `calc_next_token_logprobas` function from listing 5.7. For example, suppose we have the following prompt and answer:

```
example_prompt = "What is the capital of Germany?"
example_answer = " The capital of Germany is Berlin."

next_token_logprobas, sum_next_token_logprobas = calc_next_token_logprobas(
        model, tokenizer, device=device,
        prompt=example_prompt+example_answer,
        show=False
    )

print("Next-token logprobas:", next_token_logprobas)
print("Joint log-probability:", sum_next_token_logprobas)
```

This prints the following output. (Note that the tensor contains scores for the prompt token as well, which is why the numbers appear different from figure 5.18. This will become clearer as you read on.)

```
Next-token logprobas: tensor([-0.4512, -0.3418, -8.3125, -0.3906,
-3.8125, -3.0469, -1.1719,  0.0000, -0.0155,  0.0000,
-0.0078, -0.0752, -0.1582], dtype=torch.bfloat16)
Joint log-probability: tensor(-17.7500, dtype=torch.bfloat16)
```

We can then calculate the number of answer tokens with the following code, which yields `7`:

```
print(len(tokenizer.encode(example_answer)))
```

Then, to calculate the average log probability over the answer tokens, as shown in figure 5.18, we need to average over those seven answer tokens:

```
last_7 = next_token_logprobas[-7:]
print(last_7)
print(torch.mean(last_7))
This prints:
tensor([-1.1719,  0.0000, -0.0155,  0.0000, -0.0078, -0.0752, -0.1582],
       dtype=torch.bfloat16)
tensor(-0.2041, dtype=torch.bfloat16)
```

We can see that the resulting average, `-0.2041`, is similar to what's shown in figure 5.18.

We can combine the `calc_next_token_logprobas` code with this calculation into a new function, `avg_logprob_answer`.

Listing 5.8 Average log-probability scoring for answer tokens

```
@torch.inference_mode()
def avg_logprob_answer(model, tokenizer, prompt, answer, device="cpu"):

    prompt_ids = tokenizer.encode(prompt)  #1
    answer_ids = tokenizer.encode(answer)   #2
    full_ids = torch.tensor(prompt_ids + answer_ids, device=device)

    logits = model(full_ids.unsqueeze(0)).squeeze(0)  #2
    logprobs = torch.log_softmax(logits, dim=-1)       #2

    start = len(prompt_ids) - 1  #3
    end = full_ids.shape[0] - 1   #3


    t_idx = torch.arange(start, end, device=device)#4
    next_tokens = full_ids[start + 1 : end + 1] #4
    next_token_logps = logprobs[t_idx, next_tokens] #4


    return torch.mean(next_token_logps)#5
```

#1 Encode prompt and answer tokens separately to get the prompt length later.
#2 Same as in calc_next_token_logprobas before
#3 Index range for positions corresponding to answer tokens
#4 Same as before, except for using start and end
#5 Average over the answer token scores.

The `avg_logprob_answer` function is similar to the `calc_next_token_logprobas` function in listing 5.7, except that we calculate logprobs only for the answer tokens rather than for the whole sequence, including the prompt, and we average the calculated logprob values instead of summing them.

Let's apply this new function to the prompt and answer from earlier in this section:

```
score_1 = avg_logprob_answer(
    model, tokenizer,
    prompt="What is the capital of Germany?",
    answer=" The capital of Germany is Berlin.",
    device=device
)
print(score_1)
```

This returns `-0.2041`, as before, which indicates that we implemented the function correctly. We can now also apply this function to the nonsensical `" Bridge "` answer:

```
score_2 = avg_logprob_answer(
    model, tokenizer,
    prompt="What is the capital of Germany?",
    answer=" The capital of Germany is Bridge.",
    device=device
)
print(score_2)
```

The resulting score here is `-3.8906`, which is much lower than the `" Berlin "` score, as expected.

With the average logprob scoring function now in place, we could also calculate the scores for the MATH-500 prompt (`prompt_cot`) we defined at the beginning of this chapter and the responses we stored as `response_1` and `response_2`. I'll leave that to you as an exercise.

We've spent a lot of time in this chapter going through the concepts of next-token probability and log-probability scoring. One reason for doing so is that we can use these as scorers in the upcoming section on self-refinement. A second reason is that the concept of log probabilities will also be relevant when we

implement the reinforcement learning with verifiable rewards training in the next chapter.

At the same time, it is important not to assume logprob scoring can be a universal fix. Because it measures what the model itself strongly prefers, logprob scoring can still favor confident mistakes over correct but less strongly preferred answers. In practice, logprob scoring is best viewed as one useful scoring signal among several, not as a guaranteed tiebreaker.

EXERCISE 5.3: USING THE LOGPROB SCORER AS A TIEBREAKER IN SELF-CONSISTENCY

Using the logprob scorer instead of the heuristic scorer in exercise 5.1, extend the self-consistency implementation (`self_consistency_vote`) from the previous chapter (listing 4.17) so it can handle ties among candidate answers. Then run the two implementations on a subset of the MATH-500 dataset to see which tiebreaking method performs better.

EXERCISE 5.4: USING THE LOGPROB SCORER IN A BEST-OF-*N* SETUP

Extend the self-consistency implementation so that the final answer is chosen using the logprob scorer (`avg_logprob_answer`) rather than the heuristic scorer or majority voting, as in exercise 5.2. Then run the different implementations on a subset of the MATH-500 dataset to see which tiebreaking method performs better.

Tip: You don't need to modify the `self_consistency_vote` function itself to apply the tiebreaking. You can apply tiebreaking to the `results` dictionary returned by `self_consistency_vote` , which is used inside the `evaluate_math500_stream` function from listing 3.15.

5.7 Self-refinement through iterative feedback

Having introduced several methods for scoring LLM answers, let's now get to the core inference-scaling technique of this chapter, *self-refinement* (figure 5.19). We'll first introduce self-refinement and walk through the process manually. In the next section, as shown in figure 5.19, we'll automate the self-refinement loop and add support for the scoring methods developed earlier in the chapter (the heuristic scorer and the average logprob scorer).

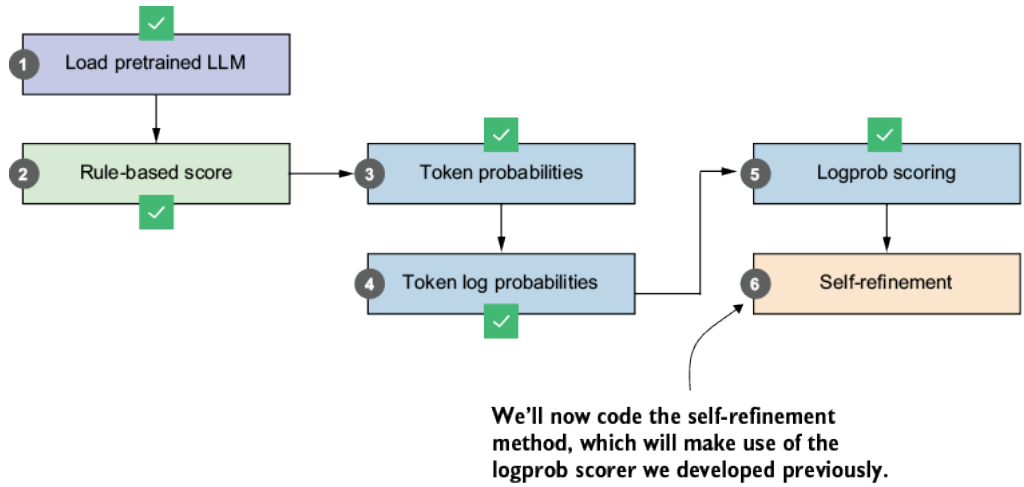


Figure 5.19 The final step in our workflow, where the logprob scorer we developed earlier is used in the self-refinement method

Self-refinement is a technique in which the LLM analyzes and refines its own answers. As shown in figure 5.20, the LLM starts with an initial answer to the prompt, as in regular LLM usage. Then it critiques the answer and refines it.

The self-refinement procedure in figure 5.20 may look complicated at first, but it's essentially just a sequential application of the text generation function to different prompts and inputs. Let's walk through it step by step with a code example.

NOTE In this example, we won't use chain-of-thought prompting. In practice, though, you can combine self-refinement with chain-of-thought prompting when working with a base model.

We'll start with the base prompt and the answer (steps 1 and 2 in figure 5.20), based on the code from sec-

tion 5.2.

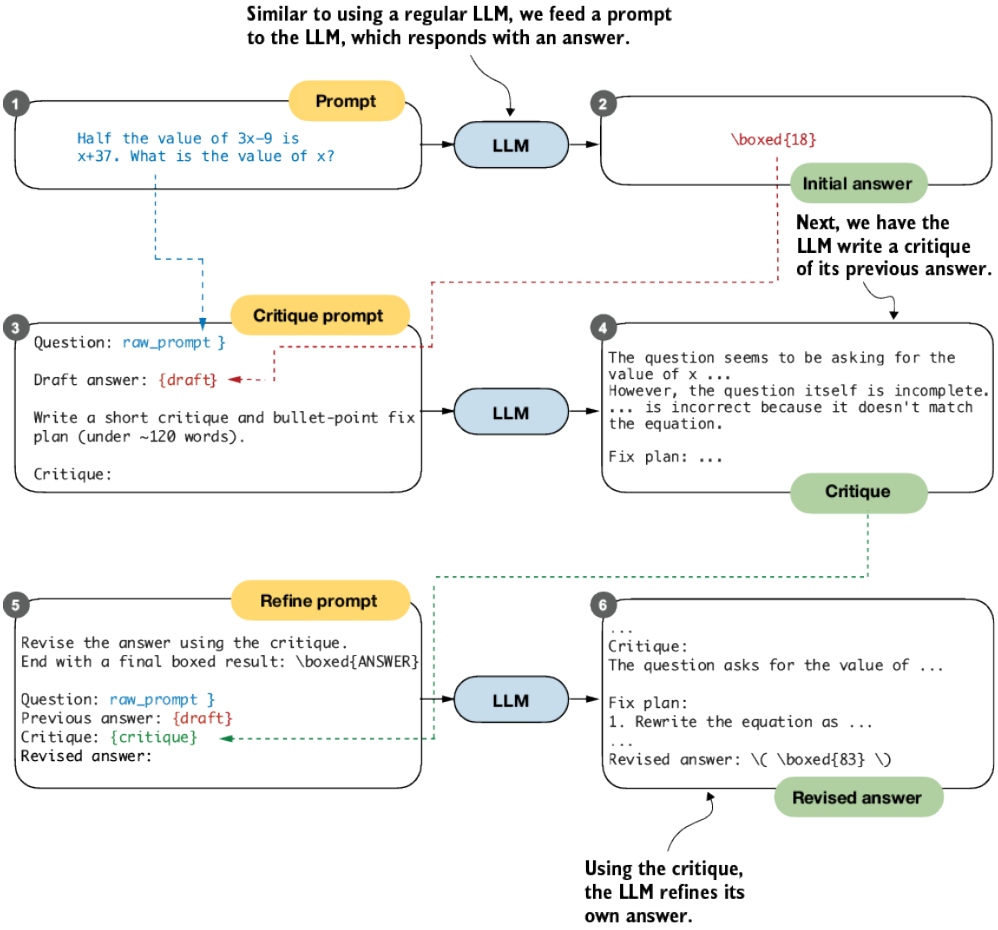


Figure 5.20 The self-refinement process. The LLM first produces an initial answer to the prompt, and then receives a critique prompt that asks it to analyze its own response and produce a short critique with a plan to refine the answer. In the final step, the model is given a refine prompt that contains the original question, its draft answer, and the critique, and it generates a revised answer that incorporates the suggested improvements.

Listing 5.9 Base prompt and answer

```
raw_prompt = (
    "Half the value of $3x-9$ is $x+37$. "
    "What is the value of $x$?"
)
prompt = render_prompt(raw_prompt)

torch.manual_seed(123)
initial_response = generate_text_stream_concat_flex(
    model, tokenizer, prompt, device,
    max_new_tokens=2048, verbose=True,
    generate_func=generate_text_top_p_stream_cache,
    temperature=0.7,
    top_p=0.9,
)
The LLM responds with " \boxed{18}", which is incorrect (the correct answer is 83).
```

Next, we have the LLM critique the answer. To do this, we write a critique prompt that includes the original question (`raw_prompt`) and the answer (`draft`), as shown in steps 3 and 4 in figure 5.20.

Listing 5.10 Critique prompt and refinement plan

```
def make_critique_prompt(raw_prompt, draft):
    return (
        "You are a meticulous reviewer. Identify logical errors, missing "
        "steps, or arithmetic mistakes. If the answer seems correct, "
        "say so briefly. Then propose a concise plan to fix issues.\n\n"
        f"Question:\n{raw_prompt}\n\n"
        f"Draft answer:\n{draft}\n\n"
        "Write a short critique and bullet-point fix plan "
        "(under ~120 words).\n"
        "Critique:"
    )


critique_prompt = make_critique_prompt(raw_prompt, initial_response)
torch.manual_seed(123)
critique = generate_text_stream_concat_flex(
    model, tokenizer, critique_prompt, device,
    max_new_tokens=2048, verbose=True,
    generate_func=generate_text_top_p_stream_cache,
    temperature=0.7,
    top_p=0.9,
)
```

The preceding critique prompt lets the LLM write an elaborate critique that even contains the correct answer itself:

```
 The question seems to have a logical error in its setup. The statement "Half the value of $3x-9$ is $:

Fix Plan:
1. Correct the equation to $\frac{1}{2}(3x-9) = x + 37$.
2. Multiply both sides by 2 to eliminate the fraction: $3x - 9 = 2(x + 37)$.
3. Distribute the 2 on the right side: $3x - 9 = 2x + 74$.
4. Subtract $2x$ from both sides: $x - 9 = 74$.
5. Add 9 to both sides: $x = 83$.
```

Note that the critique can contain factual errors. For instance, it says `" The question itself is incomplete "`, which is not true, yet it still proceeds with a correct plan for a fix. This is important because, in self-refinement, a critique can be useful even if parts of it are wrong, as long as it pushes the model toward a better revision overall.

Finally, we use this refinement prompt to revise the original answer (steps 5 and 6 in figure 5.20).

Listing 5.11 Answer refinement

```
def make_refine_prompt(raw_prompt, draft, critique):
    return (
        "Revise the answer using the critique. Keep it concise and "
        "end with a final boxed result: \\boxed{ANSWER}\n\n"
        f"Question:\n{raw_prompt}\n\n"
        f"Previous answer:\n{draft}\n\n"
        f"Critique:\n{critique}\n\n"
        "Revised answer:"
    )

refine_prompt = make_refine_prompt(raw_prompt, initial_response, critique)
torch.manual_seed(123)
revised_answer = generate_text_stream_concat_flex(
    model, tokenizer, refine_prompt, device,
    max_new_tokens=2048, verbose=True,
    generate_func=generate_text_top_p_stream_cache,
    temperature=0.7,
    top_p=0.9,
)
```

While the base model is not the best instruction follower, the refinement prompt gets the LLM to generate the correct answer:

```
...
 \boxed{83}
Final result: The value of $x$ is \boxed{83}.
```

This example illustrates a single refinement loop. In practice, it's common to repeat this loop for multiple iterations. Next, we'll code a function to automate this process.

5.8 Coding the self-refinement loop

We'll now package the previous section's manual self-refinement steps into a convenient function, simplifying the use of self-refinement and allowing the process to be repeated for a fixed number of iterations.

In addition, the function will support plugging in the scoring methods we developed earlier in the chapter (heuristic scoring and logprob scoring) to compute a score for each answer (figure 5.21). This score can then be used to decide whether a refined answer should be accepted.

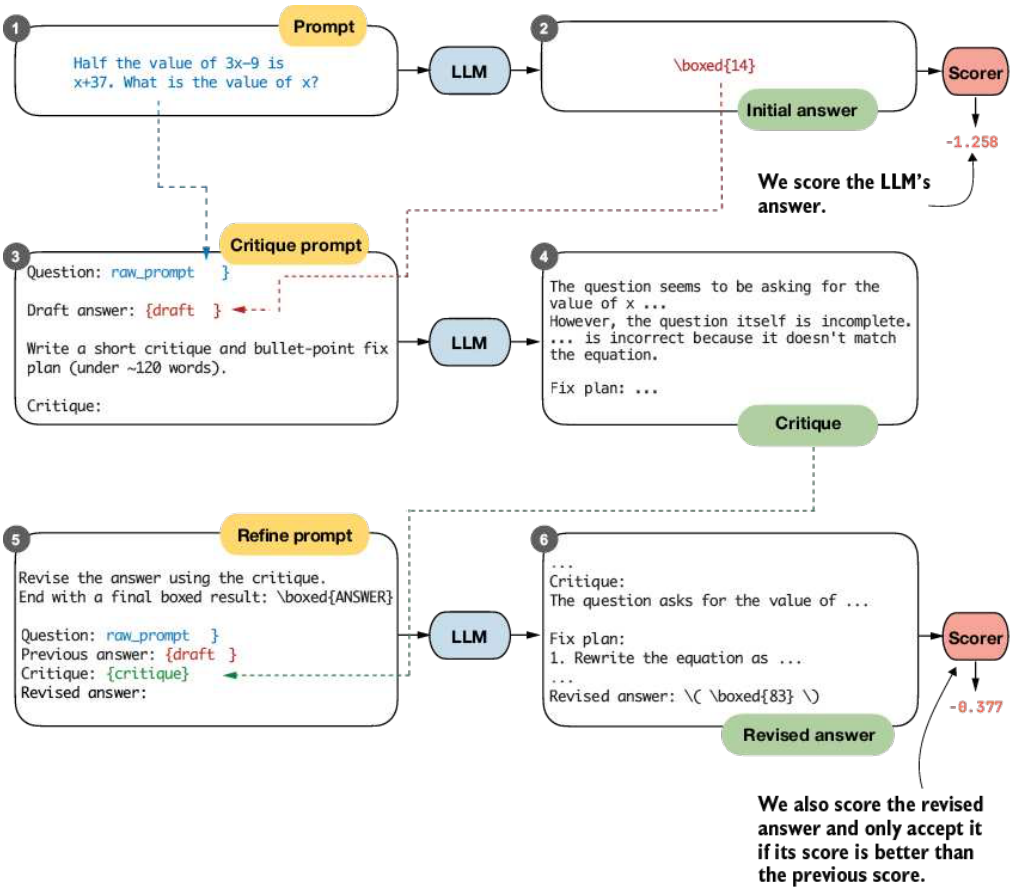


Figure 5.21 The self-refinement loop with optional scoring. The model produces an initial answer, then critiques it, and generates a revised answer based on the critique. Both answers can be evaluated with scoring functions (such as logprob scoring), and the revised answer is only accepted if its score improves on the previous one.

In figure 5.21, the revised answer receives a higher logprob score (`-0.377`) than the initial answer (`-1.258`), which makes it reasonable to accept the revision. In practice, self-refinement can also produce worse answers, especially when it's run for multiple iterations. The scorer therefore provides a way to decide whether a revised answer represents an improvement.

NOTE Scoring does not always improve the results. Whether you should use a scorer in self-refinement, and which one you should use, depends on the LLM. You can determine this by experimenting on a benchmark dataset like MATH-500.

The complete code, which combines the steps from the previous section and adds an `iterations` parameter and a scoring function (`score_fn`) option, is shown in the following listing.

Listing 5.12 Self-refinement with multiple iterations and scoring support

```
def self_refinement_loop(
    model,
    tokenizer,
    raw_prompt,
    device,
    iterations=2,
    max_response_tokens=2048,
    max_critique_tokens=256,
    score_fn=None,
    prompt_renderer=render_prompt,
    prompt_suffix="",
    verbose=False,
    temperature=0.7,
    top_p=0.9,
):
    steps = []

    prompt = prompt_renderer(raw_prompt) + prompt_suffix #1
    current_full = generate_text_stream_concat_flex(
        model=model,
```

```
    tokenizer=tokenizer,
    prompt=prompt,
    device=device,
    max_new_tokens=max_response_tokens,
    verbose=False,
    generate_func=generate_text_top_p_stream_cache,
    temperature=temperature,
    top_p=top_p,
)

current_extracted = extract_final_candidate(
    current_full, fallback="number_then_full"
)
if score_fn:
    current_score = score_fn(answer=current_full, prompt=prompt)
else:
    current_score = 0.0

for it in range(iterations):     #2
    draft_before_full = current_full
    draft_before_extracted = current_extracted
    score_before = current_score

    critique_prompt = make_critique_prompt(        #3
        raw_prompt, draft_before_full
    )
    critique_full = generate_text_stream_concat_flex(
        model=model,
        tokenizer=tokenizer,
        prompt=critique_prompt,
        device=device,
        max_new_tokens=max_critique_tokens,
        verbose=False,
        generate_func=generate_text_top_p_stream_cache,
        temperature=temperature,
        top_p=top_p,
    )

    refine_prompt = make_refine_prompt(         #4
        raw_prompt, draft_before_full, critique_full
    )
    revised_full = generate_text_stream_concat_flex(
        model=model,
        tokenizer=tokenizer,
        prompt=refine_prompt,
        device=device,
        max_new_tokens=max_response_tokens,
        verbose=False,
        generate_func=generate_text_top_p_stream_cache,
        temperature=temperature,
        top_p=top_p,
    )

    revised_extracted = extract_final_candidate(
        revised_full, fallback="number_then_full"
    )
    if score_fn:
        revised_score = score_fn(
            answer=revised_full, prompt=prompt
        )
    else:
        revised_score = 0.0

    step = {         #5
        "iteration": it + 1,
        "draft_full": draft_before_full,
        "draft_extracted": draft_before_extracted,
        "critique": critique_full,
        "revised_full": revised_full,
```

```
            "revised_extracted": revised_extracted,
            "score_before": score_before,
            "score_after": revised_score,
        }
        steps.append(step)

        if verbose:
            print(
                f"[Refinement {it+1}/{iterations}]"
                f"\nCurrent: {draft_before_extracted}"
                f"\nRevised: {revised_extracted}"
                f"\nScore before: {score_before:.3f}"
                f"\nScore after: {revised_score:.3f}"
                f"\n{'=' * 25}\n"
            )

        if revised_score >= current_score:         #6
            current_full = revised_full
            current_extracted = revised_extracted
            current_score = revised_score

    return {
        "final_full": current_full,
        "final_extracted": current_extracted,
        "steps": steps,
    }
```

#1 Initial response (draft)
#2 Run for one or more iterations.
#3 Critique the response.
#4 Refine the response.
#5 Log the results.
#6 Accept revised response if it's not worse.

The `self_refinement_loop` function runs the self-refinement procedure from the previous section for one or more iterations (`for it in range(iterations)`). At the end of each iteration, it compares the revised score to the current score (`revised_score >= current_score`) and keeps the revised answer only if its score is equal or higher.

Using a scoring function is optional. When `score_fn=None` (the default), the score is always set to `0.0`. Since `0.0 >= 0.0` evaluates to `True`, the most recent answer is always accepted when running multiple refinement iterations.

Then we run the self-refinement loop using the average logprob scorer, `avg_logprob_answer`. This function, shown in listing 5.8, takes several arguments (`model`, `tokenizer`, `prompt`, `answer`, and `device`). As you can see in the previous listing, the scoring function is called with only two arguments:

```
# ...
        if score_fn:
            revised_score = score_fn(
                answer=revised_full, prompt=prompt
            )
# ...
```

To make `avg_logprob_answer` compatible with this `score_fn` call, we can use the `partial` function from Python's built-in `functools` module to prespecify the remaining arguments.

Listing 5.13 Creating an average log-probability scorer

```
from functools import partial

avg_logprob_score = partial(
    avg_logprob_answer,
    model=model,
    tokenizer=tokenizer,
    device=device
)
```

Now we can use the `avg_logprob_score` function in the self-refinement loop:

```
torch.manual_seed(1)

results_logprob = self_refinement_loop(
    model=model,
    tokenizer=tokenizer,
    raw_prompt=raw_prompt,
    device=device,
    iterations=2,
    max_response_tokens=2048,
    max_critique_tokens=256,
    score_fn=avg_logprob_score,
    verbose=True,
    temperature=0.7,
    top_p=0.9,
)
```

The output is shown here:

```
[Refinement 1/2]
Current: 10
Revised: 83
Score before: -0.855
Score after: -0.226
========================
[Refinement 2/2]
Current: 83
Revised: 83
Score before: -0.226
Score after: -1.320
========================
```

Looking at the preceding results, we can see that the model corrected the initially incorrect answer (from `10` to `83`) in the first iteration, and the score improved from `-0.855` to `-0.226`. In the second iteration, the score gets worse, but that's okay since we already have the correct answer.

We can also access the final answer extracted from the best-scoring response, typically from its boxed content when present, via `results_logprob["final_extracted"]`, which in this case returns `83`. If you are interested in the full answer, use `results_logprob["final_full"]`. To read through the critiques and longer answers, you can print the `results_logprob` dictionary, which contains the detailed results for all iterations.

EXERCISE 5.5: USING THE HEURISTIC SCORE FOR SELF-REFINEMENT

Run `self_refinement_loop` using the `heuristic_score` function, which we defined in listing 5.3.

The previous example showed that self-refinement can help the model generate the correct answer. To get a better idea of how useful this self-refinement method really is, I ran it on MATH-500 and summarized the results in table 5.1.

Table 5.1 MATH-500 task accuracy for different self-refinement methods

	Method	Scoring	Iterations	Model	Accuracy	Time
1	Baseline (chapter 3)	-	-	Base	15.2%	10.1 min
2	Self-refinement	None	1	Base	25.0%	84.8 min
3	Self-refinement	None	2	Base	22.0%	165.4 min
4	Self-refinement	Heuristic	1	Base	21.6%	84.7 min
5	Self-refinement	Heuristic	2	Base	20.8%	151.4 min
6	Self-refinement	Avg. logprob	1	Base	21.4%	85.3 min
7	Self-refinement	Avg. logprob	2	Base	22.0%	165.3 min
8	Baseline (chapter 3)	-	-	Reasoning	48.2%	182.1 min
9	Self-refinement	None	1	Reasoning	56.6%	498.8 min
10	Self-refinement	Heuristic	1	Reasoning	57.8%	498.6 min
11	Self-refinement	Avg. logprob	1	Reasoning	48.4%	499.7 min

The accuracy values shown in table 5.1 were computed on all 500 samples in the MATH-500 test set using a `" cuda "` GPU (DGX Spark).

As shown in table 5.1, self-refinement improves performance over the base model (rows 1–7), but the improvement is very moderate, with the best accuracy achieved when no scoring is used in self-refinement. This means that both the heuristic score and the average logprob score can sometimes lead to incorrect answers being accepted over the initial correct answer.

Note that when adding `" Explain step by step. "` chain-of-thought to both the base model and the self-refinement, the model failed to improve over the base model (results not shown).

Looking at the reasoning model results (rows 8–11), we can see that the combination of self-refinement and heuristic scoring improved answer accuracy by almost 10%. (As explained in chapter 3, the reasoning model can be used by changing `which_model= " base "` to `which_model= " reasoning "` in `load_model_and_tokenizer` in listing 5.1.)

In both cases, it seems that average logprob scoring results in worse performance than no scorer or the heuristic scorer. This is likely because the average logprob score is more closely related to how natural or expected an answer looks under the model than to whether the answer is actually correct. As a result, it can favor answers that are fluent, familiar, or syntactically clean, even when they are semantically wrong. In other words, the logprob criterion can unintentionally select confident mistakes, whereas the heuristic score is more focused on the format and structure of the answer.

At this point, it may seem that we've spent a lot of time discussing the concept of average logprob scoring. Logprob scoring is a fundamental concept when working with LLMs, and it will come in handy in upcoming chapters, when we implement the reinforcement learning training procedure.

Comparing the results in table 5.1 with those in the previous chapter (table 4.1), we also observe that self-refinement is less effective than self-consistency for this model on this math task. Self-consistency remains widely used because it works well in practice, despite its simplicity.

Another method we haven't explored in this chapter is self-refinement with an external model in an

LLM-as-a-judge setup, similar to what's described in appendix F (section F.5). For instance, instead of using the heuristic score or average logprob, we can use a second LLM to compute a score and write the critique.

In November 2025, with DeepSeekMath-V2, the DeepSeek team demonstrated that self-refinement with a second LLM can be very successful, leading to gold-level performance in several math competitions. Specifically, the team proposed a new method in which they trained a second LLM to be a strong critique model and further trained their base model to become a better math problem solver in the context of self-refinement (by contrast, in this chapter, we applied self-refinement without any additional training).

This concludes our discussion of inference scaling without additional training (see figure 5.22). In the next chapter, we'll begin covering training techniques that update the model weights to improve reasoning.

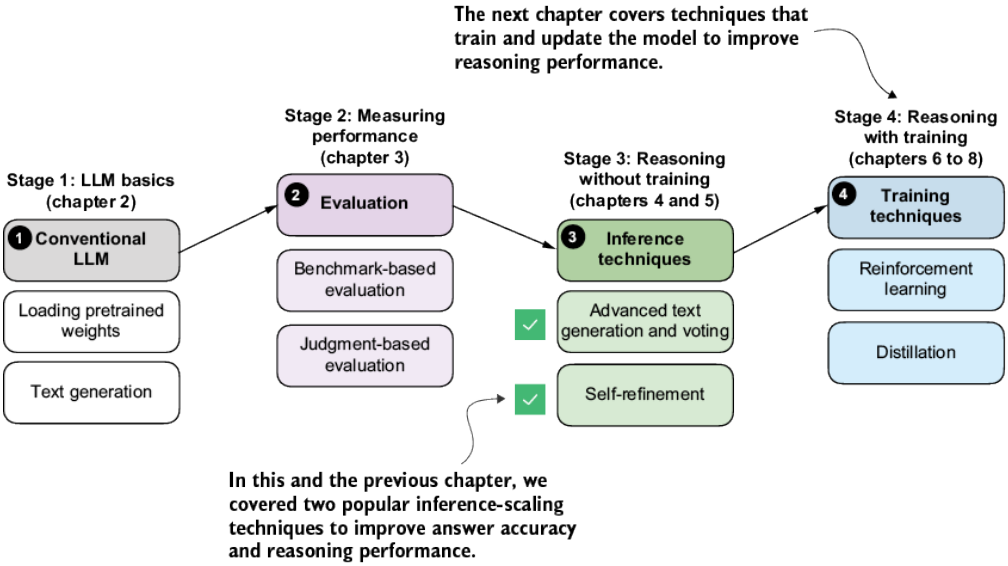


Figure 5.22 Overview of the book's progression from basic LLM use to inference-time reasoning methods and finally to training-based techniques. This chapter concludes our discussion of inference-scaling methods that don't require additional training. The next chapters introduce approaches that update model weights to further improve reasoning performance.

Summary

- Self-refinement extends the inference-time scaling ideas from the previous chapter by iteratively critiquing and improving a single answer, rather than relying on multiple independent samples as in self-consistency.
- A simple rule-based scoring function ranks model outputs by rewarding extractable final answers and shorter, more economical completions.
- Next-token scoring quantifies model confidence by converting logits into normalized probabilities and combining these into a sequence-level likelihood.
- Log probabilities replace raw probabilities to avoid numerical underflow and to turn products over many tokens into stable sums and averages.
- These scoring functions are useful beyond self-refinement, such as for breaking ties in self-consistency or implementing best-of-*N* selection strategies.
- The self-refinement procedure consists of three stages: generating an initial draft, producing a short critique and fix plan, and generating a revised answer.
- A reusable refinement function automates the self-refinement workflow across multiple iterations (refinement rounds), and it can use score-based acceptance to keep only revisions that do not degrade the computed answer's score using one of the scoring functions we developed earlier.

6 Training reasoning models with reinforcement learning

This chapter covers

- The difference between reinforcement learning with human feedback and reinforcement learning with verifiable rewards
- Training reasoning LLMs as a reinforcement learning problem with task-correctness rewards
- Sampling multiple responses per prompt to compute group-relative learning signals
- Updating the LLM weights using group-based policy optimization for improved reasoning

Reasoning performance and answer accuracy can be improved both by increasing the inference compute budget and by using specific model-training methods. This chapter, as shown in figure 6.1, focuses on reinforcement learning (RL), the most commonly used training method for reasoning models.

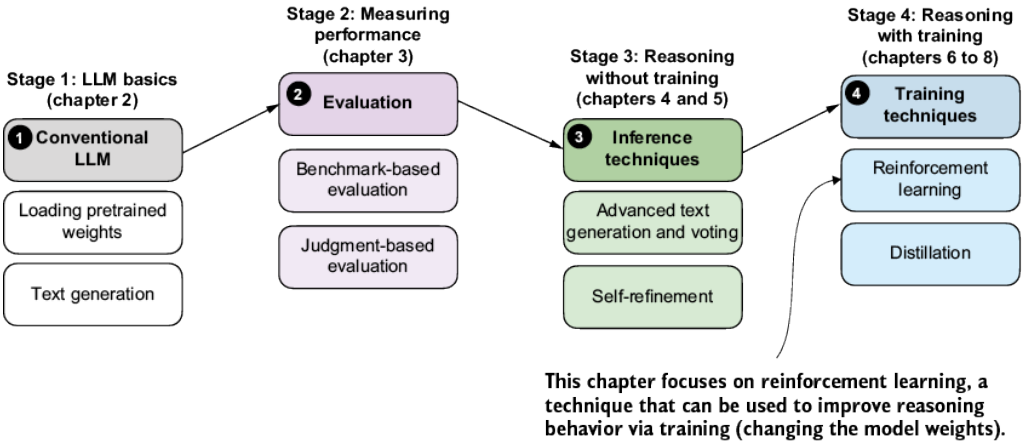


Figure 6.1 A model of the topics covered in this book. This chapter focuses on techniques that improve reasoning with additional training (stage 4). Specifically, this chapter covers reinforcement learning (RL).

We'll first go through a general introduction to RL in the context of LLMs. Then we'll discuss the two most common RL approaches used for LLMs.

6.1 Introduction to RL for LLMs

Inference-time scaling and training-time scaling are two distinct approaches for improving the reasoning performance of LLMs, as illustrated in figure 6.2. Inference-time scaling increases accuracy by spending more computation per generated answer, whereas training-time scaling improves accuracy by investing additional computation during training. This chapter focuses on training-time scaling.

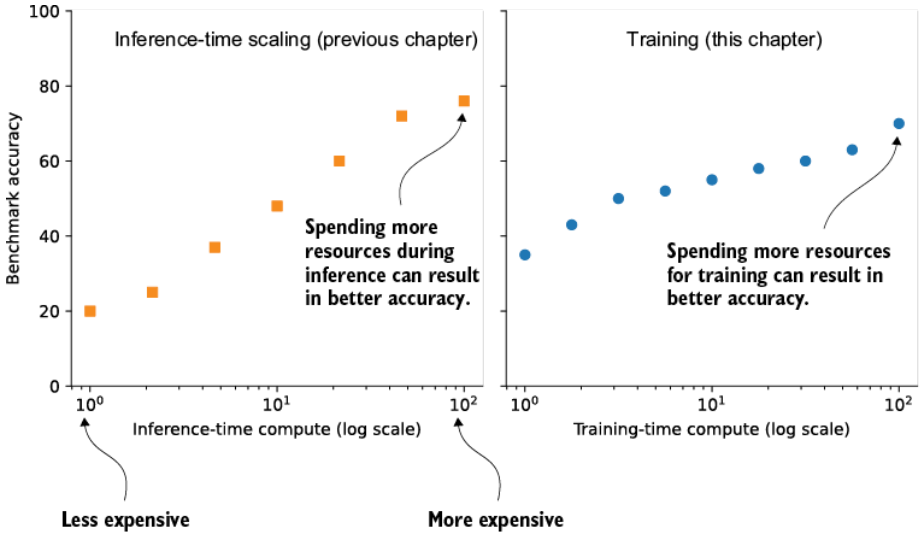


Figure 6.2 Conceptual comparison of inference-time scaling and training-time scaling. Increasing compute at inference improves accuracy by spending more resources per-answer generation, while increasing compute during training improves accuracy by investing more resources up front.

While figure 6.2 presents inference-time scaling and training-time scaling as separate concepts, in practice they can be combined. For example, after improving a model's reasoning capabilities through RL, which is the core focus of this chapter, inference-time-scaling techniques such as those introduced in chapters 4 and 5 can be applied to further boost performance.

RL focuses on how models learn from sequences of actions and their outcomes, with classic examples being agents trained to play chess, Go, or video games. While RL for LLMs builds on this research, it differs in important ways and often resembles familiar LLM-training techniques. Since this book focuses on language models, we will not cover RL in general but instead focus on how RL is applied in LLM training.

So what does RL mean for LLMs in practice? At a high level, RL is typically applied as a post-training stage on top of a pretrained language model, sometimes after instruction fine-tuning. This setup is illustrated in figure 6.3.

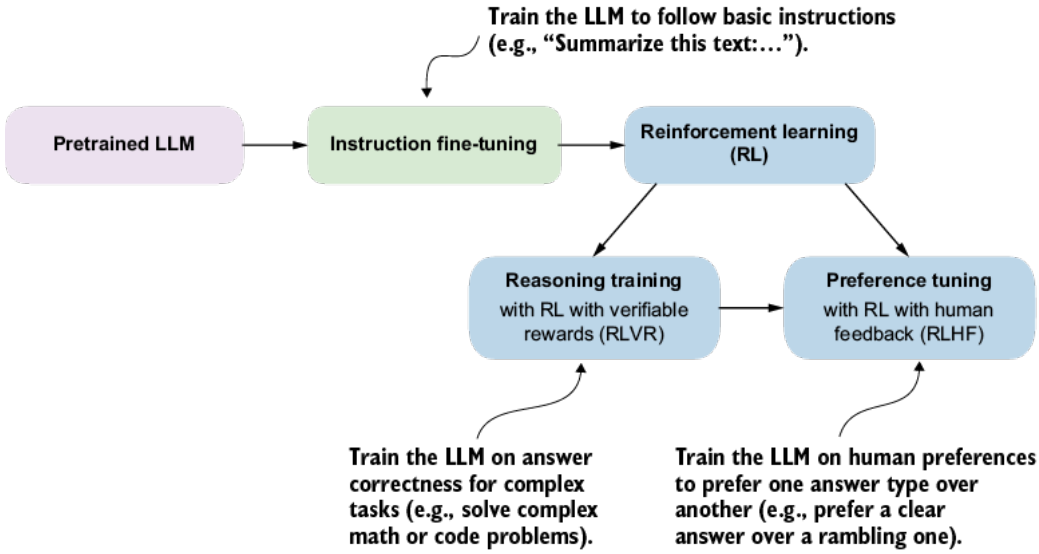


Figure 6.3 Common training stages for LLMs. The ordering of the reasoning training and preference- tuning stages can vary, and some pipelines interleave reasoning and preference tuning rather than strictly sequencing them.

There are two common RL stages for LLMs: *reasoning training* and *preference tuning*. As shown in figure 6.3, RL is usually applied after *instruction fine-tuning*, and preference tuning often follows reasoning training.

As the DeepSeek team demonstrated in its DeepSeek-R1 work, though, reasoning-focused RL can also be applied directly to the pretrained base model, skipping both instruction fine-tuning and preference tuning. The resulting model is generally weaker in reasoning accuracy than one that goes through the full training pipeline, but it still exhibits clear, robust reasoning behavior. More importantly, applying reasoning training directly to the base model avoids mixing the effects of multiple training stages, making it easier to attribute any observed improvements specifically to the reasoning training itself.

Before getting further into the details of how RL is implemented for LLMs, let's briefly compare RL with pretraining at a conceptual level. During pretraining, an LLM is trained to predict the next token in large amounts of text, and many of the model's core abilities and emergent behaviors (such as answering knowledge-based questions and following simple instructions) develop at this stage.

NOTE Pretraining is the main focus of my earlier book, *Build a Large Language Model (From Scratch)*, but familiarity with that material is not required to follow this chapter or the rest of this book.

Applying RL on top of a pretrained model is attractive because it lets us optimize whole outputs, such as answer correctness or preferences, rather than individual tokens and next-token prediction. In this sense, pretraining mainly builds knowledge, while RL shapes how the model uses that knowledge, including its reasoning behavior.

As outlined in figure 6.4, the next two subsections provide a high-level overview of how RL is used in LLM training, and they set the context for the method we'll use in the rest of this chapter.

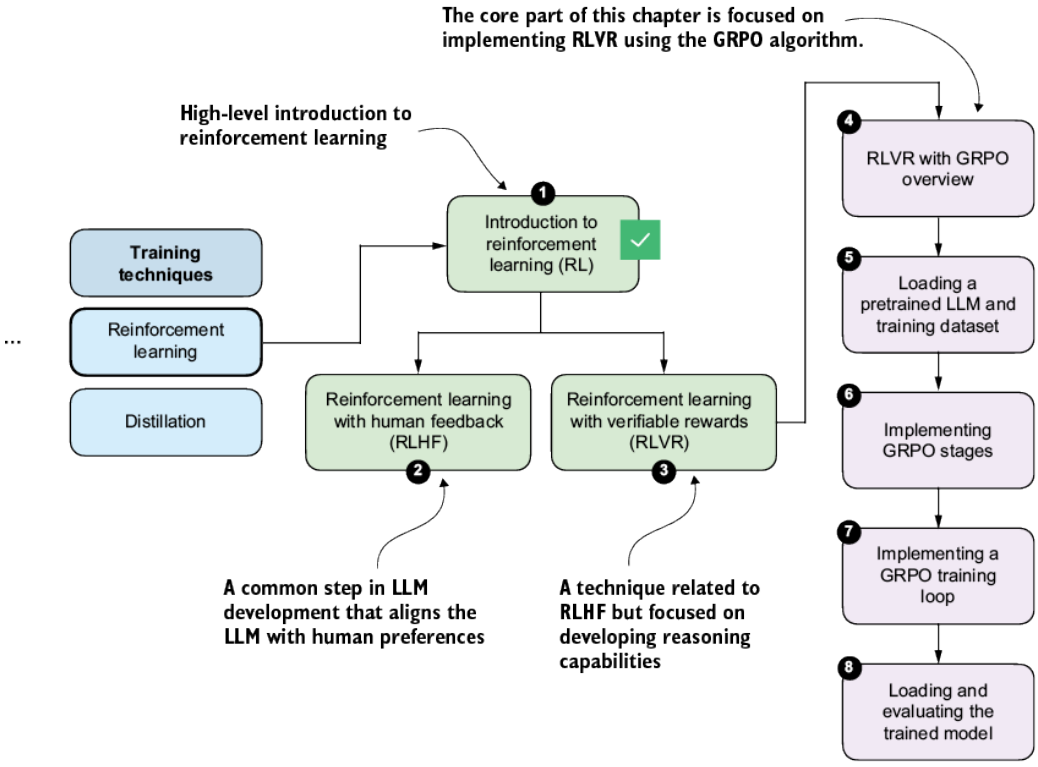


Figure 6.4 Road map of this chapter. After this section's brief introduction to RL for LLMs, we'll discuss the difference between two RL stages, RLHF and RLVR. In the rest of the chapter, we'll focus on implementing RLVR using the GRPO algorithm.

6.1.1 The original RL pipeline with human feedback (RLHF)

In 2022, the InstructGPT paper "Training language models to follow instructions with human feedback" (**https://arxiv.org/abs/2203.02155**) introduced reinforcement learning with human feedback (RLHF), a training approach that uses human preference labels to modify model behavior. In fact, RLHF was a key ingredient in turning GPT-3 into the original model used in ChatGPT, which arguably made LLMs broadly popular in 2022.

At a high level, RLHF is the most popular method for implementing the preference-tuning stage. Before RLHF, LLMs were primarily trained through pretraining and, in some cases, supervised fine-tuning, both of which are based on next-token prediction objectives. RLHF moves beyond this token-level optimization and instead optimizes models on whole outputs (in this case, RLHF optimizes for how humans rank and evaluate LLM outputs).

To make this more concrete, consider the following prompt: "I am looking to buy a laptop for programming and everyday use. What should I consider?"

The model might generate two candidate responses:

- Response A: "You should consider the CPU, RAM, storage, GPU, screen resolution, battery life, key-

board quality, port selection, thermal design, and price."

- Response B: "For programming and everyday use, focus on a fast CPU, at least 16 GB of RAM, and an SSD with at least 256 GB storage. A comfortable keyboard and good battery life also matter. A GPU may only matter if you plan to train small LLMs locally."

Both responses mention relevant points, but a human data annotator might prefer response B because it is more concrete and specific to the use case stated in the prompt. In RLHF, such pairwise preferences are collected across many prompts and used to train the model to favor responses that humans consistently rank higher.

As shown in figure 6.5, RLHF has two main steps: (1) training a reward model (which is itself an LLM), and (2) using that reward model to score the target LLM outputs and fine-tune it.

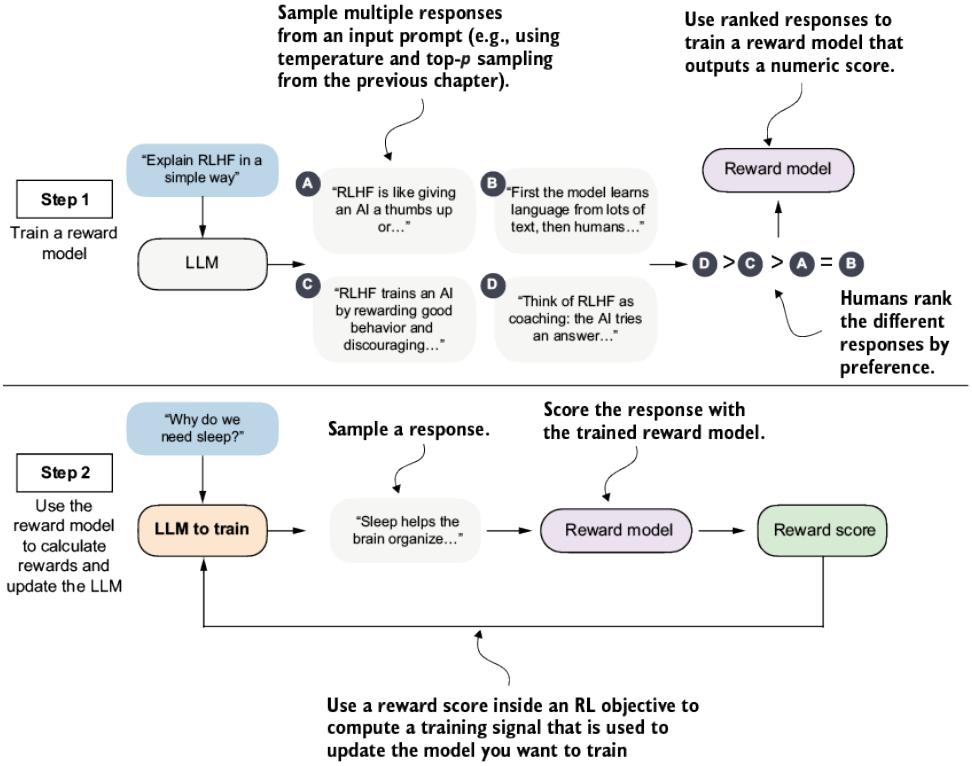


Figure 6.5 A two-stage overview of reinforcement learning with human feedback (RLHF). First, a reward model is trained on human-ranked responses to assign a preference score to each. Second, the LLM is updated using these reward scores within an RL objective to encourage preferred responses and discourage less desirable ones.

The first step in RLHF is training a *reward model*, as illustrated in the top subpanel of figure 6.5. Here, we have the LLM generate multiple responses per prompt (for example, using the temperature scaling and top-*p* filtering approach explained in chapter 4) and ask human annotators to rank the answers from best to worst based on human preference.

These human preferences are converted into training targets for the reward model using a statistical preference model that maps pairwise comparisons to relative quality scores. The reward model, which is usually a separate model initialized from a pretrained LLM, is trained to output a single-number (scalar) reward score for each response that reflects its perceived quality. The idea is that the reward model can automatically score new model outputs, eliminating the need for human annotation at every training step.

The second step in RLHF trains the LLM on the reward scores the reward model assigns to its answers (for example, a bad answer may receive –2, and a good answer may receive +2).

We're discussing RLHF here because it can be viewed as a precursor to reasoning-focused RL methods such as reinforcement learning with verifiable rewards (RLVR). While the source of the training signals for RLHF and RLVR differs, the underlying structure of the pipeline (generating responses, scoring them, and updating the model via RL) is closely related.

6.1.2 From human feedback to verifiable rewards

RLHF can be fairly involved because it requires training an additional reward model, which is often a large LLM that is also expensive to train. Reinforcement learning with verifiable rewards (RLVR) simplifies this setup by replacing the LLM reward model with verifiable rewards, which are computed deterministically and without human annotation. For example, the math verifier introduced in chapter 3 is a verifiable reward generator for math problems. Given a math problem (and a correct solution), a math verifier automatically checks whether a generated solution's final answer matches the ground truth and assigns a corresponding reward: 1 for a correct response and 0 for an incorrect one.

In RLVR, the two-step RLHF pipeline shown in figure 6.5 collapses into a single training loop that resembles step 2 in RLHF: the model generates responses, the verifier assigns rewards, and these rewards are used directly to update the model. This simplified RLVR training procedure is outlined in figure 6.6.

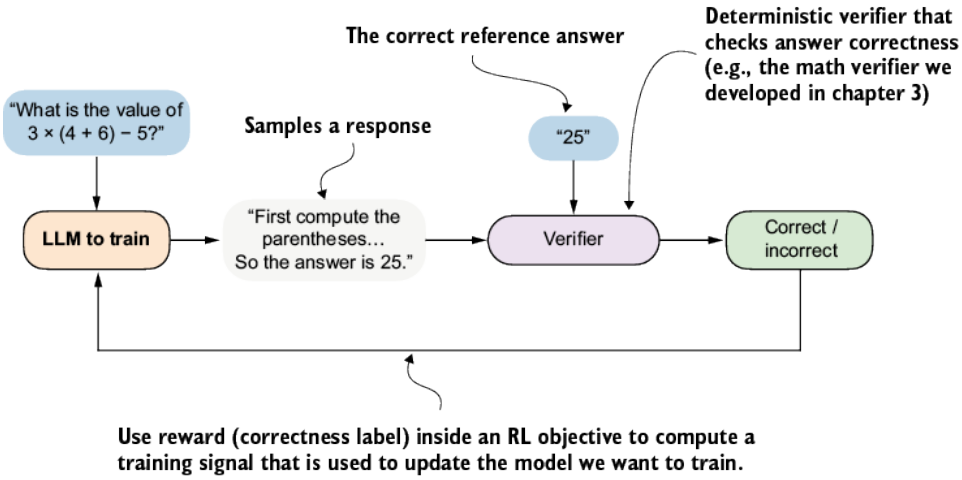


Figure 6.6 Overview of RLVR. The LLM generates a response that is evaluated by a deterministic verifier, such as the math verifier from chapter 3, which assigns a correctness label that's used as a reward signal within an RL objective to update the model.

The popularity of RLVR can largely be traced to the success of DeepSeek-R1 in 2025 (**https://arxiv.org/abs/2501.12948**), which demonstrated that strong reasoning performance can be achieved without relying on human preference data or a learned reward model. DeepSeek-R1 trained reasoning behavior by using automatically verifiable rewards, including correctness checks for math problems (similar to our verifier in chapter 3) and code compilation and execution for code-related tasks.

Code execution is outside the scope of this book because it would require training the LLM in a secure execution environment. That would mean compiling and running model-generated code inside an isolated sandbox so it cannot access the host system, private files, or external services in unsafe ways, making the setup considerably more complex than solving math problems. Either way, the idea behind verifying math tasks (checking whether an answer is correct as a binary label) and code compilation (checking whether code compiles, also as a binary label) is similar, and the RLVR training algorithm would be identical.

To summarize, RLVR offers several practical advantages over RLHF. First, it removes the need to train and maintain a separate reward model, which is often comparable in size and cost to the base LLM.

Second, RLVR requires access to reliable verification signals, which restricts it to certain domains, such as math and code. But verifiable rewards are deterministic and reproducible, meaning they produce the same result every time for the same output, which avoids the noise and inconsistency that arise in human preference annotations.

Finally, RLVR scales naturally to large training sets because, given a reference solution, rewards can be computed automatically for any number of model-generated answers without additional human labeling effort.

6.2 RLVR using GRPO

Now that we've looked at the big picture and seen how RL fits within the overall development cycle of LLMs, we'll turn to the implementation. Specifically, we'll implement RLVR to train a reasoning model, as illustrated in the chapter road map in figure 6.7. This method is similar to DeepSeek-R1-Zero, but we'll apply it at a much smaller scale, using a smaller model and less data, since a comparable full training run

would require several hundred thousand dollars in GPU compute.

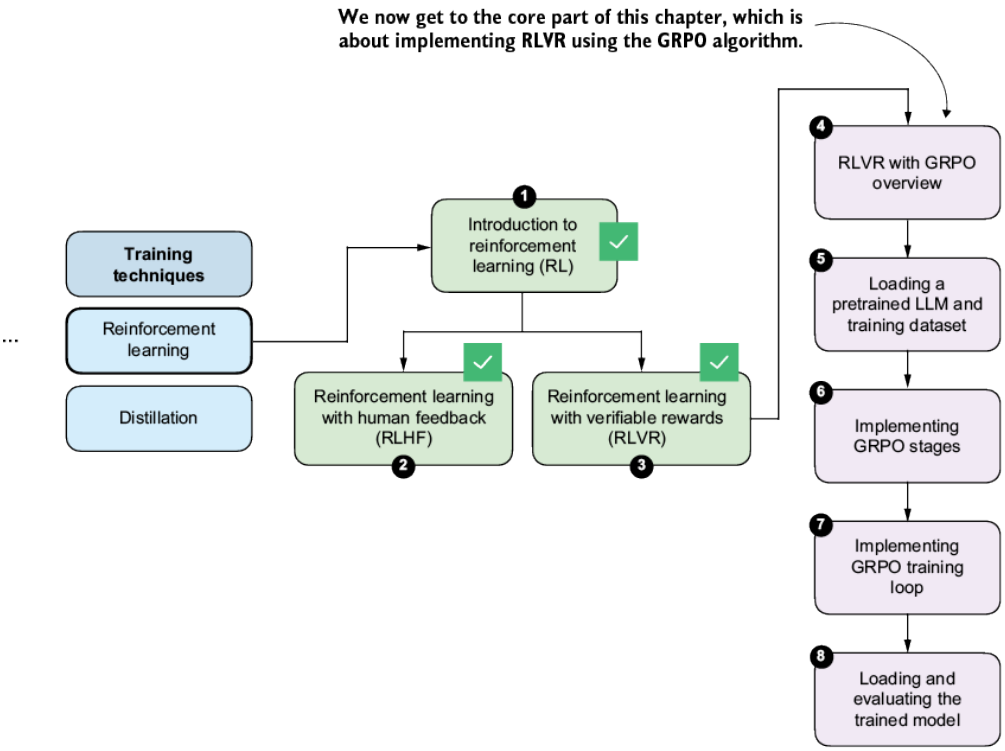


Figure 6.7 We've seen the two main RL approaches for LLMs, RLHF and RLVR. The remaining sections of this chapter focus on implementing RLVR using the GRPO algorithm, from loading the dataset to implementing the full training loop.

RLVR defines the overall training setup, namely, the use of automatically verifiable rewards. In addition, we need a concrete *policy optimization* algorithm that can be used within this RLVR framework to update the model weights and train the model.

The term "policy" is RL-specific jargon and refers, in this context, to the LLM we want to train. Specifically, we'll use the *Group Relative Policy Optimization* (GRPO) algorithm for policy optimization. In short, RLVR determines what learning signal is available, and GRPO determines how that signal is used to update the model weights.

POLICY GRADIENT ALGORITHMS

A widely used policy gradient algorithm for LLMs is *Proximal Policy Optimization* (PPO), which was popularized in RLHF. In principle, the same algorithm could also be applied in an RLVR setting.

When training the DeepSeek-R1 reasoning models, the DeepSeek team opted for a simpler alternative, GRPO, which was originally introduced in their "DeepSeekMath" paper (**https://arxiv.org/abs/2402.03300**). GRPO is more resource friendly than PPO because it doesn't require a separate value model to estimate the value function. Instead, GRPO derives its learning signal from relative comparisons within a group of sampled responses, which substantially reduces computational overhead. If you're interested, you can find a more detailed side-by-side comparison of PPO and GRPO in my article "The State of Reinforcement Learning for LLM Reasoning" (**https://magazine.sebastianraschka.com/p/the-state-of-llm-reasoning-model-training**).

In this chapter, we'll focus on implementing RLVR using GRPO, much like what DeepSeek-R1 and other popular reasoning models use. In the next chapter, we'll build on this foundation and introduce several practical extensions to GRPO that further improve training stability and reasoning performance.

6.2.1 High-level GRPO intuition via a chef analogy

The technical details of the GRPO algorithm for RLVR can initially be a bit overwhelming because it has many moving parts and a lot of domain-specific RL terminology. So before we start with a technical overview and implementation, take a look at figure 6.8, which introduces the technical terms and approach of GRPO through the analogy of a chef preparing a meal.

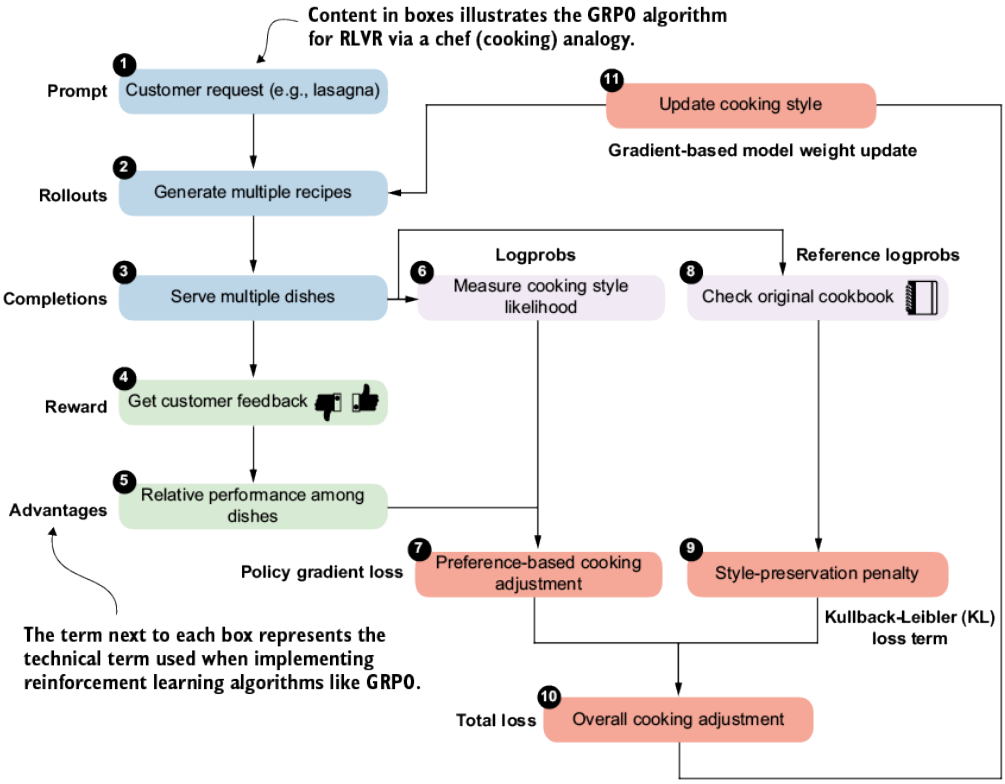


Figure 6.8 Overview of the GRPO algorithm for RLVR using a chef analogy. Multiple rollouts are generated and scored, relative advantages are computed, and a policy gradient objective with a KL-based regularization loss term is used to update the model parameters.

Walking through the flowchart in figure 6.8, imagine we're running a small food delivery service:

1. Each day, we receive a single customer request (the prompt) asking us to prepare a certain dish (for example, lasagna).
2. For that request, we cook our famous lasagna dish, but while doing so, we try out several different recipe variations (the *rollouts*).
3. After preparing the recipes, we create multiple dishes (*completions*). Note that in the LLM setting, a *rollout* refers to the entire generation process for a prompt, while a completion is the resulting output text. In practice, the two terms are often used interchangeably.
4. Customers only give us feedback (the *reward*) after tasting the entire dish, not while it is being prepared. This means we judge the final result as a whole rather than judging the individual cooking steps.
5. We compare how the dishes performed relative to each other based on the customer feedback from stage 4. This helps us identify which recipe variations received higher rewards and which received lower rewards for this particular request.
6. For each completed dish, we also keep track of how typical it was of our current cooking style, that is, how closely it followed our usual techniques and ingredient choices (*logprobs*).
7. Using relative feedback from stage 5 and how characteristic each dish was of our current cooking style from stage 6, we determine how to tweak the cooking style (*policy gradient loss*). Here, we want to reinforce choices that led to better dishes and reduce those that led to worse ones.
8. At the same time, we consult our original cookbook.
9. We then apply a style-preservation penalty that encourages trying new recipe variations but prevents overly drastic changes to our cooking style that could scare away existing customers.
10. The preference-based adjustment and the cooking style-preservation penalty are combined into a single overall measure, or overall loss, that determines how the cooking style should change. The goal is to balance customer satisfaction with consistency.
11. Finally, we update the cooking style (gradient-based model weight update) so that future dishes are more likely to satisfy customer preferences while remaining close to the original cookbook.

Even in this relatively basic analogy, GRPO still has lots of components and moving parts.

6.2.2 The high-level GRPO procedure

Following the chef analogy from the previous section, the flowchart in figure 6.9 provides a technical overview of the GRPO algorithm for RLVR, with concrete values that we will compute as we implement

GRPO step by step.

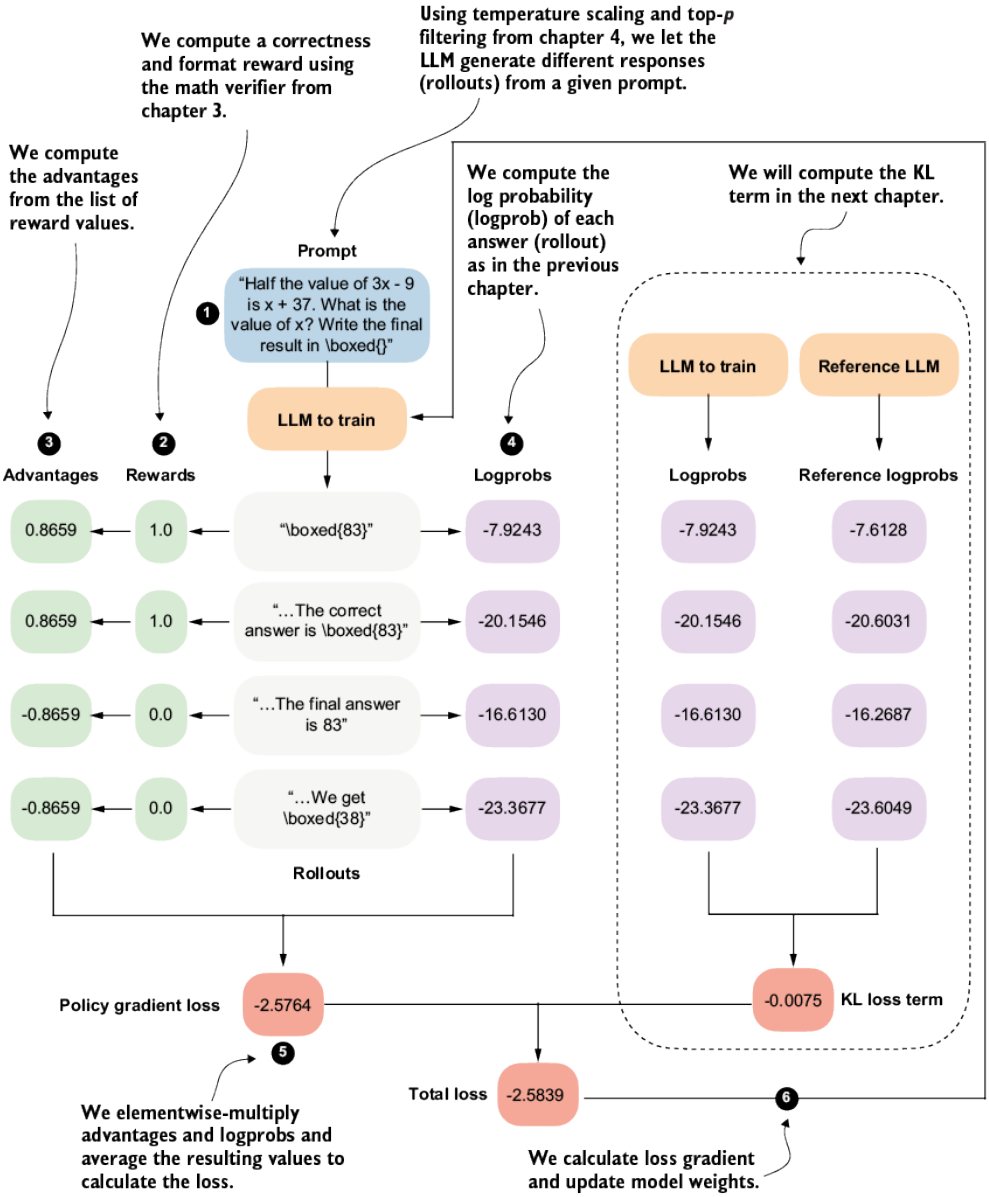


Figure 6.9 Step-by-step GRPO update for RLVR. (1) A prompt is sampled and multiple rollouts are generated. (2) Each rollout is scored using a verifiable reward. (3) Group-relative advantages are computed from these rewards. (4) The log probability of each rollout under the current model is calculated. (5) Advantages and log probabilities are combined to form the policy gradient loss. (6) A KL regularization term against a reference model is added, and the resulting total loss is used to update the model parameters.

The GRPO outline in figure 6.9 contains many technical components. We'll start by implementing a simplified version of GRPO that omits the *KL loss* term shown on the right side of the figure. In chapter 7, we'll add this missing KL loss term for completeness. The KL loss is part of the original GRPO formulation, but it is not strictly essential. Many LLM developers omit it in practice because doing so simplifies implementation and can sometimes improve modeling performance.

NOTE Readers experienced with GRPO or other policy gradient methods may wonder about the use of clipped policy ratios; we'll cover them in the next chapter.

If the overview shown in figure 6.9 looks complicated, don't worry. We will build up the algorithm step by step. For now, it's best to treat figure 6.9 as a high-level road map that will help orient us as we work through the individual pieces.

6.3 Loading a pretrained model

As in previous chapters, we'll begin by loading the pretrained model (see stage 5 in figure 6.10), which we'll then use to generate the rollouts (answers to a prompt) needed for GRPO.

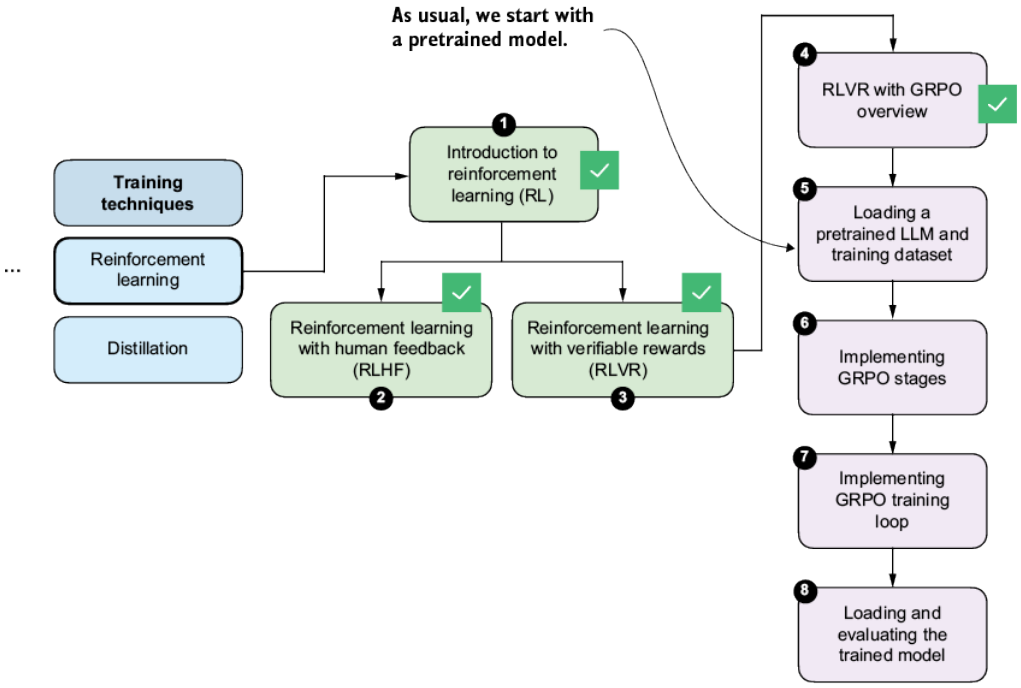


Figure 6.10 In stage 5, we load the pretrained model (this section) and dataset (next section) that we will use for the model training.

The following listing shows the same code we've used previously for loading the tokenizer and base model.

Listing 6.1 Loading the tokenizer and base model

```
import torch
from reasoning_from_scratch.ch02 import get_device
from reasoning_from_scratch.ch03 import (
     load_model_and_tokenizer
)

device = get_device()
device = torch.device("cpu")  #1

model, tokenizer = load_model_and_tokenizer(
    which_model="base",
    device=device,
    use_compile=False
)
```

#1 Delete this line to run the code on a GPU (if supported by your machine).

Note that the preceding code runs on the CPU by default, to ensure you get results that are more consistent with those shown in this chapter. Once you have gone through this chapter in full, I recommend deleting the line `device = torch.device( " cpu " )` and running the chapter code on a GPU.

Next, to ensure that the model is loaded correctly, we'll check it with the temperature and top-*p* sampler code from chapter 4 on a simple math prompt.

Listing 6.2 Generating text with temperature scaling and top-*p* sampling

```
from reasoning_from_scratch.ch03 import render_prompt
from reasoning_from_scratch.ch04 import (
    generate_text_stream_concat_flex,
    generate_text_top_p_stream_cache
)

raw_prompt = (
    "Half the value of $3x-9$ is $x+37$. "
    "What is the value of $x$?"
)
prompt = render_prompt(raw_prompt)

torch.manual_seed(0)
response = generate_text_stream_concat_flex(
    model, tokenizer, prompt, device,
    max_new_tokens=2048, verbose=True,
    generate_func=generate_text_top_p_stream_cache,
    temperature=0.9,
    top_p=0.9
)
print(response)
```

The model prints `" \boxed{58} "`, which is an incorrect answer (`83` is correct), but that's okay because the goal here was merely to make sure that the code runs without issue.

6.4 Loading a MATH training subset

Next, we'll load the dataset that we'll use to train the model with GRPO. We'll use a non-overlapping training subset derived from the original MATH dataset, which means that all MATH-500 evaluation problems are explicitly excluded from the training data. This prevents data leakage and ensures a clean separation between training and evaluation, as illustrated in figure 6.11.

NOTE For details on how this dataset was constructed, see **https://github.com/rasbt/math_full_minus_math500**.

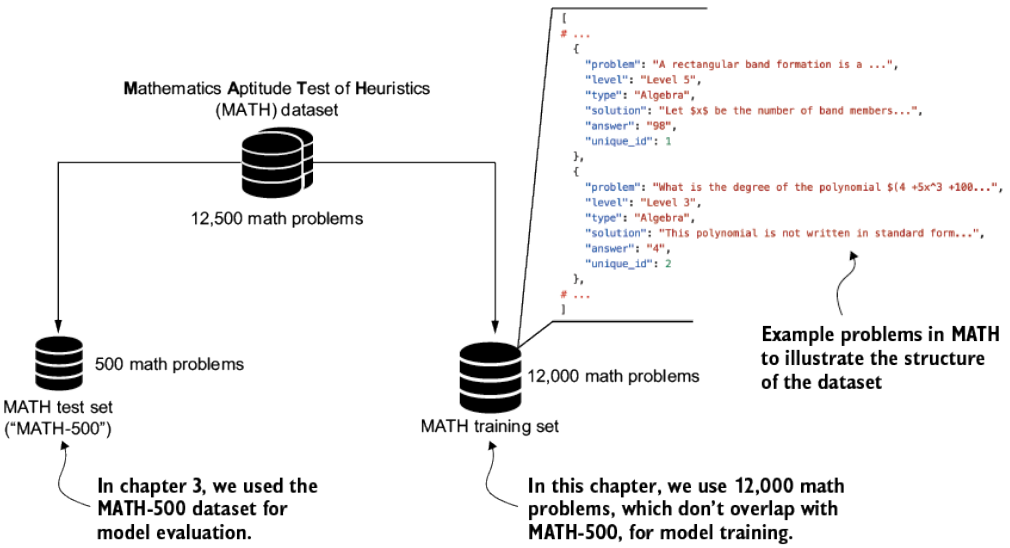


Figure 6.11 Structure and split of the MATH dataset. The full dataset contains about 12,500 problems divided into a 500-problem test set (MATH-500), which we used for model evaluation in chapter 3. A non-overlapping set of 12,000 problems is used for training in this chapter.

To load the 12,000 math problems from the MATH training set shown in figure 6.11, we'll define the following `load_math_train` function. It is similar to the `load_math500_test` function in chapter 3 except that we specify a different file path.

Listing 6.3 Loading the MATH training set

```
import json
import requests
from pathlib import Path

def load_math_train(local_path="math_train.json", save_copy=True):
    local_path = Path(local_path)

    url = (
        "https://raw.githubusercontent.com/rasbt/"
        "math_full_minus_math500/refs/heads/main/"
        "math_full_minus_math500.json"
    )
    backup_url = (  #1
        "https://f001.backblazeb2.com/file/reasoning-from-scratch/"
        "MATH/math_full_minus_math500.json"
    )

    if local_path.exists():   #2
        with local_path.open("r", encoding="utf-8") as f:
            data = json.load(f)
    else:
        try:
            r = requests.get(url, timeout=30)
            r.raise_for_status()
        except requests.RequestException:
            print("Using backup URL.")
            r = requests.get(backup_url, timeout=30)
            r.raise_for_status()

        data = r.json()

        if save_copy:  #3
            with local_path.open("w", encoding="utf-8") as f:
                json.dump(data, f, indent=2)

    return data

math_train = load_math_train()
print("Dataset size:", len(math_train))
```

#1 Alternative URL in case the GitHub-hosted file is unavailable
#2 Load from a local file if it's already downloaded.
#3 Optionally cache a local copy for future runs.

The preceding code should print `" Dataset size: 12000 "` to indicate that the dataset was loaded correctly.

Next, we can print one of the 12,000 entries to get an idea of the dataset structure. For this, we'll use the `pprint` function from the `pprint` standard library because it provides nicer formatting for dictionary entries than the regular `print` function:

```
from pprint import pprint
pprint(math_train[4])
```

This is the resulting entry (index 4 corresponds to the fifth entry in the dataset):

```
{
    "answer": "6",
    "level": "Level 3",
    "problem": (
        "Sam is hired for a 20-day period. On days that he "
        "works, he earns $\\$60. For each day that he does "
        "not work, $\\$30 is subtracted from his earnings. "
        "At the end of the 20-day period, he received "
        "$\\$660. How many days did he not work?"
    ),
    "solution": (
        "Call $x$ the number of days Sam works and $y$ the "
        "number of days he does not... Thus, Sam did not "
        "work for $\\boxed{6}$ days."
    ),
    "type": "Algebra",
    "unique_id": 4,
}
```

As you can see, each dataset example is stored as a dictionary with several fields. For training, the relevant fields are `" problem "`, which serves as the prompt, and `" answer "`, which is the target against which we verify the model's output using the math verifier.

The dataset also includes a full worked solution in the `" solution "` field. While such step-by-step solutions could in principle be used to evaluate intermediate reasoning steps, doing so would unnecessarily constrain the model and risk overfitting to a specific solution and style. Instead, we want to allow the model to explore the solution space more freely.

6.5 Sampling rollouts

Now that we've set up the pretrained base model and loaded the dataset, we are ready to implement the GRPO stages, as illustrated in figure 6.12.

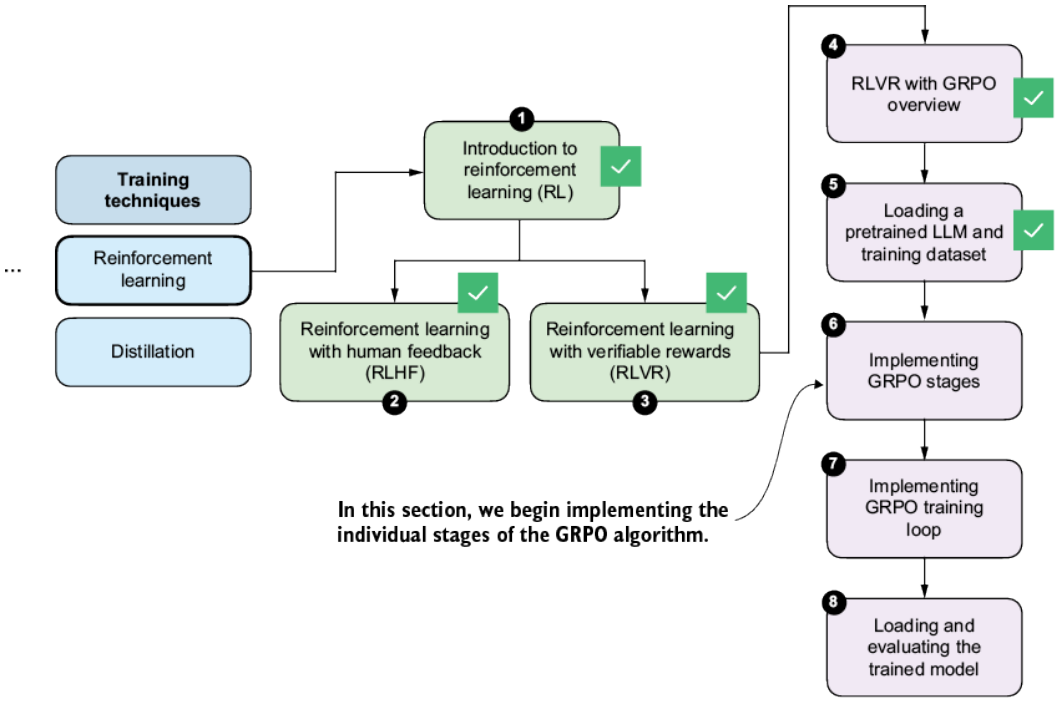


Figure 6.12 This and the following sections implement the GRPO stages needed to train the LLM via verifiable rewards on the MATH dataset.

Figure 6.13 revisits the GRPO overview we introduced earlier in a simplified form that omits the KL loss term, short for Kullback-Leibler divergence loss. We'll add it in the next chapter. We can think of the KL loss as a penalty term that discourages the updated model from drifting too far from a reference model. Figure 6.13 will serve as a simplified road map that we can refer to as we step through the algorithm's components.

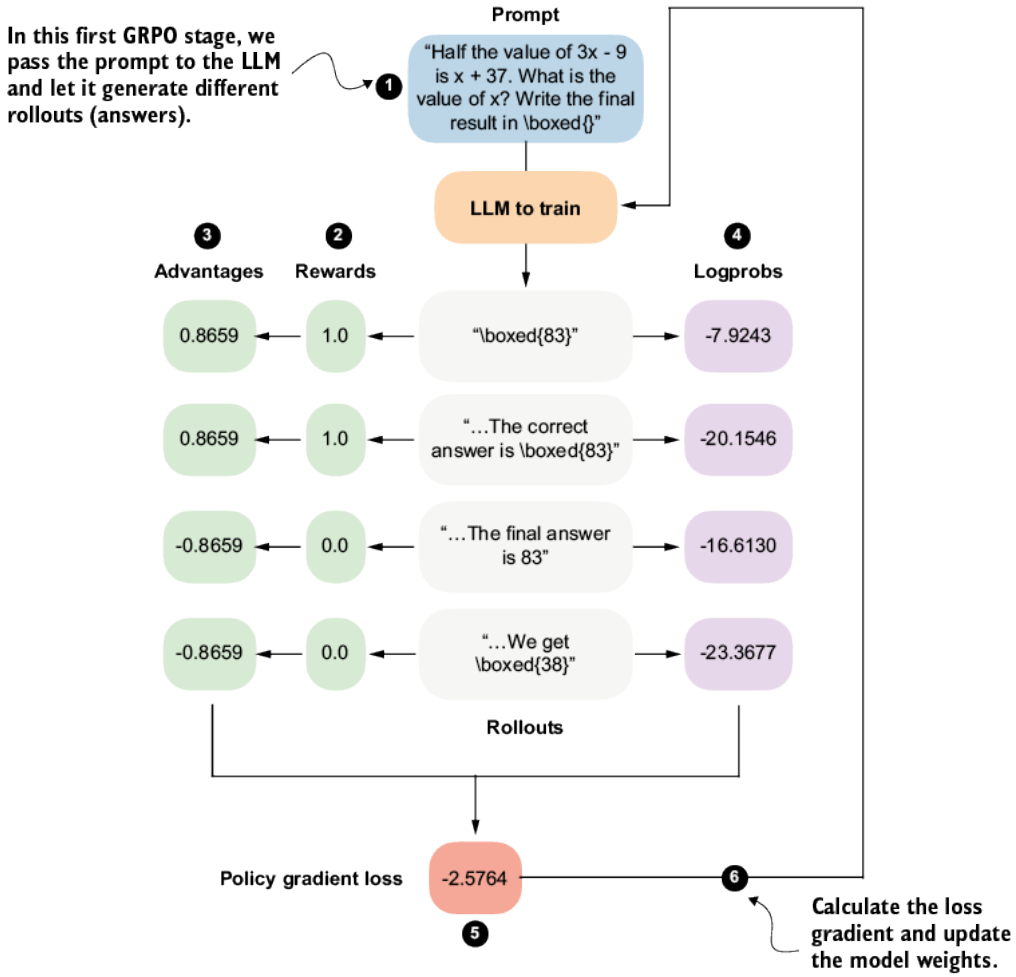


Figure 6.13 Step-by-step GRPO update for RLVR (without the KL loss term). We'll begin by prompting the LLM to generate different rollouts.

As shown in figure 6.13, we'll first use the LLM to generate multiple rollouts. Here, *rollout* is an RL term that simply refers to a complete answer generated by the model for a given prompt.

We could reuse the `generate_text_stream_concat_flex` function from listing 6.2 to generate the rollouts. When we defined that function earlier, though, we decorated it with `@inference_mode`, which disables several PyTorch features for efficiency. Since we will later perform a backward pass, `@inference_mode` makes the function incompatible with our training setup. (Here, "backward pass" refers to the step where PyTorch computes gradients from the loss so that the optimizer can update the model weights.) Instead, we need the `@torch.no_grad` decorator, which disables gradient tracking during the forward pass without switching PyTorch into inference-only mode.

Let's rewrite the function in a more compact form and call it `sample_response`. This function will generate the same text as `generate_text_stream_concat_flex(..., generate_func=generate_text_top_p_stream_cache)`. We can verify this by observing that, with the same `random_seed`, `temperature`, and `top_p` settings, both functions produce identical responses.

Listing 6.4 Defining the rollout generation function

```
from reasoning_from_scratch.qwen3 import KVCache
from reasoning_from_scratch.ch04 import top_p_filter


@torch.no_grad()
def sample_response(
    model,
    tokenizer,
    prompt,
    device,
    max_new_tokens=512,
    temperature=0.8,
    top_p=0.9,
):
    input_ids = torch.tensor(
        tokenizer.encode(prompt),
        device=device
        )

    cache = KVCache(n_layers=model.cfg["n_layers"])  #1
    model.reset_kv_cache()
    logits = model(input_ids.unsqueeze(0), cache=cache)[:, -1]

    generated = []
    for _ in range(max_new_tokens):
        if temperature and temperature != 1.0:  #2
            logits = logits / temperature

        probas = torch.softmax(logits, dim=-1)
        probas = top_p_filter(probas, top_p)  #3
        next_token = torch.multinomial(
            probas.cpu(), num_samples=1
        ).to(device)

        token_id = next_token.item()
        generated.append(token_id)

        if (
            tokenizer.eos_token_id is not None
            and token_id == tokenizer.eos_token_id
        ):
            break

        logits = model(next_token, cache=cache)[:, -1]

    full_token_ids = torch.cat(
        [input_ids,
         torch.tensor(generated, device=device, dtype=input_ids.dtype),]
    )
    return full_token_ids, input_ids.numel(), tokenizer.decode(generated)
```

#1 Cache past keys and values for efficient generation as introduced in chapter 2.
#2 Apply temperature scaling from chapter 4.
#3 Apply top-p filter from chapter 4.

Note that the function now also returns the token IDs of the prompt plus answer tokens, the number of tokens (`input_ids.numel()`), and the answer text (`tokenizer.decode(generated)`). Returning these will allow us to simplify several downstream functions later on.

Otherwise, there is nothing new here. The code is simply a leaner version of what we developed previously; it combines the `generate_text_basic_stream_cache` function from chapter 2 directly with temperature and top-*p* sampling from chapter 4.

Next, we'll call the function on an example prompt, similar to listing 6.2.

Listing 6.5 Generating rollouts with temperature scaling and top-*p* sampling

```
torch.manual_seed(0)

raw_prompt = (
    "Half the value of $3x-9$ is $x+37$. "
    "What is the value of $x$?"
)
prompt = render_prompt(raw_prompt)

token_ids, prompt_len, answer_text = sample_response(
            model=model,
            tokenizer=tokenizer,
            prompt=prompt,
            device=device,
            max_new_tokens=512,
            temperature=0.9,
            top_p=0.9,
        )

print(answer_text)
```

Previously, the `generate_text_stream_concat_flex` sampling function returned `" \boxed{58} "` . Now the `sample_response` function explicitly includes the end-of-sequence token in its response: `" \boxed{58}<|endoftext|> "` . Including this `<|endoftext|>` token is not strictly necessary, but it prevents the model from unlearning how to generate end-of-sequence tokens during very long training runs.

We could call `sample_response` multiple times to generate different rollouts, and we will do this later when implementing the full training loop. To keep this GRPO walk-through example simple and make it easier to follow the step-by-step GRPO outline in figure 6.13, we'll instead assume that the model produced the following four short responses:

```
rollouts = [
    r"\boxed{83}",
    r"The correct answer is \boxed{83}",
    r"The final answer is 83",
    r"We get \boxed{38}",
]
```

6.6 Calculating rewards

The second stage of the GRPO procedure involves computing rewards for each rollout, as shown in figure 6.14.

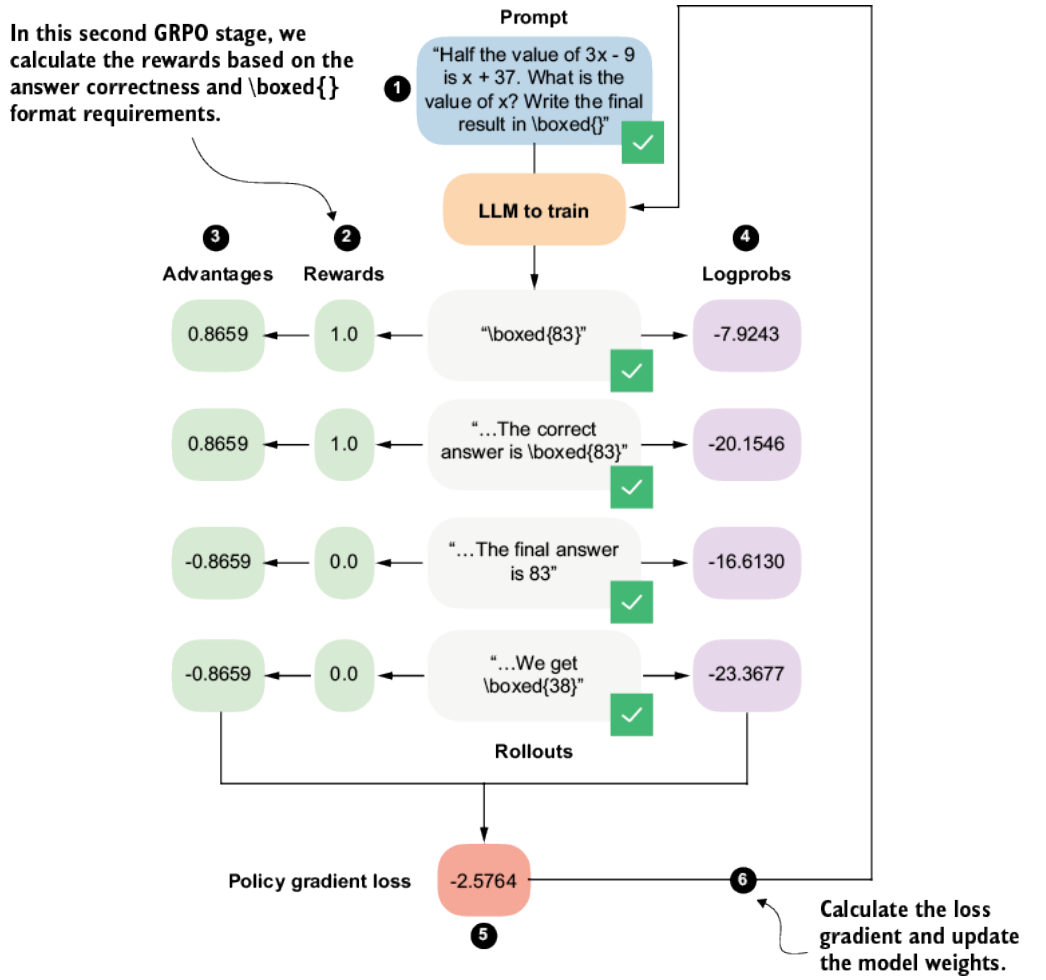


Figure 6.14 The second stage in the GRPO pipeline computes the rewards for each rollout that the LLM generated in the first stage.

The rewards are computed using the math verifier from chapter 3, and they're primarily based on answer correctness. By setting `fallback=None` in the `reward_rlvr` function in the following listing, we also include an implicit format constraint. For instance, an answer only receives a reward of `1.0` if it is both correct and expressed in the required `\boxed{}` format.

Listing 6.6 Implementing the reward function

```
from reasoning_from_scratch.ch03 import (
    extract_final_candidate, grade_answer
)

def reward_rlvr(answer_text, ground_truth):
    extracted = extract_final_candidate(
        answer_text, fallback=None  #1
    )
    if not extracted:
        return 0.0
    correct = grade_answer(extracted, ground_truth)
    return float(correct)
```

#1 fallback=None requires `\boxed{}` format to return an extracted answer.

Since the model receives a nonzero reward only if it answers correctly and writes the final answer in the `\boxed{}` format, we encourage it to learn to produce correct *and* properly formatted answers.

Let's give the `reward_rlvr` function a try and apply it to the rollouts.

Listing 6.7 Applying the reward function to all rollouts

```
rollout_rewards = []

for answer in rollouts:
    reward = reward_rlvr(answer_text=answer, ground_truth="83")
    print(f"Answer: {answer!r}")
    print(f"Reward: {reward}\n")
    rollout_rewards.append(reward)
```

The resulting outputs are as follows:

```
Answer: '\\boxed{83}'
Reward: 1.0

Answer: 'The correct answer is \\boxed{83}'
Reward: 1.0

Answer: 'The final answer is 83'
Reward: 0.0

Answer: 'We get \\boxed{38}'
Reward: 0.0
```

As you can see, the reward function works as intended and only provides a reward of `1.0` if the answer contains the correct result (`83`) and uses the `\boxed{}` format.

Note that the DeepSeek-R1 team also tried to use process reward models to score intermediate solution steps during training. These attempts were unsuccessful, and the researchers concluded that it is better to train only on final-answer correctness rewards, without intermediate rewards.

EXERCISE 6.1: ADDING FORMAT-AWARE REWARD SHAPING

Extend the `reward_rlvr` function from this chapter so that it assigns partial credit based on output format. Specifically, modify the reward function so that it returns `1.0` if the model produces the correct answer in the required `\boxed{}` format, `0.5` if the answer is correct but not boxed, and `0.0` otherwise.

6.7 Preparing learning signals from rollouts via advantages

We'll now move on from rewards to the so-called *advantages*, as shown in figure 6.15. While rewards tell us how well each individual rollout performed, advantages capture how a rollout performed relative to other rollouts generated for the same prompt.

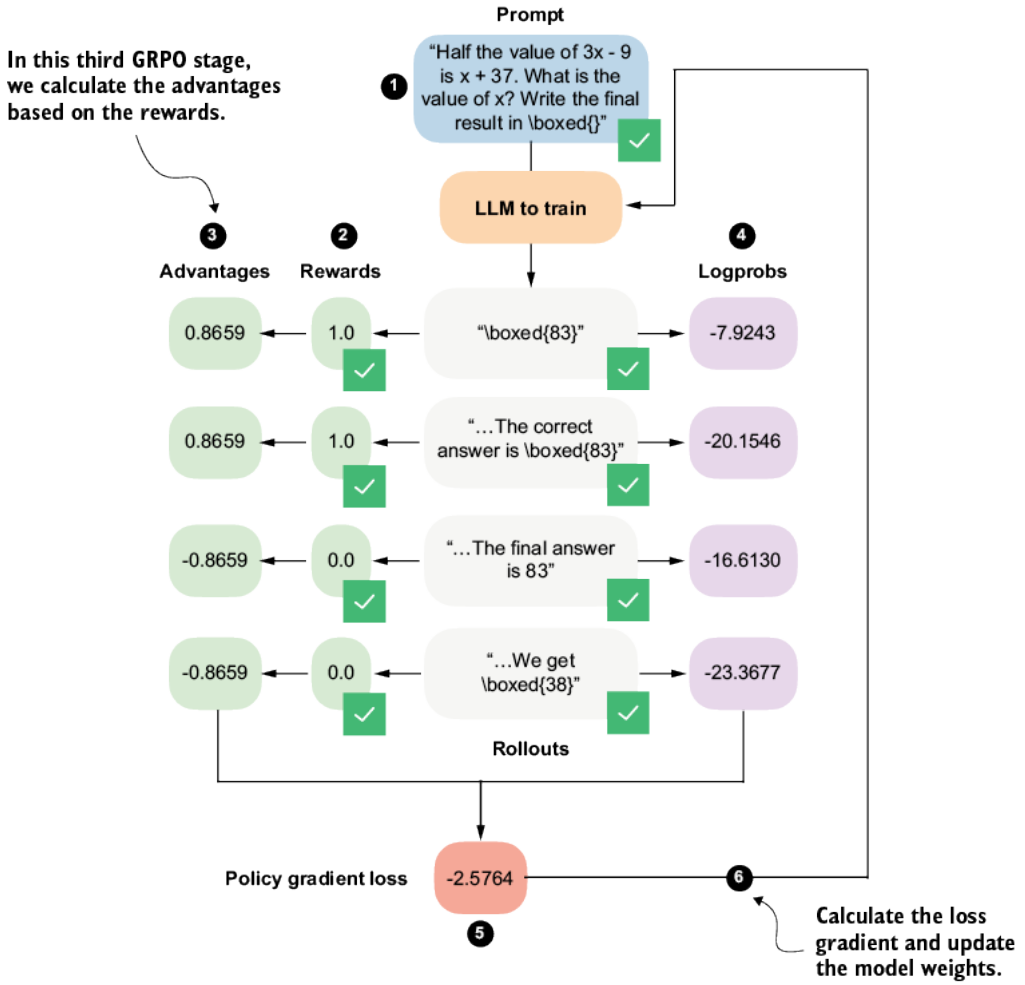


Figure 6.15 The third GRPO stage computes the advantage values from the correctness rewards for the answers (rollouts).

The advantage values shown in figure 6.15 are computed by a simple formula:

$$\text{advantages}_i = \frac{r_i - \mu_r}{\sigma_r + \epsilon}$$

Here

- r_i denotes the reward of the i-th rollout.
- μ_r is the mean reward across the group of rollouts.
- σ_r is the corresponding standard deviation.
- ϵ is a small constant added for numerical stability to avoid division-by-zero errors.

In code, we can implement the advantage calculation as follows.

Listing 6.8 Calculating advantages

```
rewards = torch.tensor(rollout_rewards, device=device)
advantages = (rewards - rewards.mean()) / (rewards.std() + 1e-4)
print(rewards)
print(advantages)
```

The rewards, which we computed in the previous section, are `[1., 1., 0., 0.]`, and the corresponding advantages are `[0.8659, 0.8659, -0.8659, -0.8659]`. These advantage values capture which responses performed better than average and which performed worse than average within the group.

NOTE The "GR" (group relative) in GRPO refers to the fact that GRPO generates multiple answers (rollouts) per prompt and compares them relative to each other to construct a learning signal.

You might still wonder what the point of converting rewards into advantages is. At a practical level, the

advantage values directly scale the gradients during the policy update. If the advantage is positive, the gradient increases the likelihood of the actions that produced that rollout. If it is negative, the gradient decreases their likelihood. Rollouts with advantages close to zero contribute very little.

EXERCISE 6.2: ZERO-ADVANTAGE CASES

Modify the rollout rewards so that all rollouts receive the same reward (for example, force all rewards to `0.0` or all to `1.0`). Run the advantage computation and verify that the resulting GRPO loss is zero. Why is this behavior desirable in Group Relative Policy Optimization?

6.8 Scoring rollouts with sequence log probabilities

We calculated the rewards and then the advantages from them. Now we'll take a separate branch from the rollouts and compute their log probabilities (logprobs), as shown in figure 6.16. Logprobs measure how likely the model considers each generated token to be under its current parameters, as discussed in the previous chapter.

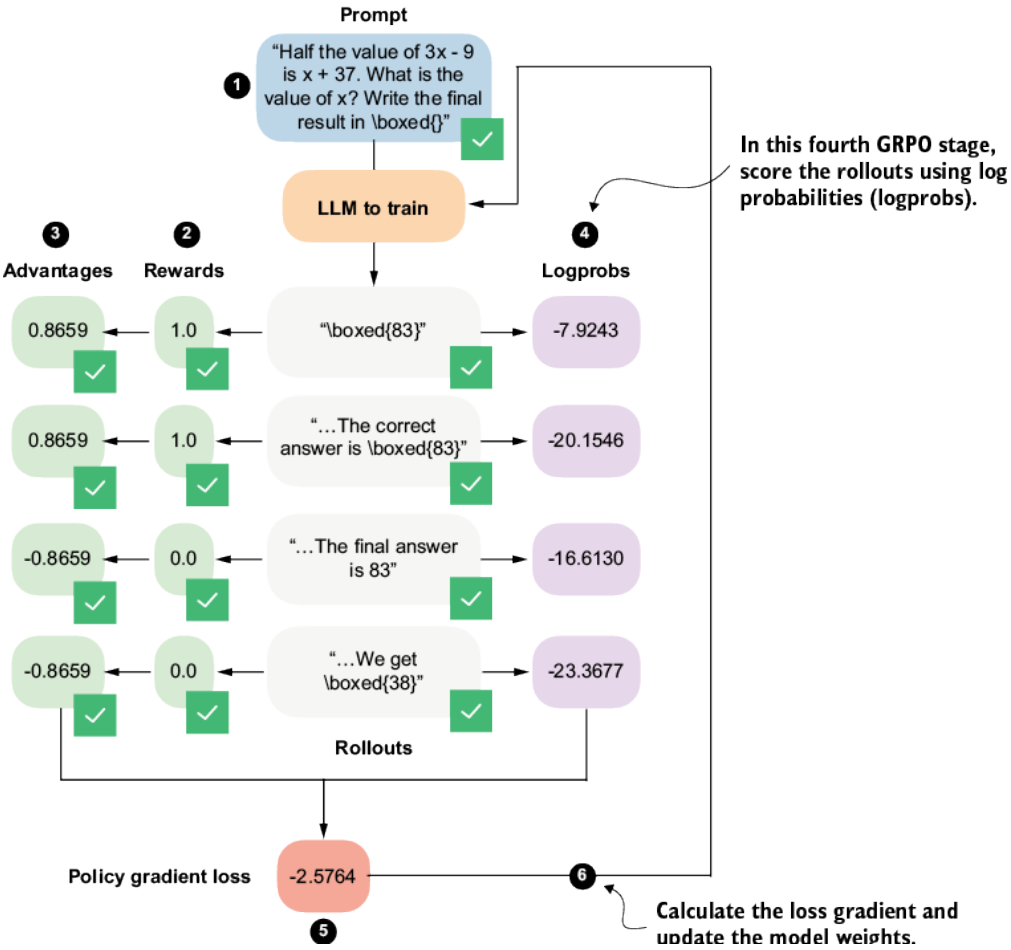


Figure 6.16 The fourth stage in the GRPO pipeline computes log probabilities for each rollout. It is related to the logprob scorer we developed in the previous chapter.

As shown in figure 6.16, these logprobs, together with the advantages, form the core ingredients of the GRPO policy gradient loss, which we will implement later on.

In the previous chapter (listing 5.8), we implemented an `avg_logprob_answer` function that computes per-token logprobs for the model's answer tokens. These values are obtained by evaluating the likelihood the model assigns to each generated token under its current parameters.

When averaged across the response, these values are often referred to as token-level logprobs and are commonly used for scoring LLM outputs. Averaging is preferred in this context because it provides length normalization, which makes the scores comparable across responses of different lengths.

MATHEMATICAL DEFINITION OF TOKEN-LEVEL LOG PROBABILITIES

Mathematically, the token-level logprobs we computed in the previous chapter can be written as

$$\frac{1}{T}\sum_{t=1}^{T}\log p_w(y_t|y_{<t},x)$$

Here

- $y1$, $y2$, ..., yT denote the tokens in the generated response of length T.
- $y<t$ represents all previously generated tokens.
- x is the input prompt.
- W denotes the model's weight parameters.

This expression is mathematically identical to the one used in the previous chapter; we simply switch from x to y to distinguish the generated output tokens from the input prompt.

The `avg_logprob_answer` function from listing 5.8 is repeated and used here to compute the logprobs for the previous `prompt` and `answer_text` for illustration.

Listing 6.9 Computing token-level log probabilities (similar to chapter 5)

```
@torch.inference_mode()
def avg_logprob_answer(model, tokenizer, prompt, answer, device="cpu"):

    prompt_ids = tokenizer.encode(prompt)
    answer_ids = tokenizer.encode(answer)
    full_ids = torch.tensor(prompt_ids + answer_ids, device=device)

    logits = model(full_ids.unsqueeze(0)).squeeze(0)
    logprobs = torch.log_softmax(logits, dim=-1)

    start = len(prompt_ids) - 1
    end = full_ids.shape[0] - 1

    t_idx = torch.arange(start, end, device=device)
    next_tokens = full_ids[start + 1 : end + 1]
    next_token_logps = logprobs[t_idx, next_tokens]

    return torch.mean(next_token_logps).item()


avg_logprob_val = avg_logprob_answer(
                    model, tokenizer,
                    prompt=prompt,
                    answer=answer_text,
                    device=device)
print(avg_logprob_val)
```

This returns `-0.061279296875`.

In GRPO, we do not use averaged token-level logprobs. Instead, we work with sequence-level logprobs. That's because GRPO assigns a single reward and advantage to each rollout, which applies to the entire generated response. Intuitively, logprob should reflect how likely the model was to generate the whole sequence, not an average per token.

We compute sequence-level logprobs by summing the logprobs of all generated tokens. (This sum corresponds to the log likelihood of producing the full response under the current model parameters.) In contrast, averaging token-level logprobs normalizes by sequence length. While this normalization is useful for scoring and comparing responses of different lengths, it would unintentionally rescale the learning signal in policy optimization and cause longer and shorter responses to contribute unevenly to the update. That kind of length-based rescaling is not part of the original GRPO formulation, which instead uses sequence-level logprobs for the whole rollout. We will revisit token-level logprobs in the next chapter

when improving the algorithm.

We can convert the token-level, length-normalized score into a sequence-level logprob by dropping the averaging step and replacing `torch.mean(next_token_logps)` with `torch.sum(next_token_logps)` in the code in listing 6.9.

The same result can also be obtained by multiplying the averaged logprob by the number of answer tokens, as long as the token count matches exactly:

```
sequence_logprob_val = avg_logprob_val * (
    len(tokenizer.encode(answer_text))
)
print(sequence_logprob_val)
```

This now returns `-16.239013671875`.

These sequence-level logprobs scale linearly with the sequence length *T*, which means that longer responses generally receive more negative logprob values. As a result, for two equally good answers, the optimization implicitly favors the shorter one, since it is cheaper in terms of likelihood. Summed logprobs therefore encourage the model to stop earlier unless producing a longer response is justified by a higher reward.

So, as mentioned before, we can replace `torch.mean` with `torch.sum` in the function to obtain sequence-level logprobs. Because we ran the function in inference mode in the previous chapter using the `@torch.inference_mode()` decorator, we need to redefine it without the decorator so that PyTorch can track gradients.

In addition, because `sample_response` from listing 6.4 already returns the `token_ids` and `prompt_len`, we can simplify the implementation by dropping the explicit tokenization step and the construction of `full_ids`.

Listing 6.10 Computing sequence-level log probabilities

```
def sequence_logprob_draft(model, token_ids, prompt_len):
    logits = model(token_ids.unsqueeze(0)).squeeze(0).float()
    logprobs = torch.log_softmax(logits, dim=-1)

    start = prompt_len - 1         #1
    end = token_ids.shape[0] - 1    #1

    t_idx = torch.arange(start, end, device=token_ids.device)
    next_tokens = token_ids[start + 1 : end + 1]
    next_token_logps = logprobs[t_idx, next_tokens]

    return torch.sum(next_token_logps) #2

print(sequence_logprob_draft(model, token_ids, prompt_len))
```

#1 Positions whose next-token probabilities we want to predict
#2 Sum log probabilities over the answer tokens.

This produces the following output:

```
tensor(-16.2998, grad_fn=<SumBackward0>)
```

The result is similar to the `-16.2421875` we got earlier (the difference is due to floating-point behavior and rounding).

PyTorch internally builds a computation graph that records each differentiable operation applied to a tensor. The `SumBackward0` entry shows that the summation of token log probabilities is part of this graph, which means gradients can propagate back through the sequence-level log probability to the mod-

el parameters.

This is exactly what we'll need for policy optimization later, when we implement the training loop to update the model weights via a backward pass. The backward pass applies the backpropagation algorithm, which is the standard method used in deep learning to compute gradients and update neural network weights.

PYTORCH COMPUTATION GRAPHS, GRADIENTS, AND BACKPROPAGATION

As the model produces an output, PyTorch records each mathematical operation that leads from the model parameters to that output. This record is called the *computation graph*. When we later run the backward pass, PyTorch uses this graph to perform backpropagation; that is, it computes gradients that describe how changes to the model's parameters would affect the final result. If an operation is part of this graph, PyTorch can compute gradients through it during training.

Understanding PyTorch's computation graphs is not required for this chapter, but you can learn more, if you're interested, in my tutorial at **https://sebastianraschka.com/teaching/pytorch-1h/#3-seeing-models-as-computation-graphs**.

This draft implementation works, but we can simplify it a bit and make it more efficient for GPUs. In particular, we can avoid explicitly constructing index ranges and instead use `torch.gather` to directly select the logprob corresponding to the generated tokens. The following listing shows a more compact, optimized version of the same computation that produces identical results while being easier to read and faster to execute.

Listing 6.11 Optimized sequence-level log probabilities code

```
def sequence_logprob(model, token_ids, prompt_len):
    logits = model(token_ids.unsqueeze(0)).squeeze(0).float()
    logprobs = torch.log_softmax(logits, dim=-1)
    selected = logprobs[:-1].gather(
        1, token_ids[1:].unsqueeze(-1)
    ).squeeze(-1)
    return torch.sum(selected[prompt_len - 1:])

print(sequence_logprob(model, token_ids, prompt_len))
```

Like the previous code, this returns `tensor(-16.2998, grad_fn=<SumBackward0>)`.

TIP We could compute these logprobs directly in the `sample_response` function, which would improve efficiency by avoiding calling `model(...)` twice, in both the `sample_response` and `sequence_logprob` functions.

Finally, with our robust and efficient sequence-level logprob function in place, we can calculate the respective logprobs of the four different rollouts.

Listing 6.12 Computing sequence-level log probabilities of all rollouts

```
rollouts = [
    r"\boxed{83}",
    r"The correct answer is \boxed{83}",
    r"The final answer is 83",
    r"We get \boxed{38}",
]

rollout_logps = []

for text in rollouts:
    token_ids = tokenizer.encode(prompt + " " + text)
    logprob = sequence_logprob(
        model=model,
        token_ids=torch.tensor(token_ids, device=device),
        prompt_len=prompt_len,
    )

    print(f"Answer:  {text}")
    print(f"Logprob: {logprob.item():.4f}\n")

    rollout_logps.append(logprob)
```

This prints the following:

```
Answer:  \boxed{83}
Logprob: -7.9243

Answer:  The correct answer is \boxed{83}
Logprob: -20.1546

Answer:  The final answer is 83
Logprob: -16.6130

Answer:  We get \boxed{38}
Logprob: -23.3677
```

As you can see, shorter and more concise answers receive higher (less negative) sequence-level logprobs than longer or more verbose responses. The exception is the last answer, which receives the lowest score. Note that this is the only answer that contains the incorrect numerical answer value (`38` instead of `83`), so this also makes intuitive sense.

Overall, this illustrates how summed logprobs naturally favor concise outputs, which is consistent with their role in GRPO when rewards and advantages are applied at the sequence level.

6.9 From advantages to policy updates via the GRPO loss

We'll now implement the fifth stage of the GRPO pipeline, where the previously computed advantages and logprobs are combined into a policy gradient loss. We'll implement the subsequent weight update (stage 6) later as part of the training loop, as shown in figure 6.17.

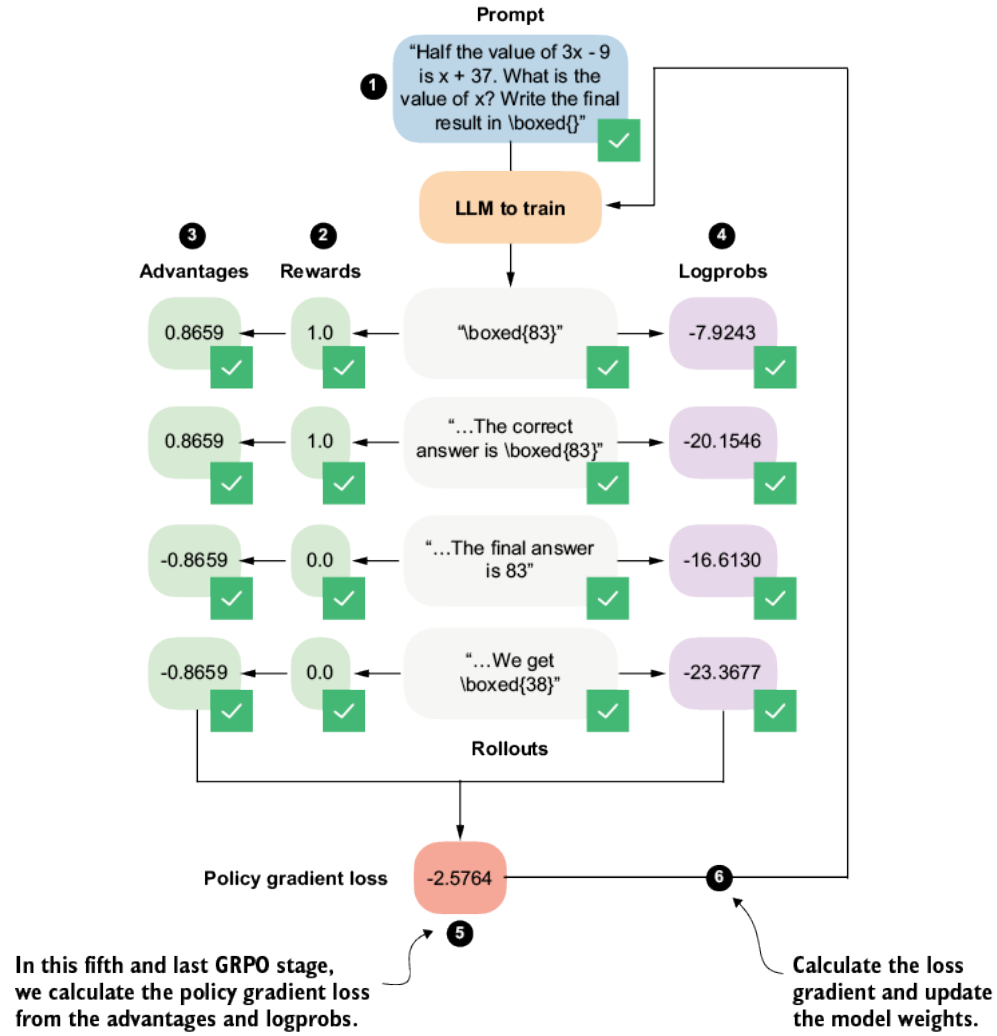


Figure 6.17 The fifth stage in the GRPO pipeline computes the policy gradient loss that we'll use to update the model. Stage 6, the model weight update, will be implemented later as part of the training loop.

First, we'll convert the list of rollout logprobs from the previous section into a PyTorch tensor. Then, we'll compute the policy gradient loss by multiplying each rollout's sequence-level logprob by its corresponding advantage, taking the mean across rollouts, and applying a negative sign. Because PyTorch optimizers are defined to *minimize* a loss, objectives that are naturally written as maximization problems must be sign-flipped.

Listing 6.13 Computing the policy gradient loss

```
logps = torch.stack(rollout_logps)
pg_loss =-(advantages.detach() * logps).mean()
print(logps)
print(pg_loss)
```

This prints the following for the logprobs:

```
tensor([
-7.9243, -20.1546, -16.6130, -23.3677],
grad_fn=<StackBackward0>)
```

The resulting policy gradient loss value is as follows:

```
tensor(-2.5764, grad_fn=<NegBackward0>)
```

Note that we use `.detach()` on the advantages because they are treated as fixed learning signals during the policy update. This prevents gradients from flowing back through the advantage computation, ensuring that only the model parameters influencing the logprobs are updated.

In policy gradient methods, we want to maximize the advantage-weighted logprob of the rollouts, since higher log probabilities for high-advantage rollouts improve the policy. By multiplying this objective by −1, we convert the maximization problem into an equivalent minimization problem. Minimizing the negative objective produces the exact same parameter updates as maximizing the original one, while remaining compatible with PyTorch's optimizer implementations.

MAXIMIZATION VS. MINIMIZATION

To illustrate sign flipping with a concrete example, suppose the objective we want to maximize is the simple scalar value $f(x) = 3$.

Under the maximization view, a larger value is better, so 3 is preferable to 2. PyTorch optimizers minimize, so they would try to *reduce* this value, which is the opposite of what we want.

Now, suppose we flip the sign and define the loss as $L(x) = -f(x) = -3$. Minimizing $L(x)$ is now equivalent to maximizing $f(x)$. For instance, if $L(x)$ decreases from −2 to −3, $f(x)$ increases from 2 to 3.

MATHEMATICAL DEFINITION OF THE POLICY GRADIENT LOSS

If you find it easier to follow the computations via mathematical notation, the policy gradient loss used in GRPO can be written as follows:

$$L_{PG} = -\frac{1}{N}\sum_{i=1}^{N} A_i \sum_{t=1}^{T_i} \log p_w\left(y_t{}^{(i)} | y_{<t}{}^{(i)}, x^{(i)}\right)$$

Here,

- N denotes the number of rollouts.
- $y_t(i)$, ..., $y_{T_i}(i)$ are the tokens of the i-th generated response of length T_i.
- $y_{<t}(i)$ represents all previously generated tokens in that response.
- $x(i)$ is the corresponding input prompt.
- A_i is the advantage of the full rollout.

The inner sum computes the sequence-level log probability of a rollout, while the outer average computes the advantage-weighted log probabilities across rollouts.

6.10 Putting everything together in a single GRPO function

We have now completed the most challenging parts of this chapter and walked through each GRPO stage in isolation. Next, we'll combine the five GRPO stages shown in figure 6.18 into a single `compute_grpo_loss` function, which we'll later use as part of the full training loop. (Note that the term "stage" here refers to a conceptual component of the GRPO loss computation, based on how we stepped through the GRPO algorithm; it's not a training step in the outer optimization loop.)

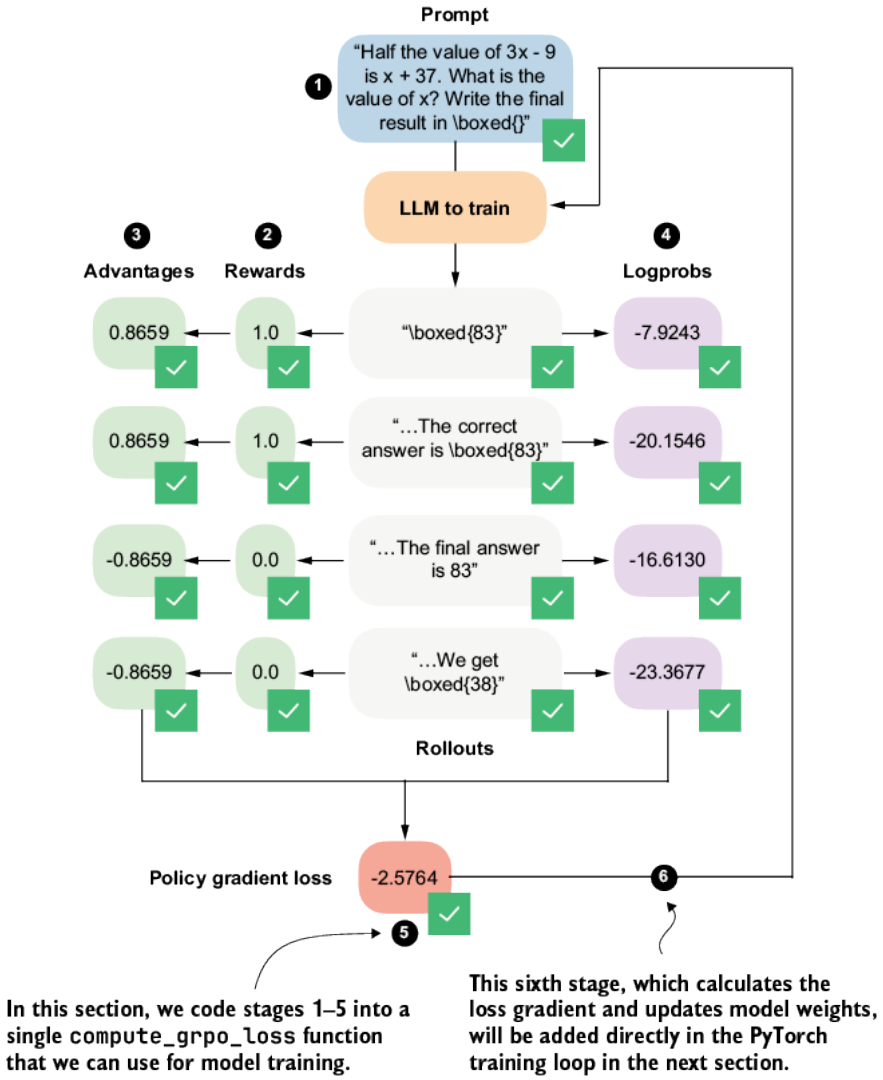


Figure 6.18 The complete GRPO workflow where (1) multiple rollouts are generated for a prompt, (2) they are assigned correctness rewards, (3) they are converted into group-relative advantages, and (4) they are combined with log probabilities to (5) compute the policy gradient loss. The loss gradients (6) will be computed and used to update the model in the next section.

The `compute_grpo_loss` function, which combines all the stages from figure 6.18 that we discussed previously, is shown in the following listing.

Listing 6.14 Combining all the GRPO stages

```
def compute_grpo_loss(
    model,
    tokenizer,
    example,
    device,
    num_rollouts=2,
    max_new_tokens=256,
    temperature=0.8,
    top_p=0.9,
):
    assert num_rollouts >= 2
    roll_logps, roll_rewards, samples = [], [], []
    prompt = render_prompt(example["problem"])

    was_training = model.training
    model.eval()

    for _ in range(num_rollouts):

        token_ids, prompt_len, text = sample_response(#1
            model=model,
            tokenizer=tokenizer,
            prompt=prompt,
            device=device,
```

```
            max_new_tokens=max_new_tokens,
            temperature=temperature,
            top_p=top_p,
        )

        reward = reward_rlvr(text, example["answer"])#2

        logp = sequence_logprob(model, token_ids, prompt_len)#3

        roll_logps.append(logp)
        roll_rewards.append(reward)
        samples.append(
            {
                "text": text,
                "reward": reward,
                "gen_len": token_ids.numel() - prompt_len,
            }
        )

    if was_training:
        model.train()

    rewards = torch.tensor(roll_rewards, device=device)#4

    advantages = (rewards - rewards.mean()) / (rewards.std() + 1e-4)#5

    logps = torch.stack(roll_logps)#6

    pg_loss = -(advantages.detach() * logps).mean()#7
    loss = pg_loss  # In the next chapter we add a KL term here

    return {
        "loss": loss.item(),
        "pg_loss": pg_loss.item(),
        "rewards": roll_rewards,
        "advantages": advantages.detach().cpu().tolist(),
        "samples": samples,
        "loss_tensor": loss,
    }
```

#1 Stage 1: Generate rollouts
#2 Stage 2.1: Compute rewards
#3 Stage 4.1: Compute logprobs
#4 Stage 2.2: Collect all rewards
#5 Stage 3: Compute advantages
#6 Stage 4.2: Collect all logprobs
#7 Stage 5: Compute policy gradient loss

The `compute_grpo_loss` function is relatively self-explanatory because it follows the same stages we implemented manually in the previous sections. Note, though, that after stage 1, the code proceeds directly to stages 2 and 4 rather than stages 3 and 4. This is purely an implementation choice that simplifies the code structure by avoiding multiple nested `for` loops while preserving the same logical sequence of operations.

Also note that the `pg_loss` (policy gradient loss) is identical to the overall loss in our examples. I intentionally omitted the KL loss term here; we'll add it in chapter 7 for completeness. As discussed earlier, GRPO is often reported to perform better on math problems when the KL loss term is omitted, which is why we are adopting this simplified objective here.

Next, let's try the new `compute_grpo_loss` function on an example from the MATH training dataset to ensure that it works. We'll only sample a small number of rollouts (`num_rollouts=2`) and generate a relatively small number of tokens (`max_new_tokens=256`) for testing purposes. It may still take several seconds for the function to return the results.

Listing 6.15 Computing GRPO stages on a MATH training example

```
torch.manual_seed(123)

stats = compute_grpo_loss(
    model=model,
    tokenizer=tokenizer,
    example=math_train[4],
    device=device,
    num_rollouts=2,
    max_new_tokens=256,
    temperature=0.8,
    top_p=0.9
)

pprint(stats)
```

The output is as follows:

```
{'advantages': [0.0, 0.0],
 'loss': -0.0,
 'loss_tensor': tensor(-0., grad_fn=<NegBackward0>),
 'pg_loss': -0.0,
 'rewards': [0.0, 0.0],
 'samples': [{'gen_len': 4, 'reward': 0.0, 'text': ' 14<|endoftext|>'},
             {'gen_len': 256,
              'reward': 0.0,
              'text': ' 4\n'
                      'To solve the problem, let's break it down step by step'
                      '...'}
```

As you can see from the `'rewards'` and `'text'` entries in the `'samples'` field, the model answers incorrectly. As discussed in section 6.4, a correct answer must include `\boxed{6}`. Since this condition is not met, the reward is zero, which in turn yields zero advantages and a zero loss. In this case, if we were training the model, the gradient would be zero and the model parameters would not be updated.

6.11 Implementing the GRPO training loop

We now have all the necessary components in place to implement the full GRPO training loop. In this final section of the chapter, we'll bring everything together to train the model via RLVR using the GRPO algorithm, as illustrated in figure 6.19.

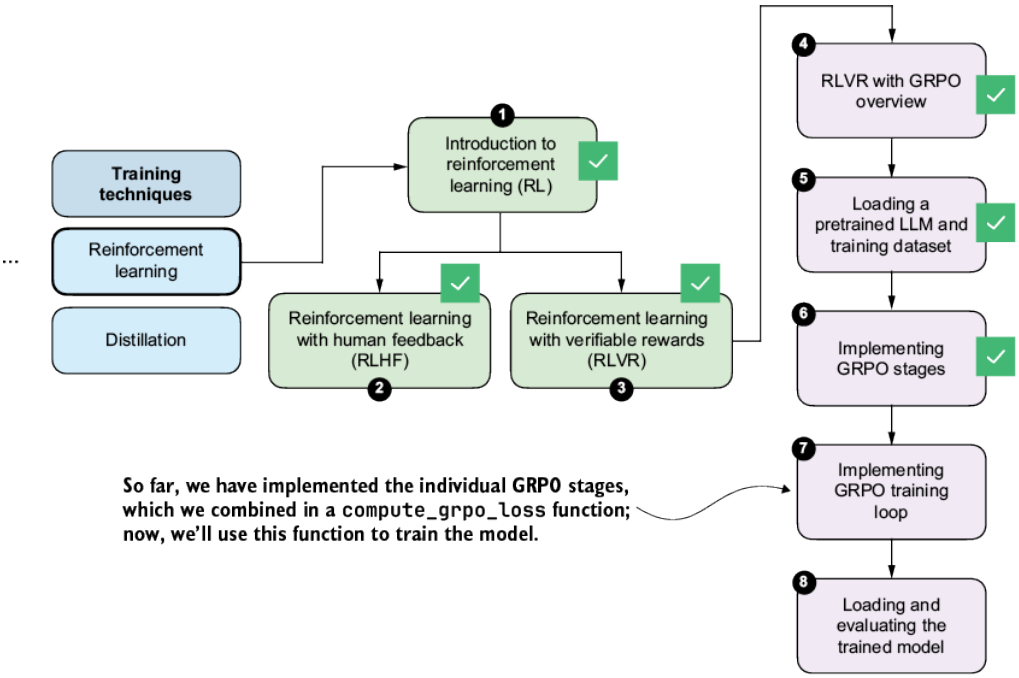


Figure 6.19 We'll now implement the training loop to update the model weights.

The training loop consists of eight main stages, as shown in figure 6.20. Most of these stages are standard components of a typical PyTorch training loop for deep neural networks, including LLMs. The only stage specific to reasoning models is stage 4, where we compute the GRPO loss rather than the standard classification loss used in many other types of deep neural networks and in LLM pretraining.

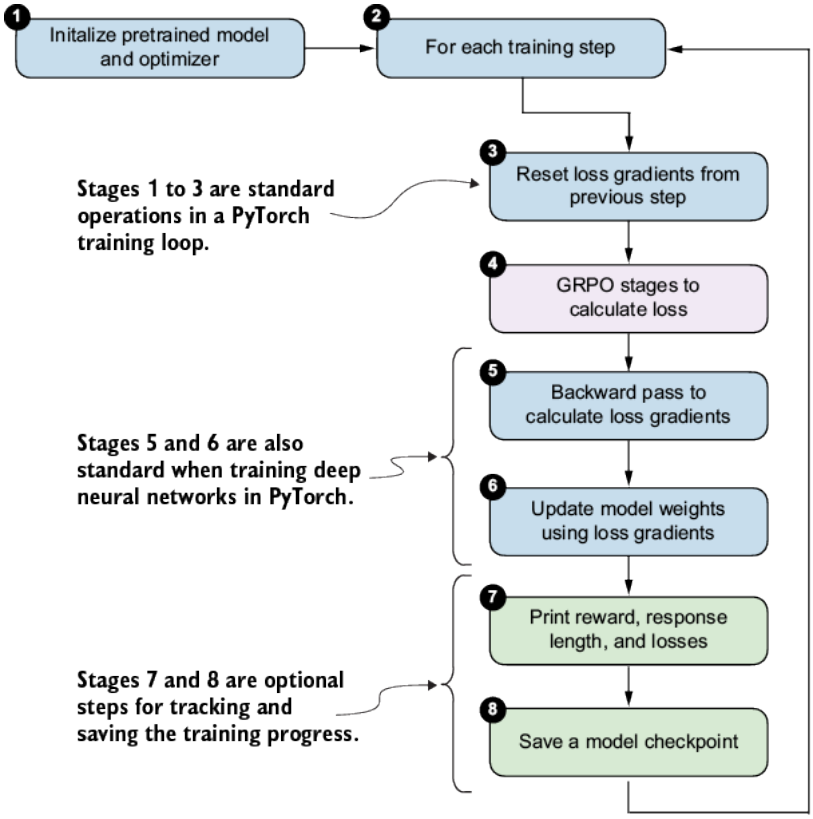


Figure 6.20 Outline of the training loop. The overall structure follows a standard deep learning training loop. The key difference lies in how the loss is computed; instead of a standard supervised objective, the loss is obtained via the GRPO stages (stage 4).

Before discussing the eight stages one by one, it's helpful to first see how they are implemented in code. Listing 6.16 shows the full training loop illustrated in figure 6.20.

Listing 6.16 Implementing the RLVR training loop

```
import time

def train_rlvr_grpo(
    model,
    tokenizer,
    math_data,
    device,
    steps=None,
    num_rollouts=2,
    max_new_tokens=256,
    temperature=0.8,
    top_p=0.9,
    lr=1e-5,
    checkpoint_every=50,
    checkpoint_dir=".",
    csv_log_path=None,

):
    if steps is None:
        steps = len(math_data)


    optimizer = torch.optim.AdamW(model.parameters(), lr=lr)#1
    model.train()
    current_step = 0

    if csv_log_path is None:
        timestamp = time.strftime("%Y%m%d_%H%M%S")
```

```
    csv_log_path = f"train_rlvr_grpo_metrics_{timestamp}.csv"
csv_log_path = Path(csv_log_path)

try:

    for step in range(steps):#2


        optimizer.zero_grad()#3

        current_step = step + 1
        example = math_data[step % len(math_data)]


        stats = compute_grpo_loss(#4
            model=model,
            tokenizer=tokenizer,
            example=example,
            device=device,
            num_rollouts=num_rollouts,
            max_new_tokens=max_new_tokens,
            temperature=temperature,
            top_p=top_p,
        )


        stats["loss_tensor"].backward()#5


        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)#6


        optimizer.step()#7


        reward_avg = torch.tensor(stats["rewards"]).mean().item()#8
        step_tokens = sum(
            sample["gen_len"] for sample in stats["samples"]
        )
        avg_response_len = (
            step_tokens / len(stats["samples"]) if stats["samples"] else 0.0
        )
        append_csv_metrics(
            csv_log_path, current_step, steps,
            stats["loss"], reward_avg, avg_response_len,
        )
        print(
            f"[Step {current_step}/{steps}] "
            f"loss={stats['loss']:.4f} "
            f"reward_avg={reward_avg:.3f} "
            f"avg_resp_len={avg_response_len:.1f}"
        )


        if checkpoint_every and current_step % checkpoint_every == 0:#9
            ckpt_path = save_checkpoint(
                model=model,
                checkpoint_dir=checkpoint_dir,
                step=current_step,
            )
            print(f"Saved checkpoint to {ckpt_path}")


except KeyboardInterrupt:#10
    ckpt_path = save_checkpoint(
        model=model,
        checkpoint_dir=checkpoint_dir,
        step=max(1, current_step),
        suffix="interrupt",
```

```
            )
            print(f"\nKeyboardInterrupt. Saved checkpoint to {ckpt_path}")
            return model

    return model


def save_checkpoint(model, checkpoint_dir, step, suffix=""):
    checkpoint_dir = Path(checkpoint_dir)
    checkpoint_dir.mkdir(parents=True, exist_ok=True)
    suffix = f"-{suffix}" if suffix else ""
    ckpt_path = (
        checkpoint_dir /
        f"qwen3-0.6B-rlvr-grpo-step{step:05d}{suffix}.pth"
    )
    torch.save(model.state_dict(), ckpt_path)
    return ckpt_path


def append_csv_metrics(#11
    csv_log_path, step_idx, total_steps,
    loss, reward_avg, avg_response_len,
):
    if not csv_log_path.exists():
        csv_log_path.write_text(
            "step,total_steps,loss,reward_avg,avg_response_len\n",
            encoding="utf-8",
        )
    with csv_log_path.open("a", encoding="utf-8") as f:
        f.write(
            f"{step_idx},{total_steps},{loss:.6f},{reward_avg:.6f},"
            f"{avg_response_len:.6f}\n"
        )
```

#1 Stage 1: Initialize optimizer (model was already initialized outside the function)
#2 Stage 2: Iterate over training steps
#3 Stage 3: Reset loss gradient (best practice to do this at the beginning of each step)
#4 Stage 4: Calculate GRPO loss
#5 Stage 5: Backward pass to calculate loss gradients
#6 Clip large gradients to improve training stability
#7 Stage 6: Update model weights using loss gradients
#8 Stage 7: Collect rewards, average response lengths, and losses
#9 Stage 8: Save model checkpoint
#10 Save a model checkpoint if we interrupt the training early
#11 Utility function to save the results to a CSV file

This function implements the full RLVR training loop using GRPO. As mentioned earlier, except for stage 4 (the GRPO loss computation), the structure mirrors a conventional PyTorch training loop where we reset gradients, backpropagate a loss, optionally clip gradients, update parameters via an optimizer step, log metrics, and periodically save checkpoints. (We save the model checkpoints, which are snapshots of the model weights, so we can load, evaluate, and use them later, as we'll discuss in the next section.)

TIP For a general introduction to training neural networks in PyTorch, see sections 3-8 of my "PyTorch in One Hour: From Tensors to Training Neural Networks on Multiple GPUs" article (**https://sebastianraschka.com/teaching/pytorch-1h/**).

The key difference here, compared with other standard training loops, is how the loss is constructed. Instead of a standard supervised objective, which is used for most neural networks, including classifiers and for pretraining LLMs, the training signal is computed via the GRPO algorithm as part of the RLVR procedure.

Let's now run the code and train the model. Earlier, we hardcoded the PyTorch device to `" cpu "` so that all intermediate results closely match those shown in the book, since `" mps "` and `" cuda "` devices can introduce small floating-point differences. Training on a CPU is very slow, though.

For convenience, the following listing includes two lines that automatically select and activate the appro-

priate device before the `train_rlvr_grpo()` function call. For instance, it will automatically switch to `" cuda "` or `" mps "` simply by running the code in the following listing.

Listing 6.17 Training the model

```
device = get_device()
model.to(device)

torch.manual_seed(0)

train_rlvr_grpo(
    model=model,
    tokenizer=tokenizer,
    math_data=math_train,
    device=device,
    steps=50,
    num_rollouts=4,
    max_new_tokens=512,
    temperature=0.8,
    top_p=0.9,
    lr=1e-5,
    checkpoint_every=5,
    checkpoint_dir=".",
)
```

Let's briefly talk about these settings before we inspect the results. The `temperature` and `top_p` settings were set within a common range that encourages some diversity in the answers while still producing coherent text for this model. You can change these settings and see how they affect the results (commonly used ranges are 0.7-0.9).

The number of steps is relatively small, at 50, even though the dataset contains 12,000 entries. This is simply to keep the runtime reasonable, and it can be increased. Even with 50 steps, though, it is sufficient for good results.

The learning rate (`lr`) can be tweaked, but it is in a reasonable range and works well.

The number of rollouts (`num_rollouts=4`) and allowed tokens (`max_new_tokens=512`) are relatively small to reduce resource requirements. If you experience out-of-memory errors, you can further reduce these values, such as by setting `num_rollouts=2` and `max_new_tokens=64` , for testing purposes.

With the current `checkpoint_every=5` setting, the model is saved every 5 steps, and each checkpoint requires approximately 1.5 GB of disk space. For longer runs, I recommend setting this number to 50 or 100. Here, it is purposefully small for testing. Also note that an additional checkpoint should be created if the training run is manually interrupted, such as by interrupting the Jupyter notebook execution with the `" Interrupt the kernel "` button.

Let's take a brief look at the run's output (where I interrupted the 50-step run at step 7):

```
Using Apple Silicon GPU (MPS)
[Step 1/50] loss=-0.0000 reward_avg=0.000 avg_resp_len=88.0
[Step 2/50] loss=-0.0000 reward_avg=0.000 avg_resp_len=7.5
[Step 3/50] loss=-0.0000 reward_avg=0.000 avg_resp_len=6.5
[Step 4/50] loss=0.0909 reward_avg=0.250 avg_resp_len=6.5
[Step 5/50] loss=1.1001 reward_avg=0.500 avg_resp_len=300.5
Saved checkpoint to qwen3-0.6B-rlvr-grpo-step00005.pth

KeyboardInterrupt. Saved checkpoint to qwen3-0.6B-rlvr-grpo-step00006-interrupt.pth
```

You can see that both the loss and reward values fluctuate, which is normal in RL training.

We've printed the average reward, which indicates the proportion of sampled responses that are correct. With our `num_rollouts=4` setting, a `reward_avg` of 0.5 means that 2 of the 4 rollouts received a posi-

tive reward. Similarly, values of 0.25 and 0.0 indicate one or zero correct responses, respectively.

The loss values should not be over-interpreted. Steps with `reward_avg=0.000` produce a near-zero loss because all rollouts receive the same reward, resulting in vanishing group-relative advantages and little to no gradient signal. Larger loss magnitudes, such as at step 3 (see row 4 in the preceding output), simply reflect bigger relative differences between rollouts and are typical for GRPO-style objectives, especially at the beginning of training.

Ideally, we want to see two main trends over time:

1. The average reward should increase when averaged over many steps, as the model learns to produce more accurate responses.
2. The reasoning accuracy should improve (we'll evaluate this in the next section).

The online code for the book contains a slightly more sophisticated version of the script that also includes an option to evaluate the model periodically on subsets of the MATH-500 test dataset to see whether the model is improving on the target task: **https://github.com/rasbt/reasoning-from-scratch/tree/main/ch06/02_rlvr_grpo_scripts_intro**.

BATCHED TRAINING

RL can be relatively resource-intensive because we have to generate multiple (long) rollouts via the LLMs for each training step. For this reason, the implementation does not support batching.

If you have access to multiple GPUs, you can use the optional version of this code, with batch and multi-GPU support: **https://github.com/rasbt/reasoning-from-scratch/tree/main/ch06/02_rlvr_grpo_scripts_intro**. This version trains the model faster.

6.12 Loading and evaluating saved model checkpoints

Now we'll discuss how to load the saved checkpoints from the RLVR training run in the previous section (see figure 6.21). We'll also evaluate the model on the MATH-500 dataset.

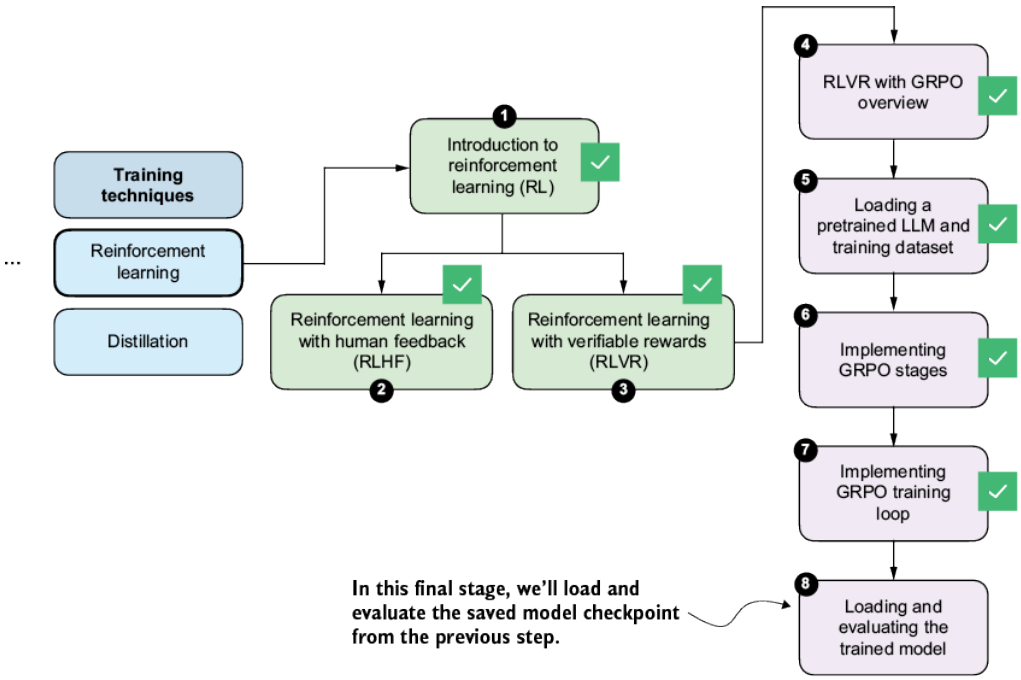


Figure 6.21 We'll now load the saved model checkpoints and evaluate them.

The saved checkpoints can be loaded using the PyTorch state dict approach described in chapter 2, where `model_path` points to the corresponding .pth checkpoint file:

```
model.load_state_dict(torch.load("qwen3-0.6B-rlvr-grpo-step00050.pth"))
```

DOWNLOADING RLVR CHECKPOINTS

If you prefer not to run the GRPO training loop locally because of its runtime cost, you can instead download my pretrained checkpoints. These can be fetched either manually or directly from Python using the following helper function, which downloads the selected checkpoint and makes it available for evaluation or further experimentation:

```
from reasoning_from_scratch.qwen3 import download_qwen3_grpo_checkpoints
download_qwen3_grpo_checkpoints(grpo_type="no_kl", step="00050")
```

Note that this checkpoint was created with settings similar to those in listing 6.17, except that `num_rollouts=4` was changed to `num_rollouts=8`.

These checkpoints are fully compatible with the model evaluation utilities introduced in chapter 3, which allow us to evaluate RL-trained models using the same MATH-500 verification pipeline.

For convenience, you can reuse the evaluation script provided in the chapter 3 bonus materials (**https://github.com/rasbt/reasoning-from-scratch/blob/main/ch03/02_math500-verifier-scripts/evaluate_math500.py**) to run evaluations on the MATH-500 dataset by specifying the desired checkpoint path. For example, to evaluate the `qwen3-0.6B-rlvr-grpo-step00050.pth` checkpoint file, you can run the aforementioned script as follows:

```
uv run evaluate_math500.py \
--dataset_size 500 \
--which_model base \
--checkpoint_path "qwen3-0.6B-rlvr-grpo-step00050.pth"
```

(If you are not a `uv` user, replace `uv run` with `python`.)

Table 6.1 compares the original base and reasoning models (rows 1 and 2) to the base model we trained via GRPO in this chapter (rows 3–5).

Table 6.1 MATH-500 task accuracy for different base and reasoning models

	Method	Step	Max tokens	Num rollouts	Accuracy	Average tokens
1	Base model (chapter 3)	-	-	-	15.2%	78.85
2	Reasoning model (chapter 3)	-	-	-	48.2%	1369.79
3	GRPO (this chapter)	50	256	4	43.2%	560.22
4	GRPO (this chapter)	50	512	4	45.6%	579.81
5	GRPO (this chapter)	50	512	8	47.4%	586.11

The accuracy column in table 6.1 refers to the accuracy on the full 500-sample MATH-500 dataset we used in chapter 3.

The max tokens column identifies the number of tokens allowed per rollout during training. If the number is 512, this encourages the LLM to provide the final boxed answer within that 512-token limit; otherwise, it will not receive a reward during training. The evaluation code was run with a maximum token limit of 2,048, allowing the LLM to generate longer responses during evaluation.

The average tokens column shows the average response length in the evaluation on the MATH-500 dataset. As you can see, compared to the reference reasoning model (row 2), our GRPO models generate shorter responses on average, as expected because of the token-length restriction during training.

Note that the DeepSeek-R1 team observed that responses grow longer over the course of training as the model begins to write intermediate (chain of thought) explanations. To maximize accuracy, it makes sense not to restrict token length too aggressively during training. Longer token lengths, especially when coupled with multiple rollouts, require more computational resources, which is why we capped them at a lower value in this chapter.

As shown in table 6.1, training the reasoning model via GRPO results in 47.4% accuracy (row 5), which is close to the accuracy of the official Qwen3 0.6B reasoning model on MATH-500. The model's accuracy could be further improved by increasing the response length, sampling more rollouts per prompt, and training for additional steps. The next chapter will introduce practical tips and techniques for monitoring and improving GRPO training outcomes.

We trained the model for only 50 steps, which already appears sufficient to unlock reasoning behavior in the base model. In my experiments, training for longer does not necessarily improve performance and can even reduce accuracy, since the original GRPO formulation can be unstable over longer runs.

It's also worth keeping in mind that this chapter implemented a simplified GRPO variant without the KL regularization term. In the next chapter, we'll return to the full GRPO objective, add the KL term back in, and discuss common improvements for making training more stable over longer runs.

If you're interested, you can find and download additional checkpoints ranging from 50 to 9,000 at **https://huggingface.co/rasbt/qwen3-from-scratch-grpo-checkpoints/tree/main/grpo_original_no_kl**.

Summary

- Reinforcement learning (RL) can be used to train LLMs on human preference labels and verifiable rewards.
- RL is typically applied as post-training on top of a pretrained base model, and it can be inserted at different stages of an LLM pipeline, including reasoning training and preference tuning.
- RL with human feedback (RLHF) optimizes for human preferences via a two-stage setup: train a reward model from ranked responses, and then use reward scores to update the LLM.
- RL with verifiable rewards (RLVR) simplifies RLHF by replacing learned reward models with deterministic, automatically computed verifiers (for example, math answer checking).
- GRPO is a policy optimization algorithm that turns verifier rewards into parameter updates. Because GRPO directly optimizes the model using sequence-level rewards without requiring a separate value model, it is particularly convenient.
- GRPO is a more resource-friendly alternative to other RL algorithms for LLMs because it avoids training a separate value model and instead derives learning signals from comparisons within a group of sampled rollouts.
- A "rollout" refers to a full model answer (completion) for a prompt; rewards, advantages, and log probabilities are computed from the rollout in later steps.
- Rewards are computed by a verifier that grants a reward only if the final answer is both correct and extractable in a required format, like `" \boxed{} "`.
- Raw rewards are transformed into advantages by normalizing each rollout reward relative to the group mean and standard deviation.
- GRPO also relies on sequence-level log probabilities, which are computed by summing token log probabilities over the generated answer tokens.
- Sequence log probabilities, together with the advantages, form the core policy-gradient objective in GRPO.
- The full GRPO loss computation is combined into a single function that performs rollout sampling, reward computation, advantage calculation, log probability computation, and policy-gradient loss calculation.
- The surrounding training loop is a standard deep learning loop, with the key difference being that the loss comes from GRPO rather than conventional classification losses.
- Training is resource intensive because each step requires generating multiple, potentially long rollouts, but even short GRPO runs can increase MATH-500 accuracy from 15% to 47%.

Previous chapter
< 5 Inference-time scaling via self-refinement

Next chapter
7 Improving GRPO for reinforcement learning >

7 Improving GRPO for reinforcement learning

This chapter covers

- Interpreting training curves and evaluation metrics
- Preventing the model from exploiting the reward signal
- Extending task-correctness rewards with additional response-formatting rewards

Previously, we implemented the GRPO algorithm for reinforcement learning with verifiable rewards (RLVR) from end to end. Now, as shown in figure 7.1, we'll pick up from that baseline and focus on what happens when we run longer training.

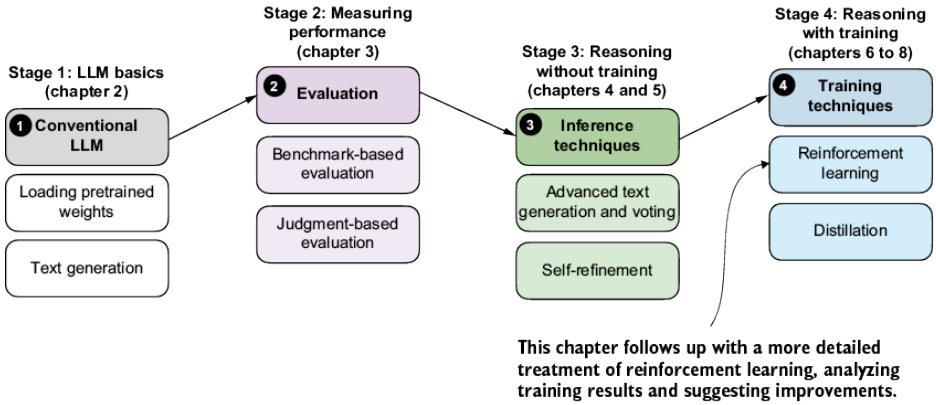


Figure 7.1 A model of the topics covered in this book. This chapter provides deeper coverage of the GRPO algorithm for reinforcement learning with verifiable rewards.

In particular, we'll discuss which metrics are worth tracking (beyond reward and accuracy), how to spot failure modes early, and why training can become unstable even when the code is "correct." As it turns out, basic GRPO can lead to training instability, so this chapter also introduces practical GRPO extensions and fixes that are used in reasoning-model training.

7.1 Improving GRPO

We implemented GRPO (Group Relative Policy Optimization) in the previous chapter; we'll now revisit the training run and analyze it more thoroughly. We'll also revisit the KL loss term that we omitted in the previous chapter and discuss some practical tips and algorithmic choices that become important in real training runs. These topics are summarized in the chapter overview in figure 7.2.

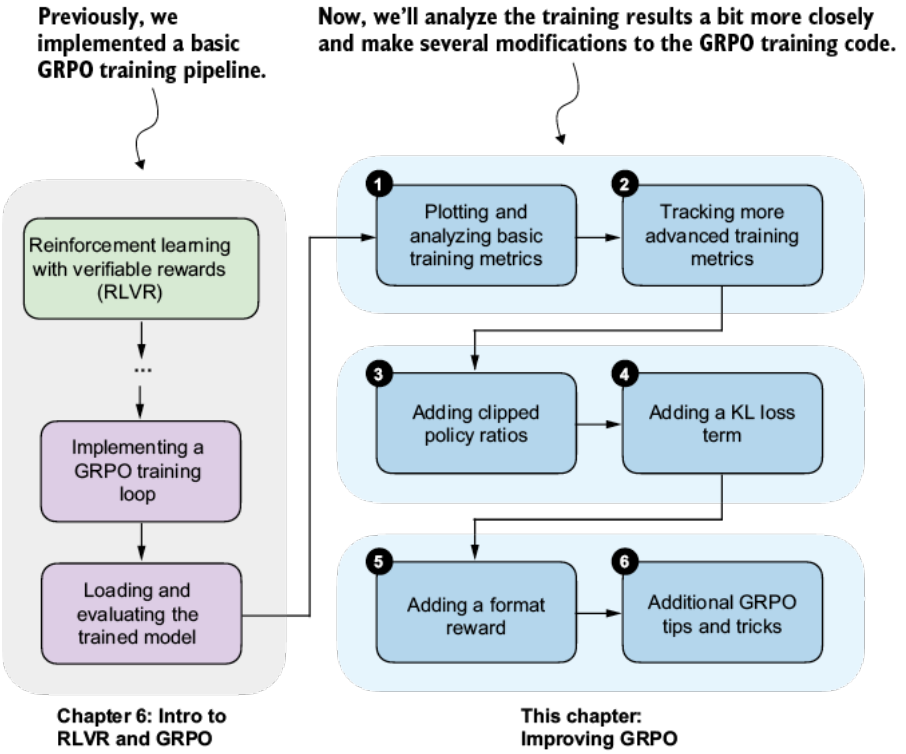


Figure 7.2 A chapter overview showing the different topics covered in this chapter

The examples in this chapter are based on actual experiments, but the results should be interpreted with care. To draw strong conclusions about the effect of individual settings, each experiment would need to be repeated multiple times and the results averaged, since randomness in sampling and optimization can lead to noticeable variation between runs. That said, the examples shown here are sufficient to illustrate the main ideas and the relevant tradeoffs.

NOTE This chapter focuses on additional technical details and practical considerations for using GRPO. If you find the content in this chapter too technical, you can move on, as the next chapter does not depend on it.

7.2 Tracking GRPO performance metrics

In chapter 6, we ran a short GRPO training loop and briefly examined the results. For example, the output of a short run was structured as follows:

```
[Step 1/50] loss=-0.0000 reward_avg=0.000 avg_resp_len=5.5
[Step 2/50] loss=-0.0000 reward_avg=0.000 avg_resp_len=6.8
[Step 3/50] loss=0.3592 reward_avg=0.250 avg_resp_len=7.8
[Step 4/50] loss=2.7401 reward_avg=0.250 avg_resp_len=56.5
[Step 5/50] loss=3.3214 reward_avg=0.500 avg_resp_len=251.2
# ...
```

Let's pick up from this point and discuss which metrics to track when training with GRPO, and how they can help interpret and debug training behavior.

7.2.1 Executing a GRPO training run

The code in chapter 6 was designed to strike a balance between implementing the complete GRPO pipeline and keeping the implementation compact enough to remain readable. For convenience, the book's supplementary materials include an equivalent script that contains the same code and can be run from the terminal (more on this later): **https://mng.bz/oZxN**.

For each GRPO improvement introduced in this chapter, we'll use related reference scripts from the supplementary materials to avoid duplicating large code blocks in the text. The helper function in listing 7.1 will download the relevant scripts from the supplementary materials and save them locally for you to use in this chapter. This will let us avoid repeating long code passages and keep the focus on the main changes in the GRPO implementations.

Listing 7.1 Helper function to download supplementary materials

```
from pathlib import Path
import requests

def download_from_github(rel_path, out=None):
    github_raw_base = (  #1
        "https://raw.githubusercontent.com/rasbt/"
        "reasoning-from-scratch/refs/heads/main/"
    )

    rel_path = Path(rel_path)

    out = Path(out) if out is not None else Path(rel_path.name)#2

    if out.exists():  #3
        size_kb = out.stat().st_size / 1e3
        print(f"{out}: {size_kb:.1f} KB (cached)")
        return out


    r = requests.get(github_raw_base + rel_path.as_posix())#4
    r.raise_for_status()

    out.write_bytes(r.content)
    size_kb = out.stat().st_size / 1e3
    print(f"{out}: {size_kb:.1f} KB")
```

#1 Base URL

#2 Use the URL filename as default output file name.

#3 Skip the download if the file already exists locally.

#4 Download the file.

Using the preceding helper function, you can download the script with the GRPO training code from chapter 6 as follows:

```
download_from_github(
    "ch06/02_rlvr_grpo_scripts_intro/rlvr_grpo_original_no_kl.py"
)
```

You should see the following output: `rlvr_grpo_original_no_kl.py: 13.4 KB`. If the downloaded file is much smaller than the size listed here, the download may not have completed correctly. In that case, double-check the URLs and see the troubleshooting guide for additional suggestions: **https://github.com/rasbt/reasoning-from-scratch/blob/main/troubleshooting.md**. The resulting script can be run from a terminal as shown in figure 7.3.

Figure 7.3 Output from a GRPO training run in a terminal, showing several training statistics, such as loss, average reward, tokens/sec throughput, and average response length

It's also possible to run the training script from a code cell in a Jupyter Notebook by prepending "`!`", that is, `!uv run rlvr_grpo_original_no_kl.py`.

If you are not a `uv` user, replace the `uv run` shown in figure 7.3 with `python`, as follows:

```
python rlvr_grpo_original_no_kl.py \
--steps 500 \
--max_new_tokens 1024
```

TIP You can add `--show_eta` to the preceding code execution command to show an estimate of the script's total runtime on your machine.

If you prefer not to run the training code yourself, which is reasonable given its computational cost, the supplementary materials provide log files from this run. You can download them as follows:

```
download_from_github(
    "ch07/02_logs/ch06_rlvr_grpo_original_no_kl_metrics.txt"
)
download_from_github(
    "ch07/02_logs/ch06_rlvr_grpo_original_no_kl_metrics.csv"
)
```

The first file, ending in `.txt`, is the plain output file that shows output statistics similar to those in figure 7.3. The second file, ending in `.csv`, is a comma-separated values file that provides more structure for this information, making it easier to extract and plot the data for further analysis.

7.2.2 Inspecting the GRPO training run

To inspect the training run, we can plot the results from the log files using Matplotlib. We'll define a plotting function that we'll use throughout this chapter.

Listing 7.2 Plotting function to visualize training results from log files

```
import csv
import matplotlib.pyplot as plt


def moving_average(values, window_fraction=0.25):

    window_size = max(1, int(window_fraction * len(values)))#1
    smoothed = []
```

```
    for i in range(len(values)):
        start_idx = max(0, i - window_size + 1)
        window_mean = sum(values[start_idx : i + 1]) / (i - start_idx + 1)
        smoothed.append(window_mean)

    return smoothed


def plot_grpo_metrics(csv_path, columns, save_as=None):
    data = {name: {"steps": [], "values": []} for name in columns}

    with Path(csv_path).open(newline="") as f: #2
        reader = csv.DictReader(f)
        for row in reader:
            if not row or not row.get("step"):
                continue

            step = int(row["step"])  #3

            for name in columns:
                value_str = row.get(name)
                if value_str:
                    data[name]["steps"].append(step)
                    data[name]["values"].append(float(value_str))


    fig, axes = plt.subplots(2, 2, sharex=True, figsize=(6, 4))#4
    axes = axes.ravel()

    for i, name in enumerate(columns):
        steps = data[name]["steps"]
        values = data[name]["values"]

        if not values: #5
            fig.delaxes(axes[i])
            continue

        if name == "eval_acc":  #6
            axes[i].bar(steps, values, width=20)
        else:
            axes[i].plot(steps, values, alpha=0.4)
            axes[i].plot(steps, moving_average(values))

        axes[i].set_ylabel(name)

    for j in (2, 3):
        if axes[j] in fig.axes:
            axes[j].set_xlabel("Step")

    plt.tight_layout()
    if save_as is not None:
        plt.savefig(save_as)
    plt.show()
```

#1 Smooth the noisy training signal to reveal longer-term trends during training.

#2 Open and read the CSV log file.

#3 Use the training step as the shared x-axis across all metrics.

#4 Create a fixed grid so loss, rewards, response length, etc., can be shown side by side.

#5 Skip metrics that are not present.

#6 Evaluation accuracy as a barplot because we don't have data for each step

The `.csv` file we downloaded has multiple columns. Using the `plot_grpo_metrics` function we defined in listing 7.2, we can plot the loss, average reward, average response length, and evaluation accuracy:

```
plot_grpo_metrics(
    "ch06_rlvr_grpo_original_no_kl_metrics.csv",
    columns=["loss", "reward_avg", "avg_response_len", "eval_acc"]
)
```

The resulting plot is shown in figure 7.4.

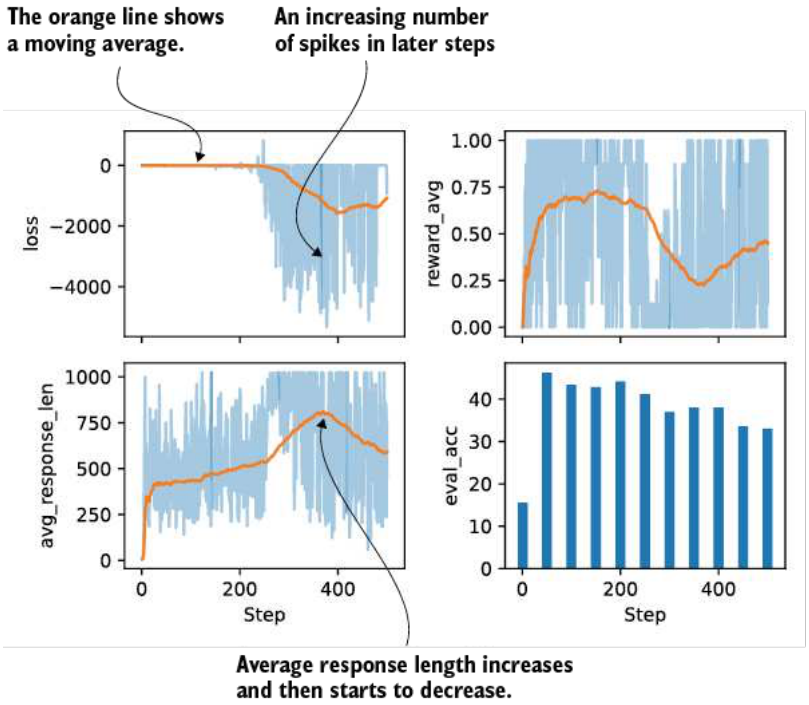


Figure 7.4 The four metrics tracked during the GRPO training run (loss, average reward, average response length, and evaluation accuracy). The orange centerline represents a moving average over the last 25% of values, which helps reveal overall trends in the otherwise noisy training signals. The evaluation accuracy is shown as a bar plot since it is computed only every 50 steps rather than at each step.

A few general observations stand out in the plots shown in figure 7.4. The average response length should initially increase, ideally, along with an improvement in accuracy, which is largely what we see here, although there is a noticeable decline later in the run, just before step 400. Compared to LLM pretraining, which is outside the scope of this book and is covered in more detail in my previous book, *Build A Large Language Model (From Scratch)*, the loss value itself is less informative and mainly serves as a sanity check. Overall, the loss should remain relatively stable. Some fluctuations are expected, but the larger spikes that appear halfway through the run are somewhat concerning.

The average reward should also increase over time. In principle, an average reward of 1.00 means that all sampled responses are correct, which is desirable, but it also means that the training signal has disappeared. At that point, further training is unlikely to be useful, and stopping early can save us time and resources.

Finally, performance on the external target task, here measured by MATH-500 accuracy, should improve. In this run, accuracy increases at first but then begins to decline, which points to problems and instabilities in the training process.

In summary, the training run shows fast gains early on, followed by diminishing returns after approximately 50 steps. One likely reason is that the underlying algorithm is not particularly stable over longer runs. Another possible explanation is that later training examples are more difficult, but this would not account for the decline in evaluation accuracy, which is computed from the same 500 MATH-500 test samples every 50 steps.

Note that the main goal of this section has been to introduce, analyze, and discuss various training metrics. We have focused on some basic ones, and in the next section we'll extend this list and look at additional metrics that are commonly tracked during GRPO training.

CALCULATING EVALUATION ACCURACY

Evaluation accuracy (`eval_acc`), here measured on the MATH-500 benchmark, is not tracked by default during training. It can be computed periodically by adding the setting `--eval_on_checkpoint 500` when running the training script, but this is not recommended because it significantly slows down training. Alternatively, it can be calculated separately after training using the `evaluate_math500.py` script introduced in chapter 3:

```
download_from_github(
    "ch03/02_math500-verifier-scripts/evaluate_math500.py"
)
```

The evaluation can then be run as follows for a given checkpoint:

```
uv run evaluate_math500.py \
  --dataset_size 500 \
  --checkpoint_path \
    "checkpoints/rlvr_grpo_original_no_kl/\
qwen3-0.6B-rlvr-grpo-step00050.pth"
```

In this run, evaluation accuracy is included in the log file. (If you are not a `uv` user, replace `uv run` with `python` .)

7.3 Tracking more advanced GRPO performance metrics

Beyond the small set of basic metrics for monitoring GRPO training runs, several additional metrics can help in understanding the training dynamics, as illustrated in figure 7.5.

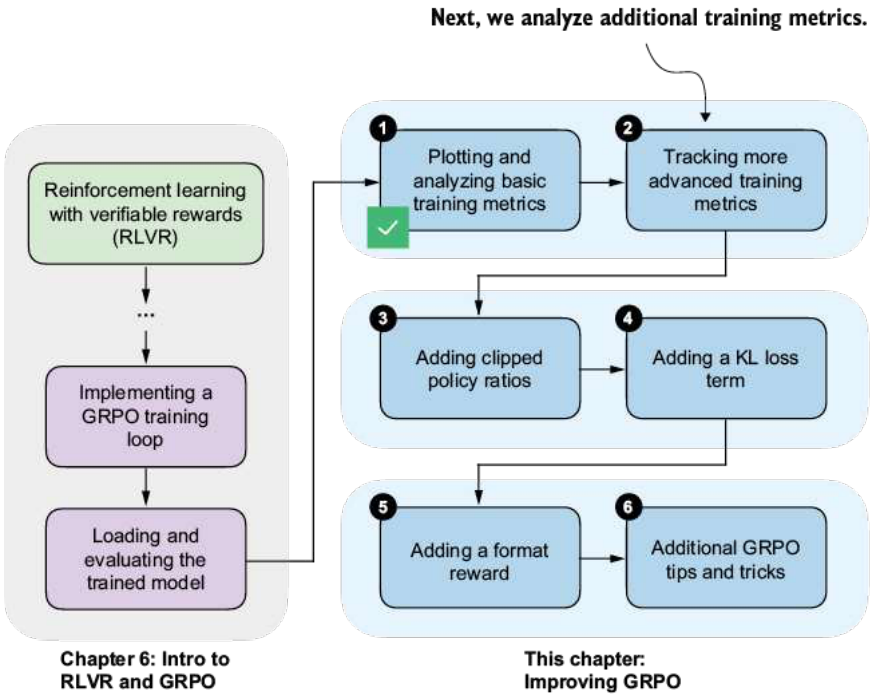


Figure 7.5 We have analyzed basic GRPO training metrics; we'll now add more advanced metrics to analyze the training run.

Two examples of the more advanced metrics mentioned in figure 7.5 are the rollout advantages already computed in the `compute_grpo_loss` function in chapter 6 and the entropy of the generated sequences.

7.3.1 Advantage tracking

As part of the GRPO algorithm, we compute so-called *advantages*, as discussed in chapter 6 and shown in figure 7.6. Beyond their role in the loss computation, these values are useful for analyzing and understanding training dynamics.

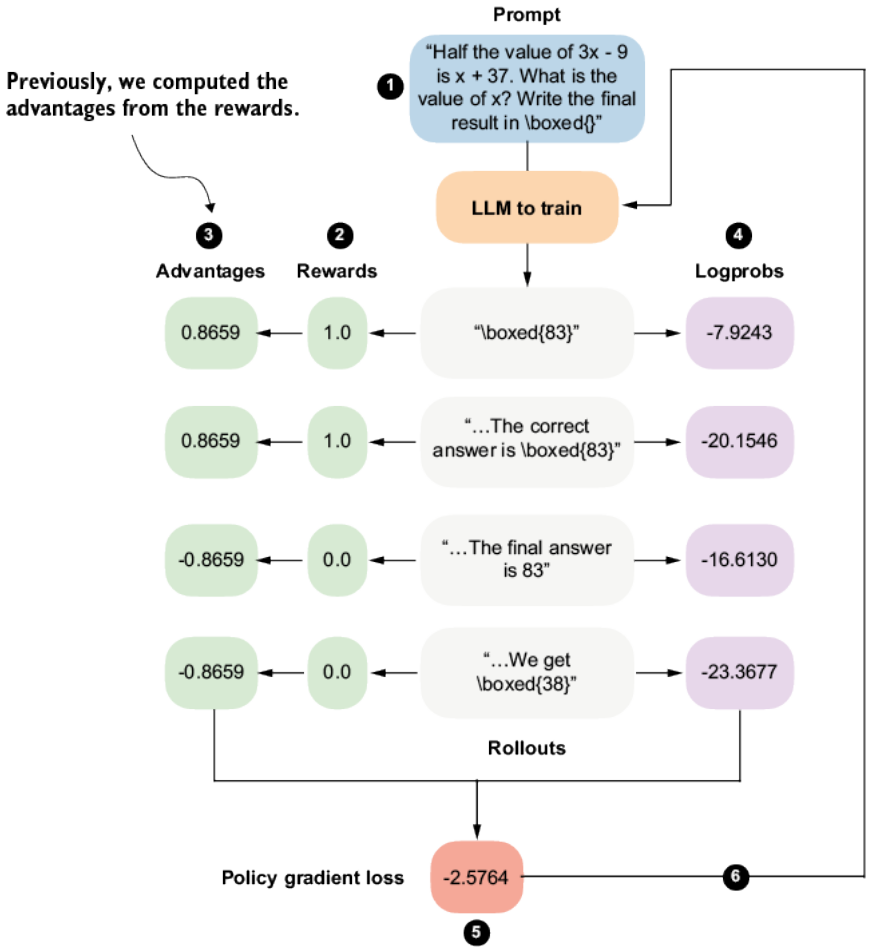


Figure 7.6 GRPO overview from chapter 6. The advantages are shown in step 3.

In particular, we compute two summary statistics derived from the advantages shown in figure 7.6, namely, their average value (the sample mean) and their standard deviation.

Listing 7.3 Computing advantage statistics

```
import torch

def compute_advantage_stats(rewards_list):

    rewards = torch.tensor(rewards_list)#1
    advantages = (rewards - rewards.mean()) / (rewards.std() + 1e-4)


    adv_avg = advantages.mean().item()#2
    adv_std = advantages.std().item()

    return advantages, adv_avg, adv_std
```

#1 The rewards and advantages below are what we already compute in GRPO.

#2 Now, we also compute the average (mean) and standard deviation (std).

As a simple illustration, consider a small sample of four reward values, similar to those shown in figure 7.6:

```
adv, adv_avg, adv_std = compute_advantage_stats([1., 1., 0., 0.])
print(f"Advantages: {adv}")
print(f"Advantage mean = {adv_avg:.4f}, std = {adv_std:.4f}")
```

The output is as follows:

```
Advantages: tensor([ 0.8659,  0.8659, -0.8659, -0.8659])
Advantage mean = 0.0000, std = 0.9998
```

Because of how advantages are computed in GRPO, their mean is always zero. In practice, this makes the mean mostly a sanity check that the implementation behaves as expected.

The standard deviation is more informative. Values close to 1 indicate a well-scaled gradient signal and are usually associated with stable updates. Very small values point to a vanishing learning signal, which often happens when rewards collapse. Very large values indicate overly spiky updates that can destabilize training.

An extreme case occurs when all rewards in the sampled group are identical. In that case, the normalized advantages are also identical, with a standard deviation of zero. As a result, the policy gradient update becomes zero, and no model weight update occurs. In other words, the model fails to learn from these cases. For example, consider the following scenario:

```
adv, adv_avg, adv_std = compute_advantage_stats([0., 0., 0., 0.])
print(f"Advantages: {adv}")
print(f"Advantage mean = {adv_avg:.4f}, std = {adv_std:.4f}")
```

which prints the following outputs:

```
Advantages: tensor([0., 0., 0., 0.])
Advantage mean = 0.0000, std = 0.0000
```

The outputs are similar if we replace the all-zero rewards in `compute_advantage_stats([0., 0., 0., 0.])` with all-one rewards in `compute_advantage_stats([1., 1., 1., 1.])`.

In practice, it is best to consider the advantage statistics together with the average reward. As mentioned earlier, an average reward of 1.0 is actually a desirable outcome, even though it means that the training signal has disappeared, because the model is answering everything correctly. At this point, it usually makes sense to stop training or to switch to more challenging examples.

7.3.2 Entropy tracking

Before we track and analyze the advantage statistics introduced in the previous section, let's introduce another metric, *entropy*.

Entropy measures how uncertain the model is when generating the next token. High entropy means the probabilities are spread across many possible tokens, which encourages exploration. Low entropy means most of the probability is concentrated on a single token, which makes the model increasingly deterministic. Low entropy can also signal training collapse, where the model stops exploring and keeps producing the same outputs.

Before computing entropy, let's briefly revisit how we calculated log-probability values (logprobs) in chapter 5, as summarized in figure 7.7.

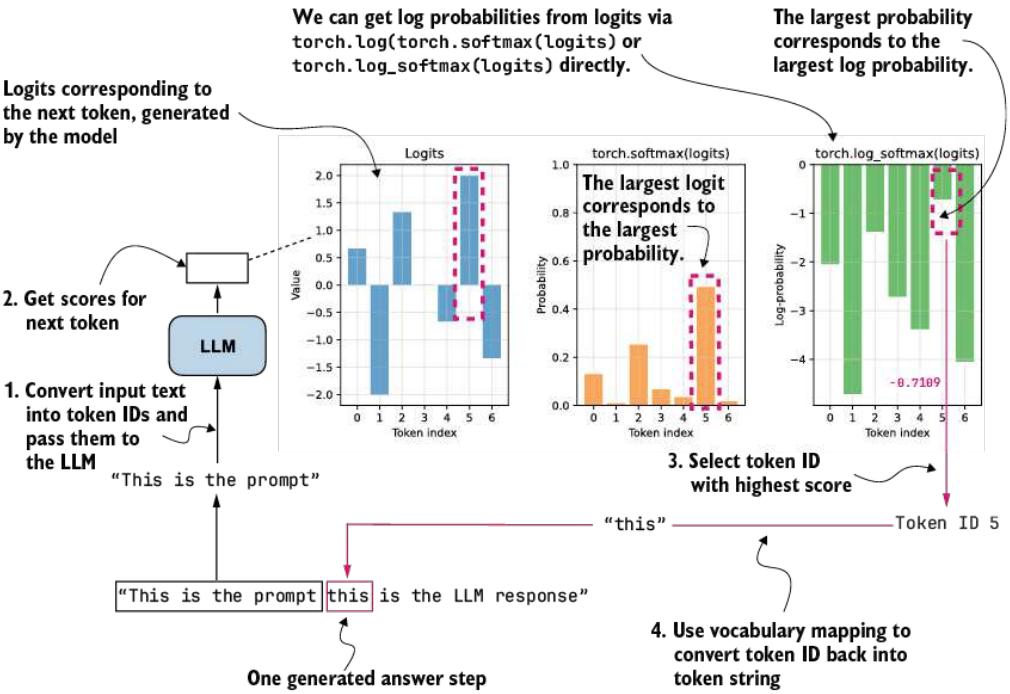


Figure 7.7 Log probability (logprob) computation of a single token (" `this` ") in the LLM's generated answer. The LLM returns the logits of the token, which are then converted to softmax probability values via `torch.softmax()` or logprob values via `torch.log_softmax()`.

In figure 7.7, instead of using real logits generated by prompting the base model, we use example logits assuming a vocabulary size of 7 (otherwise, with the original 151,000 token vocabulary, it would be impossible to visualize this concept in a plot):

```
logits = torch.tensor([
    0.6667, -2.0000,  1.3333, -0.0000, -0.6667,  2.0000, -1.3333
])
```

The following code implements the calculation shown in figure 7.7:

```
logprobs = torch.log_softmax(logits, dim=-1)
print("All token logprobs:", logprobs)

selected_token = torch.argmax(logprobs)
selected = logprobs[selected_token]
print("Selected token ID:", selected_token)
print("Selected token logprob:", selected)
```

These are the outputs:

```
All token logprobs: tensor([-2.0442, -4.7109, -1.3776,
-2.7109, -3.3776, -0.7109, -4.0442])
Selected token ID: tensor(5)
Selected token logprob: tensor(-0.7109)
```

In short, `torch.log_softmax()` computes all logprobs, `torch.argmax()` returns the index (token ID) of the largest logprob (here, 5), and `logprobs[selected_token]` returns the logprob value of that token ID (–0.7109).

Entropy is closely related to the logprobs. We compute it by multiplying each probability by its logprobs and then summing these products, as illustrated in figure 7.8. In principle, we could also track simpler quantities than entropy during training, such as the sum of probabilities or logprobs, but entropy is a widely used and easy-to-interpret measure of uncertainty in a probability distribution.

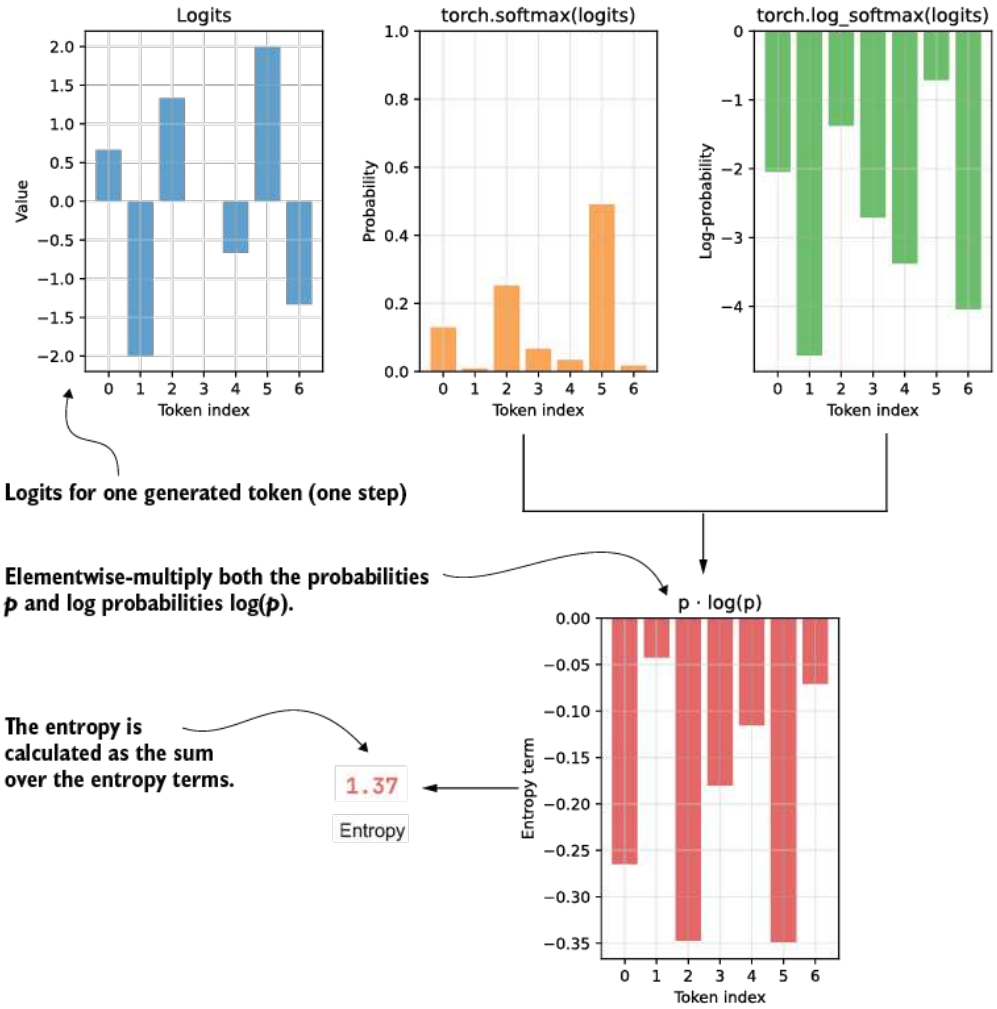


Figure 7.8 The entropy term is calculated by multiplying the token probabilities by the token logprobs.

Figure 7.8 shows an entropy of 1.37. As a rough rule of thumb,

- Very low entropy (≈ 0–0.5) means that one token dominates the distribution. The model is highly confident and behaves almost deterministically.
- Moderate entropy (≈ 1–2) means the probabilities are shared across a reasonably small set of tokens, which is typical during stable training.
- High entropy (approaching [log × vocabulary size]; here, log(7) = 1.9459) means the probabilities are spread across many tokens. In this case, the model is highly uncertain and behaves almost randomly.

In code, we can calculate entropy as follows.

Listing 7.4 Calculating entropy

```
probs = torch.softmax(logits, dim=-1)
logprobs = torch.log_softmax(logits, dim=-1)
entropy = torch.sum(-(probs * logprobs))

print("Probs:", probs)
print("Entropy:", entropy)
```

These are the resulting outputs:

```
Probs: tensor([0.1295, 0.0090, 0.2522, 0.0665, 0.0341, 0.4912, 0.0175])
Entropy: tensor(1.3700)
```

As mentioned earlier, entropy quantifies how spread out the model's probability distribution is over the vocabulary. In this specific example, the entropy of 1.37 indicates a moderate level of uncertainty. One token (with probability 0.4912) clearly dominates, but other tokens still have meaningful probabilities.

In listing 7.4, we compute the probs and logprobs separately using `torch.softmax()` and

`torch.log_softmax()`, respectively. The `torch.log_softmax()` function combines two separate function calls: `torch.log(torch.softmax())`. The `torch.exp()` function is the inverse of `torch.log()`. This means that if we only had the logprobs, we could still calculate the probs by applying `torch.exp(logprobs)`, as follows:

```
print("Probs:", torch.exp(logprobs))
```

This outputs the same probs values as before:

```
Probs: tensor([0.1295, 0.0090, 0.2522, 0.0665, 0.0341, 0.4912, 0.0175])
```

Building on this idea, we can extend the `sequence_logprob` function from the previous chapter to a `sequence_logprob_and_entropy` function that returns both the logprobs and the entropy.

Listing 7.5 Calculating sequence logprob and average entropy

```
def sequence_logprob_and_entropy(model, token_ids, prompt_len):

    logits = model(token_ids.unsqueeze(0)).squeeze(0).float() #1
    logprobs = torch.log_softmax(logits, dim=-1)   #2
 #2
    targets = token_ids[1:]       #2
    selected = logprobs[:-1].gather(1, targets.unsqueeze(-1)).squeeze(-1) #2
 #2
    selected_answer_logprobs = selected[prompt_len - 1:]   #2#2
    logp_all_steps = torch.sum(selected_answer_logprobs)   #3

    all_answer_logprobs = logprobs[:-1][prompt_len - 1:] #3
    if all_answer_logprobs.numel() == 0:   #4
        entropy_all_steps = logp_all_steps.new_tensor(0.0)
    else:
        all_answer_probs = torch.exp(all_answer_logprobs)  #5
        plogp = all_answer_probs * all_answer_logprobs  #6
        step_entropy = -torch.sum(plogp, dim=-1) #7
        entropy_all_steps = torch.mean(step_entropy) #8

    return logp_all_steps, entropy_all_steps
```

#1 Code below is identical to the sequence_logprob code in chapter 5.
#2 Logprob of the generated answer tokens (sum over answer steps)
#3 Below is the new code that calculates the entropy.
#4 A safeguard that is triggered if the model immediately returns EOS token
#5 Convert logprob to prob.
#6 Calculate elementwise p * log p.
#7 Calculate entropy for single tokens (generation steps) by summing over all plogp values in the vocabulary.
#8 Average over all answer steps to calculate average entropy for a given LLM answer.

To compute and track entropy during training, we can use this `sequence_logprob_and_entropy` function inside `compute_grpo_loss` in place of the `sequence_logprob` function.

Note that figure 7.8 illustrated the entropy computation for a single token in the rollout (`step_entropy`), whereas `sequence_logprob_and_entropy` returns the average entropy across all answer tokens; that is, `entropy_all_steps = torch.mean(step_entropy)`. For the answer `" this is the LLM response "`, the step entropy refers to the entropy at a single token position (for instance, at `" this "`), while the average entropy is the mean across all answer tokens in `" this is the LLM response "`.

With this modified `sequence_logprob_and_entropy` function, we can now use entropy as a diagnostic for the model's generation behavior during training. By tracking the average entropy of the generated answer tokens, we can monitor whether the model behaves randomly (high entropy), remains exploratory (moderate entropy), or becomes overly confident and deterministic (low entropy).

In particular, we expect entropy to gradually decrease as the model becomes more confident. A sudden

collapse to very low entropy can be a warning sign of unstable training.

7.3.3 Plotting additional GRPO metrics

Let's build on the earlier computation of advantage statistics and entropy by analyzing these quantities in a concrete training run. To do this, we could update `compute_grpo_loss` to use `sequence_log-prob_and_entropy` and then print and log the entropy in `train_rlvr_grpo`, which calls `compute_-grpo_loss` internally. To avoid duplicating a long code listing here, the full modified version is provided in the book's supplementary materials. You can download it as follows:

```
download_from_github(
    "ch07/03_rlvr_grpo_scripts_advanced/7_3_plus_tracking.py"
)
```

This code can be run in the same way as the training script we used earlier:

```
uv run 7_3_plus_tracking.py \
    --steps 500  \
    --max_new_tokens 1024
```

Since training takes a long time, you can download the resulting CSV log file and plot the new advantage statistics and entropy as follows:

```
download_from_github(
    "ch07/02_logs/7_3_plus_tracking_metrics.csv"
)
plot_grpo_metrics(
    "7_3_plus_tracking_metrics.csv",
    columns=["reward_avg", "adv_avg", "adv_std", "entropy_avg"]
)
```

We already looked at the average reward, but it's included here again for comparison. The resulting plots are shown in figure 7.9. Here, moderate entropy should still be understood as exploratory behavior, just less random than the later high-entropy regime.

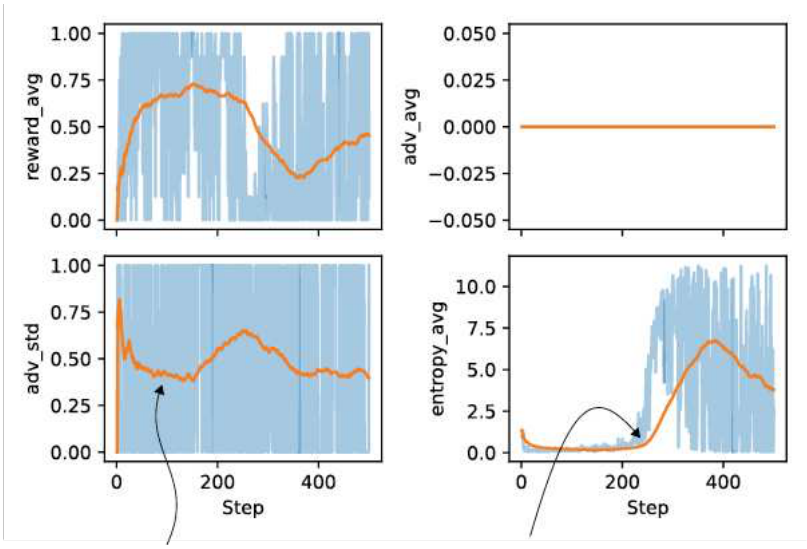


Figure 7.9 Visualizing advantage statistics and entropy tracked during the GRPO training run (next to the average reward tracked previously)

Let's begin by analyzing the advantage shown in figure 7.9. As expected with GRPO-style normalization, the average advantage stays at zero throughout training. Since advantages are computed relative to the group mean, they sum to zero by design. As mentioned earlier, this metric mainly serves as a sanity

check. If the advantage averages were to drift away from zero, that would point to a bug or a normalization issue.

The standard deviation of the advantages is more informative. Early in training, we see relatively high variance, which indicates that the model is producing rollouts with a wide range of quality. Over time, the advantage standard deviation gradually decreases and stabilizes, which means that the rollouts become more similar in quality. The key point is that as long as the advantage standard deviation remains nonzero and reasonably stable (meaning the value doesn't vary much), there is still a usable learning signal, and training is still happening.

Next, let's look at the entropy. Early in training, the entropy is relatively low and fairly flat, which indicates that the model behaves in a largely deterministic way. Increasing the sampling temperature would lead to more diverse rollouts, although it does not change the underlying entropy of the model itself.

Later in training, after roughly step 200, entropy increases quite noticeably. This means the next-token probabilities are more spread out and the model behaves more randomly. Very low entropy can also be a sign of collapse, where the model repeatedly produces the same or very similar outputs. In this run, the entropy and the increasing average reward and non-vanishing advantage standard deviation suggest somewhat healthy exploration rather than collapse. This is also consistent with the earlier MATH-500 evaluation accuracy, which stays in the 30%–40% range. That's not great, but it is not near zero.

The main takeaway here is that each metric tells a slightly different part of the story, and they are most useful when considered together and in context.

7.4 Stabilizing sequence-level GRPO using clipped policy ratios

So far, we have mainly focused on analyzing the GRPO training results. Next, we'll start adding to the GRPO algorithm itself. The version we have used so far is a simplified form of GRPO, and in this section, we'll introduce a *clipped policy ratio* in the GRPO loss, as illustrated in the overview in figure 7.10.

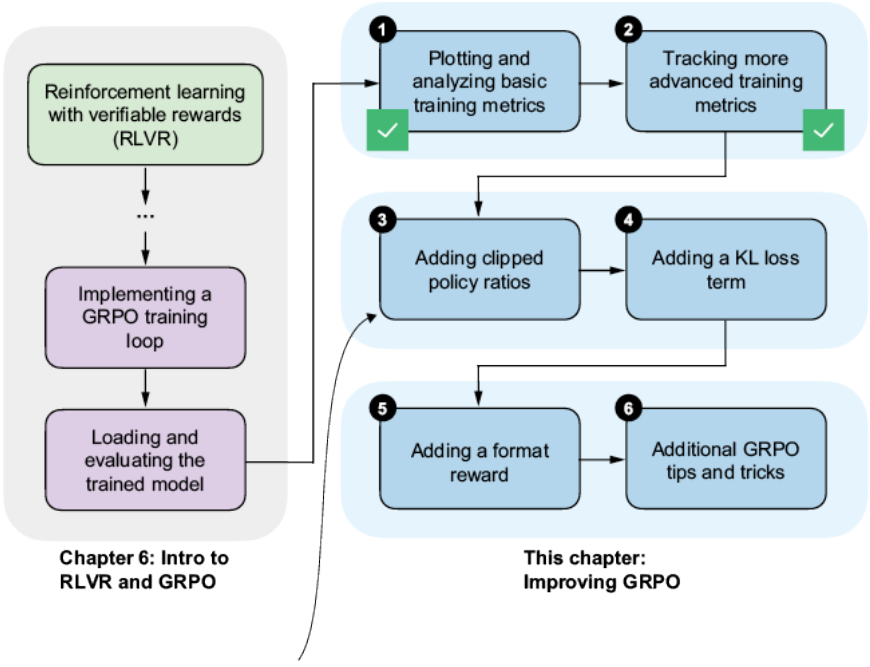


Figure 7.10 We have plotted basic and advanced GRPO training metrics. Now we'll modify the GRPO algorithm to add clipped policy ratios.

The clipped policy ratios will help limit overly large model weight updates and make training more stable, especially over longer runs. Ideally, we want to ensure that model performance doesn't noticeably decline, as we saw previously.

7.4.1 Computing clipped policy ratios

The clipped policy ratio measures how much the current policy, that is, the LLM being trained, has changed relative to an earlier version of itself. It compares sequence logprobs computed before an update step with those computed after the update. You can think of it as asking, "If the LLM previously as-

signed a certain likelihood to this answer, how much more or less likely does it consider the same answer after we adjusted its weights?"

In the GRPO pipeline shown in figure 7.11, this corresponds to comparing the logprobs from step 4, which are computed using the old weight parameters, with the logprobs produced by the updated model (updated in step 6).

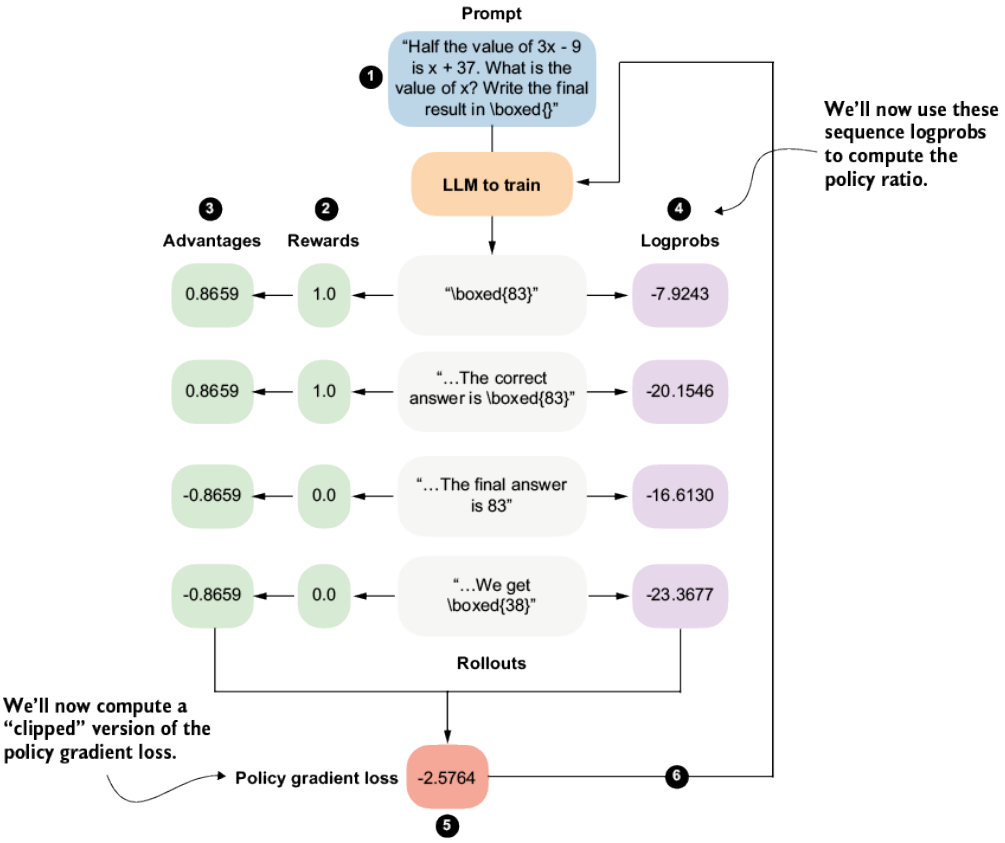


Figure 7.11 The GRPO overview from chapter 6. We'll now use the sequence logprobs from step 4 to compute policy ratios and clipped policy ratios.

This will become clearer once we implement this concept in code. But first, to recap, in chapter 6 we computed the policy gradient loss as follows.

Listing 7.6 Computing the policy gradient

```
# ...  #1


rewards = torch.tensor([1., 1., 0., 0.])#2


logprobs = torch.tensor([-7.9243, -20.1546, -16.6130, -23.3677])#3

advantages = (rewards - rewards.mean()) / (rewards.std() + 1e-4)

pg_loss = -(advantages.detach() * logprobs).mean()
print("Policy gradient loss:", pg_loss)
```

#1 Code to compute rollouts omitted for brevity
#2 Compute rewards.
#3 Compute sequence logprobs.

The resulting policy gradient value, also shown in figure 7.11, is –2.5764.

Next, we'll compute the policy ratio and clipped policy ratio, as shown in figure 7.12.The policy ratio and clipped policy ratio are computed by comparing logprobs from a previous version of the policy ("old" logprobs) with those from the current policy ("new" logprobs). The mathematical derivation is outside the scope of this chapter, but if you're interested, you can find more details in the "Proximal Policy Optimization Algorithms" paper (**https://arxiv.org/pdf/1707.06347**).

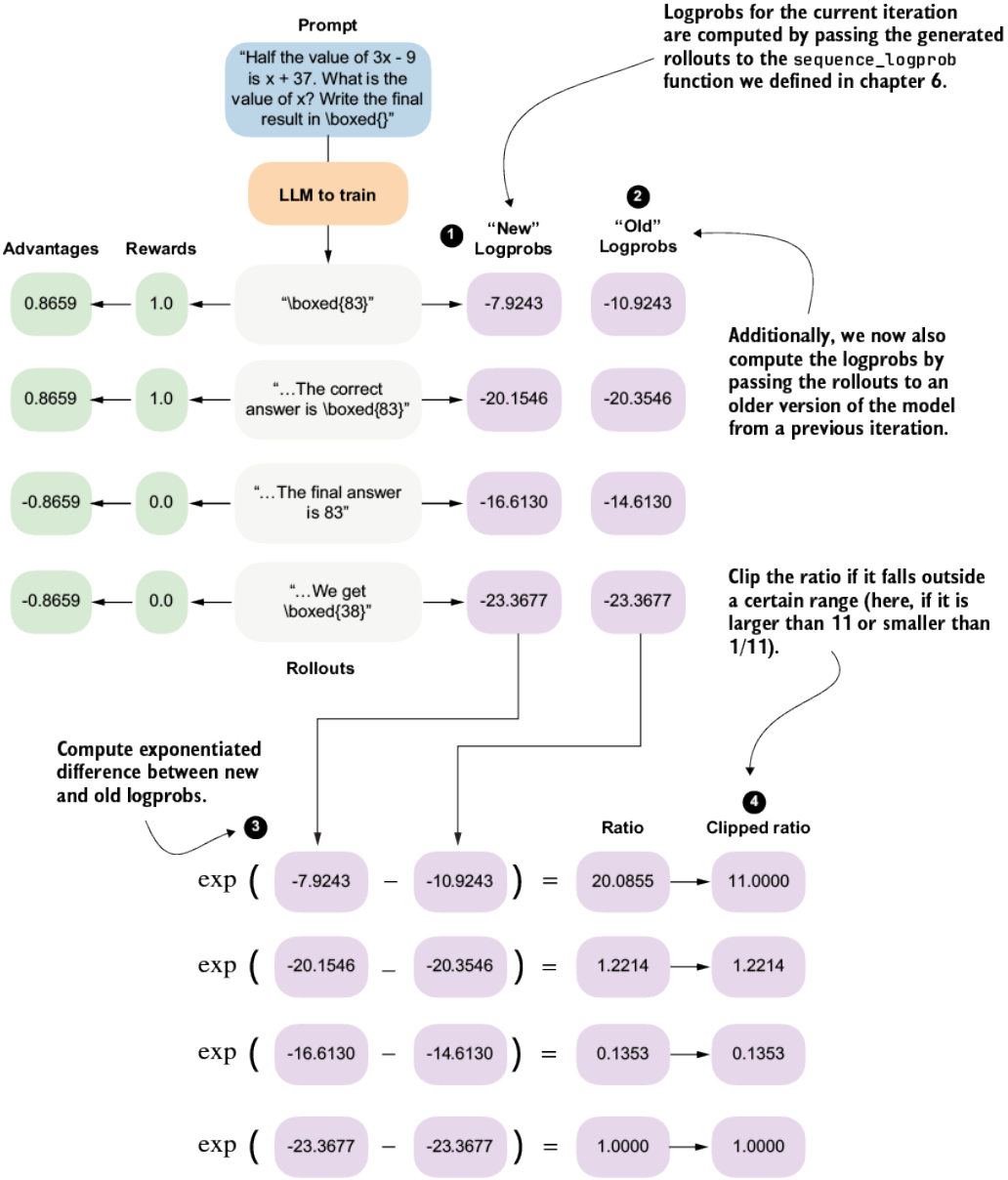


Figure 7.12 Calculating the policy ratio ("ratio") and clipped policy ratio ("clipped ratio") we've added to the GRPO, from "new" and "old" logprobs

In the following code-based illustration of the calculation shown in figure 7.12, we reuse the logprobs from listing 7.6 as `old_logps`, assume that a model update has taken place, and then compute the corresponding `new_logps` using the updated model. In practice, both are computed using the same prompt to ensure a fair, apples-to-apples comparison between the old and current policies.

Listing 7.7 Computing policy ratios and clipped policy ratios

```
new_logps = logprobs


old_logps = torch.tensor([#1
    -10.9243,   # -7.9243
    -20.3546,   # -20.1546
    -14.6130,   # -16.6130
    -23.3677,   # -23.3677
])

log_ratio = new_logps - old_logps
ratio = torch.exp(log_ratio)
clip_eps = 10.0
clipped_ratio = torch.clamp(ratio, 1.0 - clip_eps, 1.0 + clip_eps)

print("Ratio:        ", ratio)
print("Clipped ratio:", clipped_ratio)
```

#1 The values of the new_logps are shown side by side in the comments.

The resulting unclipped and clipped policy ratios follow:

```
Ratio:         tensor([20.0855,  1.2214,  0.1353,  1.0000])
Clipped ratio: tensor([11.0000,  1.2214,  0.1353,  1.0000])
```

The ratio tells us how different the old and new logprobs are. If they are identical, the ratio is 1.0.

The `clipped_ratio`, which we use to compute a clipped version of the policy gradient loss, limits how far the new policy can move away from the old one in a single update. If the new model suddenly assigns a much higher or much lower probability to a rollout than the old model, the raw ratio can become very large or very small. Without clipping, this would scale the advantage term substantially, which could lead to a very large gradient step that might destabilize training.

In practice, when using the `clip_eps` parameter from listing 7.7, it is common to clamp the ratio to the range $1 \pm$ `clip_eps`. For example, DeepSeek-R1 used `clip_eps = 10`, which corresponds to very weak clipping. Other RL training setups, such as RLHF using the PPO algorithm, often use much smaller values, such as 0.1, which results in aggressive clipping and substantially smaller per-step policy changes.

As you can see, with this very generous `clip_eps` value of `10.0`, only the first value is clipped from 20.0855 down to 11.0000.

NOTE The "eps" in `clip_eps` is short for epsilon (ε), a Greek letter commonly used in mathematics to denote a small positive quantity.

Next, we'll apply the `ratio` and `clipped_ratio` in the loss computation. Previously, we multiplied the advantages directly by the logprobs. Now, we'll instead scale the advantages by the policy ratios and use the clipped objective to limit how much each rollout can influence the update.

Listing 7.8 Computing the clipped policy gradient loss

```
adv = advantages.detach()  #1

unclipped = ratio * adv
clipped = clipped_ratio * adv

obj = torch.minimum(unclipped, clipped)#2


clipped_pg_loss = -torch.mean(obj)
policy_ratio = torch.mean(ratio)

print("Clipped policy gradient loss:", clipped_pg_loss)
print("Policy ratio:", policy_ratio)
```

#1 Treat advantages as fixed learning signals (no backprop through rewards).
#2 Limit overly large policy-ratio updates.

The resulting clipped policy gradient loss is –2.3998, which is slightly lower than the regular policy gradient loss we computed previously in this section (–2.5764). This small difference makes sense because, in our example, only one of the policy ratios was effectively clipped by the generous `clip_eps = 10.0` ratio. In general, this clipping can prevent overly aggressive updates that could destabilize training. Large, aggressive updates can move the policy too far in a single step, causing large shifts in token probabilities and leading to reward crashes, entropy collapses, or other related problems.

In addition, we compute the average policy ratio, `policy_ratio = torch.mean(ratio)`, which we can track and report during the training run. Here, the result is a relatively large 5.6106. This large policy ratio means that the updated policy (model) assigns a much higher probability to the sampled tokens than the previous policy.

7.4.2 Training with clipped policy ratios

We can apply the clipped policy ratio and policy loss computations from listings 7.7 and 7.8 to see if they

help stabilize training.

The modified code can be downloaded from the supplementary materials as follows:

```
download_from_github(
    "ch07/03_rlvr_grpo_scripts_advanced/7_4_plus_clip_ratio.py"
)
```

And we can run it as before:

```
uv run 7_4_plus_clip_ratio.py \
    --steps 500  \
    --max_new_tokens 1024
```

As discussed in the previous section, the policy ratio compares logprobs computed under the current policy with those from a previous version of the policy. In practice, this means that we first need a batch of sampled responses whose `old_logps` are measured under a fixed reference policy. We can then compare those same responses with the current model, which changes during optimization, to form the clipped policy ratio update.

DeepSeek-R1 does this at large scale by generating a large rollout pool once (8,192 responses), holding those responses fixed, and then stepping through them in 16 minibatches. At a high level, the process is as follows:

1. Make a copy of the current model as the reference policy.
2. Sample a big rollout pool using the reference policy.
3. Split those fixed rollouts into minibatches.
4. For each minibatch:
    1. Compute `old_logps` under the reference model.
    2. Compute `new_logps` under the current model (which changes after each minibatch update).
    3. Update the current model.
5. Update the reference model.

Due to resource constraints, we cannot generate as many rollouts and minibatches as DeepSeek-R1 does, so we'll use a smaller approximation. Instead of generating one huge fixed rollout pool and slicing it into minibatches, we'll generate a small batch of rollouts, update the model on that batch, and then repeat the process with a fresh batch. The procedure in the `7_4_plus_clip_ratio.py` script is as follows:

1. Make a copy of the current model as the reference policy.
2. Sample eight rollouts using the reference policy.
3. Then,
    1. Compute `old_logps` under the reference model.
    2. Compute `new_logps` under the current model.
    3. Update the current model.
4. Repeat steps 2 and 3 (by default, one more time) with different rollouts instead of using minibatches.
5. Update the reference model.

In short, DeepSeek-R1 generates a huge rollout pool once and then reuses it across minibatches before refreshing the reference model. In our code, we mimic this idea more cheaply by generating a small batch of rollouts multiple times instead.

A log file from the `7_4_plus_clip_ratio.py` training run can be downloaded and visualized as follows:

```
download_from_github(
    "ch07/02_logs/7_4_plus_clip_ratio_metrics.csv"
)
plot_grpo_metrics(
    "7_4_plus_clip_ratio_metrics.csv",
    columns=["loss", "reward_avg", "avg_response_len", "eval_acc"]
)
```

Figure 7.13 shows the resulting plot. Compared to the previous training runs, this training with clipped policy ratios is more stable, as there is no visible drop in the average reward or evaluation accuracy around the 400-step mark.

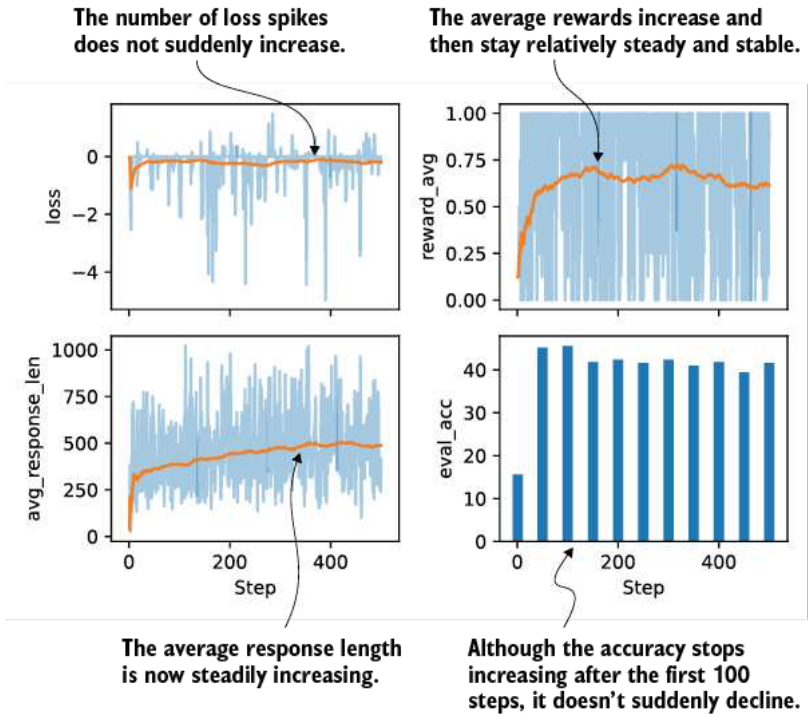


Figure 7.13 Selected metrics from a GRPO training run using clipped policy ratios

We could also plot and inspect the other metrics , such as `policy_ratio`, `adv_avg`, `adv_std`, and `entropy_avg`. I'll leave that as an exercise for you to try.

7.5 Controlling how much the model changes with a KL term

In chapter 6, we skipped using the KL loss term in the GRPO algorithm to keep the presentation manageable. To complete the GRPO algorithm, we'll now add the KL loss term, as outlined in figure 7.14.

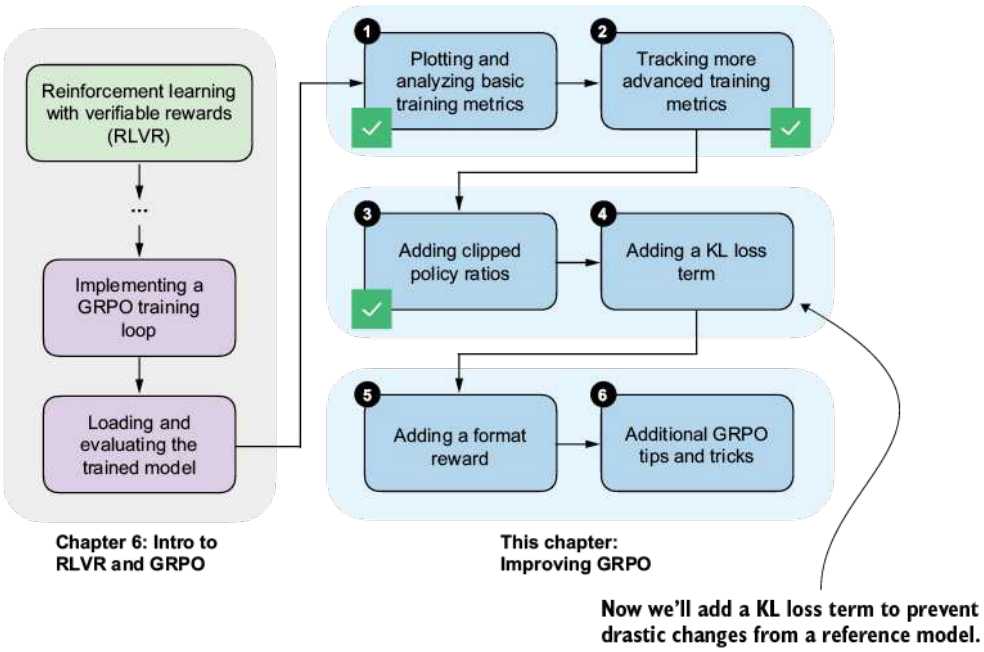


Figure 7.14 Implementing a KL loss term, which is part of the original GRPO algorithm

NOTE If you're familiar with reinforcement learning, you may recognize that what we implemented up to this point is essentially REINFORCE with group-normalized advantages. (See **https://lilian-weng.github.io/posts/2018-04-08-policy-gradient/ for information about the REINFORCE method**.)

KL is short for *Kullback–Leibler* divergence, and it measures how much the current policy, that is, the LLM being trained, deviates from a reference policy, typically the original model at the start of training. The KL term acts as a constraint (technically called a *regularizer*) that discourages overly large updates and keeps the model close to its original behavior, preventing drastic or unstable changes.

This is closely related to the clipped policy ratios we introduced earlier. With clipped policy ratios, we limited how large individual update steps could be. The KL term is another mechanism for controlling how much the model can change. While clipped policy ratios focus on limiting changes between steps, the KL term is often used to control change over the longer training trajectory.

7.5.1 Implementing the KL loss term

Figure 7.15 shows the KL loss term computation as part of the GRPO algorithm. The KL loss term calculation involves comparing logprobs and reference logprobs. It is computed by summing the differences between these corresponding logprobs.

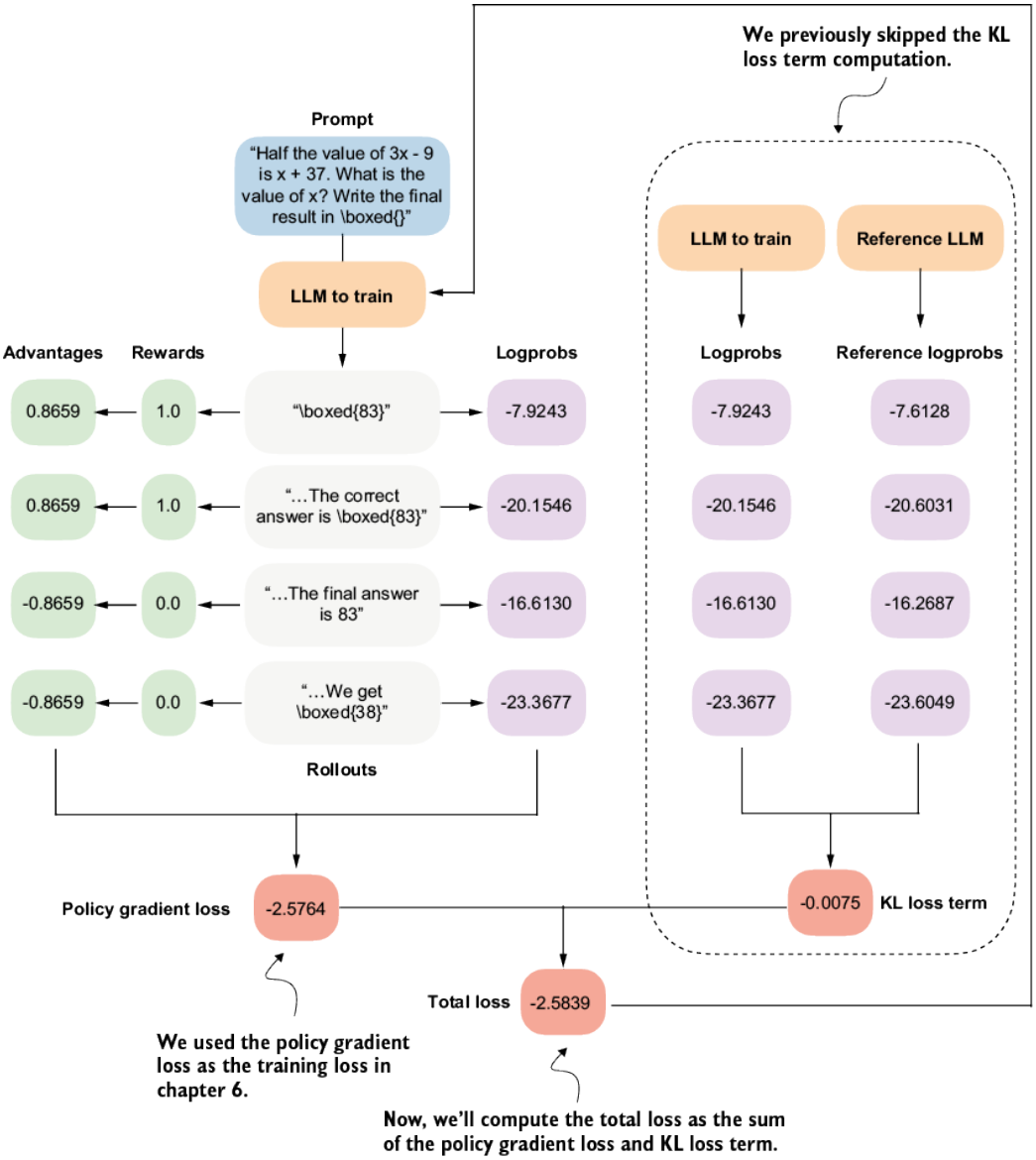


Figure 7.15 Overview of the GRPO algorithm with the KL loss term calculation added to the right

A small KL loss term value indicates that the current model and the reference model output relatively similar logprobs, meaning the current model has not changed much compared to the reference model. This is generally desirable for stability, as it prevents large, abrupt shifts in behavior. If the KL loss term remains too small for too long, it may also indicate that learning is limited.

The resulting KL loss term is then added to the policy gradient loss to compute the total loss, so weight updates that greatly increase divergence from the reference model, even if they result in a high reward, are

penalized during training.

In the clipped policy ratio section, we replaced the reference model in each iteration. For the KL loss term, the reference model used to compute the reference logprobs is the original model at the beginning of training, and it is either kept fixed or replaced relatively rarely (in the case of DeepSeek-R1, every 400 steps).

The following code illustrates how the `compute_grpo_loss` function can be modified to include this KL loss term. To avoid code duplication, listing 7.9 does not show the fully modified `compute_grpo_loss_with_kl` function. It instead focuses on the changes we need to make to the `compute_grpo_loss` function. The new additions are marked with comments or are located inside the `if kl_coeff` blocks.

Listing 7.9 Adding a KL term to the GRPO loss computation

```
import copy

kl_coeff = 0.0  #1

if kl_coeff: #2
    ref_model = copy.deepcopy(model).to(device)
    ref_model.eval()
    for p in ref_model.parameters():
        p.requires_grad = False
else:
    ref_model = None


def compute_grpo_loss_with_kl(
    model,
    ref_model,  #3
    # ...
    kl_coeff=0.02,  #4
):
    roll_logps, roll_ref_logps, roll_rewards, samples = [], [], [], []
    # ...

    for _ in range(num_rollouts):
        token_ids, prompt_len, text = sample_response(
        # ...
        )


        if kl_coeff: #5
            with torch.no_grad():
                ref_logp = sequence_logprob(
                    ref_model, token_ids, prompt_len
                )
        else:
            ref_logp = None

        reward = reward_rlvr(text, example["answer"])

        roll_rewards.append(reward)
        roll_logps.append(logp)
        if kl_coeff:
            roll_ref_logps.append(ref_logp)

        # ...

    rewards = torch.tensor(roll_rewards, device=device)
    advantages = (rewards - rewards.mean()) / (rewards.std() + 1e-4)

    logps = torch.stack(roll_logps)
    if kl_coeff:
        ref_logps = torch.stack(roll_ref_logps).detach()
```

```
        pg_loss = -(advantages.detach() * logps).mean()


        if kl_coeff: #6
            kl_loss = kl_coeff * torch.mean(logps - ref_logps)
        else:
            kl_loss = torch.tensor(0.0, device=logps.device)


        loss = pg_loss + kl_loss #7
```

#1 kl_coeff = 0.0 deactivates the KL term and the code is similar to before.
#2 Make a copy of the original model and disable gradients so it isn't updated.
#3 New: Pass the reference model.
#4 New: Specify the KL strength.
#5 Compute logprobs with the reference model.
#6 Compute the KL loss term.
#7 New: Add the KL loss term to the policy gradient loss.

Note that adding the KL loss term increases resource use, since we now need to keep an additional copy of the original model. The strength of this penalty is controlled by `kl_coeff`. Larger `kl_coeff` values add a stronger constraint on how far the current policy is allowed to change from the reference model.

The setting `kl_coeff = 0.0` at the top of listing 7.9 is included only so that this abbreviated code snippet won't crash if we run it in a code environment. In practice, `kl_coeff` is typically a small value between 0.001 and 0.05.

7.5.2 Training with a KL loss term

The following script from the book's supplementary materials implements the KL loss term code modification we've discussed:

```
download_from_github(
    "ch07/03_rlvr_grpo_scripts_advanced/7_5_plus_kl.py"
)
```

As before, we can run it as follows:

```
uv run 7_5_plus_kl.py \
    --steps 500  \
    --max_new_tokens 1024
```

Let's take a look at the log files from a training run that was executed with the preceding settings:

```
download_from_github(
    "ch07/02_logs/"
    "7_5_plus_kl_metrics.csv"
)
plot_grpo_metrics(
    "7_5_plus_kl_metrics.csv",
    columns=["loss", "reward_avg", "avg_response_len", "eval_acc"]
)
```

The resulting plots are shown in figure 7.16.

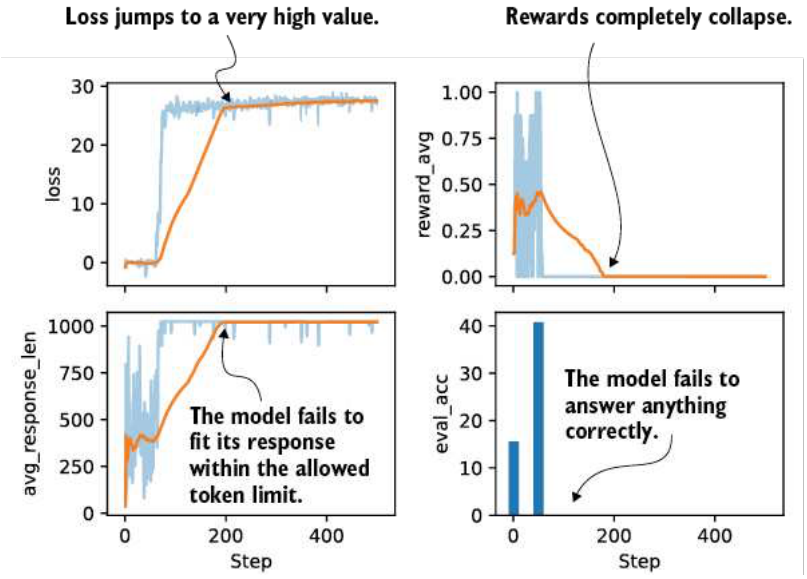


Figure 7.16 Selected metrics from a GRPO training run after adding a KL loss term. Here, loss (in the upper left) refers to the total GRPO loss, which is the sum of the policy gradient loss and the KL loss term.

As the accuracy graph at the bottom right shows, the first 50 steps improve the model's performance noticeably from slightly above 15% accuracy to approximately 40%. We then see a total collapse where the loss values explode and the average reward goes toward 0. This is accompanied by a crash in the evaluation accuracy that also goes toward 0%.

On manually inspecting the generated responses, we can see that the model eventually started producing nonsensical text and random tokens, such as `" framework.raises Self profess(import "`. There are a couple of likely reasons for this behavior.

First, the KL term is computed using summed logprobs over the answer tokens, that is, `kl_loss = kl_coeff * mean(new_logps - ref_logps)`. Here, `new_logps` are computed under the current trainable policy, while `ref_logps` are computed under the reference policy, which is the original model at the beginning of training. This reference policy is not the same as the old rollout policy in later training steps. The old policy is a recent snapshot used to generate the sampled responses, whereas the reference policy remains fixed at the initial model.

KL DIRECTION VERSUS ROLLOUT SAMPLING

KL divergence is directional. If we sampled responses from the old rollout policy and wanted to measure how much the current policy differs from that old policy, the corresponding sample-level logprob difference would be `old_logps - new_logps`.

However, the KL-style term here has a different purpose. It is intended to penalize the drift of the current policy from the fixed reference policy, so the corresponding logprob difference is `new_logps - ref_logps`.

The subtlety is that the responses are generated by the old rollout policy, not by the current policy after the update. Therefore, `mean(new_logps - ref_logps)` is a simplified sampled surrogate rather than an exact KL estimate. A more principled off-policy version would use an importance-sampling correction, which we'll discuss in the next subsection.

Because the term is not length normalized, its magnitude grows with the response length. This does not mean that longer outputs are always favored, since additional tokens can either increase or decrease the loss depending on the per-token logprob difference between the current and reference policy. However, because the rollouts are sampled from the old policy, this simple sampled surrogate is not an exact KL estimate for the current policy unless we either sample from the current policy or apply an importance-sampling correction. As a result, the objective becomes length sensitive, which can bias training toward longer outputs in some cases, as suggested by the growing response length here, and can destabilize training.

Second, once rewards collapse to zero around step 200, the KL term becomes the only remaining source

of gradient signal. When all rewards are zero, the advantages are also zero, so the policy gradient loss no longer contributes any gradient. The KL term is still active and starts to dominate the updates. In an exact KL-to-reference objective, this would pull the model back toward the reference policy. However, the simplified KL surrogate used here, `kl_loss = kl_coeff * mean(new_logps - ref_logps)`, is evaluated only on sampled rollouts from the old policy, and `ref_logps` acts as a constant during optimization. On those fixed sampled tokens, minimizing the loss mainly pushes down the current model's probability on the sampled actions, which can make the next-token distribution more spread out and raise entropy. The observed behavior is better understood as a failure mode of this sampled surrogate, which can lead to increasingly random outputs and the drop to 0.0% evaluation accuracy. If you want, you can plot the other evaluation metrics, such as `policy_ratio`, `adv_avg`, `adv_std`, and `entropy_avg`, and confirm that the entropy becomes very large.

While the KL term is a standard part of the GRPO algorithm, which is why I've introduced it here, several recent works report that models can train better without it. Examples include Dr. GRPO, Olmo 3, and DeepSeek-V3.2, all of which observe improved stability or performance when the KL term is reduced or omitted (see appendix A for more detailed references).

IMPROVING TRAINING STABILITY WITH A KL TERM

Alternatively, instead of removing the KL term, there are several ways to improve training stability. First, we can reduce `kl_coeff` from 0.02 to a smaller value (for example, 0.001) to weaken the KL penalty, although this was not sufficient in my experiments.

Since we saw that the response length keeps growing until it hits the maximum, another option is to length normalize the sequence logprobs by dividing by the number of generated tokens, which approximates average per-token logprobs. This could reduce the incentive to increase response length to accumulate larger absolute logprob differences.

A third option is to use a reweighted KL term based on *importance sampling*, as proposed by DeepSeek-V3.2. This matters because the rollouts are generated by the old policy, while the KL term is intended to constrain the current policy relative to the reference policy. Instead of using

```
kl_loss = kl_coeff * torch.mean(logps - ref_logps)
```

we could use

```
kl_loss = (
    kl_coeff * torch.mean(torch.exp(new_logps - old_logps)
    * (new_logps - ref_logps)
)
```

Here, the importance weight `torch.exp(new_logps - old_logps)` corrects for rollouts being generated by the old policy, and it downweights samples that the current policy would rarely produce and upweights those it would.

Finally, we could add a small format reward to prevent advantages from collapsing to zero (we will revisit format rewards in the next section). That said, for math data, a common recommendation is to omit the KL term altogether (e.g., Dr. GRPO, Olmo 3, and DeepSeek-V3.2), which also simplifies the code and reduces resource requirements, since we don't have to keep an additional reference model in memory.

7.6 Adding an explicit format reward

So far, we've used only one type of verifiable reward, an answer-correctness reward, when training the model. In practice, this is sufficient to train reasoning models with RLVR. It is also common to use one or more auxiliary rewards, such as a *format reward* that checks whether the generated answer follows certain formatting guidelines (figure 7.17).

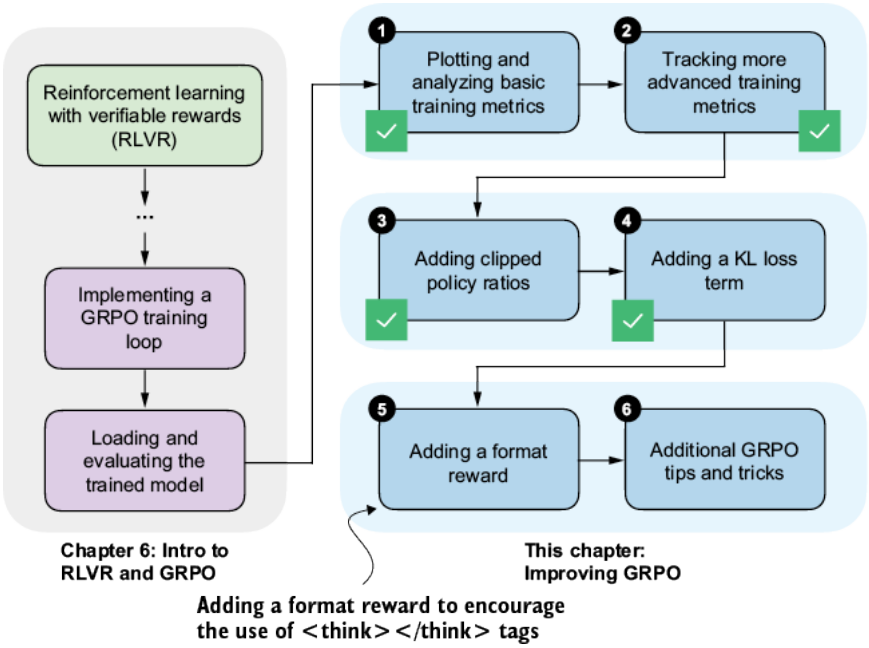


Figure 7.17 Implementing a format reward that encourages the model to generate `<think>...</think>` tokens

Specifically, we'll focus on a format reward that encourages the model to use `<think>...</think>` tokens.

7.6.1 Using <think> tokens

Some reasoning models, such as DeepSeek-R1 and Qwen3, use special `<think>` and `</think>` tokens. These are ordinary text tokens that mark the beginning and end of the model's intermediate reasoning. We can prompt the model to write its step-by-step reasoning inside `<think> ... </think>` and then provide the final answer outside these tags. This approach is optional, but it makes the output structure explicit and helps separate reasoning from the final result.

To illustrate this concept, we'll start by loading the tokenizer for the base model we used earlier.

Listing 7.10 Loading the base model tokenizer

```
from reasoning_from_scratch.ch02 import get_device
from reasoning_from_scratch.ch03 import load_model_and_tokenizer

device = get_device()
model, tokenizer_base = load_model_and_tokenizer(
    which_model="base",
    device=device,
    use_compile=False
)
```

Next, we'll use the tokenizer to encode `<think>`:

```
print(tokenizer_base.encode("<think>"))
```

The output is `[13708, 766, 29]`, which means the tokenizer is breaking up the `<think>` token into several subword tokens. This indicates that the `<think>` token is not part of the tokenizer's vocabulary.

Nowadays, when developing tokenizers and creating the embedding layers of LLMs, developers often leave unused placeholder token IDs that can be used for specific purposes when fine-tuning a model. The Qwen3 base model tokenizer has some empty placeholder slots that we can use to add `<think>`:

```
tokenizer_base._tok.add_special_tokens(
  ["<tool_response>", "</tool_response>", "<think>", "</think>"]
)
```

Note that we are also adding the `<tool_response>` and `</tool_response>` tokens in the preceding code to match the tokenizer of the Qwen3 reasoning variant, which we'll discuss shortly. But first, let's check whether the modified base tokenizer now supports the newly added `<think>` and `</think>` tokens:

```
print(tokenizer_base.encode("<think>"))
print(tokenizer_base.encode("</think>"))
```

The outputs are single tokens `[151667]` and `[151668]`, which indicates that the `<think>` and `</think>` tokens are now part of the vocabulary and are no longer broken into subword tokens.

These new tokens would also be supported by the base model, which supports token IDs from 0 to 151935, because it has a vocabulary size of 151936. We can confirm this by printing the size of the embedding layer:

```
print("Vocabulary size:", model.tok_emb.weight.shape[0])
```

Alternatively, instead of modifying the base tokenizer, we could use the reasoning model variant, whose tokenizer already supports these `<think>` tokens natively. As in chapter 2, we can load the tokenizer separately.

Listing 7.11 Loading the reasoning model's tokenizer

```
from reasoning_from_scratch.qwen3 import Qwen3Tokenizer
from reasoning_from_scratch.qwen3 import download_qwen3_small

download_qwen3_small(
    kind="reasoning", tokenizer_only=True, out_dir="qwen3"
)

tokenizer_path = Path("qwen3") / "tokenizer-reasoning.json"
tokenizer = Qwen3Tokenizer(tokenizer_file_path=tokenizer_path)
```

Now, let's use the reasoning tokenizer (`tokenizer`) instead of `tokenizer_base` to encode the token IDs:

```
print(tokenizer.encode("<think>"))
print(tokenizer.encode("</think>"))
```

As when we used `tokenizer_base`, this returns `[151667]` and `[151668]`.

Now that we have a tokenizer that supports these `<think>` `</think>` tokens, we can develop a reward function that checks whether the LLM uses them, and we'll provide a nonzero reward if it does, as illustrated in figure 7.18. This means that we can develop a function that returns 1.0 if both `<think>` and `</think>` appear in the output in the correct order; otherwise, it returns 0.0.

Listing 7.12 Implementing a format reward function

```
def reward_format(
    token_ids,
    prompt_len,
    start_think_id=151667,
    end_think_id=151668,
):
    try:
        gen = token_ids[prompt_len:].tolist()
        return float(
            gen.index(start_think_id) < gen.index(end_think_id)
        )
    except ValueError:
        return 0.0
```

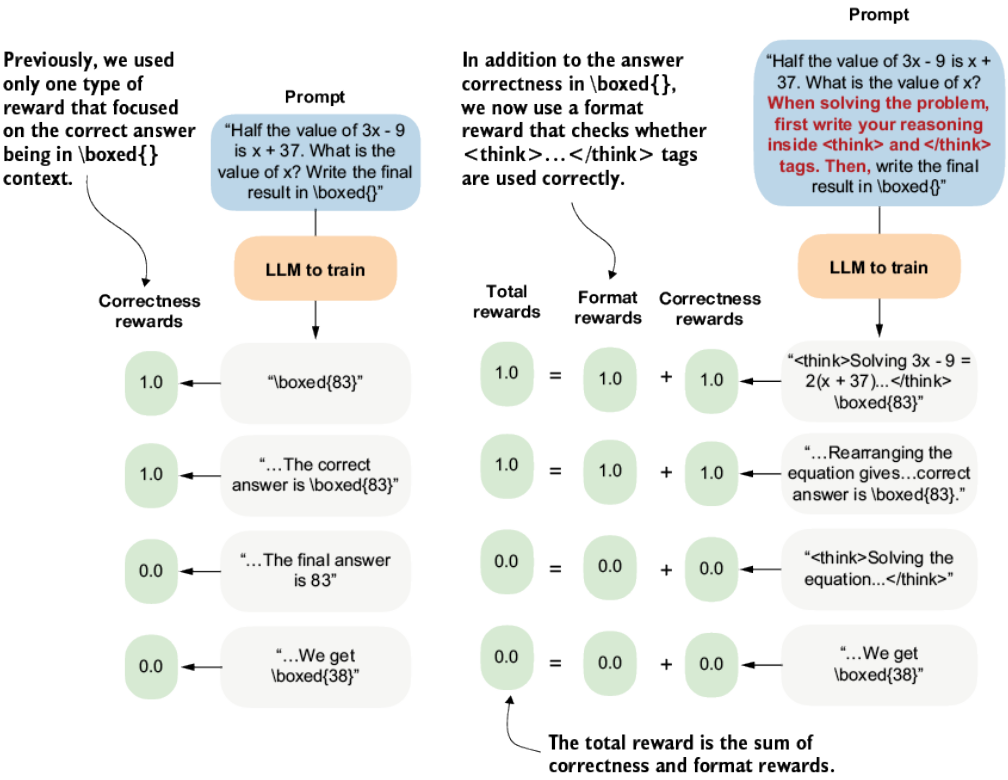


Figure 7.18 Using the previous correctness reward (left) and a correctness plus format reward (right)

Let's give this a try on a simple text:

```
prompt = "Calculate ..."
rollout = "Let's ... <think> ... </think> ..."
token_ids = tokenizer.encode(prompt + rollout)
reward_format(
    token_ids=torch.tensor(token_ids),
    prompt_len=len(tokenizer.encode(prompt))
)
```

Running the `reward_format` function in the preceding code snippet returns 1.0 as expected, since the answer (`rollout`) contains both `<think>` and `</think>` .

EXERCISE 7.1: TESTING THE <THINK> FORMAT REWARD

Run the example and confirm it returns 1.0, since both `<think>` and `</think>` appear in the correct order.

Now modify the `rollout` text in the following ways:

- Introduce a typo in one of the `<think> </think>` tags.
- Reverse the tag order.
- Remove one tag.

Verify that the reward becomes 0.0 in each case.

Now that we have this new `reward_format` function, we can use the format reward as an additional signal to train the model. For instance, we could convert `compute_grpo_loss` from chapter 6 into a `compute_grpo_loss_plus_format_reward` function, as follows (the changes are highlighted in the comments).

Listing 7.13 Updating the GRPO loss function with a format reward

```
def compute_grpo_loss_plus_format_reward(
    # ...
    format_reward_weight=1.0,  #1
):
    # ...

        logp = sequence_logprob(model, token_ids, prompt_len)
        rlvr_reward = reward_rlvr(text, example["answer"])


        format_reward = reward_format(token_ids, prompt_len)         # NEW#2
        reward = rlvr_reward + format_reward_weight * format_reward  # NEW #2

    # ...
```

#1 A setting parameter that lets us regulate the magnitude of the format reward relative to the existing correctness reward
#2 New format reward lines that modify the previous reward to now consist of a correctness and a format reward

In addition, we can modify `render_prompt` to direct the model to emit these `<think>` and `</think>` tokens explicitly.

Listing 7.14 Updated prompt rendering function for `<think>` tokens

```
def render_prompt_with_think_tokens(prompt):
    template = (
        "You are a helpful math assistant.\n"
        "When solving the problem, first write your reasoning inside <think> and </think> tags.\n"
        "Then write the final result on a new line in the exact format:\n"
        "\\boxed{ANSWER}\n\n"
        f"Question:\n{prompt}\n\nAnswer:"
    )
    return template
```

In theory, the modifications in code listings 7.11 through 7.14 should be sufficient to train a reasoning model with an additional format reward. Let's now look at how this works in practice and discuss some additional caveats.

7.6.2 Training a model to emit <think> tokens

The modifications needed to train a reasoning model with a formatting reward are implemented in a script that you can download:

```
download_from_github(
    "ch07/03_rlvr_grpo_scripts_advanced/7_6_plus_format_reward.py"
)
```

There are a few caveats to keep in mind about this script. For instance, if we train the base model with the modified prompt to emit `<think>` and `</think>` tokens, as shown in listing 7.14, the results would be poor because the base model has never seen these tokens before. The proper way to introduce these new tokens would be to pretrain or instruction fine-tune the model on data that includes `<think>` and `</think>` tokens (instruction fine-tuning is covered in the next chapter).

In this chapter, where we are focusing on reinforcement learning, we'll train the existing reasoning variant of the model, which is already familiar with `<think>` tokens. This is the same reasoning variant we introduced in chapter 3.

You can run the script as follows:

```
uv run 7_6_plus_format_reward.py \
    --steps 500  \
    --max_new_tokens 1024
```

Note that this script is already configured to use the reasoning model variant instead of the base model.

You can download a log file of this run for further analysis, because running the experiment can be resource intensive and expensive:

```
download_from_github(
    "ch07/02_logs/7_6_plus_format_reward_metrics.csv"
)
plot_grpo_metrics(
    "7_6_plus_format_reward_metrics.csv",
    columns=["loss", "reward_avg", "avg_response_len", "eval_acc"]
)
```

The resulting plot is shown in figure 7.19. As you can see, the model improves at first but then gets worse in terms of evaluation accuracy, while the average reward stays approximately constant. The decline in evaluation accuracy appears to correlate with the shorter response lengths the model produces after that point.

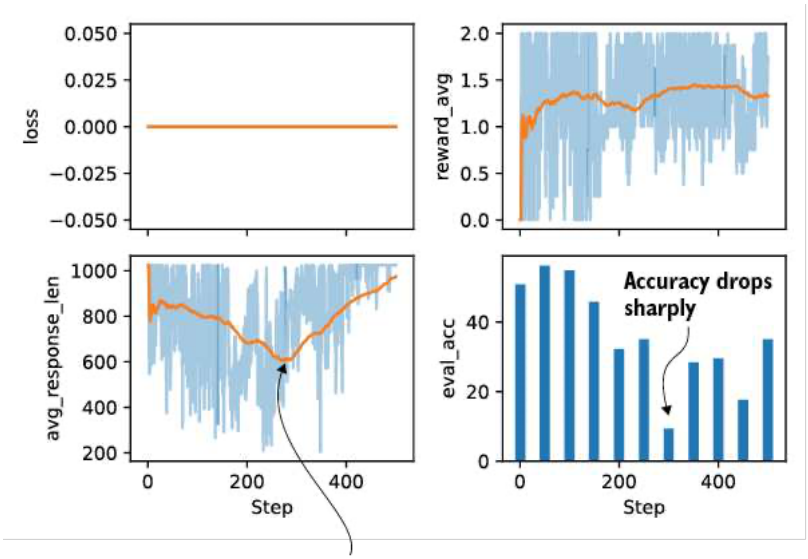


Figure 7.19 Basic metrics from a GRPO training run with a format reward

To investigate this further, let's look at some additional metrics:

```
plot_grpo_metrics(
    "7_6_plus_format_reward_metrics.csv",
    columns=["reward_avg", "format_reward_avg", "adv_std", "entropy_avg"],
)
```

The results are shown in figure 7.20. One possible explanation for the declining model performance is that the model receives too much reward simply for following the `<think>` tag format, as shown in the `format_reward_avg` plot. In other words, the model does not focus enough on answer correctness (as reflected by the drop in `eval_acc` despite the high `reward_avg`).

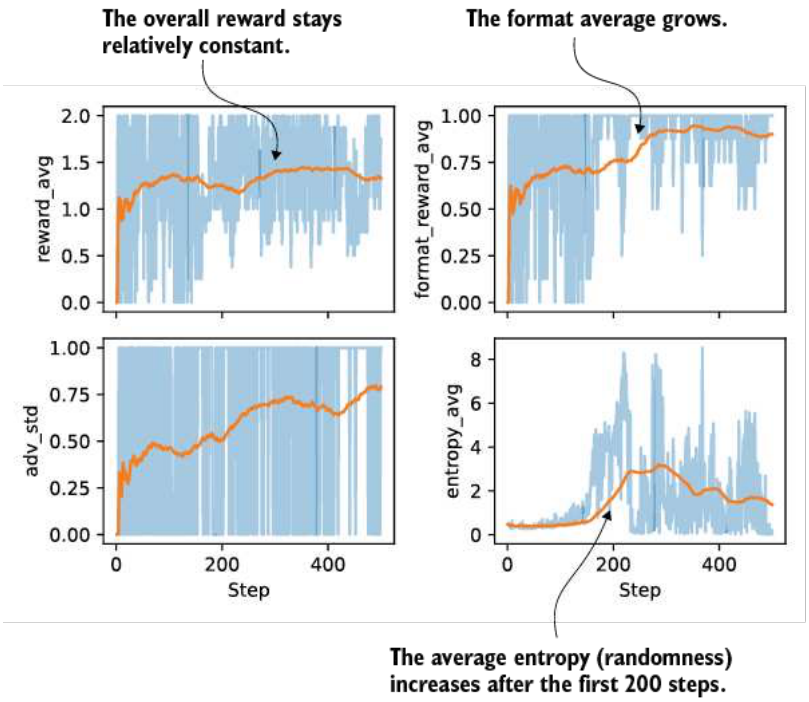


Figure 7.20 Additional metrics from a GRPO training run with a format reward

One way to address this is to reduce the strength of the format reward, such as by lowering the default `format_reward_weight` from `1.0` to `0.1`. Another option is to make the format reward conditional so it is applied only when the answer is also correct. In the current setup, the format reward is granted regardless of correctness.

A NOTE ON LOSS AND KL_LOSS

This experiment used `7_6_plus_format_reward.py` with the default settings `--kl_coeff 0.0` and `--inner_epochs 1`. Because `--kl_coeff` is set to `0.0`, the KL penalty is disabled, so the logged `kl_loss` remains `0` throughout the run.

If you want to enable KL regularization with the same settings used in section 7.5, run this:

```
uv run 7_6_plus_format_reward.py \
    --steps 500 \
    --max_new_tokens 1024 \
    --kl_coeff 0.001 \
    --inner_epochs 2
```

The logged `loss` (figure 7.19) is also 0 with the default 7.6 settings. This happens because the run uses only one inner update (`--inner_epochs 1`) and compares the model against a copy of itself from the same step. As a result, the policy ratio starts at 1, and because the advantages are normalized to have a mean of 0, the scalar policy loss evaluates to 0.

So in this case, `loss = 0` is a computational artifact caused by the default settings. It mainly reflects that the run uses a single inner epoch and no KL term, which makes the logged scalar loss uninformative.

The more useful quantities to monitor here are metrics such as `reward_avg`, `format_reward_avg`, `adv_std`, and `entropy_avg`.

EXERCISE 7.2: MAKING THE FORMAT REWARD CONDITIONAL

Modify the format reward calculation,

```
reward = rlvr_reward + format_reward_weight * format_reward
```

so that the format reward is given only if the correctness reward (`rlvr_reward`) is nonzero. Repeat the training run with the conditional format reward. Does conditioning the format reward change training behavior and final accuracy?

Note: Because running these experiments can be resource intensive, reference results are provided in appendix B.

7.6.3 More GRPO modifications, tips, and tricks

In chapter 6, we implemented a version of GRPO without the KL loss term, with clipped policy ratios, and with a format reward for introductory purposes. Then we added these concepts to build a solid foundation for understanding the original GRPO algorithm, which underlies most modern reasoning training frameworks. Figure 7.21 outlines this next step, where we'll discuss additional modifications, tips, and tricks.

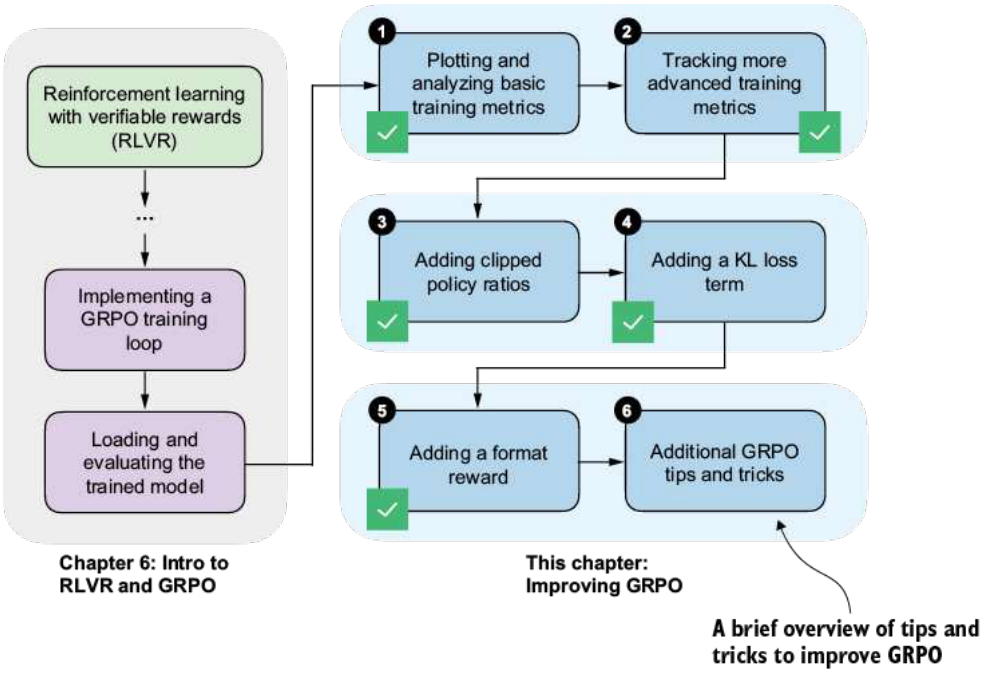


Figure 7.21 The last section in this chapter outlines additional GRPO modifications that have emerged recently.

As far as RLVR with GRPO goes, we learned that there are many knobs to tune. Besides changing the basic settings, such as the learning rate, maximum response length, and prompt format, there is a nearly infinite number of settings and modifications to try. Exploring a single modification, or a small handful of modifications, thoroughly could become a month- or year-long research project for a small research team.

Algorithms are expected to keep changing and evolving. It is important, though, to understand the fundamentals and the general ideas behind them so you can understand and appreciate these improvements.

In the months following the success of DeepSeek-R1, many researchers proposed changes to the original GRPO algorithm that improved both training stability and final performance. The following selection of suggested improvements comes from DAPO, Dr. GRPO, DeepSeek-V3.2, and others (you can find a full list of references and links in appendix A):

1. Filter out zero-gradient examples (DAPO): Skip prompts where all sampled responses receive the same reward, because these groups produce no useful advantage signal.
2. Use active sampling (DAPO): Oversample or prioritize prompts that produce a mix of correct and incorrect responses, because these provide a stronger learning signal.

3. Switch from sequence-level to token-level loss (DAPO): Compute the loss over individual tokens rather than whole responses, which can reduce length-related bias.
4. Omit the KL loss term (DAPO and Dr. GRPO): Remove the KL penalty when it hurts stability or performance, especially in math reasoning settings.
5. Use a larger clipping range (DAPO): Relax the clipping threshold so the policy can make larger updates when the reward signal is reliable.
6. Truncate importance-sampling ratios (verl): Cap very large importance weights so a few off-policy samples cannot dominate the update.
7. Remove standard-deviation normalization of advantages (Dr. GRPO): Normalize rewards relative to the group mean but avoid dividing by the reward standard deviation.
8. Tune the KL strength by domain (DeepSeek-V3.2): Use different KL coefficients for different data types, including zero KL for math tasks.
9. Reweight the KL term with importance sampling (DeepSeek-V3.2): Correct the KL estimate when rollouts are sampled from an older policy rather than the current policy.
10. Mask unsuitable off-policy sequences (DeepSeek-V3.2): Exclude rollout sequences that are too unlikely under the current policy.
11. Preserve the sampling mask for top-*p* or top-*k* sampling (DeepSeek-V3.2): Compute probabilities only over the token set that was actually available during sampling.
12. Keep the original GRPO advantage normalization (DeepSeek-V3.2): Retain the standard group-based advantage normalization when it works well empirically.
13. Normalize each reward component before aggregation (GDPO): Normalize different reward types separately before combining them into a total reward.
14. Apply sequence-level importance sampling and clipping (GSPO): Correct and clip updates at the whole-sequence level instead of only at the token level.
15. Clip importance-sampling weights rather than token updates (CISPO): Limit the importance weights directly, so off-policy corrections remain stable.

Detailed explanations, code examples, and training run results for those modifications are outside the scope of this chapter. If you're interested, you can find them in the book's supplementary materials at **https://mng.bz/nZVv.**

Summary

- Training reasoning models with GRPO can become unstable over longer runs, even when the implementation is correct and rewards initially improve.
- Interpreting GRPO training requires tracking multiple metrics jointly (average rewards, response length, evaluation accuracy, advantage statistics, and entropy):
  - Basic metrics such as loss mainly serve as sanity checks in GRPO and should not be overinterpreted in isolation.
  - Advantage statistics provide useful diagnostics: the mean should remain near zero by design, while the standard deviation reflects the strength and stability of the learning signal.
  - Entropy measures how uncertain the model is during generation. Very low entropy can signal collapse, and very large entropy can indicate unstable updates and randomness in the model responses.
- Clipped policy ratios limit how much the policy can change between updates and can substantially improve training stability over longer runs.
- Adding a KL divergence term constrains long-term drift from a reference model but can destabilize training when the rewards collapse.
- For math reasoning tasks, several recent systems report better stability and performance by omitting the KL term altogether.
- Auxiliary format rewards can improve the response structure, such as encouraging the use of `<think>` and `</think>` tokens.
- Beyond the original GRPO algorithm, many recent extensions modify advantage normalization, importance sampling, clipping strategies, and KL handling to improve stability and efficiency.

8 Distilling reasoning models for efficient reasoning

This chapter covers

- Hard and soft distillation for reasoning models
- Creating and preparing a teacher-generated reasoning dataset
- Training and evaluating a distilled student model using cross-entropy loss

Reasoning performance can be improved not only through inference-time scaling and reinforcement learning, but also through *distillation*. In distillation, a smaller student model is trained on the reasoning traces and answers generated by a larger teacher model. As shown in the overview in figure 8.1, this chapter focuses on this training-time technique.

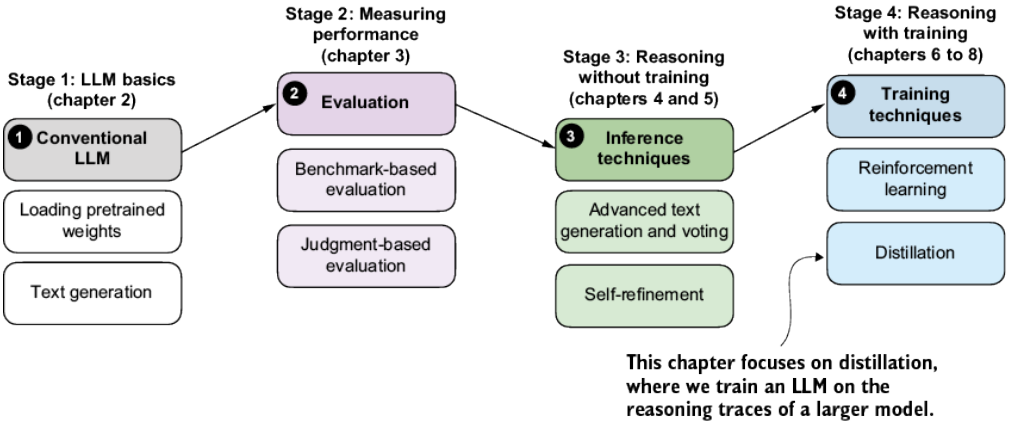


Figure 8.1 A model of the topics covered in this book. This chapter focuses on distillation, where a smaller student model is trained on reasoning traces generated by a larger teacher model.

We'll look at model distillation in general before discussing the individual steps in the process.

8.1 Introducing model distillation for reasoning tasks

Model distillation involves training a smaller LLM, the *student*, on outputs produced by a larger LLM, the *teacher*. For reasoning models, these outputs usually include not only the final answer but also the intermediate reasoning trace that leads to it.

Distillation is important because the strongest reasoning models are often too large and expensive to work with directly. For example, DeepSeek-R1 has 671 billion parameters. Systems at that scale are expensive to develop and deploy, and they are far beyond what most practitioners can run on local hardware. The team behind DeepSeek-R1 created smaller model variants by distilling the 671-billion-parameter teacher model. We will follow the same basic idea in this chapter, but at a scale that is practical for this book.

Throughout this book, we have deliberately worked with small models rather than training a very large LLM from scratch, but the workflow in this chapter is the same as for larger models. We'll use a stronger teacher to generate reasoning traces and then train a smaller student to reproduce them. In our setup, this distillation stage will take only a fraction of the time as for larger models. Our distillation training run will take about 3 hours on a DGX Spark and use about 15 GB of RAM, whereas even a few GRPO rounds in chapter 6 took around 12 hours and 70 GB of RAM on the same hardware.

Distillation can also be more effective than training a small model using reinforcement learning with verifiable rewards (RLVR). For example, the DeepSeek-R1 paper reported that its smaller distilled variants outperformed comparable models trained with reinforcement learning alone. In that setup, the largest DeepSeek-R1 model, with 671 billion parameters, acted as the teacher and generated the supervision used to train smaller student models.

There are two main types of distillation: *hard* distillation and *soft* distillation. In hard distillation, the student is trained on text generated by the teacher, so the teacher's tokens are treated as the targets. In soft distillation, the student is trained to match the teacher's probability distribution over the vocabulary by minimizing the *KL divergence*, a measure of how different two probability distributions are. These two approaches are illustrated in figure 8.2.

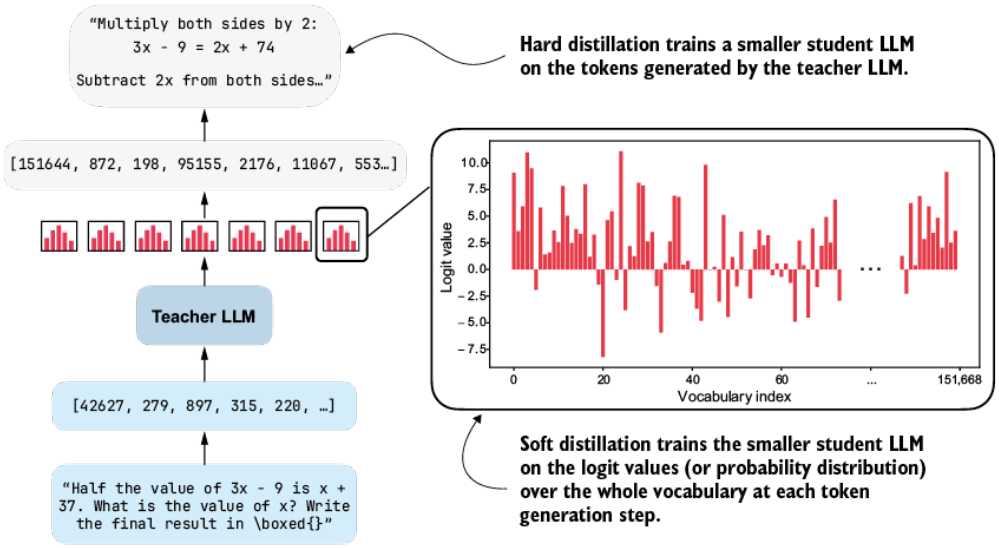


Figure 8.2 Hard distillation trains the student on teacher-generated tokens; soft distillation trains the student on the teacher's full output distribution.

As illustrated in figure 8.2, one option is pure hard distillation. Here, we use only the teacher-generated text as the training target. If you're familiar with the typical LLM training pipeline, which is covered in more detail in my previous book, *Build A Large Language Model (From Scratch)*, hard distillation is just supervised fine-tuning on synthetic data. This is also the setup used for the smaller DeepSeek-R1 distilled models. A large teacher model generates reasoning traces and answers, and the student model is fine-tuned to reproduce them. According to the DeepSeek-R1 paper, for small models this distillation approach can result in higher accuracy than training with reinforcement learning. The main practical advantage of hard distillation is that we only need access to the teacher's generated text, not its logits.

The second option is pure soft distillation. Instead of matching only the teacher's chosen tokens, the student is trained to match the teacher's full output distribution of each token over the whole vocabulary. This gives the student richer information about which alternative tokens the teacher considered plausible, but it requires access to teacher logits or log probabilities at training time.

A third option combines hard and soft distillation. This is the classic knowledge-distillation setup popularized earlier in computer vision, such as in the paper "Distilling the Knowledge in a Neural Network," by Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. In this case, we train on the teacher's actual output tokens while also encouraging the student to match the teacher's full distribution.

In practice, hard distillation is much more common for LLMs, partly because full teacher logits are usually inaccessible. Proprietary systems such as ChatGPT or Claude may expose generated text, but they generally do not expose the full vocabulary distribution needed for classical soft distillation.

WARNING Reusing generated text for distillation may be restricted by provider-specific usage policies, including the OpenAI and Anthropic terms of service, so review those policies carefully before using such outputs for training.

Even when logits are available, soft distillation is more cumbersome than hard distillation. The student and teacher usually require the same tokenizers so that their vocabulary distributions line up, so this approach is easier within the same model family. It is also much more expensive to store and use full token distributions for long reasoning traces. By comparison, storing plain text outputs is cheap and simple.

In this chapter, we'll focus on hard distillation in the style of DeepSeek-R1 because it is the more practical setup for most readers. The steps we'll discuss in this chapter are summarized in figure 8.3.

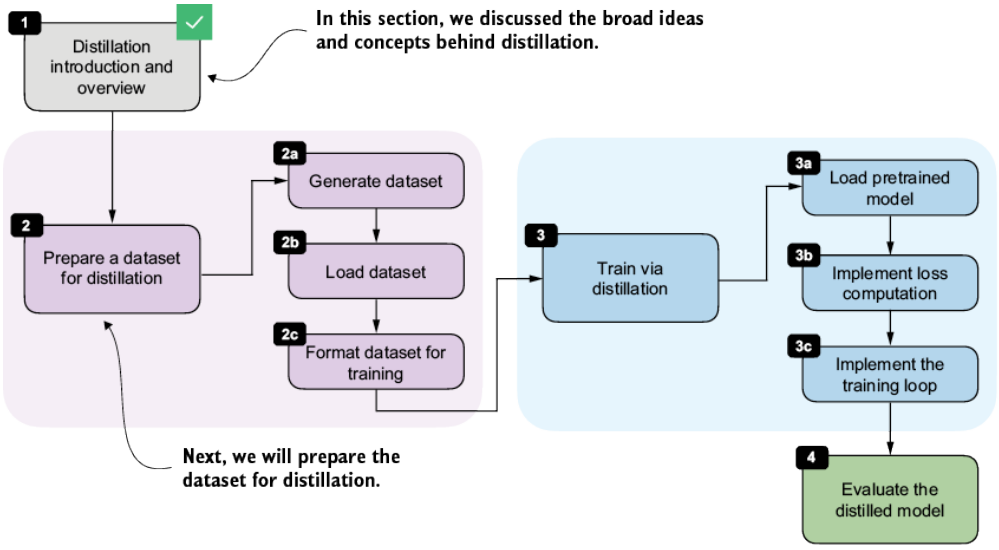


Figure 8.3 This chapter covers distillation in four main steps: (1) distillation introduction and overview; (2) dataset preparation (steps 2a-2c); (3) training via distillation (steps 3a-3c); and (4) evaluation.

With this overview in place, we can now move from the high-level ideas behind distillation to the practical steps required to implement it. In the next section, we'll begin by preparing a dataset for distillation, which will serve as the foundation for training the student model.

8.2 Generating a dataset for reasoning distillation

The first practical step in hard distillation is to create a dataset for training the student model. For us, the student will be the Qwen3 0.6B base model that we used in earlier chapters.

To build the dataset, we'll use the 12,000 math problems from the MATH split that do not overlap with the MATH-500 evaluation set. These are the same 12,000 problems we used in chapters 6 and 7 for RLVR. Instead of sampling multiple student responses and computing rewards, we'll now feed these problems to an existing reasoning model, DeepSeek-R1 (the teacher model), and collect its responses as training targets. This setup is illustrated in figure 8.4.

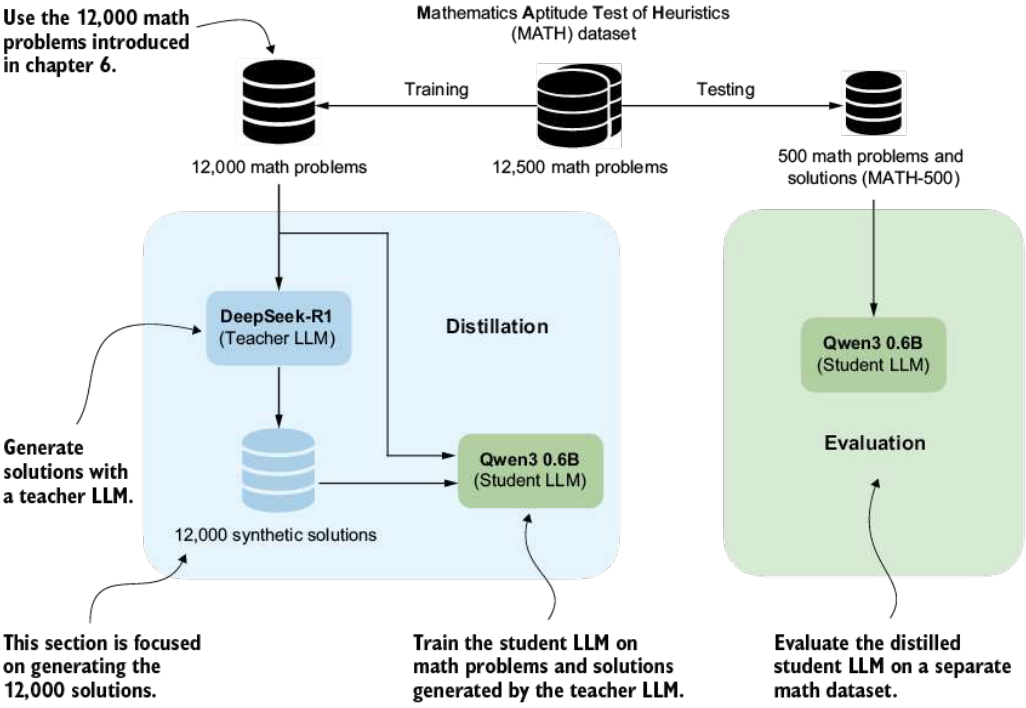


Figure 8.4 The distillation setup used in this chapter. We'll use the 12,000 non-overlapping MATH training problems to obtain synthetic solutions from DeepSeek-R1, and we'll later evaluate the distilled Qwen3 student on the separate MATH-500 test set.

In the RLVR setup in chapters 6 and 7, we trained the Qwen3 base model to produce the correct solution, and we then used a verifier to compare the model's final answer with the reference answer. The verifier produced the reward signal.

In distillation, the supervision is more direct. Instead of comparing the student's answer with the reference solution using a reward function, we'll compare the student's generated tokens with the teacher's generated tokens, as shown in figure 8.4. The teacher's response becomes the *target sequence*. This contrast with RLVR is illustrated in more detail in figure 8.5.

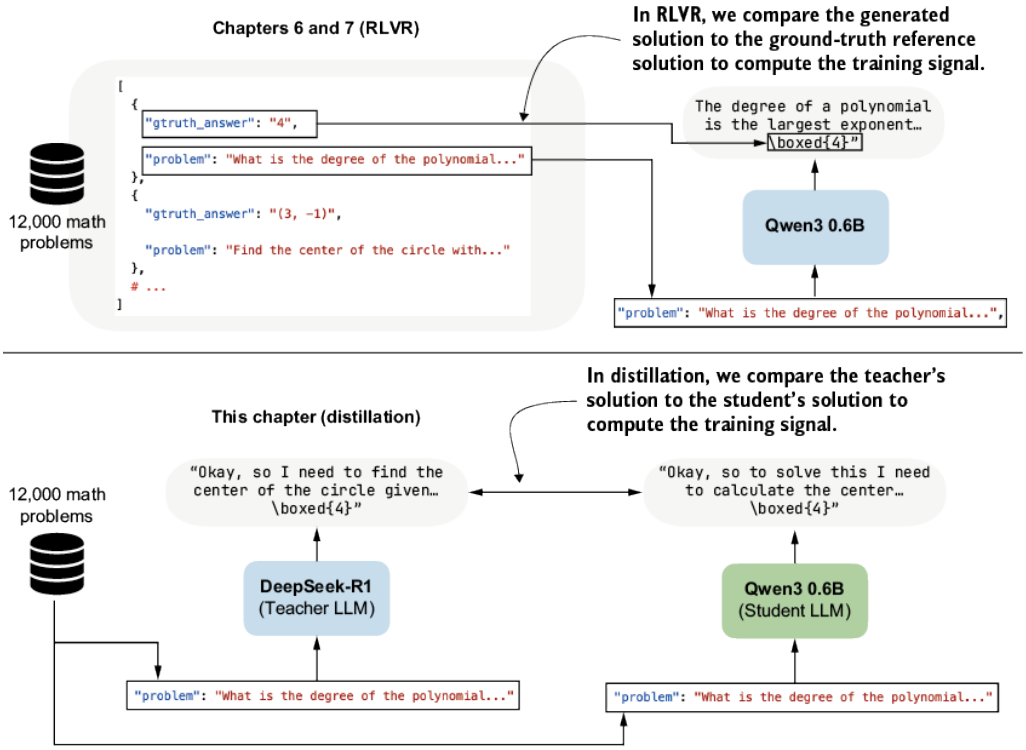


Figure 8.5 In RLVR, the generated answer is compared against the ground-truth reference solution (top subpanel), whereas in distillation the student answer is compared against the teacher-generated solution (bottom subpanel).

A practical advantage of distillation is that we can generate the teacher dataset ahead of training the student.

Because generating teacher answers for all 12,000 math problems can be time and resource intensive, I created the dataset using the 671-billion-parameter DeepSeek-R1 model hosted via OpenRouter. The full data generation cost was approximately $50 in API usage. We'll load this pregenerated dataset, so you do not need to generate it yourself to follow along. If you are curious about the data-generation process, the code and usage instructions are available in the supplementary materials at **https://mng.bz/vZvx**.

8.3 Loading the MATH training dataset for distillation

We'll now load the distilled MATH dataset generated by DeepSeek-R1 in the previous section. Each example contains a math problem together with a reasoning trace and final answer, which we can use as targets for supervised fine-tuning. This step is highlighted in figure 8.6.

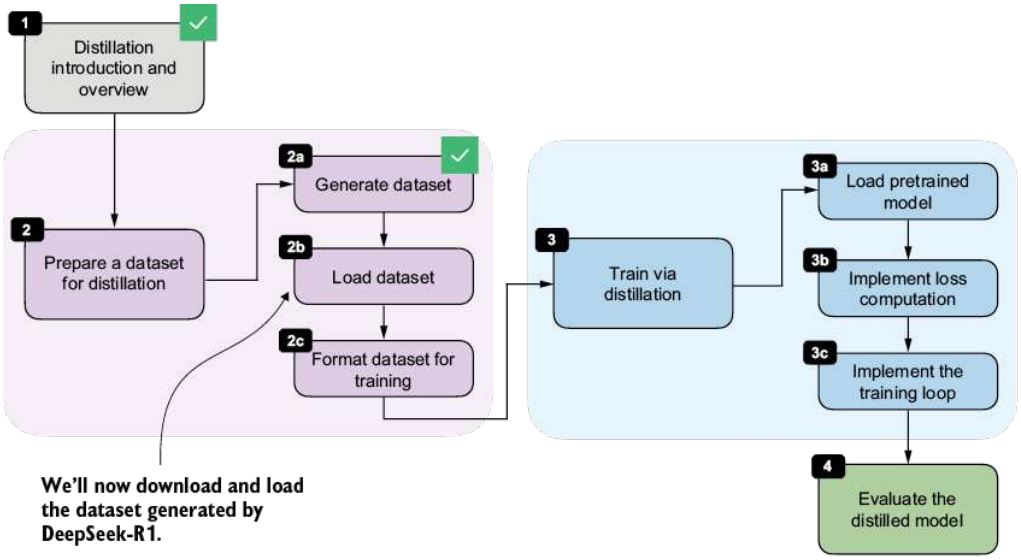


Figure 8.6 In this section, we'll load the DeepSeek-R1-generated dataset from a JSON file before preparing it for training.

I made the dataset available via the Hugging Face Hub rather than GitHub because the JSON file is approximately 107 MB, which exceeds GitHub's 100 MB file-size limit: **https://huggingface.co/datasets/rasbt/math_distill**. The following helper function downloads the selected partition if it is not already cached locally and returns it as a Python object.

Listing 8.1 Loading the distilled MATH training split

```
import json
import requests
from pathlib import Path


def load_distill_data(
    local_path=None,
    partition="deepseek-r1-math-train",
    save_copy=True,
):

    if local_path is None:
        local_path = f"{partition}.json"
    local_path = Path(local_path)

    url = (
        "https://huggingface.co/datasets/rasbt/math_distill"
        "/resolve/main/data/"
        f"{partition}.json"
    )
    backup_url = (
        "https://f001.backblazeb2.com/file/reasoning-from-scratch/"
        f"MATH/{partition}.json"
    )

    if local_path.exists(): #1
        with local_path.open("r", encoding="utf-8") as f:
            data = json.load(f)

        size_kb = local_path.stat().st_size / 1e3
        print(f"{local_path}: {size_kb:.1f} KB (cached)")
        return data

    assert partition in (
        "deepseek-r1-math-train",
        "deepseek-r1-math500",
        "qwen3-235b-a22b-math-train",
        "qwen3-235b-a22b-math500",
    )

    try:  #2
        r = requests.get(url, timeout=30)
        r.raise_for_status()
    except requests.RequestException:
        print("Using backup URL.")
        r = requests.get(backup_url, timeout=30)
        r.raise_for_status()

    data = r.json()

    if save_copy:  #3
        with local_path.open("w", encoding="utf-8") as f:
            json.dump(data, f, indent=2)

        size_kb = local_path.stat().st_size / 1e3
        print(f"{local_path}: {size_kb:.1f} KB")

    return data
```

#1 Reuse a cached copy if the dataset was already downloaded.

#2 Try downloading from Hugging Face first.

#3 Save a local copy so later runs can skip the download.

The output is as follows:

```
deepseek-r1-math-train.json: 107538.0 KB
Dataset size: 12000
```

Let's inspect one of the training examples so we can understand the dataset structure and see exactly which fields the teacher-generated data contains. For illustration, we'll use the fifth training example:

```
from pprint import pprint
pprint(math_train[4])
```

The printed output is as follows:

```
{'gtruth_answer': '6',
 'message_content': 'Sam worked \\( x \\) days and did...'
 'message_thinking': "Okay, let's see. Sam was hired for 20 days..."
 'problem': 'Sam is hired for a 20-day period...'
}
```

Each dataset entry contains the math problem itself (`problem`), the ground-truth answer (`gtruth_answer`), the teacher's reasoning trace in `message_thinking` , and the final answer in `message_content` . For distillation, the two most important fields are the reasoning trace and the final answer, because together they form the target text that the student should learn to reproduce.

The `format_distilled_answer` function in the following listing combines these two fields into a single training target by placing the reasoning trace inside `<think>...</think>` tags and then appending the final answer.

Listing 8.2 Formatting teacher responses for distillation

```
def format_distilled_answer(entry):
    content = str(entry["message_content"]).strip()
    if not content:
        raise ValueError("Missing non-empty 'message_content' field.")

    thinking = str(entry["message_thinking"]).strip()
    return f"<think>{thinking}</think>\n\n{content}"

print(format_distilled_answer(math_train[4]))
```

This is the printed output:

```
<think>Okay, let's see. Sam was hired for 20 days. Each day he works, he earns $60...So answer is 6 day
Sam worked \( x \) days and did not work \( y \) days. We know:...
Sam did not work \(\boxed{6}\) days.
```

As discussed in the previous chapter, the `<think></think>` tokens are optional. They are not required for distillation itself, though they can be useful for clearly separating the reasoning trace from the final answer. This separation becomes helpful when implementing user interfaces that hide the verbose reasoning trace from end users. Some systems, including products such as ChatGPT, may display only the final answer while hiding portions of the internal reasoning. Teaching the model to use explicit `<think>` tags makes these traces easier to parse and handle.

Since the dataset also contains ground-truth labels, we can measure how accurate the teacher model is on this set. For convenience, we'll use the `evaluate_json.py` script from chapter 3's supplementary materials, which compares generated answers with the reference answers using the verifier implemented in that chapter. This will give us a quick estimate of DeepSeek-R1's performance on the distillation dataset:

```
from reasoning_from_scratch.ch07 import download_from_github
_ = download_from_github(
    "ch03/02_math500-verifier-scripts/evaluate_json.py"
)
```

After downloading the script via the preceding Python code, we can run it in a code terminal as follows:

```
uv run evaluate_json.py \
--json_path "deepseek-r1-math-train.json" \
--gtruth_answer gtruth_answer \
--generated_text message_content
```

(If you don't use `uv`, replace `uv run` with `python`.)

This is the output:

```
Accuracy: 90.6% (10871/12000)
```

While the model is not perfect, 90.6% is still a relatively high level of accuracy. On the MATH-500 test set from chapter 3, it achieved 91.2% accuracy, which is much higher than the Qwen3 0.6B base model we are going to train (15.2% accuracy on MATH-500) and the official Qwen3 0.6B reasoning reference model (50.8% on MATH-500).

8.4 Building training examples

Next, we'll convert the raw dataset entries into training examples that can be consumed by the model. At a high level, this means formatting the prompts and answers consistently, tokenizing them, and storing the resulting token IDs together with the prompt length. This preprocessing stage is shown in figure 8.7.

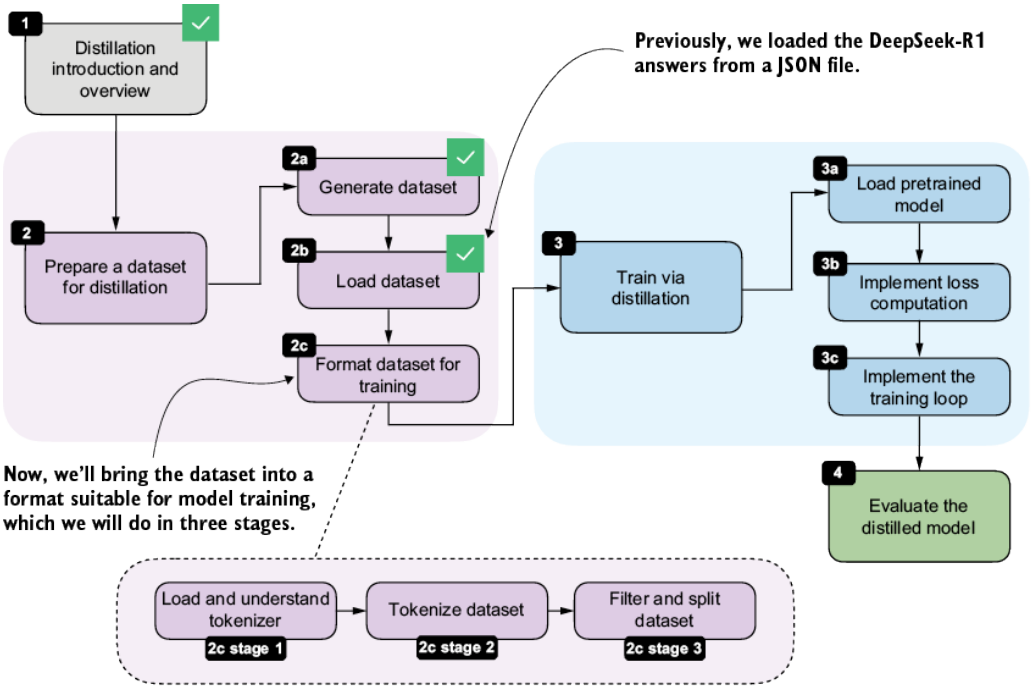


Figure 8.7 Bringing the loaded dataset into a format suitable for model training by understanding the tokenizer, tokenizing the examples, and filtering and splitting the dataset

Most of the work here is straightforward preprocessing. In the RLVR chapters, we performed similar preparation on the fly because each training example was sampled once and then discarded. Distillation is different, and we usually loop over the same examples for multiple *training epochs*. It is therefore more efficient to format and tokenize the dataset once, store the processed examples, and reuse them during training. This is also one reason distillation is often easier to iterate on than RLVR, once the teacher data has been collected.

WHAT ARE TRAINING EPOCHS?

A *training epoch*, or *epoch* for short, is a standard machine learning and deep learning term. An epoch is one complete pass through the full training dataset. For example, if we train for three epochs, the model sees each training example three times, usually in a different order each time. Multiple epochs help the model gradually improve by revisiting the same data more than once.

8.4.1 Loading and understanding the tokenizer

We'll begin with the tokenizer. Here, we are using the Qwen3 reasoning tokenizer because it supports the `<think>...</think>` tokens introduced in chapter 7.

Listing 8.3 Loading the Qwen3 reasoning tokenizer

```
from reasoning_from_scratch.qwen3 import (
    download_qwen3_small,
    Qwen3Tokenizer,
)


def load_reasoning_tokenizer(local_dir="qwen3"):
    download_qwen3_small(
        kind="reasoning", tokenizer_only=True, out_dir=local_dir
    )

    tokenizer_path = Path(local_dir) / "tokenizer-reasoning.json"
    tokenizer = Qwen3Tokenizer(
        tokenizer_file_path=tokenizer_path,
        apply_chat_template=True,
        add_generation_prompt=True,
        add_thinking=True,
    )

    return tokenizer

tokenizer = load_reasoning_tokenizer()
```

We set `apply_chat_template=True` and `add_generation_prompt=True` so that the tokenizer applies the same prompt-formatting style used for Qwen3's chat and reasoning models. The following example shows the additional wrapper tokens that are inserted automatically.

```
prompt = "Sam is hired for a 20-day period..."
prompt_ids = tokenizer.encode(prompt)
decoded_prompt = tokenizer.decode(prompt_ids)
print(decoded_prompt)
```

The formatted text is as follows:

```
<|im_start|>user
Sam is hired for a 20-day period...<|im_end|>
<|im_start|>assistant
```

The `<|im_start|>user` marks the start of the user prompt, `<|im_end|>` marks the end of the prompt, and `<|im_start|>assistant` marks the start of the model response. This chat-style wrapping is optional, but it is a common convention for instruction and chat fine-tuning.

For the target answer, we'll disable this wrapping by setting `chat_wrapped=False`. Otherwise, both the prompt and the answer would introduce their own assistant-start tokens, which is not what we want when concatenating them into a single training sequence:

```
answer = (
    "<think>Okay, let me try to solve "
    "this problem...</think> \\boxed{4}"
)
answer_ids = tokenizer.encode(answer, chat_wrapped=False)
decoded_answer = tokenizer.decode(answer_ids)
print(decoded_answer)
```

As shown, the `chat_wrapped=False` setting suppressed the chat template for the answer tokens:

```
<think>Okay, let me try to solve this problem...</think> \boxed{4}
```

For training, we need the full sequence of token IDs: the wrapped prompt, followed by the teacher answer, followed by an end-of-sequence token. This is the sequence the model will see during next-token prediction:

```
token_ids = prompt_ids + answer_ids + [tokenizer.eos_token_id]
decoded_token_ids = tokenizer.decode(token_ids)
print(decoded_token_ids)
```

The formatted string is as follows:

```
<|im_start|>user
Sam is hired for a 20-day period...<|im_end|>
<|im_start|>assistant
<think>Okay, let me try to solve this problem...</think>\boxed{4}<|im_end|>
```

It is worth emphasizing that the reasoning tokenizer is mainly convenient because it already includes the `<think></think>` tokens. In principle, though, we could also use the base tokenizer. Likewise, the chat template is not strictly required. We will keep it here because it matches the formatting used by Qwen3's own reasoning models and helps keep the training and evaluation setup consistent.

If you later evaluate the model with the scripts from earlier chapters, remember to use the `--which_model " reasoning "` setting so that the evaluation uses the same tokenizer variant.

8.4.2 Formatting and tokenizing the dataset

Now that we've loaded the tokenizer, we can apply the formatting and tokenization steps to the whole dataset, as illustrated in figure 8.8.

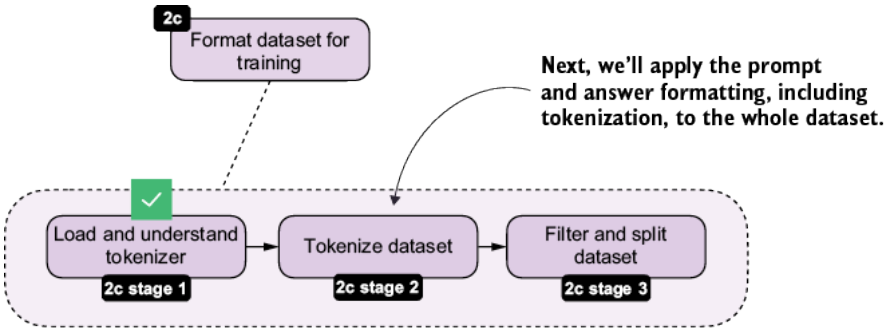


Figure 8.8 With tokenization complete, we can now apply the formatting and tokenization steps to the whole dataset.

With the tokenizer in place, we can now apply the formatting and tokenization steps consistently across the whole dataset via a `build_examples` function. The full formatting and tokenization pipeline for a single training sample is illustrated in figure 8.9.

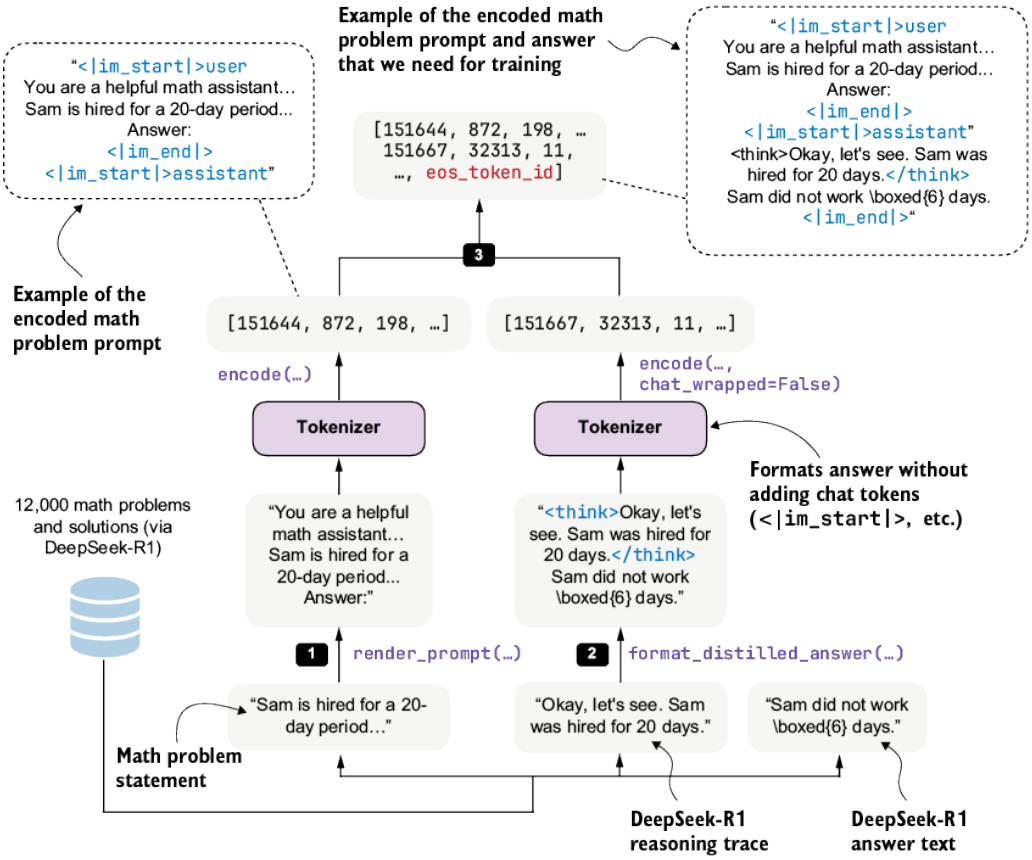


Figure 8.9 Example of the tokenization pipeline for one training sample. The math problem is rendered into the chat prompt format, the teacher reasoning trace and final answer are combined via `format_distilled_answer`, and both parts are concatenated into one token sequence.

Here, the `build_examples` function performs three steps. First, it renders and tokenizes the prompt. Second, it formats and tokenizes the teacher answer. Third, it concatenates both parts and records the prompt length so that we can later compute the loss only on the answer tokens. The following listing shows how we can implement that in code.

Listing 8.4 Building and inspecting tokenized distillation examples

```
from reasoning_from_scratch.ch03 import render_prompt


def build_examples(data, tokenizer):
    examples = []
    skipped = 0

    for entry in data:
        try:

            prompt = render_prompt(entry["problem"])#1
            prompt_ids = tokenizer.encode(prompt) #1


            target_answer = format_distilled_answer(entry)#2
            answer_ids = tokenizer.encode( #2
                target_answer, chat_wrapped=False #2
            ) #2


            token_ids = (#3
                prompt_ids + answer_ids + [tokenizer.eos_token_id] #3
            ) #3

            if len(token_ids) < 2:
                skipped += 1
                continue


            examples.append({#4
                "token_ids": token_ids, #4
                "prompt_len": len(prompt_ids), #4
            }) #4
        except (KeyError, TypeError, ValueError):

            skipped += 1#5
 #5
    return examples, skipped #5
 #5
 #5
examples, skipped = build_examples(math_train, tokenizer) #5
 #5
print("Number of examples:", len(examples)) #5
print("Number of skipped examples:", skipped) #5
```

#1 Step 1: Render the problem in the chat format
#2 Step 2: Tokenize the teacher reasoning trace and final answer
#3 Step 3: Combine prompt and answer for training
#4 Store the prompt length so we can ignore prompt tokens in the loss later.
#5 Skip the misformatted examples.

The resulting numbers after running the code in listing 8.4 are as follows:

```
Number of examples: 12000
Number of skipped examples: 0
```

Next, let's decode one of the training examples to inspect it further:

```
print(tokenizer.decode(examples[4]["token_ids"]))
```

This is the output:

```
<|im_start|>user
You are a helpful math assistant.
Answer the question and write the final result on a new line as:
\boxed{ANSWER}

Question:
Sam is hired for a 20-day period...

Answer:<|im_end|>
<|im_start|>assistant
<think>Okay, let's see. Sam was hired for 20 days.... So answer is 6 days not worked.</think>
...Sam did not work \(\boxed{6}\) days.<|im_end|>
```

Looking at the preceding output, we can confirm that it meets all the formatting requirements. For instance, it uses the chat template correctly, and the answer's reasoning trace is correctly enclosed in `<think></think>` tags. Finally, the answer ends with an end-of-sequence token, `<|im_end|>`.

8.4.3 Filtering and splitting the dataset

Once the examples are tokenized, we still need to filter long sequences and split the dataset into *training* and *validation* subsets. This step is highlighted in figure 8.10.

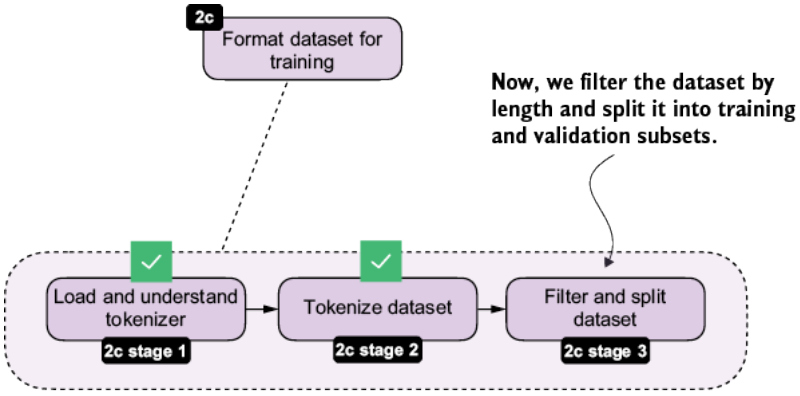


Figure 8.10 After tokenization, we filter out long sequences and split the remaining examples into training and validation subsets.

After tokenization, it's useful to inspect the sequence lengths. This tells us how long the examples are on average, which samples are extreme outliers, and how aggressive our filtering needs to be. We can then remove examples above the chosen maximum length, shuffle the remaining data with a fixed random seed, and split off a small validation set.

Let's begin by analyzing the sequence lengths.

Listing 8.5 Computing lengths and filtering long examples

```
def compute_length(examples, answer_only=False):
    lengths = []
    for ex in examples:
        total = len(ex["token_ids"])
        length = total - ex["prompt_len"] if answer_only else total
        lengths.append(length)

    avg_len = round(sum(lengths) / len(lengths))

    shortest_len = min(lengths)
    longest_len = max(lengths)
    shortest_idx = lengths.index(shortest_len)
    longest_idx = lengths.index(longest_len)

    print(f"Average: {avg_len} tokens")
    print(f"Shortest: {shortest_len} tokens (index {shortest_idx})")
    print(f"Longest: {longest_len} tokens (index {longest_idx})")

compute_length(examples)
```

This is the resulting output:

```
Average: 2946 tokens
Shortest: 236 tokens (index 10846)
Longest: 42005 tokens (index 2529)
```

As you can see, the average response length is 2,946 tokens, which is typical for reasoning models. There are outliers, though. For instance, the longest answer is 42,005 tokens, which is excessive. The index positions (`index 10846` and `index 2529`) denote the positions of the shortest and longest examples in the dataset, respectively, in case we want to inspect them.

To keep the computational costs reasonable for this distillation example, we'll filter the dataset to include only entries of up to 2,048 tokens using the code in listing 8.6. In practice, controlling sequence length is one of the main steps that makes distillation feasible on smaller hardware.

Listing 8.6 Filtering long examples

```
def filter_examples_by_max_len(examples, max_len=2048):
    filtered_examples = [
        s for s in examples
        if len(s["token_ids"]) <= max_len
    ]

    print("Original:", len(examples))
    print("Filtered:", len(filtered_examples))
    print("Removed:", len(examples) - len(filtered_examples))

    return filtered_examples


filtered_examples = filter_examples_by_max_len(examples, max_len=2048)
```

The filtering code in listing 8.6 removes 5,305 training examples:

```
Original: 12000
Filtered: 6695
Removed: 5305
```

Let's compute the dataset lengths on this new subset:

```
compute_length(filtered_examples)
```

As you can see, the average token length is now down to 1,180 tokens, and none of the formatted training examples exceed 2,048 tokens:

```
Average: 1180 tokens
Shortest: 236 tokens (index 5971)
Longest: 2048 tokens (index 5587)
```

Lastly, we'll split the dataset into training and validation sets, where the latter are used for quick evaluations throughout the training run.

Listing 8.7 Partitioning into training and validation sets

```
import random

rng = random.Random(123)
rng.shuffle(filtered_examples)

train_examples = filtered_examples[25:]
val_examples = filtered_examples[:25]

print("Number of train examples:", len(train_examples))
print("Number of validation examples:", len(val_examples))
```

The resulting numbers of training and validation examples are as follows:

```
Number of train examples: 6670
Number of validation examples: 25
```

We are keeping the validation set purposefully small so that we don't unnecessarily slow down the training loop. In addition, after training is completed, we will use the 500 samples from the MATH-500 set to evaluate the model's performance.

EXERCISE 8.1: TRAINING AND VALIDATION SET LENGTHS

Apply the `compute_length` function to the new `train_examples` and `val_examples` partitions to check whether they are balanced by sample length.

8.5 Loading a pretrained model

With the dataset preparation complete, we can turn to the actual distillation training. We'll begin by loading the pretrained Qwen3 base model, as shown in figure 8.11.

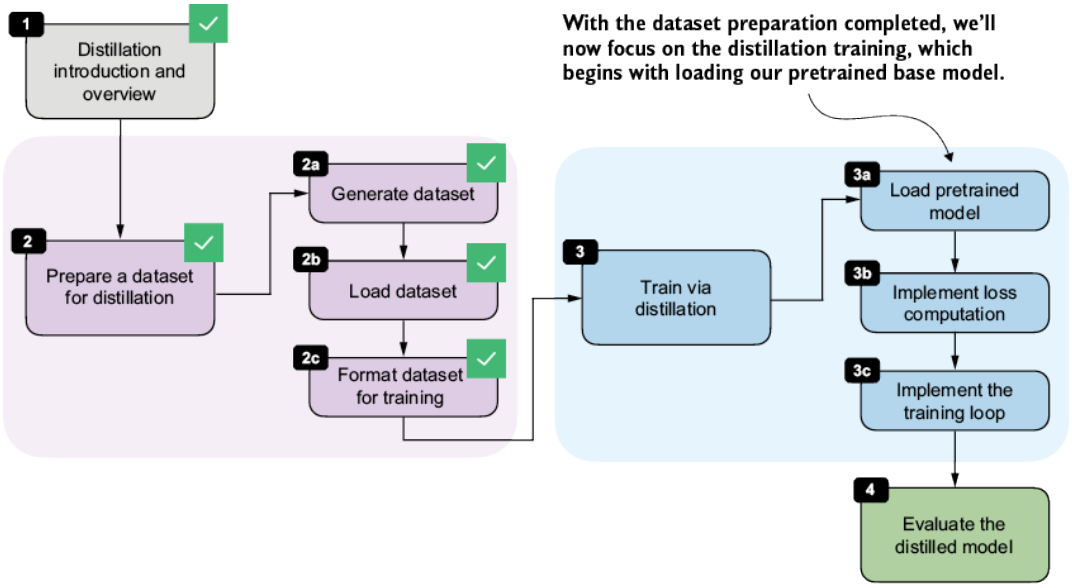


Figure 8.11 With the dataset preparation complete, we begin the distillation training by loading the pretrained Qwen3 base model.

We'll start with the pretrained Qwen3 base model, not from an RL-trained model, because distillation itself is the training stage we want to study here. This keeps our setup clean and makes it easier to attribute any improvement to the distilled reasoning traces. It also mirrors a common practical setup in which a general base model is adapted using teacher-generated data.

Listing 8.8 Loading the Qwen3 base model for distillation

```
import torch

from reasoning_from_scratch.ch02 import get_device
from reasoning_from_scratch.ch03 import (
    load_model_and_tokenizer,
)

device = get_device()

model, _ = load_model_and_tokenizer(
    which_model="base",
    device=device,
    use_compile=False,
)
```

We can ignore the loaded base tokenizer in listing 8.8 because we'll be using the tokenizer with `<think>` `</think>` token support that we loaded in section 8.4.1.

8.6 Computing the training and validation losses

Next, we implement the *cross-entropy loss* that will serve as the training signal during distillation when we implement the training loop later. We will also reuse the same computation on the validation set to monitor progress during training.

If you have a deep learning background, you may already know cross-entropy from classification tasks. The idea is the same here, except that the target class we want to predict at each position is the next token in the teacher-generated sequence.

We can connect the cross-entropy loss directly to the log-probability computations from chapters 5 and 6. As discussed there, log probability measures how much probability the model assigns to the correct next token. Higher log probability means the model is more confident in the correct token, and lower log probability means the opposite. Reusing our "The capital of Germany is Berlin" example from previous chapters, figure 8.12 recaps this.

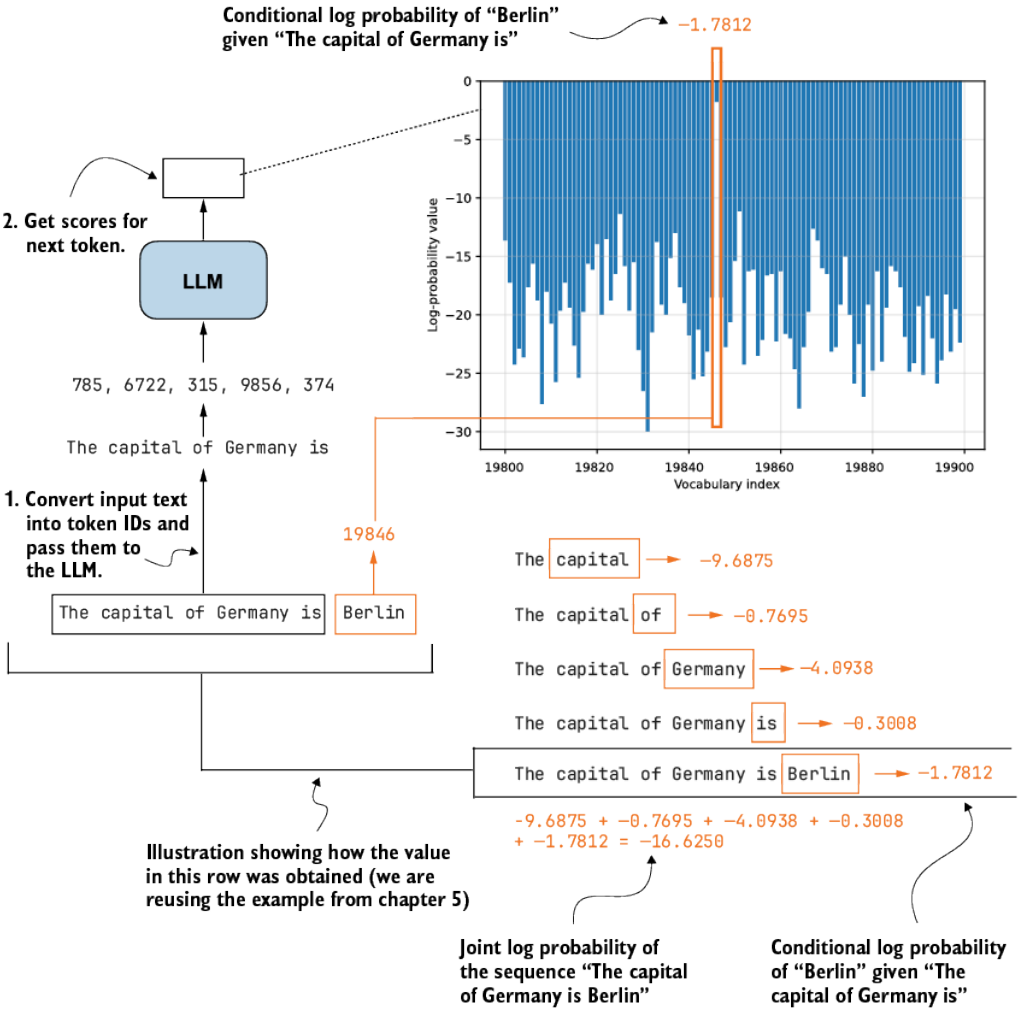


Figure 8.12 Illustration of token and sequence log probabilities. The log probabilities of the correct next tokens are summed to obtain the sequence log probability, which is the basis for the cross-entropy loss used later in this chapter.

Cross-entropy is simply the negative average of the token log probabilities shown in figure 8.12. For instance, if –16.6250 is the log probability, the negative log probability is 16.6250, and the negative average log probability (or cross-entropy) is 16.6250/5 = 3.325. Note that the 5 is there because the sequence has 5 target predictions whose log probabilities are being averaged: `" capital "`, `" of "`, `" Germany "`, `" is "`, and `" Berlin "`. Similar to log probability, the closer the cross-entropy value is to 0, the better, because the model is more confident in the correct target token.

The `sequence_logprob` function from chapter 6 performs the computation shown in figure 8.12, which makes it a useful starting point for understanding how the *distillation loss* works. In this case, we'll use the training example at index position `5730` because it is the shortest example in `train_examples`, as we can determine via `compute_length(train_examples)`, and therefore computes a bit more quickly than the other examples:

```
token_ids = train_examples[5730]["token_ids"]
prompt_len = train_examples[5730]["prompt_len"]
```

Instead of reporting the summed log probabilities returned by `sequence_logprob`, we'll average them over the number of answer tokens. Summed log probabilities grow with sequence length, so they are not directly comparable across examples with shorter or longer answers. By dividing by the number of answer tokens, we'll obtain a per-token quantity, which matches the form used by cross-entropy loss. We can compute the number of answer tokens by subtracting `prompt_len` from the total sequence length.

Listing 8.9 Computing average negative log probabilities

```
from reasoning_from_scratch.ch06 import sequence_logprob

tok = torch.tensor(token_ids, dtype=torch.long, device=device)

with torch.no_grad():
    seq_logprob = sequence_logprob(model, tok, prompt_len)
    num_answer_tokens = tok.numel() - prompt_len
    avg_logprob = -seq_logprob / num_answer_tokens
print(f"Average logprob: {avg_logprob:.2f}")
```

The resulting negative average log probability is 1.68.

We can now compute the same quantity using PyTorch's `cross_entropy` function. Although the function is usually introduced for classification, it's a natural choice here. For example, the model logits provide the predicted class distribution over the vocabulary, and the target sequence provides the correct class label at each position.

As in the previous average-logprob calculation, we'll compute the loss only over the answer tokens and ignore the prompt tokens (the prompt is the context provided as input and not something we want the model to be penalized for reproducing). This is illustrated in figure 8.13.

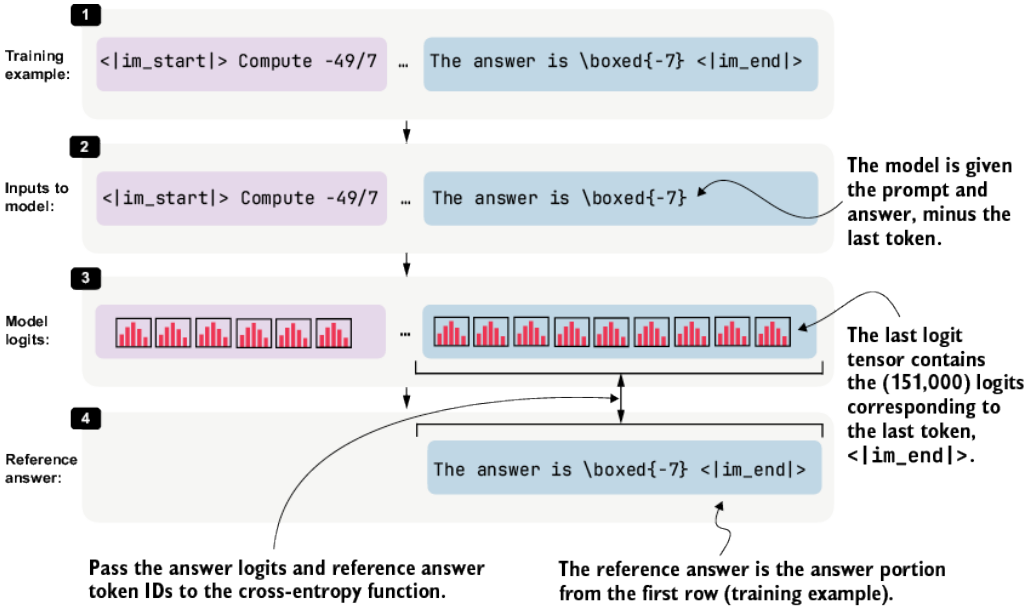


Figure 8.13 Input for the cross-entropy loss over the answer tokens. The model receives the prompt and answer, shifted by one token, as input, and the answer token logits are compared with the reference answer tokens.

During distillation, we want the student to learn the teacher's reasoning trace and final answer conditioned on the prompt. So, as shown in figure 8.13, we discard the logits and targets that correspond only to the prompt portion of the sequence when computing the cross-entropy.

Listing 8.10 Computing cross-entropy loss directly

```
input_ids = tok[:-1].unsqueeze(0)#1
target_ids = tok[1:] #1
logits = model(input_ids).squeeze(0) #1

first_answer_logit_idx = max(prompt_len - 1, 0)#2
answer_logits = logits[first_answer_logit_idx:] #2
answer_targets = target_ids[first_answer_logit_idx:] #2

with torch.no_grad():#3
    ce_mean_direct = torch.nn.functional.cross_entropy( #3
        answer_logits, answer_targets #3
    ) #3
print(f"Cross-entropy: {ce_mean_direct:.2f}") #3
```

#1 Shift the sequence by one token so each input predicts the next token target.
#2 Drop the prompt positions so the loss only covers the teacher answer.
#3 Compute the cross-entropy loss.

The resulting cross-entropy loss is 1.68, which is similar to that in listing 8.9, where we used the `sequence_logprob` function. In other words, `cross_entropy` implements the same core calculation in a more optimized way. For training, we'll therefore use the built-in `cross_entropy` function rather than our custom log-probability function.

The `compute_example_loss` helper in listing 8.11 wraps this logic into a convenient function that calculates the *answer-only loss* for a single example. "Answer-only loss" means the training loss is computed only on the teacher's answer tokens, not on the prompt tokens. The following listing puts it all together into a function that applies the whole logic, from target preparation to cross-entropy computation.

Listing 8.11 Defining the loss for one distillation example

```
def compute_example_loss(model, example, device):
    token_ids = example["token_ids"]
    prompt_len = example["prompt_len"]

    input_ids = torch.tensor(#1
        token_ids[:-1], dtype=torch.long, device=device #1
    ).unsqueeze(0) #1
    target_ids = torch.tensor( #1
        token_ids[1:], dtype=torch.long, device=device #1
    ) #1
 #1
    logits = model(input_ids).squeeze(0)

    answer_start = max(prompt_len - 1, 0)#2
    answer_logits = logits[answer_start:] #2
    answer_targets = target_ids[answer_start:] #2

    loss = torch.nn.functional.cross_entropy(#3
        answer_logits, answer_targets #3
    ) #3
    return loss #3
 #3
with torch.no_grad():#4
    loss = compute_example_loss( #4
        model, train_examples[5730], device #4
    ) #4
 #4
print(f"Loss: {loss:.2f}")
```

#1 Create input-target pairs that are shifted by one token.
#2 Ignore prompt tokens so the loss is computed on the distilled answer only.
#3 Compute the cross-entropy loss.
#4 Used to verify that the helper returns the same loss as before

The resulting loss is 1.68 again, which indicates that the function in listing 8.11 works as intended.

BATCHING

It is possible to process multiple examples in parallel by batching them together. I've omitted batching here to keep the implementation compact and lower the resource requirements. Appendix E discusses batching and throughput-oriented execution in more detail for loss computation and training in general.

Next, we'll define a small wrapper that iteratively computes the average loss across multiple examples. This will be useful both for quick sanity checks and for tracking the validation loss during training.

Listing 8.12 Evaluating loss across multiple examples

```
@torch.no_grad()
def evaluate_examples(model, examples, device):
    was_training = model.training

    model.eval()#1
    total_loss = 0.0 #1
    num_examples = 0 #1

    for example in examples:#2
        loss = compute_example_loss(model, example, device) #2
        total_loss += loss.item() #2
        num_examples += 1 #2

    if was_training:  #3
        model.train() #3

    return total_loss / num_examples  #4

train_loss = evaluate_examples(model, train_examples[:3], device)  #5
print(f"Train loss (3 examples): {train_loss:.2f}")  #5

val_loss = evaluate_examples(model, val_examples[:3], device)
print(f"Validation loss (3 examples): {val_loss:.2f}")
```

#1 Temporarily switch to evaluation mode while scoring the examples.
#2 Sum the loss over all examples.
#3 Restore training mode so this helper is safe to call during training.
#4 Average the loss over all examples.
#5 Estimate the current training loss on a small subset.

This is the output:

```
Train loss (3 examples): 0.98
Validation loss (3 examples): 1.02
```

We'll reuse this evaluation function during training. Ideally, both quantities should decrease over time, which would indicate that the student model is becoming better at matching the teacher-generated target sequences.

In practice, the *training loss*, which is the loss computed on the training examples used for optimization, is often noisier because it is measured on the examples currently being optimized and can fluctuate depending on the sample order and recent parameter updates.

The *validation loss*, which is measured on a separate held-out validation set rather than on the examples currently being used for optimization, is computed without updating the model weights. As a result, it usually provides a cleaner signal of whether the student is improving in a way that generalizes beyond the training set. For this reason, the validation loss is often the more reliable metric to watch.

8.7 Implementing the training loop for distillation

With the dataset prepared and the loss computation in place, we can now implement the training loop for distillation. This stage is highlighted in figure 8.14.

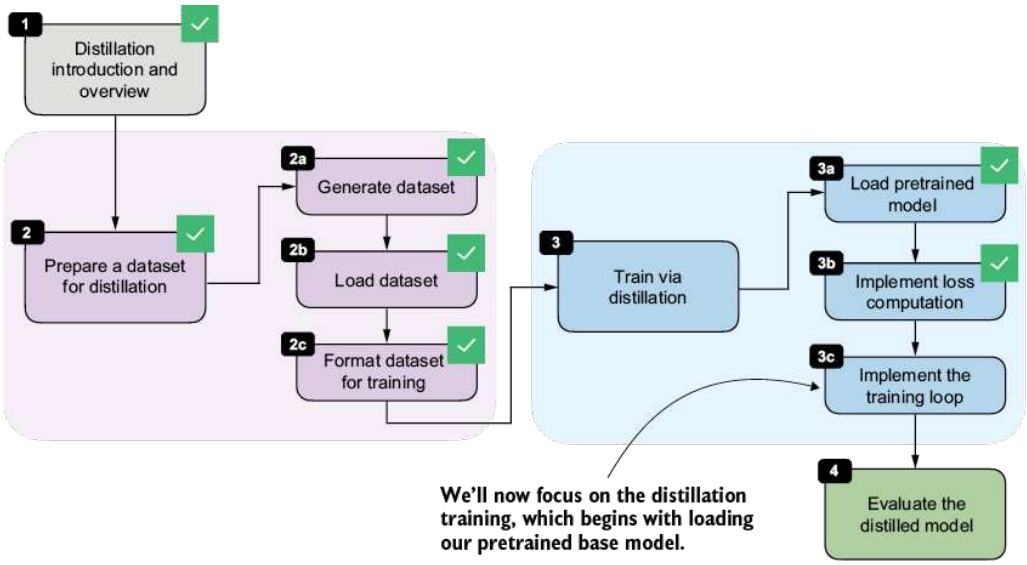


Figure 8.14 With the dataset preparation and loss computation complete, we can now turn to the training loop for distillation.

The training loop we'll use here is very similar to the one in chapter 6. The main difference is that here we'll revisit the same training set multiple times across epochs and optimize the cross-entropy distillation loss instead of the GRPO objective (GRPO loss) used in RLVR. The detailed steps are shown in figure 8.15.

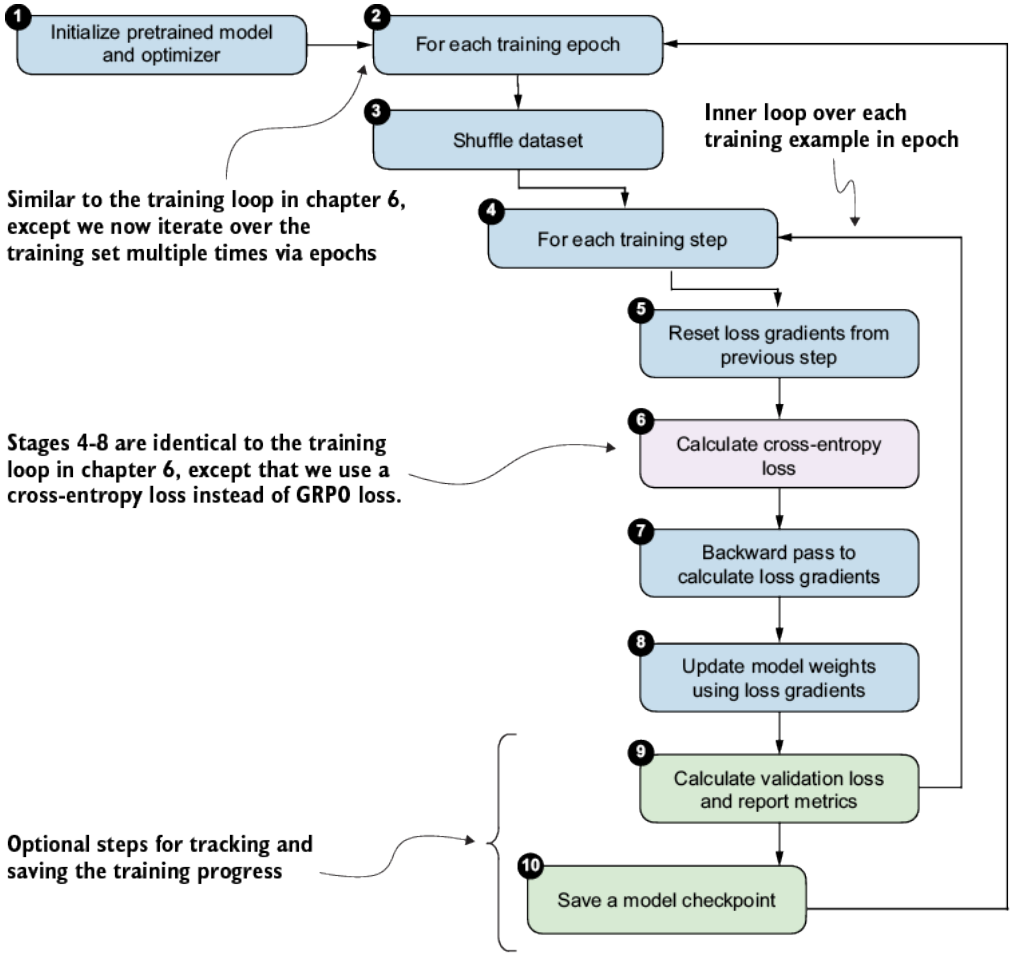


Figure 8.15 The distillation training loop. In each epoch, the training examples are shuffled, the student model computes a cross-entropy loss for each example, gradients are backpropagated, and the model weights are updated. The validation loss is reported at certain intervals to track progress.

The following `train_distillation` function implements the loop shown in figure 8.15. It shuffles the training examples at the start of each epoch, computes the loss for each example, applies an optimizer step, optionally clips large gradients, and periodically evaluates on the validation set. The metrics are also written to a CSV file so we can inspect the learning curves later.

Listing 8.13 Implementing the distillation training loop

```
import time
```

```
def train_distillation(
    model,
    train_examples,
    val_examples,
    device,
    epochs=2,
    lr=5e-6,
    grad_clip_norm=None,
    seed=123,
    log_every=50,
    checkpoint_dir="checkpoints",
    csv_log_path=None,
):

    optimizer = torch.optim.AdamW(model.parameters(), lr=lr)#1
    model.train()

    total_steps = epochs * len(train_examples)
    global_step = 0
    rng = random.Random(seed)

    if csv_log_path is None:
        timestamp = time.strftime("%Y%m%d_%H%M%S")
        csv_log_path = f"train_distill_metrics_{timestamp}.csv"
    csv_log_path = Path(csv_log_path)


    for epoch in range(1, epochs + 1):#2

        epoch_examples = list(train_examples)#3
        rng.shuffle(epoch_examples) #3


        for example in epoch_examples:#4
            global_step += 1


            optimizer.zero_grad()#5


            loss = compute_example_loss(model, example, device)#6


            loss.backward()#7


            if grad_clip_norm is not None:#8
                torch.nn.utils.clip_grad_norm_( #8
                    model.parameters(), grad_clip_norm #8
                ) #8


            optimizer.step()#9


            if log_every and global_step % log_every == 0:#10
                val_loss = evaluate_examples( #10
                    model=model, #10
                    examples=val_examples, #10
                    device=device, #10
                ) #10
                model.train() #10
                print(
                    f"[Epoch {epoch}/{epochs} "
                    f"Step {global_step}/{total_steps}] "
                    f"train_loss={loss.item():.4f} "
                    f"val_loss={val_loss:.4f}"
```

```
                )
                append_csv_metrics(
                    csv_log_path=csv_log_path,
                    epoch_idx=epoch,
                    total_steps=global_step,
                    train_loss=loss.item(),
                    val_loss=val_loss,
                )


            ckpt_path = save_checkpoint(#11
                model=model, #11
                checkpoint_dir=checkpoint_dir, #11
                step=global_step, #11
                suffix=f"epoch{epoch}", #11
            ) #11
            print(f"Saved checkpoint to {ckpt_path}") #11
        return model #11
     #11

    def save_checkpoint(model, checkpoint_dir, step, suffix=""):
        checkpoint_dir = Path(checkpoint_dir)
        checkpoint_dir.mkdir(parents=True, exist_ok=True)
        suffix = f"-{suffix}" if suffix else ""
        ckpt_path = (
            checkpoint_dir /
            f"qwen3-0.6B-distill-step{step:05d}{suffix}.pth"
        )
        torch.save(model.state_dict(), ckpt_path)
        return ckpt_path


    def append_csv_metrics(
        csv_log_path,
        epoch_idx,
        total_steps,
        train_loss,
        val_loss,
    ):
        if not csv_log_path.exists():
            csv_log_path.write_text(
                "epoch,total_steps,train_loss,val_loss\n",
                encoding="utf-8",
            )
        with csv_log_path.open("a", encoding="utf-8") as f:
            f.write(
                f"{epoch_idx},{total_steps},{train_loss:.6f},"
                f"{val_loss:.6f}\n"
            )
```

#1 Step 1: Initialize optimizer (model is already loaded)
#2 Step 2: Iterate over training epochs
#3 Step 3: Shuffle the training examples at the start of the epoch
#4 Step 4: Iterate over training examples in epoch
#5 Stage 5: Reset loss gradient
#6 Step 6: Compute the cross-entropy loss for the current example
#7 Step 7: Backpropagate gradients
#8 Optionally clip large gradients to improve training stability
#9 Step 8: Update the model weights
#10 Step 9: Periodically evaluate the current model on the validation set
#11 Step 10: Record the metrics and save a checkpoint for this epoch

With a maximum sequence length of 2,048, the full training run requires about 15 GB of GPU memory. If this is too high for your hardware, you can lower the resource requirements by filtering out longer sequences earlier in the notebook, such as by changing `max_len` from `2048` to `1024` or `512` in the `filter_examples_by_max_len` step in listing 8.6.

Let's execute a short training run.

Listing 8.14 Training the model

```
torch.manual_seed(0) #1

train_distillation(
    model,
    train_examples=train_examples[:10], #2
    val_examples=val_examples[:10],      #2
    device=device,
    epochs=2,
    lr=5e-6,
    grad_clip_norm=1.0,  #3
    seed=123,
    log_every=5,
    csv_log_path="train_distill_metrics.csv",
)
```

#1 Seed PyTorch so the short demo is reproducible.
#2 Train on a tiny subset so this notebook run stays lightweight.
#3 Same as in chapter 6

Let's briefly talk about the settings before we inspect the results. We are keeping the training run intentionally short. For instance, we use only the first 10 training examples and 10 validation examples, and we train for just 2 epochs. This is purely to keep the notebook run lightweight and fast enough for experimentation. For a real distillation run, we would of course train on many more examples, ideally, the whole training set.

The learning rate (`lr=5e-6`) is in a reasonable range for fine-tuning a pretrained model and worked well for this setup, as reflected in the loss curves discussed shortly.

The gradient clipping setting (`grad_clip_norm=1.0`) is the same as in chapter 6 and helps prevent unstable updates when a particular example produces unusually large gradients.

The `log_every=5` setting means that validation loss is measured every 5 training steps. Since this demo uses only a handful of examples, this produces frequent progress updates so that we can quickly verify that the training loop behaves as expected. In a larger run, we would usually increase this interval to reduce evaluation overhead.

Finally, the `csv_log_path= " train_distill_metrics.csv "` argument stores the training and validation losses in a CSV file so that we can inspect and plot them later.

The main goal of this run is not to achieve the best possible reasoning performance, but to confirm that the distillation pipeline works from end to end. Once that is established, we can move on to larger runs and inspect the resulting learning curves and checkpoints in more detail.

Let's now take a brief look at the run's output:

```
[Epoch 1/2 Step 5/20] train_loss=0.9648 val_loss=0.9082
[Epoch 1/2 Step 10/20] train_loss=0.9844 val_loss=0.8871
Saved checkpoint to checkpoints/qwen3-0.6B-distill-step00010-epoch1.pth
[Epoch 2/2 Step 15/20] train_loss=0.8008 val_loss=0.8707
[Epoch 2/2 Step 20/20] train_loss=0.7148 val_loss=0.8586
Saved checkpoint to checkpoints/qwen3-0.6B-distill-step00020-epoch2.pth
```

Even though this is only a tiny demonstration run, the output already shows the expected overall behavior. Both the training loss and the validation loss decrease over time, which indicates that the student model is becoming better at matching the teacher-generated target sequences. The validation loss is especially useful here because it is computed on held-out examples and therefore provides a cleaner signal that the improvement is not limited to the training samples alone.

We can also see that checkpoints are saved at the end of each epoch. This is useful because it lets us resume training later or evaluate intermediate versions of the distilled model using the MATH-500 test set.

8.8 Evaluating the distilled model

Now that we've implemented the training loop, the final stage is evaluating the distilled model, as shown in figure 8.16.

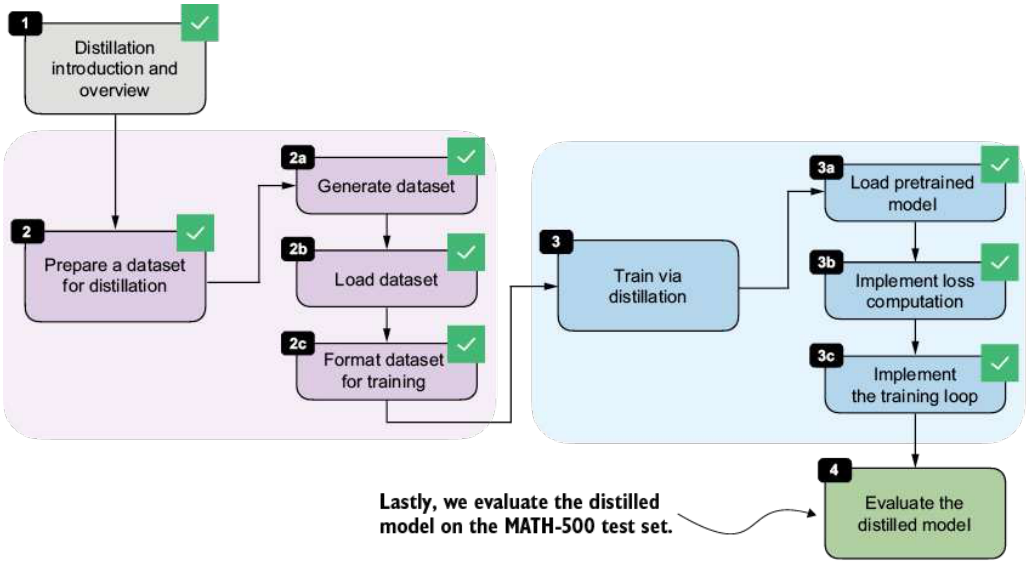


Figure 8.16 Now we'll evaluate the distilled model on the MATH-500 test set.

Instead of running the full distillation process inside the notebook, you can download a convenience script from the chapter's supplementary materials. As in the previous chapter on RLVR, it is often convenient to keep the notebook focused on the core ideas and move the longer-running training job into a standalone script:

```
from reasoning_from_scratch.ch07 import download_from_github
download_from_github(
    "ch08/04_train_with_distillation/distill.py"
)
```

After downloading the script with the preceding code, you can run it as follows in a code terminal (if you are not a `uv` user, replace `uv run` with `python`):

```
uv run distill.py \
--data_path deepseek-r1-math-train.json \
--validation_size 25 \
--epochs 3 \
--lr 1e-5 \
--max_seq_len 2048 \
--use_think_tokens \
--grad_clip 1.0
```

Using these settings, the full training run takes about 3 hours and 5 minutes on a DGX Spark and uses roughly 15.02 GB of GPU memory. (This is relatively modest compared with the earlier RLVR runs because most of the expensive work has already been moved into the one-time teacher data generation step.) If you do not want to run the training yourself, you can download the resulting metrics file and inspect it directly:

```
download_from_github(
    "ch08/03_logs/deepseek-r1-2048_distill_metrics.csv"
)
```

The following listing implements a utility function to visualize the training metrics stored in the CSV file.

Listing 8.15 Plotting distillation losses from a CSV log

```
import csv
import matplotlib.pyplot as plt


def plot_distill_metrics(csv_path="train_distill_metrics.csv"):
    total_steps, train_losses, val_losses, epoch_bounds = [], [], [], {}


    with open(csv_path, newline="", encoding="utf-8") as f:#1
        for row in csv.DictReader(f):
            step = int(row["total_steps"])
            epoch = int(row["epoch"])
            total_steps.append(step)#2
            train_losses.append(float(row["train_loss"]))
            val_losses.append(float(row["val_loss"]))
            epoch_bounds.setdefault(epoch, [step, step])[1] = step

    fig, ax = plt.subplots(figsize=(7, 4))
    ax.plot(total_steps, train_losses, label="train_loss", alpha=0.3)
    ax.plot(total_steps, val_losses, label="val_loss")
    ax.set_xlabel("Total Steps")
    ax.set_ylabel("Loss")
    ax.legend()


    epoch_axis = ax.secondary_xaxis("bottom")#3
    epoch_axis.spines["bottom"].set_position(("outward", 45)) #3
    epochs = sorted(epoch_bounds) #3
    epoch_axis.set_xticks( #3
        [
            (epoch_bounds[epoch][0] + epoch_bounds[epoch][1]) / 2
            for epoch in epochs
        ]
    )
    epoch_axis.set_xticklabels(epochs)
    epoch_axis.set_xlabel("Epoch")

    plt.tight_layout()
    plt.show()


plot_distill_metrics("deepseek-r1-2048_distill_metrics.csv")
```

#1 Load and plot the logged losses.

#2 Load and plot the logged losses.

#3 Add a second x-axis so the epoch numbers are visible below the step axis.

The resulting plot is shown in figure 8.17.

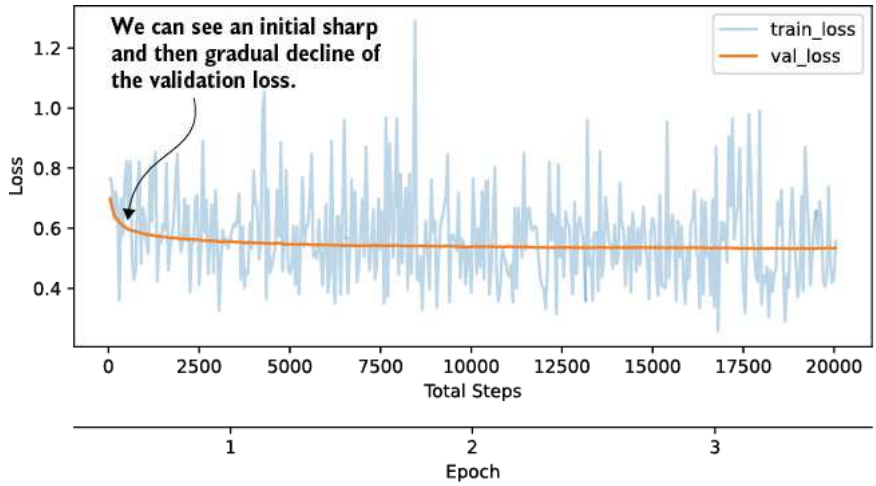


Figure 8.17 The training and validation losses of a 3-epoch distillation training run on DeepSeek-R1 reasoning traces

The training-loss curve and validation-loss curve shown in figure 8.17 should be interpreted slightly differently. The training loss is computed for each training example during training at the current step, whereas the validation loss is computed on a small, fixed, held-out set. The validation curve is therefore less noisy and is the more informative signal here. We could also compute the training loss on a subset of examples, similar to the validation loss, but this would slow down training. The training loss is much less important than the validation loss for estimating training progress.

As we would hope, the validation loss drops sharply at first and then begins to flatten out, indicating that the model is learning from the distillation data but that additional training yields diminishing returns. We could experiment with a larger learning rate or a different schedule to train more aggressively, but overall the curve looks healthy.

Just as in chapter 7, we can evaluate saved checkpoints with the verifier-based utilities from chapter 3. The following command downloads the evaluation script from the supplementary materials:

```
download_from_github(
    "ch03/02_math500-verifier-scripts/evaluate_math500.py"
)
```

Evaluate the distilled checkpoint on MATH-500 using the reasoning tokenizer in a code terminal, as follows:

```
uv run evaluate_math500.py \
--dataset_size 500 \
--which_model reasoning \
--max_new_tokens 4096 \
--checkpoint_path \
"run_11/checkpoints/distill/qwen3-0.6B-distill-step06682-epoch1.pth"
```

For the later checkpoints from the same DeepSeek-R1 run, replace `...step06682-epoch1.pth` with `...step13364-epoch2.pth`, `...step20046-epoch3.pth`, and so on.

The evaluation results for the DeepSeek-R1 run used in this chapter are summarized in table 8.1. For reference, I've also included a second run trained on Qwen3 235B-A22B teacher outputs (a 235-billion-parameter Qwen3 model).

Table 8.1 MATH-500 task accuracy for different model checkpoints

	Method	Epoch	Final validation loss	MATH-500 accuracy
1	Base Qwen3 0.6B (chapter 3)	-	-	15.2%
2	Reasoning Qwen3 0.6B (chapter 3)	-	-	48.2%
3	DeepSeek-R1	1	0.5436	30.6%
4	DeepSeek-R1	2	0.5349	32.4%
5	DeepSeek-R1	3	0.5343	33.6%
6	Qwen3 235B-A22B	1	0.4043	45.0%
7	Qwen3 235B-A22B	2	0.3963	43.8%
8	Qwen3 235B-A22B	3	0.3948	44.2%

Based on the results for the DeepSeek-R1 run shown in table 8.1, the MATH-500 accuracy improves from 30.6% after the first epoch to 33.6% after the third epoch, while the validation loss decreases from 0.5436

to 0.5343. This matches the earlier learning curve in figure 8.17, where the student clearly learns from the teacher-generated reasoning traces, but the gains begin to taper off after the initial improvement.

The Qwen3 235B-A22B run performs noticeably better in this setup. One likely reason is that the teacher and student come from the same model family, which means that the tokenizer, prompting conventions, and overall response style are more closely aligned. This can make the teacher targets easier for the smaller Qwen3 student to imitate.

Considering the MATH-500 accuracy of 45% (row 6), our distillation recipe reaches nearly the same performance as the reasoning reference model (48.2%, row 2), which is itself a distilled model but was trained on a much larger dataset generated by Qwen3 235B-A22B. This is the main tradeoff of distillation.

In general, we do not expect the smaller student to match the teacher exactly. For instance, Qwen3 235B-A22B has a MATH-500 accuracy of 92.4%, and DeepSeek-R1 has a MATH-500 accuracy of 91.2%. Still, we can recover a useful portion of the teacher's reasoning behavior in a much cheaper model. Our distilled model accuracy could also be higher if we used a larger model (e.g., a 4- or 30-billion-parameter version of Qwen3 instead of Qwen3 0.6B), but this would increase the computational cost during training.

Let's step back and connect the individual pieces we implemented into one complete workflow. Figure 8.18 summarizes the full distillation pipeline, starting with the introductory setup, followed by dataset generation and preprocessing, then the distillation training loop itself, and finally the evaluation of the distilled model on MATH-500.

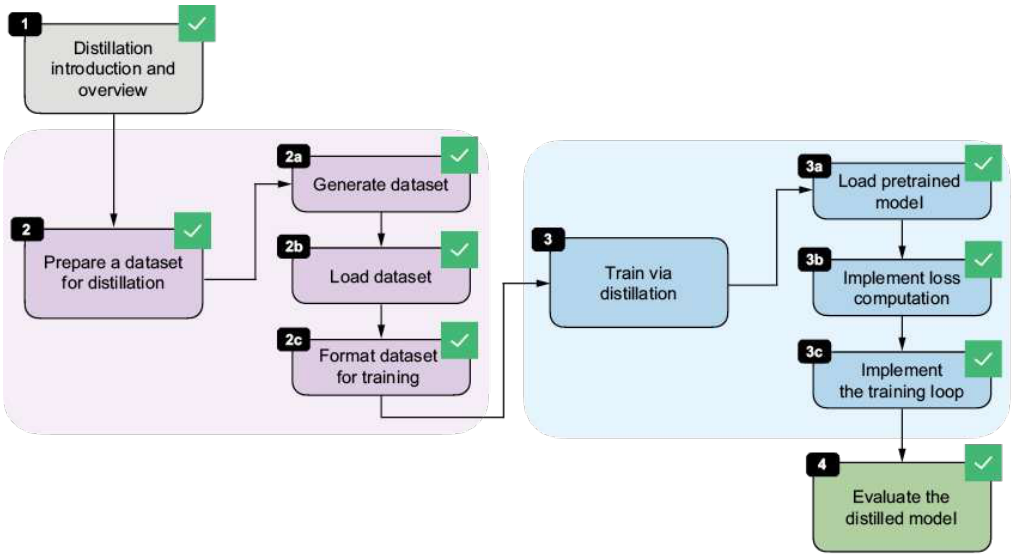


Figure 8.18 The evaluation of the distilled model completes the technical content of this chapter.

This workflow completes the core technical recipe for distilling a smaller reasoning model from a stronger teacher. In practice, each stage offers room for variation, such as changing how the teacher data is generated and modifying the training settings.

EXERCISE 8.2: DISTILLING WITHOUT <THINK> TOKENS

Repeat the distillation experiment without `--use_think_tokens` and compare the results against the version trained with reasoning traces wrapped in `<think>...</think>`. Inspect the validation loss and, if possible, evaluate the saved checkpoint on MATH-500. Then compare the results with those listed in table 8.1. How much do the explicit reasoning tags matter in this setup?

8.9 Future directions for reasoning models

Before closing the chapter, let's discuss where reasoning models are headed next.

For the foreseeable future, the broad pattern remains the same. For instance, the general strategy is to develop a stronger reasoning teacher, collect high-quality reasoning traces, and distill them into smaller student models.

One obvious direction is the continued refinement of the training recipe popularized by the DeepSeek-R1 paper, which includes both RLVR (chapters 6 and 7) for the flagship models and distillation for the small-

er models focused on computational efficiency.

A second direction is optimization at inference time. In practice, a reasoning model should not always produce answers of equal length. Some tasks benefit from short, direct responses, whereas others benefit from more detailed, multistep reasoning. This creates room for more automatic and flexible inference scaling at the application layer, where the surrounding system decides when to ask for a short answer, when to allocate more reasoning budget, and when to stop early. For example, OpenAI implemented such a system with the launch of GPT-5 in 2025, adding an "auto" mode to steer the reasoning effort and the reasoning trace generation length.

A third direction for improvement is reward generation for RLVR. Much of the current work still relies heavily on rewards based on final answers, especially in math and code. But *process rewards* that check intermediate reasoning steps, not just the final result, may provide a richer training signal and help models learn more reliable reasoning strategies. For example, the DeepSeek-Math-V2 paper (**https://arxiv.org/abs/2511.22570**) recently demonstrated that judging the whole answer during training can meaningfully improve reasoning performance.

A fourth direction is the growing role of reasoning models as the engine inside larger agent applications such as OpenAI Codex, Claude Code, and OpenClaw. In these settings, the model must not only solve a simple math or coding problem, but also plan, call tools, recover from failures, and coordinate longer workflows. This naturally pushes reward design beyond math and code correctness. We may want rewards for successful tool use, information retrieval, policy compliance, and more. In turn, this leads to *multi-reward training*, where several objectives are optimized together instead of relying on a single correctness score.

Distillation will likely remain important in this setting because it offers a practical way to transfer these richer behaviors from larger and more expensive teacher systems into smaller models that are easier to deploy, may run locally, and are more cost effective for users.

8.10 Conclusions

This completes the main technical material of the book. We'll now look at some brief pointers on what to try next, how to keep up with this fast-moving field, and where to find additional material.

8.10.1 What's next

A practical next step is to start combining the methods from this book instead of treating them as isolated techniques. For example, you could distill a smaller model from a strong teacher, continue training it with RLVR, and then apply inference-time scaling methods such as self-consistency or self-refinement at deployment time. Running these kinds of comparisons is often the fastest way to develop intuition for which method helps most in a given setting.

The appendices are also a good place to continue. They cover additional topics such as LLM architecture details, batched execution for higher throughput, and alternative evaluation approaches.

Lastly, the supplementary code repository (**https://github.com/rasbt/reasoning-from-scratch**) includes bonus material and standalone scripts that are better suited for longer runs and larger experiments than a notebook.

8.10.2 Staying up to date in a fast-moving field

I hope this book gave you a clearer picture of how modern reasoning models work in practice.

Reasoning-model research is moving quickly, and specific algorithms, datasets, and best practices will continue to change, but the core ideas in this book will remain relevant and useful. These include careful model evaluation, distinguishing between inference-time and training-time methods, and understanding how the training losses and reward signals are computed.

When new methods appear, it helps to map them back to these fundamentals. Many new techniques are best understood as variations or combinations of the building blocks we have implemented here, even when the surrounding training recipe becomes more complex.

In practice, I recommend several resources. First, you can skim recent machine learning and AI papers on arXiv (**https://arxiv.org/list/cs.LG/recent**) to spot new training and evaluation ideas early. But be aware that the volume of new papers is now so large that keeping up comprehensively is nearly impossible, so it is usually better to treat arXiv as a way to scan for themes and promising papers than to try to read everything.

Second, read technical summaries and reports for newly released models, since they often contain the most useful details about prompting conventions, benchmarks, and limitations. These are usually shared directly by the developer on the Hugging Face model hub, announcement blog articles, and social media.

Third, keep an eye on practitioner discussions on the social media platform X and in communities such as r/LocalLLaMA (**https://www.reddit.com/r/LocalLLaMA/**), where implementation details, replications, and failure cases often show up before they make it into polished papers.

AI-assistant or "deep research" tools can also be useful for monitoring these sources and summarizing new developments, but they work best as filters rather than substitutes for reading the original papers and model cards.

I also regularly write about AI and LLM topics on my blog at **https://magazine.sebastianraschka.com**.

Summary

- Distillation trains a smaller student LLM on outputs produced by a larger teacher LLM.
- Hard distillation is usually more practical than soft distillation because teacher logits are often unavailable and teacher text outputs are much cheaper to store and reuse.
- We used DeepSeek-R1 as the teacher model and Qwen3 0.6B as the student model.
- The distillation dataset was built from the 12,000 MATH training problems that do not overlap with MATH-500.
- Each training sample combines a rendered prompt with the teacher reasoning trace and final answer, optionally separated via `<think>...</think>` tags.
- For efficiency, we tokenize the dataset once, filter it by sequence length, and reuse the processed examples across multiple epochs.
- The training objective is answer-only cross-entropy, which is equivalent to the negative average log probability of the correct next tokens.
- A distillation training loop is a standard supervised learning loop that includes shuffling the training examples each epoch, computing the loss, backpropagating, updating the model weights, and tracking validation loss.
- The validation loss is the main signal to watch during training. Additionally, it is useful and recommended that you evaluate checkpoints periodically on benchmark datasets.

appendix A References and further reading

A.1 Chapter 1: Understanding reasoning models

A.1.1 References

This is the announcement article for OpenAI's o1 model, which is regarded as the first LLM-based reasoning model:

- OpenAI, "Introducing OpenAI o1-preview" (Sept. 12, 2024), **https://openai.com/index/introducing-openai-o1-preview/**

DeepSeek-R1 is the first open source reasoning model to be accompanied by a comprehensive technical report showing that reasoning emerges from reinforcement learning with verifiable rewards (a topic covered in more detail in chapter 5):

- DeepSeek-AI, Daya Guo, Dejian Yang, et al., "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning" (Jan. 22, 2025), **https://arxiv.org/abs/2501.12948**

This is the OpenAI CEO's comment on the reasoning ("chain-of-thought") capabilities of future models: "We will next ship GPT-4.5, the model we called Orion internally, as our last non-chain-of-thought model."

- Sam Altman, post on X (Feb. 12, 2025), **https://x.com/sama/status/1889755723078443244**

A research paper by AI researchers at Apple found that reasoning models are sophisticated (but very capable) pattern matchers:

- Parshin Shojaee, Iman Mirzadeh, Keivan Alizadeh, et al., "The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity" (Jun. 2025), **https://machinelearning.apple.com/research/illusion-of-thinking**

The following book is an in-depth guide to implementing and training large language models step by step:

- Sebastian Raschka, *Build a Large Language Model (From Scratch)*, (Manning, 2024), **http://mng.bz/orYv**

A.1.2 Further reading

The following article is an introduction to how DeepSeek-R1 works, providing insights into the foundations of reasoning in LLMs:

- Sebastian Raschka, "Understanding Reasoning LLMs" (Feb. 5, 2025), **https://magazine.sebastianraschka.com/p/understanding-reasoning-llms**

A.2 Chapter 2: Generating text with a pretrained LLM

A.2.1 References

You can find the official installation page for the `uv` Python package and project manager at the following URL:

- "Installing uv," **https://docs.astral.sh/uv/getting-started/installation/**

These are two user-friendly and popular cloud compute platforms with GPU support:

- Lightning AI, https://lightning.ai/
- Google Colab, https://colab.research.google.com/

The following articles provide Qwen3 resources with additional benchmark results and comparisons with other models:

- "Qwen3: Think Deeper, Act Faster" (Apr. 28, 2025), https://qwen.ai/blog?id=qwen3
- An Yang, Anfeng Li, Baosong Yang, et al., "Quen3 Technical report" (May 14, 2025), https://arxiv.org/abs/2505.09388

If you are curious about KV cache sizes for different sequence lengths, you can find a handy calculator app here:

- KV cache size calculator, https://lmcache.ai/kv_cache_calculator.html

A.2.2 A.2.2 Further reading

This is a PyTorch tutorial for those new to PyTorch or wanting a refresher:

- Sebastian Raschka, "PyTorch in One Hour: From Tensors to Training Neural Networks on Multiple GPUs tutorial" (July 1, 2025), https://sebastianraschka.com/teaching/pytorch-1h

The following are additional resources on tokenization:

- Sebastian Raschka, *Build a Large Language Model (from Scratch)*, chapter 2 (Manning, 2024), https://mng.bz/M96o
- Sebastian Raschka, "Implementing A Byte Pair Encoding (BPE) Tokenizer From Scratch" (Jan. 17, 2025), https://sebastianraschka.com/blog/2025/bpe-from-scratch.html

If you are interested in more in-depth PyTorch coverage, I recommend the following two books:

- Howard Huang, Eli Stevens, Luca Antiga, and Thomas Viehmann, *Deep Learning with PyTorch* (Manning 2026), https://www.manning.com/books/deep-learning-with-pytorch-second-edition
- Sebastian Raschka, Yuxi (Hayden) Liu, and Vahid Mirjalili, *Machine Learning with PyTorch and Scikit-Learn* (Packt Publishing, 2022), https://www.amazon.com/Machine-Learning-PyTorch-Scikit-Learn-learning/dp/1801819319/

A.3 Chapter 3: Evaluating reasoning models

A.3.1 References

The MATH-500 dataset originated from the MATH dataset, introduced in the following paper, which contains 12,500 problems across algebra, geometry, probability, number theory, and more:

- Dan Hendrycks, Collin Burns, Saurav Kadavath, et al., "Measuring Mathematical Problem Solving With the MATH Dataset" (Mar. 5, 2021), https://arxiv.org/abs/2103.03874

The MATH-500 split (created from the original MATH dataset) was proposed in the following paper:

- Hunter Lightman, Vineet Kosaraju, Yura Burda, et al., "Let's Verify Step by Step" (31 May 2023), https://arxiv.org/abs/2305.20050

A.3.2 Further reading

If you are interested in SymPy, a Python library for math and symbolic computation (not required for this book), you can learn more in this official tutorial:

- SymPy, "Introductory Tutorial," https://docs.sympy.org/latest/tutorials/intro-tutorial/index.html

The following article presents an example of a system (here, a fine-tuned LLM) that also evaluates intermediate reasoning steps:

- Shijie Xia, Xuefeng Li, Yixin Liu, et al., "Evaluating Mathematical Reasoning Beyond Accuracy," Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, Volume 1: Long Papers (2023), 6858–6877, **https://arxiv.org/pdf/2404.05692**

The following is a large-scale dataset containing 800,000 step-level correctness labels for model-generated solutions to problems from the MATH dataset:

- Hunter Lightman, Vineet Kosaraju, Yura Burda, et al., "Let's Verify Step by Step" (May 31, 2023), **https://arxiv.org/abs/2305.20050**

An article describing the rising cost of LLM evaluation found that evaluating reasoning models such as o1 on seven popular benchmarks costs approximately $1,500:

- Kyle Wiggers, "The rise of AI 'reasoning' models is making benchmarking more expensive" (Apr. 10, 2025), **https://techcrunch.com/2025/04/10/the-rise-of-ai-reasoning-models-is-making-benchmarking-more-expensive/**

The following is a comprehensive 2025 survey on LLM benchmarks:

- Shiwen Ni, Guhong Chen, Shuaimin Li, et al., "A Survey on Large Language Model Benchmarks" (Aug. 21, 2025), **https://arxiv.org/abs/2508.15361**

Rather than relying only on deterministic and symbolic verifiers, a recent research project showed that small reasoning models can themselves be used successfully as verifiers for other reasoning models:

- Ding Chen, Qingchen Yu, Pengyuan Wang, et al., "xVerify: Efficient Answer Verifier for Reasoning Model Evaluations" (Apr. 14, 2025), **https://arxiv.org/abs/2504.10481**

A.4 Chapter 4: Improving reasoning with inference-time scaling

A.4.1 References

The following paper formally described chain-of-thought prompting. Note that the paper suggested "Let's think step by step" as a prompt modification. In my experiments, I've found that "Explain step by step" performs better when using the Qwen3 base model, which is why we use it in chapter 4:

- Takeshi Kojima, Shixiang Shane Gu, Machel Reid, et al., "Large Language Models are Zero-Shot Reasoners" (May 24, 2022), **https://arxiv.org/abs/2205.11916**

The following paper focuses on self-consistency sampling with additional comparison studies:

- Xuezhi Wang, Jason Wei, Dale Schuurmans, et al., "Self-Consistency Improves Chain-of-Thought Reasoning in Language Models" (Mar. 21, 2022), **https://arxiv.org/abs/2203.11171**

A.4.2 Further reading

The following article is an overview and discussion of additional inference scaling methods:

- Sebastian Raschka, "The State of LLM Reasoning Model Inference" (Mar. 8, 2025), **https://magazine.sebastianraschka.com/p/state-of-llm-reasoning-and-inference-scaling**

A.5 Chapter 5: Inference-time scaling via self-refinement

A.5.1 References

Google keeps the methods behind its proprietary Gemini 3 model secret, but based on a recent announcement, we can speculate that it uses inference-scaling techniques similar to self-consistency or best-of-*N*: "We're pushing the boundaries of intelligence even further with Gemini 3 Deep Think. This mode meaningfully improves reasoning capabilities by exploring many hypotheses simultaneously to solve problems."

- Public announcement by Google DeepMind, which develops Gemini, on X (Dec. 4, 2025), https://x.-

com/GoogleDeepMind/status/1996658401233842624?s=20

The DeepSeekMath-V2 paper showed that self-consistency scaling can noticeably improve answer accuracy, and by combining self-consistency with their version of self-refinement (called *Best@32* in figure 2 of the following paper), the model achieved gold-level performance in several math competitions:

- Zhihong Shao, Yuxiang Luo, Chengda Lu, et al., "DeepSeekMath-V2: Towards Self-Verifiable Mathematical Reasoning" (Nov. 27, 2025), **https://arxiv.org/abs/2511.22570v1**

Instead of using a majority vote in self-consistency, we can use a scoring function to rank the different answers and select the best one. This approach is known as best-of-*N*. Still, when applicable, majority voting often gives better results:

- Junlin Wang, Shang Zhu, Jon Saad-Falcon, et al., "Think Deep, Think Fast: Investigating Efficiency of Verifier-free Inference-time-scaling Methods" (Apr. 18, 2025), **https://arxiv.org/abs/2504.14047**

A.5.2 Further reading

The following short article explains the difference between probabilities and likelihood:

- Sebastian Raschka, "What is the difference between likelihood and probability?" **https://sebastianraschka.com/faq/docs/probability-vs-likelihood.html**

A.6 Chapter 6: Training reasoning models with reinforcement learning

A.6.1 References

The InstructGPT paper demonstrated the effectiveness of RLHF and was instrumental in popularizing RLHF as a standard alignment and fine-tuning approach for LLMs:

- Long Ouyang, Jeff Wu, Xu Jiang, et al., "Training language models to follow instructions with human feedback" (Mar. 4, 2022), **https://arxiv.org/abs/2203.02155**

The following DeepSeekMath paper introduced the GRPO algorithm:

- Zhihong Shao, Peiyi Wang, Qihao Zhu, et al., "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models" (Feb. 5, 2024), **https://arxiv.org/abs/2402.03300**

The DeepSeek-R1 paper showed that strong reasoning behavior can emerge in LLMs through reinforcement learning alone (via RLVR with GRPO). This was most clearly shown in the R1-Zero variant, but combining this approach with a multistage training pipeline yields an even better reasoning model:

- DeepSeek-AI, Daya Guo, Dejian Yang, et al., "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning" (Jan. 22, 2025), **https://arxiv.org/abs/2501.12948**

A.6.2 Further reading

The following article provides a comprehensive walk-through of the DeepSeek-R1 training pipeline involving RLVR:

- Sebastian Raschka, "Understanding Reasoning LLMs: Methods and Strategies for Building and Refining Reasoning Models" (Feb. 5, 2025), **https://magazine.sebastianraschka.com/p/understanding-reasoning-llms**

This article compares GRPO and PPO for reinforcement learning in the context of LLMs:

- Sebastian Raschka, "The State of Reinforcement Learning for LLM Reasoning: Understanding GRPO and New Insights from Reasoning Model Papers" (Apr. 19, 2025), **https://magazine.sebastianraschka.com/p/the-state-of-llm-reasoning-model-training**

A.7 Chapter 7: Improving GRPO for reinforcement learning

A.7.1 References

This is the original PPO paper that introduced clipped policy ratios, which we also use here to stabilize GRPO:

- John Schulman, Filip Wolski, Prafulla Dhariwal, et al., "Proximal Policy Optimization Algorithms" (Jul. 20, 2017), **https://arxiv.org/abs/1707.06347**

The following papers recommend improvements to the GRPO algorithm:

- Qiying Yu, Zheng Zhang, Ruofei Zhu, et al., "DAPO: An Open-Source LLM Reinforcement Learning System at Scale" (Mar. 18, 2025), **https://arxiv.org/abs/2503.14476**
- Understanding R1-Zero-Like Training: A Critical Perspective" (Mar. 26, 2025, this paper introduces Dr. GRPO), **https://arxiv.org/abs/2503.20783**
- Feng Yao, Liyuan Liu, Dinghuai Zhang, et al., "Your Efficient RL Framework Secretly Brings You Off-Policy RL Training" (Aug. 5, 2025, this paper introduces VERL), **https://fengyao.notion.site/off-policy-rl**
- DeepSeek-AI, Aixin Liu, Aoxue Mei, et al., "DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models" (Dec. 2, 2025), **https://arxiv.org/abs/2512.02556**
- Shih-Yang Liu, Xin Dong, Ximing Lu, et al., "GDPO: Group reward-Decoupled Normalization Policy Optimization for Multi-reward RL Optimization" (Jan. 8, 2026), **https://arxiv.org/abs/2601.05242**
- Chujie Zheng, Shixuan Liu, Mingze Li, et al., "Group Sequence Policy Optimization" (Jul. 24, 2025, this paper introduces GSPO), **https://arxiv.org/abs/2507.18071**
- Aili Chen, Aonian Li, Bangwei Gong, et al., "MiniMax-M1: Scaling Test-Time Compute Efficiently with Lightning Attention" (Jun. 16, 2025, this paper introduces CISPO), **https://arxiv.org/abs/2506.13585**

A.7.2 Further reading

The following article provides a comparison between PPO (the original algorithm used for RLHF) and GRPO:

- Sebastian Raschka, "The State of Reinforcement Learning for LLM Reasoning" (Apr. 19, 2025), **https://magazine.sebastianraschka.com/p/the-state-of-llm-reasoning-model-training**

The following article is a good technical deep dive discussing different GRPO improvements:

- Cameron R. Wolfe, "GRPO++: Tricks for Making RL Actually Work" (Jan. 5, 2026), **https://cameronr-wolfe.substack.com/p/grpo-tricks**

A.8 Chapter 8: Distilling reasoning models for efficient reasoning

A.8.1 References

This is the original knowledge-distillation paper that popularized the combination of hard and soft distillation objectives:

- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean, "Distilling the Knowledge in a Neural Network" (Mar. 9, 2015), **https://arxiv.org/abs/1503.02531**

The DeepSeek-R1 paper described the reasoning-distillation recipe that motivated this chapter, in which a large teacher model generates reasoning traces that are then used to train smaller student models:

- DeepSeek-AI, Daya Guo, Dejian Yang, et al., "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning" (Jan. 22, 2025), **https://arxiv.org/abs/2501.12948**

The following paper on distilling large language models reports strong results for carefully designed soft-distillation objectives:

- Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang, "MiniLLM: Knowledge Distillation of Large Language Models" (Jun. 14, 2023), **https://arxiv.org/abs/2306.08543**

A.8.2 Further reading

The following chapter provides more details on supervised fine-tuning (the technique used in hard distillation) and on masking when working with batches of training examples:

- Sebastian Raschka, *Build A Large Language Model (From Scratch)*, chapter 7 (Manning, 2024), **https://www.manning.com/books/build-a-large-language-model-from-scratch**

The following article provides a practical walk-through of reasoning-model-training pipelines, including distillation and RLVR:

- Sebastian Raschka, "Understanding Reasoning LLMs: Methods and Strategies for Building and Refining Reasoning Models" (Feb. 5, 2025), **https://magazine.sebastianraschka.com/p/understanding-reasoning-llms**

A.9 Appendix F: Common approaches to LLM evaluation

A.9.1 References

This is the original paper that introduced the popular multiple-choice MMLU dataset:

- Dan Hendrycks, Collin Burns, Steven Basart, et al., "Measuring Massive Multitask Language Understanding" (Sept. 7, 2020), **https://arxiv.org/abs/2009.03300**

The following Wikipedia article provides a detailed description of the Elo rating system:

- Elo rating system, **https://en.wikipedia.org/wiki/Elo_rating_system**

The following paper describes the original methodology behind the popular LLM leaderboard:

- Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, et al., "Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference" (Mar. 7, 2024), **https://arxiv.org/abs/2403.04132**

A.9.2 Further reading

The following paper discusses the problems with leaderboards such as LM Arena:

- Shivalika Singh, Yiyang Nan, Alex Wang, et al., "The Leaderboard Illusion" (Apr. 29, 2025), **http://arxiv.org/abs/2504.20879**

The following article describes OpenAI's most recent open source model, gpt-oss, in more detail:

- Sebastian Raschka, "From GPT-2 to gpt-oss: Analyzing the Architectural Advances" (Aug. 9, 2025), **https://magazine.sebastianraschka.com/p/from-gpt-2-to-gpt-oss-analyzing-the**

The following survey compares different LLM judge approaches:

- Jiawei Gu, Xuhui Jiang, Zhichao Shi, et al., "A Survey on LLM-as-a-Judge" (Nov. 23, 2024), **https://arxiv.org/abs/2411.15594**

The method described in the following paper is an example of a small LLM fine-tuned to act as a judge:

- Mahesh Deshwal and Apoorva Chawla, "PHUDGE: Phi-3 as Scalable Judge" (May 12, 2024), **https://arxiv.org/abs/2405.08029**

appendix B Exercise solutions

The complete code examples for the exercise solutions can be found in the supplementary GitHub repository at **https://github.com/rasbt/reasoning-from-scratch**.

B.1 Chapter 2

EXERCISE 2.1: ENCODING UNKNOWN WORDS

We can use a prompt similar to "Hello, Ardwarklethyrx. Haus und Garten.", which contains a made-up word (`" Ardwarklethyrx "`) and three words in a non-English language (German):

```
prompt = "Hello, Ardwarklethyrx. Haus und Garten."
input_token_ids_list = tokenizer.encode(prompt)
for i in input_token_ids_list:
    print(f"{[i]} --> {tokenizer.decode([i])}")
```

The output is:

```
[9707] --> Hello
[11] --> ,
[1644] -->  Ar
[29406] --> dw
[838] --> ark
[273] --> le
[339] --> th
[10920] --> yr
[87] --> x
[13] --> .
[47375] -->  Haus
[2030] -->  und
[93912] -->  Garten
[13] --> .
```

As you can see, unknown words are broken into smaller subwords or even single tokens; this allows the tokenizer and LLM to handle any input.

German words are not broken down into characters or even subwords here, suggesting that the tokenizer has seen German texts during training. This also suggests that the LLM was likely trained on German texts too and should be able to handle at least some non-English languages well.

EXERCISE 2.2: RERUN CODE ON NON-CPU DEVICES

We can simply delete the line `device = torch.device( " cpu " )` in listing 2.1 and then rerun the rest of the code in chapter 2 as is. Reference numbers for the hardware I used to run the code are provided in table 2.1 at the end of chapter 2.

B.2 Chapter 3

EXERCISE 3.1: ADDING MORE TEST CASES

There are endlessly many test cases we can add. This is a selection of some interesting ones:

```
from reasoning_from_scratch.ch03 import (
    run_demos_table
)

more_tests = [
    ("check_17", "[1, 2]", "(1, 2)", True),  #1
    ("check_18", "1e-3", "0.001", True),  #2
    ("check_19", "(-3)^2", "9", True),  #3
    ("check_20", "−1", "-1", True),  #4

]
run_demos_table(more_tests)
```

#1 Different bracket types
#2 Scientific notation
#3 Algebraic simplification with caret exponent
#4 Unicode minus (U+2212) vs. ASCII hyphen-minus

The output follows:

```
Test     | Expect | Got   | Status
check_17 | True   | True  | PASS
check_18 | True   | True  | PASS
check_19 | True   | True  | PASS
check_20 | True   | False | FAIL
```

As you can see, the tests pass in all cases except for `check_20`, which swaps the regular minus sign with a Unicode version of a minus sign that looks indistinguishable to the human eye (depending on the font or editor you are using). We could fix this test case by adding one of the following lines anywhere in the `normalize_text` function:

```
text = text.replace("−", "-")
```

or

```
text = text.replace("\u2212", "-")
```

Here is another interesting test:

```
extra_tests_1 = [
    ("check_21", "Text around answer 3.", "3", True)
]
```

You can run it with the following code:

```
run_demos_table(extra_tests_1)
```

But it fails the test:

```
Test     | Expect | Got   | Status
check_21 | True   | False | FAIL

Passed 0/1
```

While it may seem that the code cannot handle cases that contain text, this is actually a poorly designed test. In practice, the `run_demos_table` function is intended specifically to test the `grade_answer` function; nothing more, nothing less.

The `grade_answer` function would never receive the entire answer in this text form because the answer would have been extracted from the text before being passed to it. If we want to test text answers, we need to call the test as follows:

```
from reasoning_from_scratch.ch03 import (
    extract_final_candidate
)
extra_tests_2 = [
    ("check_21",
     extract_final_candidate("Text around answer 3."),
     "3", True)
]
run_demos_table(extra_tests_2)
```

As you can see from the output, it now passes the test:

```
Test     | Expect | Got  | Status
check_21 | True   | True | PASS

Passed 1/1
```

EXERCISE 3.2: CALCULATING THE AVERAGE RESPONSE LENGTH

```
# ...
# below `num_correct = 0`
total_len = 0

# ...
# inside for i, row in enumerate(math_data, start=1):
# anywhere below `gen_text = ...`
total_len += len(tokenizer.encode(gen_text))

# ...
# anywhere at the bottom before the return statement
avg_len = total_len / num_examples
print(f"Average length: {avg_len:.2f} tokens")
```

Alternatively, the second option is to calculate the response lengths from the `.jsonl` files created when we ran the `evaluate_math500_stream` function in the main chapter. This way, we avoid having to rerun the evaluation.

First, we load the `.jsonl` file as follows:

```
import json
from pathlib import Path

WHICH_MODEL = "base"
dev_name = "mps"

local_path = Path(f"math500-{dev_name}.jsonl")  #1
if not local_path.exists():
    raise FileNotFoundError(
        f"{local_path} not found. Run ch03_main.ipynb to create it."
    )

results = []
with open(local_path, "r") as f:
    for line in f:
        if line.strip():
            results.append(json.loads(line))

print("Number of entries:", len(results))
```

#1 You may need to adjust this path.

Let's print the dictionary keys to get a better idea of how the `results` dataset is structured:

```
print(results[0].keys())
```

This prints the following:

```
dict_keys(['index', 'problem', 'gtruth_answer', 'generated_text',
'extracted', 'correct'])
```

Each entry has multiple keys, but we are interested only in the `generated_text` key, which contains the model's full answer.

Next, we need to load the tokenizer so we can tokenize the answer text before calculating the number of tokens. This is similar to the code we used in listing 3.1:

```
from reasoning_from_scratch.qwen3 import (
    download_qwen3_small,
    Qwen3Tokenizer
)

if WHICH_MODEL == "base":

    download_qwen3_small(
        kind="base", tokenizer_only=True, out_dir="qwen3"
    )
    tokenizer_path = Path("qwen3") / "tokenizer-base.json"
    tokenizer = Qwen3Tokenizer(tokenizer_file_path=tokenizer_path)

elif WHICH_MODEL == "reasoning":

    download_qwen3_small(
        kind="reasoning", tokenizer_only=True, out_dir="qwen3"
    )
    tokenizer_path = Path("qwen3") / "tokenizer-reasoning.json"
    tokenizer = Qwen3Tokenizer(
        tokenizer_file_path=tokenizer_path,
        apply_chat_template=True,
        add_generation_prompt=True,
        add_thinking=True,
    )
```

Now we can calculate the average length as follows, which is similar to how we could have modified the `evaluate_math500_stream` function:

```
total_len = 0

for item in results:
    num_tokens = len(tokenizer.encode(item["generated_text"]))
    total_len += num_tokens

avg_len = total_len / len(results)
print(f"Average length: {avg_len:.2f} tokens")
```

The resulting average length is as follows:

```
Average length: 98.00 tokens
```

Table B.1 lists the average lengths for the different models and subsets.

Table B.1 Average number of tokens on MATH-500

Model	Device	Average length	MATH-500 size
Base	CPU	97.30	10
Base	CUDA	96.74	500
Reasoning	CPU	891.80	10
Reasoning	CUDA	1361.21	500

As you can see from the results in table B.1, and as expected, the reasoning model generates much longer responses (in this case, approximately 10 times longer).

EXERCISE 3.3: EXTENDING OR CHANGING THE EVALUATION DATASET

To evaluate the model on a larger dataset, you can simply change `math_data[:10]` to a different slice or larger number (up to 500) in the following function call:

```
num_correct, num_examples, acc = evaluate_math500_stream(
    model, tokenizer, device,
    math_data=math_data[:10],
    max_new_tokens=2048,
    verbose=False
)
```

Table B.2 shows the accuracy values for different dataset sizes. (Since the MATH-500 test set is already shuffled, no additional shuffling was applied.) As you can see, the first 10 examples are not very representative of MATH-500 performance on the full set of 500 examples.

Table B.2 Accuracies for different MATH-500 dataset sizes

Model	Device	Accuracy	MATH-500 size
Base	CUDA	30.0%	10
Base	CUDA	34.0%	50
Base	CUDA	27.0%	100
Base	CUDA	15.3%	500
Reasoning	CUDA	90.0%	10
Reasoning	CUDA	58.0%	50
Reasoning	CUDA	56.0%	100
Reasoning	CUDA	48.2%	500

We can also create an entirely new dataset in a style similar to MATH-500. For example, this book's repository includes a dataset in MATH-500 style; we can use it by changing the filename from `math-500_test.json` to `math_new50_exercise.json` in the chapter's code (this dataset is included in the book's GitHub repository at **https://github.com/rasbt/reasoning-from-scratch/tree/main/ch03/01_main-chapter-code**).

The performance of the models is as follows:

- Base: 36.0% (18/50)
- Reasoning: 80.0% (40/50)

On the new 50-example dataset, the base model reaches 36.0% accuracy, which is close to its 34.0% accuracy on the 50-example MATH-500 subset shown in table B.2. The reasoning model reaches 80.0% accuracy on the new dataset, which is higher than its 58.0% accuracy on the same MATH-500 subset. This suggests that the reasoning model's strong performance is not limited to the original MATH-500 examples. While possible overlap with Qwen3's training data cannot be ruled out, these results do not show signs of extensive overfitting to MATH-500.

EXERCISE 3.4: EXPERIMENTING WITH DIFFERENT PROMPT TEMPLATES

We could use an alternative prompt, similar to the one suggested in the chapter, that uses the word "problem" instead of "question":

```
def render_prompt(prompt):
    template = (
        "You are a helpful math assistant.\n"
        "Solve the problem and write the final "
        "result on a new line as:\n"
        "\\boxed{ANSWER}\n\n"
        f"Problem:\n{prompt}\n\nAnswer:"
    )
    return template
```

On the 500 examples, using this prompt improves the base model's performance from 15.3% to 31.2%. It also improves the reasoning model's performance from 48.2% to 50.0%.

From these observations, we may conclude that the base model is much more sensitive to the prompt format (likely due to memorizing some prompt-formatted MATH-500 examples from the training set) than the reasoning model. The latter seems largely unaffected.

B.3 Chapter 4

EXERCISE 4.1: USING CHAIN-OF-THOUGHT PROMPTING ON MATH-500

The modification only requires adding a prompt suffix such as `" \n\nExplain step by step. "` after applying the prompt template. Only a very small portion of code in the MATH-500 evaluation function from chapter 3 needs to be updated, as follows:

```
def evaluate_math500_stream(...):
    # ...
        for i, row in enumerate(math_data, start=1):
            prompt = render_prompt(row["problem"])
            prompt += "\n\nExplain step by step."  # NEW
            gen_text = generate_text_stream_concat(
                model, tokenizer, prompt, device,
                max_new_tokens=max_new_tokens,
                verbose=verbose,
            )

     # ...
```

The improvements are shown in row 3 of table 4.1.

EXERCISE 4.2: USING TEMPERATURE SCALING AND TOP-P FILTERING ON MATH-500

Here, we replace the `generate_text_stream_concat` function with `generate_text_stream_concat_flex` and pass in `generate_text_top_p_stream_cache` as its generation function. The updated MATH-500 evaluation function from chapter 3 follows, and the changes are marked with comments labeled # NEW:

```
def evaluate_math500_stream(
    model,
    tokenizer,
    device,
    math_data,
    out_path=None,
    max_new_tokens=512,
    verbose=False,
    temperature=1.0,  # NEW
    top_p=1.0,        # NEW
):

    # ...

    with open(out_path, "w", encoding="utf-8") as f:
        for i, row in enumerate(math_data, start=1):
            prompt = render_prompt(row["problem"])
            gen_text = generate_text_stream_concat_flex( # NEW
                model, tokenizer, prompt, device,
                max_new_tokens=max_new_tokens,
                verbose=verbose,
                generate_func=generate_text_top_p_stream_cache,  # NEW
                temperature=temperature,                         # NEW
                top_p=top_p                                      # NEW
            )
    # ...
```

The difference between this modified function and the baseline from chapter 3 can be seen in rows 1 and 4 of table 4.1.

EXERCISE 4.3: USING SELF-CONSISTENCY SAMPLING ON MATH-500

Starting from the `evaluate_math500_stream` function in listing 3.15, the first modification is to replace the line `gen_text = generate_text_stream_concat(...)` with a call to `results = self_consis-`

`tency_vote(...)` from section 4.6.

The second modification adds a simple tiebreaking rule that selects the first occurrence of the most frequent answer. For instance, if the sampled results are 1, 3, 5, 3, 5, the function would return 3 because it is the first member of the most frequent group.

Since the most frequent answers are stored in `results[ " majority_winners " ]`, one straightforward way to break ties is to take the first element of this list, that is, `results[ " majority_winners " ][0]`.

Those changes are illustrated in the following code excerpts:

```
def evaluate_math500_stream(
    model,
    tokenizer,
    device,
    math_data,
    out_path=None,
    max_new_tokens=2048,
    verbose=False,
    prompt_suffix="",    # NEW
    temperature=1.0,     # NEW
    top_p=1.0,           # NEW
    seed=None,           # NEW
    num_samples=10,      # NEW
):

    if out_path is None:
        dev_name = str(device).replace(":", "-")
        out_path = Path(f"math500-{dev_name}.jsonl")

    num_examples = len(math_data)
    num_correct = 0
    start_time = time.time()

    with open(out_path, "w", encoding="utf-8") as f:
        for i, row in enumerate(math_data, start=1):
            prompt = render_prompt(row["problem"])

            ################################################################
            # NEW
            prompt += prompt_suffix
            results = self_consistency_vote(
                model=model,
                tokenizer=tokenizer,
                prompt=prompt,
                device=device,
                num_samples=num_samples,
                temperature=temperature,
                top_p=top_p,
                max_new_tokens=max_new_tokens,
                show_progress=False,
                show_long_answer=False,
                seed=seed,
            )

            # resolve ties
            if results["final_answer"] is None:
                extracted = results["majority_winners"][0]
            else:
                extracted = results["final_answer"]

            # extracted = extract_final_candidate(
            #     gen_text
            # )

            # Optionally, get long answer
```

```
            if extracted is not None:
                for idx, s in enumerate(results["short_answers"]):
                    if s == extracted:
                        long_answer = results["full_answers"][idx]
                        break
            gen_text = long_answer
            ############################################################

            is_correct = grade_answer(
                extracted, row["answer"]
            )
            num_correct += int(is_correct)
    # ...
```

The performance improvements from self-consistency sampling are summarized and discussed in table 4.1 (rows 5–7 and 9–12).

EXERCISE 4.4: EARLY STOPPING IN SELF-CONSISTENCY SAMPLING

The early stopping check can be implemented by adding a few lines of code to see whether the given answer has already been counted multiple times, or, more specifically, whether the given answer count is greater than `num_samples / 2`:

```
if early_stop and counts[short] > num_samples / 2:
    majority_winners = [short]
    final_answer = short
    break
```

The following excerpt from the modified `self_consistency_vote` function illustrates where to insert this code:

```
def self_consistency_vote(
    # ...
    early_stop=True,   # NEW
):
    # ...

        if show_progress:
            print(f"[Sample {i+1}/{num_samples}] → {short!r}")

        ######################################################
        # NEW
        # Early stop if one answer already meets >= 50% majority
        if early_stop and counts[short] > num_samples / 2:
            majority_winners = [short]
            final_answer = short
            break
        ######################################################

    if final_answer is None:
        mc = counts.most_common()
        if mc:
            top_freq = mc[0][1]
            majority_winners = [s for s, f in mc if f == top_freq]
            final_answer = mc[0][0] if len(majority_winners) == 1 else None

    return {
        "full_answers": full_answers,
        "short_answers": short_answers,
        "counts": dict(counts),
        "groups": groups,
        "majority_winners": majority_winners,
        "final_answer": final_answer,
    }
```

B.4 Chapter 5

EXERCISE 5.1: USING THE HEURISTIC SCORER AS A TIEBREAKER IN SELF-CONSISTENCY

There are many ways to implement this. Perhaps the easiest approach is to handle it outside the self-consistency function and work directly with the returned dictionary, similar to what we did in exercise 4.4, where we implemented the tiebreaking logic directly inside the `evaluate_math500_stream` function. The relevant lines are shown here:

```
# ...
from reasoning_from_scratch.ch05 import heuristic_score

def evaluate_math500_stream(
    #...
    # ...
            results = self_consistency_vote(...)
            # Majority vote winner available
            if results["final_answer"] is not None:
                extracted = results["final_answer"]

            ### NEW: Break tie with heuristic_score
            else:
                best = None
                best_score = float("-inf")
                for cand in results["majority_winners"]:
                    scores = [
                        heuristic_score(results["full_answers"][idx],
                                        prompt=prompt)
                        for idx in results["groups"][cand]
                    ]
                    score = max(scores)
                    if score > best_score:
                        best_score = score
                        best = cand
                extracted = best
            # ...
    # ...
    return num_correct, num_examples, acc
```

The results are shown in table B.3. The accuracy values and runtimes were computed on all 500 samples in the MATH-500 test set using a `" cuda "` GPU (DGX Spark).

Table B.3 MATH-500 self-consistency score with different tiebreaking

	Method	Model	Accuracy	Time
1	Baseline with chain-of-thought prompting	Base	33.4%	129.2 min
2	Self-consistency (*n*=3)	Base	43.2%	328.2 min
3	Self-consistency (*n*=3) + heuristic	Base	43.4%	326.5 min
4	Self-consistency (*n*=3) + avg. logprob	Base	44.8%	327.7 min

Row 1 in table B.3 is the baseline from chapter 4 without self-consistency. Row 2 doesn't use a scorer for tiebreaking, so if there is a tie among the answers, it chooses the one that appears first. Using a heuristic scorer (row 3) as a tiebreaker results in a slight improvement. The best, though still minimal, improvement is achieved with the logprob scorer as a tiebreaker (row 4).

EXERCISE 5.2: USING THE HEURISTIC SCORER IN A BEST-OF-N SETUP

Best-of-*N* is similar to self-consistency in that we generate multiple answers. But instead of selecting the

final answer by majority vote, we score all generated answers using a scoring function, such as `heuristic_score`, and return the highest-scoring one. There are several ways to implement this behavior, but the simplest approach is to use the existing self-consistency function from chapter 4 as a template and swap in `heuristic_score`, as follows:

```
# ...
from reasoning_from_scratch.ch05 import (
    heuristic_score
)

def self_consistency_vote( #...):
    full_answers, short_answers = [], []
    counts = Counter()
    groups = {}
    majority_winners, final_answer = [], None
    best_score, best_idx = float("-inf"), None

    for i in range(num_samples):
        if seed is not None:
            torch.manual_seed(seed + i + 1)

        answer = generate_text_stream_concat_flex(
            model=model,
            tokenizer=tokenizer,
            prompt=prompt,
            device=device,
            max_new_tokens=max_new_tokens,
            verbose=show_long_answer,
            generate_func=generate_text_top_p_stream_cache,
            temperature=temperature,
            top_p=top_p,
        )

        short = extract_final_candidate(answer, fallback="number_then_full")
        full_answers.append(answer)
        short_answers.append(short)
        counts[short] += 1

        if short in groups:
            groups[short].append(i)
        else:
            groups[short] = [i]

        score = heuristic_score(answer, prompt=prompt)

        if score > best_score:
            best_score, best_idx = score, i
        # ...
```

Table B.4 compares the baseline with two best-of-*N* variants, one using the heuristic scorer and one using average log probability to select among three sampled answers.

Table B.4 MATH-500 best-of-*N* scores with heuristic and average logprob scores

	Method	Model	Accuracy	Time
1	Baseline with chain-of-thought prompting	Base	33.4%	129.2 min
2	Best-of-*N* (*n*=3) + heuristic	Base	40.6%	327.7 min
3	Best-of-*N* (*n*=3) + avg. logprob	Base	43.2%	330.2 min

The accuracy values and runtimes shown in the table were computed on all 500 samples in the MATH-500 test set using a `" cuda "` GPU (DGX Spark).

EXERCISE 5.3: USING THE LOGPROB SCORER AS A TIEBREAKER IN SELF-CONSISTENCY

The solution is similar to exercise 5.1, except that we swap `heuristic_score` with `avg_logprob_answer`, as follows:

```
# ...
# from reasoning_from_scratch.ch05 import heuristic_score
from reasoning_from_scratch.ch05 import avg_logprob_answer

def evaluate_math500_stream(# ...)
    # ...

    # score = heuristic_score(
    #    candidate_full, prompt=prompt
    # )
    score = avg_logprob_answer(
        model=model,
        tokenizer=tokenizer,
        prompt=prompt,
        answer=candidate_full,
        device=device,
    )
   # ...
```

The results were included in table B.3 (exercise 5.1), in row 4.

EXERCISE 5.4: USING THE LOGPROB SCORER IN A BEST-OF-N SETUP

To implement best-of-*N* with a logprob scorer, we can use the code from exercise 5.2 and swap the `heuristic_score` with `avg_logprob_answer`, as follows:

```
from reasoning_from_scratch.ch05 import (
    avg_logprob_answer
)
# ...
            score = avg_logprob_answer(
                model=model,
                tokenizer=tokenizer,
                prompt=prompt,
                answer=answer,
                device=device
            )
        if score > best_score:
            best_score, best_idx = score, i
# ...
```

The resulting MATH-500 score is shown in table B.4 (exercise 5.2).

EXERCISE 5.5: USING THE HEURISTIC SCORE FOR SELF-REFINEMENT

Using the `heuristic_score` is even simpler than using the logprob score; all we need to do is change the following code:

```
from functools import partial

avg_logprob_score = partial(
    avg_logprob_answer,
    model=model,
    tokenizer=tokenizer,
    device=device
)
torch.manual_seed(0)

results_logprob = self_refinement_loop(
    model=model,
    tokenizer=tokenizer,
    raw_prompt=raw_prompt,
    device=device,
    iterations=2,
    max_response_tokens=2048,
    max_critique_tokens=256,
    score_fn=avg_logprob_score,
    verbose=True,
    temperature=0.7,
    top_p=0.9,
)
```

The updated code is

```
torch.manual_seed(0)
results_logprob = self_refinement_loop(
    model=model,
    tokenizer=tokenizer,
    raw_prompt=raw_prompt,
    device=device,
    iterations=2,
    max_response_tokens=2048,
    max_critique_tokens=256,
    score_fn=heuristic_score,  # NEW
    verbose=True,
    temperature=0.7,
    top_p=0.9,
)
```

The improvements over the baseline in chapter 3 and the self-consistency results from chapter 4 are shown in table 5.1 (rows 4, 5, and 10).

B.5 Chapter 6

EXERCISE 6.1: ADDING FORMAT-AWARE REWARD SHAPING

We can assign a partial reward (score 0.5) if no `" \boxed{} "` answer is found, using the `fallback= " number_then_full "` fallback we coded in chapter 3, as follows:

```
from reasoning_from_scratch.ch03 import (
    extract_final_candidate, grade_answer
)

def reward_rlvr(answer_text, ground_truth):

    # 1) Try to extract a boxed answer
    boxed = extract_final_candidate(
        answer_text, fallback=None
    )
    if boxed:
        correct = grade_answer(boxed, ground_truth)
        return 1.0 if correct else 0.0

    # 2) If no boxed answer is found, look for number
    unboxed = extract_final_candidate(
        answer_text, fallback="number_then_full"
    )
    if unboxed:
        correct = grade_answer(unboxed, ground_truth)
        return 0.5 if correct else 0.0

    return 0.0
```

When plugged into the chapter 6 code and trained under the same settings, the partial-reward variant achieves lower accuracy (37.8%) than the standard GRPO setup (47.4%), despite using a similar number of tokens on average, as shown in table B.5.

Table B.5 MATH-500 accuracies for strict and partial rewards

	Method	Step	Max tokens	Num rollouts	Accuracy	Average tokens
1	GRPO (chapter 6)	50	512	8	47.4%	586.11
2	GRPO partial rewards (exercise 6.1)	50	512	8	37.8%	550.33

EXERCISE 6.2: ZERO-ADVANTAGE CASES

If the rewards are all equal (for instance, they are all 0 or all 1), the advantages will all be 0 because subtracting the mean removes the shared reward value and leaves only zeros, as you can see here:

```
import torch

rollout_rewards = [0., 0., 0., 0.]
rewards = torch.tensor(rollout_rewards)
advantages = (rewards - rewards.mean()) / (rewards.std() + 1e-4)
print(advantages)
```

This returns `tensor([0., 0., 0., 0.])`.

Similarly, if we change the rollout rewards to `rollout_rewards = [0., 0., 0., 0.]`, we get the same all-zero tensor, `tensor([0., 0., 0., 0.])`.

In short, if all rewards in a group are identical, for example, all rewards are 0 or all rewards are 1, then $r_i - \mu_i = 0$ for all i rollouts. As a result, the policy gradient is zero, and the model parameters are not updated for that prompt.

This behavior is intentional. If all rollouts are equally bad or equally good, there is no relative signal to tell the model which behavior to reinforce or suppress. Intuitively, if the model answers all the questions

correctly, there is no need to update it. Conversely, if the model answers all the questions incorrectly, we don't want to update the model to reinforce that behavior.

B.6 Chapter 7

EXERCISE 7.1: TESTING THE `<THINK>` FORMAT REWARD

The following code checks that the format reward is zero if the `<think>` tokens are used incorrectly:

```
from pathlib import Path
import torch
from reasoning_from_scratch.qwen3 import Qwen3Tokenizer
from reasoning_from_scratch.qwen3 import download_qwen3_small
from reasoning_from_scratch.ch07 import reward_format

download_qwen3_small(
    kind="reasoning", tokenizer_only=True, out_dir="qwen3"
)
tokenizer_path = Path("qwen3") / "tokenizer-reasoning.json"
tokenizer = Qwen3Tokenizer(tokenizer_file_path=tokenizer_path)

prompt = "Calculate ..."

def check_case(name, rollout):
    token_ids = tokenizer.encode(prompt + rollout)
    prompt_len = len(tokenizer.encode(prompt))
    reward = reward_format(
        token_ids=torch.tensor(token_ids),
        prompt_len=prompt_len,
    )
    print(f"{name}: {reward}")

# 1) Correct case
check_case(
    "Correct order",
    "Let's ... <think> ... </think> ..."
)
# 2) Typo in tag
check_case(
    "Typo in <think>",
    "Let's ... <thnik> ... </think> ..."
)
# 3) Reversed order
check_case(
    "Reversed order",
    "Let's ... </think> ... <think> ..."
)

# 4) Missing one tag
check_case(
    "Missing </think>",
    "Let's ... <think> ..."
)
```

The output is as follows, indicating that the function requires correct `<think>` ... `</think>` tag usage to award a reward of 1.0:

```
Correct order: 1.0
Typo in <think>: 0.0
Reversed order: 0.0
Missing </think>: 0.0
```

EXERCISE 7.2: MAKING THE FORMAT REWARD CONDITIONAL

The implementation of the conditional reward is very simple; in the chapter, we discussed implementing

the overall reward as follows:

```
reward = rlvr_reward + format_reward_weight * format_reward
```

This is one way to disable the reward when the correctness reward (`rlvr_reward`) is 0.0:

```
if conditional_reward:
    format_reward *= rlvr_reward
reward = rlvr_reward + format_reward_weight * format_reward
```

To use this in practice, you can run the `7_6_plus_format_reward.py` script, which we used in section 7.6, with the `--conditional_reward` flag enabled.

You can download the log file of this run (using settings similar to those in section 7.6) and plot it as follows:

```
from reasoning_from_scratch.ch07 import download_from_github
from reasoning_from_scratch.ch07 import plot_grpo_metrics
download_from_github(
    "ch07/02_logs/7_6_plus_format_reward_conditional_metrics.csv"
)
plot_grpo_metrics(
    "7_6_plus_format_reward_conditional_metrics.csv",
    columns=["loss", "reward_avg", "avg_response_len", "eval_acc"],
)
```

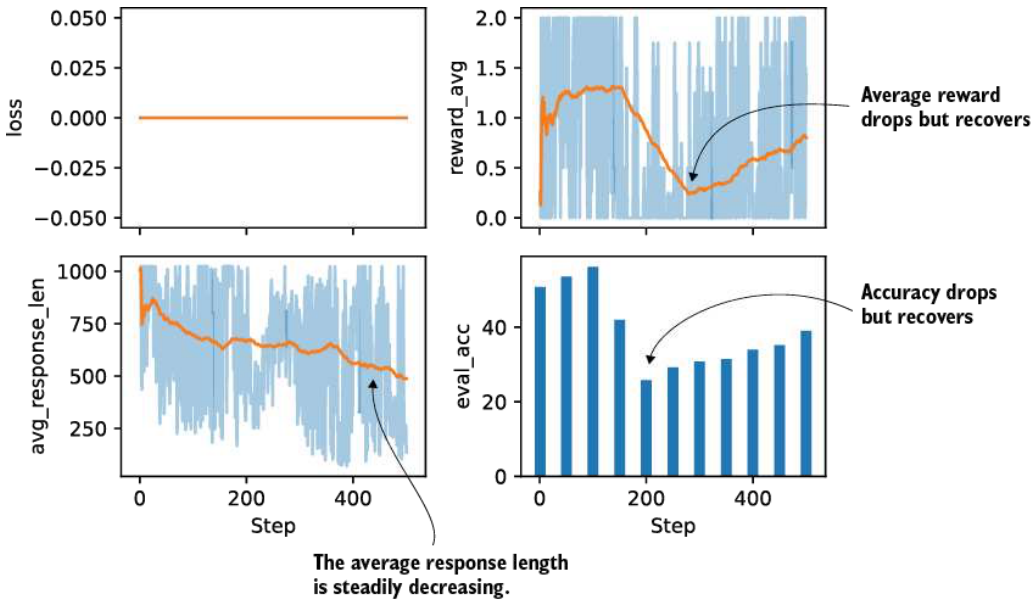


Figure B.1 Basic metrics from a GRPO training run with a conditional format reward

The plots in figure B.1 show that the evaluation accuracy and reward average take a big hit but seem to recover. Overall, despite the performance crash, this looks more stable than before, and the trend indicates that performance would improve further with longer training.

The previous plot focused on the main training metrics. To inspect the effect of the conditional format reward more directly, we can plot additional metrics from the same run:

```
plot_grpo_metrics(
    "7_6_plus_format_reward_conditional_metrics.csv",
    columns=["reward_avg", "format_reward_avg", "adv_std", "entropy_avg"],
)
```

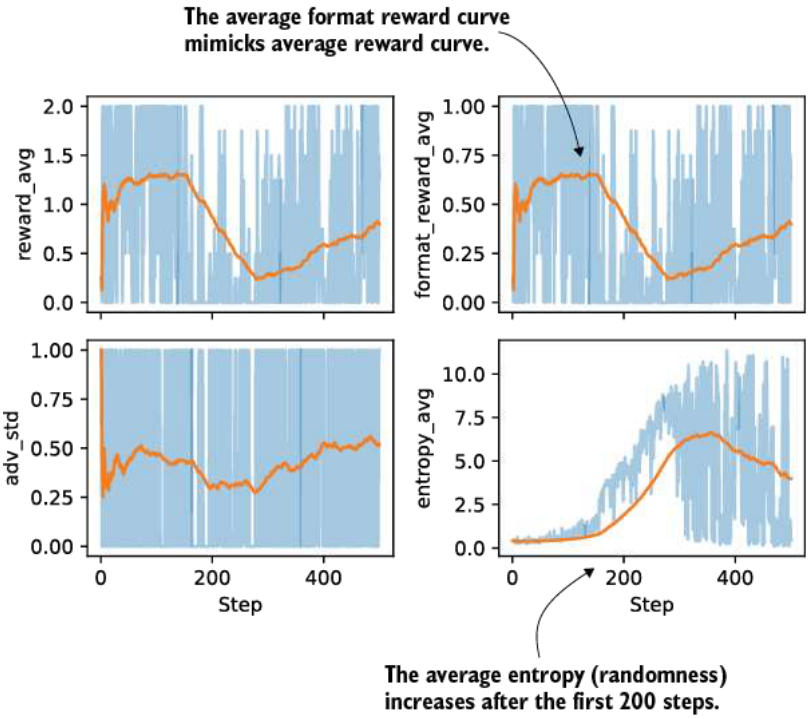


Figure B.2 Additional metrics from a GRPO training run with a conditional format reward

In figure B.2, we see the average format reward mimicking the average reward graph almost perfectly, which is a good sanity check that the conditional logic is working. The average format reward also shows how much of the total reward is coming from the format term on the subset of correct answers.

As we can see, though, since the average format reward graph echoes the average reward one, it's mainly a bonus. It looks like it's always awarded if the model is correct; this makes sense because the trained reasoning model already knows how to use `<think>...</think>` tags correctly, and we can see that it doesn't unlearn this ability.

The entropy increase is still a bit troubling, though, and could hint at training instabilities that could be addressed by other means (like tighter clipping with smaller `clip_eps`).

B.7 Chapter 8

EXERCISE 8.1: TRAINING AND VALIDATION SET LENGTHS

To calculate the training and validation answer length statistics, you can add the following commands at the end of listing 8.7 in section 8.4.3, following the partitioning:

```
compute_length(train_examples)
# Prints
# Average: 1180 tokens
# Shortest: 236 tokens (index 5730)
# Longest: 2048 tokens (index 1319)
```

As you can see, the average token length (1180 versus 1106) is fairly similar, and the datasets should be relatively balanced.

As a bonus, we can plot histograms to visualize the distributions:

```
import matplotlib.pyplot as plt

train_lengths = [len(ex["token_ids"]) for ex in train_examples]
val_lengths = [len(ex["token_ids"]) for ex in val_examples]

# Normalize counts because the validation split is much smaller
bins = range(0, max(train_lengths + val_lengths) + 64, 64)

fig, ax = plt.subplots(figsize=(7, 4))
ax.hist(train_lengths, bins=bins, density=True, alpha=0.6, label="Train")
ax.hist(val_lengths, bins=bins, density=True, alpha=0.6, label="Validation")
ax.set_xlabel("Token length")
ax.set_ylabel("Density")
ax.legend()
plt.tight_layout()
plt.show()
```

The resulting plot is shown in figure B.3.

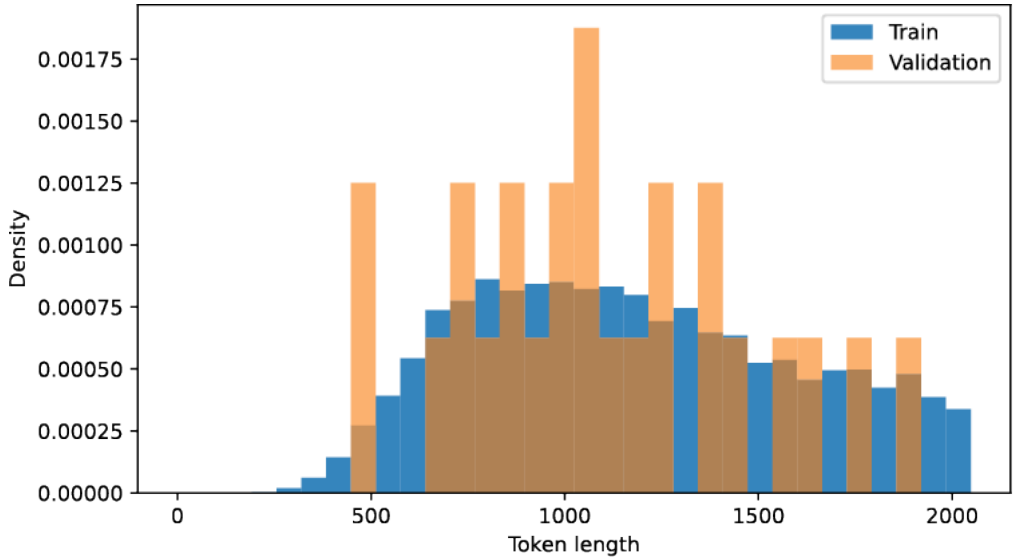


Figure B.3 Distribution of training and validation set lengths

There are far fewer validation samples, which is why the validation histogram seems a bit jagged, but as you can see, it still provides good coverage of the distribution.

EXERCISE 8.2: DISTILLING WITHOUT <THINK> TOKENS

We can replicate the runs without `<think></think>` tokens in the script execution command:

```
uv run distill.py \
--data_path deepseek-r1-math-train.json \
--validation_size 25 \
--epochs 3 \
--lr 1e-5 \
--max_seq_len 2048 \
--grad_clip 1.0
```

For the evaluation, we can use the `base` instead of `reasoning` model:

```
uv run evaluate_math500.py \
--dataset_size 500 \
--which_model base \
--max_new_tokens 4096 \
--checkpoint_path \
run_11/checkpoints/distill/qwen3-0.6B-distill-step05746-epoch1.pth
```

The results are shown in table B.6, where the new results (without `<think>` tokens) are shown first,

with the corresponding think-token results (from the chapter) in parentheses.

Table B.6 MATH-500 task accuracy with and without `<think>` tokens

	Method	Epoch	Final validation loss	MATH-500 accuracy
1	Base Qwen3 0.6B (chapter 3)	-	-	15.2%
2	Reasoning Qwen3 0.6B (chapter 3)	-	-	48.2%
3	DeepSeek-R1	1	0.5436 (0.5404)	31.8% (30.6%)
4	DeepSeek-R1	2	0.5349 (0.5339)	31.8% (32.4%)
5	DeepSeek-R1	3	0.5343 (0.5306)	30.2% (33.6%)
6	Qwen3 235B-A22B	1	0.4043 (0.3130)	44.8% (45.0%)
7	Qwen3 235B-A22B	2	0.3963 (0.3087)	39.4% (43.8%)
8	Qwen3 235B-A22B	3	0.3948 (0.3078)	39.8% (44.2%)

Interestingly, the Qwen3 model has a lower validation loss when `<think></think>` tokens are omitted, but this doesn't translate into better modeling performance. The omission of `<think>` makes the results slightly worse in almost all cases.

appendix C Qwen3 LLM source code

Although this is a "from scratch" book, that part of the title refers to the reasoning techniques, not to the LLM itself. Implementing an LLM entirely from scratch would require a separate book, such as my *Build a Large Language Model (From Scratch)*, also available from Manning (**http://mng.bz/orYv**).

If you're interested in the Qwen3 implementation used in this book, this appendix lists the source code for the `Qwen3Model` model that I implemented and that we import from the book's `reasoning_from_scratch` Python package:

```
from reasoning_from_scratch.qwen3 import Qwen3Model, Qwen3Tokenizer
```

As shown in figure C.1, the Qwen3 architecture is very similar to GPT-2, which I cover in my *Build A Large Language Model (From Scratch)* book. While familiarity with GPT-2 is not required for this book, this appendix includes comparisons to GPT-2 for readers who are familiar with it. In fact, I wrote the Qwen3 implementation by porting the GPT-2 model from my other book, piece by piece, into the Qwen3 architecture so that it follows similar style conventions and is easier to read.

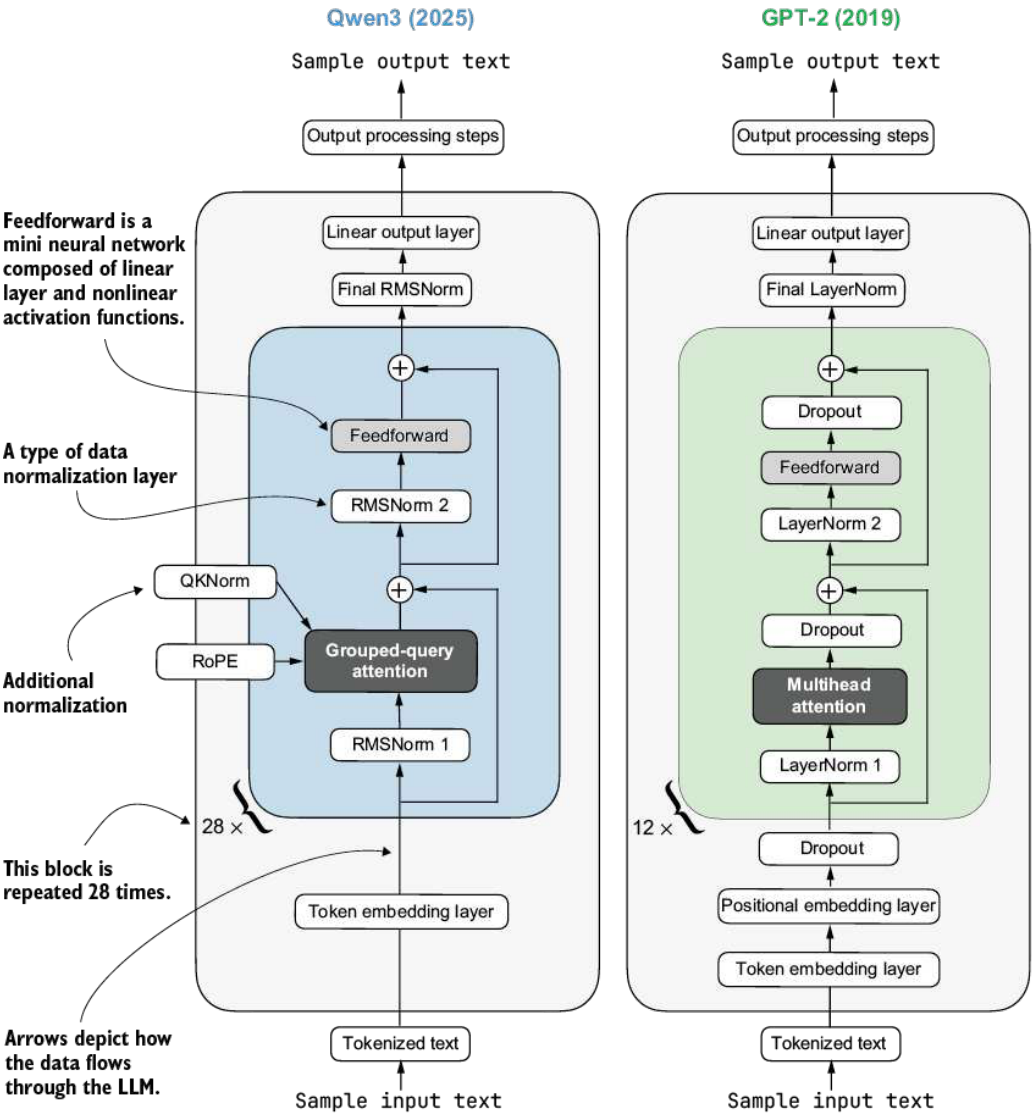


Figure C.1 Architectural comparison between Qwen3 and GPT-2. Both models process text through embedding layers and stacked transformer blocks, but they differ in certain design choices.

As shown in figure C.1, Qwen3 (released in 2025) and GPT-2 (released in 2019) are broadly similar in that they are both based on the decoder submodule of the original Transformer architecture. But some design

choices have evolved since 2019. Note that most of the design choices found in Qwen3 are not unique to Qwen3 but are found in many other contemporary LLMs, as I discussed in "The Big LLM Architecture Comparison" (**https://magazine.sebastianraschka.com/p/the-big-llm-architecture-comparison**).

If you're new to LLMs and want to understand how these architectures are implemented, I recommend starting with GPT-2. Its design is simpler to implement, which makes it an easier entry point before exploring more modern variations.

Since this book does not focus on architecture implementations, the rest of this appendix provides only a brief overview of Qwen3's code.

C.1 RMSNorm

In contrast to GPT-2, which used standard LayerNorm, the newer Qwen3 architecture uses *root mean square layer normalization* (RMSNorm). This trend has become increasingly common in recent model architectures.

RMSNorm fulfills the same core function as LayerNorm: normalizing layer activations to stabilize and improve training. But it simplifies the computation by removing the mean-centering step, as shown in figure C.2. This means that activations are still normalized, but they are not centered at 0.

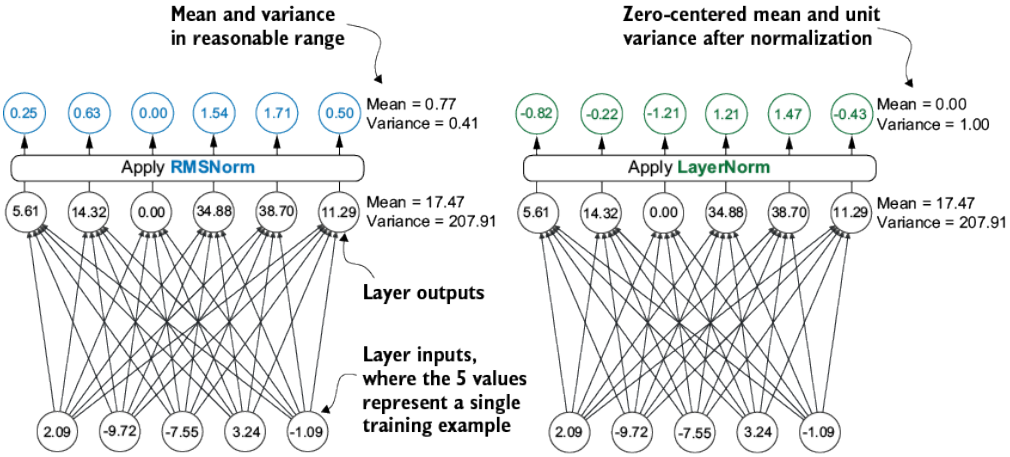


Figure C.2 Comparison of LayerNorm (used in GPT-2) and RMSNorm (used in Qwen3). LayerNorm (right) normalizes activations so that their average value (mean) is exactly zero and their spread (variance) is exactly one. RMSNorm (left) instead scales activations based on their root mean square, which does not enforce zero mean or unit variance, but still keeps the mean and variance within a reasonable range for stable training.

As you can see in figure C.2, both LayerNorm and RMSNorm scale the layer outputs to a reasonable range.

LayerNorm subtracts the mean and divides by the standard deviation so that the layer outputs have zero mean and unit variance (a variance of one and a standard deviation of one), which results in favorable gradient properties for stable training.

RMSNorm divides the inputs by the root mean square. This scales activations to a comparable magnitude without enforcing zero mean or unit variance. In the example shown in figure C.2, the mean is 0.77 and the variance is 0.41.

Both LayerNorm and RMSNorm stabilize activation scales and improve optimization, but RMSNorm is often preferred in large-scale LLMs because it is computationally cheaper. Unlike LayerNorm, RMSNorm does not use a bias (shift) term by default, which reduces the number of trainable parameters. Moreover, RMSNorm reduces the expensive mean and variance computations to a single root-mean-square operation. This reduces the number of cross-feature reductions from two to one, which lowers communication overhead on GPUs and slightly improves training efficiency.

The following listing shows what RMSNorm looks like in code.

Listing C.1 RMSNorm

```
import torch.nn as nn

class RMSNorm(nn.Module):

    def __init__(
        self,
        emb_dim,
        eps=1e-6,
        bias=False,
        qwen3_compatible=True,
    ):
        super().__init__()
        self.eps = eps
        self.qwen3_compatible = qwen3_compatible
        self.scale = nn.Parameter(torch.ones(emb_dim))
        self.shift = nn.Parameter(torch.zeros(emb_dim)) if bias else None

    def forward(self, x):
        input_dtype = x.dtype

        if self.qwen3_compatible:
            x = x.to(torch.float32)

        variance = x.pow(2).mean(dim=-1, keepdim=True)
        norm_x = x * torch.rsqrt(variance + self.eps)
        norm_x = norm_x * self.scale

        if self.shift is not None:
            norm_x = norm_x + self.shift

        return norm_x.to(input_dtype)
```

Note that, for brevity, this appendix does not provide detailed code walk-throughs for each LLM component. Instead, in section C.6, we'll integrate all the components into the `Qwen3Model` class, load the pretrained weights into it, and then use this model to generate text in section C.9.

C.2 Feedforward module

The *feedforward module* (a small multilayer perceptron) is replaced with a *gated linear unit* (GLU) variant introduced in a 2020 paper, "GLU Variants Improve Transformer" (**https://arxiv.org/abs/2002.05202**). In this design, the standard two fully connected layers are replaced with three, as shown in figure C.3.

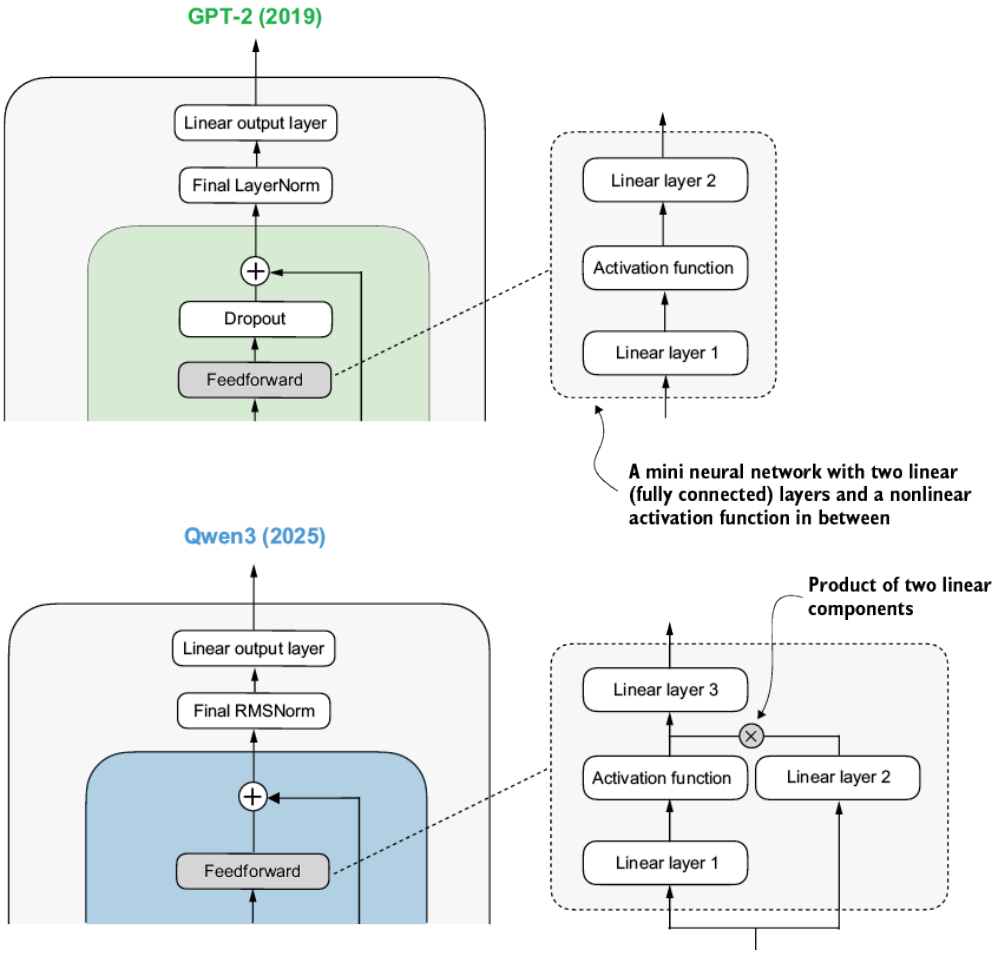


Figure C.3 In GPT-2 (top), the feedforward module consists of two fully connected (linear) layers separated by a nonlinear activation function. In Qwen3 (bottom), this module is a gated linear unit (GLU) variant that adds a third linear layer (linear layer 3) and multiplies its output elementwise with the activated output of linear layer 1.

Qwen3's feedforward module, shown in figure C.3, can be implemented as shown in the next listing.

Listing C.2 Qwen3's feedforward module

```
class FeedForward(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.fc1 = nn.Linear(
            cfg["emb_dim"], cfg["hidden_dim"], dtype=cfg["dtype"],
            bias=False
        )
        self.fc2 = nn.Linear(
            cfg["emb_dim"], cfg["hidden_dim"], dtype=cfg["dtype"],
            bias=False
        )
        self.fc3 = nn.Linear(
            cfg["hidden_dim"], cfg["emb_dim"], dtype=cfg["dtype"],
            bias=False
        )

    def forward(self, x):
        x_fc1 = self.fc1(x)
        x_fc2 = self.fc2(x)
        x = nn.functional.silu(x_fc1) * x_fc2  #1
        return self.fc3(x)
```

#1 The non-linear activation function here is a SiLU function, which will be discussed later.

At first glance, it might seem that the GLU feedforward variant used in Qwen3 should outperform the standard feedforward variant in GPT-2, simply because it adds an extra linear layer (three instead of two) and therefore appears to have more parameters.

But this intuition is misleading. The `fc1` and `fc2` layers in the GLU variant are each half the width of

the `fc1` layer in a standard feedforward module, so the GLU variant has fewer parameters.

To illustrate this with a concrete example, suppose the input dimension to linear layer 1 in figure C.3 is 1,024. This corresponds to `cfg[ " emb_dim " ]` in listing C.2. The output dimension of `fc1` is 3,072 (`cfg[ " hidden_dim " ]`). These are the actual numbers used in the Qwen3 0.6B variant. In this case, we have the following parameter counts for the GLU variant in listing C.2:

- `fc1` : 1024 × 3,072 = 3,145,728
- `fc2` : 1024 × 3,072 = 3,145,728
- `fc3` : 1024 × 3,072 = 3,145,728
- Total: 3 × 3,145,728 = 9,437,184 parameters

If we assume that `fc1` in this GLU variant has half the width typically chosen for `fc1` in a standard feedforward module, the parameter counts of the standard feedforward module would be as follows:

- `fc1` : 1024 × 2 × 3,072 = 6,291,456
- `fc2` : 1024 × 2 × 3,072 = 6,291,456
- Total: 2 × 6,291,456 = 12,582,912 parameters

While GLU variants usually have fewer parameters than regular feedforward modules, they perform better. The improvement comes from the additional multiplicative interaction introduced by the gating mechanism, `activation(x_fc1) * x_fc2`, which increases the model's expressivity. This is similar to how deeper, slimmer networks can outperform shallower, wider ones, given proper training.

Before we proceed to the next section, there is one more thing to address. Note that the feedforward module shown in figure C.3 contains an element labeled "Activation function," but we used an `nn.functional.silu` activation as a concrete example in listing C.2.

Historically, activation functions were a hot topic of debate until the deep learning community largely converged on the *rectified linear unit* (ReLU) more than a decade ago. ReLU is simple and computationally cheap, but it has a sharp kink at zero. This motivated researchers to explore smoother functions such as the *Gaussian error linear unit* (GELU) and the *sigmoid linear unit* (SiLU), as shown in figure C.4.

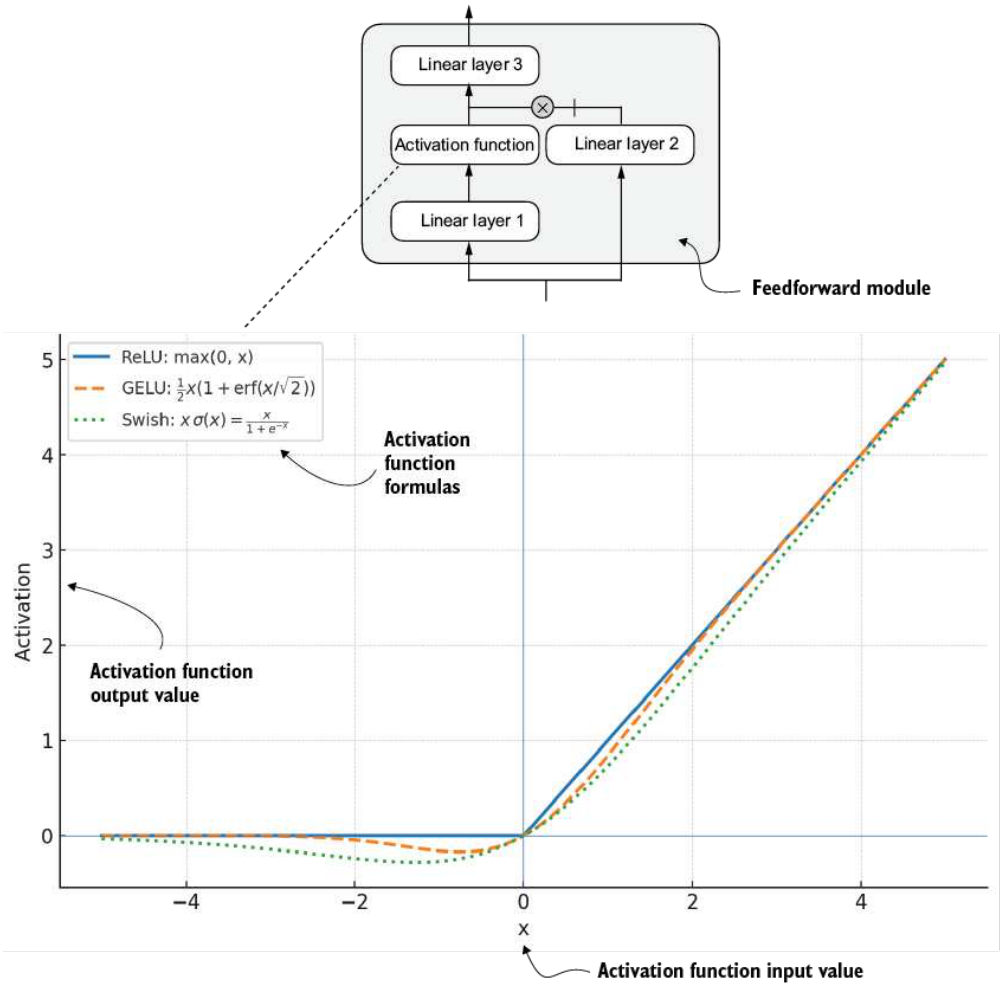


Figure C.4 Different activation functions that can be used in a feedforward module (neural network). GELU and SiLU (Swish) offer smooth alternatives to ReLU, which has a sharp kink at input zero.

GELU involves the Gaussian cumulative distribution function (CDF). Computing this CDF is slow because it uses piecewise logic and exponentials, which makes it hard to write fused, optimized GPU kernels, although a tanh approximation uses cheaper operations and runs faster with near-identical results. In short, while GELU produces smooth activation curves, it is overall computationally more expensive than simpler functions.

Newer models have largely replaced GELU with the SiLU (also known as *Swish*) function, which smoothly suppresses large negative inputs toward ~0 and is approximately linear for large positive inputs, as shown in figure C.4.

SiLU has similar smoothness, but it is slightly cheaper to compute than GELU and offers comparable modeling performance. In practice, SiLU is now used in most architectures, while GELU remains in use in only some models, such as Google's Gemma open-weight LLM. In the implementation of the feedforward module in listing C.2, the SiLU function is called via `nn.functional.silu`. The feedforward module in listing C.2 is also often called *SwiGLU*, an abbreviation derived from the terms Swish and GLU.

C.3 Rotary position embeddings

In transformer-based LLMs, positional encoding is necessary because of the attention mechanism. By default, attention treats the input tokens as if they have no order. In the original GPT architecture, absolute positional embeddings addressed this by adding a learned embedding vector for each position in the sequence, which is then added to the token embeddings.

RoPE (short for *rotary position embeddings*) uses a different approach: instead of adding position information as separate embeddings, it encodes that information by rotating the query and key vectors in the attention mechanism in a way that depends on each token's position (the attention mechanism is discussed in the next section). RoPE is an elegant idea, but it is also a large topic in itself. If you're interested, you can find more information in the original RoPE paper, "RoFormer: Enhanced Transformer with Rotary Position Embedding," at **https://arxiv.org/abs/2104.09864**. (RoPE was first introduced in 2021, and it became widely adopted with the release of the original Llama model in 2023. It has since become a staple

in modern LLMs, so it is not unique to Qwen3.)

RoPE can be implemented in two mathematically equivalent ways: the interleaved form, which pairs adjacent dimensions for rotation, or the two-halves form, which splits the dimension into cosine and sine halves for convenience. The next listing implements the two-halves variant, which can be easier to read.

Listing C.3 RoPE functions

```
import torch


def compute_rope_params(head_dim, theta_base=10_000, context_length=4096,
                        dtype=torch.float32):
    assert head_dim % 2 == 0, "Embedding dimension must be even"
    inv_freq = 1.0 / (theta_base ** (
        torch.arange(0, head_dim, 2, dtype=dtype)[: (head_dim // 2)].float()
        / head_dim
    ))
    positions = torch.arange(context_length, dtype=dtype)
    angles = positions[:, None] * inv_freq[None, :]
    angles = torch.cat([angles, angles], dim=1)

    cos = torch.cos(angles)
    sin = torch.sin(angles)

    return cos, sin


def apply_rope(x, cos, sin, offset=0):
    batch_size, num_heads, seq_len, head_dim = x.shape    #1
    assert head_dim % 2 == 0, "Head dimension must be even"

    x1 = x[..., : head_dim // 2]  # First half    #2
    x2 = x[..., head_dim // 2:]  # Second half     #2


    cos = cos[offset:offset + seq_len, :].unsqueeze(0).unsqueeze(0)
    sin = sin[offset:offset + seq_len, :].unsqueeze(0).unsqueeze(0)
    # Shape after:  (1, 1, seq_len, head_dim)

    rotated = torch.cat((-x2, x1), dim=-1)
    x_rotated = (x * cos) + (rotated * sin)

    return x_rotated.to(dtype=x.dtype)    #3
```

#1 The shape is (batch_size, num_heads, seq_len, head_dim).

#2 Split x into first half and second half.

#3 It's ok to use lower-precision after applying cos and sin rotation.

The RoPE code in listing C.3 will be used in the grouped query attention mechanism discussed in section C.4.

ROPE IMPLEMENTATION VARIANTS

If you're familiar with the original RoPE paper (**https://arxiv.org/abs/2104.09864**), you may be wondering about the particular implementation I have chosen. There are two common styles for implementing RoPE, which are mathematically equivalent. The implementations mainly differ in how the rotation matrix pairs dimensions. I chose the split-halves style because it is a bit easier to read and implement.

```
1) Split-halves style (this book, Hugging Face Transformers):
      [ x0   x1   x2   x3   x4   x5   x6   x7 ]
        |    |    |    |    |    |    |    |
        ▼    ▼    ▼    ▼    ▼    ▼    ▼    ▼
       cos  cos  cos  cos  sin  sin  sin  sin
```

This is the rotation matrix:

```
[ cosθ   -sinθ    0      0   ... ]
[ sinθ    cosθ    0      0   ... ]
[  0       0    cosθ   -sinθ ... ]
[  0       0    sinθ    cosθ ... ]
```

Here, the embedding dimensions are split into two halves, and each one is rotated in blocks.

The interleaved (even/odd) RoPE style from the original paper (and the official Llama code repository) looks like this:

```
[ x0   x1   x2   x3   x4   x5   x6   x7 ]
  |    |    |    |    |    |    |    |
  ▼    ▼    ▼    ▼    ▼    ▼    ▼    ▼
 cos  sin  cos  sin  cos  sin  cos  sin
```

This is the rotation matrix:

```
[ cosθ  -sinθ    0      0   ... ]
[ sinθ   cosθ    0      0   ... ]
[  0      0    cosθ   -sinθ ... ]
[  0      0    sinθ    cosθ ... ]
```

Here, embedding dimensions are interleaved as even/odd cosine/sine pairs.

Both layouts encode the same relative positions. The only difference is in how the dimensions are paired.

C.4 Grouped query attention

Grouped query attention (*GQA*) has become the standard, more compute- and parameter-efficient alternative to the original multihead attention (MHA) mechanism. Unlike MHA, where each head has its own set of keys and values, GQA reduces memory usage by grouping multiple heads to share the same key and value projections, as shown in figure C.5.

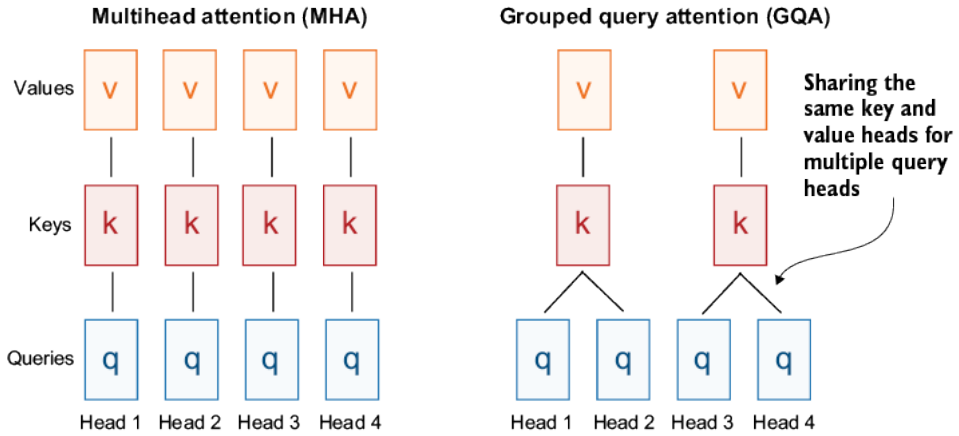


Figure C.5 A comparison of MHA and GQA. Here, the group size is 2, meaning that a key-value pair is shared by 2 queries.

The core idea behind GQA is to reduce the number of key and value heads by sharing them across multiple query heads. This both lowers the model's parameter count and reduces memory bandwidth use for key and value tensors during inference, since fewer keys and values need to be stored and retrieved from the KV cache (discussed in section C.7).

While GQA is primarily a computational efficiency workaround for MHA, ablation studies presented in the original GQA paper, "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints" (**https://arxiv.org/abs/2305.13245**), show that it performs comparably to standard MHA in LLM modeling performance.

The next listing implements the GQA mechanism with KV cache support.

Listing C.4 Grouped query attention

```
class GroupedQueryAttention(nn.Module):
    def __init__(self, d_in, num_heads, num_kv_groups, head_dim=None,
                 qk_norm=False, dtype=None):
        super().__init__()
        assert num_heads % num_kv_groups == 0

        self.num_heads = num_heads
        self.num_kv_groups = num_kv_groups
        self.group_size = num_heads // num_kv_groups

        if head_dim is None:
            assert d_in % num_heads == 0
            head_dim = d_in // num_heads

        self.head_dim = head_dim
        self.d_out = num_heads * head_dim

        self.W_query = nn.Linear(
            d_in, self.d_out, bias=False, dtype=dtype
        )
        self.W_key = nn.Linear(
            d_in, num_kv_groups * head_dim, bias=False,dtype=dtype
        )
        self.W_value = nn.Linear(
            d_in, num_kv_groups * head_dim, bias=False, dtype=dtype
        )

        self.out_proj = nn.Linear(self.d_out, d_in, bias=False, dtype=dtype)

        if qk_norm:
            self.q_norm = RMSNorm(head_dim, eps=1e-6)
            self.k_norm = RMSNorm(head_dim, eps=1e-6)
        else:
            self.q_norm = self.k_norm = None

    def forward(self, x, mask, cos, sin, start_pos=0, cache=None):
```

```
        b, num_tokens, _ = x.shape

        queries = self.W_query(x)            #1
        keys = self.W_key(x)                 #2
        values = self.W_value(x)              #2

        queries = queries.view(b, num_tokens, self.num_heads,
                               self.head_dim).transpose(1, 2)
        keys_new = keys.view(b, num_tokens, self.num_kv_groups,
                             self.head_dim).transpose(1, 2)
        values_new = values.view(b, num_tokens, self.num_kv_groups,
                                 self.head_dim).transpose(1, 2)

        if self.q_norm:
            queries = self.q_norm(queries)
        if self.k_norm:
            keys_new = self.k_norm(keys_new)

        queries = apply_rope(queries, cos, sin, offset=start_pos)
        keys_new = apply_rope(keys_new, cos, sin, offset=start_pos)

        if cache is not None:
            prev_k, prev_v = cache
            keys = torch.cat([prev_k, keys_new], dim=2)
            values = torch.cat([prev_v, values_new], dim=2)
        else:
            start_pos = 0                    #3
            keys, values = keys_new, values_new
        next_cache = (keys, values)


        keys = keys.repeat_interleave(       #4
            self.group_size, dim=1            #4
        )                                     #4
        values = values.repeat_interleave(    #4
            self.group_size, dim=1            #4
        )                                     #4
    #4
        attn_scores = queries @ keys.transpose(2, 3)
        attn_scores = attn_scores.masked_fill(mask, -torch.inf)
        attn_weights = torch.softmax(
            attn_scores / self.head_dim**0.5, dim=-1
        )

        context = (attn_weights @ values).transpose(1, 2)
        context = context.reshape(b, num_tokens, self.d_out)
        return self.out_proj(context), next_cache
```

#1 The shape is (b, num_tokens, num_heads * head_dim).

#2 The shapes are (b, num_tokens, num_kv_groups * head_dim).

#3 Reset RoPE.

#4 Expand K and V to match the number of heads.

You may have noticed that the GQA mechanism in listing C.4 also includes a `qk_norm` parameter that is not part of the standard GQA design. When `qk_norm=True`, the model applies RMSNorm separately to the query and key vectors before computing attention scores. This variant is often called QKNorm because it normalizes the query (Q) and key (K) representations. In other words, QKNorm uses the same RMSNorm operation introduced in section C.1, but applies it inside the attention mechanism rather than as a general normalization layer around the transformer block. In Qwen3, this query/key normalization is used (in addition to the regular RMSNorm layers) to help stabilize the attention scores during training.

C.5 Transformer block

The *transformer block* is the central component of an LLM, combining all the individual elements covered in this appendix so far. As shown in figure C.6, it is repeated multiple times; in the 0.6B-parameter version of Qwen3, it appears 28 times.

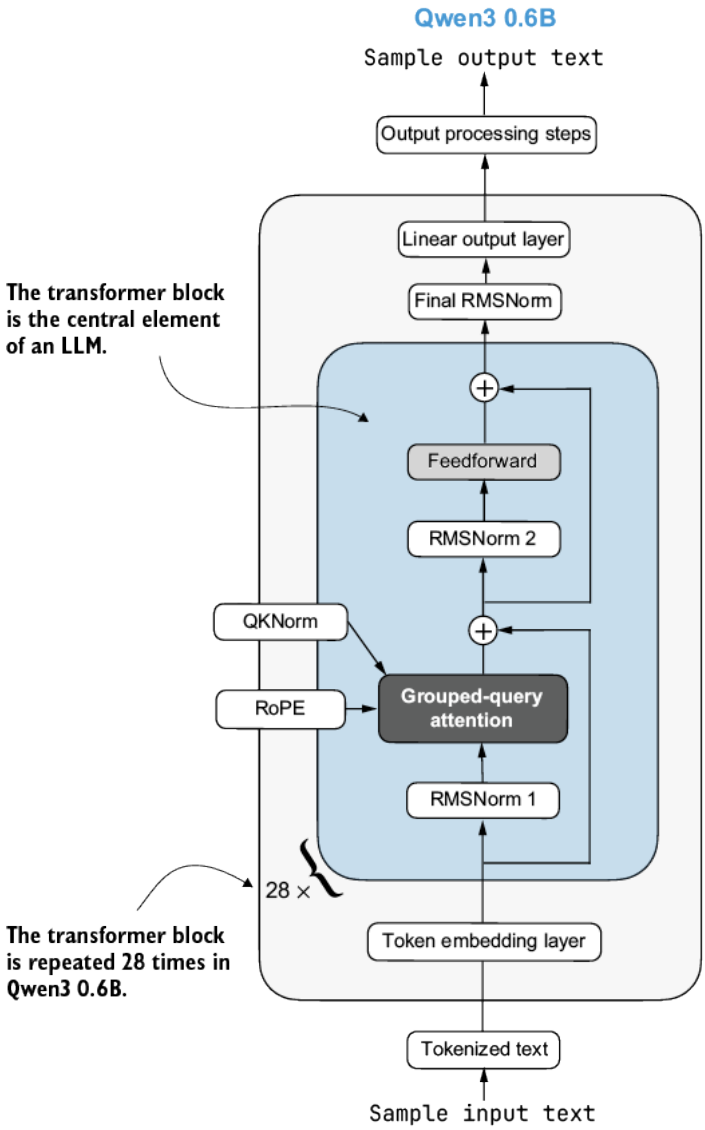


Figure C.6 The structure of the transformer block in Qwen3. Each block includes RMSNorm, RoPE, masked grouped-query attention, and a feedforward module, and it's repeated 28 times in the 0.6B-parameter model.

Listing C.5 implements the transformer block shown in figure C.6.

Listing C.5 Transformer block

```
class TransformerBlock(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.att = GroupedQueryAttention(
            d_in=cfg["emb_dim"],
            num_heads=cfg["n_heads"],
            head_dim=cfg["head_dim"],
            num_kv_groups=cfg["n_kv_groups"],
            qk_norm=cfg["qk_norm"],
            dtype=cfg["dtype"]
        )
        self.ff = FeedForward(cfg)
        self.norm1 = RMSNorm(cfg["emb_dim"], eps=1e-6)
        self.norm2 = RMSNorm(cfg["emb_dim"], eps=1e-6)

    def forward(self, x, mask, cos, sin, start_pos=0, cache=None):
        shortcut = x
        x = self.norm1(x)
        x, next_cache = self.att(
            x, mask, cos, sin, start_pos=start_pos,cache=cache
        )  #1
        x = x + shortcut

        shortcut = x
        x = self.norm2(x)
        x = self.ff(x)
        x = x + shortcut

        return x, next_cache
```

#1 The shape is (batch_size, num_tokens, emb_size).

As you can see in the listing, the transformer block simply connects the various elements we implemented in previous sections.

C.6 Main model code

In this section, we'll define the `Qwen3Model` class that we imported and used in chapter 2.

To implement the `Qwen3Model` class, the following listing follows the architecture shown in figure C.6, with the transformer block at the heart of the LLM.

Listing C.6 Main Qwen3Model code

```
class Qwen3Model(nn.Module):
    def __init__(self, cfg):
        super().__init__()

        # Main model parameters
        self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"],
                                    dtype=cfg["dtype"])

        self.trf_blocks = nn.ModuleList(
            [TransformerBlock(cfg) for _ in range(cfg["n_layers"])]
        )
        self.final_norm = RMSNorm(cfg["emb_dim"])
        self.out_head = nn.Linear(
            cfg["emb_dim"], cfg["vocab_size"],
            bias=False, dtype=cfg["dtype"]
        )

        # Reusable utilities
        if cfg["head_dim"] is None:
            head_dim = cfg["emb_dim"] // cfg["n_heads"]
        else:
```

```
            head_dim = cfg["head_dim"]
        cos, sin = compute_rope_params(
            head_dim=head_dim,
            theta_base=cfg["rope_base"],
            context_length=cfg["context_length"]
        )
        self.register_buffer("cos", cos, persistent=False)
        self.register_buffer("sin", sin, persistent=False)
        self.cfg = cfg
        self.current_pos = 0  # Track current position in KV cache

    def forward(self, in_idx, cache=None):
        tok_embeds = self.tok_emb(in_idx)
        x = tok_embeds

        num_tokens = x.shape[1]
        if cache is not None:
            pos_start = self.current_pos
            pos_end = pos_start + num_tokens
            self.current_pos = pos_end
            mask = torch.triu(
                torch.ones(
                    pos_end, pos_end, device=x.device, dtype=torch.bool
                ),
                diagonal=1
            )[pos_start:pos_end, :pos_end]
        else:
            pos_start = 0  # Not strictly necessary but helps torch.compile
            mask = torch.triu(
                torch.ones(num_tokens, num_tokens, device=x.device,
                           dtype=torch.bool),
                diagonal=1
            )

        mask = mask[None, None, :, :]                #1

        for i, block in enumerate(self.trf_blocks):
            blk_cache = cache.get(i) if cache else None
            x, new_blk_cache = block(x, mask, self.cos, self.sin,
                                     start_pos=pos_start,
                                     cache=blk_cache)
            if cache is not None:
                cache.update(i, new_blk_cache)

        x = self.final_norm(x)
        logits = self.out_head(x.to(self.cfg["dtype"]))
        return logits

    def reset_kv_cache(self):
        self.current_pos = 0
```

#1 Prefill: Shape (1, 1, T, T) to broadcast across batch and heads. Cached: Shape (1, 1, T, K+T) where T=new tokens, K=cached keys.

We already have all the main ingredients, so the `Qwen3Model` class in listing C.6 only adds a few more components around the transformer block, namely, the embedding and output layers, including one more RMSNorm layer. The code may still appear somewhat complicated because of the KV cache option.

As discussed in chapter 2, the KV cache can speed up the text generation process, but it is outside the scope of this book. If you are interested, you can learn more about KV caching in my "Understanding and Coding the KV Cache in LLMs from Scratch" article at **https://magazine.sebastianraschka.com/p/coding-the-kv-cache-in-llms**.

Note that the `Qwen3Model` class, as implemented in listing C.6, supports various model sizes (as discussed in appendix D). In chapter 2, we use the 0.6B-parameter model because it is the least resource-intensive model in the Qwen3 model family. The configuration of this model is shown in figure C.7.

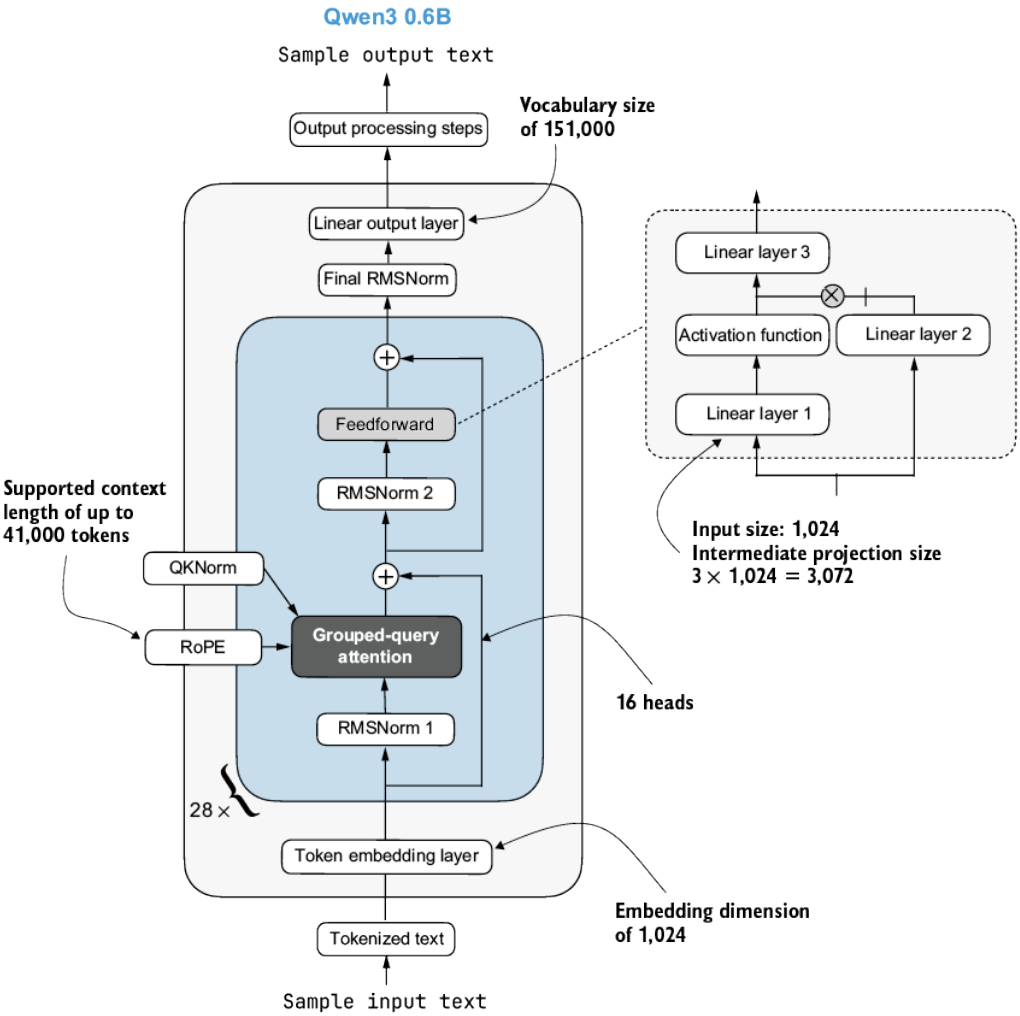


Figure C.7 Architecture of the Qwen3 0.6B model. The model consists of a token-embedding layer followed by 28 transformer blocks, each containing RMSNorm, RoPE, QKNorm, masked grouped-query attention with 16 heads, and a feedforward module with an intermediate size of 3,072.

To use the 0.6B model shown in figure C.7 via the `Qwen3Model` class, we can define the following configuration and provide it as input (`cfg=QWEN_CONFIG_06_B`) when instantiating a new `Qwen3Model` instance.

Listing C.7 Qwen3 0.6B configuration

```
QWEN_CONFIG_06_B = {
    "vocab_size": 151_936,     # Vocabulary size
    "context_length": 40_960,  # Length originally used during training
    "emb_dim": 1024,           # Embedding dimension
    "n_heads": 16,             # Number of attention heads
    "n_layers": 28,            # Number of layers
    "hidden_dim": 3072,        # Size of intermediate dim in FeedForward
    "head_dim": 128,           # Size of the heads in GQA
    "qk_norm": True,           # Whether to normalize queries & keys in GQA
    "n_kv_groups": 8,          # Key-Value groups for GQA
    "rope_base": 1_000_000.0,  # The base in RoPE's "theta"
    "dtype": torch.bfloat16,   # Lower-precision dtype to reduce memory
}
```

We will use the `QWEN_CONFIG_06_B` configuration from listing C.7 to instantiate the Qwen3 0.6B model in section C.9.

MORE PRECISE CONTEXT-LENGTH INFORMATION

For Qwen3 0.6B, the default maximum supported context length is 40,960 tokens. But the model itself was trained with only 32,768 tokens. The 40,960-token allocation reserves 32,768 tokens for model outputs (the generated text) and 8,192 tokens for typical prompts (the user's question or instruction). To put this in perspective, 40,000 tokens correspond to roughly half of the first Harry Potter book.

Since this length is sufficient for most reasoning tasks, the developers note that context-extension methods (such as Yarn) are not recommended unless the average context regularly exceeds 32,000 tokens, because enabling them in shorter contexts can slightly degrade performance.

C.7 KV cache

The KV-cache-related heavy lifting is mostly done in the `Qwen3Model` (listing C.6) and `GroupedQueryAttention` (listing C.4) code. The `KVCache`, shown in the following listing, stores the key-value pairs during text generation, which results in the speedup we experienced when enabling KV caching in chapter 2.

Listing C.8 KV cache

```
class KVCache:
    def __init__(self, n_layers):
        self.cache = [None] * n_layers

    def get(self, layer_idx):
        return self.cache[layer_idx]

    def update(self, layer_idx, value):
        self.cache[layer_idx] = value

    def get_all(self):
        return self.cache

    def reset(self):
        for i in range(len(self.cache)):
            self.cache[i] = None
```

This `KVCache` class is used inside the `generate_text_basic_stream_cache` function that we implemented in chapter 2.

C.8 Tokenizer

The *tokenizer* code is somewhat complicated because it supports a variety of special tokens, in addition to the base model and the so-called "thinking" model variant of Qwen3, which is a reasoning model. The full reimplementation of the tokenizer is shown next.

Listing C.9 Tokenizer

```
import re
from tokenizers import Tokenizer

class Qwen3Tokenizer:
    _SPECIALS = [
        "<|endoftext|>",
        "<|im_start|>", "<|im_end|>",
        "<|object_ref_start|>", "<|object_ref_end|>",
        "<|box_start|>", "<|box_end|>",
        "<|quad_start|>", "<|quad_end|>",
        "<|vision_start|>", "<|vision_end|>",
        "<|vision_pad|>", "<|image_pad|>", "<|video_pad|>",
    ]
    _SPLIT_RE = re.compile(r"(<\|[^>]+?\|>)")
```

```
    def __init__(self,
                 tokenizer_file_path="tokenizer-base.json",
                 apply_chat_template=False,
                 add_generation_prompt=False,
                 add_thinking=False):

        self.apply_chat_template = apply_chat_template
        self.add_generation_prompt = add_generation_prompt
        self.add_thinking = add_thinking

        tok_path = Path(tokenizer_file_path)
        if not tok_path.is_file():
            raise FileNotFoundError(
                f"Tokenizer file '{tok_path}' not found. "
            )

        self._tok = Tokenizer.from_file(str(tok_path))
        self._special_to_id = {t: self._tok.token_to_id(t)
                               for t in self._SPECIALS}

        self.pad_token = "<|endoftext|>"
        self.pad_token_id = self._special_to_id.get(self.pad_token)

        f = tok_path.name.lower()                       #1
        if "base" in f and "reasoning" not in f:         #1
            self.eos_token = "<|endoftext|>"             #1
        else:                                            #1
            self.eos_token = "<|im_end|>"                #1
        self.eos_token_id = self._special_to_id.get(self.eos_token)

    def encode(self, prompt, chat_wrapped=None):
        if chat_wrapped is None:
            chat_wrapped = self.apply_chat_template

        stripped = prompt.strip()
        if stripped in self._special_to_id and "\n" not in stripped:
            return [self._special_to_id[stripped]]

        if chat_wrapped:
            prompt = self._wrap_chat(prompt)

        ids = []
        for part in filter(None, self._SPLIT_RE.split(prompt)):
            if part in self._special_to_id:
                ids.append(self._special_to_id[part])
            else:
                ids.extend(self._tok.encode(part).ids)
        return ids

    def decode(self, token_ids):
        return self._tok.decode(token_ids, skip_special_tokens=False)

    def _wrap_chat(self, user_msg):
        s = f"<|im_start|>user\n{user_msg}<|im_end|>\n"
        if self.add_generation_prompt:
            s += "<|im_start|>assistant"
            if self.add_thinking:
                s += "\n"          #2
            else:
                s += "\n<think>\n\n</think>\n\n"
        return s
```

#1 Match HF behavior: chat model: <|im_end|>, base model: <|endoftext|>.

#2 Insert no <think> tag, just a new line.

My `Qwen3Tokenizer` reimplementation in listing C.9 may appear somewhat complicated because it aims to replicate the behavior of the official tokenizer released by the Qwen3 team in the Hugging Face Transformers library.

At first glance, it appears to have a few quirks. For example, when `add_thinking=True`, no `" \n<think>\n\n</think>\n\n "` tokens are inserted (where `\n` is a newline character), but when `add_thinking=False`, these tokens are added. This is intentional because the non-base Qwen3 0.6B model is a hybrid that supports both reasoning ("thinking") and standard modes.

C.9 Using the model

Let's now instantiate and use the model to confirm that the code works by reusing the text generation approach from chapter 2.

First, we'll instantiate the model using the pretrained model weights:

```
from pathlib import Path
import torch

from reasoning_from_scratch.ch02 import get_device
from reasoning_from_scratch.qwen3 import download_qwen3_small

# device = get_device()         #1
device = torch.device("cpu")

download_qwen3_small(kind="base", tokenizer_only=False, out_dir="qwen3")

tokenizer_file_path = Path("qwen3") / "tokenizer-base.json"
model_file = Path("qwen3") / "qwen3-0.6B-base.pth"

tokenizer = Qwen3Tokenizer(tokenizer_file_path=tokenizer_file_path)
model = Qwen3Model(QWEN_CONFIG_06_B)
model.load_state_dict(torch.load(model_file))

model.to(device)
```

#1 Optional: Uncomment to use automatic device picker.

The following output prints the instantiated model architecture. In particular, it shows that the model object was created with the same settings specified in listing C.7, such as the number of layers, attention heads, key-value heads, embedding size, and feed-forward dimensions:

```
✓ qwen3/qwen3-0.6B-base.pth already up-to-date
✓ qwen3/tokenizer-base.json already up-to-date
Qwen3Model(
  (tok_emb): Embedding(151936, 1024)
  (trf_blocks): ModuleList(
    (0-27): 28 x TransformerBlock(
      (att): GroupedQueryAttention(
        (W_query): Linear(in_features=1024, out_features=2048, bias=False)
        (W_key): Linear(in_features=1024, out_features=1024, bias=False)
        (W_value): Linear(in_features=1024, out_features=1024, bias=False)
        (out_proj): Linear(in_features=2048, out_features=1024, bias=False)
        (q_norm): RMSNorm()
        (k_norm): RMSNorm()
      )
      (ff): FeedForward(
        (fc1): Linear(in_features=1024, out_features=3072, bias=False)
        (fc2): Linear(in_features=1024, out_features=3072, bias=False)
        (fc3): Linear(in_features=3072, out_features=1024, bias=False)
      )
      (norm1): RMSNorm()
      (norm2): RMSNorm()
    )
  )
  (final_norm): RMSNorm()
  (out_head): Linear(in_features=1024, out_features=151936, bias=False)
)
```

Next, we'll reuse the text generation functions from chapter 2 to generate text:

```
import time

from reasoning_from_scratch.ch02 import (
   generate_text_basic_stream_cache,
   generate_stats
)

prompt = "Explain large language models in a single sentence."
input_token_ids_tensor = torch.tensor(
    tokenizer.encode(prompt),
    device=device
    ).unsqueeze(0)
max_new_tokens = 200

start_time = time.time()
generated_ids = []

for token in generate_text_basic_stream_cache(
    model=model,
    token_ids=input_token_ids_tensor,
    max_new_tokens=max_new_tokens,
    eos_token_id=tokenizer.eos_token_id
):
    token_id = token.squeeze(0).tolist()
    print(
        tokenizer.decode(token_id),
        end="",
        flush=True
    )

    next_token_id = token.squeeze(0)
    generated_ids.append(next_token_id)  # Collect generated tokens

end_time = time.time()

output_token_ids_tensor = torch.cat(generated_ids, dim=0)
generate_stats(output_token_ids_tensor, tokenizer, start_time, end_time)
```

Since the preceding code uses the same prompt as in chapter 2, the generated text matches the text from chapter 2 exactly:

```
Time: 1.46 sec
28 tokens/sec

 Large language models are artificial intelligence systems that can
understand, generate, and process human language, enabling them to
perform a wide range of tasks, from answering questions to writing
articles, and even creating creative content.
```

While we use the 0.6B-parameter variant of Qwen3 in the chapter discussions to keep this book's resource requirements lower, you can find more information on using the larger models in appendix D.

appendix D Using larger LLMs

This book uses the 0.6-billion-parameter (0.6B) Qwen3 base model because it is the smallest model in the Qwen3 family and therefore the easiest to run on consumer hardware. But the same `Qwen3Model` implementation from appendix C is not limited to the 0.6B checkpoint. We can also use it to load larger Qwen3 checkpoints with the same PyTorch code we built from scratch.

In practice, this means that once we understand how to work with the 0.6B model, moving to a larger model mainly involves three changes:

- Selecting the matching configuration dictionary
- Downloading the larger checkpoint from Hugging Face
- Loading the appropriate tokenizer for the base or reasoning variant

This appendix illustrates this process using the Qwen3 4B model as an example because it is meaningfully stronger than the 0.6B model while still being easier to handle than the larger 8B, 14B, and 32B variants.

D.1 Larger dense Qwen3 configurations

The book's repository includes configuration dictionaries for several larger Qwen3 models in the `reasoning_from_scratch.appendix_c` Python library, listed in table D.1. You can also view the source code directly at **https://github.com/rasbt/reasoning-from-scratch/blob/main/reasoning_from_scratch/appendix_c.py**.

Table D.1 Qwen3 configurations (larger than 0.6B)

Model size	Configuration Python dictionary
1.7B	QWEN3_CONFIG_1_7B
4B	QWEN3_CONFIG_4B
8B	QWEN3_CONFIG_8B
14B	QWEN3_CONFIG_14B
32B	QWEN3_CONFIG_32B

The models listed in table D.1 are the dense Qwen3 variants, similar to the Qwen3 0.6B model used in the book and discussed in appendix C. Here, "dense" means these are not the sparse mixture-of-experts (MoE) variants of Qwen3. MoE models are not supported by this book's code, but a from-scratch implementation is available here: **https://github.com/rasbt/LLMs-from-scratch/tree/main/ch05/11_qwen3**.

Figure D.1 summarizes the differences between the models in table D.1 more visually. As you can see, the different Qwen3 size variants all use the same overall architecture pattern as the 0.6B model from appendix C. They still use token embeddings, grouped-query attention, rotary position embeddings, feedforward layers, and RMS normalization. The differences involve the model dimensions, such as the embedding size, the number of layers, the number of attention heads, and the feedforward hidden dimension.

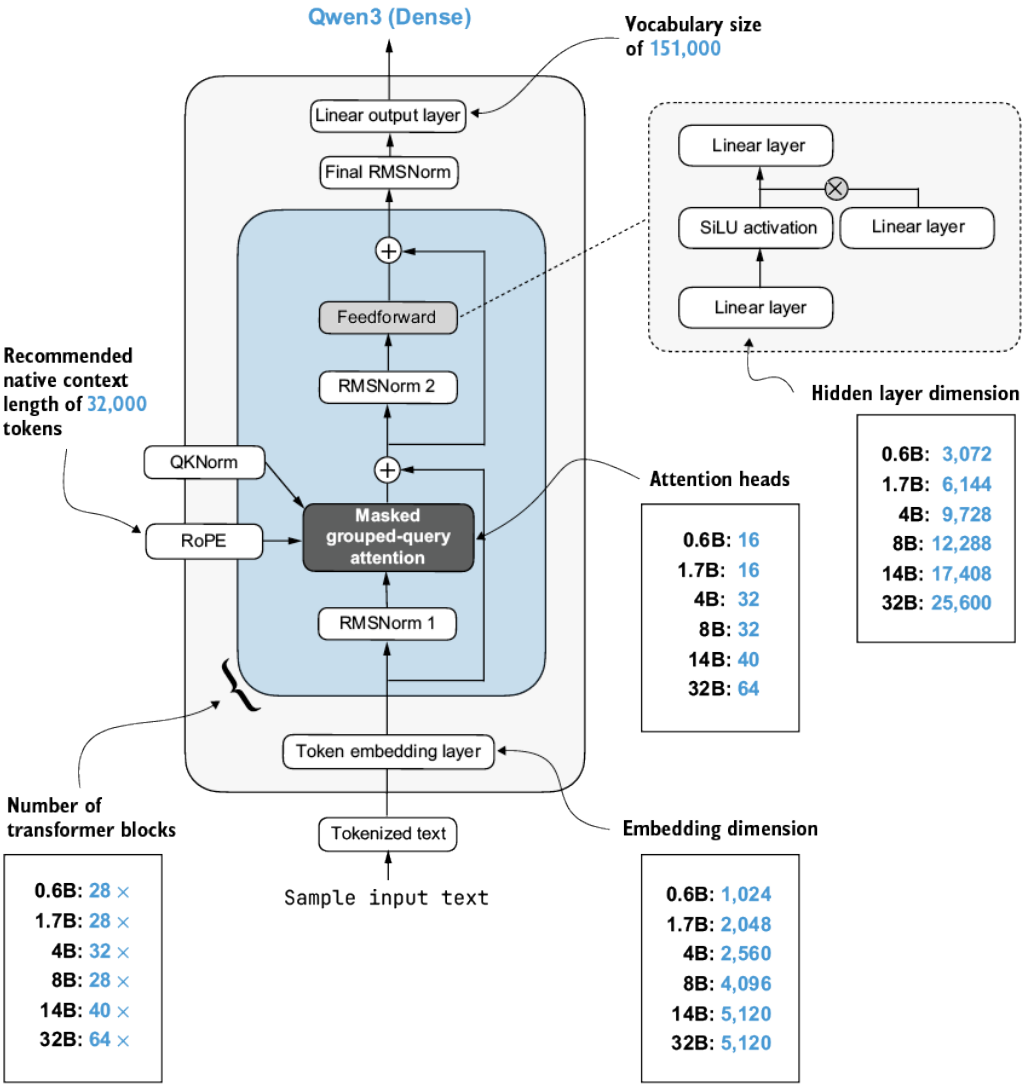


Figure D.1 Comparison of different Qwen3 variants, from 0.6 billion to 32 billion parameters

Table D.2 lists the estimated memory requirements for loading the models into RAM (and GPU RAM). As a rough lower bound, storing weights in bfloat16 precision (the common default for LLMs) requires about 2 bytes per parameter.

Table D.2 Qwen3 model sizes

Model size	Approximate memory for weights in bfloat16
1.7B	About 3.4 GB
4B	About 8 GB
8B	About 16 GB
14B	About 28 GB
32B	About 64 GB

The numbers in table D.2 are only approximate lower bounds for the weights themselves. Actual runtime memory use is higher because we also need memory for activations, temporary buffers, the KV cache, and so on. In addition, model loading can temporarily increase memory use because tensors may exist in more than one place while they are being copied into the `Qwen3Model` instance. See the supplementary materials for my *Build A Large Language Model (From Scratch)* book for more details: **https://github.-com/rasbt/LLMs-from-scratch/tree/main/ch05/08_memory_efficient_weight_loading**.

D.2 Downloading larger checkpoints overview

Unlike the 0.6B checkpoints used in this book, which I converted into PyTorch checkpoints to minimize

external code dependencies, larger official Qwen3 models are typically distributed as `safetensors` files, sometimes split across multiple shards.

The `reasoning-from-scratch` Python library provides a helper function, `download_from_huggingface_from_snapshots`, to handle this. This function, which we'll use in the next section, downloads the Hugging Face snapshot into a local directory and then loads either a single `model.safetensors` file or multiple shards referenced via a `model.safetensors.index.json` file.

Because this helper function relies on additional packages that were not required in the earlier chapters, you may need to install them first:

```
uv add huggingface_hub safetensors
```

or

```
pip install huggingface_hub safetensors
```

Once these packages are installed, loading a larger checkpoint follows the same broad pattern as before. You'll need to download the files, instantiate the model, load the weights, prepare the tokenizer, and then run the text generation, as you'll see in the next section.

D.3 Loading a larger base model

We will now walk through a complete example using the official Qwen3 4B base model. The first step, shown in listing D.1, is to download the model snapshot from the Hugging Face Model hub (**https://huggingface.co/Qwen/Qwen3-4B-Base**).

Listing D.1 Downloading model weights

```
from pathlib import Path
from reasoning_from_scratch.ch02 import get_device
from reasoning_from_scratch.appendix_c import (
    download_from_huggingface_from_snapshots,
)


device = get_device()
local_dir = Path("qwen3-4b-base")

weights = download_from_huggingface_from_snapshots(
    repo_id="Qwen/Qwen3-4B-Base",
    local_dir=local_dir,
)
```

The `local_dir` path specifies where the snapshot should be stored on disk. In this example, we keep the 4B base model in a dedicated `qwen3-4b-base` folder so that it stays separate from the smaller `qwen3` directory used throughout the book.

Next, we use the code in listing D.2 to instantiate the 4B model and load the Hugging Face weights into the from-scratch `Qwen3Model` implementation.

Listing D.2 Loading weights into `Qwen3Model`

```
from reasoning_from_scratch.qwen3 import (
    Qwen3Model,
    load_hf_weights_into_qwen,
)
from reasoning_from_scratch.appendix_c import QWEN3_CONFIG_4B


model = Qwen3Model(QWEN3_CONFIG_4B)
load_hf_weights_into_qwen(
    model,
    param_config={
        "n_layers": QWEN3_CONFIG_4B["n_layers"],
        "hidden_dim": QWEN3_CONFIG_4B["hidden_dim"],
    },
    params=weights,
)
model.to(device)
model.eval()
```

The `QWEN3_CONFIG_4B` dictionary defines the architecture dimensions for the 4B model. The `load_hf_weights_into_qwen` helper then maps the Hugging Face parameter names into the parameter names used by our own implementation. After that, `model.to(device)` moves the model to the selected device, and `model.eval()` switches the model into inference mode.

After the model is loaded, we prepare the tokenizer.

Listing D.3 Loading the base tokenizer

```
from reasoning_from_scratch.qwen3 import Qwen3Tokenizer
import shutil

tokenizer_src = local_dir / "tokenizer.json"
tokenizer_path = local_dir / "tokenizer-base.json"

if not tokenizer_path.exists():
    shutil.copyfile(tokenizer_src, tokenizer_path)

tokenizer = Qwen3Tokenizer(tokenizer_file_path=tokenizer_path)
```

The Hugging Face snapshot stores the tokenizer under the generic name `tokenizer.json`. In listing D.3, we copy it to `tokenizer-base.json` so that it is easy to distinguish from the reasoning tokenizer that we'll use in the next section.

We can now use the model for generation exactly as we did with the smaller 0.6B model in chapter 2.

Listing D.4 Generating text

```
import torch
from reasoning_from_scratch.ch02 import (
    generate_text_basic_stream_cache,
)

prompt = "Explain large language models in two sentences."
input_ids = torch.tensor(
    tokenizer.encode(prompt),
    device=device,
).unsqueeze(0)

for token in generate_text_basic_stream_cache(
    model=model,
    token_ids=input_ids,
    max_new_tokens=64,
    eos_token_id=tokenizer.eos_token_id,
):
    print(tokenizer.decode(token.squeeze(0).tolist()), end="", flush=True)
```

As in chapter 2, we encode the input prompt, add a batch dimension via `unsqueeze(0)`, and then stream the generated tokens one by one.

The output is as follows:

```
 Large language models are artificial intelligence systems that use deep learning
techniques to understand and generate human-like text. They are trained on vast
amounts of data and can perform a wide range of natural language processing tasks,
such as translation, summarization, and question answering.
```

This is a nice property of reusing the same `Qwen3Model` abstraction across model sizes. Once the checkpoint and tokenizer are set up correctly, the inference code looks essentially the same.

D.4 Loading a larger reasoning variant

The same idea also works for larger reasoning-style Qwen3 models. The architecture for a given model size stays the same. Compared to the previous section, only the checkpoint and tokenizer settings change.

For example, to load the 4B reasoning variant instead of the 4B base variant, we would

- Switch the repository ID from `Qwen/Qwen3-4B-Base` to `Qwen/Qwen3-4B`
- Copy or rename the `tokenizer.json` file to `tokenizer-reasoning.json`
- Initialize the tokenizer using the code in listing D.5

Listing D.5 Loading the reasoning tokenizer

```
tokenizer = Qwen3Tokenizer(
    tokenizer_file_path=tokenizer_path,
    apply_chat_template=True,
    add_generation_prompt=True,
    add_thinking=True,
)
```

These three tokenizer settings match the chat-style prompt formatting used by the reasoning models, as discussed in chapters 2 and 8. In other words, we still use the same 4B architecture configuration (`QWEN3_CONFIG_4B`), but we pair it with the reasoning checkpoint and the reasoning-style tokenizer behavior.

The rest of the model-loading and model-use code stays the same. We still download the snapshot, instantiate `Qwen3Model(QWEN3_CONFIG_4B)`, load the weights via `load_hf_weights_into_qwen`, move the

model to the target device, and generate text using the same streaming function from chapter 2.

D.5 Practical recommendations

The main point of this appendix is that the same from-scratch model code from appendix C can be reused across larger official Qwen3 checkpoints, provided that you select the matching configuration dictionary and tokenizer setup. This means moving to a larger model is mostly a matter of loading a different checkpoint and configuration rather than learning an entirely new concept.

If you want to experiment beyond the 4B example shown here, the 1.7B and 8B variants are natural next steps. The 1.7B model is a modest step up from 0.6B if 4B is too resource intensive for your hardware. The 8B model is substantially larger and may produce stronger outputs if your hardware can handle it.

Previous chapter
< appendix C Qwen3 LLM source code
Next chapter
appendix E Batching and throughput-oriented e >

appendix E Batching and throughput-oriented execution

In the code implemented through the book's chapters, we usually processed one prompt or example at a time. This kept the code compact and easier to understand, and it also helped keep the resource requirements more manageable. The code in this book is already expensive to run, so adding batching support everywhere would often increase complexity without providing much real benefit, depending on your available hardware.

That being said, if you have access to relatively modern GPUs with enough memory, batched execution, illustrated in figure E.1, can be very useful. For example, if we want to evaluate many problems, sample several responses per prompt, or train on multiple examples at once, batching can help increase throughput and reduce the overall time required.

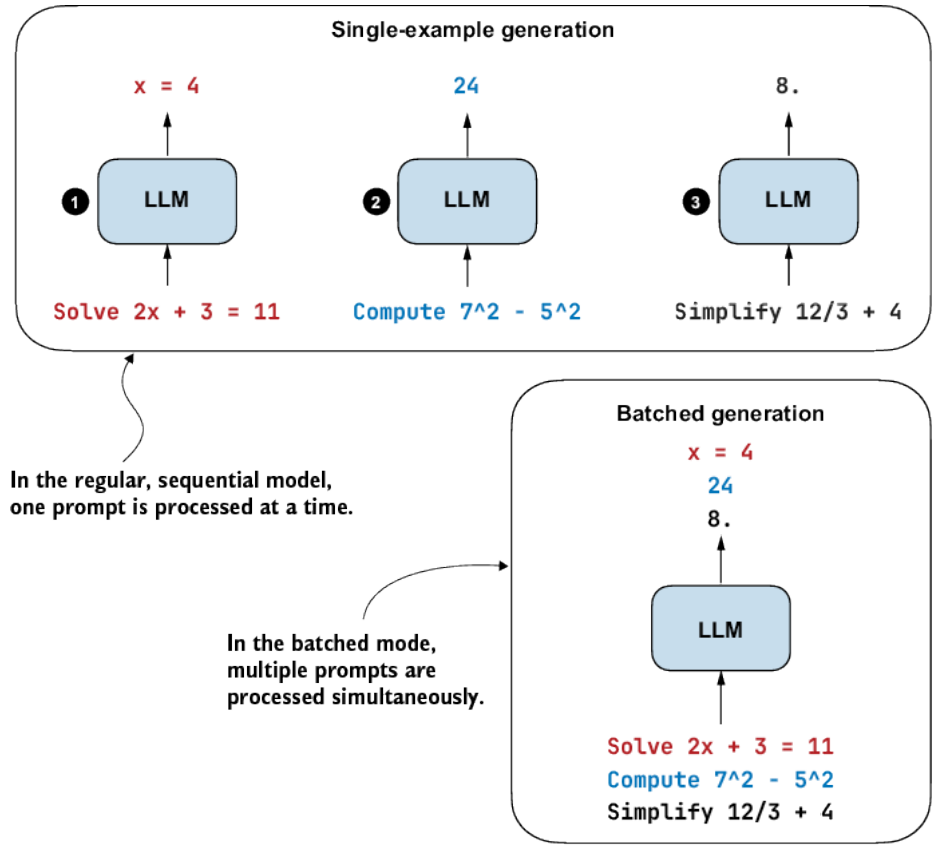


Figure E.1 An overview of single-example versus batched generation. In batched generation, several prompts are packed into one batch, processed in parallel, and then decoded together to improve throughput.

This appendix explains the broad idea behind batching and shows how you can use the batched implementations in the supplementary materials across various chapters.

E.1 Why batching helps

When we talk about computational performance, it helps to separate the following overall goals:

- Latency: how quickly we get the answer for one prompt
- Throughput: how many prompts we can process in a fixed amount of time

Single-example generation is often best used for understanding the conceptual implementation of a concept, minimizing latency, and debugging. Batching can be seenas an extension of single-example generation that primarily targets throughput. For example, if we want to evaluate hundreds of problems on

MATH-500, generate several self-consistency samples, or train on a larger distillation dataset, batching can reduce total runtime substantially on suitable hardware.

But batching is not automatically faster on every device. Small models on CPUs or less optimized GPUs may benefit only a little, or not at all, because they introduce additional code complexity, including padding and other overheads, that can offset the gains from batching parallelism.

Batching is best understood as a throughput optimization, not as a universal speedup.

E.2 Running batched generation

The main technical obstacle in batching is that prompts usually have different lengths. One math problem may tokenize to 40 tokens while another may tokenize to 120 tokens, but PyTorch tensors still have to be rectangular. So if we want to run several prompts together, we need a way to pad shorter rows and keep track of which positions are real tokens and which are just padding tokens.

Padding tokens are placeholder tokens that are added to a prompt to make it a certain length. For example, to extend a three-token prompt `" 3 + 3 "` to a five-token prompt, we may add two additional padding tokens to the right or the left: `" <pad><pad> 3 + 3 "`. (In practice, it doesn't matter which token we use as a padding token; it's common to use an end-of-sequence token like `<eos>` or `<|endoftext|>`.)

Conceptually, padding and keeping track of padded positions make batched generation more complicated than single-prompt generation. In the book, we used the `Qwen3Model` class from `reasoning_from_scratch.qwen3`, which is the plain single-example implementation explained in appendix C.

For batched generation, the book's supplementary materials therefore include a separate `Qwen3Model` class in `reasoning_from_scratch.qwen3_batched`. It can be used as a drop-in replacement, and it adds the bookkeeping needed for padding-aware execution in batching.

Let's look at a single-sequence example that looks very similar to the code we used earlier in book. In the following listing, we apply it to two very small prompts, but we still run them one after the other.

Listing E.1 Single-example generation

```
import torch

from reasoning_from_scratch.ch02 import (
    get_device,
    generate_text_basic_stream_cache,
)
from reasoning_from_scratch.ch03 import (
    load_model_and_tokenizer,
    render_prompt,
)

device = get_device()
model, tokenizer = load_model_and_tokenizer(
    which_model="base",
    device=device,
    use_compile=False,
)

for problem in ["2+2?", "3+3=6?"]:  #1
    prompt = render_prompt(problem)
    input_ids = torch.tensor(
        tokenizer.encode(prompt),
        dtype=torch.long,
        device=device,
    ).unsqueeze(0)

    for token in generate_text_basic_stream_cache(
        model=model,
        token_ids=input_ids,
        max_new_tokens=32,
        eos_token_id=tokenizer.eos_token_id,
    ):
        next_token_id = token.squeeze(0)
        print(tokenizer.decode(next_token_id.tolist()), end="", flush=True)

    print()  #2
```

#1 The two prompts we sequentially process

#2 Force a new line after each answer.

This is the resulting output:

```
 \boxed{4}
 \boxed{6}
```

The code in listing E.1 is still ordinary single-example generation. We render the prompt, tokenize it, add a batch dimension of size 1, and then stream the generated tokens as they arrive. This is conceptually simple, which is why we used this style in the book.

Listing E.2 shows the corresponding batched version. The overall workflow is similar, but now we switch to `reasoning_from_scratch.qwen3_batched`, tokenize both prompts up front, left-pad them to the same length, and generate them together.

Listing E.2 Batched generation

```
from reasoning_from_scratch.qwen3_batched import (
    generate_text_basic_batched_cache,
    load_model_and_tokenizer,
)

model, tokenizer = load_model_and_tokenizer(
    which_model="base",
    device=device,
    use_compile=False,
)

problems = ["2+2?", "3+3=6?"]
prompts = [render_prompt(problem) for problem in problems]
tokenized = [tokenizer.encode(p) for p in prompts]
pad_id = tokenizer.pad_token_id
max_len = max(len(t) for t in tokenized)

left_padded = [  #1
    [pad_id] * (max_len - len(t)) + t
    for t in tokenized
]
input_ids = torch.tensor(left_padded, dtype=torch.long, device=device)

generated = generate_text_basic_batched_cache(
    model=model,
    token_ids=input_ids,
    max_new_tokens=32,
    eos_token_id=tokenizer.eos_token_id,
    pad_id=pad_id,  #2
)

for row in generated:
    eos_pos = (row == tokenizer.eos_token_id).nonzero(as_tuple=True)[0]
    if len(eos_pos) > 0:
        row = row[:eos_pos[0]]
    print(tokenizer.decode(row.tolist()))
```

#1 Left-pad shorter sequences so all prompts in the batch have the same length.

#2 Needed so generation can distinguish real tokens from padding

The results are the same as before, but this time they are produced in parallel:

```
 \boxed{4}
 \boxed{6}
```

In this example, we used the nonstreaming batched helper, so we need to wait until the whole batch is finished before decoding and printing the results. That keeps the example simple and makes it easier to see the role of the padding before we dig into the internal masking logic.

There is also a more optimized variant of the text generation function, `generate_text_basic_batched_cache_stop`, which removes finished rows from the active compute batch instead of carrying them along until the longest row finishes. In contrast, the `generate_text_basic_batched_cache` we used in listing E.2 keeps every row in the active batch for every decode step.

In `generate_text_basic_batched_cache`, once a row has emitted EOS, the implementation forces that row to keep producing EOS so that the batch shape stays aligned, even though the row is already finished from the perspective of the final decoded output. The difference between these two functions is illustrated in figure E.2. Note that Qwen3 uses `<|endoftext|>` as the EOS token for the base model, but figure E.2 labels it simply as `<eos>` for visual compactness.

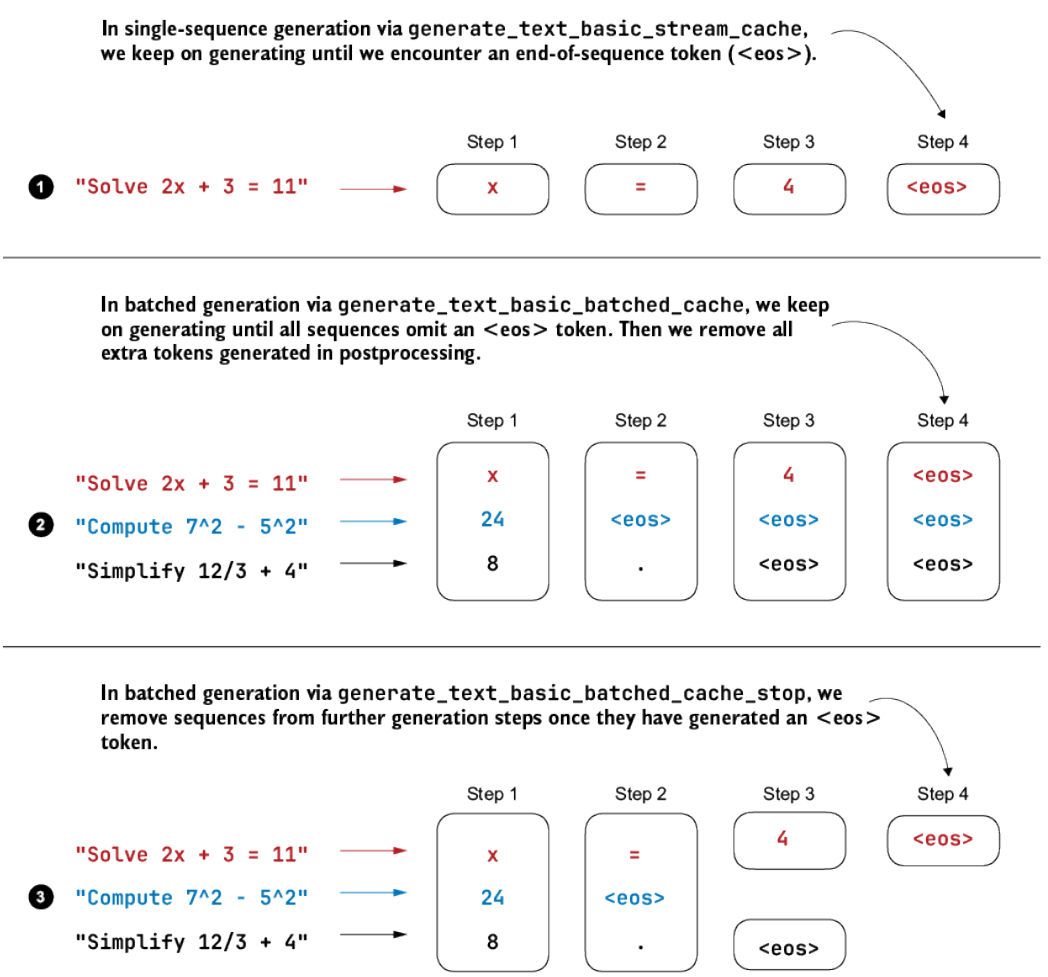


Figure E.2 Sequential generation (1) versus regular batched generation (2) and early-stop batched generation (3). The regular batched helper (2) keeps finished rows in the batch as placeholder EOS rows, whereas the `_stop` variant (3) removes finished rows from the active compute batch as soon as they emit EOS.

The next section goes into a bit more detail about how the padding is handled internally.

E.3 Padding and attention masks

In single-example mode, if we tokenize a short prompt such as `" 2+2? "`, we can pass it to the model as a simple tensor of shape `(1, 4)`:

```
input_ids = torch.tensor([[17, 10, 17, 30]])
```

Internally, the model builds a standard *causal attention mask* so that each position can attend only to itself and earlier tokens. If you are unfamiliar with causal attention masks and self-attention, my article titled "Understanding and Coding Self-Attention, Multi-Head Attention, Causal-Attention, and Cross-Attention in LLMs," provides more background information: **https://magazine.sebastianraschka.com/p/understanding-and-coding-self-attention**.

The causal attention mask for the `" 2+2? "` prompt is illustrated in figure E.3. In the causal mask, the value `1` means "masked out" and `0` means "allowed." In this figure, the first token cannot look ahead to later positions, the second token can look only at the first two positions, and so on. This is the standard autoregressive masking pattern.

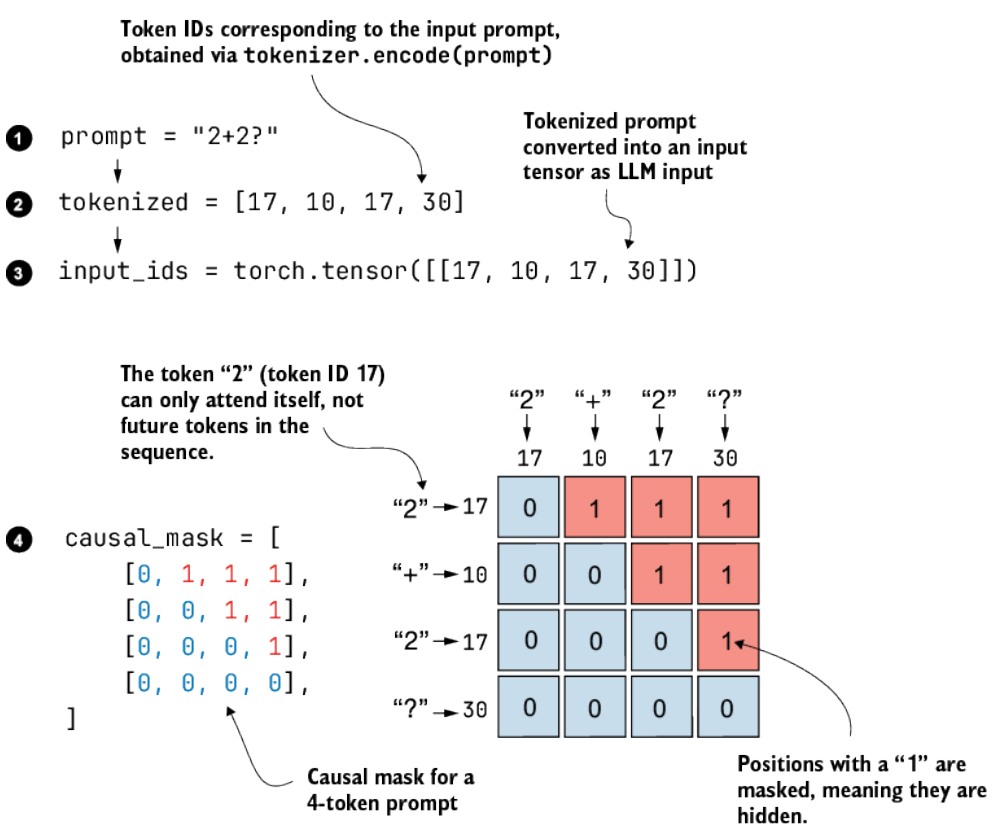


Figure E.3 The causal mask used in single-example generation. Tokens can attend only to themselves and earlier positions, which is the standard autoregressive setup.

Batching changes the situation because different prompts usually have different lengths. Suppose we process `" 2+2? "` together with the slightly longer prompt, `" 3+3=6? "`.

Since PyTorch tensors must be rectangular, the shorter row has to be padded to match the longer one, as illustrated in figure E.4 with left padding.

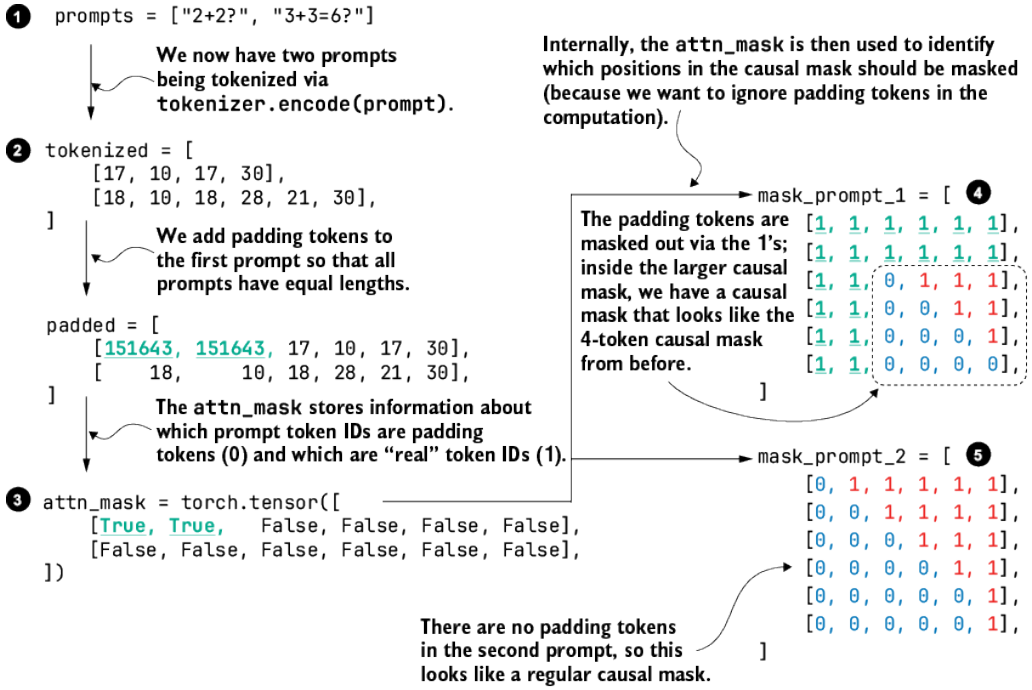


Figure E.4 Left padding in batched generation. The shorter row is padded up to the maximum sequence length in the batch, and the padding-aware mask ensures that these padded positions do not affect attention.

Figure E.4 shows that we keep an additional `attn_mask` internally. This mask is used to keep track of the padded positions, where `True` means padded and `False` means not padded. We use this additional `attn_mask` to identify the tokens in the causal mask that correspond to pad token IDs. Masking padded keys and zeroing padded queries are important steps to make batching behave similarly to the single-example execution.

In the remainder of this appendix, you will see how to use scripts from the book's supplementary materi-

als that implement the optional batched variants for different chapters.

E.4 Chapter 3: Batched MATH-500 evaluation

The book's supplementary materials include a script for the single-example evaluation method from chapter 3 that you can download and run directly using the download function we introduced in chapter 7.

Listing E.3 Downloading the single-example evaluation script

```
from reasoning_from_scratch.ch07 import download_from_github

download_from_github(
    "ch03/02_math500-verifier-scripts/evaluate_math500.py"
)
download_from_github(
    "ch03/01_main-chapter-code/math500_test.json",
    out="math500_test.json",
)
```

After downloading the script and dataset via listing E.3, we can run the single-example evaluation from the terminal as follows (if you are not using `uv`, simply replace `uv run` with `python`):

```
uv run evaluate_math500.py \
  --dataset_size 500 \
  --which_model "reasoning"
```

The supplementary material also includes a batched version that applies the batching method we discussed previously. The download is almost identical, except that we replace `evaluate_math500.py` with `evaluate_math500_batched.py`.

Listing E.4 Downloading the evaluation script with batch support

```
download_from_github(
    "ch03/02_math500-verifier-scripts/evaluate_math500_batched.py"
)
```

The use of the batched script is very similar to the nonbatched version, except that we now provide an additional `--batch_size` argument to specify how many prompts and answers the model should process in parallel:

```
uv run evaluate_math500_batched.py \
  --dataset_size 500 \
  --which_model "reasoning" \
  --batch_size 64
```

The ideal batch size depends entirely on what your hardware can handle, so it's best to start with a smaller value and scale upward until you hit the throughput and memory limit that makes sense for your setup.

We will return to a comparison of single-example and batched generation performance at the end of this appendix.

E.5 Chapter 4: Batched self-consistency sampling

The optional `self_consistency_math500_batched.py` script for chapter 4 does not mix different prompts into one padded tensor. Instead, it repeats the same prompt `num_samples` times and samples several continuations in parallel for self-consistency voting, because parallelizing over the different numbers of samples for each prompt is a natural angle we can take advantage of.

Because every row starts from the same prompt length, this script can use the regular `Qwen3Model` from `reasoning_from_scratch.qwen3` instead of `reasoning_from_scratch.qwen3_batched`. In other words, the batch dimension here is used for multiple sampled rollouts of the same prompt, not for padding together unrelated prompts of different lengths.

You can download the script as shown next.

Listing E.5 Downloading self-consistency sampling with batch support

```
download_from_github(
    "ch04/02_math500-inference-scaling-scripts/"
    "self_consistency_math500_batched.py"
)
```

To download the nonbatched version instead, simply remove the `"_batched"` part from the filename in listing E.5.

We can run the batched script as follows, and the syntax for the nonbatched script is otherwise the same:

```
uv run self_consistency_math500_batched.py \
  --which_model base \
  --temperature 0.9 \
  --top_p 0.9 \
  --num_samples 3 \
  --dataset_size 500 \
  --prompt_suffix "\n\nExplain step by step."
```

E.6 Chapter 6: Batched GRPO rollouts

Self-refinement in chapter 5 is fundamentally sequential and therefore does not benefit much from simple batching. In principle, you could run self-refinement loops for multiple inputs in parallel, but that is nontrivial to implement and is therefore not part of the book's supplementary material. Instead, the next batched example appears in the chapter 6 RLVR code.

In chapter 6, we again use the same prompt for multiple rollouts, so no padding is required. As in section E.5, the code can therefore stay with the regular `Qwen3Model` class from `reasoning_from_scratch.qwen3`. The relevant scripts can be fetched as follows.

Listing E.6 Downloading the RLVR script with batch support

```
download_from_github(
    "ch06/02_rlvr_grpo_scripts_intro/"
    "rlvr_grpo_original_no_kl_batched.py"
)
```

As before, to download the single-generation script, simply drop `"_batched"` from the filename in listing E.6.

A typical batched run looks like this:

```
uv run rlvr_grpo_original_no_kl_batched.py \
  --num_rollouts 8 \
  --batch_size 4 \
  --max_new_tokens 512
```

Here, `--batch_size` controls how many rollouts are generated in parallel within one training step. This improves throughput, but it also increases memory pressure, so you may need to reduce `--num_rollouts` or `--max_new_tokens`.

E.7 Chapter 8: Batched distillation

Chapter 8 returns to the padding-aware style from chapter 3 because the distillation examples have different prompt and answer lengths. In other words, we are again batching unrelated examples of different lengths into a single rectangular tensor, which means that masking and padding matter again. You can download the batched training script and fetch a sample training dataset as follows.

Listing E.7 Downloading the distillation script with batch support

```
from reasoning_from_scratch.ch08 import load_distill_data

download_from_github(
    "ch08/04_train_with_distillation/distill_batched.py"
)
_ = load_distill_data(
    partition="deepseek-r1-math-train",
    local_path="deepseek-r1-math-train.json",
)
```

For the nonbatched version, remove the `" _batched "` part from the filename in listing E.7. A representative run of the batched version looks like this:

```
uv run distill_batched.py \
  --data_path deepseek-r1-math-train.json \
  --dataset_size 500 \
  --validation_size 10 \
  --epochs 2 \
  --use_think_tokens \
  --batch_size 32
```

Here, `--batch_size` lets us process multiple distillation examples per optimization step. But because training also has to store activations, the resource demands can be much higher than in the chapter 3 evaluator, so chapter 8 introduces the training analogue of the padding-aware batching strategy discussed earlier in this appendix.

E.8 Single-sequence vs. batch generation

The supplementary scripts in chapters 3 and 8 batch together different-length examples, so they need padding-aware attention masks and more careful bookkeeping. Chapters 4 and 6 batch repeated copies of the same prompt or rollout setup, so they can stay with the regular model implementation and avoid the more complicated padding logic.

Either way, batched generation can result in higher throughput and shorter runtimes, at the cost of increased RAM use. Table 8.1 compares single to batched generation using the settings shown in the previous sections.

Table E.1 RAM use and runtime comparison

	Script	Batch size	RAM	H100 total time	DGX Spark total time
1	`evaluate_math500.py`	-	1.8 GB	90 min	174 min
2	`evaluate_math500_batched.py`	64	23.39 GB	16 min	108 min
3	`self_consistency_math500.py`	-	1.79 GB	252 min	340 min
4	`self_consistency_math500_batched.py`	3	2.45 GB	129 min	243 min
5	`rlvr_grpo_original_no_kl.py`	-	43.35 GB	68 min	63 min
6	`rlvr_grpo_original_no_kl_batched.py`	4	44.91 GB	19 min	23 min
7	`distill.py`	-	8.29 GB	10 min	32 min
8	`distill_batched.py`	4	8.34 GB	9 min	28 min

As shown in table E.1, the batched variants are always faster than the single-sequence variants. In most cases, the difference is very noticeable. One exception is distillation (rows 7 and 8), where batching only yields a modest advantage. This could be because the GPU is already well utilized in single-batch mode, and the extra overhead of batched generation from the additional mask generation is not worth it, given the small size of the Qwen3 6-billion-parameter model.

appendix F Common approaches to model evaluation

F.1 Understanding the main evaluation methods for LLMs

There are four common ways of evaluating trained LLMs in practice: *multiple choice*, *verifiers*, *leaderboards*, and *LLM judges*, as shown in figure F.1. Research papers, marketing materials, technical reports, and model cards (a term for LLM-specific technical reports) often include results from two or more of these categories.

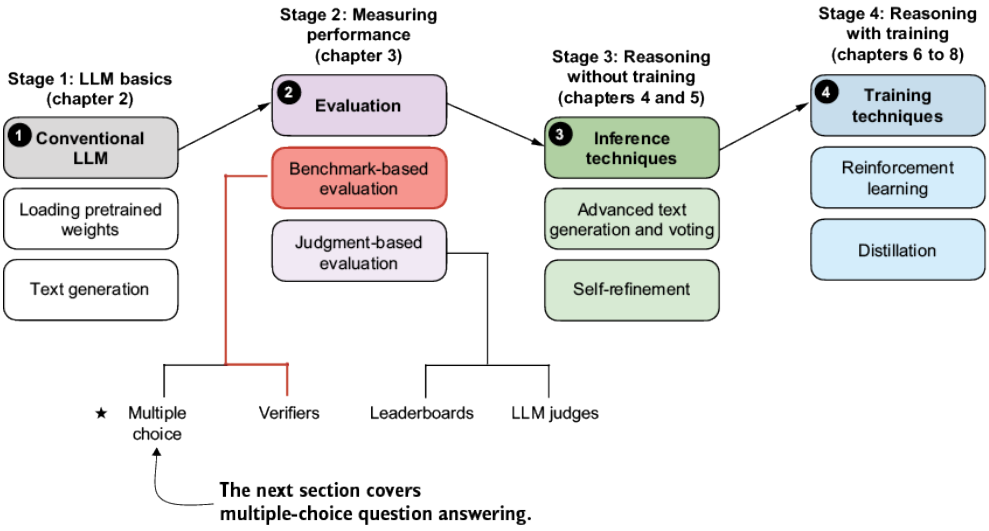


Figure F.1 A model of the topics covered in this book, with a focus on the two broad evaluation categories covered in this appendix: benchmark-based evaluation and judgment-based evaluation

As you can see in figure F.1, the four evaluation categories introduced here fall into two groups: *benchmark-based evaluation* and *judgment-based evaluation*. Other measures, such as *training loss*, *perplexity*, and *rewards*, are typically used internally during model development. They are covered in the model-training chapters (chapters 6 to 8).

F.2 Evaluating answer-choice accuracy

Historically, one of the most widely used evaluation methods has been multiple-choice benchmarks such as *MMLU* (short for Massive Multitask Language Understanding, **https://huggingface.co/datasets/cais/mmlu**).

Figure F.2 shows a single example from the MMLU dataset. The complete MMLU dataset consists of 57 subjects (from high school math to biology) and about 16 thousand multiple-choice questions in total. Performance is measured in terms of accuracy (the fraction of correctly answered questions); for example, the score would be 87.5% if 14,000 out of 16,000 questions are answered correctly.

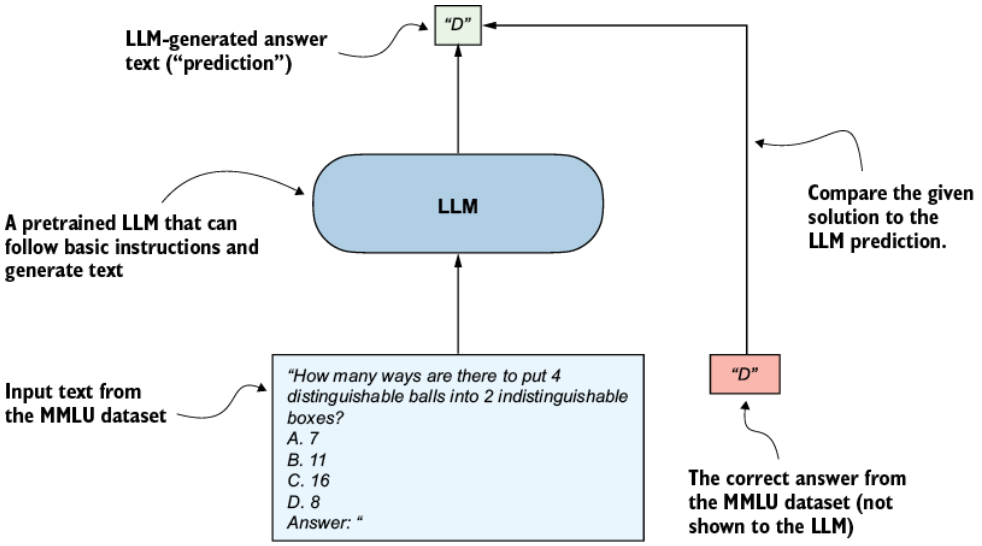


Figure F.2 Evaluating an LLM on MMLU by comparing its multiple-choice prediction with the correct answer from the dataset

Multiple-choice benchmarks, such as MMLU, test an LLM's knowledge recall in a straightforward, quantifiable way similar to standardized tests, many school exams, or theoretical driving tests.

Note that figure F.2 shows a simplified version of multiple-choice evaluation, where the model's predicted answer letter is compared directly to the correct one. Other popular methods use log-probability scoring (log probabilities are discussed in more detail in chapter 4).

The following subsections illustrate how the MMLU scoring shown in figure F.2 can be implemented in code. End-to-end MMLU scripts, including the different scoring variants, will be provided as bonus materials in this book's code repository.

F.2.1 Loading the model

Before we can evaluate the model on MMLU, we have to load the pretrained model. The following code is identical to listing 3.1.

Listing F.1 Loading a pretrained model

```
from pathlib import Path
import torch
from reasoning_from_scratch.ch02 import get_device
from reasoning_from_scratch.qwen3 import (
    download_qwen3_small, Qwen3Tokenizer,
    Qwen3Model, QWEN_CONFIG_06_B
)

device = get_device()
torch.set_float32_matmul_precision("high")  #1

# device = "cpu"  #2

WHICH_MODEL = "base"  #3

if WHICH_MODEL == "base":
    download_qwen3_small(
        kind="base", tokenizer_only=False, out_dir="qwen3"
    )
    tokenizer_path = Path("qwen3") / "tokenizer-base.json"
    model_path = Path("qwen3") / "qwen3-0.6B-base.pth"
    tokenizer = Qwen3Tokenizer(tokenizer_file_path=tokenizer_path)

elif WHICH_MODEL == "reasoning":
    download_qwen3_small(
        kind="reasoning", tokenizer_only=False, out_dir="qwen3"
    )
    tokenizer_path = Path("qwen3") / "tokenizer-reasoning.json"
    model_path = Path("qwen3") / "qwen3-0.6B-reasoning.pth"
    tokenizer = Qwen3Tokenizer(
        tokenizer_file_path=tokenizer_path,
        apply_chat_template=True,
        add_generation_prompt=True,
        add_thinking=True,
    )

else:
    raise ValueError(f"Invalid choice: WHICH_MODEL={WHICH_MODEL}")

model = Qwen3Model(QWEN_CONFIG_06_B)
model.load_state_dict(torch.load(model_path))
model.to(device)

USE_COMPILE = False  #4
if USE_COMPILE:
  torch._dynamo.config.allow_unspec_int_on_nn_module = True
  model = torch.compile(model)
```

#1 Lower precision from "highest" to enable Tensor Cores if applicable
#2 Uncomment this line if you have compatibility issues with your device.
#3 Uses the base model, similar to chapter 2, by default
#4 Optionally set to true to enable model compilation.

F.2.2 Checking the generated answer letter

Now we'll implement the simplest and perhaps most intuitive MMLU scoring method, which relies on checking whether a generated multiple-choice answer letter matches the correct answer. This is similar to what was illustrated in figure F.2.

We will work with an example from the MMLU dataset:

```
example = {
    "question": (
        "How many ways are there to put 4 distinguishable"
        " balls into 2 indistinguishable boxes?"
    ),
    "choices": ["7", "11", "16", "8"],
    "answer": "D",
}
```

Next, we'll define a function to format the LLM prompts.

Listing F.2 Formatting the LLM prompt

```
def format_prompt(example):
    return (
        f"{example['question']}\n"
        f"A. {example['choices'][0]}\n"
        f"B. {example['choices'][1]}\n"
        f"C. {example['choices'][2]}\n"
        f"D. {example['choices'][3]}\n"
        "Answer: " #1
    )
```

#1 The trailing space encourages a single-letter next token.

Let's execute the function on the MMLU example to get an idea of what the formatted LLM input looks like:

```
prompt = format_prompt(example)
print(prompt)
```

The output is as follows:

```
How many ways are there to put 4 distinguishable balls into 2
indistinguishable boxes?
A. 7
B. 11
C. 16
D. 8
Answer:
```

The preceding model prompt provides the model with a list of answer choices, and it ends with an `" Answer: "` text that encourages the model to generate the correct answer.

While this is not strictly necessary, it can sometimes be helpful to provide additional questions along with the correct answers as input so that the model can observe how it is expected to solve the task. (For example, cases where five examples are provided are known as 5-shot MMLU.) However, for current generations of LLMs, in which even the base models are quite capable, this is not required.

LOADING DIFFERENT MMLU SAMPLES

You can load examples from the MMLU dataset directly from the `datasets` library (which can be installed with `pip install datasets` or `uv add datasets`):

```
from datasets import load_dataset
configs = get_dataset_config_names("cais/mmlu")
dataset = load_dataset("cais/mmlu", "high_school_mathematics")

example = dataset["test"][0]  #1
print(example)
```

#1 Inspect the first example from the test set.

Here we used the `" high_school_mathematics "` subset. To get a list of the other subsets, use the following code:

```
from datasets import get_dataset_config_names
subsets = get_dataset_config_names("cais/mmlu")
print(subsets)
```

Next, we'll tokenize the prompt and wrap it in a PyTorch tensor object as input to the LLM (similar to what we did in chapter 2):

```
prompt_ids = tokenizer.encode(prompt)
prompt_fmt = torch.tensor(prompt_ids, device=device).unsqueeze(0)
```

Now we'll define the main scoring function, which generates a few tokens (eight by default) and extracts the first instance of the letter A, B, C, or D that the model prints.

Listing F.3 Extracting the generated letter

```
from reasoning_from_scratch.ch02 import (
    generate_text_basic_stream_cache
)

def predict_choice(
    model, tokenizer, prompt_fmt, max_new_tokens=8
):
    pred = None
    for t in generate_text_basic_stream_cache(
        model=model,
        token_ids=prompt_fmt,
        max_new_tokens=max_new_tokens,
        eos_token_id=tokenizer.eos_token_id,
    ):
        answer = tokenizer.decode(t.squeeze(0).tolist())
        for letter in answer:
            letter = letter.upper()
            if letter in "ABCD":  #1
                pred = letter
                break
        if pred:
            break
    return pred
```

#1 Stop as soon as a letter appears.

We can now check the generated letter using the function from listing F.3:

```
pred1 = predict_choice(model, tokenizer, prompt_fmt)
print(
    f"Generated letter: {pred1}\n"
    f"Correct? {pred1 == example['answer']}"
)
```

This is the result:

```
Generated letter: C
Correct? False
```

As you can see, the generated answer is incorrect (`False`) in this case.

MULTIPLE-CHOICE ANSWER FORMATS

This section implements a simplified version of multiple-choice evaluation for illustration purposes, where the model's predicted answer letter is compared directly with the correct one. In practice, more widely used variations exist, such as log-probability scoring, where we measure how likely the model considers each candidate answer rather than just checking the final letter choice. (Probability-based scoring is discussed in chapter 5.) For reasoning models, evaluation can also involve assessing the likelihood of generating the correct answer when it is provided as input.

Regardless of the variant, evaluation still amounts to checking whether the model selects correctly from the predefined answer options. Examples of these variations are included in the code repository as optional bonus material.

A limitation of multiple-choice benchmarks like MMLU is that they only measure an LLM's ability to select from predefined options and thus are not very useful for evaluating reasoning capabilities beyond checking whether, and how much, knowledge the model has forgotten compared to the base model. They do not capture free-form writing ability or real-world utility. Still, they remain a simple and useful diagnostic: a high MMLU score doesn't necessarily mean the model is strong in practical use, but a low score can highlight potential knowledge gaps.

F.3 Using verifiers to check answers

Related to the multiple-choice question answering that was discussed in the previous section, verification-based approaches quantify LLM capabilities with an accuracy metric. But unlike multiple-choice benchmarks, verification methods allow LLMs to provide a free-form answer. We then extract the relevant portion of the answer and use a verifier to compare it with the correct answer provided in the dataset, as illustrated in figure F.3.

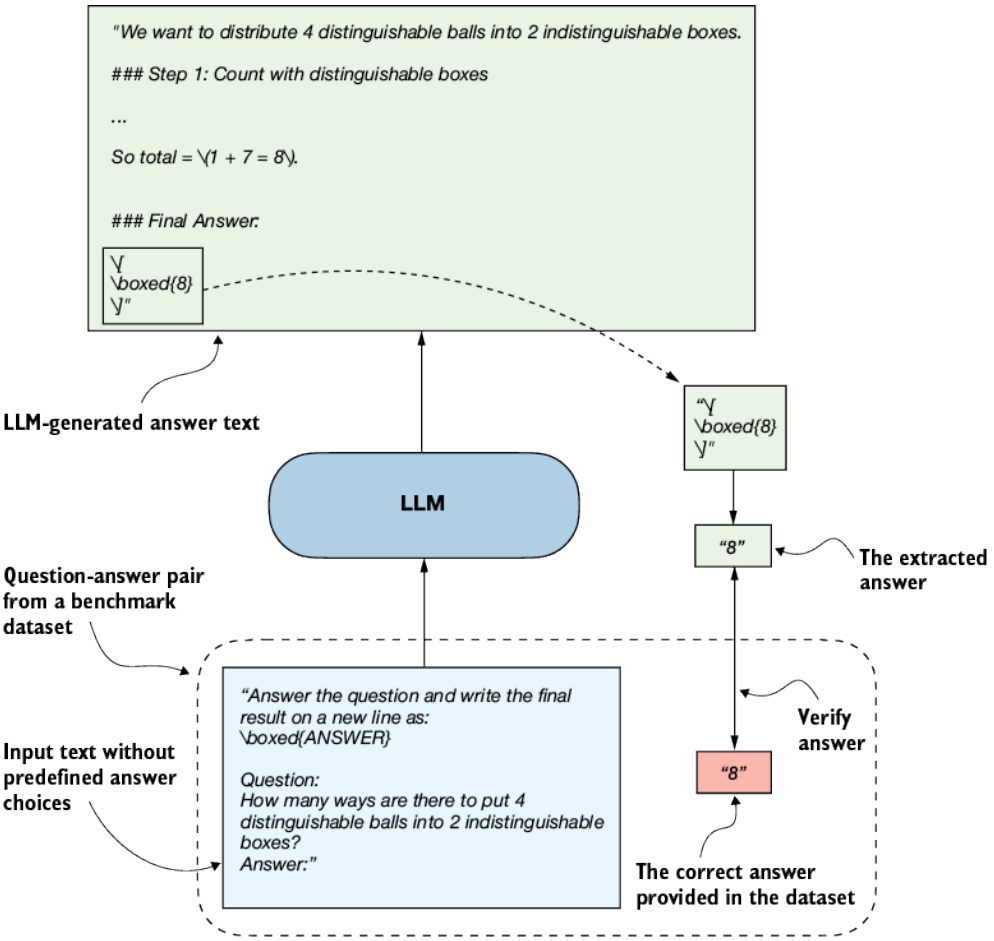


Figure F.3 Evaluating an LLM with a verification-based method in free-form question answering. The model generates a free-form answer (which may include multiple steps) and a final boxed answer, which is extracted and compared against the correct answer from the dataset.

When we compare the extracted answer with the provided answer, as shown in figure F.3, we can use external tools such as code interpreters or calculator software.

The downside is that this method can only be applied to domains that can be easily (and, ideally, deterministically) verified, such as math and code. Also, this approach can introduce additional complexity and dependencies, and it may shift part of the evaluation burden from the model itself to the external tool.

Because this method allows us to generate an unlimited number of math problem variations programmatically and it benefits from step-by-step reasoning, it has become a cornerstone of reasoning model evaluation and development. An extensive example of this method is provided in chapter 3, so we'll skip a code demonstration here.

F.4 Comparing models using preferences and leaderboards

So far, we've covered two methods that offer easily quantifiable metrics such as model accuracy. But none of the methods discussed so far evaluate LLMs in a more holistic way, including the style of the responses. In this section, as illustrated in figure F.4, we'll discuss a judgment-based method: LLM leaderboards.

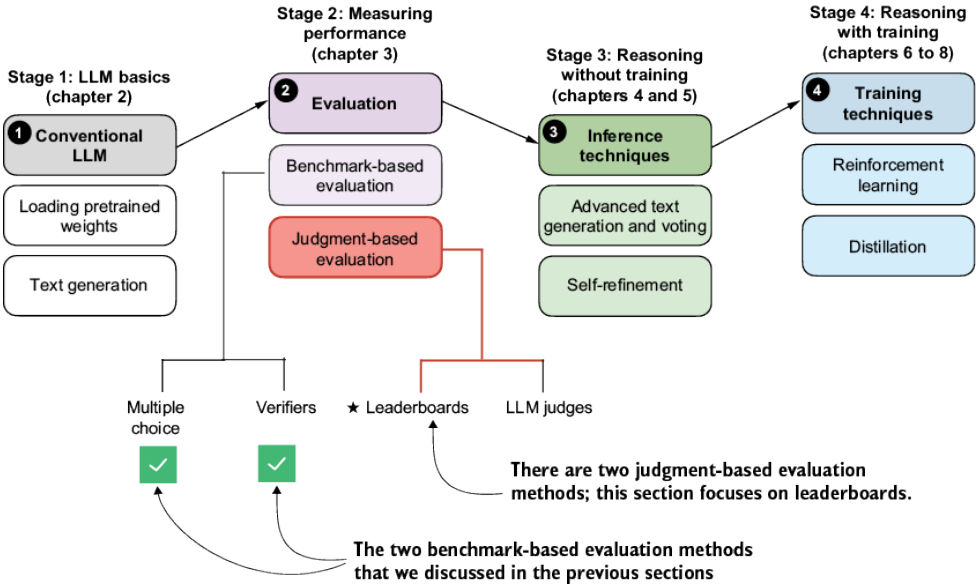


Figure F.4 A model of the topics covered in this book, with a focus on the judgment- and benchmark-based evaluation methods covered in this appendix. Having already covered benchmark-based approaches in the previous sections (multiple choice and verifiers), we'll now introduce judgment-based approaches for measuring LLM performance, with this section focusing on leaderboards.

The leaderboard method is a judgment-based approach in which models are ranked not by accuracy values or other fixed benchmark scores but by user (or another LLM's) preferences regarding their outputs. A popular leaderboard is *Arena* (formerly Chatbot Arena, **https://arena.ai/leaderboard/text**), where users compare responses from two models, either user-selected or anonymous, and vote for the one they prefer, as shown in figure F.5.

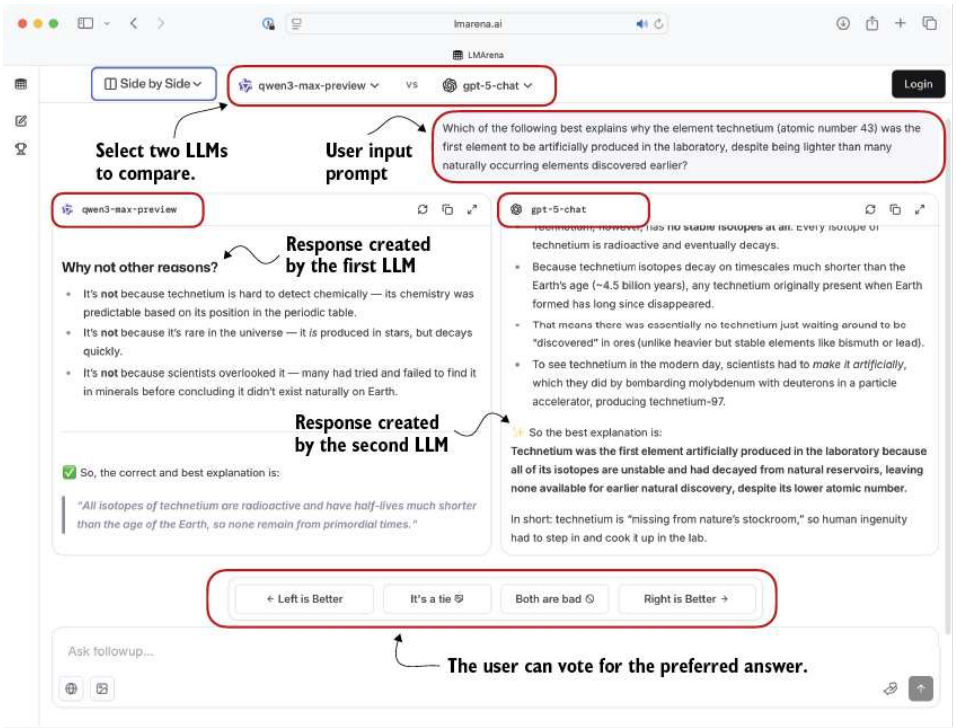


Figure F.5 Example of a judgment-based leaderboard interface (Arena). Two LLMs are given the same prompt, their responses are shown side by side, and users vote for their preferred answer.

These preference votes, which are collected as shown in figure F.5, are then aggregated across all users into a leaderboard that ranks different models by user preference. In the remainder of this section, we'll implement a simple example of a leaderboard.

Let's consider users prompting different LLMs in a setup similar to figure F.5. The following list represents pairwise votes where the first model is the winner:

```
votes = [
    ("GPT-5", "Claude-3"),
    ("GPT-5", "Llama-4"),
    ("Claude-3", "Llama-3"),
    ("Llama-4", "Llama-3"),
    ("Claude-3", "Llama-3"),
    ("GPT-5", "Llama-3"),
]
```

In this list, each tuple in the votes list represents a pairwise preference between two models, written as `(winner, loser)`. So `( " GPT-5 " , " Claude-3 " )` means that a user preferred GPT-5's answer over a Claude-3 model answer.

In the remainder of this section, we'll turn the votes list into a leaderboard. We'll use the popular Elo rating system, which was originally developed for ranking chess players. Each model will start with a baseline score. Then, after each comparison and preference vote, the model's rating will be updated. If a user prefers the current model over a highly ranked model, the current model will receive a relatively large rating update and move higher in the leaderboard. Conversely, if the current model wins against a low-ranked model, its rating increases only a little. (If the current model loses, it is updated in a similar way, but ranking points are subtracted instead of added.)

The following listing shows the code to turn these pairwise rankings into a leaderboard.

Listing F.4 Constructing a leaderboard

```
def elo_ratings(vote_pairs, k_factor=32, initial_rating=1000):
    ratings = {  #1
        model: initial_rating
        for pair in vote_pairs
        for model in pair
    }

    for winner, loser in vote_pairs:  #2


        expected_winner = 1.0 / (  #3
            1.0 + 10 ** ((ratings[loser] - ratings[winner]) / 400.0)
        )

        ratings[winner] = (  #4
            ratings[winner] + k_factor * (1 - expected_winner)
        )
        ratings[loser] = (
            ratings[loser] + k_factor * (0 - (1 - expected_winner))
        )

    return ratings
```

#1 Initialize all models with the same base rating.
#2 Update ratings after each match.
#3 Expected score for the current winner given the ratings
#4 k_factor determines sensitivity of rating updates.

The `elo_ratings` function in listing F.4 takes `votes` as input and turns it into a leaderboard, as follows:

```
ratings = elo_ratings(votes, k_factor=32, initial_rating=1000)
for model in sorted(ratings, key=ratings.get, reverse=True):
    print(f"{model:8s} : {ratings[model]:.1f}")
```

This results in the following leaderboard ranking, where the higher the score is, the better:

```
GPT-5    : 1043.7
Claude-3 : 1015.2
Llama-4  : 1000.7
Llama-3  : 940.4
```

So how does this work? For each pair, we compute the expected score of the winner using the following formula:

```
expected_winner = 1 / (1 + 10 ** ((rating_loser - rating_winner) / 400))
```

The `expected_winner` value is the model's predicted chance to win in a no-draw setting based on the current ratings. It determines how large the rating update is.

Each model starts at `initial_rating = 1000`. If the winner and loser's ratings are equal, we have `expected_winner = 0.5`, which indicates an even match. In this case, the updates are as follows:

```
rating_winner + k_factor * (1 - 0.5) = rating_winner + 16
rating_loser  + k_factor * (0 - (1 - 0.5)) = rating_loser - 16
```

If a heavy favorite (a model with a high rating) wins, we have `expected_winner ≈ 1`. The favorite gains only a small amount and the loser loses only a little:

```
rating_winner + 32 * (1 - 0.99) = rating_winner + 0.32
rating_loser  + 32 * (0 - (1 - 0.99)) = rating_loser - 0.32
```

But if an underdog (a model with a low rating) wins, we have `expected_winner ≈ 0`, and the winner gets almost the full `k_factor` points, while the loser loses about the same number of points:

```
rating_winner + 32 * (1 - 0.01) = rating_winner + 31.68
rating_loser  + 32 * (0 - (1 - 0.01)) = rating_loser - 31.68
```

ORDER MATTERS

The Elo approach updates ratings after each match (model comparisons), so later results build on ratings that have already been updated. This means the same set of outcomes, when presented in a different order, can end with slightly different final scores. This effect is usually mild, but it can happen, especially when an upset happens earlier rather than later.

To reduce this order effect, we can shuffle the `votes` pairs, run the `elo_ratings` function multiple times, and average the resulting ratings.

Leaderboard approaches such as the one described here provide a more dynamic view of model quality than static benchmark scores. But the results can be influenced by user demographics, prompt selection, and voting biases. Benchmarks and leaderboards can also be gamed, and users may select responses based on style rather than correctness. Finally, compared to automated benchmark harnesses, leaderboards do not provide instant feedback on newly developed variants, which makes them harder to use during active model development.

Arena originally used the Elo method described in this section, but it has recently transitioned to a statistical approach based on the Bradley–Terry model. The main advantage of the Bradley–Terry model is that, because it is statistically grounded, it allows for the construction of confidence intervals to express uncertainty in the rankings. In contrast to Elo ratings, the Bradley–Terry model also estimates all ratings jointly using a statistical fit over the entire dataset, which makes it immune to order effects.

To keep the reported scores in a familiar range, the Bradley–Terry model is fitted to produce values comparable to Elo. Even though the leaderboard no longer officially uses Elo ratings, the term "Elo" remains widely used by LLM researchers and practitioners when comparing models. A code example showing the Elo rating is included in this book's bonus materials at **https://github.com/rasbt/reasoning-from-scratch/tree/main/chF/03_leaderboards**.

F.5 Judging responses with other LLMs

In the early days, LLMs were evaluated using statistical and heuristics-based methods, including a measure called *BLEU*, which crudely measures how well generated text matches reference text. The problem with such metrics is that they require exact word matches and don't account for synonyms, word substitutions, and so on.

One solution to this problem, if we want to judge the written answer text as a whole, is to use relative rankings and leaderboard-based approaches, as discussed in the previous section. But a downside of leaderboards is the subjective nature of preference-based comparisons because they involve human feedback, and there are the challenges of collecting that feedback.

A related method is to use another LLM with a predefined grading *rubric* (that is, an evaluation guide) to compare an LLM's response with a reference response and judge the response quality based on that rubric, as illustrated in figure F.6.

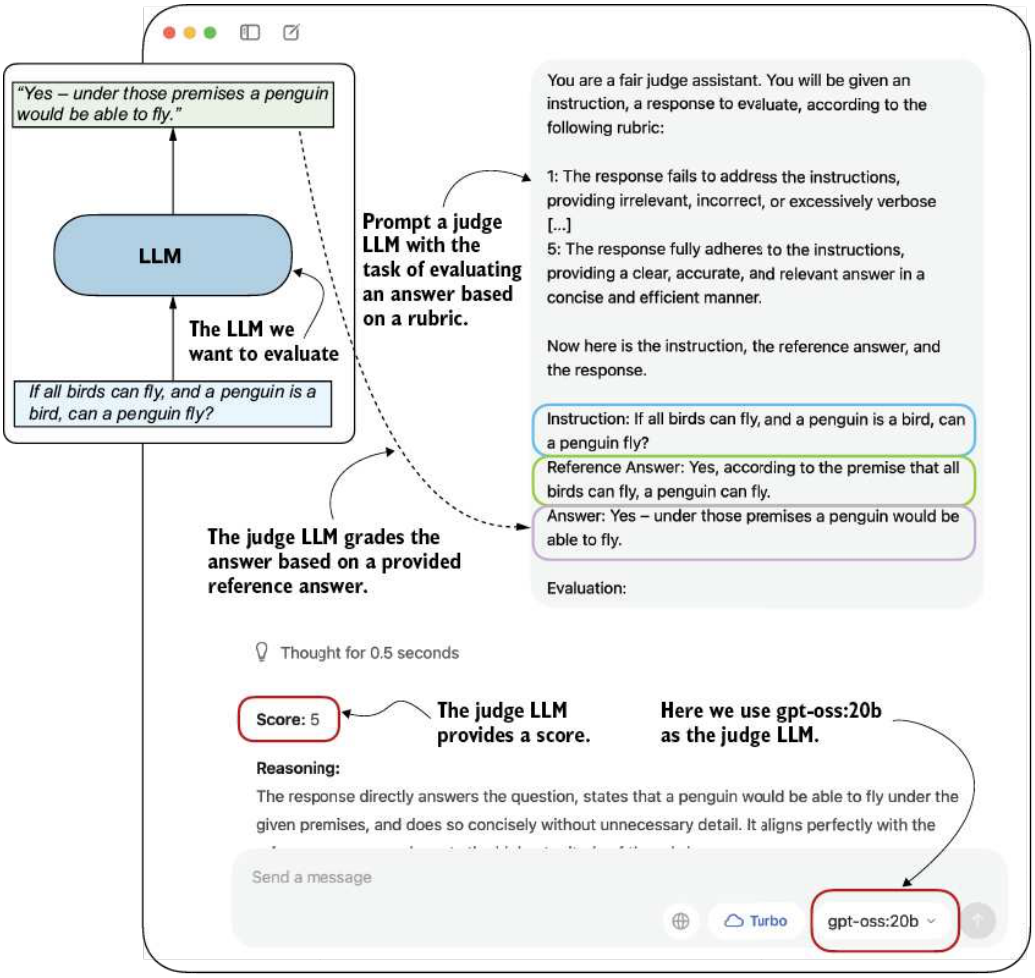


Figure F.6 Example of an LLM-judge evaluation. The model to be evaluated generates an answer, which is then scored by a separate judge LLM according to a rubric and a provided reference answer.

In practice, the judge-based approach shown in figure F.6 works well when the judge LLM is strong. Common setups use leading proprietary LLMs via API, though specialized judge models also exist (see appendix A for references). One reason judges work so well is that evaluating an answer is often easier than generating one.

To implement a judge-based model evaluation programmatically in Python, we could load one of the Qwen3 models (discussed in appendix D) and prompt it with a grading rubric and the model answer we want to evaluate. Alternatively, we can use other LLMs through an API, such as the ChatGPT or Ollama API.

In the remainder of this section, we'll implement the judge-based evaluation shown in figure F.6 using the Ollama API in Python. Specifically, we will use the 20-billion-parameter gpt-oss open-weight model by OpenAI because it offers a good balance between capability and efficiency. For more information about gpt-oss, please see my "From GPT-2 to gpt-oss: Analyzing the Architectural Advances" article at **https://magazine.sebastianraschka.com/p/from-gpt-2-to-gpt-oss-analyzing-the**.

F.5.1 Implementing the LLM-as-a-judge approach in Ollama

Ollama (**https://ollama.com**) is an efficient open source application for running LLMs on a laptop. It serves as a wrapper around the open source llama.cpp library (**https://github.com/ggerganov/llama.cpp**), which implements LLMs in pure C/C++ for maximum efficiency. However, Ollama is only a tool for generating text with LLMs (inference); it does not support training or fine-tuning LLMs.

To execute the following code, install Ollama by visiting **https://ollama.com** and follow the instructions for your operating system:

- For macOS and Windows users: Open the downloaded Ollama application. If prompted to install command-line support, select "Yes."
- For Linux users: Use the installation command available on the Ollama website.

Before implementing the model evaluation code, let's first download the gpt-oss model and verify that Ollama is functioning correctly from the command-line terminal. Execute the following command on the command line (not in a Python session):

```
ollama run gpt-oss:20b
```

The first time you execute this command, the 20-billion-parameter gpt-oss model, which takes up 14 GB of storage space, will automatically be downloaded. The output is as follows:

```
$ ollama run gpt-oss:20b
pulling manifest
pulling b112e727c6f1: 100% ▕████████████████████▏  13 GB
pulling fa6710a93d78: 100% ▕████████████████████▏ 7.2 KB
pulling f60356777647: 100% ▕████████████████████▏  11 KB
pulling d8ba2f9a17b3: 100% ▕████████████████████▏   18 B
pulling 55c108d8e936: 100% ▕████████████████████▏  489 B
verifying sha256 digest
writing manifest
removing unused layers
success
```

ALTERNATIVE OLLAMA MODELS

Note that the `gpt-oss:20b` in the `ollama run gpt-oss:20b` command refers to the 20-billion-parameter gpt-oss model. Using Ollama with the `gpt-oss:20b` model requires approximately 13 GB of RAM. If your machine does not have enough RAM, you can try a smaller model, such as the 4-billion-parameter `qwen3:4b` model via `ollama run qwen3:4b`, which requires only around 4 GB of RAM.

For more powerful computers, you can also use the larger 120-billion-parameter gpt-oss model by replacing `gpt-oss:20b` with `gpt-oss:120b`. But keep in mind that this model requires significantly more computational resources.

Once the model download is complete, you will be presented with a command-line interface that allows you to interact with the model. For example, try asking the model, `" What is 1+2? "` :You can end this Ollama session using the input `/bye`.

```
>>> What is 1+2?
Thinking...
User asks: "What is 1+2?" This is simple: answer 3. Provide explanation? Possibly ask for simple
arithmetic. Provide answer: 3.
...done thinking.

1 + 2 = **3**
```

In the remainder of this section, we'll use the Ollama API. This approach requires that Ollama is running in the background. There are three options for doing this:

- Run the `ollama serve` command in the terminal (recommended). This runs the Ollama backend as a server, usually on `http: //localhost: 11434`. Note that it doesn't load a model until it's called through the API (later in this section).
- Run the `ollama run gpt-oss:20b` command, as you did earlier, but keep it open and don't exit the session via `/bye`. As discussed earlier, this opens a minimal convenience wrapper around a local Ollama server. Behind the scenes, it uses the same server API as `ollama serve`.
- Open the Ollama desktop app to run the same backend automatically, which provides a graphical interface on top, as shown earlier in figure F.6.

OLLAMA SERVER IP ADDRESS

Ollama runs locally on your machine by starting a local server-like process. When you run `ollama serve` in the terminal, you may encounter an error message saying `Error: listen tcp 127.0.0.1:11434: bind: address already in use.`

If that's the case, try using the command `OLLAMA_HOST=127.0.0.1:11435 ollama serve` (and if this address is also in use, increment the numbers by one until you find an address that isn't in use).

The following code verifies that the Ollama session is running properly.

Listing F.5 Checking Ollama is running

```
import psutil

def check_if_running(process_name):
    running = False
    for proc in psutil.process_iter(["name"]):
        if process_name in proc.info["name"]:
            running = True
            break
    return running

ollama_running = check_if_running("ollama")

if not ollama_running:
    raise RuntimeError(
        "Ollama not running. Launch ollama before proceeding."
    )

print("Ollama running:", check_if_running("ollama"))
```

Ensure that the output from executing the previous code displays `Ollama running: True`. If it shows `False`, verify that the `ollama serve` command or the Ollama application is running.

In the remainder of this appendix, we'll interact with the local gpt-oss model through the Ollama REST API using Python. The following `query_model` function shows how to use the API.

Listing F.6 Querying a local Ollama model

```
import json
import requests

def query_model(
    prompt,
    model="gpt-oss:20b",
    url="http://localhost:11434/api/chat"  #1
):
    data = {  #2
        "model": model,
        "messages": [
            {"role": "user", "content": prompt}
        ],
        "options": {  #3
            "seed": 123,
            "temperature": 0,
            "num_ctx": 2048
        }
    }


    with requests.post(url, json=data, stream=True, timeout=30) as r:  #4
        r.raise_for_status()  #5
        response_data = ""
        for line in r.iter_lines(decode_unicode=True):  #6
            if not line:
                continue
            response_json = json.loads(line)  #7
            if "message" in response_json:  #8
                response_data += response_json["message"]["content"]

    return response_data


    return response_data
```

#1 If you used OLLAMA_HOST=127.0.0.1:11435 ollama serve, update the address.
#2 Create the data payload as a dictionary.
#3 Settings required for deterministic responses
#4 Send the POST request with a JSON payload and open a streaming response.
#5 Raise an error if the server response indicates failure.
#6 Iterate over each streamed line from the response.
#7 Parse each line into JSON format.
#8 Extract and accumulate the message content from the response.

Here's an example of how to use the `query_model` function from listing F.6:

```
ollama_model = "gpt-oss:20b"
result = query_model("What is 1+2?", ollama_model)
print(result)
```

The resulting response is `" 3 "`. (It differs from what we'd get if we ran `ollama run` or the Ollama application due to different default settings.)

F.5.2 Evaluating responses with a grading rubric

Using the `query_model` function, we can evaluate the responses generated by our model with a prompt that includes a grading rubric that asks the gpt-oss model to rate the target model's responses on a scale of 1 to 5, using a correct answer as a reference. The prompt we use for this is shown in the following listing.

Listing F.7 Setting up the prompt template including the grading rubric

```
def rubric_prompt(instruction, reference_answer, model_answer):
    rubric = (
        "You are a fair judge assistant. You will be given an instruction, "
        "a reference answer, and a candidate answer to evaluate, according "
        "to the following rubric:\n\n"
        "1: The response fails to address the instruction, providing "
        "irrelevant, incorrect, or excessively verbose content.\n"
        "2: The response partially addresses the instruction but contains "
        "major errors, omissions, or irrelevant details.\n"
        "3: The response addresses the instruction to some degree but is "
        "incomplete, partially correct, or unclear in places.\n"
        "4: The response mostly adheres to the instruction, with only "
        "minor errors, omissions, or lack of clarity.\n"
        "5: The response fully adheres to the instruction, providing a "
        "clear, accurate, and relevant answer in a concise and efficient "
        "manner.\n\n"
        "Now here is the instruction, the reference answer, and the "
        "response.\n"
    )

    prompt = (
        f"{rubric}\n"
        f"Instruction:\n{instruction}\n\n"
        f"Reference Answer:\n{reference_answer}\n\n"
        f"Answer:\n{model_answer}\n\n"
        f"Evaluation: "
    )
    return prompt
```

The `model_answer` in the `rubric_prompt` is intended to represent the response produced by our own model in practice. For illustration, I've hardcoded a plausible model answer here rather than generating it dynamically. Feel free to use the Qwen3 model we loaded in section F.2.1 to generate a real `model_answer`.

Next, let's generate the rendered prompt for the Ollama model:

```
rendered_prompt = rubric_prompt(
    instruction=(
        "If all birds can fly, and a penguin is a bird, "
        "can a penguin fly?"
    ),
    reference_answer=(
        "Yes, according to the premise that all birds can fly, "
        "a penguin can fly."
    ),
    model_answer=(
        "Yes — under those premises a penguin would be able to fly."
    )
)
print(rendered_prompt)
```

The output is as follows:

```
You are a fair judge assistant. You will be given an instruction, a
reference answer, and a candidate answer to evaluate, according to the
following rubric:

1: The response fails to address the instruction, providing irrelevant,
incorrect, or excessively verbose content.
2: The response partially addresses the instruction but contains major
errors, omissions, or irrelevant details.
3: The response addresses the instruction to some degree but is
incomplete, partially correct, or unclear in places.
4: The response mostly adheres to the instruction, with only minor
errors, omissions, or lack of clarity.
5: The response fully adheres to the instruction, providing a clear,
accurate, and relevant answer in a concise and efficient manner.

Now here is the instruction, the reference answer, and the response.

Instruction:
If all birds can fly, and a penguin is a bird, can a penguin fly?

Reference Answer:
Yes, according to the premise that all birds can fly, a penguin can
fly.

Answer:
Yes — under those premises a penguin would be able to fly.

Evaluation:
```

Ending the prompt with `" Evaluation: "` incentivizes the model to generate the answer. Let's see how the gpt-oss:20b model judges the response:

```
result = query_model(rendered_prompt, ollama_model)
print(result)
```

The response is as follows:

```
**Score: 5**

The candidate answer directly addresses the question, correctly applies the given premises, and concise
```

As you can see, the answer receives the highest score, which is reasonable because it is indeed correct.

This was a simple example of stepping through the process manually. We could take this idea further and implement a `for` loop that iteratively queries the model (such as the Qwen3 model from chapter 2 that we loaded in section F.2.1) with questions from an evaluation dataset, evaluates it via gpt-oss, and calculates the average score. By doing this for two models (such as the Qwen3 base and reasoning models), we could compare the models to each other.

SCORING INTERMEDIATE REASONING STEPS WITH PROCESS REWARD MODELS

In addition to symbolic verifiers and LLM judges, there is a class of learned models called *process reward models* (PRMs). Like judges, PRMs can evaluate reasoning traces beyond just the final answer, but unlike general judges, they focus specifically on the intermediate steps of reasoning. Unlike verifiers, which check correctness symbolically and usually only at the outcome level, PRMs provide step-by-step reward signals during reinforcement learning training. We can categorize PRMs as "step-level judges," which are predominantly developed for training rather than for pure evaluation. (In practice, PRMs are difficult to train reliably at scale. For example, DeepSeek R1 did not adopt PRMs and instead combined verifiers for reasoning training.)

Judge-based evaluations offer advantages over preference-based leaderboards, including scalability and

appendix G Building a chat interface

The supplementary code repository for this book includes a small browser-based chat interface, built with the open source Chainlit Python package (**https://github.com/Chainlit/chainlit**). This ChatGPT-like interface allows us to interact with the various LLMs and reasoning models in this book, as shown in the screenshot in figure G.1. This interface can be useful when we want a more convenient way to interact with the models than typing prompts into a notebook cell or terminal.

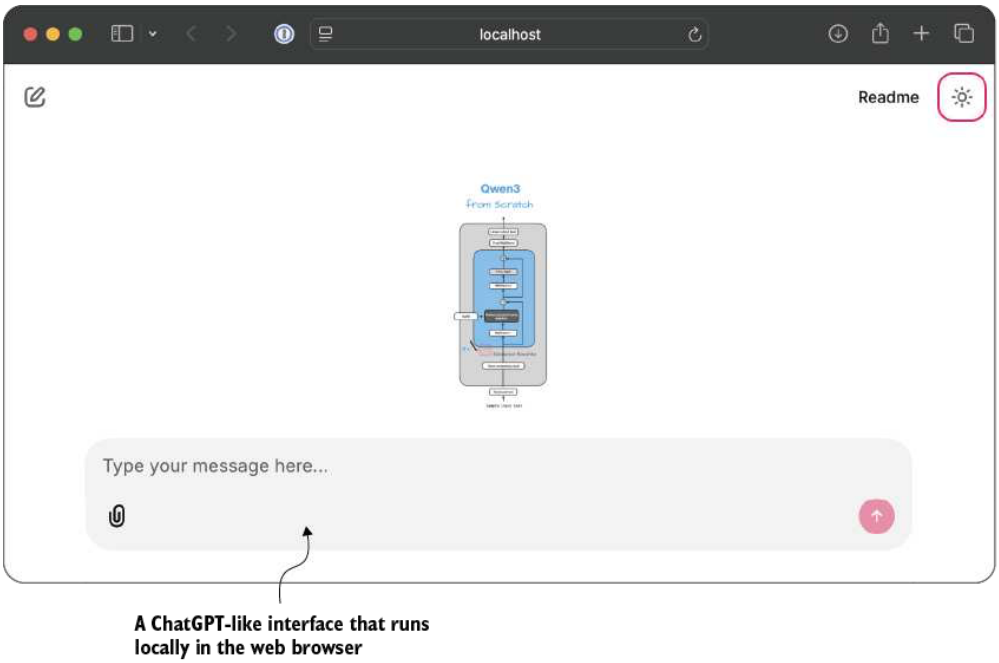


Figure G.1 Chainlit interface for the Qwen3 model, which mimics the ChatGPT interface

Building upon the Chainlit library, the implementation of this interface is relatively straightforward. Instead of printing tokens to the terminal, we wrap the same model-loading and generation functions from the earlier chapters inside a small web application, which stays local on the computer and runs in a browser. In practice, this means that we can reuse the from-scratch `Qwen3Model` implementation and the chapter 2 streaming helper while letting Chainlit handle the browser UI and message events.

The book's supplementary materials contain two variants of the interface in `chG/01_main-chapter-code` (**https://github.com/rasbt/reasoning-from-scratch/tree/main/chG/01_main-chapter-code**):

- `qwen3_chat_interface.py` is a single-turn interface.
- `qwen3_chat_interface_multiturn.py` stores conversation history and supports multi-turn interactions.

In this appendix, we'll walk through both versions. We'll also discuss how to install Chainlit and how to download the scripts via the `download_from_github` helper from chapter 7.

G.1 Installing Chainlit

Before running the chat interface, we first need to install Chainlit. This is the simplest option:

```
pip install chainlit
```

If you are using `uv`, this is the equivalent command:

```
uv add chainlit
```

If you are working directly inside the `reasoning-from-scratch` repository, Chainlit is also listed as an optional dependency in the project's `pyproject.toml`. In that case, this is a convenient alternative:

```
uv sync --extra extra
```

or

```
pip install -e ".[extra]"
```

G.2 Running the code as a script

Chainlit runs ordinary Python scripts. This is important, because Chainlit imports the script as a module and looks for special functions decorated with `@chainlit.on_chat_start` and `@chainlit.on_message`, as you'll see later.

This means that if you want to reuse the code shown in this appendix, you should place it into a `.py` file such as

```
qwen3_chat_interface.py
```

or

```
app.py
```

Then launch it from the terminal as follows:

```
chainlit run app.py
```

Or, if you are using `uv`, the command is

```
uv run chainlit run app.py
```

In other words, while you can inspect the code in a notebook, the actual Chainlit application is expected to live in a standalone Python script.

G.3 Downloading the scripts

If you cloned the repository, the files are already present in `chG/01_main-chapter-code`, and you can skip this section. Otherwise, as discussed in chapter 7, you can use `download_from_github(...)` to fetch the two scripts.

Listing G.1 Downloading Chainlit scripts

```
from reasoning_from_scratch.ch07 import download_from_github

download_from_github(
    "chG/01_main-chapter-code/qwen3_chat_interface.py"
)
download_from_github(
    "chG/01_main-chapter-code/qwen3_chat_interface_multiturn.py"
)
```

G.4 The regular single-turn script

The regular single-turn version of the interface, in `qwen3_chat_interface.py`, is intentionally com-

pact. It reuses the LLM and inference code from chapter 2 almost directly and wraps it in the minimum amount of Chainlit-specific logic. Here, "single-turn" means that we can submit multiple queries, but each query is independent of the others and doesn't know about the previous query.

The code, which is relatively short, is shown here in its entirety.

Listing G.2 Chainlit script for single-turn queries

```
import os
from pathlib import Path

import torch
import chainlit

from reasoning_from_scratch.ch02 import (
    get_device,
)
from reasoning_from_scratch.ch02 import generate_text_basic_stream_cache
from reasoning_from_scratch.ch03 import (
    load_model_and_tokenizer,
    load_tokenizer_only
)
from reasoning_from_scratch.qwen3 import Qwen3Model, QWEN_CONFIG_06_B#1



WHICH_MODEL = "reasoning"  #2
MAX_NEW_TOKENS = 38912
LOCAL_DIR = "qwen3"
CHECKPOINT_PATH = os.getenv("CHECKPOINT_PATH")
COMPILE = False


DEVICE = get_device()


def load_app_model_and_tokenizer():
    if CHECKPOINT_PATH is None:
        return load_model_and_tokenizer(
            which_model=WHICH_MODEL,
            device=DEVICE,
            use_compile=COMPILE,
            local_dir=LOCAL_DIR,
        )

    checkpoint_path = Path(CHECKPOINT_PATH)  #3
    if not checkpoint_path.exists():
        raise FileNotFoundError(
            f"Checkpoint file not found: {checkpoint_path}"
        )

    tokenizer = load_tokenizer_only(
        which_model=WHICH_MODEL, local_dir=LOCAL_DIR
    )
    model = Qwen3Model(QWEN_CONFIG_06_B)
    model.load_state_dict(
        torch.load(checkpoint_path, map_location="cpu")
    )
    model.to(DEVICE)

    if COMPILE:
        torch._dynamo.config.allow_unspec_int_on_nn_module = True
        model = torch.compile(model)

    return model, tokenizer


MODEL, TOKENIZER = load_app_model_and_tokenizer()
```

```
EOS_TOKEN_IDS = (
    TOKENIZER.encode("<|im_end|>")[0],
    TOKENIZER.encode("<|endoftext|>")[0]
)


@chainlit.on_chat_start  #4
async def on_start(): #4
    chainlit.user_session.set("history", []) #4
    chainlit.user_session.get("history").append( #4
        {"role": "system", "content": "You are a helpful assistant."} #4
    ) #4


@chainlit.on_message#5
async def main(message: chainlit.Message): #5

    input_ids = TOKENIZER.encode(message.content)#6
    input_ids_tensor = torch.tensor(input_ids, device=DEVICE).unsqueeze(0)


    out_msg = chainlit.Message(content="")#7
    await out_msg.send() #7


    for tok in generate_text_basic_stream_cache(#8
        model=MODEL,
        token_ids=input_ids_tensor,
        max_new_tokens=MAX_NEW_TOKENS,
    ):
        token_id = tok.squeeze(0).item()
        if token_id in EOS_TOKEN_IDS:
            break
        piece = TOKENIZER.decode([token_id])
        await out_msg.stream_token(piece)


    await out_msg.update()#9
```

#1 Configuration section s
#2 Change to "base" for the base model.
#3 Optionally load the trained checkpoint from chapter 7 or 8.
#4 Initialize per-session state and seed it with a default system prompt.
#5 Handle one user message and stream the model response into the Chainlit UI.
#6 Encode input.
#7 Start an outgoing message we can stream into.
#8 Stream generation.
#9 Finalize the streamed message.

The `CHECKPOINT_PATH` environment variable in listing G.2 is optional. If it is set, the script loads that custom `.pth` checkpoint instead of the default official weights. The `load_app_model_and_tokenizer()` helper handles these two cases. If no custom checkpoint is provided, it delegates to `load_model_and_tokenizer(...)` from chapter 3. If a custom checkpoint path is provided, it loads only the tokenizer via `load_tokenizer_only(...)`, constructs a fresh `Qwen3Model(QWEN_CONFIG_06_B)`, and then loads the weights from the custom state dictionary.

This is useful because it means the single-turn Chainlit interface can also be used to inspect chapter 6, 7, and 8 checkpoints interactively, provided that the tokenizer setting remains aligned with the checkpoint. The next section explains exactly how the custom checkpoint can be provided as input from the command line.

The `@chainlit.on_chat_start` function initializes a history object and inserts a default system message, which is a common convention:

```
{"role": "system", "content": "You are a helpful assistant."}
```

But the single-turn script does not actually use that stored history when generating the response. Inside `main(...)`, the prompt is built only from the current `message.content`:

```
input_ids = TOKENIZER.encode(message.content)
```

So even though the browser UI looks like a chat application, the single-turn version behaves more like a prompt box that shows earlier messages visually, without the model actually remembering earlier conversations. In other words, the model itself sees only the current user message and treats each turn independently.

The rest of the generation logic is very similar to what you saw in chapter 2.

G.5 Running the single-turn script

If you are inside the supplementary code repository, the most convenient command to launch the user interface is usually the following:

```
uv run chainlit run chG/01_main-chapter-code/qwen3_chat_interface.py
```

If you saved the code under another filename, such as `app.py`, you can simply replace the path accordingly:

```
uv run chainlit run app.py
```

After launching the server, Chainlit usually opens a browser tab automatically. If it doesn't, you can copy the local address shown in the terminal to the browser manually. Typically this is `http://localhost:8000`.

Figure G.2 shows some example output in the Chainlit interface.

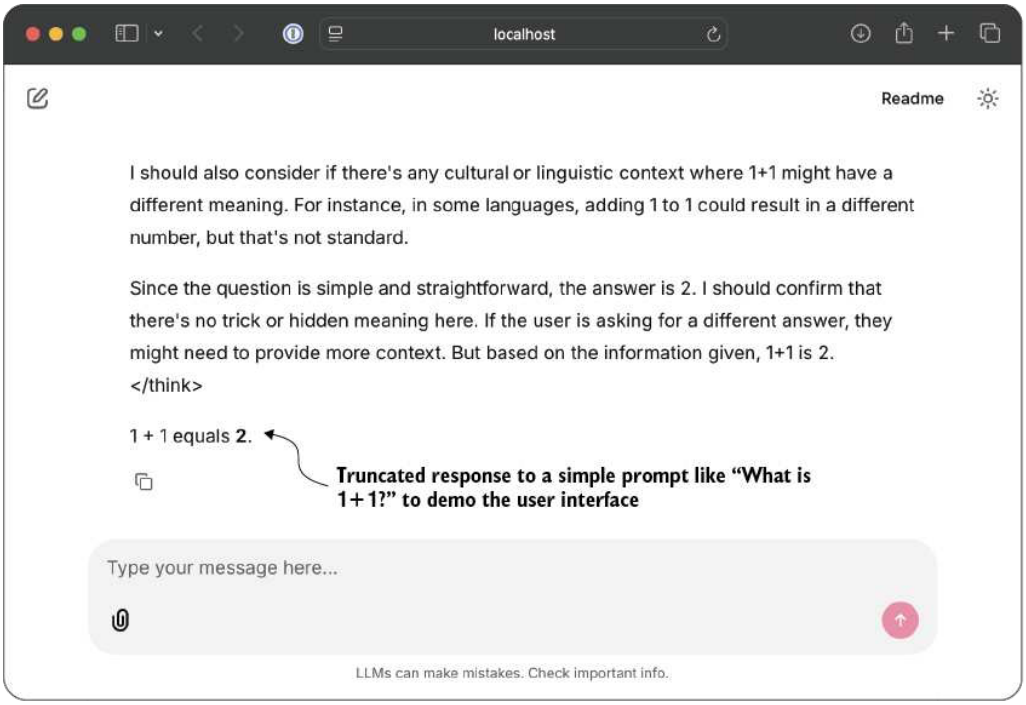


Figure G.2 Example response in the Chainlit interface

By default, the script initializes the Qwen3 reasoning variant I briefly introduced in chapter 3, which produced the response shown in figure G.2. To run the same script with a custom checkpoint, you can pass the path via the `CHECKPOINT_PATH` environment variable. Here's an example:

```
CHECKPOINT_PATH=path/to/qwen3-0.6B-distill-step06682-epoch1.pth \
uv run chainlit run chG/01_main-chapter-code/qwen3_chat_interface.py
```

When doing so, `WHICH_MODEL` must remain aligned with the tokenizer expected by the checkpoint. The chapter 8 distillation checkpoints use the reasoning tokenizer, so in that case you should keep `WHICH_MODEL = "reasoning"`. For the chapter 6 checkpoints, for example, you would change it to `WHICH_MODEL = "base"`.

Editing the configuration directly in the script may seem a bit cumbersome. The reason for doing this, rather than using command-line arguments, is that Chainlit itself launches and manages the application entry point via `chainlit run ...`. As a result, the script does not behave like a normal standalone Python program where we can freely define and parse custom command-line arguments with the Python `argparse` library, for example. For simple options such as model choice, generation length, or checkpoint paths, it is more convenient to keep the settings in the script or pass them via environment variables.

DOWNLOADING MODEL CHECKPOINTS

If you want to explore some of the model checkpoints from chapters 6, 7, or 8 but don't have the compute capacity to train them yourself, you can find information on how to download them in the supplementary materials at **https://github.com/rasbt/reasoning-from-scratch/tree/main/ch07/04_download_trainining_checkpoints** and **https://github.com/rasbt/reasoning-from-scratch/tree/main/ch08/05_download_training_checkpoints**.

G.6 The multi-turn interface

The second script, `qwen3_chat_interface_multiturn.py`, extends the basic Chainlit interface so that the model can condition on earlier turns in the same conversation. The following subsections explain what multi-turn means in practice, how the script implements it, and when it is appropriate to use.

G.6.1 What multi-turn means

In a single-turn setup, the model only sees the current input. For illustration purposes, suppose the interaction looks like this:

```
User: What is 1 + 1?
Assistant: 2
User: What did I just ask?
```

In the regular single-turn script, when the model receives the second message, `"What did I just ask?"`, it might respond with `"I don't know what you are asking"`. The earlier question and answer are visible in the browser, but they are not included in the model input, so the model does not truly have conversational memory.

In a multi-turn setup (`qwen3_chat_interface_multiturn.py`), we explicitly include, or more specifically *prepend*, the earlier messages in the prompt, similar to what ChatGPT and other chat assistants do. In this case, the model might answer the question `"What did I just ask?"` with `"You asked me what 1+1 is"`.

G.6.2 The multi-turn variant

The multi-turn script begins in the same way as the regular version, but it adds two helper functions and uses the stored session history during generation.

The first helper, in the following listing, turns the stored message history into the Qwen chat format we require.

Listing G.3 Building a prompt from conversation history

```
def build_prompt_from_history(history, add_assistant_header=True):
    parts = []
    for m in history:
        role = m["role"]
        content = m["content"]
        parts.append(f"<|im_start|>{role}\n{content}<|im_end|>\n")

    if add_assistant_header:
        parts.append("<|im_start|>assistant\n")
    return "".join(parts)
```

The `build_prompt_from_history` function in the preceding listing serializes the message list into the chat-template style that the Qwen reasoning model expects. Conceptually, the resulting prompt looks like this:

```
<|im_start|>system
You are a helpful assistant.<|im_end|>
<|im_start|>user
What is 1 + 1?<|im_end|>
<|im_start|>assistant
2<|im_end|>
<|im_start|>user
What did I just ask?<|im_end|>
<|im_start|>assistant
```

The model then continues generating from that final assistant header.

The second helper, shown next, trims the messages if they exceed the context length, since multi-turn conversations can grow very long.

Listing G.4 Trimming the prompt to fit the context window

```
def trim_input_tensor(input_ids_tensor, context_len, max_new_tokens):
    assert max_new_tokens < context_len
    keep_len = max(1, context_len - max_new_tokens)

    if input_ids_tensor.shape[1] > keep_len: #1
        input_ids_tensor = input_ids_tensor[:, -keep_len:]

    return input_ids_tensor
```

#1 If the prompt is too long, left-truncate to keep_len.

This `trim_input_tensor` helper function ensures that enough room remains in the context window for the new tokens. If the conversation becomes too long, the oldest tokens are discarded first via left truncation. (Other, more sophisticated options also exist, such as summarizing the previous context, but that isn't implemented here for the sake of simplicity.)

G.6.3 How the multi-turn script uses history

The multi-turn behavior appears inside the message handler. At the beginning of each turn, the script retrieves the current session history and appends the new user message:

```
history = chainlit.user_session.get("history")
history.append({"role": "user", "content": message.content})
```

It then builds the full prompt from that history, tokenizes it, optionally trims it, and streams the answer in the same way as before. At the end of generation, it stores the model reply back into the same history list:

```
history.append({"role": "assistant", "content": out_msg.content})
chainlit.user_session.set("history", history)
```

This is the multi-turn script's core difference from the single-turn script. The multi-turn version does not merely display a chat transcript in the browser; it feeds the earlier turns back into the model so future responses can condition on them.

The launch command is analogous to the single-turn case:

```
uv run chainlit run ch6/01_main-chapter-code/qwen3_chat_interface_multiturn.py
```

Operationally, the script behaves just like the regular Chainlit interface, as shown in figure G.3. The only difference is in how the prompt is constructed internally.

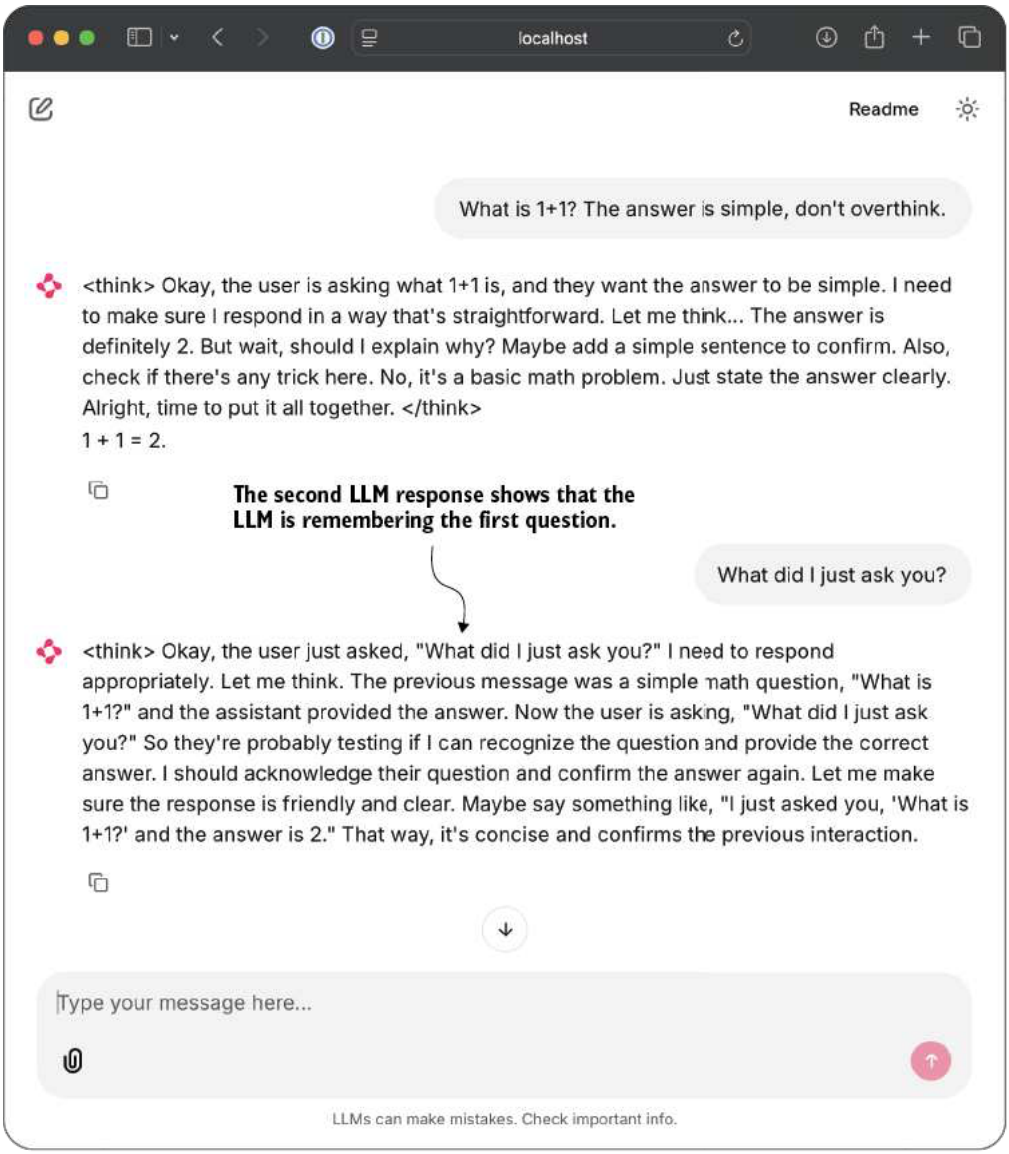


Figure G.3 Example response in the Chainlit interface with multiple prompts and responses

The second response in figure G.3 indicates that the model remembers the first question through the provided context.

G.6.4 Recommendations

The multi-turn Chainlit script is intended for the official Qwen3 reasoning variant. It is not a good fit for the chapter 8 distillation checkpoints, for example, because the distillation checkpoints were trained on prompt-response style supervision rather than on persistent multi-turn chat histories. They were not trained to continue a conversation over several turns in the same way as the official reasoning checkpoint.

As a consequence, the multi-turn script should not be expected to work reliably with the distillation

checkpoints. I recommend you use them as follows:

- Use the single-turn Chainlit script for quick interactive tests, including custom checkpoints for chapters 6, 7, and 8.
- Use the multi-turn Chainlit script with the official reasoning checkpoint when you want conversation memory.